

```
=====  
: : , 233501934139 : : 2026 :  
Python, JavaScript =====
```

```
=====
```

FILE: app.py

```
=====
```

```
# neurocad/app.py
```

```
"""NeuroCad – фабрика FastAPI-приложения."""
```

```
from fastapi import FastAPI
```

```
from .config import settings as default_settings
```

```
class NeuroCad:
```

```
    """NeuroCad – фабрика FastAPI-приложения."""
```

```
    def __init__(self, config=None, **kwargs):  
        self.config = config or default_settings
```

```
        # Логгер
```

```
        from .utils.log import Log
```

```
        self.log = Log(log_dir=str(self.config.LOG_PATH))
```

```
        # Lifespan – инициализация БД, shutdown
```

```
        from .utils.lifespan import get_lifespan
```

```
        self.app = FastAPI(  
            title=getattr(self.config, "APP_TITLE", None) or "NeuroCad",  
            debug=getattr(self.config, "DEBUG", True),  
            lifespan=get_lifespan(),  
        )
```

```
            title=getattr(self.config, "APP_TITLE", None) or "NeuroCad",
```

```
            debug=getattr(self.config, "DEBUG", True),
```

```
            lifespan=get_lifespan(),
```

```
        )
```

```
        self._setup_cors()
```

```
        self._setup_static()
```

```
        self._setup_routes()
```

```
        self._setup_middleware()
```

```
    def __getattr__(self, name):  
        return getattr(self.app, name)
```

```
    async def __call__(self, scope, receive, send):  
        await self.app(scope, receive, send)
```

```
    def _setup_cors(self):
```

```
        from .utils.cors import setup_cors
```

```
        setup_cors(self.app)
```

```
    def _setup_static(self):
```

```
        from .utils.static import setup_static
```

```
        setup_static(self.app, log=self.log)
```

```
    def _setup_routes(self):
```

```

        from .utils.routes import setup_routes
        setup_routes(self.app)

    def _setup_middleware(self):
        from .core.auth.middleware.auth import CoreAuthMiddlewareAuth
        self.app.add_middleware(CoreAuthMiddlewareAuth)

=====
FILE: cli.py
=====
# neurocad/cli.py

"""CLI for NeuroCad."""

import argparse

def cmd_run(args):
    import sys
    import os
    import uvicorn
    from .config import settings
    from .utils.paths import ensure_workdirs

    # Ensure cwd is in sys.path (so uvicorn can find main:app)
    cwd = os.getcwd()
    if cwd not in sys.path:
        sys.path.insert(0, cwd)

    # Create working dirs and files (main.py, .env, app/, etc.)
    ensure_workdirs()

    host = args.host or settings.APP_HOST
    port = args.port or settings.APP_PORT
    reload = args.reload

    print(f"neurocad run - http://{host}:{port}")
    uvicorn.run(
        "main:app",
        host=host,
        port=port,
        reload=reload,
        app_dir=cwd,
        reload_dirs=[cwd] if reload else None,
    )

def main(argv=None):
    parser = argparse.ArgumentParser(prog="neurocad", description="NeuroCad CLI")
    sub = parser.add_subparsers(dest="cmd", required=True)

    p_run = sub.add_parser("run", help="Start uvicorn")
    p_run.add_argument("--host", default=None)
    p_run.add_argument("--port", type=int, default=None)
    p_run.add_argument("--reload", action="store_true")
    p_run.set_defaults(func=cmd_run)

```

```

args = parser.parse_args(argv)
args.func(args)

if __name__ == "__main__":
    main()

=====
FILE: config.py
=====
# neurocad/config.py

from pydantic_settings import BaseSettings
from typing import Optional
from pathlib import Path

class Settings(BaseSettings):
    """
    Application settings.
    All fields have default values. .env file is optional.

    Priority for LLM settings:
    1. Database (settings table, key='llm', encrypted secrets)
    2. .env / this file
    3. Hardcoded default in the provider class
    """

    # =====
    # APPLICATION
    # =====

    # Application host.
    APP_HOST: str = "127.0.0.1"

    # Application port.
    APP_PORT: int = 8000

    # Main page URL (redirect from /).
    APP_MAIN_PAGE: str = "/core/engine/admin"

    # Root domain the application runs on.
    #
    # Used by the "Custom domains" module (base/profile/domain):
    # - to build the free third-level subdomain (<login>.<APP_DOMAIN>)
    # - to reject custom domains that end with .<APP_DOMAIN>
    #   (those are handled by the wildcard certificate, not by
    #   On-Demand TLS)
    #
    # MUST match a domain that Caddy actually serves with a wildcard
    # certificate (e.g. *.neurocad-dev.ru).
    #
    # Dev : neurocad-dev.ru
    # Prod : neurocad.ru
    APP_DOMAIN: str = "neurocad.ru"

```

```

# Debug mode.
DEBUG: bool = True

# =====
# DATABASES
# =====

# MySQL URL (optional).
DATABASE_URL: str = ""

# SQLite URL for async.
SQLITE_URL: str = "sqlite+aiosqlite:///base/neurocad.db"

# SQLite URL for sync (Alembic).
SQLITE_URL_SYNC: str = "sqlite:///base/neurocad.db"

# =====
# SECURITY
# =====

# Secret key for JWT.
SECRET_KEY: str = "your-secret-key-change-in-production"

# JWT algorithm.
ALGORITHM: str = "HS256"

# Access token lifetime (minutes).
ACCESS_TOKEN_EXPIRE_MINUTES: int = 60 * 24

# =====
# SUPERADMIN
# =====

# Default superadmin login.
SUPERADMIN_LOGIN: str = "admin"

# Default superadmin password.
SUPERADMIN_PASSWORD: str = ""

# =====
# USERS
# =====

# Auto-create a personal "Каталог статей" nav for every new user.
# When True, every registered user gets their own Nav row pointing
# at the built-in 'default' module, so they immediately have a
# place to create pages.
# Set to False (or 0 in .env) to register users without any nav.
USER_AUTO_CREATE_NAV: bool = True

# =====
# PATHS (relative to cwd)
# =====

# Media files directory.

```

```

MEDIA_PATH: Path = Path("media")

# Database directory.
BASE_PATH: Path = Path("base")

# Static files directory.
STATIC_PATH: Path = Path("static")

# Logs directory.
LOG_PATH: Path = Path("log")

# =====
# LLM – CORE
# =====

# Active provider: 'deepseek' | 'openai' | 'yandex' | 'gigachat' | 'gemini'
LLM_PROVIDER: str = "deepseek"

# Use mock LLM provider instead of a real one.
# 1 – return a canned response (no network, no tokens, instant).
# 0 – use the real provider from LLM_PROVIDER.
# Handy for testing the WS/run orchestration end-to-end.
LLM MOCK: int = 0

# Artificial delay (seconds) for the mock provider, to mimic real latency.
# Ignored when LLM MOCK = 0.
LLM MOCK_DELAY: float = 1.0

# Secret key for encrypting API keys stored in the database (Fernet).
# Generate with:
# python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
# If empty – DB-stored secrets are disabled, .env values are used.
NEUROCAD_SECRET_KEY: Optional[str] = None

# =====
# LLM – DEEPSEEK (default provider)
# =====

# DeepSeek API key.
DEEPSEEK_API_KEY: Optional[str] = None

# DeepSeek API base URL.
DEEPSEEK_BASE_URL: str = "https://api.deepseek.com"

# DeepSeek model name.
DEEPSEEK_MODEL: str = "deepseek-flash"

# Max output tokens.
DEEPSEEK_MAX_OUTPUT_TOKENS: int = 12000

# Request timeout (seconds).
DEEPSEEK_TIMEOUT: int = 150

# =====
# LLM – OPENAI
# =====

```

```

# OpenAI API key.
OPENAI_API_KEY: Optional[str] = None

# OpenAI API base URL.
OPENAI_BASE_URL: str = "https://api.openai.com/v1"

# OpenAI model name.
OPENAI_MODEL: str = "gpt-4o-mini"

# Max output tokens.
OPENAI_MAX_OUTPUT_TOKENS: int = 4096

# Request timeout (seconds).
OPENAI_TIMEOUT: int = 120

# =====
# LLM – YANDEXGPT
# =====

# Yandex Cloud API key.
YANDEX_API_KEY: Optional[str] = None

# Yandex Cloud folder ID.
YANDEX_FOLDER_ID: Optional[str] = None

# Model name.
YANDEX_MODEL: str = "yandexgpt-lite"

# Max output tokens.
YANDEX_MAX_OUTPUT_TOKENS: int = 4096

# Request timeout (seconds).
YANDEX_TIMEOUT: int = 120

# =====
# LLM – GIGACHAT
# =====

# GigaChat Authorization Key (from Sber Studio).
#
# Can be either:
#   - "client_id:client_secret" (plain)
#   - base64("client_id:client_secret") – what Sber Studio gives you
#
# The provider detects the format and uses the appropriate endpoint.
GIGACHAT_AUTH_KEY: Optional[str] = None

# OAuth scope: GIGACHAT_API_PERS | GIGACHAT_API_B2B | GIGACHAT_API_CORP
GIGACHAT_SCOPE: str = "GIGACHAT_API_PERS"

# Model name.
GIGACHAT_MODEL: str = "GigaChat-2-Lite"

# Max output tokens.
GIGACHAT_MAX_OUTPUT_TOKENS: int = 8192

```

```

# Request timeout (seconds).
GIGACHAT_TIMEOUT: int = 120

# Russian Trusted Root CA (PEM text). Optional – only needed
# if the system trust store does not include the Ministry of
# Digital Development root CA.
GIGACHAT_CA_PEM: Optional[str] = None

# =====
# LLM – GOOGLE GEMINI
# =====

# Google AI Studio API key.
GEMINI_API_KEY: Optional[str] = None

# Model name.
GEMINI_MODEL: str = "gemini-1.5-flash"

# Max output tokens.
GEMINI_MAX_OUTPUT_TOKENS: int = 8192

# Request timeout (seconds).
GEMINI_TIMEOUT: int = 120

class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"
    extra = "ignore"

settings = Settings()

=====
FILE: __init__.py
=====
# neurocad/__init__.py

"""
NeuroCad – core engine + CMS + visual editor for FastAPI.
"""

from importlib.metadata import version, PackageNotFoundError

try:
    __version__ = version("neurocad")
except PackageNotFoundError:
    __version__ = "0.0.0-dev"

from .app import NeuroCad
from .core.models.base import Base
from .config import settings

__all__ = [
    "NeuroCad",
    "Base",

```

```

    "settings",
    "__version__",
]

```

```

=====
FILE: core/models/access.py
=====

```

```

# app/core/models/access.py

```

```

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import Integer, Boolean, DateTime, Index
from datetime import datetime as dt
from .base import Base

```

```

class Access(Base):
    __tablename__ = "access"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(Integer, nullable=False)
    module_id: Mapped[int] = mapped_column(Integer, nullable=False)
    can_read: Mapped[bool] = mapped_column(Boolean, default=False, nullable=True)
    can_write: Mapped[bool] = mapped_column(Boolean, default=False, nullable=True)
    is_delete: Mapped[bool] = mapped_column(Boolean, default=False, nullable=True)
    created_at: Mapped[dt] = mapped_column(DateTime, default=dt.now, nullable=True)
    updated_at: Mapped[dt] = mapped_column(DateTime, default=dt.now, onupdate=dt.now, nullable=True)

    __table_args__ = (
        Index('idx_access_user_module', 'user_id', 'module_id', unique=True),
        Index('idx_access_is_delete', 'is_delete'),
    )

```

```

=====
FILE: core/models/auth.py
=====

```

```

# app/core/models/auth.py

```

```

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import Integer, String, DateTime, ForeignKey
from datetime import datetime
from typing import Optional
from .base import Base

```

```

class PasswordReset(Base):
    __tablename__ = "password_resets"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), nullable=False)
    token: Mapped[str] = mapped_column(String(512), nullable=False, unique=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, nullable=False, default=datetime.now)

```

```

=====
FILE: core/models/balance.py
=====

```

```

# neurocad/core/models/balance.py

```


"""

Balance – per-user limits, balances and tariff state.

One row per user. Holds:

- the user's current tariff (free / pro / llm);
- the billing day of the month (1–31);
- remaining balances (tokens, generations, money, storage, pages);
- the corresponding limits (per day / per month / total);
- the referrer (refer_id) who brought the user.

Values are integers; the tariff is stored as a small integer:

- 0 – free
- 1 – pro
- 2 – llm

Storage (`mb` / `limit\_mb`) is in MEGABYTES, not gigabytes.

The name is explicit so nobody has to guess: 100 = 100 MB,
1024 = 1 GB. UI formats it for display.

Generations (`gen`):

A single accumulated counter. It grows by `limit\_genday` every day and by `limit\_genmon` once a month (on `balance.day`). There is NO separate daily/monthly balance – one field, two accrual rates.

Example:

pro: limit_genday=1, limit_genmon=0 → +1 daily
free: limit_genday=0, limit_genmon=1 → +1 monthly
llm: limit_genday=0, limit_genmon=0 → never grows (LLM generations are unlimited via tokens)

Accrual anchor (`acc\_at`):

Date of the last lazy reset for this user. Used by the lazy "day change": on each request we compare today against `acc\_at` and top up `gen` / `tokens` accordingly. Updated on:

- daily accrual (any tariff with limit_genday > 0),
- monthly accrual (any tariff with limit_genmon > 0),
- tariff change (always reset to today).

Billing day (`day`):

The day of the month when the user last changed their tariff. All monthly accruals (limit_genmon, limit_tokens) are anchored to this day. If `day` = 31 and the target month is shorter, the last day of that month is used (28/29 Feb, 30 Apr, ...).

Convention – "0 means unlimited":

For every limit_* field, 0 means "no limit" – the guard never blocks on it. A positive value is the actual cap.

EXCEPT `limit\_tokens`: 0 means "LLM is not available" (used on the free tariff). For pro / llm it is a positive cap.

`is\_delete` is a Boolean: SQLAlchemy stores it in SQLite as INTEGER (0 = false, 1 = true), same as everywhere else in this project.

"""

```

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import (
    Integer,
    Boolean,
    Date,
    DateTime,
    ForeignKey,
    Index,
)
from datetime import datetime, date
from typing import Optional
from .base import Base

class Balance(Base):
    __tablename__ = "balance"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)

    # Owner of this balance row.
    user_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("users.id", ondelete="CASCADE"),
        nullable=False,
    )

    # Current tariff: 0 – free, 1 – pro, 2 – llm.
    tariff: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

    # Billing day of the month (1–31) – the day the user last
    # changed their tariff. Anchors all monthly accruals.
    day: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)

    # ===== REMAINING BALANCES =====

    # Accumulated generations. Grows by limit_genday (daily) and
    # limit_genmon (monthly, on `day`). Spent by LLM generations
    # on free / pro. On llm – unlimited (see limit_tokens).
    gen: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

    # Remaining LLM tokens.
    tokens: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

    # Remaining money on the account.
    sum: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

    # Storage used, in megabytes. Updated on upload / delete of media.
    mb: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

    # Number of pages (articles) created by the user.
    pages: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

    # ===== TARIFF PRICE =====

    # Tariff price.

```

```

price: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

# ===== REFERRAL =====

# ID of the referrer (the user who invited this one).
refer_id: Mapped[Optional[int]] = mapped_column(
    Integer,
    ForeignKey("users.id", ondelete="SET NULL"),
    nullable=True,
)

# ===== LIMITS =====
#
# 0 means "no limit" for every field below – except limit_tokens,
# where 0 means "LLM not available" (free tariff).

# Generation accrual rates.
# limit_genday – how much `gen` grows each day.
# limit_genmon – how much `gen` grows each month (on `day`).
limit_genday: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
limit_genmon: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

# Storage limit, in megabytes.
limit_mb: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

# Total page-creation limit.
limit_pages: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

# LLM token limit per month.
# 0 – LLM not available (free tariff).
# > 0 – cap; tokens are reset to this on `day`.
limit_tokens: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

# ===== ACCRUAL ANCHOR =====
#
# Date of the last lazy reset. Used by the lazy "day change":
# on each request we compare today against `acc_at` and top up
# `gen` / `tokens` accordingly.
#
# Updated on:
# - daily accrual (any tariff with limit_genday > 0),
# - monthly accrual (any tariff with limit_genmon > 0),
# - tariff change (always reset to today).
#
# null – the user has never been through the lazy reset.
acc_at: Mapped[Optional[date]] = mapped_column(Date, nullable=True)

# ===== HOUSEKEEPING =====

# Soft delete flag. SQLAlchemy maps Boolean to INTEGER in SQLite:
# False → 0, True → 1.
is_delete: Mapped[bool] = mapped_column(Boolean, default=False, nullable=True)

updated_at: Mapped[datetime] = mapped_column(
    DateTime,
    default=datetime.now,

```

```

        onupdate=datetime.now,
        nullable=True,
    )

    __table_args__ = (
        Index("idx_balance_user_id", "user_id"),
        Index("idx_balance_refer_id", "refer_id"),
        Index("idx_balance_tarif", "tarif"),
        Index("idx_balance_is_delete", "is_delete"),
    )

```

```

=====
FILE: core/models/base.py
=====

```

```

# neurocad/core/models/base.py

```

```

from sqlalchemy.orm import DeclarativeBase

```

```

class Base(DeclarativeBase):
    pass

```

```

# Импортируем все модели
from .access import Access
from .auth import PasswordReset
from .user import User
from .page import Page
from .nav import Nav
from .module import Module
from .setting import Setting
from .page_pres import PagePres
from .page_hist import PageHist
from .page_chat import PageChat
from .run import Run
from .balance import Balance
from .domain import Domain

```

```

=====
FILE: core/models/domain.py
=====

```

```

# neurocad/core/models/domain.py

```

```

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, DateTime, Index
from datetime import datetime
from typing import Optional
from .base import Base

```

```

class Domain(Base):

```

```

    """

```

```

    Scheduled-for-deletion custom domains.

```

```

    A row appears here when a user removes their custom domain
    (users.domain = None). If the user doesn't re-attach it within
    GRACE_DAYS, the lazy cleanup physically removes the Caddy
    certificate from disk.

```

```

name is the primary key – a domain can only be in one state
at a time: active (no row / delete_after=NULL) or scheduled
(delete_after set).
"""
__tablename__ = "domains"

name: Mapped[str] = mapped_column(String(253), primary_key=True)

# When the domain was removed.
disabled_at: Mapped[datetime] = mapped_column(DateTime, nullable=False)

# When it becomes eligible for physical deletion.
delete_after: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

# When the certificate was actually removed from disk.
# NULL = not yet removed.
deleted_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)

__table_args__ = (
    Index("idx_domains_delete_after", "delete_after"),
)

=====
FILE: core/models/__init__.py
=====

=====
FILE: core/models/module.py
=====
# app/core/models/module.py

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text, Integer, Boolean, DateTime, Index
from datetime import datetime
from typing import Optional
from .base import Base

class Module(Base):
    __tablename__ = "modules"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(128), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    url: Mapped[Optional[str]] = mapped_column(String(512), nullable=True)
    is_delete: Mapped[bool] = mapped_column(Boolean, default=False, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.now, nullable=True)

    __table_args__ = (
        Index('idx_modules_is_delete', 'is_delete'),
    )

=====
FILE: core/models/nav.py

```

```

=====
# app/core/models/nav.py

"""
Nav – user-owned navigation entry.

A Nav row is an *instance* of a module: it points at a module class
via `module_id` and belongs to a user via `user_id`. Pages and media
are scoped to a Nav, not to a module.

Hierarchy is built with `parent_id` (self-referential). `card_type`
distinguishes the kinds of entries the UI renders:
    - 'folder' – a grouping node (no module attached)
    - 'module' – a module instance
    - 'link' – a simple link (used by the default "Каталог статей")
"""

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text, Integer, Boolean, DateTime, ForeignKey, Index
from datetime import datetime
from typing import Optional
from .base import Base

class Nav(Base):
    __tablename__ = "nav"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(Integer, nullable=False)
    parent_id: Mapped[Optional[int]] = mapped_column(
        Integer,
        ForeignKey('nav.id', ondelete='CASCADE'),
        nullable=True,
    )
    card_type: Mapped[str] = mapped_column(
        String(20),
        nullable=False,
    ) # 'folder' | 'module' | 'link'
    sort_order: Mapped[int] = mapped_column(Integer, default=0, nullable=True)

    name: Mapped[str] = mapped_column(String(128), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    icon: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)

    module_id: Mapped[Optional[int]] = mapped_column(
        Integer,
        ForeignKey('modules.id', ondelete='SET NULL'),
        nullable=True,
    )

    is_delete: Mapped[bool] = mapped_column(Boolean, default=False, nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime,
        default=datetime.now,
        nullable=True,
    )

```

```

updated_at: Mapped[datetime] = mapped_column(
    DateTime,
    default=datetime.now,
    onupdate=datetime.now,
    nullable=True,
)

```

```

__table_args__ = (
    Index('idx_nav_user_id', 'user_id'),
    Index('idx_nav_parent', 'parent_id'),
    Index('idx_nav_type', 'card_type'),
    Index('idx_nav_is_delete', 'is_delete'),
    Index('idx_nav_module_id', 'module_id'),
)

```

```

=====
FILE: core/models/page_chat.py
=====

```

```

# neurocad/core/models/page_chat.py

```

```

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text, Integer, DateTime, ForeignKey, Index
from datetime import datetime as dt
from typing import Optional
from .base import Base

```

```

class PageChat(Base):

```

```

    """

```

```

    История чата с LLM по конкретной странице.

```

```

    Каждая запись – одно сообщение: роль + содержимое. Привязана к странице
    (page_id). Используется в правой панели LLM-редактора.
    """

```

```

    __tablename__ = "page_chat"

```

```

    id: Mapped[int] = mapped_column(Integer, primary_key=True)

```

```

    # FK на pages.id – к какой странице относится сообщение

```

```

    page_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("pages.id"),
        nullable=False,
    )

```

```

    # Роль отправителя:

```

```

    # 'user' – сообщение пользователя

```

```

    # 'assistant' – ответ LLM

```

```

    # 'system' – системный промпт

```

```

    role: Mapped[str] = mapped_column(String(20), nullable=False)

```

```

    # Текст сообщения

```

```

    content: Mapped[str] = mapped_column(Text, nullable=False)

```

```

    # Модель, которой был сгенерирован ответ (для assistant)

```

```

    # например: 'deepseek-chat'

```

```

model: Mapped[Optional[str]] = mapped_column(String(50))

# Учёт токенов (для assistant)
prompt_tokens: Mapped[Optional[int]] = mapped_column(Integer)
completion_tokens: Mapped[Optional[int]] = mapped_column(Integer)

created_at: Mapped[dt] = mapped_column(DateTime, default=dt.now)

__table_args__ = (
    Index('idx_page_chat_page_id', 'page_id'),
    Index('idx_page_chat_created_at', 'created_at'),
    Index('idx_page_chat_page_created', 'page_id', 'created_at'),
)

=====
FILE: core/models/page_hist.py
=====
# neurocad/core/models/page_hist.py

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text, Integer, DateTime, ForeignKey, Index
from datetime import datetime as dt
from typing import Optional
from .base import Base

class PageHist(Base):
    """
    История изменений страницы.

    Снимки полного состояния страницы после каждого изменения —
    для undo/redo в LLM-редакторе и визуальном редакторе GrapesJS.
    Привязана к конкретной странице (page_id).

    Хранит:
    - html          — HTML для рендера (без <style>).
    - css           — CSS страницы на момент снимка.
                     Для старых записей (до выделения css
                     в отдельное поле) — NULL, CSS вшит в html.
    - content_json  — полный project JSON GrapesJS (getProjectData()).
                     При откате загружается через loadProjectData()
                     и восстанавливает структуру, стили, assets.
    """
    __tablename__ = "page_hist"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)

    # FK на pages.id — к какой странице относится снимок
    page_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("pages.id"),
        nullable=False,
    )

    # Снимок полного HTML на момент изменения
    html: Mapped[str] = mapped_column(Text, nullable=False)

```



```
# Снимок полного project JSON GrapesJS (getProjectData()).
# Optional: для старых записей (созданных до появления поля) – NULL.
content_json: Mapped[Optional[str]] = mapped_column(Text)
```

```
# Снимок CSS на момент изменения (для отката).
# Optional: для старых записей, где CSS вшит в html.
css: Mapped[Optional[str]] = mapped_column(Text)
```

```
# Что стало причиной изменения:
# 'user_edit' – ручное редактирование
# 'ai_edit' – изменение через LLM
# 'preset_apply' – применение пресета
# 'rollback' – откат к предыдущему снимку
action: Mapped[Optional[str]] = mapped_column(String(50))
```

```
# Комментарий (для отладки и UI):
# 'ai_edit' → текст запроса к LLM
# 'preset_apply' → имя пресета
# 'rollback' → id снимка, к которому откатились
note: Mapped[Optional[str]] = mapped_column(Text)
```

```
created_at: Mapped[dt] = mapped_column(DateTime, default=dt.now)
```

```
__table_args__ = (
    Index('idx_page_hist_page_id', 'page_id'),
    Index('idx_page_hist_created_at', 'created_at'),
    Index('idx_page_hist_page_created', 'page_id', 'created_at'),
)
```

```
=====
FILE: core/models/page_pres.py
=====
```

```
# neurocad/core/models/page_pres.py
```

```
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text, Integer, DateTime, Index
from datetime import datetime as dt
from typing import Optional
from .base import Base
```

```
class PagePres(Base):
```

```
    """
```

```
    Global preset (page template).
```

```
    Used in LLM editor (left panel) – a library of ready-made templates.
    Not tied to a specific page.
```

```
    """
```

```
    __tablename__ = "page_pres"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(Text)
```

```
    # HTML of preset (current state)
```

```

html: Mapped[Optional[str]] = mapped_column(Text)

# CSS of preset (current state)
css: Mapped[Optional[str]] = mapped_column(Text)

# Path to PNG preview relative to media/
# e.g. "presets/1.png"
thumbnail_path: Mapped[Optional[str]] = mapped_column(String(500))

is_delete: Mapped[int] = mapped_column(Integer, default=0)
created_at: Mapped[dt] = mapped_column(DateTime, default=dt.now)
updated_at: Mapped[dt] = mapped_column(DateTime, default=dt.now, onupdate=dt.now)

__table_args__ = (
    Index('idx_page_pres_is_delete', 'is_delete'),
    Index('idx_page_pres_created_at', 'created_at'),
)

=====
FILE: core/models/page.py
=====
# app/core/models/page.py

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text, Integer, DateTime, Index, ForeignKey
from datetime import datetime as dt
from typing import Optional
from .base import Base

class Page(Base):
    __tablename__ = "pages"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)

    # Nav instance this page belongs to (a concrete object owned by a user).
    # Each Nav row = one object of some module ("class"); pages belong
    # to the object, not to the module/class.
    nav_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("nav.id"),
        nullable=False,
    )

    datetime: Mapped[dt] = mapped_column(DateTime, nullable=False)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(Text)
    logo: Mapped[Optional[str]] = mapped_column(Text)

    # HTML for display
    content: Mapped[Optional[str]] = mapped_column(Text)

    # GrapesJS JSON for editor
    content_json: Mapped[Optional[str]] = mapped_column(Text)

    # CSS страницы (источник правды).

```

```

# HTML – в content, CSS – здесь. При рендере CSS выгружается
# в static/pages/<id>.css, в HTML идёт <link> с ?v=<hash>.
css: Mapped[Optional[str]] = mapped_column(Text)

# ===== Template system =====
# is_template = True → this page can be used as a base template
#   by other pages (shown in "Наследовать от" dropdown).
# template_id = None → standalone page (renders its own content).
# template_id = <id> → inherits layout from that template page;
#   its own `content` is inserted into the [data-slot="content"] slot.
is_template: Mapped[int] = mapped_column(Integer, default=0)
template_id: Mapped[Optional[int]] = mapped_column(
    Integer,
    ForeignKey("pages.id"),
    nullable=True,
    default=None,
)

is_active: Mapped[int] = mapped_column(Integer, default=1)
is_delete: Mapped[int] = mapped_column(Integer, default=0)
created_at: Mapped[dt] = mapped_column(DateTime, default=dt.now)
updated_at: Mapped[dt] = mapped_column(DateTime, default=dt.now, onupdate=dt.now)
rss_yandex_id: Mapped[Optional[str]] = mapped_column(String(64), default=None)

__table_args__ = (
    Index('idx_pages_nav_id', 'nav_id'),
    Index('idx_pages_nav_datetime', 'nav_id', 'datetime'),
    Index('idx_pages_datetime', 'datetime'),
    Index('idx_pages_is_delete', 'is_delete'),
    Index('idx_pages_is_active', 'is_active'),
    Index('idx_pages_rss_yandex_id', 'rss_yandex_id'),
    Index('idx_pages_is_template', 'is_template'),
    Index('idx_pages_template_id', 'template_id'),
)

```

```

=====
FILE: core/models/run.py
=====

```

```

# neurocad/core/models/run.py

```

```

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Integer, Text, DateTime, ForeignKey, Index
from datetime import datetime as dt
from typing import Optional
from .base import Base

```

```

class Run(Base):
    """
    One multi-step LLM generation run.

    Lifecycle:
    pending   → created, nothing started
    planning  → step 1 (plan) is running
    filling   → step 2 (fill) is running
    effects   → step 3 (effects) is running
    """

```

done → finished successfully, final_html is set
cancelled → user pressed Stop
failed → error during any step, `error` is set

Every step writes its result to the corresponding column, so the run can be resumed or inspected even if the WebSocket connection dropped. This also makes the state visible to all uvicorn workers, since it lives in the shared SQLite database.

```
"""
__tablename__ = "runs"

id: Mapped[int] = mapped_column(Integer, primary_key=True)

# Which page this run belongs to
page_id: Mapped[int] = mapped_column(
    Integer,
    ForeignKey("pages.id"),
    nullable=False,
)

# Who started it (superadmin)
user_id: Mapped[Optional[int]] = mapped_column(
    Integer,
    ForeignKey("users.id"),
    nullable=True,
)

# Lifecycle
status: Mapped[str] = mapped_column(
    String(32),
    nullable=False,
    default="pending",
)

step: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

# Input
user_message: Mapped[str] = mapped_column(Text, nullable=False)
block_catalog: Mapped[Optional[str]] = mapped_column(Text) # JSON

# Intermediate results
plan: Mapped[Optional[str]] = mapped_column(Text) # JSON
filled: Mapped[Optional[str]] = mapped_column(Text) # JSON
effects: Mapped[Optional[str]] = mapped_column(Text) # JSON

# Final result
final_html: Mapped[Optional[str]] = mapped_column(Text)
message: Mapped[Optional[str]] = mapped_column(Text)
error: Mapped[Optional[str]] = mapped_column(Text)

# Audit
created_at: Mapped[dt] = mapped_column(DateTime, default=dt.now)
updated_at: Mapped[dt] = mapped_column(
    DateTime, default=dt.now, onupdate=dt.now
)

__table_args__ = (
```

```

        Index('idx_runs_page_id', 'page_id'),
        Index('idx_runs_user_id', 'user_id'),
        Index('idx_runs_status', 'status'),
        Index('idx_runs_page_status', 'page_id', 'status'),
    )

```

```

=====
FILE: core/models/setting.py
=====

```

```

# app/core/models/setting.py

```

```

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text, Integer, Boolean, DateTime, Index
from datetime import datetime
from typing import Optional
from .base import Base

```

```

class Setting(Base):
    __tablename__ = "settings"

    id: Mapped[int] = mapped_column(primary_key=True)
    domain: Mapped[str] = mapped_column(String(64), nullable=False)
    subsys: Mapped[str] = mapped_column(String(64), nullable=False)
    module: Mapped[str] = mapped_column(String(64), nullable=False)
    section: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
    key: Mapped[str] = mapped_column(String(128), nullable=False)
    value: Mapped[str] = mapped_column(Text, nullable=False)
    caption: Mapped[str] = mapped_column(String(64), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    is_delete: Mapped[bool] = mapped_column(Boolean, default=False, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.now, nullable=True)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.now, onupdate=datetime.now, nullable=True)

    __table_args__ = (
        Index('idx_settings_lookup', 'domain', 'subsys', 'module', 'section', 'key', unique=True),
        Index('idx_settings_is_delete', 'is_delete'),
    )

```

```

=====
FILE: core/models/user.py
=====

```

```

# app/core/models/user.py

```

```

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Integer, Boolean, DateTime, ForeignKey, Index, Text
from datetime import datetime as dt
from typing import Optional
from .base import Base

```

```

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    login: Mapped[str] = mapped_column(String(64), nullable=False, unique=True)

```

```

password: Mapped[str] = mapped_column(String(256), nullable=False)
name: Mapped[Optional[str]] = mapped_column(String(128))
email: Mapped[Optional[str]] = mapped_column(String(128))
is_superuser: Mapped[bool] = mapped_column(Boolean, default=False)
is_active: Mapped[bool] = mapped_column(Boolean, default=True)
is_delete: Mapped[bool] = mapped_column(Boolean, default=False)
last_seen: Mapped[Optional[dt]] = mapped_column(DateTime)
created_at: Mapped[dt] = mapped_column(DateTime, default=dt.now)

# Домен второго уровня, подключённый пользователем (example.com).
# NULL — пользователь не подключал кастомный домен.
# Значение — FQDN в нижнем регистре. Без unique и index —
# проверка уникальности не нужна, домен принадлежит конкретному
# человеку и проверяется через DNS.
domain: Mapped[Optional[str]] = mapped_column(String(253), nullable=True)

# Главная страница пользователя.
#
# Используется, когда пользователь заходит на свой поддомен
# (<login>.<APP_DOMAIN>) или на свой кастомный домен БЕЗ пути —
# например, https://testuser3.neurocad-dev.ru/ .
# В этом случае middleware резолвит этот id и редиректит на
# публичную страницу (/page/<nav_id>/<date>/<time>).
#
# NULL — главная не задана. Тогда показывается первая страница
# nav'a по datetime ASC; если и её нет — редирект на
# /core/engine/pages (каталог статей).
#
# ondelete='SET NULL' — если страницу удалят, поле обнулится
# само, пользователь не останется с висящей ссылкой.
home_page_id: Mapped[Optional[int]] = mapped_column(
    Integer,
    ForeignKey('pages.id', ondelete='SET NULL'),
    nullable=True,
)

# robots.txt для домена 3 уровня (<login>.<APP_DOMAIN>).
#
# NULL — пользователь никогда не открывал модальку robots.txt
# для поддомена. В этом случае /robots.txt на поддомене
# отдаёт ROBOTS_CLOSED (закрыто от всех) — дефолт для новых
# пользователей.
#
# Автоматически управляется в service.py:
# - add_custom → robots_3 = ROBOTS_CLOSED (поддомен закрыт,
# пока активен кастомный домен);
# - remove_custom → robots_3 = ROBOTS_OPEN (поддомен снова
# открыт после отключения кастомного).
#
# Text, а не String(N): содержимое robots.txt может быть
# длинным (Allow/Disallow/Crawl-delay/Sitemap на много строк),
# ограничения по длине здесь ставить не нужно.
robots_3: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

# robots.txt для домена 2 уровня (кастомного).
#

```

```

# NULL – пользователь никогда не открывал модальку robots.txt
# для кастомного домена. В этом случае /robots.txt на
# кастомном домене отдаёт ROBOTS_CLOSED (закрыто от всех).
#
# Редактируется пользователем через модальное окно в
# Профиль → Домены → «Редактировать robots.txt». Значение
# сохраняется в БД верbatim (включая пустую строку) – бэкенд
# не парсит и не валидирует синтаксис robots.txt.
#
# При подключении кастомного домена (add_custom) это поле
# инициализируется ROBOTS_OPEN, если было NULL. При отключении
# (remove_custom) не трогается – текст сохраняется на случай
# повторного подключения.
robots_2: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

```

```

__table_args__ = (
    Index('idx_users_login', 'login'),
    Index('idx_users_is_delete', 'is_delete'),
)

```

```

=====
FILE: core/auth/dependencies.py
=====

```

```

# app/core/auth/dependencies.py

```

```

from fastapi import Depends, HTTPException, status, Request
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from jose import JWTError, jwt
from datetime import datetime, timedelta
from typing import Optional, Dict, Any

```

```

from ...config import settings
from .service import CoreAuthService

```

```

security = HTTPBearer(auto_error=False)

```

```

class CoreAuthDependencies:

```

```

    """Зависимости для авторизации"""

```

```

    @staticmethod

```

```

    def create_access_token(data: dict, expires_delta: timedelta = None) -> str:

```

```

        """Создание JWT токена"""

```

```

        to_encode = data.copy()

```

```

        if expires_delta:

```

```

            expire = datetime.utcnow() + expires_delta

```

```

        else:

```

```

            expire = datetime.utcnow() + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)

```

```

        to_encode.update({"exp": expire})

```

```

        return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)

```

```

    @staticmethod

```

```

    def get_token_from_request(request: Request) -> Optional[str]:

```

```

        """Получает токен из заголовка Authorization или из cookie"""

```

```

        auth_header = request.headers.get("Authorization")

```

```

        if auth_header and auth_header.startswith("Bearer "):

```

```

        return auth_header[7:]

    token = request.cookies.get("access_token")
    if token:
        return token

    return None

@staticmethod
async def get_current_user(
    request: Request,
    credentials: HTTPAuthorizationCredentials = Depends(security)
) -> Dict[str, Any]:
    """
    Обязательная авторизация

    Возвращает данные пользователя или выбрасывает 401 ошибку с JSON
    """
    token = CoreAuthDependencies.get_token_from_request(request)

    if not token:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail={
                "success": False,
                "message": "Требуется авторизация"
            },
            headers={"WWW-Authenticate": "Bearer"}
        )

    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        user_id: int = payload.get("sub")
        if user_id is None:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail={
                    "success": False,
                    "message": "Недействительный токен"
                },
                headers={"WWW-Authenticate": "Bearer"}
            )
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail={
                "success": False,
                "message": "Недействительный токен"
            },
            headers={"WWW-Authenticate": "Bearer"}
        )

    log = request.app.state.log if hasattr(request.app.state, 'log') else None
    user = await CoreAuthService.get_user_by_id(int(user_id), log=log)

    if user is None:

```



```

        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail={
                "success": False,
                "message": "Пользователь не найден"
            },
            headers={"WWW-Authenticate": "Bearer"}
        )

    return user

@staticmethod
async def get_current_user_optional(
    request: Request,
    credentials: HTTPAuthorizationCredentials = Depends(security)
) -> Optional[Dict[str, Any]]:
    """
    Опциональная авторизация

    Возвращает данные пользователя или None если не авторизован
    """
    token = CoreAuthDependencies.get_token_from_request(request)

    if not token:
        return None

    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        user_id: int = payload.get("sub")
        if user_id is None:
            return None
    except JWTError:
        return None

    log = request.app.state.log if hasattr(request.app.state, 'log') else None
    user = await CoreAuthService.get_user_by_id(int(user_id), log=log)
    return user

```

```

# Для обратной совместимости с существующим кодом
create_access_token = CoreAuthDependencies.create_access_token
get_token_from_request = CoreAuthDependencies.get_token_from_request
get_current_user = CoreAuthDependencies.get_current_user
get_current_user_optional = CoreAuthDependencies.get_current_user_optional

```

```

=====
FILE: core/auth/route.py
=====

```

```

# app/core/auth/route.py

```

```

from fastapi import APIRouter

from .login.route import router as login_router
from .register.route import router as register_router
from .password.route import router as password_router
from .restore.route import router as restore_router

```

```
from .profile.route import router as profile_router
```

```
router = APIRouter(prefix="/auth", tags=["auth"])
```

```
# Подключаем все под-роутеры
router.include_router(login_router)
router.include_router(register_router)
router.include_router(password_router)
router.include_router(restore_router)
router.include_router(profile_router)
```

```
=====
FILE: core/auth/schema.py
=====
```

```
# app/core/auth/schema.py
```

```
from pydantic import BaseModel, Field, EmailStr
from typing import Optional
from datetime import datetime
```

```
class CoreAuthUserOutSchema(BaseModel):
    """Безопасный вывод данных пользователя"""
    id: int
    login: str
    name: Optional[str] = None
    email: Optional[str] = None
    is_superuser: bool = False
    is_active: bool = True
    created_at: Optional[datetime] = None
    last_seen: Optional[datetime] = None

    class Config:
        from_attributes = True
```

```
class CoreAuthMessageSchema(BaseModel):
    """Стандартный ответ с сообщением"""
    success: bool
    message: str
    data: Optional[dict] = None
```

```
class CoreAuthTokenResponseSchema(BaseModel):
    """Ответ с токеном (если понадобится)"""
    access_token: str
    token_type: str = "bearer"
```

```
=====
FILE: core/auth/service.py
=====
```

```
# app/core/auth/service.py
```

```
from sqlalchemy import select, or_
from datetime import datetime
from typing import Optional, Dict, Any
```

```

from ...utils.sqlite import get_db_sqlite
from ...utils.hash import get_hash_string
from ..models.base import User

def shorten_name(full_name: str) -> str:
    """Преобразует 'Иванов Иван Иванович' в 'Иванов И.И.'"""
    if not full_name:
        return full_name

    parts = full_name.strip().split()
    if len(parts) >= 2:
        surname = parts[0]
        initials = ''.join([f"{part[0]}." for part in parts[1:]])
        return f"{surname} {initials}"
    return full_name

def serialize_datetime(value):
    """Преобразует datetime в строку ISO format"""
    if isinstance(value, datetime):
        return value.isoformat()
    return value

def serialize_user(user) -> Dict[str, Any]:
    """Сериализует объект User в dict с преобразованием datetime"""
    return {
        "id": user.id,
        "login": user.login,
        "name": user.name,
        "email": user.email,
        "password": user.password,
        "is_superuser": user.is_superuser,
        "is_active": user.is_active,
        "is_delete": user.is_delete,
        "last_seen": serialize_datetime(user.last_seen),
        "created_at": serialize_datetime(user.created_at),
        "domain": getattr(user, "domain", None),
    }

class CoreAuthService:
    """Общие методы для работы с пользователями"""

    # =====
    # READ
    # =====

    @staticmethod
    async def get_user_by_id(user_id: int, log=None) -> Optional[Dict[str, Any]]:
        """Получение пользователя по ID"""
        if log:
            await log.log_info(target="auth", message=f"get_user_by_id: user_id={user_id}")

```

```

async for session in get_db_sqlite():
    stmt = select(User).where(User.id == user_id, User.is_delete == False)
    result = await session.execute(stmt)
    user = result.scalar_one_or_none()

    if user is None:
        if log:
            await log.log_warning(target="auth", message=f"User {user_id} not found")
        return None

    return serialize_user(user)

@staticmethod
async def find_user_by_login_or_email(value: str, log=None) -> Optional[Dict[str, Any]]:
    """Поиск пользователя по логину или email"""
    if log:
        await log.log_info(target="auth", message=f"find_user_by_login_or_email: {value}")

    async for session in get_db_sqlite():
        stmt = select(User).where(
            or_(User.login == value, User.email == value),
            User.is_delete == False
        )
        result = await session.execute(stmt)
        user = result.scalar_one_or_none()

        if user is None:
            return None

        return serialize_user(user)

# =====
# LOGIN / SLUG AVAILABILITY
# =====

@staticmethod
async def is_login_taken(login: str, log=None) -> bool:
    """
    True if a user with this login already exists (lower-cased).

    Comparison is done on the normalized login, so "User1" and
    "user1" collide. is_delete=True users are ignored – a login
    released by an admin is free again.
    """
    from .validators import normalize_login

    normalized = normalize_login(login)
    if not normalized:
        return False

    async for session in get_db_sqlite():
        stmt = select(User).where(
            User.login == normalized,
            User.is_delete == False,
        )
        result = await session.execute(stmt)

```

```

        return result.scalar_one_or_none() is not None
    return False

@staticmethod
async def is_slug_taken(slug: str, log=None) -> bool:
    """
    True if any existing user login slugifies to this slug.

    Two logins with different spellings can produce the same
    slug (e.g. "andrey_k" and "andrey-k"). Since the subdomain
    is built from the slug, only one of them can be registered.
    """
    from .validators import slugify_login

    s = (slug or "").strip()
    if not s:
        return False

    async for session in get_db_sqlite():
        stmt = select(User.login).where(User.is_delete == False)
        result = await session.execute(stmt)
        for (login,) in result.all():
            if slugify_login(login) == s:
                return True
    return False

# =====
# UPDATE
# =====

@staticmethod
async def update_user(user_id: int, log=None, **kwargs) -> Optional[Dict[str, Any]]:
    """Обновление данных пользователя"""
    if log:
        await log.log_info(target="auth", message=f"update_user: user_id={user_id}, fields={list(kwargs.keys())}")

    async for session in get_db_sqlite():
        stmt = select(User).where(User.id == user_id)
        result = await session.execute(stmt)
        user = result.scalar_one_or_none()

        if user is None:
            if log:
                await log.log_warning(target="auth", message=f"User {user_id} not found")
            return None

        for key, value in kwargs.items():
            if hasattr(user, key) and value is not None:
                setattr(user, key, value)

        await session.commit()
        await session.refresh(user)

        if log:
            await log.log_info(target="auth", message=f"User {user_id} updated")
        return serialize_user(user)

```

```
# =====
# CREATE
# =====

@staticmethod
async def create_user(login: str, password: str, name: Optional[str] = None,
                      email: Optional[str] = None, is_superuser: bool = False,
                      log=None) -> Optional[Dict[str, Any]]:
    """Создание нового пользователя"""
    if log:
        await log.log_info(target="auth", message=f"create_user: login={login}")

    hashed_password = get_hash_string(password)

    async for session in get_db_sqlite():
        # Проверяем существование
        stmt = select(User).where(User.login == login)
        result = await session.execute(stmt)
        existing = result.scalar_one_or_none()

        if existing:
            if log:
                await log.log_warning(target="auth", message=f"User {login} already exists")
            return None

        new_user = User(
            login=login,
            password=hashed_password,
            name=name or login,
            email=email,
            is_superuser=is_superuser,
            is_active=True,
            is_delete=False,
            created_at=datetime.now()
        )
        session.add(new_user)
        await session.commit()
        await session.refresh(new_user)

        if log:
            await log.log_info(target="auth", message=f"User {login} created with id={new_user.id}")
    return serialize_user(new_user)
```

```
=====
FILE: core/auth/validators.py
=====
```

```
# app/core/auth/validators.py
```

```
"""
```

```
Login validators.
```

```
A login must:
```

- be at least 8 characters long;
- consist of a-z, 0-9, '-' and '_';
- start with a letter.

The 8-char minimum doubles as a reserved-names rule: every system host (www, api, dev, admin, static, server, ...) and every short brandable word (studio, market, design, coffee, ...) is 7 chars or shorter, so it can never collide with a user subdomain. No separate reserved list is needed.

Storage format: the login is always saved in lower case. Two logins that differ only by case ("User1" / "user1") are the same login.

Subdomain form: the login is slugified (lowercase, a-z0-9-, '_' turned into '-') to build <slug>.neurocad.ru. Slug collisions between different logins ("andrey_k" / "andrey-k") are checked separately in the registration flow.

```
import re
from typing import Optional

LOGIN_MIN_LENGTH = 8
LOGIN_MAX_LENGTH = 64

#: a-z, 0-9, '-' and '_', starting with a letter.
_LOGIN_RE = re.compile(r"^[a-z][a-z0-9_-]*$")

def normalize_login(login: str) -> str:
    """Lower-case and trim a raw login string."""
    return (login or "").strip().lower()

def validate_login(login: str) -> Optional[str]:
    """
    Return None if the login is valid, or a Russian error message
    if not. Never raises.
    """
    s = normalize_login(login)

    if not s:
        return "Введите логин"

    if len(s) < LOGIN_MIN_LENGTH:
        return (
            f"Логин должен быть не короче {LOGIN_MIN_LENGTH} символов – "
            "короткие имена зарезервированы системой"
        )

    if len(s) > LOGIN_MAX_LENGTH:
        return f"Логин должен быть не длиннее {LOGIN_MAX_LENGTH} символов"

    if not _LOGIN_RE.match(s):
        return "Логин может содержать только латиницу, цифры, дефис и подчёркивание"

    return None
```

```

def slugify_login(login: str) -> str:
    """
    Same slugification the domain service uses to build
    <slug>.neurocad.ru. Keep the two in sync.

    - lower-case;
    - '_' becomes '-';
    - everything else that is not a-z / 0-9 / '-' is dropped;
    - leading/trailing '-' is trimmed.
    """
    s = normalize_login(login)
    s = s.replace("_", "-")
    s = re.sub(r"[^a-z0-9-]+", "-", s)
    s = s.strip("-")
    return s or "user"

```

```

FILE: core/auth/__init__.py

```

```

FILE: core/auth/login/route.py

```

```

# app/core/auth/login/route.py

from fastapi import APIRouter, Request
from fastapi.responses import JSONResponse
from datetime import timedelta
import time

from .schema import CoreAuthLoginRequestSchema
from .service import CoreAuthLoginService
from ..dependencies import CoreAuthDependencies
from ....config import settings

router = APIRouter(prefix="/login", tags=["core/auth/login"])

# Хранилище для rate limiting (в продакшене использовать Redis)
login_attempts = {}

@router.post("")
async def login(request: Request, login_data: CoreAuthLoginRequestSchema) -> JSONResponse:
    """
    Вход пользователя

    Возвращает JWT токен в HttpOnly cookie
    """
    log = request.app.state.log if hasattr(request.app.state, 'log') else None
    client_ip = request.client.host if request.client else "unknown"

    # Rate limiting
    now = time.time()
    if client_ip in login_attempts:

```



```

login_attempts[client_ip] = [t for t in login_attempts[client_ip] if now - t < 3600]
if len(login_attempts[client_ip]) >= 5:
    if log:
        await log.log_warning(target="auth", message=f"Rate limit exceeded for IP: {client_ip}")
    return JSONResponse(
        status_code=429,
        content={
            "success": False,
            "message": "Слишком много попыток. Попробуйте через час."
        }
    )

# Аутентификация
user = await CoreAuthLoginService.login(login_data.login, login_data.password, log=log)

if not user:
    # Запоминаем неудачную попытку
    if client_ip not in login_attempts:
        login_attempts[client_ip] = []
    login_attempts[client_ip].append(time.time())

    if log:
        await log.log_warning(target="auth", message=f"Failed login attempt for: {login_data.login} from IP: {client_ip}")
    return JSONResponse(
        status_code=401,
        content={
            "success": False,
            "message": "Неверный логин или пароль"
        }
    )

# Успешный вход - сбрасываем попытки
if client_ip in login_attempts:
    login_attempts[client_ip] = []

# Создаём JWT токен
access_token = CoreAuthDependencies.create_access_token(data={"sub": str(user["id"])})

# Создаём ответ с токеном в cookie
response = JSONResponse(
    status_code=200,
    content={
        "success": True,
        "message": "Вход выполнен успешно",
        "data": {
            "user": {
                "id": user["id"],
                "login": user["login"],
                "name": user["name"],
                "email": user["email"],
                "is_superuser": user["is_superuser"]
            }
        }
    }
)

```

```

# Устанавливаем HttpOnly cookie
response.set_cookie(
    key="access_token",
    value=access_token,
    httponly=True,
    secure=False, # В продакшене установить True (HTTPS)
    samesite="lax",
    max_age=settings.ACCESS_TOKEN_EXPIRE_MINUTES * 60
)

if log:
    await log.log_info(target="auth", message=f"User {user['login']} logged in successfully")
return response

```

```

@router.post("/logout")
async def logout(request: Request) -> JSONResponse:
    """
    Выход пользователя

    Удаляет HttpOnly cookie с токеном
    """
    log = request.app.state.log if hasattr(request.app.state, 'log') else None

    response = JSONResponse(
        status_code=200,
        content={
            "success": True,
            "message": "Выход выполнен успешно"
        }
    )
    response.delete_cookie("access_token")

    if log:
        await log.log_info(target="auth", message="User logged out")
    return response

```

```

=====
FILE: core/auth/login/schema.py
=====

```

```

# app/core/auth/login/schema.py

```

```

from pydantic import BaseModel, Field

```

```

class CoreAuthLoginRequestSchema(BaseModel):
    """Запрос на вход"""
    login: str = Field(..., min_length=1, max_length=100, description="Логин или email")
    password: str = Field(..., min_length=1, description="Пароль")

```

```

=====
FILE: core/auth/login/service.py
=====

```

```

# app/core/auth/login/service.py

```

```

from datetime import datetime

```

```

from typing import Optional, Dict, Any

from sqlalchemy import select

from ..service import CoreAuthService, serialize_user
from ...models.base import User
from ...utils.sqlite import get_db_sqlite
from ...utils.hash import get_hash_string

class CoreAuthLoginService:
    """Сервис для аутентификации и входа"""

    @staticmethod
    async def authenticate_user(login: str, hashed_password: str, log=None) -> Optional[Dict[str, Any]]:
        """Аутентификация пользователя по логину и хэшу пароля"""
        if log:
            await log.log_info(target="auth", message=f"authenticate_user called for login: {login}")

        async for session in get_db_sqlite():
            stmt = select(User).where(
                User.login == login,
                User.is_delete == False
            )
            result = await session.execute(stmt)
            user = result.scalar_one_or_none()

            if user is None:
                if log:
                    await log.log_warning(target="auth", message=f"User not found for login: {login}")
                return None

            if user.password != hashed_password:
                if log:
                    await log.log_warning(target="auth", message=f"Invalid password for login: {login}")
                return None

            if log:
                await log.log_info(target="auth", message=f"User authenticated: id={user.id}, login={user.login}")

            # Обновляем время последнего визита
            user.last_seen = datetime.now()
            await session.commit()
            await session.refresh(user)

            return serialize_user(user)

    @staticmethod
    async def login(login: str, password: str, log=None) -> Optional[Dict[str, Any]]:
        """Вход с автоматическим хэшированием пароля"""
        if log:
            await log.log_info(target="auth", message=f"login called for: {login}")
        hashed_password = get_hash_string(password)
        return await CoreAuthLoginService.authenticate_user(login, hashed_password, log=log)

    @staticmethod

```

```

async def update_last_seen(user_id: int, log=None) -> None:
    """Обновление времени последнего визита"""
    async for session in get_db_sqlite():
        stmt = select(User).where(User.id == user_id)
        result = await session.execute(stmt)
        user = result.scalar_one_or_none()

        if user:
            user.last_seen = datetime.now()
            await session.commit()
            if log:
                await log.log_info(target="auth", message=f"Updated last_seen for user {user_id}")

```

```

=====
FILE: core/auth/login/__init__.py
=====

```

```

=====
FILE: core/auth/password/route.py
=====

```

```

# app/core/auth/password/route.py

```

```

from fastapi import APIRouter, Depends, Request
from fastapi.responses import JSONResponse

from .schema import CoreAuthPasswordChangeRequestSchema
from .service import CoreAuthPasswordService
from ..dependencies import CoreAuthDependencies

router = APIRouter(prefix="/password", tags=["core/auth/password"])

@router.post("/change")
async def change_password(
    request: Request,
    password_data: CoreAuthPasswordChangeRequestSchema,
    current_user: dict = Depends(CoreAuthDependencies.get_current_user)
) -> JSONResponse:
    """
    Смена пароля пользователя

    Требуется авторизации
    """
    log = request.app.state.log if hasattr(request.app.state, 'log') else None

    result = await CoreAuthPasswordService.change_password(
        user_id=current_user["id"],
        current_password=password_data.current_password,
        new_password=password_data.new_password,
        new_password_confirm=password_data.new_password_confirm,
        log=log
    )

    if not result["success"]:
        return JSONResponse(

```

```

        status_code=400,
        content=result
    )

    return JSONResponse(
        status_code=200,
        content=result
    )

```

```

=====
FILE: core/auth/password/schema.py
=====

```

```

# app/core/auth/password/schema.py

```

```

from pydantic import BaseModel, Field

```

```

class CoreAuthPasswordChangeRequestSchema(BaseModel):
    """Запрос на смену пароля"""
    current_password: str = Field(..., min_length=1, description="Текущий пароль")
    new_password: str = Field(..., min_length=6, description="Новый пароль (минимум 6 символов)")
    new_password_confirm: str = Field(..., min_length=6, description="Подтверждение нового пароля")

```

```

=====
FILE: core/auth/password/service.py
=====

```

```

# app/core/auth/password/service.py

```

```

from typing import Dict, Any

```

```

from sqlalchemy import select

```

```

from ...models.base import User
from ..service import CoreAuthService
from ...utils.sqlite import get_db_sqlite
from ...utils.hash import get_hash_string

```

```

class CoreAuthPasswordService:
    """Сервис для работы с паролями"""

```

```

    @staticmethod
    async def change_password(
        user_id: int,
        current_password: str,
        new_password: str,
        new_password_confirm: str,
        log=None
    ) -> Dict[str, Any]:
        """
        Смена пароля пользователя

```

```

        Returns:

```

```

            Dict с ключами:
            - success: bool
            - message: str

```

```

"""
if log:
    await log.log_info(target="auth", message=f"change_password called for user_id: {user_id}")

# Проверяем, существует ли пользователь
user = await CoreAuthService.get_user_by_id(user_id, log=log)
if not user:
    return {
        "success": False,
        "message": "Пользователь не найден"
    }

# Проверяем текущий пароль
hashed_current = get_hash_string(current_password)
if user["password"] != hashed_current:
    if log:
        await log.log_warning(target="auth", message=f"Invalid current password for user_id: {user_id}")
    return {
        "success": False,
        "message": "Неверный текущий пароль"
    }

# Проверяем новый пароль
if new_password != new_password_confirm:
    return {
        "success": False,
        "message": "Пароли не совпадают"
    }

if len(new_password) < 6:
    return {
        "success": False,
        "message": "Пароль должен содержать минимум 6 символов"
    }

# Обновляем пароль
hashed_new = get_hash_string(new_password)
result = await CoreAuthService.update_user(user_id, log=log, password=hashed_new)

if not result:
    return {
        "success": False,
        "message": "Ошибка при обновлении пароля"
    }

if log:
    await log.log_info(target="auth", message=f"Password changed for user_id: {user_id}")
return {
    "success": True,
    "message": "Пароль успешно изменён"
}

```

```

=====
FILE: core/auth/password/__init__.py
=====

```

```

=====
FILE: core/auth/profile/route.py
=====

# app/core/auth/profile/route.py

from fastapi import APIRouter, Depends, Request
from fastapi.responses import JSONResponse

from .schema import CoreAuthProfileUpdateRequestSchema
from .service import CoreAuthProfileService
from ..dependencies import CoreAuthDependencies

router = APIRouter(prefix="/profile", tags=["core/auth/profile"])

@router.get("")
async def get_profile(
    request: Request,
    current_user: dict = Depends(CoreAuthDependencies.get_current_user)
) -> JSONResponse:
    """
    Получение профиля текущего пользователя

    Требуется авторизации
    """
    log = request.app.state.log if hasattr(request.app.state, 'log') else None

    profile = await CoreAuthProfileService.get_profile(current_user["id"], log=log)

    if not profile:
        return JSONResponse(
            status_code=404,
            content={
                "success": False,
                "message": "Пользователь не найден"
            }
        )

    return JSONResponse(
        status_code=200,
        content={
            "success": True,
            "data": {
                "user": {
                    "id": profile["id"],
                    "login": profile["login"],
                    "name": profile["name"],
                    "email": profile["email"],
                    "is_superuser": profile["is_superuser"],
                    "is_active": profile["is_active"],
                    "last_seen": profile.get("last_seen"),
                    "created_at": profile.get("created_at")
                }
            }
        }
    )

```

```

)

@router.put("")
async def update_profile(
    request: Request,
    profile_data: CoreAuthProfileUpdateRequestSchema,
    current_user: dict = Depends(CoreAuthDependencies.get_current_user)
) -> JSONResponse:
    """
    Обновление профиля текущего пользователя

    Требуется авторизации
    """
    log = request.app.state.log if hasattr(request.app.state, 'log') else None

    result = await CoreAuthProfileService.update_profile(
        user_id=current_user["id"],
        name=profile_data.name,
        email=profile_data.email,
        log=log
    )

    if not result["success"]:
        return JSONResponse(
            status_code=400,
            content=result
        )

    return JSONResponse(
        status_code=200,
        content=result
    )

```

```

=====
FILE: core/auth/profile/schema.py
=====
# app/core/auth/profile/schema.py

```

```

from pydantic import BaseModel, Field, EmailStr
from typing import Optional

```

```

class CoreAuthProfileUpdateRequestSchema(BaseModel):
    """Запрос на обновление профиля"""
    name: Optional[str] = Field(None, max_length=200, description="Полное имя")
    email: Optional[EmailStr] = Field(None, description="Email")

```

```

=====
FILE: core/auth/profile/service.py
=====
# app/core/auth/profile/service.py

```

```

from typing import Optional, Dict, Any

from ..service import CoreAuthService

```



```

class CoreAuthProfileService:
    """Сервис для работы с профилем пользователя"""

    @staticmethod
    async def get_profile(user_id: int, log=None) -> Optional[Dict[str, Any]]:
        """Получение профиля пользователя"""
        if log:
            await log.log_info(target="auth", message=f"get_profile called for user_id: {user_id}")
        return await CoreAuthService.get_user_by_id(user_id, log=log)

    @staticmethod
    async def update_profile(
        user_id: int,
        name: Optional[str] = None,
        email: Optional[str] = None,
        log=None
    ) -> Dict[str, Any]:
        """
        Обновление профиля пользователя

        Returns:
            Dict с ключами:
            - success: bool
            - message: str
            - data: dict с обновлёнными данными (при успехе)
        """
        if log:
            await log.log_info(target="auth", message=f"update_profile called for user_id: {user_id}")

        # Проверяем email на уникальность, если он указан
        if email:
            existing = await CoreAuthService.find_user_by_login_or_email(email, log=log)
            if existing and existing["id"] != user_id:
                if log:
                    await log.log_warning(target="auth", message=f"Email {email} already exists for another user")
                return {
                    "success": False,
                    "message": "Пользователь с таким email уже существует"
                }

        # Обновляем данные
        update_data = {}
        if name is not None:
            update_data["name"] = name
        if email is not None:
            update_data["email"] = email

        if not update_data:
            return {
                "success": False,
                "message": "Нет данных для обновления"
            }

        user = await CoreAuthService.update_user(user_id, log=log, **update_data)

```

```

if not user:
    return {
        "success": False,
        "message": "Ошибка при обновлении профиля"
    }

if log:
    await log.log_info(target="auth", message=f"Profile updated for user_id: {user_id}")
return {
    "success": True,
    "message": "Профиль успешно обновлён",
    "data": {
        "id": user["id"],
        "login": user["login"],
        "name": user["name"],
        "email": user["email"],
        "is_superadmin": user["is_superadmin"],
        "is_active": user["is_active"],
        "last_seen": user.get("last_seen"),
        "created_at": user.get("created_at")
    }
}

```

```

=====
FILE: core/auth/profile/__init__.py
=====

```

```

=====
FILE: core/auth/register/route.py
=====

```

```
# app/core/auth/register/route.py
```

```

from fastapi import APIRouter, Request, Depends
from fastapi.responses import JSONResponse

from .schema import CoreAuthRegisterRequestSchema
from .service import CoreAuthRegisterService
from ..dependencies import CoreAuthDependencies
from ..validators import validate_login, normalize_login, slugify_login
from ..service import CoreAuthService

```

```
router = APIRouter(prefix="/register", tags=["core/auth/register"])
```

```

# =====
# CHECK LOGIN (public, live-check for the signup form)
# =====

```

```

@router.get("/check-login")
async def check_login(request: Request, login: str) -> JSONResponse:
    """

```

Публичная проверка логина для формы регистрации.

Возвращает:

```
{
    "available": bool,
    "reason": "invalid" | "taken" | None,
    "error": str | None
}
```

Ничего не пишет в БД, только читает. Дешёвый эндпоинт — фронт вызывает его с debounce ~400 мс после последнего нажатия клавиши.

Позже, если появится нагрузка от ботов, навесьте rate-limit по IP (slowapi или свой middleware).

```
"""
log = request.app.state.log if hasattr(request.app.state, "log") else None

# ---- 1. Формат ----
err = validate_login(login)
if err:
    return JSONResponse({
        "available": False,
        "reason": "invalid",
        "error": err,
    })

normalized = normalize_login(login)
slug = slugify_login(normalized)

# ---- 2. Логин занят ----
if await CoreAuthService.is_login_taken(normalized, log=log):
    return JSONResponse({
        "available": False,
        "reason": "taken",
        "error": "Этот логин уже занят",
    })

# ---- 3. Slug занят (другой логин → тот же поддомен) ----
if await CoreAuthService.is_slug_taken(slug, log=log):
    return JSONResponse({
        "available": False,
        "reason": "taken",
        "error": "Этот логин уже занят",
    })

# ---- 4. Свободен ----
return JSONResponse({
    "available": True,
    "reason": None,
    "error": None,
})
```

```
# =====
# REGISTER
# =====
```

```
@router.post("")
```

```

async def register(request: Request, register_data: CoreAuthRegisterRequestSchema) -> JSONResponse:
    """
    Регистрация нового пользователя.
    """
    log = request.app.state.log if hasattr(request.app.state, 'log') else None

    result = await CoreAuthRegisterService.register_user(
        login=register_data.login,
        password=register_data.password,
        password_confirm=register_data.password_confirm,
        name=register_data.name,
        email=register_data.email,
        log=log
    )

    if not result["success"]:
        return JSONResponse(
            status_code=400,
            content=result
        )

    return JSONResponse(
        status_code=201,
        content=result
    )

```

```

# =====
# DELETE USER (superadmin only)
# =====

```

```

@router.delete("/{user_id}")
async def delete_user(
    request: Request,
    user_id: int,
    current_user: dict = Depends(CoreAuthDependencies.get_current_user)
) -> JSONResponse:
    """
    Удаление пользователя (мягкое удаление).

    Только для суперпользователей.
    """
    log = request.app.state.log if hasattr(request.app.state, 'log') else None

    # Проверяем права
    if not current_user.get("is_superadmin"):
        if log:
            await log.log_warning(target="auth", message=f"User {current_user['id']} tried to delete user {user_id} without p")
        return JSONResponse(
            status_code=403,
            content={
                "success": False,
                "message": "Недостаточно прав для удаления пользователя"
            }
        )

```

```

# Проверяем, что не удаляем самого себя
if current_user["id"] == user_id:
    return JSONResponse(
        status_code=400,
        content={
            "success": False,
            "message": "Нельзя удалить самого себя"
        }
    )

# Мягкое удаление (устанавливаем is_delete = True)
result = await CoreAuthService.update_user(user_id, log=log, is_delete=True)

if not result:
    return JSONResponse(
        status_code=404,
        content={
            "success": False,
            "message": "Пользователь не найден"
        }
    )

if log:
    await log.log_info(target="auth", message=f"User {user_id} deleted by {current_user['login']}")

return JSONResponse(
    status_code=200,
    content={
        "success": True,
        "message": "Пользователь успешно удалён"
    }
)

```

```

=====
FILE: core/auth/register/schema.py
=====
# app/core/auth/register/schema.py

```

```

from pydantic import BaseModel, Field, EmailStr
from typing import Optional

```

```

class CoreAuthRegisterRequestSchema(BaseModel):
    """
    Запрос на регистрацию.

    Логин: от 8 символов, начинается с буквы, содержит только
    латиницу, цифры, дефис и подчёркивание. Ограничение длины
    одновременно резервирует все системные имена (www, api, dev,
    admin, ...) — они короче 8 символов и не могут быть заняты
    пользователем.
    """
    login: str = Field(
        ...,
        min_length=8,
        max_length=64,
    )

```

```

        pattern=r"^[a-zA-Z][a-zA-Z0-9_-]*$",
        description="Логин: латиница, цифры, дефис, подчёркивание; от 8 символов",
    )
    password: str = Field(
        ...,
        min_length=8,
        description="Пароль (минимум 8 символов)",
    )
    password_confirm: str = Field(
        ...,
        min_length=8,
        description="Подтверждение пароля",
    )
    name: Optional[str] = Field(
        None,
        max_length=200,
        description="Полное имя",
    )
    email: Optional[EmailStr] = Field(
        None,
        description="Email",
    )

```

```

=====
FILE: core/auth/register/service.py
=====

```

```

# app/core/auth/register/service.py

```

```

from datetime import datetime
from typing import Optional, Dict, Any

from sqlalchemy import select

from ...models.base import User, Module, Nav
from ..service import CoreAuthService, serialize_datetime
from ..validators import (
    validate_login,
    normalize_login,
    slugify_login,
)
from ...utils.sqlite import get_db_sqlite
from ...utils.hash import get_hash_string
from ...config import settings

```

```

class CoreAuthRegisterService:
    """Сервис для регистрации пользователей"""

    @staticmethod
    async def register_user(
        login: str,
        password: str,
        password_confirm: str,
        name: Optional[str] = None,
        email: Optional[str] = None,
        log=None
    )

```

```

) -> Dict[str, Any]:
    """
    Регистрация нового пользователя.

    Порядок проверок:
    1. Формат логина (длина >= 8, разрешённые символы).
    2. Логин уже занят (точное совпадение в users.login).
    3. Slug уже занят (другой логин даёт тот же поддомен).
    4. Email уже занят.
    5. Пароли совпадают, длина пароля >= 8.
    6. Создание пользователя + auto-create nav.

    Returns:
    Dict с ключами:
    - success: bool
    - message: str
    - data: dict с данными пользователя (при успехе)
    """
    if log:
        await log.log_info(target="auth", message=f"register_user called for login: {login}")

    # ---- 1. Формат логина ----
    err = validate_login(login)
    if err:
        if log:
            await log.log_warning(target="auth", message=f"Invalid login {login!r}: {err}")
        return {
            "success": False,
            "message": err
        }

    normalized_login = normalize_login(login)
    slug = slugify_login(normalized_login)

    # ---- 2. Логин занят ----
    if await CoreAuthService.is_login_taken(normalized_login, log=log):
        if log:
            await log.log_warning(target="auth", message=f"Login {normalized_login} already exists")
        return {
            "success": False,
            "message": "Этот логин уже занят"
        }

    # ---- 3. Slug занят (другой логин → тот же поддомен) ----
    if await CoreAuthService.is_slug_taken(slug, log=log):
        if log:
            await log.log_warning(
                target="auth",
                message=f"Slug {slug!r} for login {normalized_login!r} already used"
            )
        return {
            "success": False,
            "message": "Этот логин уже занят"
        }

    # ---- 4. Email занят ----

```

```

if email:
    existing_email = await CoreAuthService.find_user_by_login_or_email(email, log=log)
    if existing_email:
        if log:
            await log.log_warning(target="auth", message=f"Email {email} already exists")
        return {
            "success": False,
            "message": "Пользователь с таким email уже существует"
        }

# ---- 5. Пароли ----
if password != password_confirm:
    return {
        "success": False,
        "message": "Пароли не совпадают"
    }

if len(password) < 8:
    return {
        "success": False,
        "message": "Пароль должен содержать минимум 8 символов"
    }

# ---- 6. Создание пользователя ----
user = await CoreAuthService.create_user(
    login=normalized_login,
    password=password,
    name=name or normalized_login,
    email=email,
    is_superuser=False,
    log=log
)

if not user:
    return {
        "success": False,
        "message": "Ошибка при создании пользователя"
    }

# ===== Auto-create personal Nav for the new user =====
#
# When USER_AUTO_CREATE_NAV is True, every new user gets
# their own "Каталог статей" nav entry pointing at the
# built-in 'default' module. This is the object that owns
# their pages; without it, the user would have no place
# to create content.
#
# Non-fatal: if the nav cannot be created (e.g. module
# missing), the user is still registered – we just log
# a warning. Failing hard here would block registration
# for a purely cosmetic reason.
if getattr(settings, "USER_AUTO_CREATE_NAV", False):
    try:
        async for session in get_db_sqlite():
            # Find the 'default' module by name (not by id –
            # the id is an implementation detail and may differ

```



```

# between installations).
mod_stmt = select(Module).where(
    Module.name == "default",
    Module.is_delete == False,
)
default_mod = (await session.execute(mod_stmt)).scalar_one_or_none()

if default_mod is None:
    if log:
        await log.log_warning(
            target="auth",
            message=(
                "USER_AUTO_CREATE_NAV is on, but module "
                "'default' not found – nav not created"
            ),
        )
    else:
        nav = Nav(
            user_id=user["id"],
            parent_id=None,
            card_type="link",
            sort_order=1,
            name="Караног статей",
            description=None,
            icon=None,
            module_id=default_mod.id,
            is_delete=False,
        )
        session.add(nav)
        await session.commit()

        if log:
            await log.log_info(
                target="auth",
                message=(
                    f"Auto-created nav id={nav.id} "
                    f"for user id={user['id']} "
                    f"(module 'default', id={default_mod.id})"
                ),
            )
    except Exception as e:
        if log:
            await log.log_warning(
                target="auth",
                message=f"Failed to auto-create nav for user {user['id']}: {e}",
            )

if log:
    await log.log_info(target="auth", message=f"User {login} registered successfully with id={user['id']}")

return {
    "success": True,
    "message": "Регистрация успешна",
    "data": {
        "id": user["id"],
        "login": user["login"],
    }
}

```

```

        "name": user["name"],
        "email": user["email"],
        "is_superadmin": user["is_superadmin"],
        "created_at": user["created_at"] # Уже строка из serialize_user
    }
}

```

```

=====
FILE: core/auth/register/__init__.py
=====

```

```

=====
FILE: core/auth/restore/route.py
=====

```

```

# app/core/auth/restore/route.py

```

```

from fastapi import APIRouter, Request
from fastapi.responses import JSONResponse

```

```

from .schema import CoreAuthRestoreRequestSchema, CoreAuthRestoreConfirmRequestSchema
from .service import CoreAuthRestoreService

```

```

router = APIRouter(prefix="/restore", tags=["core/auth/restore"])

```

```

@router.post("/request")

```

```

async def restore_request(request: Request, restore_data: CoreAuthRestoreRequestSchema) -> JSONResponse:

```

```

    """

```

```

    Запрос на восстановление пароля

```

```

    Отправляет инструкцию на email (в текущей реализации возвращает токен)

```

```

    """

```

```

    log = request.app.state.log if hasattr(request.app.state, 'log') else None

```

```

    result = await CoreAuthRestoreService.create_reset_token(
        login_or_email=restore_data.login_or_email,
        log=log
    )

```

```

    if not result["success"]:
        return JSONResponse(
            status_code=400,
            content=result
        )

```

```

    return JSONResponse(
        status_code=200,
        content=result
    )

```

```

@router.post("/confirm")

```

```

async def restore_confirm(request: Request, confirm_data: CoreAuthRestoreConfirmRequestSchema) -

```

```

> JSONResponse:

```

```

    """

```

Подтверждение восстановления пароля

Устанавливает новый пароль по токену

"""

```
log = request.app.state.log if hasattr(request.app.state, 'log') else None
```

```
result = await CoreAuthRestoreService.confirm_reset_password(
    token=confirm_data.token,
    new_password=confirm_data.new_password,
    new_password_confirm=confirm_data.new_password_confirm,
    log=log
)
```

```
if not result["success"]:
    return JSONResponse(
        status_code=400,
        content=result
    )
```

```
return JSONResponse(
    status_code=200,
    content=result
)
```

```
=====
FILE: core/auth/restore/schema.py
=====
```

```
# app/core/auth/restore/schema.py
```

```
from pydantic import BaseModel, Field, EmailStr
```

```
class CoreAuthRestoreRequestSchema(BaseModel):
    """Запрос на восстановление пароля"""
    login_or_email: str = Field(..., min_length=1, max_length=100, description="Логин или email")
```

```
class CoreAuthRestoreConfirmRequestSchema(BaseModel):
    """Запрос на подтверждение восстановления пароля"""
    token: str = Field(..., min_length=1, description="Токен сброса пароля")
    new_password: str = Field(..., min_length=6, description="Новый пароль (минимум 6 символов)")
    new_password_confirm: str = Field(..., min_length=6, description="Подтверждение нового пароля")
```

```
=====
FILE: core/auth/restore/service.py
=====
# app/core/auth/restore/service.py
```

```
from datetime import datetime, timedelta
from typing import Optional, Dict, Any
```

```
from sqlalchemy import select
```

```
from ...models.base import User
from ...models.base import PasswordReset
from ..service import CoreAuthService
```

```

from ..dependencies import CoreAuthDependencies
from ....utils.sqlite import get_db_sqlite
from ....utils.hash import get_hash_string

class CoreAuthRestoreService:
    """Сервис для восстановления пароля"""

    @staticmethod
    async def create_reset_token(login_or_email: str, log=None) -> Dict[str, Any]:
        """
        Создание токена для сброса пароля

        Returns:
            Dict с ключами:
            - success: bool
            - message: str
            - data: dict с токеном (при успехе)
        """
        if log:
            await log.log_info(target="auth", message=f"create_reset_token called for: {login_or_email}")

        # Находим пользователя
        user = await CoreAuthService.find_user_by_login_or_email(login_or_email, log=log)
        if not user:
            # Не раскрываем информацию о существовании пользователя
            if log:
                await log.log_info(target="auth", message=f"User not found for: {login_or_email}")
            return {
                "success": True,
                "message": "Если пользователь найден, инструкция отправлена на email"
            }

        # Создаём JWT токен для сброса
        reset_token = CoreAuthDependencies.create_access_token(
            data={"sub": str(user["id"]), "reset": True},
            expires_delta=timedelta(minutes=15)
        )

        # Сохраняем токен в БД
        await CoreAuthRestoreService._save_reset_token(user["id"], reset_token, log=log)

        if log:
            await log.log_info(target="auth", message=f"Reset token created for user_id: {user['id']}")

        # В реальном проекте здесь отправка email
        # А пока возвращаем токен для отладки
        return {
            "success": True,
            "message": "Инструкция отправлена на email",
            "data": {
                "token": reset_token # В продакшене убрать!
            }
        }

    @staticmethod

```

```

async def _save_reset_token(user_id: int, token: str, log=None) -> None:
    """Сохранение токена сброса пароля"""
    async for session in get_db_sqlite():
        # Удаляем старые токены для этого пользователя
        stmt = select(PasswordReset).where(PasswordReset.user_id == user_id)
        result = await session.execute(stmt)
        old_tokens = result.scalars().all()
        for t in old_tokens:
            await session.delete(t)

        # Создаём новый
        reset = PasswordReset(
            user_id=user_id,
            token=token,
            created_at=datetime.now()
        )
        session.add(reset)
        await session.commit()
        if log:
            await log.log_info(target="auth", message=f"Reset token saved for user_id: {user_id}")

    @staticmethod
    async def _verify_reset_token(token: str, log=None) -> Optional[int]:
        """Проверка токена сброса пароля"""
        async for session in get_db_sqlite():
            stmt = select(PasswordReset).where(PasswordReset.token == token)
            result = await session.execute(stmt)
            reset = result.scalar_one_or_none()

            if not reset:
                if log:
                    await log.log_warning(target="auth", message=f"Reset token not found: {token[:10]}...")
                return None

            # Проверяем срок действия (15 минут)
            if datetime.now() - reset.created_at > timedelta(minutes=15):
                if log:
                    await log.log_warning(target="auth", message=f"Reset token expired: {token[:10]}...")
                await session.delete(reset)
                await session.commit()
                return None

            return reset.user_id

    @staticmethod
    async def _invalidate_reset_token(token: str, log=None) -> None:
        """Удаление использованного токена"""
        async for session in get_db_sqlite():
            stmt = select(PasswordReset).where(PasswordReset.token == token)
            result = await session.execute(stmt)
            reset = result.scalar_one_or_none()
            if reset:
                await session.delete(reset)
                await session.commit()
                if log:
                    await log.log_info(target="auth", message=f"Reset token invalidated: {token[:10]}...")

```

```

@staticmethod
async def confirm_reset_password(
    token: str,
    new_password: str,
    new_password_confirm: str,
    log=None
) -> Dict[str, Any]:
    """
    Подтверждение сброса пароля

    Returns:
        Dict с ключами:
        - success: bool
        - message: str
    """
    if log:
        await log.log_info(target="auth", message="confirm_reset_password called")

    # Проверяем пароль
    if new_password != new_password_confirm:
        return {
            "success": False,
            "message": "Пароли не совпадают"
        }

    if len(new_password) < 6:
        return {
            "success": False,
            "message": "Пароль должен содержать минимум 6 символов"
        }

    # Проверяем токен
    user_id = await CoreAuthRestoreService._verify_reset_token(token, log=log)
    if not user_id:
        return {
            "success": False,
            "message": "Ссылка недействительна или истекла"
        }

    # Обновляем пароль
    hashed_password = get_hash_string(new_password)
    result = await CoreAuthService.update_user(user_id, log=log, password=hashed_password)

    if not result:
        return {
            "success": False,
            "message": "Ошибка при обновлении пароля"
        }

    # Удаляем использованный токен
    await CoreAuthRestoreService._invalidate_reset_token(token, log=log)

    if log:
        await log.log_info(target="auth", message=f"Password reset confirmed for user_id: {user_id}")
    return {

```

```

        "success": True,
        "message": "Пароль успешно изменён"
    }
}

```

```

=====
FILE: core/auth/restore/__init__.py
=====

```

```

=====
FILE: core/auth/middleware/auth.py
=====

```

```

# app/core/auth/middleware/auth.py

```

```

from fastapi import FastAPI, Request
from starlette.types import ASGIApp, Scope, Receive, Send

```

```

class CoreAuthMiddlewareAuth:

```

```

    """

```

```

    Pure ASGI middleware.

```

```

    Adds `request.state.user` / `request.state.is_authenticated` for HTTP.
    WebSocket connections are passed through untouched – WS auth is done
    inside the WS endpoint itself (via cookies).
    """

```

```

    def __init__(self, app: ASGIApp):
        self.app = app

```

```

    async def __call__(self, scope: Scope, receive: Receive, send: Send):

```

```

        # --- WebSocket: пропускаем без HTTP-логики ---
        if scope["type"] != "http":
            await self.app(scope, receive, send)
            return

```

```

        # --- HTTP: наша логика ---
        from ..dependencies import CoreAuthDependencies

```

```

        request = Request(scope, receive=receive)
        user = await CoreAuthDependencies.get_current_user_optional(request)
        scope.setdefault("state", {})
        scope["state"]["user"] = user
        scope["state"]["is_authenticated"] = user is not None

```

```

        await self.app(scope, receive, send)

```

```

def setup_auth_middleware(app: FastAPI):
    app.add_middleware(CoreAuthMiddlewareAuth)

```

```

=====
FILE: core/engine/route.py
=====

```

```

# neurocad/core/engine/route.py

```

```

from fastapi import APIRouter, Request, HTTPException
from fastapi.responses import HTMLResponse, JSONResponse
from neurocad.utils.templates import templates, STATIC_VERSION
from neurocad.config import settings
import json
import re
from pathlib import Path
import neurocad

from .lib.route import router as lib_router

router = APIRouter(prefix="/engine", tags=["core/engine"])
router.include_router(lib_router)

# =====
# MODULE ROOT
# =====

# Built-in modules – inside the neurocad package.
NEUROCAD_DIR = Path(neurocad.__file__).parent
BUILTIN_MOD_ROOT = NEUROCAD_DIR / "core" / "engine" / "mod"

def get_mod_root() -> Path:
    """
    Determine the module root directory.

    Priority:
    1. Path from settings.APP_JSON (if set and exists).
       If APP_JSON points to a file, its parent directory is used.
    2. "app/" directory in cwd (if it exists).
    3. Built-in neurocad/core/engine/mod/.
    """
    # 1. From settings
    app_json = getattr(settings, "APP_JSON", None)
    if app_json:
        p = Path(app_json)
        if p.is_dir() and p.exists():
            return p
        if p.is_file() and p.exists():
            return p.parent

    # 2. app/ in cwd
    user_root = Path("app")
    if user_root.exists() and user_root.is_dir():
        return user_root

    # 3. Built-in
    return BUILTIN_MOD_ROOT

# Resolved once at import time.
MOD_ROOT = get_mod_root()

# Debug mode.

```



```

DEBUG = settings.DEBUG

# =====
# AUTH HELPERS
# =====

async def _extract_current_user(request: Request):
    """
    Resolve the current authenticated user from the request.

    `CoreAuthDependencies.get_current_user` is a FastAPI dependency:
    calling it directly leaves `Depends(security)` unresolved. We
    work around that by passing `credentials=None` explicitly – the
    function reads the token from the Authorization header or the
    `access_token` cookie via `get_token_from_request(request)`, so
    the `credentials` argument is not used anyway.

    Returns the user object (may be an ORM model or a dict) or None.
    Never raises – auth failures are treated as "guest".
    """
    try:
        from neurocad.core.auth.dependencies import CoreAuthDependencies

        return await CoreAuthDependencies.get_current_user(
            request=request,
            credentials=None,
        )
    except HTTPException:
        # 401 from the dependency – guest.
        return None
    except Exception as e:
        print(f"[Engine] auth extraction failed: {e}")
        return None


def _user_id_from(user) -> int | None:
    """
    Extract the user id from either a dict or an ORM model.

    `get_current_user` is annotated -> Dict, but the underlying
    service returns an ORM User. Accept both shapes.
    """
    if user is None:
        return None
    if isinstance(user, dict):
        v = user.get("id")
    else:
        v = getattr(user, "id", None)
    try:
        return int(v) if v is not None else None
    except (TypeError, ValueError):
        return None

# =====

```

```

# UTILITIES
# =====

def _parse_path(module_path: str) -> tuple[list[str], list[str]]:
    """
    Split a URL path into path_parts and params_list.

    Rule: a segment consisting ONLY of digits → a parameter.
        Everything before the first digit → path.
        Everything after the first digit → parameters.
    """
    parts = module_path.split("/")
    path_parts: list[str] = []
    params_list: list[str] = []
    in_params = False

    for part in parts:
        if not part:
            continue
        if not in_params and part.isdigit():
            in_params = True
        if in_params:
            params_list.append(part)
        else:
            path_parts.append(part)

    return path_parts, params_list

def _find_config(path_parts: list[str]) -> tuple[Path | None, str, str]:
    """
    Find the config file for the given path.

    Returns: (config_file, module_name, page_name).
    config_file = None when nothing is found.
    """
    for length in range(len(path_parts), 0, -1):
        prefix_parts = path_parts[:length]
        prefix = "/".join(prefix_parts)
        last = prefix_parts[-1]

        # 1. Direct file: mod/{prefix}.json
        direct = MOD_ROOT / f"{prefix}.json"
        if direct.exists() and direct.is_file():
            module_name = "/".join(prefix_parts[:-1]) if length > 1 else prefix_parts[0]
            return direct, module_name, last

        # 2. Nested same-name: mod/{prefix}/{last}/{last}.json
        nested = MOD_ROOT / prefix / f"{last}.json"
        if nested.exists() and nested.is_file():
            return nested, prefix, last

    return None, "", ""

def _get_module_name_from_config(config_file: Path) -> str:

```

```

"""
Determine the module name from a config file.

module_name = the config file's parent directory relative to mod/.
If the file is directly in mod/ – its stem (filename without extension).
"""

parent = config_file.parent
if parent == MOD_ROOT:
    return config_file.stem
return str(parent.relative_to(MOD_ROOT)).replace("\\", "/")

# =====
# HOST → MODULE (via `domain` in module JSON)
# =====

#: Cache: {domain_lowercase: module_name}.
#: Refreshed automatically when the app/ tree mtime changes.
_host_module_cache: dict[str, str] = {}
_host_cache_mtime: float = 0.0

def _app_tree_mtime(root: Path) -> float:
    """
    Latest mtime across all *.json files under root (recursive).

    Used as a cheap "did anything change?" signal for the host-module
    cache. Missing files / permission errors are ignored.
    """
    latest = 0.0
    try:
        for f in root.rglob("*.json"):
            try:
                m = f.stat().st_mtime
                if m > latest:
                    latest = m
            except OSError:
                continue
    except Exception:
        pass
    return latest

def _scan_host_modules(root: Path) -> dict[str, str]:
    """
    Scan root/**/*.json and return {domain_lowercase: module_name}.

    A file is considered a module config when:
    - component == "module"
    - it has a non-empty string (or list of strings) `domain`

    Module name = the config file's stem (matches route.py convention).

    If several files declare the same domain, the first one found wins
    (rglob order is stable – alphabetical by directory).
    """

```

```

result: dict[str, str] = {}
if not root.exists() or not root.is_dir():
    return result

for cfg_file in root.rglob("*.json"):
    try:
        data = json.loads(cfg_file.read_text(encoding="utf-8"))
    except Exception:
        continue
    if data.get("component") != "module":
        continue

    raw_domain = data.get("domain")
    domains: list[str] = []
    if isinstance(raw_domain, str) and raw_domain.strip():
        domains = [raw_domain.strip().lower()]
    elif isinstance(raw_domain, list):
        domains = [
            d.strip().lower()
            for d in raw_domain
            if isinstance(d, str) and d.strip()
        ]

    if not domains:
        continue

    module_name = cfg_file.stem
    for dom in domains:
        if dom not in result:
            result[dom] = module_name

return result


def _resolve_module_by_host(host: str | None) -> str | None:
    """
    Resolve a module name from the Host header via `domain` in module JSON.

    - Strips the port.
    - Lowercases.
    - Also tries the `www.` variant (declares "example.com" → matches
      both "example.com" and "www.example.com", and vice versa).
    - Returns the module name (folder name) or None.

    Cache invalidates automatically when any JSON under MOD_ROOT
    changes its mtime.
    """
    global _host_module_cache, _host_cache_mtime

    if not host:
        return None

    host = host.strip().lower()
    if ":" in host:
        host = host.split(":", 1)[0]
    if not host:

```

```

        return None

# Skip IPs / localhost.
if host in ("localhost", "127.0.0.1", "::1"):
    return None
if host.replace(".", "").isdigit():
    return None

# Refresh cache if the app/ tree changed.
current_mtime = _app_tree_mtime(MOD_ROOT)
if current_mtime != _host_cache_mtime or not _host_module_cache:
    _host_module_cache = _scan_host_modules(MOD_ROOT)
    _host_cache_mtime = current_mtime

# Direct hit.
if host in _host_module_cache:
    return _host_module_cache[host]

# www. variants – try both directions.
if host.startswith("www."):
    alt = host[4:]
else:
    alt = "www." + host
if alt in _host_module_cache:
    return _host_module_cache[alt]

return None

def _resolve_links(obj, base_url: str):
    """
    Recursively walk the config and rewrite relative links in the
    'href' field to absolute paths prefixed with base_url.
    """
    if isinstance(obj, list):
        return [_resolve_links(item, base_url) for item in obj]
    if isinstance(obj, dict):
        result = {}
        for key, value in obj.items():
            if (
                key == 'href'
                and isinstance(value, str)
                and value.startswith('/')
                and not value.startswith('//')
            ):
                result[key] = base_url + value
            else:
                result[key] = _resolve_links(value, base_url)
        return result
    return obj

def _substitute_vars(obj, context: dict):
    """
    Recursively walk the config and replace {{ var }} with values from
    context. Supports nested keys: {{ page.title }}.

```

IMPORTANT: if a key is not in context – leave {{ var }} as-is, so frontend markers ({{ pages }}, {{ nav }}, {{ word }}) are not removed.

```
"""
if isinstance(obj, list):
    return [_substitute_vars(item, context) for item in obj]
if isinstance(obj, dict):
    return {k: _substitute_vars(v, context) for k, v in obj.items()}
if isinstance(obj, str):
    def replacer(match):
        key = match.group(1).strip()
        value = _resolve_key(key, context)
        if value is None:
            return match.group(0)
        return str(value)
    return re.sub(r"\{\s*([^\s]+)\s*\}", replacer, obj)
return obj
```

```
def _resolve_key(key: str, context: dict):
    """
    Resolve a dotted key like 'page.title' against context.
    Returns None when the key is missing.
    """
    parts = key.split(".")
    value = context
    for part in parts:
        if isinstance(value, dict) and part in value:
            value = value[part]
        else:
            return None
    return value
```

```
# =====
# API – returns the assembled JSON config
# =====
```

```
@router.get("/api/{module_path:path}")
async def engine_api(module_path: str, request: Request):
    """
    Return the assembled JSON config.

    URL without .json: /core/engine/api/app/page/20260914/153910
```

Algorithm:

1. Parse the URL: path_parts + params_list (digits).
2. Resolve Host → module when path_parts is empty.
3. Find the config file via _find_config.
4. Apply default_page, extend.
5. Substitute {{ ... }} from params_list.
6. Rewrite links for dev mode.

```
"""
path_parts, params_list = _parse_path(module_path)
```

```

# If the API path is empty, resolve the module from the Host header.
# This mirrors engine_module's behaviour: /core/engine/api/ with
# Host: dev.neurocad.ru → module "dev.neurocad.ru" (if declared).
if not path_parts:
    host_module = _resolve_module_by_host(request.headers.get("host"))
    module = host_module or "default"
    path_parts = [module, module]

config_file, module_name, page_name = _find_config(path_parts)

if not config_file:
    raise HTTPException(status_code=404, detail=f"Config for {module_path} not found")

with open(config_file, 'r', encoding='utf-8') as f:
    config = json.load(f)

# 1. Handle default_page
if config.get('default_page'):
    dp_name = config['default_page']
    dp_path = config_file.parent / f"{dp_name}.json"
    if dp_path.exists() and dp_path.is_file():
        with open(dp_path, 'r', encoding='utf-8') as f:
            dp_config = json.load(f)
            config = {**config, **dp_config}

# 2. Handle extend
if config.get('extend'):
    base_name = config['extend']
    base_path = config_file.parent / f"{base_name}.json"
    if base_path.exists() and base_path.is_file():
        with open(base_path, 'r', encoding='utf-8') as f:
            base_config = json.load(f)

    config_components = config.get('components', [])
    merged = {**base_config, **config}

    merged['component'] = base_config.get('component', 'base')

    merged_components = []
    if base_config.get('components'):
        merged_components.extend(base_config['components'])
    if config_components:
        merged_components.extend(config_components)
    merged['components'] = merged_components

    merged.pop('default_page', None)
    merged.pop('extend', None)
    config = merged

# 3. Substitute {{ ... }}
context = {
    "module_name": module_name,
    "page_name": page_name,
    "params_list": params_list,
}
if len(params_list) > 0:

```

```

        context["page_date"] = params_list[0]
    if len(params_list) > 1:
        context["page_time"] = params_list[1]
    if len(params_list) > 2:
        context["page_extra"] = params_list[2:]

    config = _substitute_vars(config, context)

    # 4. Drop service fields
    config.pop('default_page', None)
    config.pop('extend', None)

    # 5. Rewrite links for dev
    if DEBUG:
        base_url = f"/core/engine/{module_name}"
        config = _resolve_links(config, base_url)

    return JsonResponse(config)

# =====
# HTML – returns the module page
# =====

@router.get("/{module_path:path}", response_class=HTMLResponse)
async def engine_module(request: Request, module_path: str):
    """
    Return the HTML page for a module.

    Algorithm:
    1. Parse the URL: path_parts + params_list (digits).
    2. Resolve Host → module when path_parts is empty.
    3. Find the config file by path_parts.
    4. If not found → 404.
    5. Read the config (auth_required, auth_redirect).
    6. Compute nav_id:
        - if the URL looks like /core/engine/pages/<nav_id>/...
          - take it from the URL;
        - otherwise – resolve from the session (current user's first nav);
        - if nothing – None.
    7. Pass module_name, config_path, params_list, nav_id to the template.
    """
    path_parts, params_list = _parse_path(module_path)

    # If the URL path is empty (/core/engine/ hit directly) – resolve
    # the module from the Host header. Fallback: "default".
    if not path_parts:
        host_module = _resolve_module_by_host(request.headers.get("host"))
        module = host_module or "default"
        path_parts = [module, module]

    config_file, module_name, page_name = _find_config(path_parts)

    if not config_file:
        raise HTTPException(status_code=404, detail=f"Page {module_path} not found")

```



```

# config_path for the API = path of the file relative to MOD_ROOT, no .json
config_path = str(config_file.relative_to(MOD_ROOT)).replace(".json", "").replace("\\", "/")

auth_required = False
auth_redirect = None

try:
    with open(config_file, 'r', encoding='utf-8') as f:
        config = json.load(f)
        auth_required = config.get('auth_required', False)
        auth_redirect = config.get('auth_redirect', None)
except Exception as e:
    print(f"[Engine] Error loading config for {module_path}: {e}")

# ===== nav_id =====
# Priority:
# 1. URL form /core/engine/pages/<nav_id>/... - nav_id is params_list[0]
#    when path_parts[0] == "pages".
# 2. Session - first nav of the authenticated user (by id ASC).
#    This is what makes admin pages carry a nav_id, so the frontend
#    can build /core/engine/pages/<nav_id>/<date>/<time> links.
nav_id = None

if path_parts and path_parts[0] == "pages" and len(params_list) > 0:
    try:
        nav_id = int(params_list[0])
    except (TypeError, ValueError):
        nav_id = None

if nav_id is None:
    # Fall back to the current user's first nav.
    try:
        from sqlalchemy import select
        from neurocad.core.models.nav import Nav
        from neurocad.utils.sqlite import get_db_sqlite

        current_user = await _extract_current_user(request)
        user_id = _user_id_from(current_user)

        if user_id:
            async for session in get_db_sqlite():
                stmt = (
                    select(Nav)
                    .where(Nav.user_id == user_id, Nav.is_delete == False)
                    .order_by(Nav.id.asc())
                    .limit(1)
                )
                nav = (await session.execute(stmt)).scalar_one_or_none()
                if nav:
                    nav_id = nav.id
                break
    except Exception as e:
        print(f"[Engine] nav_id resolution failed: {e}")
        nav_id = None

return templates.TemplateResponse(

```

```

        request=request,
        name="core/engine/engine.html",
        context={
            "module_name": module_name,
            "config_path": config_path,
            "params_list": params_list,
            "nav_id": nav_id,
            "static_version": STATIC_VERSION,
            "is_authenticated": True,
            "username": "Гость",
            "auth_required": auth_required,
            "auth_redirect": auth_redirect,
        }
    )

```

```

# =====
# BLOCK – returns a block JSON config by $ref
# =====

```

```

@router.get("/block/{block_path:path}")
async def engine_block_config(block_path: str):
    """
    Return the JSON config of a block by $ref (used by the frontend
    to inline sub-configs into the page config).
    """
    full_path = MOD_ROOT / block_path
    if not full_path.exists() or not full_path.is_file():
        raise HTTPException(status_code=404, detail=f"Block {block_path} not found")

    with open(full_path, 'r', encoding='utf-8') as f:
        config = json.load(f)

    return JSONResponse(config)

```

```

=====
FILE: core/engine/__init__.py
=====

```

```

=====
FILE: core/engine/lib/word/constants.py
=====
# neurocad/core/engine/lib/word/constants.py

```

```

"""
Constants shared across the word module.

Kept in a dedicated module so that both `service.py` and the
editor I/O helpers can import them without creating a circular
import (media.py needs MEDIA_URL; service.py needs media.py).

```

```

Namespace: CoreEngineLibWordConstants
"""

```

```

#: Public URL prefix for media files. The matching filesystem

```

```

#: directory is `media/<nav_id>`.
MEDIA_URL = "/media"

#: Special filenames that keep their original name on upload
#: (no random suffix) – site root files.
SPECIAL_NAMES = {"favicon.ico", "robots.txt", "sitemap.xml"}

=====
FILE: core/engine/lib/word/css_builder.py
=====
# neurocad/core/engine/lib/word/css_builder.py

"""
Build the full CSS of a page at save time.

Source of truth:
- effects/registry.json – effects (id = class, file = CSS path)
- blocks/manifest.json – blocks (css = file, classes = root classes)

No CSS parsing. Files are taken WHOLE, filtered by class presence in
the page HTML.

Order (same as in the editor canvas and word.js):
    content.css → blocks/*.css → fx/*.css → custom page CSS
"""

import json
import re
from pathlib import Path
from typing import List, Set

# -----
# Paths
# -----

# css_builder.py = ../word/css_builder.py
# editor/         = ../word/editor/
_WORD_DIR = Path(__file__).resolve().parent
_EDITOR_DIR = _WORD_DIR / "editor"

CONTENT_CSS = _EDITOR_DIR / "css" / "content.css"

EFFECTS_DIR = _EDITOR_DIR / "effects"
EFFECTS_REGISTRY = EFFECTS_DIR / "registry.json"

BLOCKS_DIR = _EDITOR_DIR / "blocks"
BLOCKS_MANIFEST = BLOCKS_DIR / "manifest.json"

# -----
# Manifest loading (cached in memory)
# -----

_effects_cache = None
_blocks_cache = None

```

```

def _load_json(path: Path):
    with path.open("r", encoding="utf-8") as f:
        return json.load(f)

def _effects() -> List[dict]:
    global _effects_cache
    if _effects_cache is None:
        data = _load_json(EFFECTS_REGISTRY)
        _effects_cache = data.get("effects", [])
    return _effects_cache

def _blocks() -> List[dict]:
    global _blocks_cache
    if _blocks_cache is None:
        data = _load_json(BLOCKS_MANIFEST)
        _blocks_cache = data.get("blocks", [])
    return _blocks_cache

# -----
# Class extraction
# -----

_CLASS_RE = re.compile(r'class="([^\"]+)"', re.IGNORECASE)

def extract_classes(html: str) -> Set[str]:
    """
    Collect all class names used in the HTML.

    Not CSS parsing – just attribute extraction.
    """
    if not html:
        return set()
    classes: Set[str] = set()
    for chunk in _CLASS_RE.findall(html):
        for name in chunk.split():
            if name:
                classes.add(name)
    return classes

# -----
# File reading
# -----

def _read_text(path: Path) -> str:
    try:
        return path.read_text(encoding="utf-8")
    except OSError:
        return ""

```

```

# -----
# Build
# -----

def build_full_page_css(content: str, custom_css: str = "") -> str:
    """
    Assemble the full CSS for one page.

    content    – page HTML (used only to extract class names)
    custom_css – the page's own CSS from GrapesJS Style Manager

    Returns the combined CSS as a single string.
    """
    used = extract_classes(content or "")

    parts: List[str] = []

    # 1. content.css – always, in full (variables, base classes).
    parts.append(_read_text(CONTENT_CSS))

    # 2. Blocks – whole file if any of its root classes is used.
    #    Order: as in blocks/manifest.json.
    for b in _blocks():
        css_file = b.get("css")
        if not css_file:
            continue
        classes = b.get("classes") or []
        if any(c in used for c in classes):
            parts.append(_read_text(BLOCKS_DIR / css_file))

    # 3. Effects – one file per effect, by class match.
    #    Order: as in effects/registry.json.
    for e in _effects():
        effect_id = e.get("id")
        if not effect_id or effect_id not in used:
            continue
        css_file = e.get("file")
        if not css_file:
            continue
        parts.append(_read_text(EFFECTS_DIR / css_file))

    # 4. Custom page CSS – from GrapesJS.
    if custom_css and custom_css.strip():
        parts.append(custom_css)

    return "\n\n".join(p for p in parts if p and p.strip())

=====
FILE: core/engine/lib/word/route.py
=====
# neurocad/core/engine/lib/word/route.py

"""
Word API routes – page content, history, assets.

```

Endpoints:

GET	/bydatetime/{date}/{time}	– page by date/time
GET	/item/{item_id}	– page by ID
PUT	/page_id	– save page content
GET	/page_id/history	– list snapshots (metadata)
GET	/page_id/history/{hist_id}	– one full snapshot
DELETE	/page_id/history/{hist_id}	– delete one snapshot
DELETE	/page_id/history	– clear all snapshots
POST	/page_id/rollback/{hist_id}	– roll back to a snapshot
GET	/assets	– list media for a nav
POST	/assets/upload	– upload media for a nav

Scoping:

Every endpoint resolves a nav instance. If ?nav_id=<id> is given, it is used as-is; otherwise the backend falls back to the current user's first nav (by id ASC). Pages and media are scoped to Nav.id, not to a module.

Permissions:

Read endpoints (bydatetime, item, history, assets) – public or authenticated; write endpoints (save, delete, clear, rollback, upload) – any authenticated user. Guests cannot write.

Sub-routers:

- llm/* (presets, chat, history)
- editor/* (effects, and future editor sub-APIs)

Namespace: CoreEngineLibWord*

"""

```
from fastapi import APIRouter, Depends, HTTPException, Query, UploadFile, Request
from fastapi.responses import JSONResponse
from typing import List, Optional
```

```
from sqlalchemy import select
```

```
from neurocad.core.auth.dependencies import get_current_user
from neurocad.core.models.nav import Nav
from neurocad.utils.sqlite import get_db_sqlite
from .service import CoreEngineLibWordService
from .schema import (
    CoreEngineLibWordSaveRequest,
    CoreEngineLibWordSaveResponse,
    CoreEngineLibWordAssetsResponse,
    CoreEngineLibWordUploadResponse,
)
```

```
# LLM router (presets, chat, history)
from .llm.route import router as llm_router
```

```
# Editor router (effects, and future editor sub-APIs)
from .editor.route import router as editor_router
```

```
router = APIRouter(prefix="/word", tags=["core/engine/lib/word"])
```

```

# Include LLM router (all /llm/* endpoints)
router.include_router(llm_router)

# Include editor router (all /editor/* endpoints)
router.include_router(editor_router)

# =====
# NAV RESOLUTION
# =====

async def _resolve_nav_id(
    request: Request,
    explicit_nav_id: Optional[int],
    current_user: Optional[dict] = None,
) -> int:
    """
    Resolve the nav instance for the current request.

    Priority:
    1. Explicit nav_id (from query / path) – used as-is.
    2. First nav of the current user (ORDER BY id ASC, is_delete=0).

    Raises:
    401 if there is no authenticated user and no explicit nav_id.
    404 if the current user has no nav at all.
    """
    if explicit_nav_id is not None:
        return explicit_nav_id

    if current_user is None:
        current_user = await get_current_user(request)

    user_id = current_user.get("id") if isinstance(current_user, dict) else None
    if user_id is None:
        raise HTTPException(status_code=401, detail="Not authenticated")

    async for session in get_db_sqlite():
        stmt = (
            select(Nav)
            .where(Nav.user_id == user_id, Nav.is_delete == False)
            .order_by(Nav.id.asc())
            .limit(1)
        )
        nav = (await session.execute(stmt)).scalar_one_or_none()
        if not nav:
            raise HTTPException(
                status_code=404,
                detail="No nav found for the current user",
            )
        return nav.id

    raise HTTPException(status_code=500, detail="DB error")

# =====

```

```

# ONE PAGE BY DATETIME
# =====

@router.get("/bydatetime/{date}/{time}")
async def get_word_page_by_datetime(
    date: str,
    time: str,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
) -> JSONResponse:
    """
    Get page by date and time.

    Example:
        /core/engine/lib/word/bydatetime/20260914/153910?nav_id=2

    date = "20260914" (YYYYMMDD)
    time = "153910" (HHMMSS)

    Public endpoint (needed for page display).
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id)

    item = await CoreEngineLibWordService.get_by_datetime(
        date, time, nav_id=resolved_nav_id
    )

    if not item:
        raise HTTPException(status_code=404, detail="Page not found")

    return JSONResponse({
        "success": True,
        "data": item,
    })

# =====
# ONE PAGE BY ID
# =====

@router.get("/item/{item_id}")
async def get_word_page_by_id(
    item_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
) -> JSONResponse:
    """
    Get page by ID.

    Example:
        /core/engine/lib/word/item/2?nav_id=2

    Public endpoint.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id)

```



```

    item = await CoreEngineLibWordService.get_by_id(item_id, nav_id=resolved_nav_id)

    if not item:
        raise HTTPException(status_code=404, detail="Page not found")

    return JSONResponse({
        "success": True,
        "data": item,
    })

# =====
# SAVE CONTENT
# =====

@router.put("/{page_id}")
async def save_word_content(
    page_id: int,
    data: CoreEngineLibWordSaveRequest,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """
    Save page content (HTML + GrapesJS JSON + CSS).

    Before update, a snapshot of the current state is written
    to page_hist with action='user_edit' (see service.save_content).

    Any authenticated user.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    result = await CoreEngineLibWordService.save_content(
        page_id=page_id,
        nav_id=resolved_nav_id,
        content=data.content,
        content_json=data.content_json,
        css=data.css,
        user_note=current_user.get("username") or current_user.get("email"),
    )

    if not result:
        raise HTTPException(status_code=404, detail="Page not found")

    return JSONResponse({
        "success": True,
        "data": result,
    })

# =====
# HISTORY – LIST
# =====

@router.get("/{page_id}/history")

```

```

async def get_word_history(
    page_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """
    List all snapshots for a page, newest first.

    Does NOT include html / content_json / css (heavy) – only metadata.
    Use GET /{page_id}/history/{hist_id} for a full snapshot.

    Any authenticated user.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    items = await CoreEngineLibWordService.list_history(
        page_id, nav_id=resolved_nav_id
    )

    if items is None:
        raise HTTPException(status_code=404, detail="Page not found")

    return JSONResponse({
        "success": True,
        "data": items,
    })

# =====
# HISTORY – ONE ITEM (full snapshot)
# =====

@router.get("/{page_id}/history/{hist_id}")
async def get_word_history_item(
    page_id: int,
    hist_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """
    Get one full snapshot (html + content_json + css).

    Any authenticated user.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    item = await CoreEngineLibWordService.get_history_item(
        hist_id=hist_id,
        page_id=page_id,
        nav_id=resolved_nav_id,
    )

    if not item:
        raise HTTPException(status_code=404, detail="Snapshot not found")

```

```

    return JsonResponse({
        "success": True,
        "data": item,
    })

# =====
# HISTORY – DELETE ONE SNAPSHOT
# =====

@router.delete("/{page_id}/history/{hist_id}")
async def delete_word_history_item(
    page_id: int,
    hist_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JsonResponse:
    """
    Delete ONE snapshot from a page's history.

    The page itself is NOT touched – only the PageHist row is removed.

    Any authenticated user.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    ok = await CoreEngineLibWordService.delete_history_item(
        page_id=page_id,
        hist_id=hist_id,
        nav_id=resolved_nav_id,
    )

    if not ok:
        raise HTTPException(
            status_code=404,
            detail="Page or snapshot not found",
        )

    return JsonResponse({
        "success": True,
        "data": {"deleted": hist_id},
    })

# =====
# HISTORY – CLEAR ALL SNAPSHOTS
# =====

@router.delete("/{page_id}/history")
async def clear_word_history(
    page_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),

```

```

) -> JSONResponse:
    """
    Delete ALL snapshots of a page.

    The page itself is NOT touched – only the PageHist rows
    that reference it.

    Any authenticated user.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    ok = await CoreEngineLibWordService.clear_history(
        page_id=page_id,
        nav_id=resolved_nav_id,
    )

    if not ok:
        raise HTTPException(status_code=404, detail="Page not found")

    return JSONResponse({
        "success": True,
        "data": {"cleared": True},
    })

# =====
# HISTORY – ROLLBACK
# =====

@router.post("/{page_id}/rollback/{hist_id}")
async def rollback_word_content(
    page_id: int,
    hist_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """
    Roll the page back to the given snapshot.

    Before rollback, the current state is snapshotted into page_hist
    (action='user_edit'), then the snapshot is applied, and a new
    record with action='rollback' is written (audit trail).

    Any authenticated user.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    result = await CoreEngineLibWordService.rollback(
        page_id=page_id,
        nav_id=resolved_nav_id,
        hist_id=hist_id,
        user_note=current_user.get("username") or current_user.get("email"),
    )

    if not result:

```

```

        raise HTTPException(status_code=404, detail="Page or snapshot not found")

    return JSONResponse({
        "success": True,
        "data": result,
    })

# =====
# LIST ASSETS
# =====

@router.get("/assets")
async def list_word_assets(
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """
    Get list of all images from media/<nav_id>/.

    Used by GrapesJS Asset Manager. Any authenticated user.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    assets = await CoreEngineLibWordService.list_assets(resolved_nav_id)

    return JSONResponse({
        "success": True,
        "data": assets,
    })

# =====
# UPLOAD ASSETS
# =====

@router.post("/assets/upload")
async def upload_word_assets(
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
) -> JSONResponse:
    """
    Upload images to media/<nav_id>/.
    Used by GrapesJS Asset Manager.

    Notes:
    - Read form ONCE (Starlette does not allow re-reading).
    - Detect UploadFile by duck-typing (filename/read/content_type),
      not via isinstance – UploadFile classes differ across
      Starlette / FastAPI versions.
    - Accept any field name: 'files', 'files[]', 'file', 'upload'.
    - Authorization – manually, via get_current_user(request).
    """
    # ===== READ FORM ONCE =====
    form = await request.form()

```

```

# ===== LOGGING =====
print("=" * 60)
print("[UPLOAD] POST /assets/upload")
print("[UPLOAD] Content-Type:", request.headers.get("content-type"))
print("[UPLOAD] form keys:", list(form.keys()))

# ===== HELPER: detect UploadFile without isinstance =====
def is_upload_file(obj) -> bool:
    return (
        hasattr(obj, "filename")
        and hasattr(obj, "read")
        and hasattr(obj, "content_type")
        and callable(getattr(obj, "read", None))
    )

# ===== COLLECT FILES =====
all_files: List[UploadFile] = []
for key, value in form.multi_items():
    print(f"[UPLOAD] {key} → type={type(value).__name__}")
    if is_upload_file(value):
        all_files.append(value)
        print(f"[UPLOAD] ✓ UploadFile: name={value.filename}, "
              f"content_type={value.content_type}")
    else:
        print(f"[UPLOAD] ✗ not a file: {value!r}")

print("[UPLOAD] Total files:", len(all_files))
print("=" * 60)

# ===== AUTHORIZATION =====
current_user = await get_current_user(request)
if not current_user or not current_user.get("id"):
    raise HTTPException(status_code=401, detail="Not authenticated")

# ===== RESOLVE NAV =====
resolved_nav_id = await _resolve_nav_id(
    request, nav_id, current_user=current_user
)

# ===== SAVE =====
if not all_files:
    raise HTTPException(status_code=400, detail="No files provided")

try:
    urls = await CoreEngineLibWordService.upload_assets(all_files, resolved_nav_id)
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Upload error: {str(e)}")

return JSONResponse({
    "success": True,
    "data": urls,
})

```

=====

FILE: core/engine/lib/word/schema.py

```

=====
# app/core/engine/lib/word/schema.py

"""
Word schemas – page content, history, assets.

Pydantic models for the word API. Read and write shapes for pages,
history snapshots, and the media library. Pages are scoped to a nav
instance (nav_id) – that scoping is carried in the URL / query, not
in the request bodies, so it is not part of these schemas.

Namespace: CoreEngineLibWord*
"""

from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import datetime as dt

# =====
# RESPONSE – single page (for bydatetime and item)
# =====

class CoreEngineLibWordItemResponse(BaseModel):
    """One page for display / editing"""
    id: int
    nav_id: int
    datetime: dt
    title: str
    description: Optional[str] = None
    logo: Optional[str] = None
    content: Optional[str] = None
    content_json: Optional[str] = None
    css: Optional[str] = None
    is_active: int
    is_delete: int
    created_at: Optional[dt] = None
    updated_at: Optional[dt] = None
    rss_yandex_id: Optional[str] = None

    class Config:
        from_attributes = True

# =====
# RESPONSE – wrapper for a single page
# =====

class CoreEngineLibWordPageResponse(BaseModel):
    """Response wrapper for a single page"""
    success: bool = True
    data: CoreEngineLibWordItemResponse

# =====
# SAVE CONTENT

```

```

# =====

class CoreEngineLibWordSaveRequest(BaseModel):
    """
    Request body for saving page content from the editor.

    content      = HTML for display (without <style>).
    content_json = GrapesJS JSON (string).
    css          = page CSS (from editor.getCss()).
    """
    content: Optional[str] = Field(None, description="HTML for display")
    content_json: Optional[str] = Field(None, description="GrapesJS JSON")
    css: Optional[str] = Field(None, description="Page CSS")

class CoreEngineLibWordSaveResponse(BaseModel):
    """Response for saving content"""
    success: bool = True
    data: dict

# =====
# ASSETS (MEDIA LIBRARY)
# =====

class CoreEngineLibWordAsset(BaseModel):
    """Single asset (image)"""
    src: str = Field(..., description="Image URL")
    name: str = Field(..., description="File name")
    type: str = Field("image", description="Asset type")

class CoreEngineLibWordAssetsResponse(BaseModel):
    """Response with a list of assets"""
    success: bool = True
    data: List[CoreEngineLibWordAsset]

class CoreEngineLibWordUploadResponse(BaseModel):
    """Response for asset upload"""
    success: bool = True
    data: List[str] = Field(..., description="URLs of uploaded files")

# =====
# CHANGE HISTORY (page_hist)
# =====

class CoreEngineLibWordHistoryItem(BaseModel):
    """
    Metadata for a single history snapshot.

    Does NOT include html / content_json / css (heavy fields) –
    for a full snapshot use GET /{page_id}/history/{hist_id}.
    """
    id: int

```



```

    action: Optional[str] = Field(
        None,
        description="user_edit | ai_edit | preset_apply | rollback",
    )
    note: Optional[str] = None
    created_at: Optional[dt] = None

    class Config:
        from_attributes = True

class CoreEngineLibWordHistoryListResponse(BaseModel):
    """Response with a list of snapshots (metadata, no html/content_json/css)"""
    success: bool = True
    data: List[CoreEngineLibWordHistoryItem]

class CoreEngineLibWordHistoryItemResponse(BaseModel):
    """
    Full history snapshot – html + content_json + css.

    Used for preview and for rollback.
    """
    id: int
    page_id: int
    html: str
    content_json: Optional[str] = None
    css: Optional[str] = None
    action: Optional[str] = None
    note: Optional[str] = None
    created_at: Optional[dt] = None

    class Config:
        from_attributes = True

class CoreEngineLibWordHistoryItemFullResponse(BaseModel):
    """Response wrapper for a single full snapshot"""
    success: bool = True
    data: CoreEngineLibWordHistoryItemResponse

class CoreEngineLibWordRollbackResponse(BaseModel):
    """
    Response for a rollback to a snapshot.

    Returns updated content / content_json / css / updated_at –
    so the frontend can refresh its state without a second GET.
    """
    success: bool = True
    data: dict

=====
FILE: core/engine/lib/word/service.py
=====
# neurocad/core/engine/lib/word/service.py

```

"""

Word service – page content, history, and media.

Read/write access to Page and PageHist records, scoped to a nav instance (Page.nav_id). Media lives under media/<nav_id>/.

Main responsibilities:

- Load pages by date/time or ID.
- Save page content (HTML + GrapesJS JSON + CSS) and write a snapshot of the previous state to page_hist.
- List / fetch / roll back / delete snapshots.
- List and upload media for a nav.

CSS handling:

The full page CSS is rebuilt from scratch on every save (content.css + used blocks/*.css + used fx/*.css + custom CSS) and frozen into Page.css. The public page loads only that file (plus the wrapper public.css) – it never scans the editor directory at runtime.

Media handling (smart data-URI extraction):

On save, `content` and `content\_json` are passed through `editor.io.media.extract\_from\_html` / `extract\_from\_json`. Data-URIs larger than the per-URI threshold (50 KB) – or any URIs at all, if the page-wide sum exceeds 1 MB – are written to `media/<nav\_id>/<uuid>.<ext>` and replaced with public URLs. Small icons (< 50 KB) stay inline, so pasting a tiny image via the editor is still possible.

Constants:

`MEDIA\_URL` and `SPECIAL\_NAMES` live in `word/constants.py`. They are imported here – and re-exported, so existing callers that did `from ..service import MEDIA\_URL` keep working.

Namespace: CoreEngineLibWord*

"""

```
import os
import re
import uuid
from pathlib import Path
from datetime import datetime, timedelta
from typing import Optional, Dict, Any, List
from fastapi import UploadFile

from sqlalchemy import select
from sqlalchemy import delete as sa_delete

from ....models.base import Page
from ....models.page_hist import PageHist
from ....utils.sqlite import get_db_sqlite

# Public pages module – provides the static namespace for page CSS files.
# The URL prefix is what view.js uses for the <link> tag.
from ..pages.public.service import PAGES_CSS_DIR, PAGES_CSS_URL
```

```

from .....utils.css import ensure_css_file

# Full-CSS builder – assembles content.css + used blocks/*.css +
# used fx/*.css + custom page CSS, at every save.
from .css_builder import build_full_page_css

# Constants – MEDIA_URL and SPECIAL_NAMES live in a dedicated
# module so that editor/io/media.py can import MEDIA_URL without
# a circular import through this module.
from .constants import MEDIA_URL, SPECIAL_NAMES

# Smart data-URI extraction – moves large inline images from
# `content` / `content_json` into media/<nav_id>/.
#
# `editor` is a SUBPACKAGE of `word` (word/editor/), so the import
# is `editor...` – NOT `..editor...`. The latter would resolve to
# `lib/editor`, which does not exist.
from .editor.io.media import extract_from_html, extract_from_json

# =====
# RE-EXPORT
# =====
# MEDIA_URL and SPECIAL_NAMES are imported from `.constants`
# above. They stay available as `service.MEDIA_URL` and
# `service.SPECIAL_NAMES` – nothing to do here. Any module that
# previously imported them from `word.service` keeps working.

class CoreEngineLibWordService:
    """Service for page content and assets."""

    # =====
    # PAGE BY DATETIME
    # =====

    @staticmethod
    async def get_by_datetime(
        date: str,
        time: str,
        nav_id: int,
    ) -> Optional[Dict[str, Any]]:
        """
        Find page by date and time within a nav instance.

        date = "20260914" (YYYYMMDD)
        time = "153910" (HHMMSS)

        datetime in DB has microseconds (15:39:10.666406),
        so we search in range [dt_start, dt_start + 1 sec).
        """
        try:
            dt_start = datetime.strptime(f"{date}{time}", "%Y%m%d%H%M%S")
        except ValueError as e:
            print(f"[Word] Invalid date/time: {date} {time} – {e}")
            return None

```

```

dt_end = dt_start + timedelta(seconds=1)

async for session in get_db_sqlite():
    stmt = select(Page).where(
        Page.nav_id == nav_id,
        Page.datetime >= dt_start,
        Page.datetime < dt_end,
        Page.is_delete == 0,
    )
    result = await session.execute(stmt)
    page = result.scalar_one_or_none()

    if not page:
        return None

    return _page_to_dict(page)

return None

# =====
# PAGE BY ID
# =====

@staticmethod
async def get_by_id(
    page_id: int,
    nav_id: int,
) -> Optional[Dict[str, Any]]:
    """Find page by ID within a nav instance."""
    async for session in get_db_sqlite():
        stmt = select(Page).where(
            Page.id == page_id,
            Page.nav_id == nav_id,
            Page.is_delete == 0,
        )
        result = await session.execute(stmt)
        page = result.scalar_one_or_none()

        if not page:
            return None

        return _page_to_dict(page)

    return None

# =====
# SAVE CONTENT
# =====

@staticmethod
async def save_content(
    page_id: int,
    nav_id: int,
    content: Optional[str],
    content_json: Optional[str],

```

```

css: Optional[str] = None,
user_note: Optional[str] = None,
) -> Optional[Dict[str, Any]]:
    """

```

Save page content and write a snapshot to page_hist.

Flow:

0. Smart media extraction – move large data-URIs from
`content` and `content\_json` into media/<nav_id>/.
(see editor.io.media – threshold policy: 50 KB per URI,
1 MB per page; small icons stay inline)
1. Find page.
2. Rebuild the full page CSS from scratch:
content.css + used blocks/*.css + used fx/*.css + custom.
(see css_builder.build_full_page_css)
3. If html / content_json / css actually changed – write a
snapshot of the CURRENT (pre-save) state to page_hist
with action='user_edit'. Duplicates are skipped.
4. Update Page.content / Page.content_json / Page.css.
5. Commit.
6. Write the page's CSS to a static file under
PAGES_CSS_DIR / PAGES_CSS_URL so the public page can
<link> to it (see word/view.js → buildArticle).

user_note – optional comment for the snapshot (e.g. username).

Returns updated data or None if page not found.

"""

```

# ===== 0. Smart media extraction =====
# The caller (word/route.py) already resolved nav_id for the
# current user. It is the SAME nav_id that owns the page and
# the media folder.
if content:
    content, _map_html = extract_from_html(content, nav_id, force=False)
if content_json:
    content_json, _map_json = extract_from_json(content_json, nav_id, force=False)

async for session in get_db_sqlite():
    stmt = select(Page).where(
        Page.id == page_id,
        Page.nav_id == nav_id,
        Page.is_delete == 0,
    )
    result = await session.execute(stmt)
    page = result.scalar_one_or_none()

    if not page:
        return None

    # ===== Snapshot BEFORE update =====
    old_html = page.content or ""
    old_json = page.content_json
    old_css = page.css or ""

    # Determine new values (fallback to old if None)

```

```

new_html = content if content is not None else old_html
new_json = content_json if content_json is not None else old_json

# Custom CSS from the editor (Style Manager). May be None
# for callers that only save HTML.
custom_css = css if css is not None else ""

# Rebuild the full page CSS from scratch on EVERY save.
# content.css + used blocks/*.css + used fx/*.css + custom.
# This freezes the CSS at save time: the result is written
# to page.css and no longer depends on the editor runtime.
new_css = build_full_page_css(new_html, custom_css)

# Only snapshot if something actually changed
changed = (
    new_html != old_html
    or new_json != old_json
    or new_css != old_css
)

if changed and (old_html or old_css):
    # Check for duplicates – skip if last snapshot is identical
    is_dup = await _is_duplicate_snapshot(
        session, page_id, old_html, old_json, old_css
    )
    if not is_dup:
        snapshot = PageHist(
            page_id=page_id,
            html=old_html,
            content_json=old_json,
            css=old_css,
            action="user_edit",
            note=user_note,
        )
        session.add(snapshot)

# ===== Update Page =====
if content is not None:
    page.content = content
if content_json is not None:
    page.content_json = content_json

# Always write the rebuilt CSS – even when custom_css is
# empty, content.css + blocks + effects still need to be
# frozen into page.css.
page.css = new_css

page.updated_at = datetime.now()
await session.commit()
await session.refresh(page)

# ===== Write static CSS file for the public page =====
# Non-fatal: if the write fails, the DB save is still valid.
_write_page_css_file(page.id, page.css)

return {

```

```

        "id": page.id,
        "title": page.title,
        "updated_at": page.updated_at.isoformat() if page.updated_at else None,
    }

    return None

# =====
# HISTORY – LIST
# =====

@staticmethod
async def list_history(
    page_id: int,
    nav_id: int,
) -> Optional[List[Dict[str, Any]]]:
    """
    List all snapshots for a page, newest first.

    Does NOT return html / content_json / css (heavy) – only metadata.
    For full snapshot – use get_history_item().

    Returns None if page not found, [] if no snapshots.
    """
    async for session in get_db_sqlite():
        # Verify page belongs to this nav
        page_stmt = select(Page).where(
            Page.id == page_id,
            Page.nav_id == nav_id,
            Page.is_delete == 0,
        )
        page_res = await session.execute(page_stmt)
        page = page_res.scalar_one_or_none()
        if not page:
            return None

        stmt = (
            select(PageHist)
            .where(PageHist.page_id == page_id)
            .order_by(PageHist.created_at.desc())
        )
        result = await session.execute(stmt)
        snapshots = result.scalars().all()

        return [
            {
                "id": s.id,
                "action": s.action,
                "note": s.note,
                "created_at": s.created_at.isoformat() if s.created_at else None,
            }
            for s in snapshots
        ]

    return None

```

```

# =====
# HISTORY – ONE ITEM
# =====

@staticmethod
async def get_history_item(
    hist_id: int,
    page_id: int,
    nav_id: int,
) -> Optional[Dict[str, Any]]:
    """
    Get one full snapshot (html + content_json + css).

    Returns None if the snapshot does not exist or the page
    does not belong to this nav.
    """
    async for session in get_db_sqlite():
        page_stmt = select(Page).where(
            Page.id == page_id,
            Page.nav_id == nav_id,
            Page.is_delete == 0,
        )
        page_res = await session.execute(page_stmt)
        page = page_res.scalar_one_or_none()
        if not page:
            return None

        stmt = select(PageHist).where(
            PageHist.id == hist_id,
            PageHist.page_id == page_id,
        )
        result = await session.execute(stmt)
        s = result.scalar_one_or_none()
        if not s:
            return None

        return {
            "id": s.id,
            "page_id": s.page_id,
            "html": s.html,
            "content_json": s.content_json,
            "css": s.css,
            "action": s.action,
            "note": s.note,
            "created_at": s.created_at.isoformat() if s.created_at else None,
        }

    return None

# =====
# HISTORY – ROLLBACK
# =====

@staticmethod
async def rollback(
    page_id: int,

```



```

    nav_id: int,
    hist_id: int,
    user_note: Optional[str] = None,
) -> Optional[Dict[str, Any]]:
    """

```

Roll the page back to the given snapshot.

Flow:

1. Find page and snapshot.
2. Snapshot the CURRENT (pre-rollback) state into page_hist with action='user_edit' – so the rollback itself is undoable.
3. Set Page.content / Page.content_json / Page.css to the snapshot values.
4. Write a new record with action='rollback' – audit trail.
5. Commit.
6. Overwrite the static CSS file so the public page picks up the rolled-back CSS (see word/view.js → buildArticle).

NOTE: we do NOT rebuild the CSS here. The snapshot already holds the frozen CSS that was in effect when it was taken – restoring it as-is is exactly what "rollback" should do.

Returns updated data or None if page / snapshot not found.

```

"""
async for session in get_db_sqlite():
    # ----- Find page -----
    page_stmt = select(Page).where(
        Page.id == page_id,
        Page.nav_id == nav_id,
        Page.is_delete == 0,
    )
    page_res = await session.execute(page_stmt)
    page = page_res.scalar_one_or_none()
    if not page:
        return None

    # ----- Find snapshot -----
    hist_stmt = select(PageHist).where(
        PageHist.id == hist_id,
        PageHist.page_id == page_id,
    )
    hist_res = await session.execute(hist_stmt)
    snapshot = hist_res.scalar_one_or_none()
    if not snapshot:
        return None

    # ----- Snapshot current state (before rollback) -----
    current_html = page.content or ""
    current_json = page.content_json
    current_css = page.css or ""

    if current_html or current_css:
        is_dup = await _is_duplicate_snapshot(
            session, page_id, current_html, current_json, current_css
        )
        if not is_dup:

```

```

        session.add(PageHist(
            page_id=page_id,
            html=current_html,
            content_json=current_json,
            css=current_css,
            action="user_edit",
            note=user_note,
        ))

# ----- Apply snapshot -----
page.content = snapshot.html
page.content_json = snapshot.content_json
page.css = snapshot.css
page.updated_at = datetime.now()

# ----- Audit trail: new record with action='rollback' -----
session.add(PageHist(
    page_id=page_id,
    html=snapshot.html,
    content_json=snapshot.content_json,
    css=snapshot.css,
    action="rollback",
    note=f"rollback to snapshot id={hist_id}",
))

await session.commit()
await session.refresh(page)

# ===== Overwrite static CSS file with the rolled-back CSS =====
_write_page_css_file(page.id, page.css)

return {
    "id": page.id,
    "title": page.title,
    "content": page.content,
    "content_json": page.content_json,
    "css": page.css,
    "updated_at": page.updated_at.isoformat() if page.updated_at else None,
}

return None

# =====
# HISTORY – DELETE ONE SNAPSHOT
# =====

@staticmethod
async def delete_history_item(
    page_id: int,
    hist_id: int,
    nav_id: int,
) -> bool:
    """
    Delete ONE snapshot from a page's history.

    The page itself is NOT touched – only the PageHist row.

```

Verification:

- the page must exist and belong to the current nav;
- the snapshot must exist and belong to that page.

Returns True on success, False otherwise.

```
"""
async for session in get_db_sqlite():
    # Verify page belongs to this nav
    page_stmt = select(Page).where(
        Page.id == page_id,
        Page.nav_id == nav_id,
        Page.is_delete == 0,
    )
    page_res = await session.execute(page_stmt)
    page = page_res.scalar_one_or_none()
    if not page:
        return False

    # Find the snapshot
    hist_stmt = select(PageHist).where(
        PageHist.id == hist_id,
        PageHist.page_id == page_id,
    )
    hist_res = await session.execute(hist_stmt)
    snapshot = hist_res.scalar_one_or_none()
    if not snapshot:
        return False

    await session.delete(snapshot)
    await session.commit()
    return True

return False
```

```
# =====
# HISTORY – CLEAR ALL SNAPSHOTS
# =====
```

@staticmethod

```
async def clear_history(
    page_id: int,
    nav_id: int,
) -> bool:
    """
    Delete ALL snapshots of a page.
```

The page itself is NOT touched – only the PageHist rows that reference it.

Verification:

- the page must exist and belong to the current nav.

Returns True on success, False otherwise.

```
"""
async for session in get_db_sqlite():
```

```

    # Verify page belongs to this nav
    page_stmt = select(Page).where(
        Page.id == page_id,
        Page.nav_id == nav_id,
        Page.is_delete == 0,
    )
    page_res = await session.execute(page_stmt)
    page = page_res.scalar_one_or_none()
    if not page:
        return False

    await session.execute(
        sa_delete(PageHist).where(PageHist.page_id == page_id)
    )
    await session.commit()
    return True

return False

# =====
# LIST ASSETS – media/<nav_id>/
# =====

@staticmethod
async def list_assets(nav_id: int) -> List[Dict[str, str]]:
    """
    Get list of all images from media/<nav_id>/.

    Each nav instance owns its own media folder – assets are
    isolated per nav, not per module. This matches the data
    model: a nav is the object that owns pages, and media
    belongs to the object too.

    Returns list of dicts: { src, name, type }.
    """
    media_dir = Path("media") / str(nav_id)
    media_dir.mkdir(parents=True, exist_ok=True)

    assets: List[Dict[str, str]] = []

    for filepath in sorted(media_dir.rglob('*')):
        if not filepath.is_file():
            continue

        ext = filepath.suffix.lower()
        if ext not in ['.jpg', '.jpeg', '.png', '.gif', '.svg', '.webp']:
            continue

        rel = filepath.relative_to(media_dir).as_posix()

        assets.append({
            "src": f"{MEDIA_URL}/{nav_id}/{rel}",
            "name": filepath.name,
            "type": "image",
        })

```

```

    return assets

# =====
# UPLOAD ASSETS – media/<nav_id>/
# =====

@staticmethod
async def upload_assets(
    files: List[UploadFile],
    nav_id: int,
) -> List[str]:
    """
    Save uploaded files to media/<nav_id>/.

    Each nav instance owns its own media folder – assets are
    isolated per nav, not per module.

    Special names (favicon.ico, robots.txt, sitemap.xml) – kept as-is.
    Other files – get a random 8-hex suffix.

    Returns list of URLs of uploaded files.
    """
    media_dir = Path("media") / str(nav_id)
    media_dir.mkdir(parents=True, exist_ok=True)

    uploaded_urls: List[str] = []

    for uploaded_file in files:
        original_name = uploaded_file.filename or "file"
        clean_name = re.sub(r'^a-zA-Z0-9_.\-|', '_', original_name)

        if clean_name in SPECIAL_NAMES:
            final_name = clean_name
        else:
            name, ext = os.path.splitext(clean_name)
            suffix = uuid.uuid4().hex[:8]
            final_name = f"{name}_{suffix}{ext}"

        save_path = media_dir / final_name

        with open(save_path, "wb") as buffer:
            while True:
                chunk = await uploaded_file.read(1024 * 64)
                if not chunk:
                    break
                buffer.write(chunk)

        await uploaded_file.close()

        uploaded_urls.append(f"{MEDIA_URL}/{nav_id}/{final_name}")

    return uploaded_urls

# =====
# HELPERS

```

```

# =====

def _write_page_css_file(page_id: int, css: Optional[str]) -> None:
    """
    Write the page's CSS to a static file under PAGES_CSS_DIR and
    return the URL – but we do not return it here; the public page
    builds the URL itself (see word/view.js → buildArticle).

    Non-fatal: if the write fails (permissions, disk full), the DB
    save is still valid – the public page will just miss the CSS
    until the next save.

    Empty CSS → no file written. Existing file is left on disk; it
    will simply not be requested by the public page (view.js falls
    back to the embedded <style> path).
    """
    if not css or not css.strip():
        return
    try:
        url = ensure_css_file(
            page_id=page_id,
            css=css,
            directory=PAGES_CSS_DIR,
            url_prefix=PAGES_CSS_URL,
        )
        if url:
            print(f"[Word] page CSS file written: {url}", flush=True)
    except Exception as e:
        print(f"[Word] failed to write page CSS file for page {page_id}: {e}", flush=True)

def _page_to_dict(page) -> Dict[str, Any]:
    """Serialize Page model to dict."""
    return {
        "id": page.id,
        "nav_id": page.nav_id,
        "datetime": page.datetime.isoformat() if page.datetime else None,
        "title": page.title,
        "description": page.description,
        "logo": page.logo,
        "content": page.content,
        "content_json": page.content_json,
        "css": page.css,
        "is_active": page.is_active,
        "is_delete": page.is_delete,
        "created_at": page.created_at.isoformat() if page.created_at else None,
        "updated_at": page.updated_at.isoformat() if page.updated_at else None,
        "rss_yandex_id": page.rss_yandex_id,
        "is_template": int(page.is_template) if page.is_template is not None else 0,
        "template_id": page.template_id,
    }

async def _is_duplicate_snapshot(
    session,
    page_id: int,

```

```

    html: str,
    content_json: Optional[str],
    css: Optional[str],
) -> bool:
    """
    Return True if the LAST snapshot for this page has identical
    html AND content_json AND css – to skip writing duplicates.
    """

    stmt = (
        select(PageHist)
        .where(PageHist.page_id == page_id)
        .order_by(PageHist.created_at.desc())
        .limit(1)
    )
    result = await session.execute(stmt)
    last = result.scalar_one_or_none()
    if not last:
        return False

    return (
        last.html == html
        and last.content_json == content_json
        and (last.css or "") == (css or "")
    )

```

```

=====
FILE: core/engine/lib/word/__init__.py
=====

```

```

=====
FILE: core/engine/lib/word/llm/dumper.py
=====

```

```

# neurocad/core/engine/lib/word/llm/dumper.py

```

```

"""

```

```

Dump every prompt sent to the LLM and the raw response back to disk.

```

```

Namespace: CoreEngineLibWordLlmDumper

```

```

Layout:

```

```

    <cwd>/log/llm-prompts/run-<id>/
    meta.txt
    NN-<agent>-system.txt
    NN-<agent>-user.txt
    NN-<agent>-user-chunk-N.txt
    NN-<agent>-history.txt
    NN-<agent>-response.txt
    NN-<agent>-response-chunk-N.txt

```

```

`agent` is the agent name: router, help, create, fill, effect, none.
A running counter `NN` keeps the files ordered.

```

```

meta.txt also records the provider and model that handled the run,
so a dumped run is self-describing (useful when switching providers
from the admin UI).

```

```

If `run_id` is None, dumping is skipped.
"""

import datetime
from pathlib import Path
from typing import Any, Dict, List, Optional

class CoreEngineLibWordLlmDumper:
    """Dump prompts and responses to disk, one run per folder."""

    #: Root directory, relative to cwd.
    ROOT_DIR = Path("log") / "llm-prompts"

    #: Per-run counter for file ordering: run_id -> next index.
    _counters: Dict[Any, int] = {}

    # =====
    # LOW-LEVEL
    # =====

    @staticmethod
    def _run_dir(run_id: Any) -> Path:
        return CoreEngineLibWordLlmDumper.ROOT_DIR / f"run-{run_id}"

    @staticmethod
    def _ensure_run_dir(run_id: Any) -> Path:
        d = CoreEngineLibWordLlmDumper._run_dir(run_id)
        d.mkdir(parents=True, exist_ok=True)
        return d

    @staticmethod
    def _next_index(run_id: Any) -> int:
        idx = CoreEngineLibWordLlmDumper._counters.get(run_id, 0) + 1
        CoreEngineLibWordLlmDumper._counters[run_id] = idx
        return idx

    @staticmethod
    def _write(run_id: Any, filename: str, content: str) -> None:
        if run_id is None:
            return
        try:
            d = CoreEngineLibWordLlmDumper._ensure_run_dir(run_id)
            path = d / filename
            with open(path, "w", encoding="utf-8") as f:
                f.write(content or "")
        except Exception as e:
            print(
                f"[dumper] failed to write {filename} for run={run_id}: {e!r}",
                flush=True,
            )

    # =====
    # META
    # =====

```



```

@staticmethod
def dump_meta(
    run_id: Any,
    page_id: int,
    user_message: str,
    block_catalog: Optional[List[Dict[str, Any]]] = None,
    provider_name: Optional[str] = None,
    model: Optional[str] = None,
) -> None:
    """
    Dump a short summary of the run.

    provider_name / model – the LLM that handled this run.
    Both are optional: if the provider could not be resolved
    (or is the mock), they are written as '-'.
    """
    if run_id is None:
        return
    # reset counter for a new run
    CoreEngineLibWordLlmDumper._counters[run_id] = 0

    catalog_size = len(block_catalog or [])
    now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    provider_str = provider_name or "-"
    model_str = model or "-"

    content = (
        f"run_id      = {run_id}\n"
        f"page_id       = {page_id}\n"
        f"user_message = {user_message!r}\n"
        f"catalog_size  = {catalog_size}\n"
        f"created_at    = {now}\n"
        f"provider      = {provider_str}\n"
        f"model         = {model_str}\n"
    )
    CoreEngineLibWordLlmDumper._write(run_id, "meta.txt", content)

# =====
# REQUEST (prompt)
# =====

@staticmethod
def dump_step(
    run_id: Any,
    agent_name: str,
    system_prompt: str,
    user_content: str,
    history: Optional[List[Dict[str, str]]] = None,
    chunk_index: Optional[int] = None,
) -> None:
    """
    Dump one LLM request.

    agent_name: 'router' | 'help' | 'create' | 'fill' | 'effect' | ...

```

```

chunk_index: None for single-request agents, 1..N for chunked fill.
"""
if run_id is None:
    return

idx = CoreEngineLibWordLlmDumper._next_index(run_id)
prefix = f"{idx:02d}-{agent_name}"

# System prompt
CoreEngineLibWordLlmDumper._write(
    run_id, f"{prefix}-system.txt", system_prompt
)

# User content
if chunk_index is not None:
    CoreEngineLibWordLlmDumper._write(
        run_id, f"{prefix}-user-chunk-{chunk_index}.txt", user_content
    )
else:
    CoreEngineLibWordLlmDumper._write(
        run_id, f"{prefix}-user.txt", user_content
    )

# History – only once per agent call
if history and (chunk_index is None or chunk_index == 1):
    lines = []
    for m in history:
        role = m.get("role", "?")
        content = m.get("content", "")
        lines.append(f"=== role={role} ===")
        lines.append(content)
        lines.append("")
    CoreEngineLibWordLlmDumper._write(
        run_id, f"{prefix}-history.txt", "\n".join(lines).rstrip()
    )

# =====
# RESPONSE
# =====

@staticmethod
def dump_response(
    run_id: Any,
    agent_name: str,
    raw_response: str,
    chunk_index: Optional[int] = None,
) -> None:
    """
    Dump the raw LLM response for one request.

    agent_name: same as in dump_step.
    chunk_index: matches dump_step.
    """
    if run_id is None:
        return

```

```

# Reuse the counter established by dump_step so request and
# response share the same NN.
current = CoreEngineLibWordLlmDumper._counters.get(run_id, 0)
prefix = f"{current:02d}-{agent_name}"

if chunk_index is not None:
    filename = f"{prefix}-response-chunk-{chunk_index}.txt"
else:
    filename = f"{prefix}-response.txt"

CoreEngineLibWordLlmDumper._write(run_id, filename, raw_response or "")

=====
FILE: core/engine/lib/word/llm/mock.py
=====
# neurocad/core/engine/lib/word/llm/mock.py

"""
Mock LLM provider.

Namespace: CoreEngineLibWordLlmMock

Same interface as the real providers (utils/llm/*.py):
    async generate_completion(messages) -> str

Returns canned responses for the three steps (plan / fill / effects)
so the whole WS flow can be exercised without burning tokens.
"""

import asyncio
import json

class CoreEngineLibWordLlmMock:
    """Mock LLM provider. No network, no tokens."""

    name = "mock"

    def __init__(self, delay=0.5):
        self.delay = delay
        self.calls = []

    async def generate_completion(self, messages):
        self.calls.append(messages)
        await asyncio.sleep(self.delay)

        system = messages[0]["content"] if messages else ""

        if "архитектор" in system:
            return json.dumps({"plan": [
                {"block_id": "core-hero", "purpose": "Баннер"},
                {"block_id": "core-features", "purpose": "Преимущества"},
                {"block_id": "core-footer", "purpose": "Подвал"},
            ]}, ensure_ascii=False)

        if "редактор контента" in system:

```

```

        ids = [l[4:-4].strip() for l in messages[-1]["content"].split("\n")
                 if l.startswith("=== ") and l.endswith(" ===")]
    return json.dumps({"filled": [
        {"block_id": bid,
         "html": f'<section data-b="{bid}"><h2>{bid}</h2><p>текст</p></section>'}
        for bid in ids
    ], "remaining": []}, ensure_ascii=False)

    if "дизайнер" in system:
        user = messages[-1]["content"]
        i = user.find("HTML:\n")
        html = user[i + 6:] if i != -1 else user
        return json.dumps({"html": html + "<style>.effect-mock{opacity:1}</style>"},
                           ensure_ascii=False)

    return json.dumps({"error": "unknown prompt"})

```

```

=====
FILE: core/engine/lib/word/llm/parser.py
=====

```

```

# neurocad/core/engine/lib/word/llm/parser.py

```

```

"""

```

```

Parse LLM responses into structured data.

```

```

Namespace: CoreEngineLibWordLlmParser

```

```

The LLM may return:

```

- plain JSON,
- JSON wrapped in markdown (``json ... ``),
- JSON with some text before/after,
- plain HTML (if it ignores the JSON instruction).

```

The parser is tolerant: it tries several strategies and falls back to
a safe default instead of raising. This keeps a single bad response
from breaking the whole editor flow.
"""

```

```

import json
from typing import Dict

```

```

class CoreEngineLibWordLlmParser:

```

```

    """Tolerant LLM response parser."""

```

```

    # =====
    # LOW-LEVEL HELPERS
    # =====

```

```

    @staticmethod

```

```

    def clean_html_response(text: str) -> str:

```

```

        """

```

```

        Remove markdown wrappers (``html ... `` or `` ... ``)
        if the LLM added them around HTML.
        """

```

```

        if not text:

```

```

        return text

s = text.strip()

if s.startswith('```'):
    # Drop the first line (``` or ```html)
    first_nl = s.find('\n')
    if first_nl != -1:
        s = s[first_nl + 1:]
    # Drop trailing ```
    if s.endswith('```'):
        s = s[:-3]

return s.strip()

@staticmethod
def _strip_markdown_wrapper(text: str) -> str:
    """
    Remove a leading ```json / ``` and a trailing ``` from the text.
    Returns the inner content (may still not be valid JSON).
    """
    s = text.strip()
    if s.startswith('```'):
        first_nl = s.find('\n')
        if first_nl != -1:
            s = s[first_nl + 1:]
        if s.endswith('```'):
            s = s[:-3]
    s = s.strip()
    return s

# =====
# PUBLIC API
# =====

@staticmethod
def extract_fields(data: dict) -> Dict[str, str]:
    """
    Extract message / html from a parsed JSON dict.

    Always returns both keys, even if the model omitted one.
    Missing `message` -> "Done."
    Missing `html` -> "".
    """
    message = str(data.get("message", "")).strip() or "Done."
    html = str(data.get("html", "")).strip()
    return {"message": message, "html": html}

@staticmethod
def parse_llm_response(text: str) -> Dict[str, str]:
    """
    Parse an LLM response into {message, html}.

    Expected JSON shape:
    {"message": "...", "html": "..."}

```

Tries, in order:

1. Strip markdown wrapper (``json ... ``).
2. Find the first {...} block inside the text and parse it.
3. Try to parse the whole string as JSON.
4. Fall back: treat the whole text as HTML.

Never raises. On any failure, returns a safe default.

"""

if not text:

 return {"message": "Empty LLM response.", "html": ""}

s = CoreEngineLibWordLlmParser._strip_markdown_wrapper(text)

1. Try to find a JSON object inside the text

start = s.find('{')

end = s.rfind('}')

if start != -1 and end != -1 and end > start:

 candidate = s[start:end + 1]

 try:

 data = json.loads(candidate)

 if isinstance(data, dict):

 return CoreEngineLibWordLlmParser.extract_fields(data)

 except json.JSONDecodeError:

 pass

2. Try to parse the whole string as JSON

try:

 data = json.loads(s)

 if isinstance(data, dict):

 return CoreEngineLibWordLlmParser.extract_fields(data)

except json.JSONDecodeError:

 pass

3. Fallback: not JSON – treat the cleaned text as HTML

return {

 "message": "Done. Changes applied.",

 "html": CoreEngineLibWordLlmParser.clean_html_response(s),

}

=====

FILE: core/engine/lib/word/llm/route.py

=====

neurocad/core/engine/lib/word/llm/route.py

"""

LLM editor API: presets + chat history + chat stream.

Namespace: CoreEngineLibWordLlmRoute

HTTP endpoints:

GET	/core/engine/lib/word/llm/presets	– list presets
POST	/core/engine/lib/word/llm/presets	– create preset
GET	/core/engine/lib/word/llm/presets/{id}	– get one preset
PUT	/core/engine/lib/word/llm/presets/{id}	– update preset
DELETE	/core/engine/lib/word/llm/presets/{id}	– soft delete
POST	/core/engine/lib/word/llm/presets/{id}/restore	– restore

```

POST    /core/engine/lib/word/llm/presets/{id}/thumbnail    - upload thumbnail
DELETE  /core/engine/lib/word/llm/presets/{id}/thumbnail    - delete thumbnail

GET     /core/engine/lib/word/llm/chat/{page_id}/history    - chat history
DELETE  /core/engine/lib/word/llm/chat/{page_id}/history    - clear history
GET     /core/engine/lib/word/llm/chat/{page_id}/run/active - active run

```

WebSocket:

```

WS      /core/engine/lib/word/llm/ws/{page_id}              - chat stream

```

Note: sending a chat message is done via the WebSocket only.

The old HTTP endpoint `POST /chat/{page_id}` was removed – it did the whole plan → fill → effects flow, which is now handled by the agent dispatcher in `ws.py`.

Permissions:

- Read endpoints (GET list, GET one, GET history, GET active run) are public.
- Write endpoints (create / update / delete / restore preset, upload / delete thumbnail, clear chat history) require any authenticated user.
- The WebSocket endpoint checks authentication inside its handler.

WebSocket handler binding

```

-----
`llm_ws_endpoint` is a regular async method on EndpointMixin (see
llm/websocket/), not a @staticmethod. The module `llm/ws.py`
instantiates CoreEngineLibWordLlmWS once and exports the bound
method:

```

```

_ws_instance = CoreEngineLibWordLlmWS()
llm_ws_endpoint = _ws_instance.llm_ws_endpoint

```

We import that bound method here. Do NOT pass

``CoreEngineLibWordLlmWS.llm_ws_endpoint`` – that would be an unbound function with a stray ``self`` parameter, and FastAPI would silently refuse to register the route (WebSocket would then fail with 1006, because the path is not mounted).

```

"""

```

```

from fastapi import APIRouter, Depends, HTTPException, UploadFile, File, Query
from fastapi.responses import JSONResponse

```

```

from neurocad.core.auth.dependencies import get_current_user

```

```

from .service import CoreEngineLibWordLlmService
from .runs import CoreEngineLibWordLlmRuns
from .schema import (
    CoreEngineLibWordLlmPresetCreate,
    CoreEngineLibWordLlmPresetUpdate,
)
from .ws import llm_ws_endpoint

```

```

router = APIRouter(prefix="/llm", tags=["core/engine/lib/word/llm"])

```

```

# =====
# PRESETS – LIST
# =====

@router.get("/presets")
async def list_presets(
    include_deleted: bool = Query(False, description="Include deleted"),
) -> JSONResponse:
    """List presets. Public endpoint."""
    result = await CoreEngineLibWordLlmService.list_presets(include_deleted=include_deleted)

    return JSONResponse({
        "success": True,
        "data": result["items"],
        "total": result["total"],
    })

# =====
# PRESETS – GET ONE
# =====

@router.get("/presets/{preset_id}")
async def get_preset(preset_id: int) -> JSONResponse:
    """Get one preset by ID. Public endpoint."""
    item = await CoreEngineLibWordLlmService.get_preset(preset_id)

    if not item:
        raise HTTPException(status_code=404, detail="Preset not found")

    return JSONResponse({
        "success": True,
        "data": item,
    })

# =====
# PRESETS – CREATE
# =====

@router.post("/presets")
async def create_preset(
    data: CoreEngineLibWordLlmPresetCreate,
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """Create preset. Any authenticated user."""
    item = await CoreEngineLibWordLlmService.create_preset(
        name=data.name,
        description=data.description,
        html=data.html,
        css=data.css,
    )

    if not item:
        raise HTTPException(status_code=400, detail="Failed to create preset")

```



```

    return JsonResponse({
        "success": True,
        "data": item,
    })

# =====
# PRESETS – UPDATE
# =====

@router.put("/presets/{preset_id}")
async def update_preset(
    preset_id: int,
    data: CoreEngineLibWordLlmPresetUpdate,
    current_user: dict = Depends(get_current_user),
) -> JsonResponse:
    """Update preset. Any authenticated user."""
    item = await CoreEngineLibWordLlmService.update_preset(
        preset_id=preset_id,
        name=data.name,
        description=data.description,
        html=data.html,
        css=data.css,
    )

    if not item:
        raise HTTPException(status_code=404, detail="Preset not found")

    return JsonResponse({
        "success": True,
        "data": item,
    })

# =====
# PRESETS – SOFT DELETE
# =====

@router.delete("/presets/{preset_id}")
async def delete_preset(
    preset_id: int,
    current_user: dict = Depends(get_current_user),
) -> JsonResponse:
    """Soft delete preset. Any authenticated user."""
    result = await CoreEngineLibWordLlmService.delete_preset(preset_id)

    if not result:
        raise HTTPException(status_code=404, detail="Preset not found")

    return JsonResponse({
        "success": True,
        "message": "Preset deleted",
    })

```

```

# =====
# PRESETS – RESTORE
# =====

@router.post("/presets/{preset_id}/restore")
async def restore_preset(
    preset_id: int,
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """Restore deleted preset. Any authenticated user."""
    result = await CoreEngineLibWordLlmService.restore_preset(preset_id)

    if not result:
        raise HTTPException(status_code=404, detail="Preset not found")

    return JSONResponse({
        "success": True,
        "message": "Preset restored",
    })

# =====
# PRESETS – UPLOAD THUMBNAIL
# =====

@router.post("/presets/{preset_id}/thumbnail")
async def upload_thumbnail(
    preset_id: int,
    file: UploadFile = File(...),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """Upload PNG thumbnail for preset. Any authenticated user."""
    try:
        result = await CoreEngineLibWordLlmService.upload_thumbnail(preset_id, file)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Upload error: {str(e)}")

    if not result:
        raise HTTPException(status_code=404, detail="Preset not found")

    return JSONResponse({
        "success": True,
        "data": result,
    })

# =====
# PRESETS – DELETE THUMBNAIL
# =====

@router.delete("/presets/{preset_id}/thumbnail")
async def delete_thumbnail(
    preset_id: int,
    current_user: dict = Depends(get_current_user),

```

```

) -> JSONResponse:
    """Delete preset thumbnail. Any authenticated user."""
    result = await CoreEngineLibWordLlmService.delete_thumbnail(preset_id)

    if not result:
        raise HTTPException(status_code=404, detail="Preset not found")

    return JSONResponse({
        "success": True,
        "message": "Thumbnail deleted",
    })

# =====
# CHAT – HISTORY
# =====

@router.get("/chat/{page_id}/history")
async def get_chat_history(page_id: int) -> JSONResponse:
    """
    Chat history for a page. Public endpoint – needed to show history
    when opening the LLM editor.
    """
    messages = await CoreEngineLibWordLlmService.load_chat_history(page_id)

    return JSONResponse({
        "success": True,
        "data": messages,
    })

# =====
# CHAT – CLEAR HISTORY
# =====

@router.delete("/chat/{page_id}/history")
async def clear_chat_history(
    page_id: int,
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """
    Clear chat history for a page.

    Deletes all rows from page_chat for this page. Any authenticated user.
    """
    deleted = await CoreEngineLibWordLlmService.clear_chat_history(page_id)

    return JSONResponse({
        "success": True,
        "deleted": deleted,
    })

# =====
# CHAT – ACTIVE RUN
# =====

```

```

@router.get("/chat/{page_id}/run/active")
async def get_active_run(page_id: int) -> JSONResponse:
    """
    Return the latest unfinished run for a page, or null.

    Used by the frontend after a WebSocket reconnect: if a run is
    still active, the client shows a progress bubble and waits for
    it to finish (or cancels it).

    Public endpoint – same as chat history. The run belongs to the
    page, not to a specific user.
    """
    run = await CoreEngineLibWordLlmRuns.get_active_run_for_page(page_id)

    return JSONResponse({
        "success": True,
        "data": run,
    })

# =====
# WEBSOCKET – CHAT STREAM
# =====
#
# Mounted on the same router, so the final path is:
# /core/engine/lib/word/llm/ws/{page_id}
#
# `llm_ws_endpoint` is a bound method of an instance of
# CoreEngineLibWordLlmWS, exported by llm/ws.py. See the module
# docstring for why we cannot use the unbound class attribute.

router.add_api_websocket_route("/ws/{page_id}", llm_ws_endpoint)

=====
FILE: core/engine/lib/word/llm/runs.py
=====
# neurocad/core/engine/lib/word/llm/runs.py

"""
Service layer for Run records.

Namespace: CoreEngineLibWordLlmRuns

A "Run" is one multi-step LLM generation: plan → fill → effects.
Runs are stored in the shared SQLite database, so:
    - state survives server restarts;
    - all uvicorn workers see the same runs;
    - a run can be inspected or resumed after a WebSocket reconnect.

All methods are async and use the project's SQLite helper
(`get_db_sqlite`). They never raise on "not found" – they return None
or an empty list instead, matching the style of the LLMService.
"""

import json

```

```

from datetime import datetime
from typing import Any, Dict, List, Optional

from sqlalchemy import select, desc

from .....models.base import Run
from .....utils.sqlite import get_db_sqlite

class CoreEngineLibWordLlmRuns:
    """Static service for Run records."""

    # =====
    # SERIALIZATION HELPERS
    # =====

    @staticmethod
    def _dumps(value: Any) -> Optional[str]:
        """JSON-encode a value, or return None for None."""
        if value is None:
            return None
        try:
            return json.dumps(value, ensure_ascii=False)
        except (TypeError, ValueError):
            return None

    @staticmethod
    def _loads(text: Optional[str]) -> Any:
        """JSON-decode a value, or return None for None/empty."""
        if not text:
            return None
        try:
            return json.loads(text)
        except json.JSONDecodeError:
            return None

    @staticmethod
    def _run_to_dict(run: Run, include_heavy: bool = False) -> Dict[str, Any]:
        """
        Convert a Run model to a dict.

        `include_heavy=False` (default) omits block_catalog, plan,
        filled, final_html – useful when listing runs for a page.
        Set to True to get everything.
        """
        data = {
            "id": run.id,
            "page_id": run.page_id,
            "user_id": run.user_id,
            "status": run.status,
            "step": run.step,
            "user_message": run.user_message,
            "message": run.message,
            "error": run.error,
            "created_at": run.created_at.isoformat() if run.created_at else None,
            "updated_at": run.updated_at.isoformat() if run.updated_at else None,

```

```

    }
    if include_heavy:
        data.update({
            "block_catalog": CoreEngineLibWordLlmRuns._loads(run.block_catalog),
            "plan": CoreEngineLibWordLlmRuns._loads(run.plan),
            "filled": CoreEngineLibWordLlmRuns._loads(run.filled),
            "effects": CoreEngineLibWordLlmRuns._loads(run.effects),
            "final_html": run.final_html,
        })
    return data

# =====
# CREATE
# =====

@staticmethod
async def create_run(
    page_id: int,
    user_message: str,
    block_catalog: Optional[List[Dict[str, Any]]] = None,
    user_id: Optional[int] = None,
) -> Optional[Dict[str, Any]]:
    """
    Create a new run in status 'pending'.

    Returns the run dict (with heavy fields included), or None on error.
    """
    async for session in get_db_sqlite():
        run = Run(
            page_id=page_id,
            user_id=user_id,
            status="pending",
            step=0,
            user_message=user_message,
            block_catalog=CoreEngineLibWordLlmRuns._dumps(block_catalog),
            created_at=datetime.now(),
            updated_at=datetime.now(),
        )
        session.add(run)
        await session.commit()
        await session.refresh(run)
        return CoreEngineLibWordLlmRuns._run_to_dict(run, include_heavy=True)

    return None

# =====
# GET
# =====

@staticmethod
async def get_run(run_id: int) -> Optional[Dict[str, Any]]:
    """Get a run by id with all fields, or None."""
    async for session in get_db_sqlite():
        stmt = select(Run).where(Run.id == run_id)
        result = await session.execute(stmt)
        run = result.scalar_one_or_none()

```

```

        if not run:
            return None
        return CoreEngineLibWordLlmRuns._run_to_dict(run, include_heavy=True)

    return None

@staticmethod
async def get_active_run_for_page(page_id: int) -> Optional[Dict[str, Any]]:
    """
    Return the latest run for a page that is not yet finished.

    Active statuses: pending, planning, filling, effects.
    Returns None if there is no active run.
    """
    active_statuses = ("pending", "planning", "filling", "effects")
    async for session in get_db_sqlite():
        stmt = (
            select(Run)
            .where(
                Run.page_id == page_id,
                Run.status.in_(active_statuses),
            )
            .order_by(desc(Run.created_at))
            .limit(1)
        )
        result = await session.execute(stmt)
        run = result.scalar_one_or_none()
        if not run:
            return None
        return CoreEngineLibWordLlmRuns._run_to_dict(run, include_heavy=True)

    return None

@staticmethod
async def list_runs_for_page(
    page_id: int,
    limit: int = 20,
) -> List[Dict[str, Any]]:
    """
    List recent runs for a page (light fields only).

    Heavy fields (block_catalog, plan, filled, final_html) are
    omitted. Use `get_run(id)` to fetch the full data for one run.
    """
    async for session in get_db_sqlite():
        stmt = (
            select(Run)
            .where(Run.page_id == page_id)
            .order_by(desc(Run.created_at))
            .limit(limit)
        )
        result = await session.execute(stmt)
        rows = result.scalars().all()
        return [
            CoreEngineLibWordLlmRuns._run_to_dict(r, include_heavy=False)
            for r in rows
        ]

```

```

    ]

    return []

# =====
# UPDATE
# =====

@staticmethod
async def update_run_status(
    run_id: int,
    status: str,
    step: Optional[int] = None,
    message: Optional[str] = None,
    error: Optional[str] = None,
) -> bool:
    """Update lifecycle fields of a run."""
    async for session in get_db_sqlite():
        stmt = select(Run).where(Run.id == run_id)
        result = await session.execute(stmt)
        run = result.scalar_one_or_none()
        if not run:
            return False

        run.status = status
        if step is not None:
            run.step = step
        if message is not None:
            run.message = message
        if error is not None:
            run.error = error
        run.updated_at = datetime.now()

        await session.commit()
        return True

    return False

@staticmethod
async def set_run_plan(run_id: int, plan: List[Dict[str, Any]]) -> bool:
    """Save the plan (step 1 result)."""
    async for session in get_db_sqlite():
        stmt = select(Run).where(Run.id == run_id)
        result = await session.execute(stmt)
        run = result.scalar_one_or_none()
        if not run:
            return False
        run.plan = CoreEngineLibWordLlmRuns._dumps(plan)
        run.updated_at = datetime.now()
        await session.commit()
        return True

    return False

@staticmethod
async def set_run_filled(run_id: int, filled: Dict[str, str]) -> bool:

```



```

        """Save the filled blocks (step 2 result)."""
        async for session in get_db_sqlite():
            stmt = select(Run).where(Run.id == run_id)
            result = await session.execute(stmt)
            run = result.scalar_one_or_none()
            if not run:
                return False
            run.filled = CoreEngineLibWordLlmRuns._dumps(filled)
            run.updated_at = datetime.now()
            await session.commit()
            return True

        return False

    @staticmethod
    async def set_run_final(
        run_id: int,
        final_html: str,
        message: Optional[str] = None,
    ) -> bool:
        """Save the final HTML and mark the run as done."""
        async for session in get_db_sqlite():
            stmt = select(Run).where(Run.id == run_id)
            result = await session.execute(stmt)
            run = result.scalar_one_or_none()
            if not run:
                return False
            run.final_html = final_html
            run.status = "done"
            run.message = message
            run.updated_at = datetime.now()
            await session.commit()
            return True

        return False

# =====
# DELETE / CANCEL
# =====

    @staticmethod
    async def cancel_run(run_id: int, reason: str = "cancelled by user") -> bool:
        """
        Mark a run as cancelled.

        Does not delete the record – cancelled runs stay for inspection.
        """
        return await CoreEngineLibWordLlmRuns.update_run_status(
            run_id, status="cancelled", message=reason
        )

=====
FILE: core/engine/lib/word/llm/schema.py
=====
# neurocad/core/engine/lib/word/llm/schema.py

```

```

"""
Pydantic schemas for LLM editor (presets, chat, history).

Namespace: CoreEngineLibWordLlm*
"""

from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import datetime as dt

# =====
# PRESETS (page_pres)
# =====

class CoreEngineLibWordLlmPresetBase(BaseModel):
    """Base preset fields"""
    name: str = Field(..., min_length=1, max_length=255, description="Название")
    description: Optional[str] = Field(None, description="Описание")
    html: Optional[str] = Field(None, description="HTML пресета")
    css: Optional[str] = Field(None, description="CSS пресета")

class CoreEngineLibWordLlmPresetCreate(CoreEngineLibWordLlmPresetBase):
    """Create preset"""
    pass

class CoreEngineLibWordLlmPresetUpdate(BaseModel):
    """Update preset – all fields optional"""
    name: Optional[str] = Field(None, min_length=1, max_length=255)
    description: Optional[str] = None
    html: Optional[str] = None
    css: Optional[str] = None

class CoreEngineLibWordLlmPresetItem(BaseModel):
    """Preset for response"""
    id: int
    name: str
    description: Optional[str] = None
    html: Optional[str] = None
    css: Optional[str] = None
    thumbnail_path: Optional[str] = None
    is_delete: int
    created_at: Optional[dt] = None
    updated_at: Optional[dt] = None

    class Config:
        from_attributes = True

class CoreEngineLibWordLlmPresetListResponse(BaseModel):
    """Response with preset list"""
    success: bool = True
    data: List[CoreEngineLibWordLlmPresetItem]

```

```

total: int = 0

class CoreEngineLibWordLlmPresetItemResponse(BaseModel):
    """Response with a single preset"""
    success: bool = True
    data: CoreEngineLibWordLlmPresetItem

# =====
# THUMBNAIL UPLOAD
# =====

class CoreEngineLibWordLlmPresetThumbnailResponse(BaseModel):
    """Response for thumbnail upload"""
    success: bool = True
    data: dict # {"thumbnail_path": "presets/1.png"}

# =====
# CHAT
# =====

class CoreEngineLibWordLlmBlockItem(BaseModel):
    """
    One block in the catalog sent from the frontend.

    The catalog is assembled by chat.js from the currently registered
    GrapesJS blocks and sent with every chat message. The backend embeds
    it into the system prompt, so the model assembles pages from the
    existing blocks instead of inventing markup.
    """
    id: str = Field(..., description="GrapesJS block id, e.g. 'core-hero'")
    label: str = Field(..., description="Human-readable name, e.g. 'Hero (баннер)')")
    category: str = Field(..., description="Category name, e.g. 'Секции'")
    html: str = Field("", description="Ready-to-use HTML fragment")

class CoreEngineLibWordLlmChatMessageCreate(BaseModel):
    """Request to send a chat message"""
    message: str = Field(..., min_length=1, description="Текст сообщения пользователя")
    block_catalog: Optional[List[CoreEngineLibWordLlmBlockItem]] = Field(
        None,
        description="Каталог блоков, собранный на фронте. Передаётся в системный промпт.",
    )

class CoreEngineLibWordLlmChatMessage(BaseModel):
    """Single chat message"""
    id: Optional[int] = None
    role: str = Field(..., description="'user' | 'assistant' | 'system'")
    content: str
    model: Optional[str] = None
    prompt_tokens: Optional[int] = None
    completion_tokens: Optional[int] = None
    created_at: Optional[dt] = None

```

```

class Config:
    from_attributes = True

class CoreEngineLibWordLlmChatHistoryResponse(BaseModel):
    """Response with chat history"""
    success: bool = True
    data: List[CoreEngineLibWordLlmChatMessage]

class CoreEngineLibWordLlmChatSendResponse(BaseModel):
    """Response to send message"""
    success: bool = True
    data: dict # { user_message, assistant_message, html, css }

=====
FILE: core/engine/lib/word/llm/service.py
=====
# neurocad/core/engine/lib/word/llm/service.py

"""
LLM service: presets (CRUD + thumbnails) and chat history (page_chat).

Namespace: CoreEngineLibWordLlmService

Note: LLM calls (routing, agents) live in ws.py + agent/*.py.
This service only handles DB-side operations: presets and chat history.
"""

import os
import re
import uuid
import json
from pathlib import Path
from datetime import datetime
from typing import Optional, Dict, Any, List

from sqlalchemy import select, func, delete
from fastapi import UploadFile

from .....models.base import PagePres, PageChat
from .....utils.sqlite import get_db_sqlite

class CoreEngineLibWordLlmService:
    """LLM editor service: presets and chat history."""

    # =====
    # PATHS / CONFIG
    # =====

    # Media is relative to cwd: <project>/media/
    MEDIA_ROOT = Path("media")
    PRESETS_DIR = MEDIA_ROOT / "presets"
    MEDIA_URL = "/media"

```

```

THUMBNAİL_EXTENSIONS = {'.png', '.jpg', '.jpeg', '.webp'}

#: How many previous messages to include as dialogue context.
HISTORY_LIMIT = 20

# =====
# LIST PRESETS
# =====

@staticmethod
async def list_presets(
    include_deleted: bool = False,
) -> Dict[str, Any]:
    """Get presets list."""
    async for session in get_db_sqlite():
        stmt = select(PagePres)

        if not include_deleted:
            stmt = stmt.where(PagePres.is_delete == 0)

        stmt = stmt.order_by(PagePres.created_at.asc())

        result = await session.execute(stmt)
        rows = result.scalars().all()

        items = []
        for preset in rows:
            items.append({
                "id": preset.id,
                "name": preset.name,
                "description": preset.description,
                "html": preset.html,
                "css": preset.css,
                "thumbnail_path": preset.thumbnail_path,
                "is_delete": preset.is_delete,
                "created_at": preset.created_at.isoformat() if preset.created_at else None,
                "updated_at": preset.updated_at.isoformat() if preset.updated_at else None,
            })

        return {
            "items": items,
            "total": len(items),
        }

    return {"items": [], "total": 0}

# =====
# GET PRESET
# =====

@staticmethod
async def get_preset(preset_id: int) -> Optional[Dict[str, Any]]:
    """Get one preset by ID."""
    async for session in get_db_sqlite():
        stmt = select(PagePres).where(

```

```

        PagePres.id == preset_id,
        PagePres.is_delete == 0,
    )
    result = await session.execute(stmt)
    preset = result.scalar_one_or_none()

    if not preset:
        return None

    return {
        "id": preset.id,
        "name": preset.name,
        "description": preset.description,
        "html": preset.html,
        "css": preset.css,
        "thumbnail_path": preset.thumbnail_path,
        "is_delete": preset.is_delete,
        "created_at": preset.created_at.isoformat() if preset.created_at else None,
        "updated_at": preset.updated_at.isoformat() if preset.updated_at else None,
    }

    return None

# =====
# CREATE PRESET
# =====

@staticmethod
async def create_preset(
    name: str,
    description: Optional[str] = None,
    html: Optional[str] = None,
    css: Optional[str] = None,
) -> Optional[Dict[str, Any]]:
    """Create a new preset."""
    async for session in get_db_sqlite():
        new_preset = PagePres(
            name=name.strip(),
            description=description,
            html=html,
            css=css,
            is_delete=0,
            created_at=datetime.now(),
            updated_at=datetime.now(),
        )
        session.add(new_preset)
        await session.commit()
        await session.refresh(new_preset)

    return {
        "id": new_preset.id,
        "name": new_preset.name,
        "description": new_preset.description,
        "html": new_preset.html,
        "css": new_preset.css,
        "thumbnail_path": new_preset.thumbnail_path,
    }

```

```

        "is_delete": new_preset.is_delete,
        "created_at": new_preset.created_at.isoformat() if new_preset.created_at else None,
        "updated_at": new_preset.updated_at.isoformat() if new_preset.updated_at else None,
    }

    return None

# =====
# UPDATE PRESET
# =====

@staticmethod
async def update_preset(
    preset_id: int,
    name: Optional[str] = None,
    description: Optional[str] = None,
    html: Optional[str] = None,
    css: Optional[str] = None,
) -> Optional[Dict[str, Any]]:
    """Update preset. Only provided fields are updated."""
    async for session in get_db_sqlite():
        stmt = select(PagePres).where(
            PagePres.id == preset_id,
            PagePres.is_delete == 0,
        )
        result = await session.execute(stmt)
        preset = result.scalar_one_or_none()

        if not preset:
            return None

        if name is not None:
            preset.name = name.strip()
        if description is not None:
            preset.description = description
        if html is not None:
            preset.html = html
        if css is not None:
            preset.css = css

        preset.updated_at = datetime.now()
        await session.commit()
        await session.refresh(preset)

    return {
        "id": preset.id,
        "name": preset.name,
        "description": preset.description,
        "html": preset.html,
        "css": preset.css,
        "thumbnail_path": preset.thumbnail_path,
        "is_delete": preset.is_delete,
        "created_at": preset.created_at.isoformat() if preset.created_at else None,
        "updated_at": preset.updated_at.isoformat() if preset.updated_at else None,
    }

```

```

        return None

# =====
# SOFT DELETE
# =====

@staticmethod
async def delete_preset(preset_id: int) -> bool:
    """Soft delete preset (is_delete = 1)."""
    async for session in get_db_sqlite():
        stmt = select(PagePres).where(
            PagePres.id == preset_id,
            PagePres.is_delete == 0,
        )
        result = await session.execute(stmt)
        preset = result.scalar_one_or_none()

        if not preset:
            return False

        preset.is_delete = 1
        preset.updated_at = datetime.now()
        await session.commit()
        return True

    return False

# =====
# RESTORE
# =====

@staticmethod
async def restore_preset(preset_id: int) -> bool:
    """Restore deleted preset (is_delete = 0)."""
    async for session in get_db_sqlite():
        stmt = select(PagePres).where(
            PagePres.id == preset_id,
            PagePres.is_delete == 1,
        )
        result = await session.execute(stmt)
        preset = result.scalar_one_or_none()

        if not preset:
            return False

        preset.is_delete = 0
        preset.updated_at = datetime.now()
        await session.commit()
        return True

    return False

# =====
# UPLOAD THUMBNAIL
# =====

```



```

@staticmethod
async def upload_thumbnail(
    preset_id: int,
    file: UploadFile,
) -> Optional[Dict[str, Any]]:
    """Upload PNG thumbnail for preset."""
    async for session in get_db_sqlite():
        stmt = select(PagePres).where(
            PagePres.id == preset_id,
            PagePres.is_delete == 0,
        )
        result = await session.execute(stmt)
        preset = result.scalar_one_or_none()

        if not preset:
            return None

        original_name = file.filename or "thumbnail.png"
        ext = Path(original_name).suffix.lower()
        if ext not in CoreEngineLibWordLlmService.THUMBNAIL_EXTENSIONS:
            raise ValueError(
                f"Invalid extension: {ext}. "
                f"Allowed: {' '.join(CoreEngineLibWordLlmService.THUMBNAIL_EXTENSIONS)}"
            )

        CoreEngineLibWordLlmService.PRESETS_DIR.mkdir(parents=True, exist_ok=True)

        final_name = f"{preset_id}{ext}"
        save_path = CoreEngineLibWordLlmService.PRESETS_DIR / final_name

        # Remove stale thumbnails with other extensions.
        # Failures are ignored – the new file overwrites the same name.
        for old_ext in CoreEngineLibWordLlmService.THUMBNAIL_EXTENSIONS:
            old_file = CoreEngineLibWordLlmService.PRESETS_DIR / f"{preset_id}{old_ext}"
            if old_file.exists() and old_file != save_path:
                try:
                    old_file.unlink()
                except Exception:
                    pass

        with open(save_path, "wb") as buffer:
            while True:
                chunk = await file.read(1024 * 64)
                if not chunk:
                    break
                buffer.write(chunk)
        await file.close()

        thumbnail_path = f"presets/{final_name}"
        preset.thumbnail_path = thumbnail_path
        preset.updated_at = datetime.now()
        await session.commit()
        await session.refresh(preset)

    return {
        "id": preset.id,

```

```

        "thumbnail_path": thumbnail_path,
    }

    return None

# =====
# DELETE THUMBNAIL
# =====

@staticmethod
async def delete_thumbnail(preset_id: int) -> bool:
    """Delete preset thumbnail (file + DB record)."""
    async for session in get_db_sqlite():
        stmt = select(PagePres).where(
            PagePres.id == preset_id,
            PagePres.is_delete == 0,
        )
        result = await session.execute(stmt)
        preset = result.scalar_one_or_none()

        if not preset:
            return False

        for ext in CoreEngineLibWordLlmService.THUMBNAIL_EXTENSIONS:
            file_path = CoreEngineLibWordLlmService.PRESETS_DIR / f"{preset_id}{ext}"
            if file_path.exists():
                try:
                    file_path.unlink()
                except Exception:
                    pass

        preset.thumbnail_path = None
        preset.updated_at = datetime.now()
        await session.commit()
        return True

    return False

# =====
# CHAT – HISTORY
# =====

@staticmethod
async def load_chat_history(page_id: int) -> List[Dict[str, Any]]:
    """Load full chat history by page, from old to new."""
    async for session in get_db_sqlite():
        stmt = (
            select(PageChat)
            .where(PageChat.page_id == page_id)
            .order_by(PageChat.created_at.asc(), PageChat.id.asc())
        )
        result = await session.execute(stmt)
        rows = result.scalars().all()

    return [
        {

```

```

        "id": m.id,
        "role": m.role,
        "content": m.content,
        "model": m.model,
        "prompt_tokens": m.prompt_tokens,
        "completion_tokens": m.completion_tokens,
        "created_at": m.created_at.isoformat() if m.created_at else None,
    }
    for m in rows
]

return []

# =====
# CHAT – CLEAR HISTORY
# =====

@staticmethod
async def clear_chat_history(page_id: int) -> int:
    """
    Delete all chat messages for a page.

    Returns the number of deleted rows.
    """
    async for session in get_db_sqlite():
        stmt = delete(PageChat).where(PageChat.page_id == page_id)
        result = await session.execute(stmt)
        await session.commit()
        return result.rowcount or 0

    return 0

# =====
# CHAT – SAVE MESSAGE
# =====

@staticmethod
async def save_chat_message(
    page_id: int,
    role: str,
    content: str,
    model: Optional[str] = None,
    prompt_tokens: Optional[int] = None,
    completion_tokens: Optional[int] = None,
) -> Optional[Dict[str, Any]]:
    """Save one chat message."""
    async for session in get_db_sqlite():
        msg = PageChat(
            page_id=page_id,
            role=role,
            content=content,
            model=model,
            prompt_tokens=prompt_tokens,
            completion_tokens=completion_tokens,
            created_at=datetime.now(),
        )

```

```

        session.add(msg)
        await session.commit()
        await session.refresh(msg)

    return {
        "id": msg.id,
        "role": msg.role,
        "content": msg.content,
        "model": msg.model,
        "prompt_tokens": msg.prompt_tokens,
        "completion_tokens": msg.completion_tokens,
        "created_at": msg.created_at.isoformat() if msg.created_at else None,
    }

    return None

```

```

# =====
# CHAT – LOAD HISTORY AS MESSAGES
# =====

```

```

@staticmethod
async def load_history_messages(
    page_id: int,
    limit: Optional[int] = None,
) -> List[Dict[str, str]]:
    """
    Load the last `limit` messages from page_chat and return them
    as a list of {"role": "user"|"assistant", "content": "..."}
    dicts, oldest → newest.
    """
    if limit is None:
        limit = CoreEngineLibWordLlmService.HISTORY_LIMIT

    rows = await CoreEngineLibWordLlmService.load_chat_history(page_id)
    tail = rows[-limit:] if len(rows) > limit else rows
    return [
        {"role": m["role"], "content": m["content"]}
        for m in tail
        if m.get("role") in ("user", "assistant") and m.get("content")
    ]

```

```

=====
FILE: core/engine/lib/word/llm/step.py
=====

```

```

# neurocad/core/engine/lib/word/llm/step.py

```

```

"""
Multi-step LLM flow: plan → fill → effects → svg-illustrations.

```

```

Namespace: CoreEngineLibWordLlmStep

```

This module is a thin facade: each static method delegates to the corresponding module in `steps/`. The split exists so that the four steps – which are large and independent – live in their own files.

```

steps/plan.py      – step_plan

```

```

steps/fill.py      - step_fill
steps/effects.py   - step_effects
steps/svg.py       - step_svg_illustrations
steps/common.py    - shared helpers (logging, JSON/SVG parsing, ...)

```

The public API of `CoreEngineLibWordLlmStep` is unchanged: all existing callers (agent/create.py, agent/fill.py, agent/effect.py, ...) continue to work without modification.

Generator protocol

Every step is an async generator that yields progress events and, in the end, yields a final result. The caller (an agent in agent/) consumes the events and forwards them to the client.

Every yield is a dict with a "type" field:

Progress events (intermediate, may repeat):

```

{"type": "step",          "step": "planning", "message": "..."}
{"type": "plan",          "blocks": [...]}
{"type": "step",          "step": "filling",  "message": "..."}
{"type": "fill_progress", "request": 1, "total": 3, "block_ids": [...]}
{"type": "step",          "step": "effects",  "message": "..."}
{"type": "step",          "step": "svg",      "message": "..."}
{"type": "svg_progress",  "current": 2, "total": 6, "alt": "..."}

```

Final event (exactly one at the end):

```

{"type": "result", "html": "...", "message": "..."}

```

On error:

```

{"type": "error", "message": "..."}

```

Logging: uses provider.log (app.state.log, passed via the provider).

"""

```

from .steps.plan import step_plan
from .steps.fill import step_fill
from .steps.effects import step_effects
from .steps.svg import step_svg_illustrations

```

```

class CoreEngineLibWordLlmStep:

```

```

    """Static multi-step LLM flow (facade over steps/*.py)."""

```

```

    step_plan = staticmethod(step_plan)
    step_fill = staticmethod(step_fill)
    step_effects = staticmethod(step_effects)
    step_svg_illustrations = staticmethod(step_svg_illustrations)

```

=====

FILE: core/engine/lib/word/llm/utils.py

=====

```

# neurocad/core/engine/lib/word/llm/utils.py

```

"""

Small helpers shared across the LLM package.

Namespace: CoreEngineLibWordLlmUtils

Converts between the single `page.content` string and the (html, css) pair. No imports from other modules in this package – keep it dependency-free.

"""

import re

class CoreEngineLibWordLlmUtils:

"""Static helpers for the LLM editor."""

@staticmethod

def split_html_css(content: str) -> tuple[str, str]:

"""

Split page.content into (html, css).

If content contains a `

```
"""
```

Base interface for LLM editor agents.

Every agent receives the same input context and returns the same result shape:

```
{
    "message":      str,          # text for the chat
    "html":         str | None,   # full page HTML (replaces canvas)
    "selector":     str | None,   # element marker to replace
    "element_html": str | None,   # new HTML for that element
}
```

The agent may also emit intermediate events to the client via the `emit` callback. Typical events:

```
{"type": "step", "step": "planning", "message": "..."}
{"type": "plan", "blocks": [...]}
{"type": "fill_progress", "request": 1, "total": 3, "block_ids": [...]}
```

`emit` is optional. If it is None, the agent runs silently (used by non-streaming callers).

No `__init__.py` in this package – modules are imported directly.

```
from abc import ABC, abstractmethod
from typing import Any, Awaitable, Callable, Dict, Optional
```

```
#: Type of the emit callback: async function that sends one JSON event.
EmitFn = Callable[[Dict[str, Any]], Awaitable[None]]
```

```
class CoreEngineLibWordLlmAgentBase(ABC):
    """Base class for all LLM editor agents."""
```

```
#: Human-readable agent name (also used by the dumper).
name: str = "base"
```

```
@abstractmethod
async def run(
    self,
    *,
    provider,
    user_message: str,
    page_id: int,
    run_id: Any,
    emit: Optional[EmitFn] = None,
    selection: Optional[Dict[str, Any]] = None,
    block_catalog: Optional[list] = None,
    current_html: Optional[str] = None,
    history: Optional[list] = None,
) -> Dict[str, Any]:
    """
    Run the agent.
```

Input context:

- provider – LLM provider instance (created by ws.py)
- user_message – what the user typed
- page_id – page being edited
- run_id – run record id (for dumping)
- emit – async callback to send intermediate events to the client (optional; None means silent)
- selection – element marker {"selector", "tag", "classes", "outer_html"} or legacy {"block_id"}
- block_catalog – list of blocks from the frontend
- current_html – current page HTML (if any)
- history – previous chat messages

Output:

```
{
    "message": str,
    "html": str | None,
    "selector": str | None,
    "element_html": str | None,
}
```

"""

raise NotImplementedError

```
=====
FILE: core/engine/lib/word/llm/agent/create_effect.py
=====
```

```
# neurocad/core/engine/lib/word/llm/agent/create_effect.py
```

```
"""
```

```
Agent: create_effect.
```

Builds a DRAFT of a new effect from a user's free-form request.

This agent does NOT write anything to disk. It returns a JSON draft:

```
{
    "id": "fx-shimmer-dots",
    "label": "Мерцающие точки",
    "hint": "Мелкие точки с плавным мерцанием",
    "css": ".core-engine-lib-word-blocks .fx-shimmer-dots { ... }",
    "media": "<svg viewBox='0 0 24 24'>...</svg>"
}
```

The draft is applied to the selected element immediately by the frontend (see chat/create.js → CreateSession.handleDraft). The user judges the result by looking at the element, not by reading CSS.

The user may ask for tweaks ("сделай точки крупнее", "переименуй"). In that case the agent is called again with the previous draft embedded in the prompt, and returns a new draft. The frontend re-applies it live.

Saving is NOT done through the chat. When the user is satisfied they click the SAVE icon on the pending block in the palette. Only then does the frontend POST /editor/effects to persist the effect.

Agent output shape:

```
{
  "message": "Готово. Я применил эффект к выделенному элементу. ...",
  "draft": { "id": ..., "label": ..., "hint": ..., "css": ..., "media": ... },
}
```

The `message` is intentionally short and technical-detail-free: the user works as a "vibecoder" and should not see CSS, ids, labels, or hints in the chat log. The `draft` is structured data – the frontend uses it to apply the effect; the chat UI does not display it.

If the model fails to produce a valid draft, `draft` is None and `message` explains what happened.

Id collisions

The client sends `existing\_effect\_ids` – the list of ids already registered in the palette. The prompt embeds this list and asks the LLM not to pick any of them. `\_sanitize\_draft` also rejects a draft whose id is already taken, so we never hand a colliding id to the frontend only for it to fail on POST with 409. If the id is taken, the agent returns `draft=None` with a message asking the user to rephrase – the model will generate a fresh draft next turn.

Safety:

- The model's CSS and SVG are sanitized here so the user sees a VALID draft. If sanitization strips too much, the agent returns a clear message instead of a broken draft.
- The draft's `id` is validated against `existing\_effect\_ids` so the user cannot save a draft that would collide with a real effect.

Namespace: CoreEngineLibWordLlmAgentCreateEffect
"""

```
import json
import re
from typing import Any, Dict, List, Optional

from .base import CoreEngineLibWordLlmAgentBase
from ..prompts.create_effect import build_create_effect_prompt

# =====
# CONSTANTS
# =====

#: Same pattern the service validates against on POST.
EFFECT_ID_RE = re.compile(r"^\fx-[a-z0-9][a-z0-9-]{0,63}$")

#: Forbidden constructs in the draft CSS.
_FORBIDDEN_CSS = [
    re.compile(r"@import", re.IGNORECASE),
    re.compile(r"expression\s*\(", re.IGNORECASE),
```

```

re.compile(r"javascript\s*:", re.IGNORECASE),
re.compile(r"behavior\s*:", re.IGNORECASE),
re.compile(r"-moz-binding", re.IGNORECASE),
re.compile(r"url\s*(\s*['\"]?\s*javascript:", re.IGNORECASE),
]

#: Forbidden constructs in the draft SVG.
_FORBIDDEN_SVG = [
    re.compile(r"<script", re.IGNORECASE),
    re.compile(r"javascript\s*:", re.IGNORECASE),
    re.compile(r"on\w+\s*=", re.IGNORECASE),          # onload=, onclick=, ...
    re.compile(r"<foreignObject", re.IGNORECASE),
    re.compile(r"<use\s+[\^>]*href\s*=\s*['\"]https?:", re.IGNORECASE),
]

```

```

class CoreEngineLibWordLlmAgentCreateEffect(CoreEngineLibWordLlmAgentBase):
    """Build a draft of a new effect from a user request."""

```

```

    name = "create_effect"

```

```

    async def run(
        self,
        *,
        provider,
        user_message: str,
        page_id: int,
        run_id: Any,
        emit=None,
        selection: Optional[Dict[str, Any]] = None,
        block_catalog: Optional[list] = None,
        current_html: Optional[str] = None,
        history: Optional[list] = None,
        # Optional: the previous draft (if the user is iterating).
        previous_draft: Optional[Dict[str, Any]] = None,
        # List of effect ids already registered in the palette. Used to
        # ask the LLM not to pick a colliding id, and to reject a draft
        # that still collides.
        existing_effect_ids: Optional[List[str]] = None,
    ) -> Dict[str, Any]:

```

```

        # ---- emit: step at the start ----

```

```

        if emit:
            await emit({
                "type": "step",
                "step": "create_effect",
                "message": "Придумываю эффект...",
            })

```

```

        # ---- normalize existing ids ----

```

```

        existing_ids = self._normalize_existing_ids(existing_effect_ids)
        print(
            f"[create_effect] existing ids: {len(existing_ids)}",
            flush=True,
        )

```

```

system_prompt = build_create_effect_prompt(existing_ids)

# ---- user content: request + (optional) previous draft ----
user_content = self._build_user_content(user_message, previous_draft)

# ---- dump prompt ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name="create_effect",
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
    )
except Exception as e:
    print(f"[create_effect] dump request failed: {e}", flush=True)

# ---- build messages ----
messages = [{"role": "system", "content": system_prompt}]
if history:
    for m in history:
        role = m.get("role")
        content = m.get("content")
        if role in ("user", "assistant") and content:
            messages.append({"role": role, "content": str(content)})
messages.append({"role": "user", "content": user_content})

# ---- provider call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[create_effect] LLM error: {e}", flush=True)
    return {
        "message": f"⚠ Ошибка LLM: {e}",
        "draft": None,
    }

# ---- dump response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="create_effect",
        raw_response=raw,
    )
except Exception as e:
    print(f"[create_effect] dump response failed: {e}", flush=True)

# ---- parse JSON draft ----
data = self._parse_draft(raw)
if not data:
    print(
        f"[create_effect] failed to parse a draft from: "
        f"{str(raw)[:200]}!", flush=True,
    )

```

```

    )
    return {
        "message": "⚠ Не удалось сгенерировать черновик эффекта. Попробуйте переформулировать запрос.",
        "draft": None,
    }

# ---- sanitize draft (rejects id collisions) ----
try:
    draft = self._sanitize_draft(data, existing_ids)
except ValueError as e:
    print(f"[create_effect] draft rejected: {e}", flush=True)
    return {
        "message": f"⚠ Модель вернула некорректный черновик: {e}. Попробуйте ещё раз.",
        "draft": None,
    }

# ---- human-readable message (short, no CSS) ----
message = self._render_draft_message(draft)

return {
    "message": message,
    "draft": draft,
}

# =====
# EXISTING IDS
# =====

@staticmethod
def _normalize_existing_ids(
    raw: Optional[List[Any]],
) -> List[str]:
    """
    Normalize the incoming `existing_effect_ids` list.

    - strip whitespace;
    - drop empty entries;
    - drop non-string/int values;
    - deduplicate while preserving order.
    """
    if not raw:
        return []

    seen: set = set()
    result: List[str] = []
    for item in raw:
        if not isinstance(item, (str, int)):
            continue
        s = str(item).strip()
        if not s or s in seen:
            continue
        seen.add(s)
        result.append(s)
    return result

# =====

```

```

# USER CONTENT
# =====

@staticmethod
def _build_user_content(
    user_message: str,
    previous_draft: Optional[Dict[str, Any]],
) -> str:
    """
    Compose the user-side message.

    If the user is iterating on a previous draft ("переименуй в ...",
    "сделай круги чаще"), the previous draft is embedded so the
    model can revise it instead of starting from scratch.
    """
    parts = [user_message.strip()]

    if previous_draft:
        parts.append("")
        parts.append("--- BEGIN PREVIOUS DRAFT ---")
        parts.append(json.dumps(previous_draft, ensure_ascii=False, indent=2))
        parts.append("--- END PREVIOUS DRAFT ---")
        parts.append("")
        parts.append(
            "Это предыдущий черновик. Отредактируй его согласно новому "
            "запросу пользователя и верни обновлённый JSON."
        )

    return "\n".join(parts)

# =====
# PARSE
# =====

@staticmethod
def _parse_draft(raw: str) -> Optional[dict]:
    """
    Parse the LLM response into a draft dict.

    Tolerates markdown fences and surrounding prose. Requires the
    following keys: id, label, css. hint and media are optional
    but will be filled with defaults if missing.
    """
    if not raw:
        return None

    s = raw.strip()

    # ---- strip markdown fence ----
    if s.startswith("```"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("```"):
            s = s[:-3]
        s = s.strip()

```

```

# ---- try the whole string ----
data = CoreEngineLibWordLlmAgentCreateEffect._try_json(s)

# ---- try the first {...} block ----
if not data:
    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        data = CoreEngineLibWordLlmAgentCreateEffect._try_json(
            s[start:end + 1]
        )

if not data:
    return None

# ---- required keys ----
if not isinstance(data, dict):
    return None
if not data.get("id") or not data.get("label") or not data.get("css"):
    return None

return data

@staticmethod
def _try_json(text: str) -> Optional[dict]:
    try:
        data = json.loads(text)
    except Exception:
        return None
    return data if isinstance(data, dict) else None

# =====
# SANITIZE
# =====

@staticmethod
def _sanitize_draft(
    data: dict,
    existing_ids: Optional[List[str]] = None,
) -> dict:
    """
    Validate and clean a draft.

    Raises ValueError with a human-readable reason if the draft
    cannot be used. Otherwise returns a normalized dict with the
    keys: id, label, hint, css, media.

    `existing_ids` – if the draft's id is in this list, the draft
    is rejected. The caller then asks the user to rephrase; the
    LLM will generate a fresh draft on the next turn.
    """
    effect_id = str(data.get("id", "")).strip()
    label = str(data.get("label", "")).strip()
    hint = str(data.get("hint", "")).strip()
    css = str(data.get("css", "")).strip()

```

```

media = str(data.get("media", "")).strip()

# ---- id ----
if not EFFECT_ID_RE.match(effect_id):
    raise ValueError(
        f"некорректный id `{effect_id}` – ожидается fx-<english-slug>"
    )

# ---- id collision ----
if existing_ids and effect_id in existing_ids:
    raise ValueError(
        f"id `{effect_id}` уже занят другим эффектом – "
        f"придумайте другое имя"
    )

# ---- label ----
if not label:
    raise ValueError("пустое название")
if len(label) > 80:
    raise ValueError("слишком длинное название (макс. 80)")

# ---- hint ----
if len(hint) > 200:
    hint = hint[:197] + "..."

# ---- css ----
if not css:
    raise ValueError("пустой CSS")

for re_bad in _FORBIDDEN_CSS:
    if re_bad.search(css):
        raise ValueError(
            f"CSS содержит запрещённую конструкцию: {re_bad.pattern}"
        )

# The outer selector must reference the draft's own id.
expected_selector = f".core-engine-lib-word-blocks .{effect_id}"
if expected_selector not in css:
    raise ValueError(
        f"CSS не содержит ожидаемый селектор `{expected_selector}`"
    )

# ---- media ----
if media:
    for re_bad in _FORBIDDEN_SVG:
        if re_bad.search(media):
            raise ValueError(
                f"SVG содержит запрещённую конструкцию: {re_bad.pattern}"
            )
    # Must look like an svg tag.
    if "<svg" not in media.lower():
        raise ValueError("media не похож на SVG")

return {
    "id": effect_id,
    "label": label,

```

```

        "hint": hint,
        "css": css,
        "media": media,
    }

# =====
# MESSAGE
# =====

@staticmethod
def _render_draft_message(draft: dict) -> str:
    """
    Build the human-readable message that accompanies the draft.

    IMPORTANT: this message is intentionally minimal. The user
    works as a "vibecoder" and judges the result by LOOKING AT
    THE ELEMENT, not by reading CSS or ids.

    So this method:
    - does NOT include the id, label, hint, or CSS;
    - does NOT ask "does the draft look good?" – the user
      cannot judge CSS text;
    - tells the user to LOOK AT THE ELEMENT and either keep
      the effect (by clicking the SAVE icon on the pending
      block in the palette) or describe a tweak.

    The structured `draft` dict is sent separately by the WS
    layer ({ type: "effect_draft", draft: {...} }) and used by
    chat/create.js to apply the effect. It is NOT rendered in
    the chat.
    """
    return (
        "Готово. Я применил эффект к выделенному элементу.\n\n"
        "Если понравилось – нажмите кнопку **«Сохранить»** на новом блоке в палитре "
        "(иконка-дискета). Если хотите поправить – просто опишите, что изменить "
        "(например: «сделай точки крупнее»).\n"
    )

```

```

=====
FILE: core/engine/lib/word/llm/agent/create_page.py
=====

```

```

# neurocad/core/engine/lib/word/llm/agent/create_page.py

```

```

"""

```

```

Agent: create_page – creates OR edits a page.

```

```

Two top-level modes, chosen by `current_html`:

```

```

CREATE (current_html empty)

```

```

Plan-then-generate-and-review:

```

```

Phase 1 – plan: one short LLM call returning section names.

```

```

Phase 2 – for each section:

```

```

    a) generate (HTML + CSS),

```

```

    b) QA pass – a second LLM call that finds and
       fixes common defects (image overflow, empty
       buttons, invisible text, missing @media, ...).

```


Only the QA-approved section is streamed to the client. The user never sees a broken section.

EDIT (current_html non-empty)

Single LLM call. The current page HTML + CSS is embedded into the user message; the model returns the WHOLE updated page.

Logging / dumps

Every step writes through the shared `log()` helper from
`..steps.common` and dumps prompts / responses via
`CoreEngineLibWordLlmDumper` – same pattern as fill.py, effects.py.

Namespace: CoreEngineLibWordLlmAgentCreatePage
"""

```
import json
import re
from typing import Any, Dict, List, Optional, Tuple

from .base import CoreEngineLibWordLlmAgentBase
from ..prompts.create_page import (
    build_plan_prompt,
    build_plan_user_message,
    build_section_prompt,
    build_section_user_message,
    build_review_prompt,
    build_review_user_message,
    build_edit_prompt,
    build_edit_user_message,
    MAX_SECTIONS,
    MIN_SECTIONS,
)
from ..steps.common import build_messages, log
from ..dumper import CoreEngineLibWordLlmDumper

class CoreEngineLibWordLlmAgentCreatePage(CoreEngineLibWordLlmAgentBase):
    """Create OR edit a page."""

    name = "create_page"

    STEP_MAX_TOKENS = 4096
    PLAN_MAX_TOKENS = 512
    REVIEW_MAX_TOKENS = 4096
    EDIT_MAX_TOKENS = 32000

    #: Whether to run the QA pass after each generated section.
    #: Set to False to save tokens / time at the cost of possible
    #: layout bugs in the streamed result.
    REVIEW_ENABLED = True

    # =====
    # ENTRY POINT
    # =====
```

```

async def run(
    self,
    *,
    provider,
    user_message: str,
    page_id: int,
    run_id: Any,
    emit=None,
    selection: Optional[Dict[str, Any]] = None,
    block_catalog: Optional[list] = None,
    current_html: Optional[str] = None,
    history: Optional[list] = None,
) -> Dict[str, Any]:

    has_current = bool(current_html and current_html.strip())
    new_page_requested = self._is_new_page_request(user_message)

    log(
        provider, "info",
        f"create_page start: page_id={page_id} run={run_id!r} "
        f"current_html={len(current_html or '')} chars, "
        f"history={len(history or [])} msgs, "
        f"new_page_requested={new_page_requested}, "
        f"review_enabled={self.REVIEW_ENABLED}",
    )

    if has_current and not new_page_requested:
        return await self._run_edit(
            provider=provider,
            user_message=user_message,
            page_id=page_id,
            run_id=run_id,
            emit=emit,
            current_html=current_html,
            history=history,
        )

    return await self._run_create(
        provider=provider,
        user_message=user_message,
        page_id=page_id,
        run_id=run_id,
        emit=emit,
        history=history,
    )

# =====
# NEW-PAGE HINT
# =====

@staticmethod
def _is_new_page_request(text: str) -> bool:
    t = (text or "").lower()
    hints = (
        "с нуля",
        "с чистого листа",
    )

```

```

        "новую страницу",
        "новый сайт",
        "забуди всё",
        "переделай полностью",
        "перегенерируй с нуля",
        "удали всё и",
    )
    return any(h in t for h in hints)

# =====
# EDIT MODE
# =====

async def _run_edit(
    self,
    *,
    provider,
    user_message: str,
    page_id: int,
    run_id: Any,
    emit,
    current_html: str,
    history: Optional[list],
) -> Dict[str, Any]:

    if emit:
        await emit({
            "type": "step",
            "step": "edit_page",
            "message": "Обновляю страницу...",
        })

    current_css = await self._load_current_page_css()

    system_prompt = build_edit_prompt()
    user_content = build_edit_user_message(
        user_request=user_message,
        current_html=current_html,
        current_css=current_css,
    )

    log(
        provider, "info",
        f"edit request: system={len(system_prompt)} chars, "
        f"user={len(user_content)} chars "
        f"(html={len(current_html)}, css={len(current_css)}), "
        f"max_tokens={self.EDIT_MAX_TOKENS}",
    )

    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name="create_page_edit",
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
    )

```

```

messages = build_messages(system_prompt, user_content, history)

log(
    provider, "info",
    f"edit messages: total={len(messages)} "
    f"(system=1, history={max(0, len(messages) - 2)}, user=1)",
)

try:
    raw = await provider.generate_completion(
        messages,
        max_tokens=self.EDIT_MAX_TOKENS,
    )
except Exception as e:
    log(provider, "error", f"edit LLM error: {e}")
    return self._fail(f"⚠ Ошибка LLM: {e}")

log(provider, "info", f"edit response: raw_len={len(raw or '')}")

CoreEngineLibWordLlmDumper.dump_response(
    run_id=run_id,
    agent_name="create_page_edit",
    raw_response=raw,
)

html, css = self._extract_html_and_css(raw)

if not html and raw:
    s = raw.strip()
    if s.startswith("<"):
        log(
            provider, "warning",
            "edit: raw response starts with '<', using as-is HTML"
        )
        html = s
        css = ""

if not html:
    log(
        provider, "error",
        f"edit non-html answer: {str(raw)[:300]!r}"
    )
    return self._fail(
        "⚠ Модель вернула некорректный ответ. Попробуйте ещё раз."
    )

log(
    provider, "info",
    f"edit parsed: html={len(html)} chars, css={len(css)} chars"
)

html = self._ensure_scope(html)

return {
    "message": "Страница обновлена.",

```

```

        "html": html,
        "css": css,
        "selector": None,
        "element_html": None,
        "streamed": False,
    }

# =====
# CREATE MODE (generate + review per section)
# =====

async def _run_create(
    self,
    *,
    provider,
    user_message: str,
    page_id: int,
    run_id: Any,
    emit,
    history: Optional[list],
) -> Dict[str, Any]:

    if emit:
        await emit({
            "type": "step",
            "step": "create_page",
            "message": "Планирую структуру страницы...",
        })

    # ---- Phase 1: plan ----
    plan = await self._get_plan(
        provider=provider,
        user_message=user_message,
        run_id=run_id,
    )

    if not plan:
        return self._fail(
            "⚠ Не удалось спланировать страницу. Попробуйте ещё раз."
        )

    if emit:
        await emit({
            "type": "step",
            "step": "create_page_plan",
            "message": "План: " + " ", ".join(plan),
        })

    # ---- Phase 2: generate + review each section ----
    total = len(plan)
    html_chunks: List[str] = []
    css_chunks: List[str] = []
    sections_done: List[str] = []

    for i, target in enumerate(plan):
        step_num = i + 1

```

```

sections_remaining = plan[i + 1:]

# ---- 2a. generate ----
if emit:
    await emit({
        "type": "step",
        "step": f"create_page_{step_num}",
        "message": (
            f"Шаг {step_num} из {total}: "
            f"генерирую секцию «{target}»..."
        ),
    })

step_html, step_css = await self._generate_section(
    provider=provider,
    user_message=user_message,
    run_id=run_id,
    history=history,
    target=target,
    step_num=step_num,
    total=total,
    sections_done=sections_done,
    sections_remaining=sections_remaining,
)

if not step_html:
    log(
        provider, "warning",
        f"step {step_num} '{target}' empty after generate"
    )
    continue

# ---- 2b. QA review ----
if self.REVIEW_ENABLED:
    if emit:
        await emit({
            "type": "step",
            "step": f"create_page_review_{step_num}",
            "message": (
                f"Шаг {step_num} из {total}: "
                f"проверяю секцию «{target}»..."
            ),
        })

reviewed_html, reviewed_css = await self._review_section(
    provider=provider,
    user_message=user_message,
    run_id=run_id,
    history=history,
    target=target,
    step_num=step_num,
    total=total,
    section_html=step_html,
    section_css=step_css,
)

```

```

        # Accept the reviewed version only if it parsed AND
        # still looks like the same section (non-empty html).
        if reviewed_html:
            step_html = reviewed_html
            if reviewed_css:
                step_css = reviewed_css
        else:
            log(
                provider, "warning",
                f"step {step_num} '{target}' review failed, "
                f"keeping original"
            )

        # ---- accumulate + emit ----
        sections_done.append(target)
        html_chunks.append(step_html)
        if step_css:
            css_chunks.append(step_css)

        is_last = (step_num == total)

        if emit:
            await emit({
                "type": "page_step",
                "step": step_num,
                "total_hint": total,
                "section_name": target,
                "html": step_html,
                "css": step_css,
                "done": is_last,
            })

        # ---- aggregate ----
        full_html = "\n".join(h for h in html_chunks if h).strip()
        full_css = self.dedupe_imports(
            "\n\n".join(c for c in css_chunks if c).strip()
        )

        if not full_html:
            log(provider, "error", "no sections produced")
            return self._fail(
                "⚠ Модель не сгенерировала ни одной секции. Попробуйте ещё раз."
            )

        full_html = self._ensure_scope(full_html)

        log(
            provider, "info",
            f"create done: {len(sections_done)} sections, "
            f"html={len(full_html)} chars, css={len(full_css)} chars"
        )

        return {
            "message": (
                f"Страница собрана: {len(sections_done)} "
                f"{self._plural_sections(len(sections_done))}."
            )
        }

```

```

        ),
        "html": full_html,
        "css": full_css,
        "selector": None,
        "element_html": None,
        "streamed": True,
    }

# =====
# GENERATE ONE SECTION
# =====

async def _generate_section(
    self,
    *,
    provider,
    user_message: str,
    run_id: Any,
    history: Optional[list],
    target: str,
    step_num: int,
    total: int,
    sections_done: List[str],
    sections_remaining: List[str],
) -> Tuple[str, str]:

    system_prompt = build_section_prompt(
        target_section=target,
        sections_done=sections_done,
        sections_remaining=sections_remaining,
    )
    user_content = build_section_user_message(user_message)

    log(
        provider, "info",
        f"step {step_num}/{total} '{target}' generate: "
        f"system={len(system_prompt)} chars, "
        f"max_tokens={self.STEP_MAX_TOKENS}",
    )

    messages = build_messages(system_prompt, user_content, history)

    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name=f"create_page_section_{step_num}_{target}",
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
    )

    try:
        raw = await provider.generate_completion(
            messages,
            max_tokens=self.STEP_MAX_TOKENS,
        )
    except Exception as e:

```



```

        log(
            provider, "error",
            f"step {step_num} '{target}' LLM error: {e}"
        )
        return ("", "")

    log(
        provider, "info",
        f"step {step_num}/{total} '{target}' response: "
        f"raw_len={len(raw or '')}"
    )

    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name=f"create_page_section_{step_num}_{target}",
        raw_response=raw,
    )

    parsed = self._extract_step(raw)
    step_html = ""
    step_css = ""

    if parsed is not None:
        step_html = (parsed.get("html") or "").strip()
        step_css = (parsed.get("css") or "").strip()

    if not step_html:
        legacy_html, legacy_css = self._extract_html_and_css(raw)
        if legacy_html:
            step_html = legacy_html
            step_css = legacy_css or ""

    if not step_html:
        log(
            provider, "warning",
            f"step {step_num} '{target}' unparsable: "
            f"{str(raw)[:200]}!"
        )
        return ("", "")

    log(
        provider, "info",
        f"step {step_num}/{total} '{target}' parsed: "
        f"html={len(step_html)} chars, css={len(step_css)} chars"
    )

    return (step_html, step_css)

# =====
# REVIEW ONE SECTION
# =====

async def _review_section(
    self,
    *,
    provider,

```

```

user_message: str,
run_id: Any,
history: Optional[list],
target: str,
step_num: int,
total: int,
section_html: str,
section_css: str,
) -> Tuple[str, str]:
    """
    QA pass over ONE generated section.

    Returns (html, css) with fixes applied, or ("", "") if the
    review failed (in which case the caller keeps the original).
    """
    system_prompt = build_review_prompt(target)
    user_content = build_review_user_message(
        target_section=target,
        section_html=section_html,
        section_css=section_css,
        user_request=user_message,
    )

    log(
        provider, "info",
        f"step {step_num}/{total} '{target}' review: "
        f"system={len(system_prompt)} chars, "
        f"user={len(user_content)} chars, "
        f"max_tokens={self.REVIEW_MAX_TOKENS}",
    )

    messages = build_messages(system_prompt, user_content, history)

    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name=f"create_page_review_{step_num}_{target}",
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
    )

    try:
        raw = await provider.generate_completion(
            messages,
            max_tokens=self.REVIEW_MAX_TOKENS,
        )
    except Exception as e:
        log(
            provider, "error",
            f"step {step_num} '{target}' review LLM error: {e}"
        )
        return ("", "")

    log(
        provider, "info",
        f"step {step_num}/{total} review response: "

```

```

        f"raw_len={len(raw or '')}"
    )

    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name=f"create_page_review_{step_num}_{target}",
        raw_response=raw,
    )

    parsed = self._extract_step(raw)
    if parsed is None:
        log(
            provider, "warning",
            f"step {step_num} '{target}' review unparsable: "
            f"{str(raw)[:200]!r}"
        )
        return ("", "")

    reviewed_html = (parsed.get("html") or "").strip()
    reviewed_css = (parsed.get("css") or "").strip()

    if not reviewed_html:
        log(
            provider, "warning",
            f"step {step_num} '{target}' review empty html"
        )
        return ("", "")

    fixed = parsed.get("fixed") or []
    notes = parsed.get("notes") or ""

    log(
        provider, "info",
        f"step {step_num}/{target} review done: "
        f"html={len(reviewed_html)} chars, css={len(reviewed_css)} chars, "
        f"fixed={fixed!r}, notes={notes!r}"
    )

    return (reviewed_html, reviewed_css)

# =====
# LOAD CURRENT CSS
# =====

@staticmethod
async def _load_current_page_css() -> str:
    return ""

# =====
# PHASE 1 – PLAN
# =====

async def _get_plan(
    self,
    *,
    provider,

```

```

        user_message: str,
        run_id: Any,
    ) -> Optional[List[str]]:
        system_prompt = build_plan_prompt()
        user_content = build_plan_user_message(user_message)

        log(
            provider, "info",
            f"plan request: system={len(system_prompt)} chars, "
            f"user={len(user_content)} chars, "
            f"max_tokens={self.PLAN_MAX_TOKENS}",
        )

        CoreEngineLibWordLlmDumper.dump_step(
            run_id=run_id,
            agent_name="create_page_plan",
            system_prompt=system_prompt,
            user_content=user_content,
            history=None,
        )

        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_content},
        ]

        try:
            raw = await provider.generate_completion(
                messages,
                max_tokens=self.PLAN_MAX_TOKENS,
            )
        except Exception as e:
            log(provider, "error", f"plan LLM error: {e}")
            return None

        log(provider, "info", f"plan response: raw_len={len(raw or '')}")

        CoreEngineLibWordLlmDumper.dump_response(
            run_id=run_id,
            agent_name="create_page_plan",
            raw_response=raw,
        )

        plan = self._parse_plan(raw)
        if not plan:
            log(
                provider, "warning",
                f"plan parse failed: {str(raw)[:200]}!"
            )
            return None

        log(provider, "info", f"plan parsed: {plan}")
        return plan

    @staticmethod
    def _parse_plan(text: str) -> Optional[List[str]]:

```

```

if not text:
    return None

s = text.strip()

if s.startswith("`"):
    nl = s.find("\n")
    if nl != -1:
        s = s[nl + 1:]
    if s.endswith("`"):
        s = s[:-3]
    s = s.strip()

data = None

try:
    data = json.loads(s)
except Exception:
    pass

if data is None:
    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        try:
            data = json.loads(s[start:end + 1])
        except Exception:
            data = None

if not isinstance(data, dict):
    return None

raw_sections = data.get("sections") or []
if not isinstance(raw_sections, list):
    return None

name_re = re.compile(r"^[a-z][a-z0-9_]{1,40}$")
cleaned: List[str] = []
seen: set = set()

for item in raw_sections:
    if not isinstance(item, str):
        continue
    name = item.strip().lower()
    name = re.sub(r"[\s\~]+", "_", name)
    if not name_re.match(name):
        continue
    if name in seen:
        continue
    seen.add(name)
    cleaned.append(name)

if len(cleaned) < MIN_SECTIONS:
    return None

return cleaned[:MAX_SECTIONS]

```

```

# =====
# STEP PARSING
# =====

@staticmethod
def _extract_step(text: str) -> Optional[Dict[str, Any]]:
    if not text:
        return None

    s = text.strip()

    if s.startswith("`"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("`"):
            s = s[:-3]
        s = s.strip()

    data = CoreEngineLibWordLlmAgentCreatePage._try_json(s)
    if isinstance(data, dict) and "html" in data:
        return data

    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        chunk = s[start:end + 1]
        data = CoreEngineLibWordLlmAgentCreatePage._try_json(chunk)
        if isinstance(data, dict) and "html" in data:
            return data

    return None

@staticmethod
def _try_json(s: str) -> Optional[Any]:
    try:
        return json.loads(s)
    except Exception:
        return None

# =====
# LEGACY PARSING
# =====

@staticmethod
def _extract_html_and_css(text: str) -> tuple[str, str]:
    if not text:
        return "", ""

    s = text.strip()

    if s.startswith("`"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]

```

```

        if s.endswith("```"):
            s = s[:-3]
        s = s.strip()

    try:
        data = json.loads(s)
        if isinstance(data, dict):
            html = data.get("html") or ""
            css = data.get("css") or ""
            if isinstance(html, str) and html.strip():
                return html.strip(), (css or "").strip()
    except Exception:
        pass

    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        head = s[start:start + 60]
        if '"html"' in head or '"html"' in s[start:end + 1][:200]:
            try:
                data = json.loads(s[start:end + 1])
                if isinstance(data, dict):
                    html = data.get("html") or ""
                    css = data.get("css") or ""
                    if isinstance(html, str) and html.strip():
                        return html.strip(), (css or "").strip()
            except Exception:
                pass

    style_re = re.compile(
        r"<style\b[^\>]*>(.*?)</style>",
        re.DOTALL | re.IGNORECASE,
    )
    css_chunks = [m.group(1).strip() for m in style_re.finditer(s)]
    css = "\n\n".join(c for c in css_chunks if c)
    html = style_re.sub("", s).strip()

    if html.startswith("<") and ">" in html:
        return html, css

    return "", ""

# =====
# POST-PROCESSING
# =====

@staticmethod
def _dedupe_imports(css: str) -> str:
    if not css:
        return css

    seen: set = set()
    out_lines: List[str] = []

    for line in css.splitlines():
        stripped = line.strip()

```

```

        if stripped.startswith("@import"):
            if stripped in seen:
                continue
            seen.add(stripped)
            out_lines.append(line)

    return "\n".join(out_lines)

@staticmethod
def _ensure_scope(html: str) -> str:
    if not html:
        return html
    if "core-engine-lib-word-blocks" in html[:2000]:
        return html

    return (
        '<div class="core-engine-lib-word-blocks">\n'
        f'{html}\n'
        '</div>'
    )

```

```

# =====
# ERROR SHAPE
# =====

```

```

@staticmethod
def _fail(message: str) -> Dict[str, Any]:
    return {
        "message": message,
        "html": None,
        "css": None,
        "selector": None,
        "element_html": None,
        "streamed": True,
    }

```

```

# =====
# SMALL HELPERS
# =====

```

```

@staticmethod
def _plural_sections(n: int) -> str:
    n10 = n % 10
    n100 = n % 100
    if n10 == 1 and n100 != 11:
        return "секция"
    if 2 <= n10 <= 4 and not (12 <= n100 <= 14):
        return "секции"
    return "секций"

```

```

=====
FILE: core/engine/lib/word/llm/agent/create.py
=====

```

```

# neurocad/core/engine/lib/word/llm/agent/create.py

```

```

"""

```


Agent: create.

Build a brand-new page from the block catalog. Runs the existing multi-step flow: plan → fill → effects → svg. Returns the full page HTML, which replaces the canvas.

This is the "default" behaviour of the editor: the user describes what they want, the agent picks blocks, fills them, adds effects, and finally draws SVG illustrations for every placeholder image.

Intermediate events (step, plan, fill_progress, svg_progress) are forwarded to the client through the `emit` callback, so the user can see each stage.

Step 3 (effects)

The effect catalog comes from the SAME source as the palette: `CoreEngineLibWordEffectsService.list\_effects()` reads `word/editor/effects/registry.json` and returns every effect that is really available in the editor. We pass that list into `step\_effects(effect\_catalog=...)`, which forwards it into the effects prompt. As a result, the model only ever sees effects that actually exist – no more inventing `fx-\*` classes that are not in the registry.

Step 4 (svg illustrations)

`step\_svg\_illustrations` walks the assembled HTML and replaces every `` that still points at `placeholder.svg` with a generated inline ``. One request per image, strictly sequential, up to three attempts per image. On failure the placeholder is kept. The image's class and alt are transferred onto the generated svg, so existing CSS for `.image` / `.gallery\_\_item` etc. still applies.

Element wrapping

Blocks from the "Элементы" category (headings, paragraphs, buttons, images, ...) arrive from step 2 as BARE elements (`<h2>...</h2>`, `"""

```
from html.parser import HTMLParser
from typing import Any, Dict, List, Optional, Tuple
```

```
from .base import CoreEngineLibWordLlmAgentBase
```

```
#: Top-level tags that already act as page sections. Anything with
#: one of these as its ROOT tag is left as-is; everything else is
#: wrapped in `
```

```

SECTION_TAGS = {
    "section",
    "article",
    "header",
    "footer",
    "aside",
    "main",
    "nav",
}

#: Wrapper used for bare elements. Matches `core-section` from the
#: block catalog, so styling (padding, max-width, .container) works.
WRAPPER_TAG = "section"
WRAPPER_CLASS = "section"

class _TopLevelScanner(HTMLParser):
    """
    Split an HTML string into top-level node strings.

    The scanner tracks nesting depth. A node starts at the first
    opening tag when depth == 0, and ends when depth returns to 0.

    Text at the top level (whitespace, comments) is ignored – we
    only care about real elements. Anything unexpected (unbalanced
    tags, parse errors) is swallowed and the original string is
    kept intact by the caller.
    """

    def __init__(self):
        super().__init__(convert_charrefs=False)
        self._parts: List[str] = [] # accumulated slices
        self._depth = 0
        self._node_start = -1 # index in the SOURCE string
        self._raw = "" # full source, set later
        self._void_tags = {
            "area", "base", "br", "col", "embed", "hr", "img", "input",
            "link", "meta", "param", "source", "track", "wbr",
        }

    def parse_string(self, html: str) -> List[str]:
        self._raw = html
        self._parts = []
        self._depth = 0
        self._node_start = -1
        try:
            self.feed(html)
            self.close()
        except Exception:
            # On any parser failure – return the whole string as a
            # single "node", so the caller wraps it as one piece.
            return [html] if html.strip() else []

        # Flush any trailing slice.
        if self._node_start != -1 and self._node_start < len(html):
            self._parts.append(html[self._node_start:])

```

```

        return self._parts

# ---- parser callbacks ----

def handle_starttag(self, tag, attrs):
    tag_l = tag.lower()
    if self._depth == 0 and self._node_start == -1:
        self._node_start = self.getpos_offset()
    if tag_l not in self._void_tags:
        self._depth += 1
    else:
        # Void element: if it is the only thing at top level, it
        # is a complete node by itself.
        if self._depth == 0:
            start = self._node_start if self._node_start != -1 else self.getpos_offset()
            end = self.getpos_offset() + self._raw_tag_len()
            self._parts.append(self._raw[start:end])
            self._node_start = -1

def handle_startendtag(self, tag, attrs):
    # Self-closing syntax: `

```

```

        return len(self._raw)

def _raw_tag_len(self) -> int:
    """
    Approximate the length of the tag currently being closed by
    scanning forward from the current position to the next `>`.
    """
    pos = self.getpos_offset()
    gt = self._raw.find(">", pos)
    return (gt - pos + 1) if gt != -1 else 1

class CoreEngineLibWordLlmAgentCreate(CoreEngineLibWordLlmAgentBase):
    """Build a new page from the block catalog."""

    name = "create"

    async def run(
        self,
        *,
        provider,
        user_message: str,
        page_id: int,
        run_id: Any,
        emit=None,
        selection: Optional[Dict[str, Any]] = None,
        block_catalog: Optional[list] = None,
        current_html: Optional[str] = None,
        history: Optional[list] = None,
    ) -> Dict[str, Any]:

        from ..step import CoreEngineLibWordLlmStep

        catalog = block_catalog or []
        history = history or []

        # ---- STEP 1: PLAN ----
        plan: list = []
        async for event in CoreEngineLibWordLlmStep.step_plan(
            user_message, catalog, provider,
            history=history,
            run_id=run_id,
            agent_name="create",
        ):
            if emit:
                await emit(event)

            etype = event.get("type")
            if etype == "error":
                return {
                    "message": f"⚠ {event['message']}",
                    "html": None,
                    "selector": None,
                    "element_html": None,
                }
            if etype == "result":

```

```

        plan = event["plan"]

if not plan:
    return {
        "message": "Не удалось построить план блоков.",
        "html": None,
        "selector": None,
        "element_html": None,
    }

# ---- STEP 2: FILL ----
filled: dict = {}
async for event in CoreEngineLibWordLlmStep.step_fill(
    user_message, plan, catalog, provider,
    history=history,
    run_id=run_id,
    agent_name="create",
):
    if emit:
        await emit(event)

    etype = event.get("type")
    if etype == "error":
        return {
            "message": f"△ {event['message']}",
            "html": None,
            "selector": None,
            "element_html": None,
        }
    if etype == "result":
        filled = event["filled"]

# ---- Assemble ----
parts = []
for item in plan:
    html = filled.get(item["block_id"])
    if html:
        parts.append(html)
assembled_html = "\n".join(parts)

if not assembled_html:
    return {
        "message": "Не удалось заполнить блоки.",
        "html": None,
        "selector": None,
        "element_html": None,
    }

# ---- Wrap bare elements in <section class="section"> ----
# Blocks from the "Элементы" category (and any other source
# that produced a bare tag) get a section wrapper here, so
# nothing ends up floating in <body> without spacing.
assembled_html = self._wrap_bare_elements(assembled_html)

# ---- Load effect catalog (for step 3) ----
effect_catalog = self._load_effect_catalog(provider)

```

```

# ---- STEP 3: EFFECTS ----
after_effects_html = assembled_html
async for event in CoreEngineLibWordLlmStep.step_effects(
    user_message, assembled_html, provider,
    history=history,
    effect_catalog=effect_catalog,
    run_id=run_id,
    agent_name="create",
):
    if emit:
        await emit(event)

    if event.get("type") == "result":
        after_effects_html = event["html"]

# ---- STEP 4: SVG ILLUSTRATIONS ----
# Replace placeholder images with generated inline SVGs –
# one request per image, up to three attempts each. On
# failure the placeholder stays. See step_svg_illustrations
# for the full rules.
final_html = after_effects_html
async for event in CoreEngineLibWordLlmStep.step_svg_illustrations(
    user_message, after_effects_html, provider,
    history=history,
    run_id=run_id,
    agent_name="create",
):
    if emit:
        await emit(event)

    if event.get("type") == "result":
        final_html = event["html"]

return {
    "message": f"Страница собрана из {len(plan)} блоков.",
    "html": final_html,
    "selector": None,
    "element_html": None,
}

# =====
# ELEMENT WRAPPING
# =====

def _wrap_bare_elements(self, html: str) -> str:
    """
    Wrap every top-level node that is NOT already a section-like
    container in `<section class="section">...</section>`.

    Top-level nodes are found by depth-tracking in a small HTML
    parser. On any parse error we return the input unchanged –
    we would rather keep the page as-is than mangle it.

    @param html – assembled page HTML
    @returns html with bare elements wrapped

```

```

"""
if not html or not html.strip():
    return html

try:
    scanner = _TopLevelScanner()
    nodes = scanner.parse_string(html)
except Exception:
    # Fallback: don't touch anything.
    return html

if not nodes:
    return html

wrapped: List[str] = []
for node in nodes:
    if self._is_section_like(node):
        wrapped.append(node)
    else:
        stripped = node.strip()
        if not stripped:
            continue
        wrapped.append(
            f'<{WRAPPER_TAG} class="{WRAPPER_CLASS}">'
            f"{stripped}"
            f"</{WRAPPER_TAG}>"
        )

return "\n".join(wrapped)

@staticmethod
def _is_section_like(node: str) -> bool:
    """
    True if the first tag of `node` is one of SECTION_TAGS.

    `node` may start with whitespace or comments; we skip them
    to find the first `<tag`.
    """
    if not node:
        return False
    s = node.lstrip()
    if not s.startswith("<"):
        return False
    # Skip `<!-- ... -->` comments.
    while s.startswith("<!--"):
        end = s.find("-->")
        if end == -1:
            return False
        s = s[end + 3:].lstrip()
        if not s.startswith("<"):
            return False
    # Extract tag name.
    i = 1
    while i < len(s) and (s[i].isalnum() or s[i] in "-_"):
        i += 1
    tag = s[1:i].lower()

```

```

        return tag in SECTION_TAGS

# =====
# EFFECT CATALOG
# =====

def _load_effect_catalog(self, provider) -> list:
    """
    Load the list of effects that are actually available in the
    palette.

    Source of truth: `word/editor/effects/registry.json`, read
    through `CoreEngineLibWordEffectsService.list_effects()`.
    Returns a list of dicts:

        [{id, label, hint, file, media, builtin, order}, ...]

    If anything fails (service unavailable, JSON parse error,
    file missing), returns an empty list – the effects prompt
    then falls back to its hard-coded set. The create flow is
    NOT aborted: a missing catalog is a soft error, not a fatal
    one.
    """
    try:
        from ...editor.effects.service import (
            CoreEngineLibWordEffectsService,
        )
    except Exception as e:
        self._log_catalog_error(provider, f"import failed: {e}")
        return []

    try:
        effects = CoreEngineLibWordEffectsService.list_effects()
    except Exception as e:
        self._log_catalog_error(provider, f"list_effects failed: {e}")
        return []

    if not isinstance(effects, list):
        self._log_catalog_error(provider, "list_effects returned non-list")
        return []

    return effects

@staticmethod
def _log_catalog_error(provider, message: str) -> None:
    """Best-effort logging – never raises."""
    try:
        log = getattr(provider, "log", None)
        if log is not None:
            fn = getattr(log, "log_warning_sync", None)
            if fn is not None:
                fn(target="create", message=f"effect catalog: {message}")
                return
    except Exception:
        pass
    # Fallback: stdout

```



```
print(f"[create] effect catalog error: {message}", flush=True)
```

```
=====
FILE: core/engine/lib/word/llm/agent/edit.py
=====
```

```
# neurocad/core/engine/lib/word/llm/agent/edit.py
```

```
"""
```

```
Agent: edit.
```

Edits the CSS of ONE existing effect (fx-*) at the user's request.

This agent does NOT touch the page HTML, does NOT add classes to elements, and does NOT pick from a fixed catalog. It writes CSS for a single effect file, replacing its content entirely.

The user is in "effect edit mode": the frontend has already opened the effect for editing and sends the current CSS with every request. The agent returns a new full CSS for the same effect. The frontend applies it live to the iframe (as a <style> override), and only when the user confirms does the frontend PUT it back to the server.

Input:

effect_id	– e.g. "fx-shadow-top-n"
effect_css	– current CSS of that effect (the draft, not the file)
user_message	– what the user wants changed
effect_label	– current human label (optional)
existing_effect_ids	– every id currently registered in the palette (list of strings). The model uses this to avoid proposing an id that is already taken when it decides to rename the effect.

Output:

```
{
  "message": "Готово.",
  "css":      ".core-engine-lib-word-blocks .fx-shadow-top-n { ... }",
  "new_id":   "fx-new-slug" | None,
  "new_label": "Новое название" | None,
}
```

`new\_id` and `new\_label` are OPTIONAL. They are only filled when the edit CHANGES THE MEANING of the effect (e.g. "звёзды" → "радуга"). For cosmetic edits ("сделай тень сильнее") both are None and the client falls back to an in-place PUT.

`new\_id`, when present, must:

- match EFFECT_ID_RE (same pattern as create_effect);
- NOT be present in existing_effect_ids.

If either condition fails, `new\_id` and `new\_label` are dropped and the agent falls back to an in-place edit.

RESPONSE FORMAT TOLERANCE:

The model is ASKED to return JSON {css, new_id, new_label}, but we accept two more shapes as fallbacks:

- (a) raw CSS with the CURRENT id → in-place edit;
- (b) raw CSS with a NEW id → rename (new_id extracted from the CSS selector).

This makes the agent robust to a model that ignores the JSON schema and just returns CSS with a renamed selector. We extract the new id from the selector and validate it against existing_effect_ids, the same as if the model had returned it in a JSON field.

The agent never returns HTML – the element_update path is not used here. The caller (ws.py) sends the result as a separate `css\_update` event, not as `element\_update`.

Safety:

If the model returns something that does not look like CSS for a plausible effect selector, the agent returns css=None. The client then skips the update entirely, so the effect file is not damaged.

```
import json
import re
from typing import Any, Dict, List, Optional, Tuple

from .base import CoreEngineLibWordLlmAgentBase
from ..prompts.edit_effect import build_effect_edit_prompt

#: Same pattern create_effect validates against.
EFFECT_ID_RE = re.compile(r"^(fx-[a-z0-9][a-z0-9-]{0,63})$")

#: Extract the FIRST `fx-<slug>` selector from a CSS string.
#: The slug is a sequence of lowercase letters, digits and dashes.
_CSS_FX_SELECTOR_RE = re.compile(r"\.(fx-[a-z0-9][a-z0-9-]{0,63})(?![\w-])")

class CoreEngineLibWordLlmAgentEdit(CoreEngineLibWordLlmAgentBase):
    """Edit the CSS of one existing effect."""

    name = "edit"

    async def run(
        self,
        *,
        provider,
        user_message: str,
        page_id: int,
        run_id: Any,
        emit=None,
        selection: Optional[Dict[str, Any]] = None,
        block_catalog: Optional[List] = None,
        current_html: Optional[str] = None,
        history: Optional[List] = None,
        effect_id: Optional[str] = None,
        effect_css: Optional[str] = None,
        effect_label: Optional[str] = None,
        existing_effect_ids: Optional[List[str]] = None,
```

```

) -> Dict[str, Any]:

# ---- emit: step at the start ----
if emit:
    await emit({
        "type": "step",
        "step": "edit",
        "message": "Готовлю правку эффекта...",
    })

# ---- guard: effect_id and effect_css are required ----
if not effect_id or not effect_css:
    return {
        "message": "Режим правки эффекта не активен.",
        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
    }

# ---- normalize the list of existing ids ----
existing_ids = [
    str(x).strip()
    for x in (existing_effect_ids or [])
    if isinstance(x, (str, int)) and str(x).strip()
]
existing_ids = [x for x in existing_ids if x != effect_id]

# ---- build system prompt from the current CSS + id + list ----
system_prompt = build_effect_edit_prompt(
    effect_id=effect_id,
    current_css=effect_css,
    effect_label=effect_label or "",
    existing_effect_ids=existing_ids,
)

# ---- user content: just the request ----
user_content = user_message.strip()

# ---- dump prompt ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name="edit",
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
    )
except Exception as e:
    print(f"[edit] dump request failed: {e}", flush=True)

# ---- build messages ----
messages = [{"role": "system", "content": system_prompt}]
if history:
    for m in history:

```

```

        role = m.get("role")
        content = m.get("content")
        if role in ("user", "assistant") and content:
            messages.append({"role": role, "content": str(content)})
messages.append({"role": "user", "content": user_content})

# ---- provider call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[edit] LLM error: {e}", flush=True)
    return {
        "message": f"⚠ Ошибка LLM: {e}",
        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
    }

# ---- dump response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="edit",
        raw_response=raw,
    )
except Exception as e:
    print(f"[edit] dump response failed: {e}", flush=True)

# ---- parse: css + optional new_id / new_label ----
# This handles three shapes:
# 1) JSON {css, new_id, new_label}
# 2) raw CSS with the CURRENT id
# 3) raw CSS with a NEW id (extracted from the selector)
css, parsed_new_id, parsed_new_label = self._parse_response(
    raw, effect_id
)

if not css:
    print(
        f"[edit] model returned a non-css answer: "
        f"{str(raw)[:160]}!", flush=True,
    )
    return {
        "message": "⚠ Модель вернула некорректный ответ. Попробуйте ещё раз.",
        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
    }

# ---- validate optional new_id / new_label ----
new_id, new_label = self._validate_new_id(
    parsed_new_id,

```

```

        parsed_new_label,
        effect_id,
        existing_ids,
    )

    return {
        "message": "Готово.",
        "css": css,
        "effect_id": effect_id,
        "new_id": new_id,
        "new_label": new_label,
    }

# =====
# PARSE
# =====

@classmethod
def _parse_response(
    cls,
    text: str,
    current_id: str,
) -> Tuple[str, Optional[str], Optional[str]]:
    """
    Extract (css, new_id, new_label) from the model response.

    Handles three response shapes:

    A. JSON object with a "css" field (and optional new_id /
       new_label fields) – the shape we ask for.

    B. Raw CSS that uses the CURRENT id in its outer selector
       – no rename. new_id = None.

    C. Raw CSS that uses a NEW id in its outer selector – we
       extract the new id from the selector and return it as
       new_id, so the client can do the rename.

    Returns (css, new_id, new_label). css is "" if nothing usable
    was found.
    """
    if not text:
        return ("", None, None)

    s = text.strip()

    # ---- strip markdown fence ----
    if s.startswith("```"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("```"):
            s = s[:-3]
        s = s.strip()

    # ---- try JSON with a "css" field ----

```

```

try:
    data = json.loads(s)
    if isinstance(data, dict):
        css = data.get("css")
        if css:
            return (
                str(css).strip(),
                data.get("new_id"),
                data.get("new_label"),
            )
except Exception:
    pass

# ---- try the first {...} JSON block ----
start = s.find("{")
end = s.rfind("}")
if start != -1 and end != -1 and end > start:
    head = s[start:start + 40]
    if '"' in head:
        try:
            data = json.loads(s[start:end + 1])
            if isinstance(data, dict):
                css = data.get("css")
                if css:
                    return (
                        str(css).strip(),
                        data.get("new_id"),
                        data.get("new_label"),
                    )
        except Exception:
            pass

# ---- fallback: raw CSS ----
# Verify it looks like CSS: contains braces AND at least one
# `.fx-<slug>` selector.
if "{" not in s or "}" not in s:
    return ("", None, None)

m = _CSS_FX_SELECTOR_RE.search(s)
if not m:
    return ("", None, None)

selector_slug = m.group(1)

# Case B: raw CSS with the CURRENT id → in-place edit.
if selector_slug == current_id:
    return (s, None, None)

# Case C: raw CSS with a NEW id → rename. The new label
# defaults to None (the client falls back to using the new
# id as a label).
return (s, selector_slug, None)

# =====
# VALIDATION
# =====

```

```

@staticmethod
def _validate_new_id(
    raw_new_id: Any,
    raw_new_label: Any,
    current_id: str,
    existing_ids: List[str],
) -> Tuple[Optional[str], Optional[str]]:
    """
    Sanitize the optional new id / label proposed by the model.

    Returns (new_id, new_label), or (None, None) if:
    - new_id is missing or empty;
    - new_id equals current_id (no actual rename);
    - new_id does not match EFFECT_ID_RE;
    - new_id is in existing_ids (would collide with another
      effect already registered in the palette).

    In every rejection case the caller falls back to an in-place
    PUT – the user keeps their current effect name.
    """
    if not raw_new_id:
        return (None, None)

    new_id = str(raw_new_id).strip()
    if not new_id:
        return (None, None)

    if new_id == current_id:
        print(
            f"[edit] model proposed the same id as current "
            f"({new_id}) – ignoring rename",
            flush=True,
        )
        return (None, None)

    if not EFFECT_ID_RE.match(new_id):
        print(
            f"[edit] model proposed an invalid new_id "
            f"({new_id!r}) – ignoring rename",
            flush=True,
        )
        return (None, None)

    if new_id in existing_ids:
        print(
            f"[edit] model proposed an already-taken new_id "
            f"({new_id!r}) – ignoring rename. existing: {existing_ids}",
            flush=True,
        )
        return (None, None)

    new_label = (str(raw_new_label).strip() if raw_new_label else "")
    if not new_label:
        # No label from the model – fall back to the new id so
        # the palette still has something readable.

```

```

        new_label = new_id
    if len(new_label) > 80:
        new_label = new_label[:77] + "..."

    return (new_id, new_label)

```

```

=====
FILE: core/engine/lib/word/llm/agent/effect.py
=====

```

```

# neurocad/core/engine/lib/word/llm/agent/effect.py

```

```

"""

```

```

Agent: effect.

```

Two responsibilities, chosen by the user's request:

EFFECT MODE (default)

Add visual effects to ONE selected element on the canvas:
backgrounds, gradients, animations, shadows, hover states. Any
visual change that lives in a <style> block goes here – static
background and animation alike.

IMAGE MODE (triggered by "нарисуй картинку" / "сгенерируй иллюстрацию")

Replace a placeholder inside the selected element with a
generated inline <svg>. One LLM request, up to three attempts.
The 's class and alt are transferred onto the generated svg,
so existing CSS keeps applying. If the selection itself is the
 (the marker is on it), the marker is transferred to the
new <svg>.

The selected element is whatever the user clicked in the editor – a
whole block (<section>) or any inner node (<div>, <h1>, <a>,).

Returns:

```

{
    "selector":      "sel-abc12345",  # the marker from selection
    "element_html": "<section ...>...</section>",
    "message":      "Эффект добавлен." | "Иллюстрация создана.",
}

```

The client replaces the element with `data-selected-id="sel-abc12345"`
by the new HTML.

Marker integrity:

The selection marker `data-selected-id="sel-..."` is the ONLY way
the client can find the element back on the canvas. The model is
asked to preserve it verbatim (rule 2 in EFFECT_SYSTEM_PROMPT), but
models drift – they drop the attribute, change its value, move it
to another tag, or insert extra whitespace.

To defend against that, this agent does TWO things after parsing:

1. If the expected marker is missing from the parsed HTML, the
whole response is REJECTED (selector=None) – the client skips
the update, the page is not damaged.

2. If a `data-selected-id` attribute is present but its value is NOT the expected selector, the response is also REJECTED.
Silent substitution is worse than a clear error: if we rewrote the value, the client would replace a DIFFERENT element than the one the user had selected.

IMAGE MODE has the same guarantees: after the `<img> → <svg>` replacement we re-check the marker. If the marker was on the `<img>` itself, it is moved onto the new `<svg>`; if it was on an ancestor, it must still be there unchanged.

Intermediate events: a single `step` event is emitted at the start, then the LLM answer is returned via the standard result shape.

Safety:

If the model returns something that does not look like HTML (an error message, an explanation, an empty string, a bare JSON object without the "html" field), the agent returns `selector=None` and `element_html=None`. The client will then skip the update entirely, so the page is not damaged by a bad response.

"""

```
import json
import re
from typing import Any, Dict, Optional, Tuple

from .base import CoreEngineLibWordLlmAgentBase
from ..prompts.svg import build_svg_illustration_prompt

#: Matches `data-selected-id="<value>"` in either single or double
#: quotes. The value can contain latin letters, digits, dashes and
#: underscores – everything we generate as a marker.
_SELECTED_ID_RE = re.compile(
    r""data-selected-id\s*=\s*(?:"([^\"]*)"|'([^']*)'|'')""",
    re.IGNORECASE,
)

#: Matches a full `<img ...>` tag (self-closing or not).
_IMG_TAG_RE = re.compile(
    r"<img\b[^\>]*?>",
    re.IGNORECASE | re.DOTALL,
)

#: Attribute extractor inside a raw tag string.
_ATTR_RE = re.compile(
    r""\b(?:P<name>[a-zA-Z_-][a-zA-Z0-9_-]*)\s*=\s*(?:"(?P<dq>[^\"]*)"|'(?P<sq>[^\']*)'|'')""",
    re.DOTALL,
)

#: Max attempts for one image.
_MAX_SVG_ATTEMPTS = 3

#: Default viewBox if the model did not provide one.
_DEFAULT_VIEWBOX = "0 0 800 600"
```

#: Keywords that select IMAGE MODE instead of EFFECT MODE.
#: Case-insensitive substring match against the user message.

```
_IMAGE_INTENT_KEYWORDS = (  
    "нарисуй",  
    "нарисовать",  
    "сгенерируй картинку",  
    "сгенерируй изображение",  
    "сгенерируй иллюстрацию",  
    "сделай картинку",  
    "сделай изображение",  
    "сделай иллюстрацию",  
    "картинку",  
    "изображение",  
    "иллюстрацию",  
    "иллюстрация",  
    "svg",  
)
```

```
class CoreEngineLibWordLlmAgentEffect(CoreEngineLibWordLlmAgentBase):  
    """Add visual effects OR generate an SVG for one selected element."""
```

```
    name = "effect"
```

```
    # =====  
    # EFFECT MODE – system prompt  
    # =====
```

```
    EFFECT_SYSTEM_PROMPT = """Ты – дизайнер для CMS NeuroCad.
```

Тебе дают ОДИН HTML-элемент (это может быть секция, div, заголовок,
кнопка – любой узел) и запрос пользователя.

Ты добавляешь к нему визуальные эффекты.

ФОРМАТ ОТВЕТА – ТОЛЬКО ВАЛИДНЫЙ JSON:

```
{  
    "selector": "sel-abc12345",  
    "html": "<section ...>...</section>\\n<style>.effect-xxxx { ... }</style>"  
}
```

ПРАВИЛА:

1. НЕ меняй структуру HTML и классы. Только добавляй эффекты.
2. Корневой тег и все его атрибуты – СОХРАНИ как есть, включая `data-selected-id`. ЗНАЧЕНИЕ `data-selected-id` ДОЛЖНО остаться РОВНО таким, как во входе, символ в символ.
3. Для эффектов используй <style>-блок:
 - сгенерируй уникальный префикс, например effect-a3f7;
 - добавь его как класс к корневому элементу;
 - в конце блока вставь <style>, каждое правило начинается с этого префикса.
4. Внутри <style> используй var(--theme-*) для цветов.
5. Можно: @keyframes, linear-gradient, radial-gradient, conic-gradient, transform, box-shadow, transition, filter, backdrop-filter, mask.
6. Не добавляй inline-стили (style="...").
7. Поле "selector" – ровно то, что было во входе.
8. Поле "html" – ТОЛЬКО HTML этого элемента + <style>.

9. Не добавляй пояснения до или после JSON.
10. Не оборачивай в markdown.

ГЛАВНОЕ ПРАВИЛО – ДЕЛАЙ РОВНО ТО, ЧТО ПРОСЯТ:

11. Не расширяй задачу. «Закрась жёлтым» = ОДНО правило
background-color: <жёлтый>. НЕ градиент, НЕ анимация,
НЕ ::before, НЕ box-shadow, НЕ hover.
12. «Сделай градиент» = linear-gradient или radial-gradient,
и точка. Без анимации, без свечения.
13. «Добавь анимацию» = @keyframes + animation. Без смены фона,
без hover.
14. Если пользователь не сказал «сделай красиво», «добавь эффектов»,
«укрась» – НЕ добавляй больше ОДНОГО визуального приёма.
15. Простой цвет – это background-color: #xxxxxx.
НЕ background-image. НЕ градиент.
16. Если не уверен – сделай минимальный вариант.

```

async def run(
    self,
    *,
    provider,
    user_message: str,
    page_id: int,
    run_id: Any,
    emit=None,
    selection: Optional[Dict[str, Any]] = None,
    block_catalog: Optional[list] = None,
    current_html: Optional[str] = None,
    history: Optional[list] = None,
) -> Dict[str, Any]:

    selection = selection or {}
    selector = selection.get("selector")
    target_html = selection.get("outer_html")

    # ---- IMAGE MODE ----
    # If the user asked for a picture and there is a selection,
    # we go straight into SVG generation. No <style> prompt, no
    # effects.
    if self._is_image_intent(user_message):
        if emit:
            await emit({
                "type": "step",
                "step": "image",
                "message": "Рисую иллюстрацию...",
            })

        if not selector or not target_html:
            return {
                "message": "Сначала выделите картинку или блок с картинкой.",
                "html": None,
                "selector": None,
                "element_html": None,
            }

```

```

        return await self._run_image_mode(
            provider=provider,
            user_message=user_message,
            selector=selector,
            target_html=target_html,
            history=history,
            run_id=run_id,
            agent_name="effect-image",
        )

# ---- EFFECT MODE (default) ----
if emit:
    await emit({
        "type": "step",
        "step": "effect",
        "message": "Добавляю эффекты...",
    })

if not selector or not target_html:
    return {
        "message": "Не выделен элемент. Кликните по элементу на странице и повторите.",
        "html": None,
        "selector": None,
        "element_html": None,
    }

user_content = (
    f"Запрос пользователя: {user_message}\n\n"
    f"selector: {selector}\n\n"
    f"HTML элемента:\n{target_html}"
)

# ---- dump prompt ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name="effect",
        system_prompt=self.EFFECT_SYSTEM_PROMPT,
        user_content=user_content,
        history=history,
    )
except Exception as e:
    print(f"[effect] dump request failed: {e}", flush=True)

# ---- build messages ----
messages = [{"role": "system", "content": self.EFFECT_SYSTEM_PROMPT}]
if history:
    for m in history:
        role = m.get("role")
        content = m.get("content")
        if role in ("user", "assistant") and content:
            messages.append({"role": role, "content": str(content)})
messages.append({"role": "user", "content": user_content})

# ---- provider call ----

```

```

try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[effect] LLM error: {e}", flush=True)
    return {
        "message": f"⚠ Ошибка LLM: {e}",
        "html": None,
        "selector": None,
        "element_html": None,
    }

# ---- dump response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="effect",
        raw_response=raw,
    )
except Exception as e:
    print(f"[effect] dump response failed: {e}", flush=True)

# ---- parse ----
new_html = self._parse(raw, expected_selector=selector)

# ---- safety: html must look like html AND keep the marker ----
if not self._looks_like_html(new_html):
    print(
        f"[effect] model returned a non-html answer: "
        f"{new_html[:120]!r}",
        flush=True,
    )
    return {
        "message": "⚠ Модель вернула некорректный ответ. Попробуйте ещё раз.",
        "html": None,
        "selector": None,
        "element_html": None,
    }

# ---- safety: the marker must be EXACTLY the expected one ----
marker_ok, found_marker = self._check_marker(new_html, selector)
if not marker_ok:
    print(
        f"[effect] marker mismatch: expected {selector!r}, "
        f"found {found_marker!r} – refusing to send element_update",
        flush=True,
    )
    return {
        "message": (
            "⚠ Модель потеряла маркер выделенного элемента. "
            "Попробуйте ещё раз или выделите элемент заново."
        ),
        "html": None,
        "selector": None,
        "element_html": None,
    }
}

```

```

    return {
        "message": "Эффект добавлен.",
        "html": None,
        "selector": selector,
        "element_html": new_html,
    }

# =====
# IMAGE MODE
# =====

async def _run_image_mode(
    self,
    *,
    provider,
    user_message: str,
    selector: str,
    target_html: str,
    history: Optional[list],
    run_id: Any,
    agent_name: str,
) -> Dict[str, Any]:
    """
    Replace a placeholder <img> inside the selected element with
    a generated inline <svg>.

    Handles two shapes of the selection:
    - the selection IS the <img> (marker on the img) – the
      marker is moved onto the new <svg>;
    - the selection is an ancestor (marker on the section) –
      the first <img> inside is replaced, marker stays where it
      is.

    On failure (no <img> in the selection, or all attempts fail),
    the original HTML is returned untouched and the message
    explains what happened.
    """
    # ---- find the first <img> in the selected html ----
    img_match = _IMG_TAG_RE.search(target_html)
    if not img_match:
        return {
            "message": "В выделенном элементе нет картинки для замены.",
            "html": None,
            "selector": None,
            "element_html": None,
        }

    img_tag = img_match.group(0)
    alt = self._attr(img_tag, "alt") or user_message or "иллюстрация"
    css_class = self._attr(img_tag, "class") or ""

    # Does the marker live ON the <img> itself?
    marker_on_img = self._check_marker(img_tag, selector)[0]

    # ---- attempt up to N times ----

```

```

svg = None
last_reason = ""
for attempt in range(1, _MAX_SVG_ATTEMPTS + 1):
    svg, reason = await self._try_generate_svg(
        provider=provider,
        user_message=user_message,
        alt=alt,
        history=history,
        run_id=run_id,
        agent_name=agent_name,
        attempt=attempt,
    )
    if svg:
        break
    last_reason = reason or "invalid response"

if not svg:
    print(
        f"[effect-image] gave up after {_MAX_SVG_ATTEMPTS} attempts: "
        f"{last_reason}",
        flush=True,
    )
    return {
        "message": "⚠ Не удалось нарисовать иллюстрацию. Попробуйте ещё раз.",
        "html": None,
        "selector": None,
        "element_html": None,
    }

# ---- merge class/alt/marker onto the svg ----
svg = self._merge_attrs_into_svg(
    svg,
    alt=alt,
    css_class=css_class,
    marker=selector if marker_on_img else "",
)

# ---- build the new element html ----
# Replace ONLY the first <img> occurrence.
new_html = (
    target_html[:img_match.start()]
    + svg
    + target_html[img_match.end():]
)

# ---- safety: the marker must survive ----
marker_ok, found_marker = self._check_marker(new_html, selector)
if not marker_ok:
    print(
        f"[effect-image] marker mismatch: expected {selector!r}, "
        f"found {found_marker!r} – refusing to send element_update",
        flush=True,
    )
    return {
        "message": (
            "⚠ Потерян маркер выделенного элемента. "

```

```

        "Попробуйте ещё раз или выделите элемент заново."
    ),
    "html": None,
    "selector": None,
    "element_html": None,
}

return {
    "message": "Иллюстрация создана.",
    "html": None,
    "selector": selector,
    "element_html": new_html,
}

async def _try_generate_svg(
    self,
    *,
    provider,
    user_message: str,
    alt: str,
    history: Optional[list],
    run_id: Any,
    agent_name: str,
    attempt: int,
) -> Tuple[Optional[str], str]:
    """
    One attempt: ask the LLM for an SVG for this alt text.

    Returns (svg, reason). `svg` is a cleaned `<svg>...</svg>`
    string, or None. `reason` explains why on failure (empty on
    success).
    """
    system_prompt = build_svg_illustration_prompt()
    user_content = (
        f"Общий запрос страницы: {user_message}\n\n"
        f"Alt изображения: {alt}\n\n"
        f"Попытка: {attempt}"
    )

    messages = [{"role": "system", "content": system_prompt}]
    if history:
        for m in history:
            role = m.get("role")
            content = m.get("content")
            if role in ("user", "assistant") and content:
                messages.append({"role": role, "content": str(content)})
    messages.append({"role": "user", "content": user_content})

    # ---- dump ----
    try:
        from ..dumper import CoreEngineLibWordLlmDumper
        CoreEngineLibWordLlmDumper.dump_step(
            run_id=run_id,
            agent_name=agent_name,
            system_prompt=system_prompt,
            user_content=user_content,

```



```

        history=history,
    )
except Exception:
    pass

# ---- call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[effect-image] LLM error: {e}", flush=True)
    return (None, f"LLM error: {e}")

# ---- dump ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name=agent_name,
        raw_response=raw,
    )
except Exception:
    pass

# ---- extract + validate ----
svg = self._extract_svg(raw)
if not svg:
    return (None, "no <svg> in response")

ok, reason = self._validate_svg(svg)
if not ok:
    return (None, reason)

return (svg, "")

# =====
# INTENT
# =====

@staticmethod
def _is_image_intent(user_message: str) -> bool:
    """
    True if the user's request looks like "generate a picture".

    The check is a case-insensitive substring match against a
    short keyword list. If the user writes both "эффект" and
    "картинка" – we treat it as an image request, because the
    word "картинка" is far more specific than "эффект".
    """
    if not user_message:
        return False
    low = user_message.lower()
    return any(k in low for k in _IMAGE_INTENT_KEYWORDS)

# =====
# PARSE
# =====

```

```

@staticmethod
def _parse(text: str, expected_selector: str = "") -> str:
    """
    Extract the `html` field from the model response.

    Tolerates markdown wrappers and surrounding text. If the model
    returned JSON without `html`, falls back to the whole text.

    `expected_selector` is currently unused here – the marker
    integrity check happens later in `_check_marker`. The arg is
    kept for symmetry and future use.
    """
    if not text:
        return ""

    s = text.strip()

    # strip markdown fence
    if s.startswith("```"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("```"):
            s = s[:-3]
        s = s.strip()

    # try the whole string
    try:
        data = json.loads(s)
        if isinstance(data, dict):
            html = data.get("html", "")
            if html:
                return str(html).strip()
    except Exception:
        pass

    # try the first {...} block
    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        try:
            data = json.loads(s[start:end + 1])
            if isinstance(data, dict):
                html = data.get("html", "")
                if html:
                    return str(html).strip()
        except Exception:
            pass

    # fallback: treat the whole response as HTML
    return s

# =====
# SVG HELPERS
# =====

```

```

@staticmethod
def _extract_svg(text: str) -> Optional[str]:
    """
    Pull the first `<svg>...</svg>` out of the model response.

    Tolerates markdown fences and surrounding prose. If the
    response is JSON with an "svg" field, that is preferred.
    """
    if not text:
        return None

    s = text.strip()

    if s.startswith("`"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("`"):
            s = s[:-3]
        s = s.strip()

    # JSON with an "svg" field.
    data = CoreEngineLibWordLlmAgentEffect._extract_json(s)
    if isinstance(data, dict):
        svg_field = data.get("svg")
        if isinstance(svg_field, str) and svg_field.strip():
            return svg_field.strip()

    start = s.lower().find("<svg")
    if start == -1:
        return None
    end = s.lower().rfind("</svg>")
    if end == -1 or end <= start:
        return None
    return s[start:end + len("</svg>")].strip()

@staticmethod
def _validate_svg(svg: str) -> Tuple[bool, str]:
    """
    Cheap sanity check on the extracted SVG.
    """
    if not svg:
        return (False, "empty")

    low = svg.lower()

    if not low.startswith("<svg"):
        return (False, "does not start with <svg")

    if "</svg>" not in low:
        return (False, "no closing </svg>")

    if "<script" in low:
        return (False, "contains <script>")

```

```

    if "javascript:" in low:
        return (False, "contains javascript:")

    if re.search(r"\bon[a-z]+\s*=", low):
        return (False, "contains event handler")

    if re.search(r'""(?:href|xlink:href)\s*=\s*["']\s*(?:https?:)?//""', low):
        return (False, "contains external href")

    return (True, "")

@staticmethod
def _merge_attrs_into_svg(
    svg: str,
    *,
    alt: str,
    css_class: str,
    marker: str,
) -> str:
    """
    Put `class`, `alt` (as aria-label) and – if the marker was on
    the replaced <img> – the `data-selected-id` marker onto the
    root <svg>.
    """
    if not svg:
        return svg

    m = re.match(r"(<svg\b)([>]*?)(>|/>)", svg, flags=re.IGNORECASE | re.DOTALL)
    if not m:
        return svg

    head = m.group(1)
    attrs = m.group(2) or ""
    close = m.group(3) or ">"
    rest = svg[m.end():]

    if "viewBox" not in attrs and "viewbox" not in attrs.lower():
        attrs = f' viewBox="{_DEFAULT_VIEWBOX}"' + attrs

    if css_class and "class" not in attrs.lower():
        attrs += f' class="{css_class}"'

    if "role=" not in attrs.lower():
        attrs += ' role="img"'

    if alt and "aria-label=" not in attrs.lower():
        safe_alt = alt.replace("'", "&quot;")
        attrs += f' aria-label="{safe_alt}"'

    if marker and "data-selected-id=" not in attrs.lower():
        attrs += f' data-selected-id="{marker}"'

    return f"{head}{attrs}{close}{rest}"

@staticmethod
def _attr(tag: str, name: str) -> Optional[str]:
    """

```

```

Extract an attribute value from a raw tag string. Case-
insensitive, handles double- and single-quoted values.
"""
if not tag:
    return None
target = name.lower()
for m in _ATTR_RE.finditer(tag):
    if m.group("name").lower() == target:
        return m.group("dq") if m.group("dq") is not None else m.group("sq")
return None

@staticmethod
def _extract_json(text: str) -> Optional[dict]:
    if not text:
        return None
    s = text.strip()
    if s.startswith("`"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("`"):
            s = s[:-3]
        s = s.strip()
    try:
        data = json.loads(s)
        if isinstance(data, dict):
            return data
    except json.JSONDecodeError:
        pass
    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        try:
            data = json.loads(s[start:end + 1])
            if isinstance(data, dict):
                return data
        except json.JSONDecodeError:
            pass
    return None

# =====
# SAFETY
# =====

@staticmethod
def _looks_like_html(text: str) -> bool:
    """
    Cheap sanity check that the model returned actual HTML and not
    an error message, an explanation, or an empty string.
    """
    if not text:
        return False
    s = str(text).strip()
    if not s:
        return False
    if s.startswith("△"):

```

```

        return False
    if not s.startswith("<"):
        return False
    if ">" not in s:
        return False
    head = s[:60].lower()
    for bad in ("ошибка", "error", "ошибк", "превышено"):
        if bad in head:
            return False
    if "data-selected-id" not in s:
        return False
    return True

```

```

@staticmethod
def _check_marker(html: str, expected: str) -> Tuple[bool, Optional[str]]:
    """
    Verify that `data-selected-id` in `html` equals `expected`.
    """
    if not html or not expected:
        return (False, None)

    matches = _SELECTED_ID_RE.findall(html)
    if not matches:
        return (False, None)

    found_values = [(a or b).strip() for (a, b) in matches]
    first = found_values[0] if found_values else None

    for v in found_values:
        if v == expected:
            return (True, v)

    return (False, first)

```

```

=====
FILE: core/engine/lib/word/llm/agent/fill.py
=====

```

```

# neurocad/core/engine/lib/word/llm/agent/fill.py

```

```

"""

```

```

Agent: fill.

```

Fill ONE selected element with text, alt's and links. The selected element is whatever the user clicked in the editor – a whole block (<section>) or any inner node (<div>, <h1>, <p>, <a>, ...).

Returns:

```

{
    "selector":      "sel-abc12345",  # the marker from selection
    "element_html": "<section ...>...</section>",
    "message":      "Элемент заполнен.",
}

```

The client replaces the element with `data-selected-id="sel-abc12345"` by the new HTML.

Intermediate events: a single `step` event is emitted at the start ("Заполняю..."), then the LLM answer is returned via the standard result shape.

```
from typing import Any, Dict, Optional

from .base import CoreEngineLibWordLlmAgentBase

class CoreEngineLibWordLlmAgentFill(CoreEngineLibWordLlmAgentBase):
    """Fill one selected element with content."""

    name = "fill"

    SYSTEM_PROMPT = """Ты — редактор контента для CMS NeuroCad.

Тебе дают ОДИН HTML-элемент (секция, div, заголовок, абзац, ссылка —
любой узел) и запрос пользователя.
Ты заполняешь этот элемент текстом, alt'ами и ссылками.

ФОРМАТ ОТВЕТА — ТОЛЬКО ВАЛИДНЫЙ JSON:
{
  "selector": "sel-abc12345",
  "html": "<section ...>...</section>"
}

ПРАВИЛА:
1. НЕ меняй структуру HTML и классы. Только тексты, alt, href.
2. Корневой тег и все его атрибуты — СОХРАНИ как есть, включая
   `data-selected-id`.
3. Поле "selector" — ровно то, что было во входе.
4. Поле "html" — ТОЛЬКО HTML этого элемента, без <html>, <head>, <body>.
5. Не добавляй inline-стили (style="...") и не вставляй <style>.
6. Сохраняй все существующие классы и вложенность.
7. Тексты — на русском, в нейтрально-деловом тоне.
8. Не добавляй пояснения до или после JSON.
9. Не оборачивай в markdown.
10. Если запрос не про заполнение контентом — верни элемент без
    изменений.
"""

    async def run(
        self,
        *,
        provider,
        user_message: str,
        page_id: int,
        run_id: Any,
        emit=None,
        selection: Optional[Dict[str, Any]] = None,
        block_catalog: Optional[list] = None,
        current_html: Optional[str] = None,
        history: Optional[list] = None,
    ) -> Dict[str, Any]:
```

```

# ---- emit: step at the start ----
if emit:
    await emit({
        "type": "step",
        "step": "fill",
        "message": "Заполняю текстом...",
    })

selection = selection or {}
selector = selection.get("selector")
target_html = selection.get("outer_html")

if not selector or not target_html:
    return {
        "message": "Не выделен элемент. Кликните по элементу на странице и повторите.",
        "html": None,
        "selector": None,
        "element_html": None,
    }

user_content = (
    f"Запрос пользователя: {user_message}\n\n"
    f"selector: {selector}\n\n"
    f"HTML элемента:\n{target_html}"
)

# ---- dump prompt ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name="fill",
        system_prompt=self.SYSTEM_PROMPT,
        user_content=user_content,
        history=history,
    )
except Exception as e:
    print(f"[fill] dump request failed: {e}", flush=True)

# ---- build messages ----
messages = [{"role": "system", "content": self.SYSTEM_PROMPT}]
if history:
    for m in history:
        role = m.get("role")
        content = m.get("content")
        if role in ("user", "assistant") and content:
            messages.append({"role": role, "content": str(content)})
messages.append({"role": "user", "content": user_content})

# ---- provider call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[fill] LLM error: {e}", flush=True)
    return {
        "message": f"⚠ Ошибка LLM: {e}",
    }

```



```

        "html": None,
        "selector": None,
        "element_html": None,
    }

# ---- dump response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="fill",
        raw_response=raw,
    )
except Exception as e:
    print(f"[fill] dump response failed: {e}", flush=True)

# ---- parse ----
new_html = self._parse(raw)
if not new_html:
    return {
        "message": "Не удалось разобрать ответ модели.",
        "html": None,
        "selector": None,
        "element_html": None,
    }

    return {
        "message": "Элемент заполнен.",
        "html": None,
        "selector": selector,
        "element_html": new_html,
    }

# =====
# PARSE
# =====

@staticmethod
def _parse(text: str) -> str:
    """
    Extract the `html` field from the model response.

    Tolerates markdown wrappers and surrounding text. If the model
    returned JSON without `html`, falls back to the whole text.
    """
    import json

    if not text:
        return ""

    s = text.strip()

    # strip markdown fence
    if s.startswith("```"):
        nl = s.find("\n")
        if nl != -1:

```

```

        s = s[nl + 1:]
        if s.endswith("```"):
            s = s[:-3]
        s = s.strip()

# try the whole string
try:
    data = json.loads(s)
    if isinstance(data, dict):
        html = data.get("html", "")
        if html:
            return str(html).strip()
except Exception:
    pass

# try the first {...} block
start = s.find("{")
end = s.rfind("}")
if start != -1 and end != -1 and end > start:
    try:
        data = json.loads(s[start:end + 1])
        if isinstance(data, dict):
            html = data.get("html", "")
            if html:
                return str(html).strip()
    except Exception:
        pass

# fallback: treat the whole response as HTML
return s

```

```

=====
FILE: core/engine/lib/word/llm/agent/generate_logo.py
=====

```

```

# neurocad/core/engine/lib/word/llm/agent/generate_logo.py

```

```

"""

```

```

Agent: generate_logo.

```

Generate ONE SVG logo from a page title + description. Thin wrapper around the LLM: build the prompt, call the provider, extract and validate the SVG. Nothing is saved to disk here – the caller (the HTTP endpoint POST /editor/images/generate) does that through CoreEngineLibWordImagesService.

This agent is NOT registered in _AGENTS and is NOT called by the WebSocket dispatcher. It is invoked directly from the HTTP endpoint, because the logo flow is not part of the create / fill / effects / svg pipeline.

Retries

```

-----

```

The LLM call is retried on transient failures – CloudFront / CDN throttling (HTTP 403), rate limits (429), upstream errors (5xx), and network timeouts. Backoff is exponential: BASE_DELAY, then BASE_DELAY * 2. A clean response that simply lacks <svg> is NOT

retried – the model refused or produced invalid output, and a second attempt is unlikely to change that.

Return shape:

```
{ "svg": "<svg ...></svg>" }      - on success
{ "svg": None, "error": "<reason>" }  - on failure
"""
```

```
import asyncio
from typing import Any, Dict, Optional

from .base import CoreEngineLibWordLlmAgentBase
from ..prompts.logo import build_logo_prompt
from ..steps.common import (
    build_messages,
    extract_svg,
    log,
    validate_svg,
)

# =====
# RETRY
# =====

#: How many times to attempt the LLM call (1 = no retry).
MAX_ATTEMPTS = 3

#: Base delay for exponential backoff (seconds).
#: Attempt 1 fails → wait BASE_DELAY.
#: Attempt 2 fails → wait BASE_DELAY * 2.
BASE_DELAY = 2.0

#: Substrings in the error text that mark a transient failure
#: worth retrying. Matched against the whole error string.
#: 403 – CloudFront / CDN throttling (DeepSeek sometimes)
#: 429 – rate limit
#: 5xx – upstream server errors
#: timeout / timed out – network, no response
RETRY_MARKERS = (
    "403",
    "429",
    "500",
    "502",
    "503",
    "504",
    "timeout",
    "timed out",
    "Timeout",
)

class CoreEngineLibWordLlmAgentGenerateLogo(CoreEngineLibWordLlmAgentBase):
    """Generate one SVG logo from a title + description."""

    name = "generate_logo"
```

```

async def run(
    self,
    *,
    provider,
    user_message: str,
    page_id: int,
    run_id: Any,
    emit=None,
    selection: Optional[Dict[str, Any]] = None,
    block_catalog: Optional[list] = None,
    current_html: Optional[str] = None,
    history: Optional[list] = None,
    alt: str = "",
) -> Dict[str, Any]:
    """
    Generate one SVG logo.

    `user_message` – the prompt sent by the caller (usually
    "Заголовок: ... Описание: ..."). `alt` – an optional hint
    repeated to the model for context.

    Returns {"svg": str} on success, or {"svg": None, "error": str}.
    """
    system_prompt = build_logo_prompt()

    user_content = user_message.strip()
    if alt and alt.strip():
        user_content = f"{user_content}\n\nAlt: {alt.strip()}"

    messages = build_messages(system_prompt, user_content, history or [])

    # =====
    # LLM call with retry on transient failures.
    #
    # Attempts: MAX_ATTEMPTS (default 3).
    # Backoff:  BASE_DELAY * 2^(n-1) – 2s, 4s.
    # Retry:    only when the error text contains one of
    #           RETRY_MARKERS. A clean "no <svg> in response"
    #           from a working model is NOT retried.
    # =====

    last_error: str = ""
    svg: Optional[str] = None
    attempt: int = 0

    for attempt in range(1, MAX_ATTEMPTS + 1):
        try:
            raw = await provider.generate_completion(messages)
        except Exception as e:
            last_error = f"LLM error: {e}"
            log(provider, "warning",
                f"generate_logo attempt {attempt}/{MAX_ATTEMPTS} "
                f"failed: {last_error}")
        else:
            svg = extract_svg(raw)

```

```

        if svg:
            break
        last_error = "no <svg> found in response"
        log(provider, "warning",
            f"generate_logo attempt {attempt}/{MAX_ATTEMPTS}: "
            f"no <svg> in response")

    # Retry only on transient failures.
    if not any(marker in last_error for marker in RETRY_MARKERS):
        break

    if attempt < MAX_ATTEMPTS:
        delay = BASE_DELAY * (2 ** (attempt - 1))
        log(provider, "info",
            f"generate_logo: retry in {delay}s")
        await asyncio.sleep(delay)

    if not svg:
        log(provider, "error",
            f"generate_logo failed after {attempt} attempt(s): "
            f"{last_error}")
        return {"svg": None, "error": last_error or "no <svg> in response"}

    ok, reason = validate_svg(svg)
    if not ok:
        log(provider, "warning", f"generate_logo rejected: {reason}")
        return {"svg": None, "error": f"validation failed: {reason}"}

    log(provider, "info", "generate_logo: ok")
    return {"svg": svg}

```

```

=====
FILE: core/engine/lib/word/llm/agent/help.py
=====

```

```

# neurocad/core/engine/lib/word/llm/agent/help.py

```

```

"""

```

```

Agent: help.

```

```

Answers questions about the NeuroCad editor only. Off-topic questions
are refused. Uses a small FAQ + rules block – no vector search yet.

```

```

No page modifications: returns html=None and selector=None.
"""

```

```

from typing import Any, Dict, Optional

```

```

from .base import CoreEngineLibWordLlmAgentBase

```

```

class CoreEngineLibWordLlmAgentHelp(CoreEngineLibWordLlmAgentBase):
    """Answer NeuroCad questions from a static FAQ + rules block."""

```

```

    name = "help"

```

```

    FAQ = """ДОКУМЕНТАЦИЯ NEUROCAD:

```

РЕДАКТОР

- Редактор построен на GrapesJS.
- Страница собирается из готовых блоков (core-hero, core-features, core-gallery, core-faq, core-cta, core-contacts, core-footer и др.).
- Каталог блоков отображается в левой панели, вкладка «Блоки».
- Перетащить блок на канвас можно мышкой.
- Сохранение страницы – автоматическое.

СТИЛИ

- Классы стилей заданы заранее. Свои классы создавать нельзя.
- Inline-стили (style="...") запрещены.
- Для уникальных визуальных эффектов используется <style> с уникальным префиксом, например effect-a3f7.
- Внутри <style> можно использовать var(--theme-*) для совместимости с темами.

ПРЕСЕТЫ

- Пресеты хранятся в левой панели, вкладка «Пресеты».
- Кнопка «+» сохраняет текущий канвас как новый пресет.
- Клик по пресету применяет его HTML и CSS к канвасу.
- Thumbnail загружается через POST /presets/{id}/thumbnail.

LLM-ЧАТ

- Чат открывается справа.
- Введите запрос – модель соберёт или изменит страницу.
- Кнопка «стоп» (■) отменяет текущий запрос.
- Кнопка «очистить чат» (иконка) удаляет историю сообщений.
- История сохраняется в БД и подгружается при открытии страницы.

ВЫДЕЛЕНИЕ ЭЛЕМЕНТА

- Команды «заполни выделенный блок» и «оформи фон выделенного» работают, если элемент выделен мышкой на канвасе.
- Выделить можно любой элемент – целый блок (<section>), либо внутренний узел (<div>, <h1>, <p>, <a>).
- Если ничего не выделено – появится сообщение «Выделите элемент на странице мышкой и повторите запрос».
- Выделить элемент можно кликом по нему на канвасе.

ВЕЖЛИВОСТЬ

- Приветствия («привет», «здравствуйте», «добрый день») – коротко поздоровайся и предложи помощь по редактору.
- Благодарности («спасибо», «благодарю», «спс», «thanks», «thank you», «класс», «круто», «отлично», «супер») – ответь коротко и тепло: «Пожалуйста!», «Рад помочь!», «Обращайтесь!». Не отказывай.
- Прощания («пока», «до свидания», «bye») – коротко попрощайся: «До связи!», «Удачи!».
- Одобрение/реакция на результат («круто», «то что нужно», «получилось») – порадуйся вместе с пользователем, коротко. Не отказывай.
- Если после вежливости пользователь хочет продолжить – предложи, с чем ещё помочь по редактору.

ЧЕГО НЕЛЬЗЯ ДЕЛАТЬ

- Писать в чате не по теме редактора – будет отказ. НО вежливость (приветствие, благодарность, прощание, реакция на результат) – это НЕ «не по теме», на неё всегда отвечай тепло и коротко.

- Использовать inline-стили.
- Создавать свои CSS-классы.
- Ломать структуру блоков вручную.

ПРАВИЛА ОТВЕТА:

1. Отвечай кратко (1–3 предложения).
2. Если ответа в документации нет – честно скажи:
«В документации этого пока нет».
3. Если это вежливость – приветствие, благодарность, прощание, реакция на результат – ответ коротко и тепло (см. раздел ВЕЖЛИВОСТЬ). НЕ отказывай.
4. Если вопрос не про NeuroCad и это НЕ вежливость – вежливо откажись: «Я помогаю только с редактором NeuroCad. Спросите про блоки, пресеты, стили, чат или работу редактора».
5. Не придумывай функции, которых нет в документации.
6. Не предлагай пользователю вставлять SVG, <style> или inline-стили вручную – этим занимается агент effect.
7. Если пользователь спрашивает «как оформить фон выделенного» – ответ: «Выделите элемент мышкой и напишите „оформи фон выделенного“. Агент добавит эффект автоматически.»

"""

```

async def run(
    self,
    *,
    provider,
    user_message: str,
    page_id: int,
    run_id: Any,
    emit=None,
    selection: Optional[Dict[str, Any]] = None,
    block_catalog: Optional[list] = None,
    current_html: Optional[str] = None,
    history: Optional[list] = None,
) -> Dict[str, Any]:

    # ---- emit: step at the start ----
    if emit:
        await emit({
            "type": "step",
            "step": "help",
            "message": "Ищу ответ...",
        })

    # ---- dump request ----
    try:
        from ..dumper import CoreEngineLibWordLlmDumper
        CoreEngineLibWordLlmDumper.dump_step(
            run_id=run_id,
            agent_name="help",
            system_prompt=self.FAQ,
            user_content=user_message,
            history=history,
        )
    except Exception as e:
        print(f"[help] dump request failed: {e}", flush=True)

```

```

# ---- build messages ----
messages = [{"role": "system", "content": self.FAQ}]
if history:
    for m in history:
        role = m.get("role")
        content = m.get("content")
        if role in ("user", "assistant") and content:
            messages.append({"role": role, "content": str(content)})
messages.append({"role": "user", "content": user_message})

# ---- provider call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[help] LLM error: {e}", flush=True)
    return {
        "message": "Не удалось получить ответ. Попробуйте ещё раз.",
        "html": None,
        "selector": None,
        "element_html": None,
    }

# ---- dump response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="help",
        raw_response=raw,
    )
except Exception as e:
    print(f"[help] dump response failed: {e}", flush=True)

answer = (raw or "").strip()
if not answer:
    answer = "В документации этого пока нет."

return {
    "message": answer,
    "html": None,
    "selector": None,
    "element_html": None,
}

```

```

=====
FILE: core/engine/lib/word/llm/agent/none.py
=====

```

```

# neurocad/core/engine/lib/word/llm/agent/none.py

```

```

"""

```

```

Agent: none.

```

The request is not related to the NeuroCad editor. No LLM call is made – the agent returns a canned message immediately, so nothing is spent.

No intermediate events are emitted – the response is instant.

"""

```
from typing import Any, Dict, Optional
```

```
from .base import CoreEngineLibWordLlmAgentBase
```

```
class CoreEngineLibWordLlmAgentNone(CoreEngineLibWordLlmAgentBase):
    """Reject off-topic requests without touching the LLM."""

    name = "none"

    #: Canned reply for off-topic requests.
    MESSAGE = (
        "Я помогаю только с редактором NeuroCad. "
        "Спросите про блоки, пресеты, стили, чат или работу редактора."
    )

    async def run(
        self,
        *,
        provider,
        user_message: str,
        page_id: int,
        run_id: Any,
        emit=None,
        selection: Optional[Dict[str, Any]] = None,
        block_catalog: Optional[list] = None,
        current_html: Optional[str] = None,
        history: Optional[list] = None,
    ) -> Dict[str, Any]:
        # No emit – the answer is instant, nothing to stream.
        return {
            "message": self.MESSAGE,
            "html": None,
            "selector": None,
            "element_html": None,
        }
```

```
=====
FILE: core/engine/lib/word/llm/agent/rename.py
=====
```

```
# neurocad/core/engine/lib/word/llm/agent/rename.py
```

"""

Agent: rename.

Proposes a NEW label and SVG miniature for an existing effect, based on its current CSS. Does NOT change the id or the CSS – both are returned exactly as received.

This is a specialised variant of the `edit` agent, triggered from the UI when the user clicks the "pencil" (rename) button on an active effect block. Unlike `edit`, this agent does not ask the LLM to rewrite anything – it only asks for a human-readable label and a

new icon.

Why the id does NOT change:

Renaming by changing the id would require rewriting the CSS selector, re-registering the block under the new id, deleting the old effect and replacing the class on every element that uses it. Any failure along that chain leaves the canvas broken. Keeping the id intact means the effect keeps its class `.fx-<id>`, its `.css` file, and every element that uses it keeps working – only the name in the palette and the icon change.

Input:

effect_id	– e.g. "fx-rotating-globe"
effect_css	– current CSS (returned unchanged)
user_message	– system-generated, but kept for logging
effect_label	– current human label (optional)

Output:

```
{
  "message": "Готово.",
  "css": "<the same CSS we received>",
  "effect_id": "<the same id we received>",
  "new_label": "Короткое название",
  "new_media": "<svg viewBox='0 0 24 24'>...</svg>",
}
```

Safety:

- ``new_label`` must be non-empty. If it is empty, the agent returns ``css=None`` so the client skips the update entirely.
- ``new_media`` is OPTIONAL: if the model did not produce a valid SVG (or forgot it entirely), the agent returns an empty string, and the client falls back to reusing the OLD miniature.

The agent never touches the page HTML.

"""

```
import json
import re
from typing import Any, Dict, List, Optional

from .base import CoreEngineLibWordLlmAgentBase
from ..prompts.rename_effect import build_effect_rename_prompt

#: A very cheap sanity check for the SVG media. The prompt asks for
#: a 24x24 <svg> one-liner; we only need to be sure it's an SVG at
#: all – the client will fall back to the old miniature if not.
_SVG_HINT_RE = re.compile(r"<\s*svg\b", re.IGNORECASE)

class CoreEngineLibWordLlmAgentRename(CoreEngineLibWordLlmAgentBase):
    """Propose a new label/media for an existing effect."""

    name = "rename"

    async def run(
```

```

self,
*,
provider,
user_message: str,
page_id: int,
run_id: Any,
emit=None,
selection: Optional[Dict[str, Any]] = None,
block_catalog: Optional[list] = None,
current_html: Optional[str] = None,
history: Optional[list] = None,
effect_id: Optional[str] = None,
effect_css: Optional[str] = None,
effect_label: Optional[str] = None,
existing_effect_ids: Optional[List[str]] = None,
) -> Dict[str, Any]:

# ---- emit: step at the start ----
if emit:
    await emit({
        "type": "step",
        "step": "rename",
        "message": "Придумываю новое название...",
    })

# ---- guard ----
if not effect_id or not effect_css:
    return {
        "message": "Режим переименования не активен.",
        "css": None,
        "effect_id": None,
        "new_label": None,
        "new_media": None,
    }

# ---- system prompt ----
# `existing_effect_ids` is kept in the signature for API
# symmetry with the other agents, but is not used here – the
# id of the effect does not change, so there is nothing to
# collide with.
system_prompt = build_effect_rename_prompt(
    effect_id=effect_id,
    current_css=effect_css,
    effect_label=effect_label or "",
)

user_content = (user_message or "").strip() or (
    "Придумай новое название и иконку для этого эффекта, "
    "id и CSS не меняй."
)

# ---- dump prompt ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,

```

```

        agent_name="rename",
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
    )
except Exception as e:
    print(f"[rename] dump request failed: {e}", flush=True)

# ---- build messages ----
messages = [{"role": "system", "content": system_prompt}]
if history:
    for m in history:
        role = m.get("role")
        content = m.get("content")
        if role in ("user", "assistant") and content:
            messages.append({"role": role, "content": str(content)})
messages.append({"role": "user", "content": user_content})

# ---- provider call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[rename] LLM error: {e}", flush=True)
    return {
        "message": f"⚠ Ошибка LLM: {e}",
        "css": None,
        "effect_id": None,
        "new_label": None,
        "new_media": None,
    }

# ---- dump response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="rename",
        raw_response=raw,
    )
except Exception as e:
    print(f"[rename] dump response failed: {e}", flush=True)

# ---- parse ----
parsed = self._parse_response(raw)

new_label = parsed.get("new_label")
new_media = parsed.get("new_media")

# ---- validate the label ----
clean_label = self._validate_label(new_label)
if not clean_label:
    print(
        f"[rename] model did not return a usable new_label: "
        f"{str(raw)[:160]}!r}",
        flush=True,
    )

```

```

    return {
        "message": "⚠ Не удалось подобрать новое название. Попробуйте ещё раз.",
        "css": None,
        "effect_id": None,
        "new_label": None,
        "new_media": None,
    }

# ---- media validation ----
# The media is OPTIONAL: if the model did not produce a valid
# SVG (or forgot it entirely), we return an empty string and
# the client falls back to reusing the OLD miniature.
clean_media = self._validate_media(new_media)

    return {
        "message": "Готово.",
        "css": effect_css,          # UNCHANGED
        "effect_id": effect_id,    # UNCHANGED
        "new_label": clean_label,
        "new_media": clean_media,  # may be "" if invalid/missing
    }

# =====
# PARSE
# =====

@staticmethod
def _parse_response(text: str) -> Dict[str, Any]:
    """
    Extract `new_label` and `new_media` from the model response.

    Accepts JSON: { "new_label": "...",
                    "new_media": "<svg ...>...</svg>" }.
    Also tolerates an accidental "css" or "new_id" field – they
    are ignored, we never use the model's CSS or id in this agent.
    """
    result: Dict[str, Any] = {
        "new_label": None,
        "new_media": None,
    }

    if not text:
        return result

    s = text.strip()

    # strip markdown fence
    if s.startswith("```"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("```"):
            s = s[:-3]
        s = s.strip()

    # try the whole string

```

```

data = CoreEngineLibWordLlmAgentRename._try_json(s)

# try the first {...} JSON block
if data is None:
    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        data = CoreEngineLibWordLlmAgentRename._try_json(
            s[start:end + 1]
        )

if isinstance(data, dict):
    result["new_label"] = data.get("new_label")
    result["new_media"] = data.get("new_media")

return result

@staticmethod
def _try_json(text: str) -> Optional[dict]:
    try:
        data = json.loads(text)
    except Exception:
        return None
    return data if isinstance(data, dict) else None

# =====
# VALIDATION
# =====

@staticmethod
def _validate_label(raw_label: Any) -> str:
    """
    Sanitize the required `new_label` field.

    Rules:
    - must be non-empty after stripping;
    - must not exceed 80 characters (cut off at 77 with "...");
    - surrounding quotes (single, double, « », " ") are
      stripped – models sometimes wrap the label in them.

    Returns the clean label, or "" if the payload is unusable.
    """
    if not raw_label:
        return ""

    s = str(raw_label).strip()
    if not s:
        return ""

    # Strip a matching pair of quotes at both ends.
    pairs = [("'", "'"), ('"', '"'), ("«", "»"), ("\"", "\"")]
    for open_q, close_q in pairs:
        if s.startswith(open_q) and s.endswith(close_q) and len(s) >= 2:
            s = s[1:-1].strip()
            break

```

```

    if not s:
        return ""

    if len(s) > 80:
        s = s[:77] + "..."

    return s

@staticmethod
def _validate_media(raw_media: Any) -> str:
    """
    Sanitize the optional `new_media` field.

    The client can survive without a fresh miniature: if we return
    an empty string, it will reuse the OLD SVG from the previous
    block. So this function is intentionally lax – it only checks
    that the payload looks like an SVG at all.

    Returns the SVG as a string, or "" if the payload is not
    usable.
    """
    if not raw_media:
        return ""

    s = str(raw_media).strip()
    if not s:
        return ""

    if not _SVG_HINT_RE.search(s):
        print(
            f"[rename] new_media does not look like SVG "
            f"(first 60 chars: {s[:60]!r}) – dropping",
            flush=True,
        )
        return ""

    # Collapse internal newlines – the prompt asks for a one-liner
    # but models occasionally emit multi-line SVG. A single line
    # is easier to embed into the block model and to log.
    s = " ".join(s.split())

    return s

```

```

=====
FILE: core/engine/lib/word/llm/agent/router.py
=====

```

```

# neurocad/core/engine/lib/word/llm/agent/router.py

```

```

"""

```

Router – decides which agent handles the request.

One short LLM call. Input: user message + state (selection, page presence) + recent history. Output: {"agent": ..., "target": ...}.

Key responsibilities

```

-----

```

1. Pick the right agent for a NEW request (create / create_page / fill / effect / help / none).
2. Handle RETRY messages ("попробуй ещё раз", "повтори", "заново", "ещё раз"). The router sees recent history and knows what the user was trying – so it can route the retry to the same agent, instead of falling into `help`.
3. Handle EDIT requests for an existing page. `create\_page` has two modes (create / edit); the router only decides "this is a page request", the agent picks its own mode from `current\_html`.

Selection format

Client sends `selection` in one of two shapes:

- {"selector": "sel-abc12345", "tag": "section", "classes": [...], "outer_html": "..."} – preferred
- {"block_id": "core-hero"} – legacy

`selector` wins; `block\_id` is the fallback.

Image requests

`effect` handles both visual effects (backgrounds, gradients, animations) AND image generation (draw / generate / replace an SVG). The router only routes BOTH to `effect`, never to `fill`. Rule 7 in SYSTEM_PROMPT enforces this.

Page generation – two flavours

- `create` – build a page from OUR ready blocks (plan → fill → effects → svg).
- `create\_page` – build OR edit a page in one LLM call with free-form HTML+CSS. Pick it for creative / nonstandard pages, or when the user asks to change an existing page.

Rule 8 in SYSTEM_PROMPT explains how to choose between the two.

Logging: uses provider.log (app.state.log, passed via the provider).

"""

```
import json
import re
from typing import Any, Dict, List, Optional
```

```
class CoreEngineLibWordLlmRouter:
```

```
    """Pick the right agent for a user request."""
```

```
    AGENTS = {
```

```
        "create": "Собрать страницу ИЗ НАШИХ ГОТОВЫХ БЛОКОВ (пошагово: план → заполнение → эффекты → svg). Типовые лендинги",
        "create_page": "Создать ИЛИ отредактировать страницу свободным HTML+CSS. Креативные страницы и правки",
        "fill": "Заполнить ВЫДЕЛЕННЫЙ элемент текстом / alt'ами / ссылками (НЕ картинками)",
        "effect": "Визуальный эффект ИЛИ сгенерированная картинка/SVG для ВЫДЕЛЕННОГО элемента",
        "help": "Вопрос про сам редактор NeuroCad",
        "none": "Запрос не связан с редактором",
```



```
}
```

```
# Retry-like phrases. If the user message matches one of these  
# AND there is recent history, the LLM is asked to re-run the  
# previous action.
```

```
_RETRY_RE = re.compile(  
    r"\b(попробуй\s+(ещё|еще)\s+раз|"  
    r"повтори|заново|"  
    r"переделай|"  
    r"ещё\s+раз|еще\s+раз|"  
    r"давай\s+снова|"  
    r"try\s+again|retry|redo)\b",  
    re.IGNORECASE,  
)
```

```
SYSTEM_PROMPT = """"Ты — маршрутизатор запросов в редакторе страниц NeuroCad.
```

Твоя задача — выбрать ОДИН агент, который обрабатывает запрос пользователя.

ДОСТУПНЫЕ АГЕНТЫ:

```
{agents}
```

СОСТОЯНИЕ РЕДАКТОРА:

```
{state}
```

ПОСЛЕДНИЕ СООБЩЕНИЯ В ЧАТЕ (свежие — внизу):

```
{history}
```

ПРАВИЛА:

1. Ответь ТОЛЬКО валидным JSON, без markdown и пояснений.
2. Формат: `{{"agent": "<имя>", "target": "<selector | null>"}}`
3. Поле "target" заполняй ТОЛЬКО для агентов fill и effect.
Скопируй туда значение `selector` из состояния редактора дословно.
4. Если запрос не связан с редактором (анекдоты, погода, политика) — верни "none".
5. Если непонятно — верни "help".
6. RETRY: если пользователь просит повторить («попробуй ещё раз», «повтори», «заново», «переделай») — посмотри на предыдущий запрос пользователя в истории и верни ТОТ ЖЕ агент, который был бы выбран для него. Не отправляй такой запрос в help.
7. Картинки: если пользователь просит НАРИСОВАТЬ / СГЕНЕРИРОВАТЬ / СДЕЛАТЬ картинку, SVG, иллюстрацию, «заменить плейсхолдер» — верни agent="effect", даже если в запросе есть слово «заполни».
8. Страница / лендинг / сайт — выбор между create и create_page:
 - "create" — только если пользователь явно просит «из блоков», «по блокам», «собери из готовых»;
 - "create_page" — во всех остальных случаях: «сгенерируй страницу», «сделай лендинг», «создай сайт», креативные темы, нестандартный дизайн. Также если страница уже есть и пользователь просит её ИЗМЕНИТЬ целиком («поменяй заголовок», «убери секцию», «добавь блок с ценами»), но при этом НЕ выделен конкретный элемент для точечной правки.
9. Правка выделенного элемента (fill / effect) имеет приоритет над правкой страницы: если есть выделение и пользователь просит изменить именно его — используй fill или effect.

ПРИМЕРЫ:

- "Сделай лендинг для салона" → {"agent": "create_page", "target": null}}
- "Собери страницу из блоков" → {"agent": "create", "target": null}}
- "Сгенерируй крутую страницу о Bore" → {"agent": "create_page", "target": null}}
- "Сделай блог о путешествиях" → {"agent": "create_page", "target": null}}
- "Поменяй заголовок в hero" (есть страница, нет выделения) → {"agent": "create_page", "target": null}}
- "Убери секцию с отзывами" → {"agent": "create_page", "target": null}}
- "Добавь блок с ценами после features" → {"agent": "create_page", "target": null}}
- "Заполни выделенный блок" → {"agent": "fill", "target": "sel-abc12345"}}
- "Сделай анимацию этому блоку" → {"agent": "effect", "target": "sel-abc12345"}}
- "Нарисуй картинку вместо плейсхолдера" → {"agent": "effect", "target": "sel-abc12345"}}
- "Как сохранить пресет?" → {"agent": "help", "target": null}}
- "Расскажи анекдот" → {"agent": "none", "target": null}}

ПРИМЕР RETRY:

История:

user: Собери лендинг для приюта кошек

assistant: ⚠ Модель вернула некорректный ответ. Попробуйте ещё раз.

Текущий запрос: "Попробуй ещё раз"

Ответ: {"agent": "create_page", "target": null}}

Верни ОДИН JSON-объект. Начни с {{ и закончи }}.

"""

```
# =====  
# LOG HELPER  
# =====
```

```
@staticmethod  
def _log(provider, level: str, message: str) -> None:  
    log = getattr(provider, "log", None)  
    if log is None:  
        return  
    fn = getattr(log, f"log_{level}_sync", None)  
    if fn is None:  
        return  
    try:  
        fn(target="router", message=message)  
    except Exception:  
        pass
```

```
# =====  
# RETRY DETECTION  
# =====
```

```
@classmethod  
def _is_retry(cls, user_message: str) -> bool:  
    """True if the message is a retry-like phrase."""  
    if not user_message:  
        return False  
    return bool(cls._RETRY_RE.search(user_message))
```

```
# =====  
# STATE  
# =====
```

```

@staticmethod
def _build_state(
    selection: Optional[Dict[str, Any]],
    current_html: Optional[str],
) -> str:
    lines = []

    if selection and selection.get("selector"):
        tag = selection.get("tag") or "?"
        classes = selection.get("classes") or []
        cls = ", ".join(str(c) for c in classes) if classes else "-"
        lines.append(f"- выделен элемент: <{tag}> с классами: {cls}")
        lines.append(f"- selector: {selection['selector']}")
    elif selection and selection.get("block_id"):
        lines.append(f"- выделен блок: {selection['block_id']}")
    else:
        lines.append("- выделен элемент: нет")

    if current_html and current_html.strip():
        lines.append("- страница уже есть: да (её можно редактировать)")
    else:
        lines.append("- страница уже есть: нет (пустая)")

    return "\n".join(lines)

```

```

# =====
# HISTORY
# =====

```

```

@staticmethod
def _build_history_block(
    history: Optional[List[Dict[str, str]]],
    max_turns: int = 4,
) -> str:
    """
    Render the last few user / assistant turns as a compact block.

    The router only needs enough context to understand a RETRY
    ("what was the user asking for before?") and to see whether
    the previous attempt succeeded or failed. Trim each message
    so the prompt stays small.
    """
    if not history:
        return "(пока ничего)"

    # Keep only the last N messages, in order.
    tail = history[-max_turns:]

    lines = []
    for m in tail:
        role = (m.get("role") or "").lower()
        content = (m.get("content") or "").strip()
        if not content:
            continue
        # Truncate each message so a long assistant reply does

```

```

        # not blow up the router prompt.
        if len(content) > 400:
            content = content[:400] + "..."
        if role == "user":
            lines.append(f" user: {content}")
        elif role == "assistant":
            lines.append(f" assistant: {content}")

    return "\n".join(lines) if lines else "(пока ничего)"

# =====
# ROUTE
# =====

@staticmethod
async def route(
    provider,
    user_message: str,
    selection: Optional[Dict[str, Any]] = None,
    current_html: Optional[str] = None,
    history: Optional[List[Dict[str, str]]] = None,
    run_id: Any = None,
    emit=None,
) -> Dict[str, Any]:
    """
    Ask the LLM which agent should handle the request.

    Returns {"agent": "<name>", "target": "<selector | null>"}.
    On any failure, falls back to {"agent": "help", "target": None}.

    `history` is the recent chat history (user / assistant).
    Used to resolve RETRY messages – the router sees what the
    user asked last and routes the retry to the same agent.

    `emit` is an optional async callback for progress frames.
    """
    if emit:
        await emit({
            "type": "step",
            "step": "routing",
            "message": "Определяю действие...",
        })

    agents_block = "\n".join(
        f"- {name}: {desc}"
        for name, desc in CoreEngineLibWordLlmRouter.AGENTS.items()
    )
    state_block = CoreEngineLibWordLlmRouter._build_state(
        selection, current_html,
    )
    history_block = CoreEngineLibWordLlmRouter._build_history_block(history)

    system_prompt = CoreEngineLibWordLlmRouter.SYSTEM_PROMPT.format(
        agents=agents_block,
        state=state_block,
        history=history_block,
    )

```

```

)

messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": user_message},
]

# ---- dump router request ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name="router",
        system_prompt=system_prompt,
        user_content=user_message,
    )
except Exception as e:
    CoreEngineLibWordLlmRouter._log(
        provider, "warning", f"dump request failed: {e}",
    )

try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    CoreEngineLibWordLlmRouter._log(provider, "error", f"LLM error: {e}")
    return {"agent": "help", "target": None}

# ---- dump router response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="router",
        raw_response=raw,
    )
except Exception as e:
    CoreEngineLibWordLlmRouter._log(
        provider, "warning", f"dump response failed: {e}",
    )

# ---- parse ----
data = CoreEngineLibWordLlmRouter._extract_json(raw)
if not data:
    CoreEngineLibWordLlmRouter._log(
        provider, "warning", f"failed to parse response: {raw!r}",
    )
    # Rule-based fallback: if it's a retry, do not fall into help.
    if CoreEngineLibWordLlmRouter._is_retry(user_message):
        return {"agent": "create_page", "target": None}
    return {"agent": "help", "target": None}

agent = str(data.get("agent", "help")).strip()
target = data.get("target")
if target is not None:
    target = str(target).strip() or None

```

```

# ---- validate agent name ----
if agent not in CoreEngineLibWordLlmRouter.AGENTS:
    CoreEngineLibWordLlmRouter._log(
        provider, "warning",
        f"unknown agent {agent!r}, falling back to help",
    )
    agent = "help"
    target = None

# ---- sanity: fill/effect REQUIRE a selection ----
# If the model routed to fill/effect but there is no
# selection, the dispatcher would bounce the request anyway
# ("выделите элемент"). Better to catch it here and route to
# a page-level agent instead.
if agent in ("fill", "effect"):
    has_selection = bool((selection or {}).get("selector"))
    if not has_selection:
        CoreEngineLibWordLlmRouter._log(
            provider, "info",
            f"agent={agent} but no selection - "
            f"falling back to create_page",
        )
        agent = "create_page"
        target = None

result = {"agent": agent, "target": target}
CoreEngineLibWordLlmRouter._log(
    provider, "info", f"{user_message!r} -> {result}",
)
return result

# =====
# PARSING
# =====

@staticmethod
def _extract_json(text: str) -> Optional[dict]:
    """Tolerant JSON extraction – same pattern as in step.py."""
    if not text:
        return None

    s = text.strip()

    if s.startswith("`"):
        first_nl = s.find("\n")
        if first_nl != -1:
            s = s[first_nl + 1:]
        if s.endswith("`"):
            s = s[:-3]
        s = s.strip()

    try:
        data = json.loads(s)
        if isinstance(data, dict):
            return data
    except json.JSONDecodeError:

```

```

        pass

    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        try:
            data = json.loads(s[start:end + 1])
            if isinstance(data, dict):
                return data
        except json.JSONDecodeError:
            pass

    return None

```

```

=====
FILE: core/engine/lib/word/llm/agent/__init__.py
=====

```

```

=====
FILE: core/engine/lib/word/llm/prompts/create_effect.py
=====

```

```

# neurocad/core/engine/lib/word/llm/prompts/create_effect.py

```

```

"""

```

```

Effect create prompt – generate a DRAFT for a brand-new effect from
scratch.

```

```

Used when the user asks to create a NEW effect (see
llm/agent/create_effect.py). The model returns a JSON draft:

```

```

{
  "id":    "fx-<english-slug>",
  "label": "<название на русском>",
  "hint":  "<короткое описание на русском>",
  "css":   ".core-engine-lib-word-blocks .<id> { ... }",
  "media": "<svg viewBox='0 0 24 24'>...</svg>"
}

```

Nothing is written to disk here. The draft is sent to the user in the chat; the user reviews it and may ask for renames or tweaks. The save happens on an explicit "сохрани".

The SVG_EXAMPLES_BLOCK constant below is a set of reference icons the model uses as style examples – same 24x24 viewBox, same fill/stroke palette, same level of detail. They are NOT registered anywhere; they exist only inside this prompt.

Existing effect ids

```

-----

```

The caller passes `existing\_effect\_ids` – the list of ids already registered in the palette. If the list is non-empty, it is embedded into the prompt with a clear rule: do NOT pick any of these ids. The model still chooses its own slug; the list is only a "do not use" set, so a colliding draft never reaches the client.

No static imports beyond the standard library. This module is pure – it only builds a string.

```
from typing import Any, List, Optional
```

```
# =====
# SVG ICON EXAMPLES
# =====
#
```

```
# Reference icons for the create-effect flow. The model uses these
# as style examples – same 24x24 viewBox, same fill/stroke palette,
# same level of detail.
```

```
#
# These are NOT registered anywhere. They exist only in the prompt.
```

```
SVG_EXAMPLES_BLOCK = """\
```

```
Пример 1 – концентрические круги (радиальный градиент с кольцами):
```

```
<svg viewBox="0 0 24 24" width="20" height="20"><rect x="2" y="2" width="20" height="20" rx="3" fill="#f1f5f9"/><g fill="n
width="0.9"><circle cx="12" cy="12" r="3"/><circle cx="12" cy="12" r="6"/><circle cx="12" cy="12" r="9"/></g></svg>
```

```
Пример 2 – угол сверху-слева (диагональный градиент):
```

```
<svg viewBox="0 0 24 24" width="20" height="20"><rect x="2" y="2" width="20" height="20" rx="3" fill="#f1f5f9"/><path d="M
```

```
Пример 3 – мелкие точки (radial-gradient в повторении):
```

```
<svg viewBox="0 0 24 24" width="20" height="20"><rect x="2" y="2" width="20" height="20" rx="3" fill="#f1f5f9"/><g fill="#f
```

```
Пример 4 – тень сверху (box-shadow inset):
```

```
<svg viewBox="0 0 24 24" width="20" height="20"><rect x="2" y="2" width="20" height="20" rx="3" fill="#f1f5f9"/><rect x="2" y="2" width="20" height="20" rx="3" fill="none" stroke="black" stroke-width="1px" opacity="0.5"/><defs><linearGradient id="s-top" x1="0" y1="0" x2="0" y2="1"><stop offset="0%" stop-color="#0f172a"/><stop offset="100%" stop-color="#0f172a" stop-opacity="0"/></linearGradient></defs></svg>
"""
```

```
def _render_existing_ids_block(
    existing_effect_ids: Optional[List[Any]],
) -> str:
```

```
    """
    Render the "do not use these ids" section, or an empty string.

    - Normalizes the list: strips whitespace, drops empties, dedups.
    - If nothing remains, returns "" so the prompt stays short.
    """
    if not existing_effect_ids:
        return ""
```

```
    seen: set = set()
    ids: List[str] = []
    for item in existing_effect_ids:
        if not isinstance(item, (str, int)):
            continue
        s = str(item).strip()
        if not s or s in seen:
            continue
        seen.add(s)
```



```

        ids.append(s)

    if not ids:
        return ""

    lines = "\n".join(f"    - `{x}`" for x in ids)
    return (
        "\nЗАНЯТЫЕ ID (нельзя использовать ни один из них):\n"
        f"{lines}\n"
    )

def build_create_effect_prompt(
    existing_effect_ids: Optional[List[Any]] = None,
) -> str:
    """
    System prompt for the effect-create flow.

    The model returns a JSON draft with id, label, hint, css and
    media (SVG). Nothing is written to disk here; the draft goes to
    the chat and the save happens on an explicit "сохрани".

    @param existing_effect_ids – ids already registered in the
        palette. The prompt embeds them as a "do not use" set, so the
        model does not propose a colliding slug. If the list is empty
        or None, the section is omitted.
    @returns the full system prompt as a string
    """
    existing_block = _render_existing_ids_block(existing_effect_ids)

    return f"""Ты – дизайнер CSS-эффектов для CMS NeuroCad.
```

Пользователь описывает, какой фон или визуальный эффект он хочет.
 Ты придумываешь эффект и возвращаешь ЧЕРНОВИК в виде JSON.
 Черновик показывается пользователю в чате – ничего не сохраняется,
 пока пользователь не подтвердит.

ФОРМАТ ОТВЕТА – ТОЛЬКО ВАЛИДНЫЙ JSON:

```

{{
  "id":    "fx-<english-slug>",
  "label": "<название на русском>",
  "hint":  "<короткое описание на русском>",
  "css":   ".core-engine-lib-word-blocks .<id> {{ ... }}",
  "media": "<svg viewBox='0 0 24 24'>...</svg>"
}}
```

ПРАВИЛА ID:

1. Начинается с `fx-`.
2. Только строчные латинские буквы, цифры и дефисы.
3. Отражает СУТЬ эффекта (что видно в CSS), не транслит:
 - мерцающие точки → fx-shimmer-dots
 - градиентный уголок → fx-corner-gradient
 - пульсирующая тень → fx-pulse-shadow
 - диагональные полосы → fx-diagonal-stripes
4. Длина – до 64 символов.

```
{existing_block}
```

ПРАВИЛА CSS:

5. Внешний селектор ОБЯЗАТЕЛЬНО:

`\.core-engine-lib-word-blocks .<id>\`

6. НЕ добавляй ``background-color`` и ``color``. Цвет фона и текста задаются панелью стилей, а не эффектами.

7. НЕ добавляй ``@import``, ``url(...)``, ``expression(...)``, ``javascript:``, ``behavior:``, ``-moz-binding``.

8. Можно использовать: ``background-image`` (linear-gradient, radial-gradient, repeating-linear-gradient), ``background-size``, ``background-position``, ``box-shadow``, ``filter``, ``mask``, ``@keyframes`` + ``animation``, ``::before`` / ``::after`` (все вложенные селекторы – с тем же префиксом, что и внешний).

ПРАВИЛА MEDIA (SVG-иконка):

9. `viewBox` ОБЯЗАТЕЛЬНО ``0 0 24 24``. Атрибуты `width` и `height` НЕ указывай.

10. Иконка отражает ВИЗУАЛЬНУЮ СУТЬ CSS:

- радиальный градиент с кольцами → рисуем кольца;
- линейный диагональный градиент → рисуем диагональные полосы;
- тень → рисуем мягкую полосу тени;
- точки → рисуем точки.

11. Стиль – как у примеров ниже. Светло-серый фон ``#f1f5f9`` с закруглением ``rx="3"``, узор – ``#cbd5e1`` или ``#94a3b8``.

12. SVG должен быть валидным XML, одной строкой, без переводов строк внутри. Кавычки – двойные.

ПРИМЕРЫ ИКОНОК:

{SVG_EXAMPLES_BLOCK}

ПРИМЕРЫ ЗАПРОСОВ И ОТВЕТОВ:

Запрос: «Мерцающие точки по фону»

Ответ:

```
{{"id": "fx-shimmer-dots", "label": "Мерцающие точки", "hint": "Мелкие точки с плавным мерцанием", "css": ".core-engine-lib-word-blocks .fx-shimmer-dots {{ background-image: radial-gradient(#cbd5e1 1px, transparent 1.5px); background-size: 12px 12px; animation: fx-shimmer-dots-pulse 3s ease-in-out infinite; }} @keyframes fx-shimmer-dots-pulse {{ 0%, 100% {{ opacity: 1; }} 50% {{ opacity: 0.6; }} }}", "media": "<svg viewBox=\\\"0 0 24 24\\\"><rect x=\\\"2\\\" y=\\\"2\\\" width=\\\"20\\\" height=\\\"20\\\" rx=\\\"3\\\" fill=\\\"#f1f5f9\\\"></rect></svg>"}}
```

Запрос: «Градиентный уголок в правом верхнем углу»

Ответ:

```
{{"id": "fx-corner-gradient", "label": "Градиентный уголок", "hint": "Плавный градиент от правого верхнего угла", "css": ".core-engine-lib-word-blocks .fx-corner-gradient {{ background-image: linear-gradient(225deg, #94a3b8 0, #cbd5e1 49%, transparent 49%, transparent 51%, #f1f5f9 51%); background-repeat: no-repeat; }}", "media": "<svg viewBox=\\\"0 0 24 24\\\"><rect x=\\\"2\\\" y=\\\"2\\\" width=\\\"20\\\" height=\\\"20\\\" rx=\\\"3\\\" fill=\\\"#f1f5f9\\\"></rect></svg>"}}
```

ГЛАВНОЕ ПРАВИЛО:

13. Делай ровно то, что просят. Не расширяй задачу.

14. Если непонятно – придумай минимальный вариант по запросу.
15. Ответ – ТОЛЬКО валидный JSON. НЕ оборачивай в markdown.
НЕ добавляй пояснения до или после JSON.
16. Если в разделе «ЗАНЯТЫЕ ID» перечислены id – НИ ОДИН из них использовать нельзя. Придумай новый уникальный id, которого нет в этом списке.

"""

```
=====
FILE: core/engine/lib/word/llm/prompts/create_page.py
=====
```

```
# neurocad/core/engine/lib/word/llm/prompts/create_page.py
```

"""

Prompts for the create_page agent.

The agent has FOUR modes, each backed by its own prompt builder:

1. PLAN – one short call. Returns the LIST of sections:

```
{ "sections": [ "hero", "features", "footer" ] }
```

 No HTML, no CSS. Structure is a single decision.
2. SECTION – one call per section from the plan. Each call knows exactly which section it produces and returns:

```
{ "section_name": "hero",  
  "html": "<section ...>",&br/>  "css": ".hero { ... }" }
```
3. REVIEW – one call per generated section. QA pass that finds and fixes common defects (images overflowing, empty buttons, invisible text, missing @media, ...). Returns the same section, possibly corrected, plus a "fixed" list describing what changed.
4. EDIT – one call that rewrites an existing page. Takes the current HTML + CSS as context and returns the whole updated page:

```
{ "html": "...", "css": "..." }
```

Why plan-then-generate

Stepwise mode relied on the model to not repeat section names and to stop on its own. deepseek-flash cannot do either – a 10-step run produced 10 hero sections and never stopped. Splitting the concerns fixes it:

- structure is a single deterministic decision;
- each per-section call is focused and cannot drift;
- the driver stops after the last item of the plan.

Shared invariants (HTML/CSS split, no frameworks, etc.) are stored in module-level constants and composed into each prompt. This keeps the rules in ONE place and stops them drifting between the four modes.

No static imports beyond the standard library.

"""

```

from typing import List, Optional

# =====
# LIMITS
# =====

#: Hard cap on sections in a single page. The planner is asked for
#: 4–7; the driver clamps to this as a safety net.
MAX_SECTIONS = 7

#: Lower bound – a plan with fewer items is almost certainly a
#: parsing failure, not a real design choice.
MIN_SECTIONS = 3

# =====
# SHARED RULES (single source of truth)
# =====
#
# These snippets are composed into the prompts below. Keeping them
# here stops the rules from drifting between plan / section /
# review / edit – one place to change, all prompts updated.

_LANGUAGE_RULES = """\
ЯЗЫК:
- Весь видимый текст (заголовки, кнопки, подписи, alt) – только
  на русском.
- Технические имена (class, id, CSS-свойства) – на английском."""

_SPLIT_RULES = """\
РАЗДЕЛЕНИЕ HTML И CSS (критически важно):
- HTML содержит ТОЛЬКО разметку. Ни одного <style>, ни одной
  ссылки <link rel="stylesheet">.
- CSS – отдельная строка. БЕЗ тега <style>, БЕЗ обёрток,
  БЕЗ комментариев-заголовков. Просто правила.
- GrapesJS парсит HTML и CSS по разным каналам. Если смешать –
  редактор молча выбросит дерево компонентов."""

_NO_FRAMEWORKS_RULES = """\
ЗАПРЕЩЕНО:
- <script>, <iframe>, <object>, <embed>.
- Внешние  и .
- Tailwind, Bootstrap и любые CSS-фреймворки.
- Markdown-обёртка вокруг ответа (без ```json и ```).
- Любой текст до или после JSON."""

_STYLE_RULES = """\
СТИЛЬ:
- Готовый сайт, а не черновик.
- Палитра: 2–3 цвета + нейтральные.
- Типографика: h1 ≈ 48–64px, h2 ≈ 32–40px, текст ≈ 16–18px.
- Скругления, тени, аккуратные отступы – да.
- Никаких «Lorem ipsum», «текст здесь», «заголовок».
- Внешние шрифты через @import url('https://fonts.googleapis.com/...')

```

разрешены ТОЛЬКО в первой секции страницы.

АДАПТИВНОСТЬ:

- @media для (max-width: 1024px) и (max-width: 768px).
- На мобильном – одна колонка, крупные отступы."

```
# =====  
# PHASE 1 – PLAN  
# =====
```

```
def build_plan_prompt() -> str:  
    """  
    System prompt for the planning call.  
  
    Asks for a JSON object with a single "sections" array of  
    snake_case identifiers. No HTML, no CSS, no "done".  
  
    @returns the full system prompt as a string  
    """  
    return """"Ты – планировщик лендингов.
```

Тебе дают описание страницы. Ты возвращаешь СПИСОК СЕКЦИЙ, из которых она должна состоять – сверху вниз.

ФОРМАТ ОТВЕТА – СТРОГО ВАЛИДНЫЙ JSON:

```
{  
  "sections": ["hero", "features", "pricing", "testimonials", "faq", "footer"]  
}
```

ПРАВИЛА:

1. От 4 до 7 секций.
2. Имя секции – короткий английский snake_case: hero, features, about, pricing, testimonials, faq, gallery, cta, contact, footer, stats, team, menu, services, program, reviews, ...
3. Все имена уникальны. Ни одного повтора.
4. Порядок – как у нормального лендинга:
 - первая: hero (или header),
 - последняя: footer (или cta).
5. Секции должны соответствовать теме запроса.
6. Возвращай ТОЛЬКО JSON – без HTML, без CSS, без пояснений.

ПРИМЕРЫ:

Запрос: «собери лендинг для приюта кошек»

Ответ: {"sections": ["hero", "about", "cats", "adopt", "donate", "footer"]}

Запрос: «сделай страницу для кофейни»

Ответ: {"sections": ["hero", "menu", "about", "gallery", "contact", "footer"]}

Запрос: «промо для онлайн-курса по Python»

Ответ: {"sections": ["hero", "program", "author", "reviews", "pricing", "cta", "footer"]}

Запрос: «сайт стоматологии»

Ответ: {"sections": ["hero", "services", "doctors", "reviews", "prices", "contact", "footer"]}

Верни ОДИН JSON-объект. Начни с { и закончи }."""

```
def build_plan_user_message(user_request: str) -> str:
    """User message for the planning call."""
    text = (user_request or "").strip()
    return (
        f"Запрос на страницу:\n\n{text}\n\n"
        'Верни JSON: {"sections": [...]}.'
    )
```

```
# =====
# PHASE 2 – ONE SECTION
# =====
```

```
def _format_names(names: Optional[List[str]], empty: str) -> str:
    """Render a list of section names as a numbered block."""
    if not names:
        return empty
    return "\n".join(f" {i + 1}. {n}" for i, n in enumerate(names))
```

```
def build_section_prompt(
    target_section: str,
    sections_done: Optional[List[str]] = None,
    sections_remaining: Optional[List[str]] = None,
) -> str:
```

```
    """
```

```
    System prompt for ONE per-section call.
```

```
@param target_section    the exact section to produce now
@param sections_done      sections already generated (for
                           style continuity)
@param sections_remaining sections still to come (so the
                           model doesn't cram everything into
                           the current section)
```

```
@returns the full system prompt as a string
    """
```

```
done_block = _format_names(
    sections_done,
    "(пока ничего – это первая секция)",
)
remaining_block = _format_names(
    sections_remaining,
    "(последняя секция страницы)",
)
```

```
if not sections_done:
    # First section: it sets the visual language for the whole
    # page. Be explicit about what that means.
    style_block = ""\n
```

```
Ты открываешь страницу – задаёшь палитру, шрифт, скругления и
общий ритм отступов. Последующие секции будут к ним подстраиваться.
Определи всё это сейчас и держись выбора во всех следующих шагах."""
```

```

else:
    style_block = """\
СОХРАНЯЙ ЕДИНЫЙ СТИЛЬ со уже сгенерированными секциями:
- та же палитра,
- тот же шрифт,
- те же скругления,
- тот же ритм отступов.
Новую палитру и новый шрифт НЕ вводи.""

    return f"""\
Ты – веб-дизайнер и фронтенд-разработчик.

Ты собираешь лендинг пошагово. На этом шаге – РОВНО ОДНА секция:
«{target_section}».

НЕ генерируй другие секции. НЕ генерируй всю страницу. Один
корневой элемент: <section> (или <header>/<footer>/<nav>, если
так логичнее по смыслу).

{_LANGUAGE_RULES}

УЖЕ СГЕНЕРИРОВАНО:
{done_block}

ПОСЛЕ ЭТОЙ СЕКЦИИ БУДУТ:
{remaining_block}

{style_block}

ФОРМАТ ОТВЕТА – СТРОГО ВАЛИДНЫЙ JSON:
{{
  "section_name": "{target_section}",
  "html": "<section class=\\\"{target_section}\\\">...</section>",
  "css": ".{target_section} {{ ... }}"
}}

{_SPLIT_RULES}

КЛАССЫ:
- Префикс = имя секции. Всё внутри .{target_section} начинается
  с .{target_section}__* – например .{target_section}__title,
  .{target_section}__lead, .{target_section}__grid.
- Не используй классы из нашего редактора (.section, .container,
  .card, .btn, .grid) – у тебя своя система.

КАРТИНКИ:
- Inline <svg>...</svg>, либо
- <div class="{target_section}__placeholder"></div> с фоновым
  градиентом в CSS.
- Никаких внешних URL.

{_STYLE_RULES}

{_NO_FRAMEWORKS_RULES}

ПРИМЕР ОТВЕТА (секция features):

```

```
{{
  "section_name": "features",
  "html": "<section class=\\\"features\\\"><div class=\\\"features__inner\\\"><h2 class=\\\"features__title\\\">Почему нас выб
  \"css\": \".features {{ padding: 80px 24px; }} .features__inner {{ max-width: 1100px; margin: 0 auto; }} .features__title {{
size: 40px; margin: 0 0 40px; text-align: center; }} .features__grid {{ display: grid; grid-template-
columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 24px; }} .features__card {{ padding: 32px 24px; border-
radius: 16px; background: #f7f7f9; }} @media (max-width: 768px) {{ .features {{ padding: 60px 16px; }} .features__title {{
size: 28px; }} }}\"
}}
```

Верни ОДИН JSON-объект. Начни с {{ и закончи }}. """

```
def build_section_user_message(user_request: str) -> str:
    """User message for a per-section call."""
    text = (user_request or "").strip()
    return f"Запрос на страницу:\n\n{text}"
```

```
# =====
# PHASE 3 – REVIEW ONE SECTION (QA)
# =====
```

```
def build_review_prompt(target_section: str) -> str:
    """
    System prompt for the QA pass over ONE generated section.

    The model gets a section (HTML + CSS) and must find and fix a
    fixed list of common defects. It must NOT redesign, NOT add
    blocks, NOT change the style. Only fix bugs.

    If nothing is wrong, it returns the section unchanged with
    "fixed": [].

    @param target_section the section name (for the JSON echo)
    @returns the full system prompt as a string
    """
    return f"""\
```

Ты – QA-инженер для сгенерированной секции лендинга.

Тебе дают ОДНУ секцию страницы (HTML + CSS) и просьбу найти и исправить типичные дефекты вёрстки.

ЭТО НЕ РЕДИЗАЙН. ЭТО ТОЧЕЧНАЯ ПРАВКА БАГОВ.

НЕ меняй дизайн, НЕ добавляй новые блоки, НЕ переписывай тексты, НЕ вводи новые классы, НЕ меняй палитру.

ИЩИ И ИСПРАВЛЯЙ ТОЛЬКО ЭТИ ДЕФЕКТЫ:

1. КАРТИНКИ ЗА ГРАНИЦАМИ.

- Если в HTML есть или <svg>, а в CSS нет `max-width: 100%;` для них – добавь правило.
Например: `{target\_section} img {{ max-width: 100%; height: auto; }}`
- Если у картинки прописан фиксированный width больше контейнера – замени на `max-width: 100%;`.

2. ПУСТЫЕ КНОПКИ И ССЫЛКИ.
 - `<button></button>` без текста → подставь осмысленный русский текст по смыслу кнопки.
 - `<a href="#"></a>` без текста → либо добавь текст, либо удали.
3. НЕВИДИМЫЙ ТЕКСТ.
 - Если у текстового блока ``color: #fff`` (белый) или ``#ffffff``, а фон секции белый/светлый – поменяй color на тёмный (например, `#1a1a1a`).
 - Если текст и фон совпадают – исправь.
4. ОТСУТСТВУЮЩИЕ @media.
 - Если секция содержит grid/flex с несколькими колонками, но нет ``@media (max-width: 768px)`` – добавь упрощение до 1 колонки для мобильных.
5. ДУБЛИРУЮЩИЕСЯ @import.
 - Оставь только ПЕРВЫЙ, остальные удали.
6. ШИРИНА КОНТЕЙНЕРА.
 - Если у внутреннего контейнера ``width: <число>px`` с большим числом (960, 1100, 1200, 1440) – замени на ``max-width: <число>px; margin: 0 auto; padding: 0 24px;``
7. OVERFLOW.
 - Добавь ``overflow-x: hidden;`` на корневую секцию, чтобы ничего не вылезало за экран.

ФОРМАТ ОТВЕТА – СТРОГО ВАЛИДНЫЙ JSON:

```
{
  "section_name": "{target_section}",
  "html": "<section class=\\\"{target_section}\\\">...</section>",
  "css": ".{target_section} {{ ... }}",
  "fixed": ["images_overflow", "empty_button"],
  "notes": "кратко что исправил"
}
```

ЕСЛИ ДЕФЕКТОВ НЕТ:

Верни исходные html и css ДОСЛОВНО (символ в символ) и ``"fixed": []``. НЕ ПЫТАЙСЯ улучшить то, что уже работает.

ЧЕГО НЕ ДЕЛАТЬ:

- Не добавляй новых секций или блоков внутри секции.
- Не вводи новых CSS-классов без необходимости.
- Не добавляй новых @import.
- Не удаляй существующие @media.
- Не оборачивай ответ в markdown (без `` `` `json` и `` ``).
- Не добавляй пояснений до или после JSON.

Верни ОДИН JSON-объект. Начни с `{{` и закончи `}}`.

"""

```
def build_review_user_message(
    target_section: str,
```

```

    section_html: str,
    section_css: str,
    user_request: str,
) -> str:
    """
    User message for the QA pass.

    @param target_section  section name (echoed for the model)
    @param section_html    the generated HTML to review
    @param section_css     the generated CSS to review
    @param user_request    the original prompt (for context)
    @returns the user message as a string
    """

    parts = []

    parts.append(f"Секция: {target_section}")
    parts.append("")
    parts.append("Исходный запрос пользователя:")
    parts.append((user_request or "").strip() or "(без текста)")
    parts.append("")

    parts.append("--- BEGIN SECTION HTML ---")
    parts.append((section_html or "").strip() or "(пусто)")
    parts.append("--- END SECTION HTML ---")
    parts.append("")

    parts.append("--- BEGIN SECTION CSS ---")
    parts.append((section_css or "").strip() or "(пусто)")
    parts.append("--- END SECTION CSS ---")
    parts.append("")

    parts.append(
        "Проверь секцию по списку дефектов из системного промпта. "
        "Верни JSON."
    )

    return "\n".join(parts)

# =====
# PHASE 4 – EDIT AN EXISTING PAGE
# =====

def build_edit_prompt() -> str:
    """
    System prompt for EDIT mode.

    The model gets the current page HTML + CSS and a request that
    describes what to change. It returns the whole updated page in
    the same {"html": ..., "css": ...} shape as single-shot mode.

    The prompt is emphatic about minimal intervention – deepseek-
    flash likes to "improve" the whole page when asked to tweak a
    single button.

    @returns the full system prompt as a string

```

```
"""
```

```
return f"""\
```

Ты — редактор лендинга.

Тебе дают ТЕКУЩУЮ страницу (HTML + CSS) и запрос на правку.

Ты возвращаешь ОБНОВЛЁННУЮ страницу ЦЕЛИКОМ.

ЭТО НЕ ГЕНЕРАЦИЯ С НУЛЯ. ЭТО ТОЧЕЧНАЯ ПРАВКА.

ФОРМАТ ОТВЕТА — СТРОГО ВАЛИДНЫЙ JSON:

```
{{  
  "html": "<весь HTML страницы>",  
  "css": "<весь CSS страницы>"  
}}
```

ГЛАВНОЕ ПРАВИЛО — МИНИМАЛЬНОЕ ВМЕШАТЕЛЬСТВО:

1. Сохрани ВСЁ, что не просили менять:
 - секции, не упомянутые в запросе — оставь как есть;
 - палитру, шрифты, отступы, скругления — не трогай;
 - тексты, которые не просили менять — оставь дословно.
2. Меняй ТОЛЬКО то, что просит пользователь. Примеры:
 - «поменяй цвет кнопки» → поменяй цвет кнопки, больше ничего;
 - «убери секцию с отзывами» → удали её и связанный CSS;
 - «добавь блок с ценами после features» → добавь новую секцию после features, остальное не трогай;
 - «сделай заголовок крупнее» → измени только размер заголовка.
3. НЕ регенерируй страницу с нуля. Если текущий стиль кажется тебе некрасивым — не трогай его, пока не попросят. Твоя задача — правка, а не улучшение.
4. Не добавляй новых секций, не удаляй существующих, не меняй тексты, если это явно не указано в запросе.

ФОРМАТ:

- Один корневой контейнер `<div class="core-engine-lib-word-blocks">` уже присутствует в HTML. Сохрани его.
- Если в исходном CSS был `@import` шрифта — оставь его первой строкой.

{_SPLIT_RULES}

{_NO_FRAMEWORKS_RULES}

ПРИМЕРЫ:

Запрос: «поменяй цвет кнопки в hero на зелёный»

→ в CSS находишь `.hero__btn` и меняешь `background`. Остальное — дословно как было.

Запрос: «убери секцию Отзывы»

→ в HTML удаляешь `<section class="testimonials">...</section>`, в CSS — все правила `.testimonials*`. Остальное — как было.

Запрос: «добавь после features блок с ценами»

→ в HTML после секции features вставляешь

<section class="pricing">...</section>, в CSS добавляешь
правила .pricing*. Остальное – как было.

Запрос: «сделай заголовок hero крупнее»

→ в CSS увеличиваешь font-size у .hero__title. Остальное –
как было.

Верни ОДИН JSON-объект. Начни с {{ и закончи }}. """

```
def build_edit_user_message(
    user_request: str,
    current_html: str,
    current_css: str,
) -> str:
    """
    User message for the edit call.

    Embeds the current page (HTML + CSS) as reference blocks, then
    the request. The CSS block is skipped entirely if the CSS is
    empty, so the message is not littered with empty headers.

    @param user_request    raw text from the chat
    @param current_html    the page HTML currently on the canvas
    @param current_css     the page CSS currently on the canvas
    @returns the user message as a string
    """
    parts: List[str] = []

    parts.append("ЗАПРОС НА ПРАВКУ:")
    parts.append((user_request or "").strip() or "(без текста)")
    parts.append("")

    parts.append("--- BEGIN CURRENT PAGE HTML ---")
    parts.append((current_html or "").strip() or "(пусто)")
    parts.append("--- END CURRENT PAGE HTML ---")
    parts.append("")

    if current_css and current_css.strip():
        parts.append("--- BEGIN CURRENT PAGE CSS ---")
        parts.append(current_css.strip())
        parts.append("--- END CURRENT PAGE CSS ---")
        parts.append("")

    parts.append(
        "Верни обновлённую страницу ЦЕЛИКОМ: "
        '{"html": "...", "css": "..."}.'
        "Меняй только то, что просит пользователь. "
        "Всё остальное сохрани ДОСЛОВНО."
    )

    return "\n".join(parts)
```

=====

FILE: core/engine/lib/word/llm/prompts/edit_effect.py

```
=====
# neurocad/core/engine/lib/word/llm/prompts/edit_effect.py
```

```
"""
```

Effect edit prompt – rewrite the CSS of ONE existing effect.

Used when the user is in "edit effect" mode (see editor/effects/index.js → trigger 'word:effect-edit-start'). The model receives the current CSS of ONE effect and writes a new full CSS for the same effect, based on the user's request.

The model may ALSO propose a rename (new_id + new_label) – but ONLY if the edit changes the MEANING of the effect (e.g. "звёзды" → "радыга"). For cosmetic edits ("сделай тень сильнее") it must NOT propose a rename, and the client falls back to an in-place PUT.

The response is a JSON object:

```
{
  "css":      ".core-engine-lib-word-blocks .<id> { ... }",
  "new_id":   "fx-new-slug"   (optional, or null),
  "new_label": "Новое название" (optional, or null)
}
```

The prompt is deliberately SHORT and SCHEMA-FIRST: the format block is at the top, the rules are numbered one-liners, the examples are concrete JSON. Long prose makes the model drift and return raw CSS instead of JSON.

No static imports beyond the standard library. This module is pure – it only builds a string.

```
"""
```

```
from typing import List, Optional
```

```
def build_effect_edit_prompt(
    effect_id: str,
    current_css: str,
    effect_label: str = "",
    existing_effect_ids: Optional[List[str]] = None,
) -> str:
    """
```

System prompt for the effect-edit flow.

The model returns a JSON object with the new full CSS and, optionally, a new id / label if the edit changes the meaning of the effect.

@param effect_id	– current id, e.g. "fx-rotating-globe"
@param current_css	– current CSS of the effect
@param effect_label	– current human label (optional, for context)
@param existing_effect_ids	– ids of all OTHER effects, so the model does not pick a taken one when it proposes a rename

```

@returns the full system prompt as a string
"""

existing = [str(x).strip() for x in (existing_effect_ids or []) if str(x).strip()]
existing = [x for x in existing if x != effect_id]
if existing:
    existing_block = "\n".join(f" - {x}" for x in existing)
else:
    existing_block = " (нет других эффектов)"

label_line = f'Текущее название: "{effect_label}".' if effect_label else ""

return f"""Ты – редактор CSS-эффектов для CMS NeuroCad.

```

Пользователь редактирует ОДИН эффект. Ты возвращаешь JSON с новым CSS.

СХЕМА ОТВЕТА – ТОЛЬКО ЭТОТ JSON, БЕЗ MARKDOWN, БЕЗ ПОЯСНЕНИЙ:

```

{{
  "css": ".core-engine-lib-word-blocks .{effect_id} {{ ... }}",
  "new_id": null,
  "new_label": null
}}
```

ПОЛЯ:

"css" – ОБЯЗАТЕЛЬНО. Полный CSS эффекта.
 "new_id" – null, ИЛИ строка вида "fx-..." при смене смысла.
 "new_label" – null, ИЛИ короткое название по-русски (вместе с new_id).

КОНТЕКСТ:

```

id:      {effect_id}
{label_line}
Занятые id (нельзя брать для new_id):
{existing_block}
```

ПРАВИЛА:

1. Внешний селектор CSS: ``.core-engine-lib-word-blocks .{effect_id}``.
2. Не добавляй ``background-color`` и ``color``.
3. Не добавляй ``@import``, ``url(...)``, ``expression(...)``, ``javascript:``.
4. Можно: `background-image`, `box-shadow`, `filter`, `mask`, `@keyframes`, `::before`/`::after` (с тем же префиксом).
5. `new_id` заполняй ТОЛЬКО при смене СМЫСЛА эффекта.
6. Примеры смены смысла:
 - "звёзды" → "радуга" → `new_id: "fx-rainbow-arc"`
 - "точки" → "полосы" → `new_id: "fx-diagonal-stripes"`
 - "уголок" → "рамка" → `new_id: "fx-inner-frame"`
7. Примеры КОСМЕТИКИ (`new_id: null`):
 - "тень сильнее", "круги чаще", "уголок больше".
8. `new_id`: только строчные латинские, цифры, дефисы, префикс ``fx-``.
9. `new_id` не должен быть в списке занятых и не равен текущему `id`.

ПРИМЕРЫ ОТВЕТОВ:

Косметика:

```

{{"css": ".core-engine-lib-word-blocks .{effect_id} {{ box-shadow: inset 0 8px 20px rgba(15,23,42,0.35); }}", "new_id": null}}
```

Смена смысла:

```
{{"css": ".core-engine-lib-word-blocks .fx-light-hatch-strokes {{ background-image: repeating-linear-gradient(112deg, rgba(100,116,139,0.22) 0 2px, transparent 2px 10px); }}", "new_id": "fx-light-hatch-strokes", "new_label": "Тонкие штрихи"}}
```

ТЕКУЩИЙ CSS:

```
{current_css}
```

ОТВЕТ — ТОЛЬКО JSON. Всегда включай поле "css".
"""

```
=====
FILE: core/engine/lib/word/llm/prompts/edit.py
=====
```

```
# neurocad/core/engine/lib/word/llm/agent/edit.py
```

```
"""
```

Agent: edit.

Edits the CSS of ONE existing effect (fx-*) at the user's request.

This agent does NOT touch the page HTML, does NOT add classes to elements, and does NOT pick from a fixed catalog. It writes CSS for a single effect file, replacing its content entirely.

The user is in "effect edit mode": the frontend has already opened the effect for editing and sends the current CSS with every request. The agent returns a new full CSS for the same effect. The frontend applies it live to the iframe (as a <style> override), and only when the user confirms does the frontend PUT it back to the server.

Input:

effect_id	– e.g. "fx-shadow-top-n"
effect_css	– current CSS of that effect (the draft, not the file)
user_message	– what the user wants changed
effect_label	– current human label (optional)
existing_effect_ids	– every id currently registered in the palette (list of strings). The model uses this to avoid proposing an id that is already taken when it decides to rename the effect.

Output:

```
{
  "message": "Готово.",
  "css":      ".core-engine-lib-word-blocks .fx-shadow-top-n { ... }",
  "new_id":   "fx-new-slug" | None,
  "new_label": "Новое название" | None,
}
```

`new\_id` and `new\_label` are OPTIONAL. They are only filled when the edit CHANGES THE MEANING of the effect (e.g. "звёзды" → "радуга"). For cosmetic edits ("сделай тень сильнее") both are None and the client falls back to an in-place PUT.

`new\_id`, when present, must:

- match EFFECT_ID_RE (same pattern as create_effect);

- NOT be present in existing_effect_ids.

If either condition fails, `new\_id` and `new\_label` are dropped and the agent falls back to an in-place edit.

RESPONSE FORMAT TOLERANCE:

The model is ASKED to return JSON {css, new_id, new_label}, but we accept two more shapes as fallbacks:

- (a) raw CSS with the CURRENT id → in-place edit;
- (b) raw CSS with a NEW id → rename (new_id extracted from the CSS selector).

This makes the agent robust to a model that ignores the JSON schema and just returns CSS with a renamed selector. We extract the new id from the selector and validate it against existing_effect_ids, the same as if the model had returned it in a JSON field.

The agent never returns HTML – the element_update path is not used here. The caller (ws.py) sends the result as a separate `css\_update` event, not as `element\_update`.

Safety:

If the model returns something that does not look like CSS for a plausible effect selector, the agent returns css=None. The client then skips the update entirely, so the effect file is not damaged.

```
import json
import re
from typing import Any, Dict, List, Optional, Tuple

from .base import CoreEngineLibWordLlmAgentBase
from ..prompts.edit_effect import build_effect_edit_prompt

#: Same pattern create_effect validates against.
EFFECT_ID_RE = re.compile(r"^fx-[a-z0-9][a-z0-9-]{0,63}$")

#: Extract the FIRST `fx-<slug>` selector from a CSS string.
#: The slug is a sequence of lowercase letters, digits and dashes.
_CSS_FX_SELECTOR_RE = re.compile(r"\.(fx-[a-z0-9][a-z0-9-]{0,63})(?![\w-])")

class CoreEngineLibWordLlmAgentEdit(CoreEngineLibWordLlmAgentBase):
    """Edit the CSS of one existing effect."""

    name = "edit"

    async def run(
        self,
        *,
        provider,
        user_message: str,
        page_id: int,
        run_id: Any,
```



```

emit=None,
selection: Optional[Dict[str, Any]] = None,
block_catalog: Optional[list] = None,
current_html: Optional[str] = None,
history: Optional[list] = None,
effect_id: Optional[str] = None,
effect_css: Optional[str] = None,
effect_label: Optional[str] = None,
existing_effect_ids: Optional[List[str]] = None,
) -> Dict[str, Any]:

# ---- emit: step at the start ----
if emit:
    await emit({
        "type": "step",
        "step": "edit",
        "message": "Готовлю правку эффекта...",
    })

# ---- guard: effect_id and effect_css are required ----
if not effect_id or not effect_css:
    return {
        "message": "Режим правки эффекта не активен.",
        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
    }

# ---- normalize the list of existing ids ----
existing_ids = [
    str(x).strip()
    for x in (existing_effect_ids or [])
    if isinstance(x, (str, int)) and str(x).strip()
]
existing_ids = [x for x in existing_ids if x != effect_id]

# ---- build system prompt from the current CSS + id + list ----
system_prompt = build_effect_edit_prompt(
    effect_id=effect_id,
    current_css=effect_css,
    effect_label=effect_label or "",
    existing_effect_ids=existing_ids,
)

# ---- user content: just the request ----
user_content = user_message.strip()

# ---- dump prompt ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name="edit",
        system_prompt=system_prompt,
        user_content=user_content,

```

```

        history=history,
    )
except Exception as e:
    print(f"[edit] dump request failed: {e}", flush=True)

# ---- build messages ----
messages = [{"role": "system", "content": system_prompt}]
if history:
    for m in history:
        role = m.get("role")
        content = m.get("content")
        if role in ("user", "assistant") and content:
            messages.append({"role": role, "content": str(content)})
messages.append({"role": "user", "content": user_content})

# ---- provider call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[edit] LLM error: {e}", flush=True)
    return {
        "message": f"⚠ Ошибка LLM: {e}",
        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
    }

# ---- dump response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="edit",
        raw_response=raw,
    )
except Exception as e:
    print(f"[edit] dump response failed: {e}", flush=True)

# ---- parse: css + optional new_id / new_label ----
# This handles three shapes:
# 1) JSON {css, new_id, new_label}
# 2) raw CSS with the CURRENT id
# 3) raw CSS with a NEW id (extracted from the selector)
css, parsed_new_id, parsed_new_label = self._parse_response(
    raw, effect_id
)

if not css:
    print(
        f"[edit] model returned a non-css answer: "
        f"{str(raw)[:160]}!", flush=True,
    )
    return {
        "message": "⚠ Модель вернула некорректный ответ. Попробуйте ещё раз.",

```

```

        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
    }

# ---- validate optional new_id / new_label ----
new_id, new_label = self._validate_new_id(
    parsed_new_id,
    parsed_new_label,
    effect_id,
    existing_ids,
)

return {
    "message": "Готово.",
    "css": css,
    "effect_id": effect_id,
    "new_id": new_id,
    "new_label": new_label,
}

# =====
# PARSE
# =====

@classmethod
def _parse_response(
    cls,
    text: str,
    current_id: str,
) -> Tuple[str, Optional[str], Optional[str]]:
    """
    Extract (css, new_id, new_label) from the model response.

    Handles three response shapes:

    A. JSON object with a "css" field (and optional new_id /
       new_label fields) – the shape we ask for.

    B. Raw CSS that uses the CURRENT id in its outer selector
       – no rename. new_id = None.

    C. Raw CSS that uses a NEW id in its outer selector – we
       extract the new id from the selector and return it as
       new_id, so the client can do the rename.

    Returns (css, new_id, new_label). css is "" if nothing usable
    was found.
    """
    if not text:
        return ("", None, None)

    s = text.strip()

# ---- strip markdown fence ----

```

```

if s.startswith("`"):
    nl = s.find("\n")
    if nl != -1:
        s = s[nl + 1:]
    if s.endswith("`"):
        s = s[:-3]
    s = s.strip()

# ---- try JSON with a "css" field ----
try:
    data = json.loads(s)
    if isinstance(data, dict):
        css = data.get("css")
        if css:
            return (
                str(css).strip(),
                data.get("new_id"),
                data.get("new_label"),
            )
except Exception:
    pass

# ---- try the first {...} JSON block ----
start = s.find("{")
end = s.rfind("}")
if start != -1 and end != -1 and end > start:
    head = s[start:start + 40]
    if "'" in head:
        try:
            data = json.loads(s[start:end + 1])
            if isinstance(data, dict):
                css = data.get("css")
                if css:
                    return (
                        str(css).strip(),
                        data.get("new_id"),
                        data.get("new_label"),
                    )
        except Exception:
            pass

# ---- fallback: raw CSS ----
# Verify it looks like CSS: contains braces AND at least one
# ` .fx-<slug>` selector.
if "{" not in s or "}" not in s:
    return ("", None, None)

m = _CSS_FX_SELECTOR_RE.search(s)
if not m:
    return ("", None, None)

selector_slug = m.group(1)

# Case B: raw CSS with the CURRENT id → in-place edit.
if selector_slug == current_id:
    return (s, None, None)

```

```

# Case C: raw CSS with a NEW id → rename. The new label
# defaults to None (the client falls back to using the new
# id as a label).
return (s, selector_slug, None)

# =====
# VALIDATION
# =====

@staticmethod
def _validate_new_id(
    raw_new_id: Any,
    raw_new_label: Any,
    current_id: str,
    existing_ids: List[str],
) -> Tuple[Optional[str], Optional[str]]:
    """
    Sanitize the optional new id / label proposed by the model.

    Returns (new_id, new_label), or (None, None) if:
    - new_id is missing or empty;
    - new_id equals current_id (no actual rename);
    - new_id does not match EFFECT_ID_RE;
    - new_id is in existing_ids (would collide with another
      effect already registered in the palette).

    In every rejection case the caller falls back to an in-place
    PUT – the user keeps their current effect name.
    """
    if not raw_new_id:
        return (None, None)

    new_id = str(raw_new_id).strip()
    if not new_id:
        return (None, None)

    if new_id == current_id:
        print(
            f"[edit] model proposed the same id as current "
            f"({new_id}) – ignoring rename",
            flush=True,
        )
        return (None, None)

    if not EFFECT_ID_RE.match(new_id):
        print(
            f"[edit] model proposed an invalid new_id "
            f"({new_id!r}) – ignoring rename",
            flush=True,
        )
        return (None, None)

    if new_id in existing_ids:
        print(
            f"[edit] model proposed an already-taken new_id "

```

```

        f"({new_id!r}) – ignoring rename. existing: {existing_ids}",
        flush=True,
    )
    return (None, None)

new_label = (str(raw_new_label).strip() if raw_new_label else "")
if not new_label:
    # No label from the model – fall back to the new id so
    # the palette still has something readable.
    new_label = new_id
if len(new_label) > 80:
    new_label = new_label[:77] + "..."

return (new_id, new_label)

```

```

=====
FILE: core/engine/lib/word/llm/prompts/effects.py
=====

```

```

# neurocad/core/engine/lib/word/llm/prompts/effects.py

```

```

"""

```

Step 3 of the three-step page-building flow: EFFECTS.

The model does NOT write CSS. It receives the assembled page HTML and a catalog of effects that are actually available in the editor's palette (loaded from registry.json). It adds ONE of the ready-made `fx-\*` classes to a target element.

The catalog is now passed IN, instead of being hard-coded here. This means:

- adding a new effect = editing fx/*.css + registry.json – no change to this module;
- the model only ever sees effects that are REALLY available; if the palette has 8 effects, the prompt lists exactly those 8.

If the caller passes an empty / missing catalog (should not happen in normal flow), we fall back to a small hard-coded set so the prompt still makes sense.

No static imports beyond the standard library. This module is pure – it only builds a string.

```

"""

```

```

from typing import Any, Dict, List, Optional

```

```

# =====
# FALLBACK CATALOG
# =====
#
# Used ONLY when the caller does not pass a real catalog (e.g. an
# error loading registry.json). In normal flow the caller passes the
# live list from CoreEngineLibWordEffectsService.list_effects().

```

```

FALLBACK_EFFECTS: List[Dict[str, Any]] = [

```

```

    {"id": "fx-shadow-top-n", "label": "Тень сверху", "hint": "Мягкая тень у верхнего края"},
    {"id": "fx-corner-tl-n", "label": "Угол сверху-слева", "hint": "Акцентный уголок в левом верхнем углу"},
    {"id": "fx-corner-tr-n", "label": "Угол сверху-справа", "hint": "Акцентный уголок в правом верхнем углу"},
    {"id": "fx-circles-n", "label": "Круги", "hint": "Концентрические круги в центре"},
]

```

```

def _render_catalog(effects: Optional[List[Dict[str, Any]]]) -> str:

```

```

    """

```

```

    Turn a list of effect dicts into a bullet list for the prompt.

```

```

    Each line is:

```

```

    - `fx-<id>` – <label>: <hint>

```

```

    If `effects` is empty or missing, returns FALLBACK_EFFECTS.

```

```

    """

```

```

    src = effects if effects else FALLBACK_EFFECTS

```

```

    lines: List[str] = []

```

```

    for e in src:

```

```

        eid = str(e.get("id", "")).strip()

```

```

        if not eid:

```

```

            continue

```

```

        label = str(e.get("label", eid)).strip()

```

```

        hint = str(e.get("hint", "")).strip()

```

```

        if hint and hint != label:

```

```

            lines.append(f"- `{eid}` – {label}: {hint}")

```

```

        else:

```

```

            lines.append(f"- `{eid}` – {label}")

```

```

    if not lines:

```

```

        lines.append("(нет доступных эффектов)")

```

```

    return "\n".join(lines)

```

```

def build_effects_prompt(

```

```

    effects: Optional[List[Dict[str, Any]]] = None,

```

```

) -> str:

```

```

    """

```

```

    System prompt for step 3 (effects).

```

```

    The model does NOT write CSS. It receives the assembled page HTML
    and a catalog of effects that are actually available in the
    editor's palette. It adds ONE of the ready-made `fx-*` classes to
    a target element.

```

```

    @param effects – list of {id, label, hint, ...} dicts from

```

```

        CoreEngineLibWordEffectsService.list_effects().

```

```

        If None or empty, FALLBACK_EFFECTS is used.

```

```

    @returns the full system prompt as a string

```

```

    """

```

```

    catalog_block = _render_catalog(effects)

```

```

    return f"""Ты – дизайнер для CMS NeuroCad.

```

```

    Тебе дана готовая HTML-страница, собранная из блоков, и список

```

доступных фоновых эффектов. Твоя задача – применить эффекты к секциям страницы.

ФОРМАТ ОТВЕТА – ТОЛЬКО ВАЛИДНЫЙ JSON:

```
{{"html": "<section class=\\\"section hero\\\">...</section>"}}
```

ДОСТУПНЫЕ ЭФФЕКТЫ (классы из effects.css):

```
{catalog_block}
```

ПРАВИЛА:

1. Используй ТОЛЬКО классы из списка выше. Не изобретай свои.
2. Применяй эффекты К СЕКЦИЯМ (тег <section>), а не к внутренним элементам (h2, p, div.container). Эффекты – фоновые, они рассчитаны на всю секцию.
3. Эффекты можно применять РАЗНЫЕ к разным секциям. Не бойся использовать 2–3 разных эффекта на одной странице, если секций несколько.
4. НЕ добавляй <style>-блоки. НЕ добавляй inline-стили (style="..."). Эффекты уже определены в effects.css.
5. Структура HTML не меняется. Только добавляется класс к тегу <section>.
6. Не более ОДНОГО эффекта `fx-\*` на одну секцию. Если у секции уже есть класс `fx-\*` – замени его, а не добавляй второй.
7. Не добавляй эффект, если у секции есть свой фон (класс с background или явный `style="background..."`) – иначе фон сломается.
8. Если на странице ровно одна секция – примени один эффект.
9. Если пользователь не просит эффекты – верни HTML без изменений.
10. Не добавляй пояснения до или после JSON.

ПРИМЕРЫ:

Запрос: «Сделай фон секции в точку»

Было: <section class="section hero">...</section>

Стало: <section class="section hero fx-dots-sm-n">...</section>

Запрос: «Добавь шахматный фон карточке»

Было: <div class="card features__item">...</div>

Стало: <div class="card features__item fx-checker-n">...</div>

Запрос: «Сделай фон с волнами»

Было: <section class="section cta">...</section>

Стало: <section class="section cta fx-waves-n">...</section>

Запрос: «Добавь рамку блоку с контактами»

Было: <section class="section contacts">...</section>

Стало: <section class="section contacts fx-border-inset-n">...</section>

ГЛАВНОЕ ПРАВИЛО – ДЕЛАЙ РОВНО ТО, ЧТО ПРОСЯТ:

11. Не расширяй задачу. «Сделай фон в точку» = ОДИН класс `fx-dots-sm-n` на секции. Больше ничего.
12. Если непонятно, к какому элементу применить – не добавляй эффект вообще. Лучше ничего, чем эффект не на том месте.

13. Если подходящего эффекта нет в списке – не добавляй ничего.
 Не придумывай свой класс.
 """

```
=====
FILE: core/engine/lib/word/llm/prompts/fill.py
=====
```

```
# neurocad/core/engine/lib/word/llm/prompts/fill.py
```

"""

Step 2 of the three-step page-building flow: FILL.

The model receives a set of blocks (with their HTML) and fills in text, alt attributes, and links according to the user's request. It does NOT change structure or classes – EXCEPT in one specific case: when a block from the "Элементы" category arrives as a bare element (`<h2>...</h2>`, `<p>...</p>`, `<a>...</a>`, `<img>`, ...), it must be wrapped in a section, otherwise it ends up floating in <body> with no spacing.

Two builders live here:

```
build_fill_prompt()
    System prompt: rules for filling, expected JSON shape.

build_fill_user_message(user_message, chunk)
    User-side message: the original user request + the HTML of
    the blocks in `chunk` (a subset that fits into one request).
```

Response shape expected from the model:

```
{
  "filled": [
    {"block_id": "core-hero", "html": "<section ...>...</section>"},
    {"block_id": "core-features", "html": "<section ...>...</section>"}
  ],
  "remaining": []
}
```

No static imports beyond the standard library. This module is pure – it only builds strings.
 """

```
from typing import Any, Dict, List
```

```
def build_fill_prompt() -> str:
```

"""

System prompt for step 2 (filling).

The model receives a set of blocks with their HTML and fills in text, alts, and links. It does NOT change structure or classes – except for wrapping bare elements in a section (see rule 10).

@returns the full system prompt as a string

"""

```
return """Ты – редактор контента для CMS NeuroCad.
```

Тебе даны готовые HTML-блоки. Ты заполняешь их текстом, alt'ами и ссылками в соответствии с запросом пользователя.

ФОРМАТ ОТВЕТА – ТОЛЬКО ВАЛИДНЫЙ JSON:

```
{
  "filled": [
    {"block_id": "core-hero", "html": "<section ...>...</section>"},
    {"block_id": "core-features", "html": "<section ...>...</section>"}
  ],
  "remaining": []
}
```

ПРАВИЛА:

1. НЕ меняй структуру HTML и классы. Только тексты, alt, href.
2. Для каждой картинки с placeholder поставь осмысленный alt. Саму картинку оставь как есть (src не меняй).
3. Если пользователь просит ссылки на другие страницы – используй `href="page:slug"` (например, `href="page:about"`).
4. Не добавляй inline-стили (`style="..."`) и не вставляй `

```

<section class="section">
  <div class="container">
    <a href="#" class="btn" data-block="core-btn">Кнопка</a>
  </div>
</section>

```

11. Если корневой тег блока УЖЕ секционный контейнер (`<section>`, `<article>`, `<header>`, `<footer>`, `<aside>`, `<main>`, `<nav>`) – ОСТАВЬ его как есть. Не добавляй вторую обёртку.

Было (block_id: core-section):

```

<section class="section" data-block="core-section">...</section>

```

Стало:

```

<section class="section" data-block="core-section">...</section>
(без изменений)

```

12. Внутренние классы, атрибуты и вложенность – НЕ трогай. Обёртка добавляется СНАРУЖИ исходного элемента.

"""

```

def build_fill_user_message(
    user_message: str,
    chunk: List[Dict[str, Any]],
) -> str:
    """
    User-side message for step 2 (filling).

    The message starts with the original user request, then lists the
    blocks in `chunk` – each preceded by a `=== <block_id> ===` line,
    followed by the block's HTML.

    @param user_message – the user's original request as a string
    @param chunk         – list of {block_id, html} dicts that fit in
                           one request
    @returns the full user-side message as a string
    """
    parts: List[str] = [f"Запрос пользователя: {user_message}", ""]

    for item in chunk:
        bid = item.get("block_id", "?")
        html = item.get("html", "")
        parts.append(f"=== {bid} ===")
        parts.append(html)
        parts.append("")

    return "\n".join(parts).rstrip()

```

```

=====
FILE: core/engine/lib/word/llm/prompts/legacy.py
=====

```

```

# neurocad/core/engine/lib/word/llm/prompts/legacy.py

```

"""

Legacy prompts – kept for reference only.

This module holds prompt strings that are NOT used by any current code path. They were part of the original single-file `prompt.py` before the split into `prompts/`. They are kept here so that:

- the history of the prompts is not lost when someone reads the package;
- a future caller that wants the old single-step editor can still import it without resurrecting the whole original file.

Nothing in this module is imported by the rest of the package, and nothing in the codebase currently calls these strings. If you are sure they will never be needed, this file can be deleted.

Contents:

HTML_EDITOR_SYSTEM_PROMPT – the original single-step HTML editor prompt (pre-plan/fill/effects). It was replaced by the three-step flow (plan → fill → effects) and is no longer referenced.

"""

```
# =====  
# SINGLE-STEP PROMPT (legacy, kept for reference)  
# =====
```

HTML_EDITOR_SYSTEM_PROMPT = """Ты – редактор HTML для CMS NeuroCad.

Пользователь даёт текущий HTML-фрагмент страницы и запрос на русском.
Ты возвращаешь JSON-объект с двумя полями:

- "message": короткое сообщение пользователю (1 предложение).
- "html": изменённый HTML-фрагмент.

ФОРМАТ ОТВЕТА:

1. Ответ – ТОЛЬКО валидный JSON. Начинается с `{`, заканчивается `}`.
2. НЕ оборачивай в markdown (без ``json, без ``).
3. НЕ добавляй пояснения до или после JSON.
4. Поле "message" – короткое, нейтральное: «Готово», «Изменения применены».
5. Поле "html" – ТОЛЬКО HTML. Без `<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`.

СБОРКА СТРАНИЦЫ – СТРОГИЕ ПРАВИЛА:

6. Используй только классы, которые уже встречаются в текущем HTML.
Не изобретай новые классы.
7. ЗАПРЕЩЕНО добавлять inline-стили (`style="..."`).
8. Для уникальных визуальных эффектов используй `<style>`-блок с уникальным префиксом, например `effect-a3f7``:
 - добавь префикс как класс к целевому блоку,
 - в конце блока вставь `<style>`, каждое правило начинается с префикса,
 - внутри `<style>` можно использовать `var(--theme-*)`` и медиа-запросы.
9. Если HTML пустой – создай новый по запросу.
10. Если HTML непустой – измени его минимально.
11. Если не понял запрос – верни исходный HTML без изменений,
message: «Не понял запрос, попробуйте переформулировать».

"""

```

__all__ = ["HTML_EDITOR_SYSTEM_PROMPT"]

=====
FILE: core/engine/lib/word/llm/prompts/logo.py
=====
# neurocad/core/engine/lib/word/llm/prompts/logo.py

"""
Logo prompt for the "generate logo" flow.

The model receives a page title and (optionally) a short description,
and returns ONE inline `` logo – a compact symbol, not an
illustration. Used by `llm/agent/generate_logo.py`.

Style contract (baked into the prompt):
- square viewBox "0 0 128 128";
- flat, 2–4 colors, no gradients, no shadows, no filters;
- palette drawn from CSS variables with hard-coded fallbacks
  (so the logo still works when a theme variable is missing);
- one recognizable symbol – no background, no frame, no text;
- no <script>, no event handlers, no javascript:, no data:image;
- self-contained, one line, double quotes.

No static imports. This module is pure – it only builds a string.
"""

def build_logo_prompt() -> str:
    """
    System prompt for one logo.

    The user-side message (composed by the caller) carries:
    - the page title,
    - the page description (optional),
    - the alt text (same as title, for context).

    @returns the full system prompt as a string
    """
    return """Ты – дизайнер логотипов для CMS NeuroCad.

Тебе дают заголовок и краткое описание статьи. Ты рисуешь SVG-логотип
(иконку-символ), который подходит этой статье по смыслу.

ФОРМАТ ОТВЕТА – ТОЛЬКО ВАЛИДНЫЙ JSON:
{
  "svg": "<svg viewBox=\\\"0 0 128 128\\\" xmlns=\\\"http://www.w3.org/2000/svg\\\">...</svg>"
}

СОДЕРЖИМОЕ SVG:
1. Тег <svg> ОДИН. Без вложенных <svg>, без <symbol>, без <use>,
   без внешних ссылок.
2. viewBox ОБЯЗАТЕЛЬНО "0 0 128 128" (квадрат). Атрибуты width и
   height НЕ указывай – размер задаётся вёрсткой.
3. Формат – плоский (flat), без градиентов, без теней, без
   фильтров. 2–4 цвета максимум.
4. Цвета бери из CSS-переменных темы с числовым fallback, например:

```

```
fill="var(--theme-accent, #3b82f6)"
fill="var(--theme-text-muted, #94a3b8)"
fill="var(--theme-bg-soft, #f1f5f9)"
stroke="var(--theme-text, #1e293b)"
```

Fallback-цвет подставляй всегда: если переменная не задана, SVG должен всё равно выглядеть осмысленно.

5. Один смысловой объект – символ, иконка. НЕ рисуй фон, рамку, полосы, подложку. Логотип должен читаться на прозрачном фоне.
6. Текст внутри SVG НЕ рисуй. Логотип – только символ.
7. Никаких `<script>`, никаких `onload / onclick`, никакого `javascript:`, никаких `data:image/png` и прочего растра, никаких `http`-ссылок.
8. Тег `<svg>` и весь контент – ОДНОЙ СТРОКОЙ, без переводов строк внутри. Кавычки – двойные.

КОМПОЗИЦИЯ:

9. Главный объект – в центре кадра. Он занимает примерно 70% площади, оставляя поля по краям.
10. Силуэт должен быть УЗНАВАЕМ по заголовку: если статья про кино – камера; если про природу – дерево или лист; если про строительство – дом или инструмент; если про обучение – книга или карандаш.
11. Избегай сложных деталей: только простые фигуры (круги, прямоугольники, линии).

ПРИМЕРЫ:

- заголовок «История кино» → кинокамера.
- заголовок «Родовое поместье» → дом с деревом.
- заголовок «Сад и огород» → росток или лист.
- заголовок «Нейрокад» → абстрактный символ ИИ.

ГЛАВНОЕ ПРАВИЛО:

12. Логотип должен быть УЗНАВАЕМ по заголовку.
13. Если не получается нарисовать объект целиком – рисуй минимальный узнаваемый силуэт.
14. Ответ – ТОЛЬКО валидный JSON. НЕ оборачивай в `markdown`. НЕ добавляй пояснения до или после JSON.

"""

```
=====
FILE: core/engine/lib/word/llm/prompts/plan.py
=====
```

```
# neurocad/core/engine/lib/word/llm/prompts/plan.py
```

"""

Step 1 of the three-step page-building flow: PLAN.

Given a catalog of available blocks (id + label + category, no HTML), the model picks which blocks to use and in what order, and returns a JSON object with a "plan" array:

```
{
  "plan": [
    {"block_id": "core-hero",    "purpose": "Главный баннер"},
    {"block_id": "core-features", "purpose": "Ключевые преимущества"},
    ...
  ]
}
```

```
]
}
```

Each item carries a short "purpose" hint that the next step (fill) uses to decide what kind of content goes into the block.

Layout blocks (grids, containers, flex-shell) are EXCLUDED from the catalog before the prompt is built – the model must not use them. Composition is done with ready-made sections that already contain their own grids (core-features, core-steps, core-gallery, ...).

No static imports beyond the standard library. This module is pure – it only builds a string.

```
"""
```

```
from typing import Any, Dict, List, Optional
```

```
# =====
# LLM-VISIBLE FILTER
# =====
```

```
#: Categories that must NOT be shown to the model at the planning step.
#: "Разметка" – grids, containers, flex-shell: they are for manual
#: assembly in the editor only; the LLM composes pages from ready-made
#: sections.
```

```
_EXCLUDED_CATEGORIES = {"Разметка"}
```

```
#: Explicit block ids to exclude even if their category changes.
#: Belt-and-suspenders: if someone renames the category or moves a
#: block, the id-based filter still catches it.
```

```
_EXCLUDED_BLOCK_IDS = {
    "core-container",
    "core-grid-2",
    "core-grid-3",
    "core-grid-4",
    "core-grid-auto",
    "core-flex-shell",
}
```

```
def _filter_for_llm(
    block_catalog: Optional[List[Dict[str, Any]]],
) -> List[Dict[str, Any]]:
    """
```

```
    Remove layout blocks from the catalog before the prompt is built.
```

```
    Excludes by category (primary) and by explicit block id (fallback).
    Returns a new list – the input is not mutated.
    """
```

```
    if not block_catalog:
        return []
```

```
    out: List[Dict[str, Any]] = []
    for b in block_catalog:
        cat = b.get("category") or ""
```

```

        bid = b.get("id") or ""
        if cat in _EXCLUDED_CATEGORIES:
            continue
        if bid in _EXCLUDED_BLOCK_IDS:
            continue
        out.append(b)
    return out

```

```

def build_plan_prompt(
    block_catalog: Optional[List[Dict[str, Any]]] = None,
) -> str:
    """
    System prompt for step 1 (planning).

    The block catalog is embedded WITHOUT HTML – only id, label and
    category. The model chooses which blocks to use and in what
    order, and returns a JSON object with a "plan" array.

    Layout blocks (category "Разметка" + the known layout ids) are
    filtered out here, so the model never sees them.

    Each plan item:
        {"block_id": "core-hero", "purpose": "Главный баннер"}

    @param block_catalog – list of dicts with keys:
        id      (str)
        label   (str)
        category (str) – optional, defaults to "other"
    @returns the full system prompt as a string
    """
    # ---- Filter out layout blocks (never shown to the model) ----
    block_catalog = _filter_for_llm(block_catalog)

    if not block_catalog:
        catalog_block = "(каталог блоков пуст)"
    else:
        # Group by category, keep the order of first appearance
        by_cat: Dict[str, List[Dict[str, Any]]] = {}
        for b in block_catalog:
            cat = b.get("category") or "other"
            by_cat.setdefault(cat, []).append(b)

        lines: List[str] = []
        for cat, blocks in by_cat.items():
            lines.append(f"### {cat}")
            for b in blocks:
                bid = b.get("id", "?")
                label = b.get("label", bid)
                lines.append(f"- `{bid}` – {label}")
            lines.append("")
        catalog_block = "\n".join(lines).rstrip()

    return f"""Ты – архитектор страницы для CMS NeuroCad.

```

Пользователь описывает, какую страницу он хочет.

Ты выбираешь подходящие блоки из каталога и расставляешь их в нужном порядке. HTML блоков ты не видишь — только их названия и назначение.

ФОРМАТ ОТВЕТА — ТОЛЬКО ВАЛИДНЫЙ JSON:

```
{{
  "plan": [
    {"block_id": "core-hero", "purpose": "Главный баннер"},
    {"block_id": "core-features", "purpose": "Ключевые преимущества"},
    ...
  ]
}}
```

ПРАВИЛА:

1. Используй ТОЛЬКО те block_id, которые есть в каталоге ниже.
2. Порядок в plan = порядок блоков на странице (сверху вниз).
3. Поле "purpose" — одна короткая фраза (до 60 символов), зачем этот блок на этой странице. Это подсказка для следующего шага.
4. Не добавляй пояснения до или после JSON.
5. Не оборачивай в markdown.
6. Если запрос не про создание/изменение страницы — верни пустой plan.
7. Блоки категории «Разметка» (сетки, контейнеры, flex-shell) использовать НЕЛЬЗЯ. Собирай страницу только из готовых секций и элементов. Если для композиции нужна сетка — используй секции, которые уже включают её внутри (например, core-features, core-steps, core-gallery).

КАТАЛОГ БЛОКОВ:

```
{catalog_block}
"""
```

```
=====
FILE: core/engine/lib/word/llm/prompts/rename_effect.py
=====
```

```
# neurocad/core/engine/lib/word/llm/prompts/rename_effect.py
```

```
"""
```

Effect rename prompt — propose a NEW label and SVG miniature for an existing effect. The id and CSS are NOT changed.

Used when the user clicks the "pencil" (rename) button on an active effect block (see llm/agent/rename.py). The model is asked to look at the current CSS and propose:

- a new label (short Russian name)
- a new SVG miniature (24x24 icon that reflects the effect)

The `id` and the CSS are intentionally left alone: the effect keeps its class `fx-<id>` and its .css file. Only the human-readable name in the palette and the icon are updated. This is the safest possible "rename" — nothing on the canvas can break because the selector does not change.

Response shape:

```
{
```

```

    "new_label": "Короткое название",
    "new_media": "<svg viewBox='0 0 24 24'>...</svg>"
}

```

The prompt is deliberately SCHEMA-FIRST and explicitly tells the model NOT to think about CSS rewriting or picking a new id – that is a separate flow (see `edit_effect.py` and `create_effect.py`). Long prose makes the model drift; the short form keeps it focused on label and icon.

No static imports beyond the standard library. This module is pure – it only builds a string.

```

from typing import List, Optional

```

```

def build_effect_rename_prompt(
    effect_id: str,
    current_css: str,
    effect_label: str = "",
    existing_effect_ids: Optional[List[str]] = None,
) -> str:
    """

```

System prompt for the effect-rename flow.

The model returns a JSON object with a new label and a new SVG miniature. The `id` and the CSS are NOT to be changed – the client reuses them as-is.

```

@param effect_id          – current id, e.g. "fx-rotating-globe"
                           (kept for context only; NOT changed)
@param current_css        – current CSS of the effect (read-only)
@param effect_label       – current human label (optional, for context)
@param existing_effect_ids – unused in this flow; kept in the
                           signature for API symmetry with the
                           other prompt builders

@returns the full system prompt as a string
"""
label_line = f'Текущее название: "{effect_label}".' if effect_label else ""

return f"""Ты – редактор названий CSS-эффектов для CMS NeuroCad.

```

Задача: придумать для эффекта НОВОЕ короткое русское название (label) и НОВУЮ SVG-иконку (media) на основе того, что делает его CSS.

ВАЖНО:

- id эффекта НЕ меняется. Он остаётся `{effect\_id}`.
- CSS НЕ меняется. Он остаётся как есть – его вернёт клиент.
- От тебя нужны РОВНО два поля: `new_label` и `new_media`.

СХЕМА ОТВЕТА – ТОЛЬКО ЭТОТ JSON, БЕЗ MARKDOWN, БЕЗ ПОЯСНЕНИЙ:

```

{{
  "new_label": "...",

```

```
"new_media": "<svg viewBox='0 0 24 24'>...</svg>"
}}
```

КОНТЕКСТ:

```
id эффекта (не меняется): {effect_id}
{label_line}
```

ПРАВИЛА ДЛЯ new_label:

1. Короткое название по-русски (до 80 символов).
2. Отражает СУТЬ эффекта – что пользователь видит на экране.
3. Без кавычек, без лишних символов.

ПРАВИЛА ДЛЯ new_media (SVG-иконка):

4. viewBox ОБЯЗАТЕЛЬНО `0 0 24 24`. Атрибуты width и height НЕ указывай.
5. Иконка отражает ВИЗУАЛЬНУЮ СУТЬ CSS:
 - радиальный градиент с кольцами → рисуем кольца;
 - линейный диагональный градиент → рисуем диагональные полосы;
 - тень → рисуем мягкую полосу тени;
 - точки → рисуем точки.
6. Стил – как в остальных эффектах: светло-серый фон `#f1f5f9` с закруглением `rx="3"`, узор – `#cbd5e1` или `#94a3b8`.
7. SVG должен быть валидным XML, ОДНОЙ СТРОКОЙ, без переводов строк внутри. Кавычки – двойные.

ПРИМЕР ОТВЕТА:

```
{{
  "new_label": "Тонкая штриховка медленно дрейфует",
  "new_media": "<svg viewBox='0 0 24 24'><rect x='2' y='2' width='20' height='20' rx='3' fill='white' stroke='black' stroke-width='1'><line x1='2' y1='8' x2='22' y2='4'><line x1='2' y1='14' x2='22' y2='10'></svg>"
}}
```

ТЕКУЩИЙ CSS ЭФФЕКТА (только для анализа, НЕ переписывай):

```
{current_css}
```

ОТВЕТ – ТОЛЬКО JSON. Поля "new_label" и "new_media" обязательны.

```
"""
```

```
=====
FILE: core/engine/lib/word/llm/prompts/rename.py
=====
```

```
# neurocad/core/engine/lib/word/llm/agent/rename.py
```

```
"""
```

```
Agent: rename.
```

Proposes a NEW id, label AND SVG miniature for an existing effect, based on its current CSS. Does NOT change the CSS – the returned CSS is the same as the input.

This is a specialised variant of the `edit` agent, triggered from the UI when the user clicks the "pencil" (rename) button on an active effect block. Unlike `edit`, this agent does not ask the LLM to rewrite anything – it only asks for a name and an icon.

Input:

```
effect_id          - e.g. "fx-rotating-globe"
effect_css          - current CSS (returned unchanged)
user_message       - system-generated, but kept for logging
effect_label       - current human label (optional)
existing_effect_ids - ids of all OTHER effects, so the model does
                    not pick a taken one
```

Output:

```
{
  "message": "Готово.",
  "css":     "<the same CSS we received>",
  "effect_id": "<the same id we received>",
  "new_id":   "fx-...",
  "new_label": "Короткое название",
  "new_media": "<svg viewBox='0 0 24 24'>...</svg>",
}
```

Safety:

- `new\_id` must match EFFECT_ID_RE and must NOT be in `existing\_effect\_ids`. If either condition fails, the agent returns `css=None` so the client skips the update entirely.
- `new\_label` must be non-empty. If it is empty, the agent falls back to using `new\_id` as the label.
- `new\_media` must look like an SVG (contains "<svg"). If it does not, the agent returns an empty string, and the client falls back to reusing the OLD miniature instead.

The agent never touches the page HTML.

"""

```
import json
import re
from typing import Any, Dict, List, Optional, Tuple

from .base import CoreEngineLibWordLlmAgentBase
from ..prompts.rename_effect import build_effect_rename_prompt
```

```
#: Same pattern create_effect and edit validate against.
EFFECT_ID_RE = re.compile(r"^fx-[a-z0-9][a-z0-9-]{0,63}$")
```

```
#: A very cheap sanity check for the SVG media. The prompt asks for
#: a 24x24 <svg> one-liner; we only need to be sure it's an SVG at
#: all – the client will fall back to the old miniature if not.
_SVG_HINT_RE = re.compile(r"<\s*svg\b", re.IGNORECASE)
```

```
class CoreEngineLibWordLlmAgentRename(CoreEngineLibWordLlmAgentBase):
    """Propose a new id/label/media for an existing effect."""
```

```
    name = "rename"
```

```
    async def run(
        self,
        *,
```

```

provider,
user_message: str,
page_id: int,
run_id: Any,
emit=None,
selection: Optional[Dict[str, Any]] = None,
block_catalog: Optional[list] = None,
current_html: Optional[str] = None,
history: Optional[list] = None,
effect_id: Optional[str] = None,
effect_css: Optional[str] = None,
effect_label: Optional[str] = None,
existing_effect_ids: Optional[List[str]] = None,
) -> Dict[str, Any]:

# ---- emit: step at the start ----
if emit:
    await emit({
        "type": "step",
        "step": "rename",
        "message": "Придумываю новое название...",
    })

# ---- guard ----
if not effect_id or not effect_css:
    return {
        "message": "Режим переименования не активен.",
        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
        "new_media": None,
    }

# ---- normalize the list of existing ids ----
existing_ids = [
    str(x).strip()
    for x in (existing_effect_ids or [])
    if isinstance(x, (str, int)) and str(x).strip()
]
existing_ids = [x for x in existing_ids if x != effect_id]

# ---- system prompt ----
system_prompt = build_effect_rename_prompt(
    effect_id=effect_id,
    current_css=effect_css,
    effect_label=effect_label or "",
    existing_effect_ids=existing_ids,
)

user_content = (user_message or "").strip() or (
    "Придумай новое имя, id и иконку для этого эффекта, CSS не меняй."
)

# ---- dump prompt ----
try:

```

```

        from ..dumper import CoreEngineLibWordLlmDumper
        CoreEngineLibWordLlmDumper.dump_step(
            run_id=run_id,
            agent_name="rename",
            system_prompt=system_prompt,
            user_content=user_content,
            history=history,
        )
    except Exception as e:
        print(f"[rename] dump request failed: {e}", flush=True)

# ---- build messages ----
messages = [{"role": "system", "content": system_prompt}]
if history:
    for m in history:
        role = m.get("role")
        content = m.get("content")
        if role in ("user", "assistant") and content:
            messages.append({"role": role, "content": str(content)})
messages.append({"role": "user", "content": user_content})

# ---- provider call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    print(f"[rename] LLM error: {e}", flush=True)
    return {
        "message": f"⚠ Ошибка LLM: {e}",
        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
        "new_media": None,
    }

# ---- dump response ----
try:
    from ..dumper import CoreEngineLibWordLlmDumper
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name="rename",
        raw_response=raw,
    )
except Exception as e:
    print(f"[rename] dump response failed: {e}", flush=True)

# ---- parse ----
parsed = self._parse_response(raw)

new_id = parsed.get("new_id")
new_label = parsed.get("new_label")
new_media = parsed.get("new_media")

if not new_id:
    print(
        f"[rename] model did not return a usable new_id: "

```

```

        f"{str(row)[:160]!r}",
        flush=True,
    )
    return {
        "message": "⚠ Не удалось подобрать новое имя. Попробуйте ещё раз.",
        "css": None,
        "effect_id": None,
        "new_id": None,
        "new_label": None,
        "new_media": None,
    }

    ok, validated_id, validated_label = self._validate(
        new_id, new_label, effect_id, existing_ids
    )
    if not ok:
        return {
            "message": "⚠ Модель предложила неподходящее имя. Попробуйте ещё раз.",
            "css": None,
            "effect_id": None,
            "new_id": None,
            "new_label": None,
            "new_media": None,
        }

    # ---- media validation ----
    # The media is OPTIONAL: if the model did not produce a valid
    # SVG (or forgot it entirely), we return an empty string and
    # the client falls back to reusing the OLD miniature.
    clean_media = self._validate_media(new_media)

    return {
        "message": "Готово.",
        "css": effect_css,          # UNCHANGED
        "effect_id": effect_id,    # UNCHANGED
        "new_id": validated_id,
        "new_label": validated_label,
        "new_media": clean_media,  # may be "" if invalid/missing
    }

# =====
# PARSE
# =====

@staticmethod
def _parse_response(text: str) -> Dict[str, Any]:
    """
    Extract `new_id`, `new_label` and `new_media` from the model
    response.

    Accepts JSON: { "new_id": "fx-...", "new_label": "...",
                    "new_media": "<svg ...>...</svg>" }.
    Also tolerates an accidental "css" field – it is ignored, we
    never use the model's CSS in this agent.
    """
    result: Dict[str, Any] = {

```

```

        "new_id": None,
        "new_label": None,
        "new_media": None,
    }

    if not text:
        return result

    s = text.strip()

    # strip markdown fence
    if s.startswith("```"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("```"):
            s = s[:-3]
        s = s.strip()

    # try the whole string
    data = CoreEngineLibWordLlmAgentRename._try_json(s)

    # try the first {...} JSON block
    if data is None:
        start = s.find("{")
        end = s.rfind("}")
        if start != -1 and end != -1 and end > start:
            data = CoreEngineLibWordLlmAgentRename._try_json(
                s[start:end + 1]
            )

    if isinstance(data, dict):
        result["new_id"] = data.get("new_id")
        result["new_label"] = data.get("new_label")
        result["new_media"] = data.get("new_media")

    return result

@staticmethod
def _try_json(text: str) -> Optional[dict]:
    try:
        data = json.loads(text)
    except Exception:
        return None
    return data if isinstance(data, dict) else None

# =====
# VALIDATION
# =====

@staticmethod
def _validate(
    raw_new_id: Any,
    raw_new_label: Any,
    current_id: str,
    existing_ids: List[str],

```



```

) -> Tuple[bool, Optional[str], Optional[str]]:
    """
    Validate `new_id`/`new_label`. Returns
    (ok, validated_id, validated_label).
    """
    if not raw_new_id:
        return (False, None, None)

    new_id = str(raw_new_id).strip()
    if not new_id:
        return (False, None, None)

    if new_id == current_id:
        print(f"[rename] model proposed the same id ({new_id}) – rejected", flush=True)
        return (False, None, None)

    if not EFFECT_ID_RE.match(new_id):
        print(f"[rename] invalid new_id ({new_id!r}) – rejected", flush=True)
        return (False, None, None)

    if new_id in existing_ids:
        print(
            f"[rename] already-taken new_id ({new_id!r}) – rejected. "
            f"existing: {existing_ids}",
            flush=True,
        )
        return (False, None, None)

    new_label = (str(raw_new_label).strip() if raw_new_label else "")
    if not new_label:
        new_label = new_id
    if len(new_label) > 80:
        new_label = new_label[:77] + "..."

    return (True, new_id, new_label)

```

@staticmethod

```
def _validate_media(raw_media: Any) -> str:
```

```
    """
```

Sanitize the optional `new\_media` field.

The client can survive without a fresh miniature: if we return an empty string, it will reuse the OLD SVG from the previous block. So this function is intentionally lax – it only checks that the payload looks like an SVG at all.

Returns the SVG as a string, or "" if the payload is not usable.

```
    """
```

```
    if not raw_media:
        return ""
```

```
    s = str(raw_media).strip()
    if not s:
        return ""
```

```

if not _SVG_HINT_RE.search(s):
    print(
        f"[rename] new_media does not look like SVG "
        f"(first 60 chars: {s[:60]!r}) – dropping",
        flush=True,
    )
    return ""

# Collapse internal newlines – the prompt asks for a one-liner
# but models occasionally emit multi-line SVG. A single line
# is easier to embed into the block model and to log.
s = " ".join(s.split())

return s

```

```

=====
FILE: core/engine/lib/word/llm/prompts/svg.py
=====

```

```

# neurocad/core/engine/lib/word/llm/prompts/svg.py

```

```

"""

```

Step 4 of the page-building flow: SVG illustrations.

The model receives ONE image alt text (plus the overall page request for context) and returns ONE inline ``<svg>`` illustration that matches the alt text.

One request per image. The caller (``step_svg_illustrations`` in `step.py`) is responsible for:

- finding the placeholder images,
- calling this prompt once per image,
- retrying up to three times on failure,
- replacing ``<img>`` with the returned ``<svg>``,
- transferring ``class`` and ``alt`` onto the svg.

This module only builds the system prompt string. It knows nothing about the HTTP layer, the LLM provider, or the HTML that surrounds the image.

Style contract (baked into the prompt):

- flat, 3–4 colors, no gradients;
- palette draws from CSS variables: `var(--theme-accent, ...)`, `var(--theme-text-muted, ...)`, `var(--theme-bg-soft, ...)`, etc., with hard-coded fallbacks so the svg still works when a theme variable is missing;
- `viewBox "0 0 800 600"` (4:3, matches the placeholder aspect);
- no text inside the svg (labels belong in HTML around it);
- no `<script>`, no external hrefs, no base64, no raster images;
- self-contained: every shape, path, color lives in the svg.

No static imports beyond the standard library. This module is pure – it only builds a string.

```

"""

```

```

def build_svg_illustration_prompt() -> str:

```

"""

System prompt for one SVG illustration.

The user-side message (composed by the caller) carries:

- the overall page request (for tone / context),
- the alt text of THIS image (the main subject),
- the attempt number (1..3).

@returns the full system prompt as a string

"""

return """Ты – иллюстратор для CMS NeuroCad.

Тебе дают alt-текст ОДНОЙ картинки и общий запрос страницы. Ты рисуешь SVG-иллюстрацию, которая подходит этой картинке по смыслу.

ФОРМАТ ОТВЕТА – ТОЛЬКО ВАЛИДНЫЙ JSON:

```
{  
  "svg": "<svg viewBox=\\\"0 0 800 600\\\" xmlns=\\\"http://www.w3.org/2000/svg\\\">...</svg>"  
}
```

СОДЕРЖИМОЕ SVG:

1. Тег <svg> ОДИН. Без вложенных <svg>, без <symbol>, без <use>, без внешних ссылок.
2. viewBox ОБЯЗАТЕЛЬНО "0 0 800 600". Атрибуты width и height НЕ указывай – размер задаётся вёрсткой.
3. Формат – плоский (flat), без градиентов, без теней, без фильтров. 3–4 цвета максимум.
4. Цвета бери из CSS-переменных темы с числовым fallback, например:
fill="var(--theme-accent, #3b82f6)"
fill="var(--theme-text-muted, #94a3b8)"
fill="var(--theme-bg-soft, #f1f5f9)"
stroke="var(--theme-text, #1e293b)"
Fallback-цвет подставляй всегда: если переменная не задана, SVG должен всё равно выглядеть осмысленно.
5. Один-два смысловых объекта на всю картинку. НЕ рисуй рамку, фон-подложку или декоративные полосы по краям – фон задаётся CSS страницы.
6. Текст внутри SVG НЕ рисуй. Подписи – задача HTML вокруг картинки.
7. Никаких <script>, никаких onload / onclick, никакого javascript:, никаких data:image/png и прочего растра, никаких http-ссылок.
8. Тег <svg> и весь контент – ОДНОЙ СТРОКОЙ, без переводов строк внутри. Кавычки – двойные.

КОМПОЗИЦИЯ:

9. Главный объект – в центре кадра. Он занимает примерно 60–70% площади, оставляя поля по краям.
10. Если в alt упомянуто окружение (павильон, локация, студия) – добавь 1–2 простых силуэта окружения на заднем плане.
11. Если в alt есть действие (съёмка, работа, чтение) – покажи один-два силуэта людей в этом действии. Без деталей лиц.
12. Избегай «сложных» сцен: детализация должна оставаться на уровне простых фигур (круги, прямоугольники, линии).

ПРИМЕРЫ ALT И ИДЕЙ:

alt: «Съёмочная площадка художественного фильма в павильоне киностудии»
идея: большой прямоугольник-павильон, внутри — силуэт камеры
на штативе, за ней — прямоугольник-экран.

alt: «Оператор работает с камерой на натурных съёмочных работах»
идея: силуэт человека с камерой на плече, за ним — линия горизонта.

alt: «Монтажная студия и рабочее место режиссёра монтажа»
идея: стол, два монитора (прямоугольники), силуэт человека за столом.

alt: «Актёрский состав на читке сценария»
идея: круглый стол, вокруг — три-четыре силуэта, на столе — листы.

ГЛАВНОЕ ПРАВИЛО:

13. Иллюстрация должна быть УЗНАВАЕМА по alt-тексту. Если alt про камеру — должна быть камера, а не абстрактные круги.
14. Если не получается нарисовать объект целиком — рисуй минимальный узнаваемый силуэт.
15. Ответ — ТОЛЬКО валидный JSON. НЕ оборачивай в markdown.
НЕ добавляй пояснения до или после JSON.

"""

```
=====
FILE: core/engine/lib/word/llm/prompts/__init__.py
=====
```

```
=====
FILE: core/engine/lib/word/llm/steps/common.py
=====
```

```
# neurocad/core/engine/lib/word/llm/steps/common.py
```

"""

Common helpers for the multi-step LLM flow (plan / fill / effects / svg).

Namespace: CoreEngineLibWordLlmSteps

This module contains only pure helpers — no side effects, no state.
Every step module (plan.py, fill.py, effects.py, svg.py) imports from
here instead of duplicating the same code.

NOT INTENDED to be imported from outside the `steps/` package.
"""

```
import json
import re
from typing import Any, Dict, List, Optional, Tuple
```

```
# =====
# CONSTANTS
# =====
```

```
#: Fallback viewBox if the model does not provide one.
DEFAULT_VIEWBOX = "0 0 800 600"
```



```

        fn(target="step", message=message)
    except Exception:
        pass

# =====
# MESSAGES BUILDER
# =====

def build_messages(
    system_prompt: str,
    user_content: str,
    history: Optional[List[Dict[str, str]]] = None,
) -> List[Dict[str, str]]:
    """
    Build the messages array for the LLM.

    Order:
    - system prompt
    - previous dialogue turns (if any)
    - current user message
    """
    messages: List[Dict[str, str]] = [{"role": "system", "content": system_prompt}]

    if history:
        for m in history:
            role = m.get("role")
            content = m.get("content")
            if role in ("user", "assistant") and content:
                messages.append({"role": role, "content": str(content)})

    messages.append({"role": "user", "content": user_content})
    return messages

# =====
# JSON / SVG EXTRACTION
# =====

def extract_json(text: str) -> Optional[dict]:
    """
    Pull a JSON object out of the model response.

    Tolerates markdown fences and surrounding prose. Returns the first
    valid JSON object found, or None.
    """
    if not text:
        return None

    s = text.strip()
    if s.startswith("```"):
        first_nl = s.find("\n")
        if first_nl != -1:
            s = s[first_nl + 1:]
        if s.endswith("```"):
            s = s[:-3]

```

```

    s = s.strip()

    try:
        data = json.loads(s)
        if isinstance(data, dict):
            return data
    except json.JSONDecodeError:
        pass

    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        try:
            data = json.loads(s[start:end + 1])
            if isinstance(data, dict):
                return data
        except json.JSONDecodeError:
            pass

    return None

def extract_svg(text: str) -> Optional[str]:
    """
    Pull the first `...</svg>` out of the model response.

    Tolerates markdown fences and surrounding prose. If the response
    is JSON with an "svg" field, that is preferred.
    """
    if not text:
        return None

    s = text.strip()

    # Strip a single markdown fence.
    if s.startswith("```"):
        nl = s.find("\n")
        if nl != -1:
            s = s[nl + 1:]
        if s.endswith("```"):
            s = s[:-3]
        s = s.strip()

    # JSON with an "svg" field.
    data = extract_json(s)
    if isinstance(data, dict):
        svg_field = data.get("svg")
        if isinstance(svg_field, str) and svg_field.strip():
            return svg_field.strip()

    # First <svg ...> ... </svg> in the response.
    start = s.lower().find("<svg")
    if start == -1:
        return None
    end = s.lower().rfind("</svg>")
    if end == -1 or end <= start:

```

```

        return None
    return s[start:end + len("</svg>")].strip()

def validate_svg(svg: str) -> Tuple[bool, str]:
    """
    Cheap sanity check on the extracted SVG.

    Rules:
    - non-empty;
    - starts with `<svg`;
    - contains `</svg>`;
    - does NOT contain `<script`;
    - does NOT contain `on\w+=` (event handlers);
    - does NOT contain `javascript:`;
    - does NOT contain `href="http` (external references).
    """
    if not svg:
        return (False, "empty")

    low = svg.lower()

    if not low.startswith("<svg"):
        return (False, "does not start with <svg")

    if "</svg>" not in low:
        return (False, "no closing </svg>")

    if "<script" in low:
        return (False, "contains <script>")

    if "javascript:" in low:
        return (False, "contains javascript:")

    if re.search(r"\bon[a-z]+\s*=", low):
        return (False, "contains event handler")

    if re.search(r'""(?:href|xlink:href)\s*=\s*["']\s*(?:https?:)?//""', low):
        return (False, "contains external href")

    return (True, "")

def merge_attrs_into_svg(svg: str, *, alt: str, css_class: str) -> str:
    """
    Put `class` and `alt` (as `aria-label`) onto the root `<svg>`.

    The original `<img class="image_gallery__item" alt="...">`
    becomes `<svg class="image_gallery__item" aria-label="..."
    role="img" viewBox="...">`. That way the existing CSS for
    `image` / `gallery__item` still applies.
    """
    if not svg:
        return svg

    m = re.match(r"(<svg\b)([^\>]*?)(>|/>)", svg, flags=re.IGNORECASE | re.DOTALL)

```



```

if not m:
    return svg

head = m.group(1)
attrs = m.group(2) or ""
close = m.group(3) or ">"
rest = svg[m.end():]

if "viewBox" not in attrs and "viewbox" not in attrs.lower():
    attrs = f' viewBox="{DEFAULT_VIEWBOX}"' + attrs

if css_class and "class" not in attrs.lower():
    attrs += f' class="{css_class}"'

if "role=" not in attrs.lower():
    attrs += ' role="img"'
if alt and "aria-label=" not in attrs.lower():
    safe_alt = alt.replace("'", "&quot;")
    attrs += f' aria-label="{safe_alt}"'

return f"{head}{attrs}{close}{rest}"

def img_attr(img_tag: str, name: str) -> Optional[str]:
    """
    Extract an attribute value from a raw `<img ...>` tag string.

    Returns the value or None. Case-insensitive on the attribute
    name. Handles both double- and single-quoted values.
    """
    if not img_tag:
        return None

    target = name.lower()
    for m in IMG_ATTR_RE.finditer(img_tag):
        if m.group("name").lower() == target:
            return m.group("dq") if m.group("dq") is not None else m.group("sq")
    return None

# =====
# RETRY / CHUNKING
# =====

def is_retryable_error(err: Optional[str]) -> bool:
    """
    True if the given error string should be retried, False if
    retrying would only make things worse.

    Non-retryable: rate limit, timeout, connection problems.
    Everything else is considered retryable.
    """
    if not err:
        return True

    low = err.lower()

```

```

for marker in NON_RETRYABLE_MARKERS:
    if marker in low:
        return False
return True

def split_into_chunks(
    ordered_ids: List[str],
    html_by_id: Dict[str, str],
    limit: int,
) -> List[List[str]]:
    """Split a list of block ids into chunks whose total HTML size <= limit."""
    chunks: List[List[str]] = []
    current: List[str] = []
    current_size = 0

    for bid in ordered_ids:
        size = len(html_by_id.get(bid, ""))

        if size > limit:
            if current:
                chunks.append(current)
                current = []
                current_size = 0
            chunks.append([bid])
            continue

        if current and current_size + size > limit:
            chunks.append(current)
            current = []
            current_size = 0

        current.append(bid)
        current_size += size

    if current:
        chunks.append(current)

    return chunks

```

```

=====
FILE: core/engine/lib/word/llm/steps/effects.py
=====

```

```

# neurocad/core/engine/lib/word/llm/steps/effects.py

```

```

"""

```

Step 3 of the multi-step LLM flow: EFFECTS.

Applies ready-made effects from the palette to the blocks.

The model does NOT write CSS. It receives the assembled page HTML and the catalog of effects that are actually available in the editor, and adds ONE of the ready-made `fx-\*` classes to each section.

Namespace: CoreEngineLibWordLlmSteps

```

"""

```

```

from typing import Any, AsyncGenerator, Dict, List, Optional

from ..prompts.effects import build_effects_prompt
from ..dumper import CoreEngineLibWordLlmDumper
from .common import build_messages, extract_json, log

async def step_effects(
    user_message: str,
    assembled_html: str,
    provider,
    history: Optional[List[Dict[str, str]]] = None,
    effect_catalog: Optional[List[Dict[str, Any]]] = None,
    run_id: Any = None,
    agent_name: str = "create",
) -> AsyncGenerator[Dict[str, Any], None]:
    """
    Step 3: apply ready-made effects from the palette to the blocks.

    `effect_catalog` is the source of truth for the prompt: if it
    contains 8 effects, the model sees exactly those 8; if it is
    empty or missing, `build_effects_prompt` falls back to a small
    hard-coded set.
    """
    catalog_count = len(effect_catalog) if effect_catalog else 0

    log(
        provider, "info",
        f"effects start: agent={agent_name} run={run_id!r} "
        f"html={len(assembled_html)} chars, catalog={catalog_count}",
    )

    yield {
        "type": "step",
        "step": "effects",
        "message": "Добавляю эффекты...",
    }

    system_prompt = build_effects_prompt(effect_catalog)
    user_content = f"Запрос: {user_message}\n\nHTML:\n{assembled_html}"

    messages = build_messages(system_prompt, user_content, history)

    # ---- dump prompt ----
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name=agent_name,
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
    )

    try:
        raw = await provider.generate_completion(messages)
    except Exception as e:

```

```

        log(provider, "error", f"effects LLM error: {e}")
        yield {"type": "result", "html": assembled_html}
        return

# ---- dump response ----
CoreEngineLibWordLlmDumper.dump_response(
    run_id=run_id,
    agent_name=agent_name,
    raw_response=raw,
)

parsed = parse_effects(raw)
html = parsed.get("html") or assembled_html

yield {"type": "result", "html": html}

def parse_effects(text: str) -> Dict[str, str]:
    """Parse the model's effects response into {html}."""
    data = extract_json(text)
    if not data:
        return {"html": text.strip()}

    html = data.get("html", "")
    if not isinstance(html, str):
        html = ""
    return {"html": html.strip()}

=====
FILE: core/engine/lib/word/llm/steps/fill.py
=====
# neurocad/core/engine/lib/word/llm/steps/fill.py

"""
Step 2 of the multi-step LLM flow: FILL.

Fills the chosen blocks with text, alts, links.

Namespace: CoreEngineLibWordLlmSteps
"""

from typing import Any, AsyncGenerator, Dict, List, Optional

from ..prompts.fill import build_fill_prompt, build_fill_user_message
from ..dumper import CoreEngineLibWordLlmDumper
from .common import build_messages, extract_json, log, split_into_chunks

#: Default max HTML size (bytes) per fill request.
DEFAULT_CHUNK_LIMIT = 15_000

#: When the model reports "remaining" non-empty, shrink the limit
#: for the next attempt.
CHUNK_LIMIT_SHRINK = 0.7

#: Max number of fill attempts to avoid infinite loops.

```

```
MAX_FILL_ATTEMPTS = 10
```

```
async def step_fill(
    user_message: str,
    plan: List[Dict[str, Any]],
    block_catalog: List[Dict[str, Any]],
    provider,
    history: Optional[List[Dict[str, str]]] = None,
    chunk_limit: Optional[int] = None,
    run_id: Any = None,
    agent_name: str = "create",
) -> AsyncGenerator[Dict[str, Any], None]:
    """
    Step 2: fill the chosen blocks with text, alts, links.
    """
    if chunk_limit is None:
        chunk_limit = DEFAULT_CHUNK_LIMIT

    log(
        provider, "info",
        f"fill start: agent={agent_name} run={run_id!r} "
        f"plan={len(plan)} blocks, catalog={len(block_catalog)}",
    )

    yield {
        "type": "step",
        "step": "filling",
        "message": "Заполняю текстом...",
    }

    html_by_id: Dict[str, str] = {
        b["id"]: b.get("html", "") for b in block_catalog
    }

    ordered_ids = [p["block_id"] for p in plan]
    if not ordered_ids:
        yield {"type": "result", "filled": {}}
        return

    filled: Dict[str, str] = {}
    remaining = list(ordered_ids)
    current_limit = chunk_limit
    attempts = 0

    while remaining and attempts < MAX_FILL_ATTEMPTS:
        attempts += 1

        chunks = split_into_chunks(remaining, html_by_id, current_limit)
        total = len(chunks)

        for i, chunk in enumerate(chunks, start=1):
            yield {
                "type": "fill_progress",
                "request": i,
                "total": total,
```

```

        "block_ids": chunk,
    }

    chunk_payload = [
        {"block_id": bid, "html": html_by_id.get(bid, "")}
        for bid in chunk
    ]
    system_prompt = build_fill_prompt()
    user_content = build_fill_user_message(user_message, chunk_payload)

    messages = build_messages(system_prompt, user_content, history)

    # ---- dump prompt ----
    CoreEngineLibWordLLmDumper.dump_step(
        run_id=run_id,
        agent_name=agent_name,
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
        chunk_index=i,
    )

    try:
        raw = await provider.generate_completion(messages)
    except Exception as e:
        log(provider, "error", f"fill LLM error: {e}")
        yield {"type": "error", "message": f"LLM error: {e}"}
        return

    # ---- dump response ----
    CoreEngineLibWordLLmDumper.dump_response(
        run_id=run_id,
        agent_name=agent_name,
        raw_response=raw,
        chunk_index=i,
    )

    parsed = parse_fill(raw)
    for item in parsed.get("filled", []):
        bid = item.get("block_id")
        html = item.get("html", "")
        if bid and html:
            filled[bid] = html

    new_remaining = [bid for bid in ordered_ids if bid not in filled]
    if new_remaining == remaining:
        current_limit = int(current_limit * CHUNK_LIMIT_SHRINK)
        if current_limit < 500:
            log(provider, "error", "fill chunk limit too small, giving up")
            break
    remaining = new_remaining

    yield {"type": "result", "filled": filled}

def parse_fill(text: str) -> Dict[str, Any]:

```

```

"""Parse the model's fill response into {filled, remaining}."""
data = extract_json(text)
if not data:
    return {"filled": [], "remaining": []}

filled_raw = data.get("filled") or []
remaining_raw = data.get("remaining") or []

filled: List[Dict[str, str]] = []
for item in filled_raw:
    if not isinstance(item, dict):
        continue
    bid = item.get("block_id")
    html = item.get("html", "")
    if bid:
        filled.append({"block_id": str(bid), "html": str(html)})

remaining = [str(x) for x in remaining_raw if x]
return {"filled": filled, "remaining": remaining}

=====
FILE: core/engine/lib/word/llm/steps/plan.py
=====
# neurocad/core/engine/lib/word/llm/steps/plan.py

"""
Step 1 of the multi-step LLM flow: PLAN.

Given a catalog of available blocks (id + label + category, no HTML),
the model picks which blocks to use and in what order, and returns a
JSON object with a "plan" array.

Namespace: CoreEngineLibWordLlmSteps
"""

from typing import Any, AsyncGenerator, Dict, List, Optional

from ..prompts.plan import build_plan_prompt
from ..dumper import CoreEngineLibWordLlmDumper
from .common import build_messages, extract_json, log

async def step_plan(
    user_message: str,
    block_catalog: List[Dict[str, Any]],
    provider,
    history: Optional[List[Dict[str, str]]] = None,
    run_id: Any = None,
    agent_name: str = "create",
) -> AsyncGenerator[Dict[str, Any], None]:
    """
    Step 1: choose which blocks to use and in what order.
    """
    log(
        provider, "info",
        f"plan start: agent={agent_name} run={run_id!r} "

```

```

        f"blocks={len(block_catalog)} history={len(history) if history else 0}",
    )

    yield {
        "type": "step",
        "step": "planning",
        "message": "Выбираю подходящие блоки...",
    }

    system_prompt = build_plan_prompt(block_catalog)
    user_content = f"Запрос: {user_message}"
    messages = build_messages(system_prompt, user_content, history)

    # ---- dump full prompt ----
    CoreEngineLibWordLlmDumper.dump_step(
        run_id=run_id,
        agent_name=agent_name,
        system_prompt=system_prompt,
        user_content=user_content,
        history=history,
    )

    try:
        raw = await provider.generate_completion(messages)
    except Exception as e:
        log(provider, "error", f"plan LLM error: {e}")
        yield {"type": "error", "message": f"LLM error: {e}"}
        return

    # ---- dump response ----
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name=agent_name,
        raw_response=raw,
    )

    plan = parse_plan(raw)
    if not plan:
        yield {"type": "error", "message": "Не удалось разобрать план блоков."}
        return

    yield {"type": "plan", "blocks": plan}
    yield {"type": "result", "plan": plan}

def parse_plan(text: str) -> List[Dict[str, Any]]:
    """Parse the model's plan response into a list of {block_id, purpose}."""
    data = extract_json(text)
    if not data:
        return []

    raw_plan = data.get("plan") or []
    result: List[Dict[str, Any]] = []
    for item in raw_plan:
        if not isinstance(item, dict):
            continue

```



```

        bid = item.get("block_id")
        if not bid:
            continue
        result.append({
            "block_id": str(bid),
            "purpose": str(item.get("purpose", "")).strip(),
        })
    return result

```

```

=====
FILE: core/engine/lib/word/llm/steps/svg.py
=====

```

```

# neurocad/core/engine/lib/word/llm/steps/svg.py

```

```

"""

```

```

Step 4 of the multi-step LLM flow: SVG ILLUSTRATIONS.

```

Walks the assembled HTML, finds every `` that still points at `placeholder.svg`, and asks the LLM to produce an inline `` for each of them – one request per image, strictly sequentially, up to three attempts per image.

Every successfully generated SVG is ALSO saved to the images registry (editor/images/), so it appears in the "Изображения" palette in the editor. The save is best-effort: if it fails, the SVG still goes into the page HTML.

```

Namespace: CoreEngineLibWordLlmSteps

```

```

"""

```

```

import asyncio
from typing import Any, AsyncGenerator, Dict, List, Optional, Tuple

```

```

from ..prompts.svg import build_svg_illustration_prompt
from ..dumper import CoreEngineLibWordLlmDumper
from .common import (
    INTER_IMAGE_DELAY_S,
    IMG_PLACEHOLDER_RE,
    MAX_SVG_ATTEMPTS,
    RETRY_DELAY_S,
    build_messages,
    extract_svg,
    img_attr,
    is_retryable_error,
    log,
    merge_attrs_into_svg,
    validate_svg,
)

```

```

async def step_svg_illustrations(
    user_message: str,
    assembled_html: str,
    provider,
    history: Optional[List[Dict[str, str]]] = None,
    run_id: Any = None,

```

```

    agent_name: str = "create",
) -> AsyncGenerator[Dict[str, Any], None]:
    """
    Step 4: replace placeholder images with generated inline SVGs.

    See the module docstring for the full description.
    """
    # ---- Collect placeholder images ----
    matches = list(IMG_PLACEHOLDER_RE.finditer(assembled_html))
    total = len(matches)

    log(
        provider, "info",
        f"svg start: agent={agent_name} run={run_id!r} "
        f"placeholders={total}",
    )

    if total == 0:
        # Nothing to do – return the html untouched.
        yield {"type": "result", "html": assembled_html}
        return

    yield {
        "type": "step",
        "step": "svg",
        "message": f"Рисую иллюстрации: {total}...",
    }

    # We rebuild the html piece by piece, replacing each match.
    # `cursor` tracks our position in the original string.
    parts: List[str] = []
    cursor = 0

    for idx, m in enumerate(matches, start=1):
        img_tag = m.group(0)
        alt = img_attr(img_tag, "alt") or ""
        cls = img_attr(img_tag, "class") or ""

        yield {
            "type": "svg_progress",
            "current": idx,
            "total": total,
            "alt": alt,
        }

        # ---- Append the untouched prefix before this match ----
        parts.append(assembled_html[cursor:m.start()])
        cursor = m.end()

        # ---- Attempt up to N times ----
        svg = None
        for attempt in range(1, MAX_SVG_ATTEMPTS + 1):
            svg, err = await _try_generate_svg(
                provider=provider,
                user_message=user_message,
                alt=alt,

```

```

        history=history,
        run_id=run_id,
        agent_name=agent_name,
        attempt=attempt,
    )
    if svg:
        break

# ---- Decide whether to retry ----
if is_retryable_error(err):
    if attempt < MAX_SVG_ATTEMPTS:
        log(
            provider, "info",
            f"svg retry {attempt + 1}/{MAX_SVG_ATTEMPTS} "
            f"after {RETRY_DELAY_S}s for alt={alt!r}",
        )
        await asyncio.sleep(RETRY_DELAY_S)
        continue
    log(
        provider, "warning",
        f"svg attempt {attempt}/{MAX_SVG_ATTEMPTS} failed "
        f"for alt={alt!r}",
    )
    continue
else:
    log(
        provider, "warning",
        f"svg non-retryable error for alt={alt!r}: {err}",
    )
    break

if svg:
    # Transfer class and alt from the <img> onto the <svg>.
    svg = merge_attrs_into_svg(svg, alt=alt, css_class=cls)

    # Save the generated SVG into the images registry, so
    # it appears in the "Изображения" palette. Non-fatal:
    # if the save fails, the SVG still goes into the page
    # HTML – we just do not add it to the palette.
    try:
        from ...editor.images.store import save_generated_svg
        image_id = save_generated_svg(
            svg=svg, alt=alt, source="llm",
        )
        if image_id:
            log(
                provider, "info",
                f"svg saved to registry: {image_id}",
            )
        else:
            log(
                provider, "warning",
                f"svg save failed for alt={alt!r}",
            )
    except Exception as e:
        log(

```

```

        provider, "warning",
        f"save_generated_svg raised: {e}",
    )

    parts.append(svg)
    log(provider, "info", f"svg ok for alt={alt!r}")
else:
    # Leave the placeholder untouched.
    parts.append(img_tag)
    log(
        provider, "warning",
        f"svg gave up for alt={alt!r} – placeholder kept",
    )

    # ---- Pause between images (but not after the last one) ----
    if idx < total and INTER_IMAGE_DELAY_S > 0:
        await asyncio.sleep(INTER_IMAGE_DELAY_S)

# ---- Append the tail ----
parts.append(assembled_html[cursor:])

yield {"type": "result", "html": "".join(parts)}

async def _try_generate_svg(
    *,
    provider,
    user_message: str,
    alt: str,
    history: Optional[List[Dict[str, str]]],
    run_id: Any,
    agent_name: str,
    attempt: int,
) -> Tuple[Optional[str], str]:
    """
    One attempt: ask the LLM for an SVG for this alt text.

    Returns (svg, err):
    - on success: (svg_string, "")
    - on failure: (None, reason_string)
    """
    system_prompt = build_svg_illustration_prompt()
    user_content = (
        f"Общий запрос страницы: {user_message}\n\n"
        f"Alt изображения: {alt}\n\n"
        f"Попытка: {attempt}"
    )

    messages = build_messages(system_prompt, user_content, history)

    # ---- dump prompt ----
    try:
        CoreEngineLibWordLlmDumper.dump_step(
            run_id=run_id,
            agent_name=f"{agent_name}-svg",
            system_prompt=system_prompt,

```

```

        user_content=user_content,
        history=history,
    )
except Exception:
    pass

# ---- call ----
try:
    raw = await provider.generate_completion(messages)
except Exception as e:
    log(provider, "error", f"svg LLM error: {e}")
    return (None, f"LLM error: {e}")

# ---- dump response ----
try:
    CoreEngineLibWordLlmDumper.dump_response(
        run_id=run_id,
        agent_name=f"{agent_name}-svg",
        raw_response=raw,
    )
except Exception:
    pass

# ---- extract and validate ----
svg = extract_svg(raw)
if not svg:
    return (None, "no <svg> found in response")

ok, reason = validate_svg(svg)
if not ok:
    log(provider, "warning", f"svg rejected: {reason}")
    return (None, f"validation failed: {reason}")

return (svg, "")

```

```

=====
FILE: core/engine/lib/word/llm/websocket/agents.py
=====

```

```

# neurocad/core/engine/lib/word/llm/websocket/agents.py

```

```

"""

```

Agent registry – maps agent names to their classes.

Used by DispatchMixin to look up the class by the name the router returned (or by the name hardcoded in the effect_edit / effect_rename / create_effect flows).

Keys are the agent names the router and the frontend use. Values are the classes – DispatchMixin instantiates them per run:

```

agent = _AGENTS["fill"]()
result = await agent.run(...)

```

All agents inherit CoreEngineLibWordLlmAgentBase and share the same `run` signature (see agent/base.py).

The "edit", "rename" and "create_effect" agents are NOT returned by the router in a normal start flow – they are only reachable via their explicit WS types. To be safe, DispatchMixin and the endpoint's router-handling block both reject them if the router returns them anyway.

Namespace: CoreEngineLibWordLlmWS (via Base + mixins)

"""

```
from ..agent.none import CoreEngineLibWordLlmAgentNone
from ..agent.help import CoreEngineLibWordLlmAgentHelp
from ..agent.create import CoreEngineLibWordLlmAgentCreate
from ..agent.create_page import CoreEngineLibWordLlmAgentCreatePage
from ..agent.fill import CoreEngineLibWordLlmAgentFill
from ..agent.effect import CoreEngineLibWordLlmAgentEffect
from ..agent.edit import CoreEngineLibWordLlmAgentEdit
from ..agent.rename import CoreEngineLibWordLlmAgentRename
from ..agent.create_effect import CoreEngineLibWordLlmAgentCreateEffect
```

#: Name → agent class.

#:

```
#: none          – off-topic requests; no LLM call, no output
#: help          – questions about the editor itself
#: create        – build a full page from our blocks (multi-step)
#: create_page   – build a full page in ONE LLM request (free-form)
#: fill          – fill one selected element with text/links/alts
#: effect        – add one ready-made effect class to one element
#: edit          – rewrite the CSS of one existing effect
#: rename        – propose a NEW id/label for an existing effect
#:               (CSS is not changed)
#: create_effect – generate a draft for a NEW effect
```

#:

#: edit / rename / create_effect bypass the router: they are

#: dispatched directly from the endpoint by WS message type.

```
_AGENTS = {
    "none":          CoreEngineLibWordLlmAgentNone,
    "help":          CoreEngineLibWordLlmAgentHelp,
    "create":        CoreEngineLibWordLlmAgentCreate,
    "create_page":   CoreEngineLibWordLlmAgentCreatePage,
    "fill":          CoreEngineLibWordLlmAgentFill,
    "effect":        CoreEngineLibWordLlmAgentEffect,
    "edit":          CoreEngineLibWordLlmAgentEdit,
    "rename":        CoreEngineLibWordLlmAgentRename,
    "create_effect": CoreEngineLibWordLlmAgentCreateEffect,
}
```

```
=====
FILE: core/engine/lib/word/llm/websocket/auth.py
=====
```

```
# neurocad/core/engine/lib/word/llm/websocket/auth.py
```

"""

AuthMixin – authentication and logging helpers for the WebSocket endpoint.

Two responsibilities:

1. `_authenticate_ws(websocket)`
Decodes the `access_token` cookie (JWT) and looks up the user via `CoreAuthService`. Returns the user dict, or `None` if anything fails.
The endpoint closes the connection with code 4403 when this returns `None` or when the user is not a superadmin.

On success, the resolved user is stored on ``websocket.state``:

- `websocket.state.user` – full user dict
- `websocket.state.user_id` – `int(user.id)`

Downstream mixins (`DispatchMixin`) read those instead of re-decoding the cookie on every run.

2. `_log(websocket, level, message, **kwargs)`
Best-effort logger. Writes through ``websocket.app.state.log`` (the shared app logger) if it exists, otherwise to `stderr`.
Supports both async and sync logger methods, and both plain (``log_info``) and ``_sync`` (``log_info_sync``) variants.

These methods used to be `@staticmethod` on `CoreEngineLibWordLlmWS`. They are now regular methods on a mixin, so they are called as ``self._authenticate_ws(...)`` / ``self._log(...)`` from the endpoint and from the dispatchers.

Namespace: `CoreEngineLibWordLlmWS` (via Base + mixins)
"""

```
import asyncio
import inspect
from typing import Any, Optional

from fastapi import WebSocket
```

```
class AuthMixin:
    """Authentication and logging for the WebSocket endpoint."""

    # =====
    # AUTH
    # =====

    async def _authenticate_ws(self, websocket: WebSocket) -> Optional[dict]:
        """
        Resolve the current user from the `access_token` cookie.

        Returns:
            The user dict on success, or None on any failure.
            The caller (endpoint) treats None as "close connection
            with code 4403".

        On success, also stores:
            websocket.state.user – full user dict
            websocket.state.user_id – int(user.id)

        Notes:
            - Cookie is read BEFORE websocket.accept(), which is fine
```

```

        for FastAPI/Starlette: cookies are available on the scope
        before the handshake completes.
    - JWT decode uses the project-wide SECRET_KEY and ALGORITHM.
    - User lookup goes through CoreAuthService (async).
    """
    try:
        token = websocket.cookies.get("access_token")
        if not token:
            return None

        from jose import jwt, JWTError
        from neurocad.config import settings

        try:
            payload = jwt.decode(
                token,
                settings.SECRET_KEY,
                algorithms=[settings.ALGORITHM],
            )
        except JWTError:
            return None

        user_id = payload.get("sub")
        if user_id is None:
            return None

        from neurocad.core.auth.service import CoreAuthService
        user = await CoreAuthService.get_user_by_id(int(user_id))

        # Cache the resolved identity on the websocket so downstream
        # mixins (dispatch) can read it without re-decoding the cookie.
        if user:
            try:
                websocket.state.user = user
                websocket.state.user_id = int(user.get("id"))
            except Exception as e:
                print(f"[ws-auth] failed to cache user on state: {e}", flush=True)

        return user

    except Exception as e:
        print(f"[ws-auth] unexpected error: {e}", flush=True)
        return None

# =====
# LOGGING BRIDGE
# =====

async def _log(
    self,
    websocket: WebSocket,
    level: str,
    message: str,
    **kwargs: Any,
) -> None:
    """

```


Write a log line through app.state.log, else to stderr.

The app logger may expose:

- async methods (`log\_info`, `log\_warning`, ...)
- sync methods (`log\_info\_sync`, ...)

We try async first, then sync, then fall through to stderr.

Never raises – logging must not break the WS handler.

"""

try:

 log = getattr(websocket.app.state, "log", None)

except Exception:

 log = None

if log is not None:

 try:

 # ---- async variant ----

 fn = getattr(log, f"log_{level}", None)

 if fn is None:

 fn = getattr(log, "log_info", None)

 if fn is not None:

 if inspect.iscoroutinefunction(fn):

 # Fire-and-forget: don't block the caller on the

 # log write.

 asyncio.create_task(

 fn(target="ws", message=message, **kwargs)

)

 else:

 fn(target="ws", message=message, **kwargs)

 return

 # ---- sync variant ----

 fn_sync = getattr(log, f"log_{level}_sync", None)

 if fn_sync is None:

 fn_sync = getattr(log, "log_info_sync", None)

 if fn_sync is not None:

 fn_sync(target="ws", message=message, **kwargs)

 return

except Exception as e:

 print(f"[ws][log-fail] {e!r} :: {message}", flush=True)

 return

---- fallback: stderr ----

print(f"[ws][{level}] {message}", flush=True)

=====

FILE: core/engine/lib/word/llm/websocket/base.py

=====

neurocad/core/engine/lib/word/llm/websocket/base.py

"""

CoreEngineLibWordLlmWS – the WebSocket endpoint class, assembled
from mixins.

This class exists only to combine the mixins and hold the small bits of class-level state that cannot live in a mixin:

- HISTORY_LIMIT – how many previous chat messages to feed back into an agent as `history`.
- _cancel_events – {run_id: asyncio.Event}. Shared across all connections on purpose: a run_id is unique per run, and only one connection owns it at a time. Kept on the class (not on `self`) so the dispatcher, the endpoint, and any future helper can reach the same dict via `self.\_cancel\_events`.

The `\_AGENTS` registry itself lives in `agents.py` – this file does NOT re-declare it. `dispatch.py` imports it directly:

```
from .agents import _AGENTS
```

Mixins are combined in this order:

```
EndpointMixin – llm_ws_endpoint (main loop), small helpers
DispatchMixin – _run_agent / _run_effect_edit / _run_create_effect
ProviderMixin – _get_provider
CancelMixin – _get_cancel_event / _clear_cancel_event /
              _finish_cancelled / _finish_error / _safe_send
HistoryMixin – _load_history / _load_current_page_html
AuthMixin – _authenticate_ws / _log
```

MRO is linear and has no method-name collisions – each mixin owns a distinct set of methods.

Used by ws.py:

```
from .websocket.base import CoreEngineLibWordLlmWS
_ws_instance = CoreEngineLibWordLlmWS()
llm_ws_endpoint = _ws_instance.llm_ws_endpoint
```

Namespace: CoreEngineLibWordLlmWS

```
"""
```

```
import asyncio
from typing import Dict
```

```
from .auth import AuthMixin
from .history import HistoryMixin
from .cancel import CancelMixin
from .provider import ProviderMixin
from .dispatch import DispatchMixin
from .endpoint import EndpointMixin
```

```
class CoreEngineLibWordLlmWS(
    EndpointMixin,
    DispatchMixin,
    ProviderMixin,
    CancelMixin,
    HistoryMixin,
```

```

    AuthMixin,
):
    """WebSocket endpoint + agent dispatcher for the LLM editor."""

    #: How many previous messages (user + assistant) to include as
    #: context when an agent runs. Read by HistoryMixin._load_history.
    HISTORY_LIMIT = 20

    #: Cancel flags – one asyncio.Event per active run.
    #: Class-level on purpose: shared across all connections.
    #: Each mixin reaches this dict via `self._cancel_events`.
    _cancel_events: Dict[int, asyncio.Event] = {}

```

```

=====
FILE: core/engine/lib/word/llm/websocket/cancel.py
=====

```

```

# neurocad/core/engine/lib/word/llm/websocket/cancel.py

```

```

"""
CancelMixin – cancellation flags, terminal send helpers, and the
last-resort error path for the WebSocket endpoint.

```

Responsibilities:

1. `_get_cancel_event(run_id)`
Returns the `asyncio.Event` associated with a run. Creates it on first access. The event is SET by ``type: "cancel"`` messages and by WebSocket disconnect / supersession by a new run. Long-running agents check ``event.is_set()`` after every external call.
2. `_clear_cancel_event(run_id)`
Drops the event once the run is fully finished. Called in the ``finally`` block of every dispatcher.
3. `_finish_cancelled(websocket, run_id)`
Marks the run as cancelled in the DB, sends ``{type: "cancelled"}`` to the client. Idempotent – safe to call from multiple paths.
4. `_finish_error(websocket, run_id, message)`
Marks the run as failed in the DB, sends ``{type: "error"}`` to the client. Used as the last-resort handler from every dispatcher's `except-block`.
5. `_safe_send(websocket, payload)`
Sends a JSON payload, swallowing `WebSocketDisconnect` and `RuntimeError` (closed socket). All outgoing WS traffic goes through this – no exception ever escapes.

Cancel events live in a class-level dict on the base class (``_cancel_events``). Mixins access it via ``self._cancel_events`` – the dict itself is shared across all connections on purpose, because a `run_id` is unique per run and only one connection owns it at a time.

These methods used to be `@staticmethod` on `CoreEngineLibWordLlmWS`. They are now regular methods on a mixin, so they are called as ``self._get_cancel_event(...)`` / ``self._safe_send(...)``.

```

Namespace: CoreEngineLibWordLlmWS (via Base + mixins)
"""

import asyncio
from typing import Dict

from fastapi import WebSocket, WebSocketDisconnect

class CancelMixin:
    """Cancellation flags and terminal send helpers."""

    #: Per-run cancel events. Actual dict is declared on the base
    #: class (`_cancel_events: Dict[int, asyncio.Event] = {}`).
    #: Declared here only for static type-checkers – a mixin cannot
    #: hold instance state on its own.
    _cancel_events: Dict[int, asyncio.Event]

    # =====
    # CANCEL EVENTS
    # =====

    def _get_cancel_event(self, run_id: int) -> asyncio.Event:
        """
        Return the cancel event for a run, creating it on first use.

        Never returns None: if the key is missing, a fresh Event is
        created and stored. The caller does not need to check for
        existence before `.is_set()` / `.set()`.
        """
        ev = self._cancel_events.get(run_id)
        if ev is None:
            ev = asyncio.Event()
            self._cancel_events[run_id] = ev
        return ev

    def _clear_cancel_event(self, run_id: int) -> None:
        """
        Drop the cancel event once the run is finished.

        Called from the `finally` block of every dispatcher. If the
        key is already gone – no-op.
        """
        self._cancel_events.pop(run_id, None)

    # =====
    # TERMINAL SEND HELPERS
    # =====

    async def _finish_cancelled(self, websocket: WebSocket, run_id: int) -> None:
        """
        Mark a run as cancelled in the DB and notify the client.

        The DB write is best-effort – if it fails (network, DB down),
        we still send `{type: "cancelled"}`. The client must not be

```

```

left waiting.
"""

from ..runs import CoreEngineLibWordLlmRuns

try:
    await CoreEngineLibWordLlmRuns.update_run_status(
        run_id,
        status="cancelled",
        message="Отменено пользователем",
    )
except Exception as e:
    print(f"[ws-cancel] failed to update run {run_id}: {e}", flush=True)

    await self._safe_send(websocket, {"type": "cancelled", "run_id": run_id})

async def _finish_error(
    self,
    websocket: WebSocket,
    run_id: int,
    message: str,
) -> None:
    """
    Mark a run as failed in the DB and notify the client.

    Used as the last-resort handler from every dispatcher's
    except-block. The DB write is best-effort; the client always
    receives `{type: "error"}` with the human-readable message.
    """
    from ..runs import CoreEngineLibWordLlmRuns

    try:
        await CoreEngineLibWordLlmRuns.update_run_status(
            run_id,
            status="failed",
            message="Ошибка",
            error=message,
        )
    except Exception as e:
        print(f"[ws-error] failed to update run {run_id}: {e}", flush=True)

    await self._safe_send(websocket, {"type": "error", "message": message})

# =====
# SAFE SEND
# =====

async def _safe_send(self, websocket: WebSocket, payload: dict) -> None:
    """
    Send JSON, swallowing the two expected "socket is gone" errors.

    Called from everywhere (routers, agents' `emit` callbacks,
    finish handlers). Should never raise – the WS handler must not
    crash because the client closed the tab.
    """
    try:
        await websocket.send_json(payload)

```

```
except (WebSocketDisconnect, RuntimeError):
    pass
```

```
=====
FILE: core/engine/lib/word/llm/websocket/dispatch.py
=====
```

```
# neurocad/core/engine/lib/word/llm/websocket/dispatch.py
```

```
"""
```

DispatchMixin – four run methods, one per WS request type.

```
_run_agent          – type: "start" (router + agent)
_run_effect_edit    – type: "effect_edit"
_run_effect_rename  – type: "effect_rename"
_run_create_effect  – type: "create_effect"
```

Every method:

1. `_check_balance()` – tokens + free guard (`llm_allowed`).
2. `_run_agent` only: if the router picks the "create" agent, additionally checks `gen > 0` on pro (`logo_allowed`).
3. Runs the agent.
4. `_charge_tokens()` – tokens -= `provider.tokens_used`.
5. `_run_agent` only: `_charge_gen()` – `gen -= 1` for create on pro.
6. Clears the cancel event.

Cancel semantics: the server never aborts an in-flight LLM request.

We wait for the current call to finish, then check

``cancel_event.is_set()`` and stop.

Balance errors:

```
'llm_not_available' – free tariff; LLM is disabled.
'tokens_exhausted'  – pro / llm; tokens have run out.
'gen_exhausted'     – pro; gen has run out (generation request).
'no_balance'        – no Balance row for this user.
```

Result frames sent to the client (see ``_run_agent``):

Streamed agents (create_page in stepwise mode)

The agent emits ``page_step`` frames itself via its ``emit`` callback (which routes to ``_safe_send``). Each frame carries one section:

```
{ "type": "page_step", "step": N, "total_hint": M,
  "section_name": "hero",
  "html": "<section ...>", "css": ".hero { ... }",
  "done": false }
```

The front end accumulates ``html`` / ``css`` and re-applies them to the canvas after every step. The final frame has ``done: true``.

For these agents the dispatcher does NOT send a bulk

``html_update`` / ``page_css_update`` at the end – the content has already been delivered section-by-section. This is signalled by the agent's return value: ``result["streamed"]` is `True``.

Non-streamed agents (all others)

```

-----
    html_update      - replace the whole canvas (create / fill)
    page_css_update  - replace the whole page CSS (create_page
                      when it responds in one shot)
    element_update   - replace one element (fill / effect)
    assistant_message - text for the chat
    done             - end of the run

Namespace: CoreEngineLibWordLlmWS (via Base + mixins)
"""

import traceback
from datetime import datetime
from typing import Any, Dict, List, Optional

from fastapi import WebSocket
from sqlalchemy import select

from .agents import _AGENTS

# Human-readable messages for balance errors. Kept next to the
# guard so the wording lives in one place.
_BALANCE_ERROR_MESSAGES = {
    "llm_not_available": (
        "LLM-чат недоступен на тарифе Free. "
        "Перейдите на тариф Pro или LLM, чтобы пользоваться чатом."
    ),
    "tokens_exhausted": (
        "Закончились токены LLM на этот месяц. "
        "Они восстановятся в расчётный день или при смене тарифа."
    ),
    "gen_exhausted": (
        "Закончились генерации страниц на этот месяц. "
        "Они восстановятся в расчётный день или при смене тарифа."
    ),
    "no_balance": (
        "Не удалось определить ваш тариф. "
        "Обратитесь к администратору."
    ),
}

class DispatchMixin:
    """Run methods for the four WS request types."""

    # =====
    # BALANCE GUARD
    # =====

    async def _send_balance_error(
        self,
        websocket: WebSocket,
        run_id: int,
        err: str,
        user_id: Optional[int] = None,

```

```

) -> None:
    """
    Send a balance error frame + done, update the run status.

    `err` is the code from BalanceChecked (llm_not_available,
    tokens_exhausted, gen_exhausted, no_balance).
    """
    from ..runs import CoreEngineLibWordLlmRuns

    message = _BALANCE_ERROR_MESSAGES.get(err, "Лимит исчерпан.")

    try:
        await CoreEngineLibWordLlmRuns.update_run_status(
            run_id,
            status="failed",
            message="Лимит исчерпан",
            error=err,
        )
    except Exception:
        pass

    await self._safe_send(
        websocket,
        {"type": "error", "code": err, "message": message},
    )
    await self._safe_send(
        websocket, {"type": "done", "run_id": run_id}
    )

async def _check_balance(
    self,
    websocket: WebSocket,
    run_id: int,
) -> Optional[Any]:
    """
    Check whether the current user may run an LLM request.

    Uses BalanceChecked.llm_allowed – blocks free users and
    users with no tokens left.

    Returns the Balance object on success, or None if blocked
    (error frame already sent).
    """
    from neurocad.core.engine.lib.balance.checked import BalanceChecked

    user_id = getattr(websocket.state, "user_id", None)
    if not user_id:
        await self._log(
            websocket, "warning",
            f"[balance] run {run_id}: no user_id on websocket.state",
        )
        await self._send_balance_error(
            websocket, run_id, "no_balance", user_id=None,
        )
    return None

```



```

log = getattr(websocket.app.state, "log", None)

bal, err = await BalanceChecked.llm_allowed(user_id, log=log)
if err:
    await self._log(
        websocket, "info",
        f"[balance] run {run_id} blocked for user {user_id}: {err}",
    )
    await self._send_balance_error(
        websocket, run_id, err, user_id=user_id,
    )
    return None

return bal

async def _check_generation(
    self,
    websocket: WebSocket,
    run_id: int,
) -> bool:
    """
    Additional guard for generation requests (create-agent).

    Uses BalanceChecked.logo_allowed – on pro also requires
    gen > 0. Returns True if the request may proceed, False if
    blocked (error frame already sent).
    """
    from neurocad.core.engine.lib.balance.checked import BalanceChecked

    user_id = getattr(websocket.state, "user_id", None)
    if not user_id:
        return False

    log = getattr(websocket.app.state, "log", None)

    _bal, err = await BalanceChecked.logo_allowed(user_id, log=log)
    if err:
        await self._log(
            websocket, "info",
            f"[balance] run {run_id} generation blocked "
            f"for user {user_id}: {err}",
        )
        await self._send_balance_error(
            websocket, run_id, err, user_id=user_id,
        )
        return False

    return True

# =====
# CHARGE
# =====

async def _charge_tokens(
    self,
    websocket: WebSocket,

```

```

        provider: Any,
    ) -> None:
        """
        Charge `provider.tokens_used` to the current user's Balance.

        Called after the agent returns successfully. If the provider
        has no `tokens_used` (0), nothing happens.

        Updates Balance.tokens (clamped at 0) and updated_at.
        """
        from neurocad.core.models.balance import Balance
        from neurocad.utils.sqlite import get_db_sqlite

        user_id = getattr(websocket.state, "user_id", None)
        usage = int(getattr(provider, "tokens_used", 0) or 0)

        if not user_id or usage <= 0:
            return

        try:
            async for session in get_db_sqlite():
                stmt = select(Balance).where(
                    Balance.user_id == user_id,
                    Balance.is_delete.is_(False),
                )
                bal = (await session.execute(stmt)).scalar_one_or_none()
                if bal is None:
                    return
                bal.tokens = max(0, (bal.tokens or 0) - usage)
                bal.updated_at = datetime.now()
                await session.commit()
                await self._log(
                    websocket, "info",
                    f"[balance] user {user_id} charged {usage} tokens "
                    f"({left {bal.tokens}})",
                )
            return
        except Exception as e:
            await self._log(
                websocket, "warning",
                f"[balance] charge tokens failed for user {user_id}: {e}",
            )

    async def _charge_gen(
        self,
        websocket: WebSocket,
        agent_name: str,
    ) -> None:
        """
        Charge 1 generation for the create-agent on pro.

        Free / llm – no charge (free is blocked earlier; llm is
        unlimited). Only runs when agent_name == "create".

        Called after agent.run() returns, so an LLM failure costs
        nothing.

```

```

"""
from neurocad.core.models.balance import Balance
from neurocad.utils.sqlite import get_db_sqlite

if agent_name != "create":
    return

user_id = getattr(websocket.state, "user_id", None)
if not user_id:
    return

try:
    async for session in get_db_sqlite():
        stmt = select(Balance).where(
            Balance.user_id == user_id,
            Balance.is_delete.is_(False),
        )
        bal = (await session.execute(stmt)).scalar_one_or_none()
        if bal is None:
            return
        # Only pro pays with gen.
        if bal.tarif != 1:
            return
        bal.gen = max(0, (bal.gen or 0) - 1)
        bal.updated_at = datetime.now()
        await session.commit()
        await self._log(
            websocket, "info",
            f"[balance] user {user_id} charged 1 gen "
            f"(left {bal.gen})",
        )
        return
except Exception as e:
    await self._log(
        websocket, "warning",
        f"[balance] charge gen failed for user {user_id}: {e}",
    )

# =====
# TYPE: start – router then agent
# =====

async def _run_agent(
    self,
    websocket: WebSocket,
    page_id: int,
    run_id: int,
    user_message: str,
    block_catalog: list,
    selection: Optional[Dict[str, Any]] = None,
    history: Optional[List[Dict[str, str]]] = None,
) -> None:
    """
    Route the request to one agent and send the result to the client.

    Steps:

```

- 0a. Balance guard – refuse if the user cannot run an LLM call.
- 1. Resolve the provider (mock or real, from the DB).
- 2. Dump meta for this run (for post-mortem debugging).
- 3. Load the current page HTML (for router state + agents).
- 4. Ask the router which agent should handle the request.
- 0b. If the router picked "create" – generation guard (gen > 0 on pro).
- 5. Run that agent and send its result to the client.
- 6. Charge tokens + (for create on pro) 1 gen.

Result frames – see the module docstring for the full list.

In short:

- streamed agents (create_page stepwise) emit their own `page\_step` frames; the dispatcher does not re-send the full html/css afterwards.
- non-streamed agents get one `html\_update` / `page\_css\_update` / `element\_update` from the dispatcher.

"""

```
from ..runs import CoreEngineLibWordLlmRuns
from ..service import CoreEngineLibWordLlmService
from ..dumper import CoreEngineLibWordLlmDumper
from ..agent.router import CoreEngineLibWordLlmRouter
```

```
# ---- 0a. BALANCE GUARD (tokens + free) ----
```

```
bal = await self._check_balance(websocket, run_id)
if bal is None:
    return
```

```
cancel_event = self._get_cancel_event(run_id)
```

```
log = getattr(websocket.app.state, "log", None)
provider = await self._get_provider(log=log)
```

```
history = history or []
```

```
CoreEngineLibWordLlmDumper.dump_meta(
    run_id=run_id,
    page_id=page_id,
    user_message=user_message,
    block_catalog=block_catalog,
    provider_name=provider.name,
    model=provider.model,
)
```

```
current_html = await self._load_current_page_html(page_id)
```

```
async def _emit(msg: dict) -> None:
    await self._safe_send(websocket, msg)
```

```
try:
```

```
    # ---- ROUTER ----
    await CoreEngineLibWordLlmRuns.update_run_status(
        run_id,
        status="planning",
        step=1,
        message="Определяю действие...",
    )
```

```

routing = await CoreEngineLibWordLlmRouter.route(
    provider=provider,
    user_message=user_message,
    selection=selection,
    current_html=current_html,
    run_id=run_id,
    emit=_emit,
)
agent_name = routing.get("agent", "help")
target = routing.get("target")

# "edit", "create_effect" and "rename" are only reachable
# via their explicit WS types. Fall back to help if the
# router returned them in a start flow.
if agent_name in ("edit", "create_effect", "rename"):
    agent_name = "help"
    target = None

if cancel_event.is_set():
    await self._finish_cancelled(websocket, run_id)
    return

# ---- 0b. GENERATION GUARD (only for create) ----
if agent_name == "create":
    ok = await self._check_generation(websocket, run_id)
    if not ok:
        return

# ---- NO TARGET FOR fill / effect ----
has_element = bool((selection or {}).get("selector"))
if agent_name in ("fill", "effect") and not has_element:
    msg_text = "Выделите элемент на странице мышкой и повторите запрос."
    await self._log(
        websocket,
        "info",
        f"[run-agent] run {run_id} {agent_name} without selection "
        f"– asking user to select",
    )
    try:
        await CoreEngineLibWordLlmService.save_chat_message(
            page_id=page_id,
            role="assistant",
            content=msg_text,
        )
    except Exception as e:
        await self._log(
            websocket,
            "warning",
            f"[ws] save assistant msg failed: {e}",
        )
    await CoreEngineLibWordLlmRuns.update_run_status(
        run_id,
        status="failed",
        message="Не выделен элемент",
        error="selection required",

```

```

    )
    await self._safe_send(
        websocket,
        {"type": "assistant_message", "content": msg_text},
    )
    await self._safe_send(
        websocket,
        {"type": "done", "run_id": run_id},
    )
    return

# ---- AGENT ----
agent_cls = _AGENTS.get(agent_name)
if agent_cls is None:
    agent_cls = _AGENTS["help"]

await CoreEngineLibWordLlmRuns.update_run_status(
    run_id,
    status="filling",
    step=2,
    message="Обрабатываю запрос...",
)

agent = agent_cls()
result = await agent.run(
    provider=provider,
    user_message=user_message,
    page_id=page_id,
    run_id=run_id,
    emit=_emit,
    selection=selection,
    block_catalog=block_catalog,
    current_html=current_html,
    history=history,
)

if cancel_event.is_set():
    await self._finish_cancelled(websocket, run_id)
    return

# ---- CHARGE ----
await self._charge_tokens(websocket, provider)
await self._charge_gen(websocket, agent_name)

# ---- RESULT ----
message = result.get("message") or ""
html = result.get("html")
css = result.get("css")
selector = result.get("selector")
element_html = result.get("element_html")
is_streamed = bool(result.get("streamed"))

# Sanity: element_html must look like HTML.
if selector and element_html:
    trimmed = str(element_html).strip()
    if not trimmed.startswith("<"):

```

```

        selector = None
        element_html = None

    try:
        await CoreEngineLibWordLlmService.save_chat_message(
            page_id=page_id,
            role="assistant",
            content=message,
        )
    except Exception as e:
        await self._log(
            websocket,
            "warning",
            f"[ws] save assistant msg failed: {e}",
        )

    # ---- store final HTML for run history ----
    # In streamed mode the agent returns the aggregated page
    # HTML (wrapped in `.core-engine-lib-word-blocks`) for
    # storage only. The client has already accumulated it
    # from the `page_step` frames.
    if html and str(html).strip().startswith("<"):
        final_html = html
    elif element_html and str(element_html).strip().startswith("<"):
        final_html = element_html
    else:
        final_html = ""
    await CoreEngineLibWordLlmRuns.set_run_final(
        run_id,
        final_html,
        message=message,
    )

    # ---- html_update – replace canvas ----
    # Skipped for streamed agents: the agent has already
    # emitted one `page_step` per section, and the front
    # end accumulated them onto the canvas. Sending a bulk
    # html_update here would cause a visible "snap" and is
    # not needed.
    if not is_streamed:
        if html and str(html).strip().startswith("<"):
            await self._safe_send(
                websocket,
                {"type": "html_update", "html": html},
            )

    # ---- page_css_update – replace page CSS ----
    if css and str(css).strip():
        await self._safe_send(
            websocket,
            {"type": "page_css_update", "css": css},
        )

    # ---- element_update – replace one element (fill / effect) ----
    # Never streamed; element updates always go through here.
    if selector and element_html:

```

```

        await self._safe_send(
            websocket,
            {
                "type": "element_update",
                "selector": selector,
                "html": element_html,
            },
        )

    if message:
        await self._safe_send(
            websocket,
            {"type": "assistant_message", "content": message},
        )

    await self._safe_send(
        websocket,
        {"type": "done", "run_id": run_id},
    )

except Exception as e:
    await self._log(
        websocket,
        "error",
        f"[run-agent] run {run_id} crashed: {e}\n{traceback.format_exc()}",
    )
    await self._finish_error(
        websocket,
        run_id,
        f"Внутренняя ошибка: {e}",
    )

finally:
    self._clear_cancel_event(run_id)

# =====
# TYPE: effect_edit
# =====

async def _run_effect_edit(
    self,
    websocket: WebSocket,
    page_id: int,
    run_id: int,
    user_message: str,
    effect_id: str,
    effect_css: str,
    effect_label: Optional[str] = None,
    existing_effect_ids: Optional[List[str]] = None,
    history: Optional[List[Dict[str, str]]] = None,
) -> None:
    """Run the "edit" agent directly."""
    from ..runs import CoreEngineLibWordLlmRuns
    from ..service import CoreEngineLibWordLlmService
    from ..dumper import CoreEngineLibWordLlmDumper

```



```

# ---- BALANCE GUARD ----
bal = await self._check_balance(websocket, run_id)
if bal is None:
    return

cancel_event = self._get_cancel_event(run_id)

log = getattr(websocket.app.state, "log", None)
provider = await self._get_provider(log=log)

history = history or []

CoreEngineLibWordLlmDumper.dump_meta(
    run_id=run_id,
    page_id=page_id,
    user_message=user_message,
    block_catalog=[],
    provider_name=provider.name,
    model=provider.model,
)

async def _emit(msg: dict) -> None:
    await self._safe_send(websocket, msg)

try:
    await CoreEngineLibWordLlmRuns.update_run_status(
        run_id,
        status="filling",
        step=1,
        message="Готовлю правку эффекта...",
    )

    agent = _AGENTS["edit"]()
    result = await agent.run(
        provider=provider,
        user_message=user_message,
        page_id=page_id,
        run_id=run_id,
        emit=_emit,
        selection=None,
        block_catalog=None,
        current_html=None,
        history=history,
        effect_id=effect_id,
        effect_css=effect_css,
        effect_label=effect_label,
        existing_effect_ids=existing_effect_ids,
    )

    if cancel_event.is_set():
        await self._finish_cancelled(websocket, run_id)
        return

# ---- CHARGE TOKENS ----
await self._charge_tokens(websocket, provider)

```

```

message = result.get("message") or ""
css = result.get("css")
result_effect_id = result.get("effect_id")
new_id = result.get("new_id")
new_label = result.get("new_label")

try:
    await CoreEngineLibWordLlmService.save_chat_message(
        page_id=page_id,
        role="assistant",
        content=message,
    )
except Exception as e:
    await self._log(
        websocket,
        "warning",
        f"[ws] save assistant msg failed: {e}",
    )

final_html = css if css else ""
await CoreEngineLibWordLlmRuns.set_run_final(
    run_id,
    final_html,
    message=message,
)

if css and result_effect_id:
    payload: Dict[str, Any] = {
        "type": "css_update",
        "effect_id": result_effect_id,
        "css": css,
        "new_id": new_id,
        "new_label": new_label,
    }
    await self._safe_send(websocket, payload)

if message:
    await self._safe_send(
        websocket,
        {"type": "assistant_message", "content": message},
    )

await self._safe_send(
    websocket,
    {"type": "done", "run_id": run_id},
)

except Exception as e:
    await self._log(
        websocket,
        "error",
        f"[run-effect-edit] run {run_id} crashed: {e}\n"
        f"{traceback.format_exc()}",
    )
    await self._finish_error(
        websocket,

```

```

        run_id,
        f"Внутренняя ошибка: {e}",
    )

    finally:
        self._clear_cancel_event(run_id)

# =====
# TYPE: effect_rename
# =====

async def _run_effect_rename(
    self,
    websocket: WebSocket,
    page_id: int,
    run_id: int,
    user_message: str,
    effect_id: str,
    effect_css: str,
    effect_label: Optional[str] = None,
    existing_effect_ids: Optional[List[str]] = None,
    history: Optional[List[Dict[str, str]]] = None,
) -> None:
    """Run the "rename" agent directly."""
    from ..runs import CoreEngineLibWordLlmRuns
    from ..service import CoreEngineLibWordLlmService
    from ..dumper import CoreEngineLibWordLlmDumper

    # ---- BALANCE GUARD ----
    bal = await self._check_balance(websocket, run_id)
    if bal is None:
        return

    cancel_event = self._get_cancel_event(run_id)

    log = getattr(websocket.app.state, "log", None)
    provider = await self._get_provider(log=log)

    history = history or []

    CoreEngineLibWordLlmDumper.dump_meta(
        run_id=run_id,
        page_id=page_id,
        user_message=user_message,
        block_catalog=[],
        provider_name=provider.name,
        model=provider.model,
    )

    async def _emit(msg: dict) -> None:
        await self._safe_send(websocket, msg)

    try:
        await CoreEngineLibWordLlmRuns.update_run_status(
            run_id,
            status="filling",

```

```

        step=1,
        message="Придумываю новое название...",
    )

agent = _AGENTS["rename"]()
result = await agent.run(
    provider=provider,
    user_message=user_message,
    page_id=page_id,
    run_id=run_id,
    emit=_emit,
    selection=None,
    block_catalog=None,
    current_html=None,
    history=history,
    effect_id=effect_id,
    effect_css=effect_css,
    effect_label=effect_label,
    existing_effect_ids=existing_effect_ids,
)

if cancel_event.is_set():
    await self._finish_cancelled(websocket, run_id)
    return

# ---- CHARGE TOKENS ----
await self._charge_tokens(websocket, provider)

message = result.get("message") or ""
css = result.get("css")
result_effect_id = result.get("effect_id")
new_label = result.get("new_label")
new_media = result.get("new_media")

try:
    await CoreEngineLibWordLlmService.save_chat_message(
        page_id=page_id,
        role="assistant",
        content=message,
    )
except Exception as e:
    await self._log(
        websocket,
        "warning",
        f"[ws] save assistant msg failed: {e}",
    )

final_html = css if css else ""
await CoreEngineLibWordLlmRuns.set_run_final(
    run_id,
    final_html,
    message=message,
)

if css and result_effect_id:
    payload: Dict[str, Any] = {

```

```

        "type": "css_update",
        "effect_id": result_effect_id,
        "css": css,
        "new_label": new_label,
        "new_media": new_media,
    }
    await self._safe_send(websocket, payload)

    if message:
        await self._safe_send(
            websocket,
            {"type": "assistant_message", "content": message},
        )

    await self._safe_send(
        websocket,
        {"type": "done", "run_id": run_id},
    )

except Exception as e:
    await self._log(
        websocket,
        "error",
        f"[run-effect-rename] run {run_id} crashed: {e}\n"
        f"{traceback.format_exc()}",
    )
    await self._finish_error(
        websocket,
        run_id,
        f"Внутренняя ошибка: {e}",
    )

finally:
    self._clear_cancel_event(run_id)

# =====
# TYPE: create_effect
# =====

async def _run_create_effect(
    self,
    websocket: WebSocket,
    page_id: int,
    run_id: int,
    user_message: str,
    previous_draft: Optional[Dict[str, Any]] = None,
    existing_effect_ids: Optional[List[str]] = None,
    history: Optional[List[Dict[str, str]]] = None,
) -> None:
    """Run the "create_effect" agent directly."""
    from ..runs import CoreEngineLibWordLlmRuns
    from ..service import CoreEngineLibWordLlmService
    from ..dumper import CoreEngineLibWordLlmDumper

    # ---- BALANCE GUARD ----
    bal = await self._check_balance(websocket, run_id)

```

```

if bal is None:
    return

cancel_event = self._get_cancel_event(run_id)

log = getattr(websocket.app.state, "log", None)
provider = await self._get_provider(log=log)

history = history or []

CoreEngineLibWordLlmDumper.dump_meta(
    run_id=run_id,
    page_id=page_id,
    user_message=user_message,
    block_catalog=[],
    provider_name=provider.name,
    model=provider.model,
)

async def _emit(msg: dict) -> None:
    await self._safe_send(websocket, msg)

try:
    await CoreEngineLibWordLlmRuns.update_run_status(
        run_id,
        status="filling",
        step=1,
        message="Придумываю эффект...",
    )

    agent = _AGENTS["create_effect"]()
    result = await agent.run(
        provider=provider,
        user_message=user_message,
        page_id=page_id,
        run_id=run_id,
        emit=_emit,
        selection=None,
        block_catalog=None,
        current_html=None,
        history=history,
        previous_draft=previous_draft,
        existing_effect_ids=existing_effect_ids,
    )

    if cancel_event.is_set():
        await self._finish_cancelled(websocket, run_id)
        return

    # ---- CHARGE TOKENS ----
    await self._charge_tokens(websocket, provider)

    message = result.get("message") or ""
    draft = result.get("draft")

    try:

```

```

        await CoreEngineLibWordLlmService.save_chat_message(
            page_id=page_id,
            role="assistant",
            content=message,
        )
    except Exception as e:
        await self._log(
            websocket,
            "warning",
            f"[ws] save assistant msg failed: {e}",
        )

    await CoreEngineLibWordLlmRuns.set_run_final(
        run_id,
        "",
        message=message,
    )

    if draft:
        await self._safe_send(
            websocket,
            {"type": "effect_draft", "draft": draft},
        )

    if message:
        await self._safe_send(
            websocket,
            {"type": "assistant_message", "content": message},
        )

    await self._safe_send(
        websocket,
        {"type": "done", "run_id": run_id},
    )

except Exception as e:
    await self._log(
        websocket,
        "error",
        f"[run-create-effect] run {run_id} crashed: {e}\n"
        f"{traceback.format_exc()}",
    )
    await self._finish_error(
        websocket,
        run_id,
        f"Внутренняя ошибка: {e}",
    )

finally:
    self._clear_cancel_event(run_id)

```

```

=====
FILE: core/engine/lib/word/llm/websocket/endpoint.py
=====

```

```

# neurocad/core/engine/lib/word/llm/websocket/endpoint.py

```

"""

EndpointMixin – the WebSocket endpoint and its main loop.

This is the only entry point of the LLM editor over WS:

`/core/engine/lib/word/llm/ws/{page_id}`

One connection per open editor tab. The loop:

1. Authenticates the user (cookie JWT).
2. Accepts the connection (any authenticated user).
3. Reads messages in a loop and dispatches by `type`:

```
start          – full flow: router → agent (see _run_agent)
effect_edit    – direct: "edit" agent (see _run_effect_edit)
effect_rename  – direct: "rename" agent (see _run_effect_rename)
create_effect  – direct: "create_effect" agent
                  (see _run_create_effect)
cancel         – sets the cancel flag for the current run
ping           – replies with pong
```

Anything else → `error` frame.

4. On disconnect (client closed tab, network drop): sets the cancel flag for the current run, marks it cancelled in the DB, and returns. The endpoint does not try to keep the connection alive.

One task per connection:

The dispatcher runs in `asyncio.create\_task(...)` so the main loop stays responsive to `cancel` and `ping` while an agent is working. Only ONE task can be active at a time – a new `start` (or effect_edit, or effect_rename, or create_effect) refuses to run while a task is in flight and sends an `error` frame instead.

Cancel semantics:

Cancelling does NOT abort the in-flight LLM request. The cancel event is checked by the dispatcher after the current call finishes. See dispatch.py for the exact check points.

This method used to live on CoreEngineLibWordLlmWS as a @staticmethod.

It is now a regular method on a mixin – called as

``self.llm_ws_endpoint(...)`` from route.py (via the wrapper in ws.py).

Namespace: CoreEngineLibWordLlmWS (via Base + mixins)

"""

```
import asyncio
import json
import traceback
from typing import Optional

from fastapi import WebSocket, WebSocketDisconnect

from .agents import _AGENTS
```



```

class EndpointMixin:
    """WebSocket endpoint + per-connection main loop."""

    async def llm_ws_endpoint(self, websocket: WebSocket, page_id: int) -> None:
        """WebSocket endpoint. Mounted as /ws/{page_id}."""
        print(f"!!!! [ws] ENDPOINT REACHED page_id={page_id} !!!!!", flush=True)

        await self._log(
            websocket,
            "info",
            f"[ws] === incoming connection for page {page_id} ===",
        )

        # ---- Auth ----
        user = await self._authenticate_ws(websocket)
        await self._log(websocket, "info", f"[ws] auth result: user={user}")

        if not user:
            await websocket.close(code=4403)
            return

        # Any authenticated user may use the LLM chat.
        # (Previously this required is_superuser – removed so that
        # regular users can also use the editor.)

        await websocket.accept()
        await self._log(
            websocket,
            "info",
            f"[ws] page {page_id} connected (user_id={user.get('id')})",
        )

        # Per-connection state.
        current_task: Optional[asyncio.Task] = None
        current_run_id: Optional[int] = None

        try:
            while True:
                # ---- Receive ----
                try:
                    msg = await websocket.receive_json()
                except json.JSONDecodeError as e:
                    await self._log(websocket, "warning", f"[ws] invalid JSON: {e}")
                    await self._safe_send(
                        websocket,
                        {"type": "error", "message": "Некорректный JSON"},
                    )
                    continue

                mtype = msg.get("type")

                # ---- Keepalive ----
                if mtype == "ping":
                    await self._safe_send(websocket, {"type": "pong"})
                    continue

```

```

await self._log(
    websocket,
    "info",
    f"[ws] received: type={mtype}, keys={list(msg.keys())}",
)

# =====
# TYPE: start – full flow through the router
# =====
if mtype == "start":
    print("=" * 60, flush=True)
    print("[START-IN] raw message from frontend:", flush=True)
    print(
        json.dumps(msg, ensure_ascii=False, indent=2)[:4000],
        flush=True,
    )
    print("=" * 60, flush=True)

    # Supersede any active run for this page.
    await self._supersede_active_run(page_id, current_run_id)

    # Refuse a new start while a task is in flight.
    if current_task and not current_task.done():
        await self._safe_send(
            websocket,
            {
                "type": "error",
                "message": "Предыдущий запрос ещё выполняется",
            },
        )
        continue

    user_message = msg.get("message", "").strip()
    block_catalog = msg.get("block_catalog") or []
    selection = msg.get("selection") or None

    if not user_message:
        await self._safe_send(
            websocket,
            {"type": "error", "message": "Пустой запрос"},
        )
        continue

    history = await self._load_history(page_id)
    await self._persist_user_message(page_id, user_message)

    run = await self._create_run(
        page_id=page_id,
        user_message=user_message,
        block_catalog=block_catalog,
        user_id=user.get("id"),
    )
    if not run:
        await self._safe_send(
            websocket,
            {"type": "error", "message": "Не удалось создать run"},

```

```

        )
        continue

    current_run_id = run["id"]
    self._get_cancel_event(current_run_id)

    await self._safe_send(
        websocket,
        {"type": "run_started", "run_id": current_run_id},
    )

    current_task = asyncio.create_task(
        self._run_agent(
            websocket=websocket,
            page_id=page_id,
            run_id=current_run_id,
            user_message=user_message,
            block_catalog=block_catalog,
            selection=selection,
            history=history,
        )
    )

# =====
# TYPE: effect_edit – direct edit agent, no router
# =====
elif mtype == "effect_edit":
    print("=" * 60, flush=True)
    print("[EFFECT-EDIT-IN] raw message from frontend:", flush=True)
    print(
        json.dumps(msg, ensure_ascii=False, indent=2)[:4000],
        flush=True,
    )
    print("=" * 60, flush=True)

    await self._supersede_active_run(page_id, current_run_id)

    if current_task and not current_task.done():
        await self._safe_send(
            websocket,
            {
                "type": "error",
                "message": "Предыдущий запрос ещё выполняется",
            },
        )
        continue

    user_message = msg.get("message", "").strip()
    effect_id = (msg.get("effect_id") or "").strip()
    effect_css = msg.get("effect_css") or ""
    # Current human label + list of ids already taken
    # by OTHER effects. Forwarded to the edit agent so
    # it can decide whether to propose a rename
    # (new_id + new_label).
    effect_label = msg.get("effect_label") or None
    raw_existing = msg.get("existing_effect_ids") or []

```

```

existing_effect_ids = [
    str(x).strip()
    for x in raw_existing
    if isinstance(x, (str, int)) and str(x).strip()
]

if not user_message:
    await self._safe_send(
        websocket,
        {"type": "error", "message": "Пустой запрос"},
    )
    continue

if not effect_id or not effect_css:
    await self._safe_send(
        websocket,
        {
            "type": "error",
            "message": "Режим правки эффекта не активен",
        },
    )
    continue

await self._log(
    websocket,
    "info",
    f"[ws] effect_edit effect_id={effect_id} "
    f"label={effect_label!r} "
    f"existing={len(existing_effect_ids)}",
)

history = await self._load_history(page_id)
await self._persist_user_message(page_id, user_message)

run = await self._create_run(
    page_id=page_id,
    user_message=user_message,
    block_catalog=[],
    user_id=user.get("id"),
)

if not run:
    await self._safe_send(
        websocket,
        {"type": "error", "message": "Не удалось создать run"},
    )
    continue

current_run_id = run["id"]
self._get_cancel_event(current_run_id)

await self._safe_send(
    websocket,
    {"type": "run_started", "run_id": current_run_id},
)

current_task = asyncio.create_task(

```

```

        self._run_effect_edit(
            websocket=websocket,
            page_id=page_id,
            run_id=current_run_id,
            user_message=user_message,
            effect_id=effect_id,
            effect_css=effect_css,
            effect_label=effect_label,
            existing_effect_ids=existing_effect_ids,
            history=history,
        )
    )

# =====
# TYPE: effect_rename – direct rename agent, no router
# =====
elif mtype == "effect_rename":
    print("=" * 60, flush=True)
    print("[EFFECT-RENAME-IN] raw message from frontend:", flush=True)
    print(
        json.dumps(msg, ensure_ascii=False, indent=2)[:4000],
        flush=True,
    )
    print("=" * 60, flush=True)

    await self._supersede_active_run(page_id, current_run_id)

    if current_task and not current_task.done():
        await self._safe_send(
            websocket,
            {
                "type": "error",
                "message": "Предыдущий запрос ещё выполняется",
            },
        )
        continue

    user_message = msg.get("message", "").strip()
    effect_id = (msg.get("effect_id") or "").strip()
    effect_css = msg.get("effect_css") or ""
    effect_label = msg.get("effect_label") or None
    raw_existing = msg.get("existing_effect_ids") or []
    existing_effect_ids = [
        str(x).strip()
        for x in raw_existing
        if isinstance(x, (str, int)) and str(x).strip()
    ]

    if not effect_id or not effect_css:
        await self._safe_send(
            websocket,
            {
                "type": "error",
                "message": "Режим переименования не активен",
            },
        )

```

```

        continue

    await self._log(
        websocket,
        "info",
        f"[ws] effect_rename effect_id={effect_id} "
        f"label={effect_label!r} "
        f"existing={len(existing_effect_ids)}",
    )

    history = await self._load_history(page_id)
    await self._persist_user_message(
        page_id,
        user_message or "переименование эффекта",
    )

    run = await self._create_run(
        page_id=page_id,
        user_message=user_message or "переименование эффекта",
        block_catalog=[],
        user_id=user.get("id"),
    )
    if not run:
        await self._safe_send(
            websocket,
            {"type": "error", "message": "Не удалось создать run"},
        )
        continue

    current_run_id = run["id"]
    self._get_cancel_event(current_run_id)

    await self._safe_send(
        websocket,
        {"type": "run_started", "run_id": current_run_id},
    )

    current_task = asyncio.create_task(
        self._run_effect_rename(
            websocket=websocket,
            page_id=page_id,
            run_id=current_run_id,
            user_message=user_message,
            effect_id=effect_id,
            effect_css=effect_css,
            effect_label=effect_label,
            existing_effect_ids=existing_effect_ids,
            history=history,
        )
    )

# =====
# TYPE: create_effect – direct draft agent, no router
# =====
elif mtype == "create_effect":
    print("=" * 60, flush=True)

```

```

print("[CREATE-EFFECT-IN] raw message from frontend:", flush=True)
print(
    json.dumps(msg, ensure_ascii=False, indent=2)[:4000],
    flush=True,
)
print("=" * 60, flush=True)

await self._supersede_active_run(page_id, current_run_id)

if current_task and not current_task.done():
    await self._safe_send(
        websocket,
        {
            "type": "error",
            "message": "Предыдущий запрос ещё выполняется",
        },
    )
    continue

user_message = msg.get("message", "").strip()
previous_draft = msg.get("previous_draft") or None

# Ids already registered in the palette. Passed to
# the create_effect agent so it can tell the LLM
# not to pick a colliding id.
raw_existing = msg.get("existing_effect_ids") or []
existing_effect_ids = [
    str(x).strip()
    for x in raw_existing
    if isinstance(x, (str, int)) and str(x).strip()
]

if not user_message:
    await self._safe_send(
        websocket,
        {"type": "error", "message": "Пустой запрос"},
    )
    continue

await self._log(
    websocket,
    "info",
    f"[ws] create_effect existing={len(existing_effect_ids)}",
)

history = await self._load_history(page_id)
await self._persist_user_message(page_id, user_message)

run = await self._create_run(
    page_id=page_id,
    user_message=user_message,
    block_catalog=[],
    user_id=user.get("id"),
)
if not run:
    await self._safe_send(

```

```

        websocket,
        {"type": "error", "message": "Не удалось создать run"},
    )
    continue

    current_run_id = run["id"]
    self._get_cancel_event(current_run_id)

    await self._safe_send(
        websocket,
        {"type": "run_started", "run_id": current_run_id},
    )

    current_task = asyncio.create_task(
        self._run_create_effect(
            websocket=websocket,
            page_id=page_id,
            run_id=current_run_id,
            user_message=user_message,
            previous_draft=previous_draft,
            existing_effect_ids=existing_effect_ids,
            history=history,
        )
    )

# =====
# TYPE: cancel
# =====
elif mtype == "cancel":
    await self._log(
        websocket,
        "info",
        f"[ws] cancel received for run {current_run_id}",
    )
    if current_run_id:
        ev = self._cancel_events.get(current_run_id)
        if ev:
            ev.set()
        await self._cancel_run_in_db(
            current_run_id,
            "cancelled by user",
        )

# =====
# Unknown type
# =====
else:
    await self._safe_send(
        websocket,
        {
            "type": "error",
            "message": f"Неизвестный тип сообщения: {mtype}",
        },
    )

# ---- Client closed the socket ----

```



```

except WebSocketDisconnect as e:
    await self._log(
        websocket,
        "info",
        f"[ws] page {page_id} disconnected: "
        f"code={e.code} reason={e.reason!r}",
    )
    if current_run_id:
        ev = self._cancel_events.get(current_run_id)
        if ev:
            ev.set()
            await self._cancel_run_in_db(
                current_run_id,
                "websocket disconnected",
            )

# ---- Unexpected error in the main loop ----
except Exception as e:
    await self._log(
        websocket,
        "error",
        f"[ws] unexpected error in main loop: {e}\n"
        f"{traceback.format_exc()}",
    )
    raise

# ---- Always: clear the cancel event for the last run ----
finally:
    if current_run_id:
        self._clear_cancel_event(current_run_id)
    await self._log(
        websocket,
        "info",
        f"[ws] handler for page {page_id} finished",
    )

# =====
# SMALL HELPERS USED ONLY BY THE ENDPOINT
# =====

async def _supersede_active_run(
    self,
    page_id: int,
    current_run_id: Optional[int],
) -> None:
    """
    Cancel any DB-registered active run for this page, and set its
    cancel event if we own it. Called at the start of every
    start / effect_edit / effect_rename / create_effect handler.

    Safe to call when there is no active run – a no-op.
    """
    from ..runs import CoreEngineLibWordLlmRuns

    try:
        active = await CoreEngineLibWordLlmRuns.get_active_run_for_page(page_id)

```

```

except Exception as e:
    print(f"[ws] get_active_run failed: {e}", flush=True)
    return

if not active:
    return

try:
    await CoreEngineLibWordLlmRuns.cancel_run(
        active["id"],
        "superseded by a new run",
    )
except Exception as e:
    print(f"[ws] cancel_run failed: {e}", flush=True)

# If we still hold the event in this process, set it.
ev = self._cancel_events.get(active["id"])
if ev:
    ev.set()

async def _persist_user_message(self, page_id: int, text: str) -> None:
    """
    Save the user's message into page_chat. Best-effort – a failure
    here is logged but does not abort the run.
    """
    from ..service import CoreEngineLibWordLlmService

    try:
        await CoreEngineLibWordLlmService.save_chat_message(
            page_id=page_id,
            role="user",
            content=text,
        )
    except Exception as e:
        print(f"[ws] save user msg failed: {e}", flush=True)

async def _create_run(
    self,
    page_id: int,
    user_message: str,
    block_catalog: list,
    user_id: Optional[int],
) -> Optional[dict]:
    """
    Create a run row in the DB. Returns the run dict, or None on
    any failure (the caller sends an error frame).
    """
    from ..runs import CoreEngineLibWordLlmRuns

    try:
        return await CoreEngineLibWordLlmRuns.create_run(
            page_id=page_id,
            user_message=user_message,
            block_catalog=block_catalog,
            user_id=user_id,
        )

```

```

except Exception as e:
    print(f"[ws] create_run failed: {e}", flush=True)
    return None

async def _cancel_run_in_db(self, run_id: int, reason: str) -> None:
    """
    Mark a run as cancelled in the DB. Best-effort – a failure is
    logged and swallowed.
    """
    from ..runs import CoreEngineLibWordLlmRuns

    try:
        await CoreEngineLibWordLlmRuns.cancel_run(run_id, reason)
    except Exception as e:
        print(f"[ws] cancel_run({run_id}) failed: {e}", flush=True)

```

```

=====
FILE: core/engine/lib/word/llm/websocket/history.py
=====

```

```

# neurocad/core/engine/lib/word/llm/websocket/history.py

```

```

"""
HistoryMixin – history and page-context helpers for the WebSocket endpoint.

```

Two responsibilities:

1. `_load_history(page_id)`
Loads the last `HISTORY_LIMIT` messages from ``page_chat`` and returns them in the OpenAI/Anthropic-compatible shape:

```

[{"role": "user"|"assistant", "content": "..."}, ...]

```

Only user/assistant messages are kept – system prompts are constructed fresh by each agent. Non-string content is dropped.

`HISTORY_LIMIT` is defined on the base class (20) – this mixin reads it via ``self.HISTORY_LIMIT``.

2. `_load_current_page_html(page_id)`
Loads ``Page.content`` for the current page (or None if missing).
Used by:
 - the router – to know whether the page already has content;
 - the fill/effect agents – to see the current markup.

Read-only. Never raises.

These methods used to be `@staticmethod` on `CoreEngineLibWordLlmWS`. They are now regular methods on a mixin, so they are called as ``self._load_history(...)`` / ``self._load_current_page_html(...)``.

Import depth note

```

-----
This file lives one level deeper than the old ws.py:

```

```

llm/ws.py           ← old location
llm/websocket/history.py ← new location (+1 level)

```

Relative import depth from llm/websocket/history.py:

```
..          → llm/
...         → word/
....        → lib/
.....      → engine/
.....      → core/
.....      → neurocad/    ← utils/, models/ live here
```

So imports into neurocad/ need SEVEN dots, imports into neurocad.core/ need SIX. Anything that was `from ....X` in ws.py becomes `from .....X` here if X lives under neurocad.core, and `from .....X` if X lives directly under neurocad.

For imports into llm/ itself (`..service`, `..runs`, `..dumper`, `..agent.\*`, `..mock`), two dots are enough – the parent of websocket/ is llm/.

Namespace: CoreEngineLibWordLlmWS (via Base + mixins)
"""

```
from typing import Dict, List, Optional
```

```
class HistoryMixin:
```

```
    """History and page-context helpers for the WebSocket endpoint."""
```

```
    # =====
    # CHAT HISTORY
    # =====
```

```
    async def _load_history(self, page_id: int) -> List[Dict[str, str]]:
        """
```

```
        Load the last HISTORY_LIMIT messages from page_chat.
```

```
        Returns a list of {"role", "content"} dicts, oldest first.
        Empty list on any failure – the caller treats a missing
        history as "no prior context".
        """
```

```
        # Import here, not at module level: the service module pulls
        # in the whole app + DB stack. Keeping the import local to the
        # method means this mixin can be importable even when the DB
        # is not yet configured (useful for tooling / static checks).
        from ..service import CoreEngineLibWordLlmService
```

```
        try:
```

```
            rows = await CoreEngineLibWordLlmService.load_chat_history(page_id)
        except Exception as e:
            print(f"[ws-history] load failed for page {page_id}: {e}", flush=True)
            return []
```

```
        limit = self.HISTORY_LIMIT
        tail = rows[-limit:] if len(rows) > limit else rows
```

```
        return [
            {"role": m["role"], "content": m["content"]}
        ]
```

```

        for m in tail
        if m.get("role") in ("user", "assistant") and m.get("content")
    ]

# =====
# CURRENT PAGE HTML
# =====

async def _load_current_page_html(self, page_id: int) -> Optional[str]:
    """
    Load the current page HTML from the Page table.

    Returns the HTML string, or None if the page is missing,
    deleted, or the read fails. Never raises.
    """
    try:
        # Seven dots for neurocad.utils, six for neurocad.core.models.
        # See the module docstring for the full count. Do not
        # "clean up" the dots without checking the tree – this
        # file is one level deeper than ws.py used to be.
        from .....models.base import Page
        from .....utils.sqlite import get_db_sqlite
        from sqlalchemy import select

        async for session in get_db_sqlite():
            stmt = select(Page).where(
                Page.id == page_id,
                Page.is_delete == 0,
            )
            result = await session.execute(stmt)
            page = result.scalar_one_or_none()
            if not page:
                return None
            return page.content or None

    except Exception as e:
        print(f"[ws] load current page failed: {e}", flush=True)
        return None

    return None

```

```

=====
FILE: core/engine/lib/word/llm/websocket/provider.py
=====

```

```

# neurocad/core/engine/lib/word/llm/websocket/provider.py

```

```

"""

```

ProviderMixin – resolve the LLM provider for one run.

The provider is the object every agent talks to via
`provider.generate\_completion(messages)`. Which provider is used
is decided at run time:

- LLM MOCK=1 (env / settings) → a mock provider, no network calls.
Used in tests and local development when no real API key is
configured.

- Otherwise → the factory ``get_provider(log=...)`` reads the active provider name and its config from the DB (``settings`` table, key ``llm``) and returns an instance wired to that vendor.

``log`` – `app.state.log`, so the provider can write its own diagnostics (OAuth refresh failures, TLS/CA issues) into the shared log file. It is passed through, not stored.

The factory call is async because it hits the DB. Therefore ``_get_provider`` is async too.

These methods used to be `@staticmethod` on `CoreEngineLibWordLlmWS`. Now they are regular methods on a mixin – called as ``self._get_provider(log=...)``.

Import depth note

This file lives one level deeper than the old `ws.py`:

```
llm/ws.py           ← old location
llm/websocket/provider.py ← new location (+1 level)
```

Relative import depth from `llm/websocket/provider.py`:

```
..          → llm/
...         → word/
....        → lib/
.....       → engine/
.....       → core/
.....       → neurocad/ ← utils.llm.factory lives here
```

So the factory import needs SEVEN dots, not six. Anything that was ``from .....X`` in `ws.py` becomes ``from .....X`` here if X lives under the neurocad package root (`utils/`, `config`, etc.).

For imports into `llm/` itself (``..mock``, ``..runs``, ``..service``, ``..dumper``, ``..agent.*``), two dots are enough – the parent of `websocket/` is `llm/`.

Namespace: `CoreEngineLibWordLlmWS` (via Base + mixins)

```
from typing import Any, Optional
```

```
class ProviderMixin:
```

```
    """Resolve the LLM provider (mock or real) for one run."""
```

```
    async def _get_provider(self, log: Optional[Any] = None):
```

```
        """
```

```
        Resolve the LLM provider.
```

```
        Returns an object exposing at least:
```

- `name` : str, provider identifier (e.g. "openai")
- `model` : str, active model id
- `generate_completion(messages)` -> str (async)

`log` is passed to the factory so the provider can write diagnostics into the shared app log. It is not stored on the provider instance.

Never returns None: the factory raises on misconfiguration, and the caller (dispatcher) catches and reports it as a run error. That is intentional – a run without a provider cannot produce a result.

```
"""
# Imported lazily, so this mixin stays importable even when
# the settings module or the LLM factory are not yet
# configured (useful for static checks).
from neurocad.config import settings

if settings.LLM MOCK:
    # Mock provider – no network, deterministic answers.
    # Delay comes from settings so tests can speed it up.
    from ..mock import CoreEngineLibWordLlmMock
    return CoreEngineLibWordLlmMock(delay=settings.LLM MOCK_DELAY)

# Real provider – resolved from the DB via the factory.
# Seven dots, because this file is one level deeper than the
# old ws.py. See the module docstring for the full count.
from .....utils.llm.factory import get_provider
return await get_provider(log=log)
```

```
=====
FILE: core/engine/lib/word/llm/websocket/__init__.py
=====
```

```
=====
FILE: core/engine/lib/word/editor/route.py
=====
```

```
# neurocad/core/engine/lib/word/editor/route.py
```

```
"""
```

Editor sub-router for the Word component.

Namespace: CoreEngineLibWordEditorRoute

Mounted from word/route.py with no extra prefix – the parent router already carries prefix="/word", and this router adds "/editor".

Final paths (after word router's prefix):

/core/engine/lib/word/editor/...

Currently mounted here:

/core/engine/lib/word/editor/effects/...	– effects editor API
/core/engine/lib/word/editor/images/...	– images editor API
/core/engine/lib/word/editor/io/...	– import / export API

New editor sub-APIs (toolbar, history, templates, ...) go here.

```
"""
```

```
from fastapi import APIRouter
```

```

from .effects.route import router as effects_router
from .images.route import router as images_router
from .io.route import router as io_router


router = APIRouter(
    prefix="/editor",
    tags=["core/engine/lib/word/editor"],
)


# Effects editor API
router.include_router(effects_router)


# Images editor API
router.include_router(images_router)


# Import / export API
router.include_router(io_router)


=====
FILE: core/engine/lib/word/editor/__init__.py
=====


=====
FILE: core/engine/lib/pages/route.py
=====
# neurocad/core/engine/lib/pages/route.py


from fastapi import APIRouter, Depends, HTTPException, Query, Request
from fastapi.responses import JSONResponse
from typing import Optional


from sqlalchemy import select


from neurocad.core.auth.dependencies import get_current_user
from neurocad.core.models.nav import Nav
from neurocad.utils.sqlite import get_db_sqlite
from neurocad.core.engine.lib.balance.checked import BalanceChecked
from .service import CoreEngineLibPagesService
from .schema import (
    CoreEngineLibPagesItemCreate,
    CoreEngineLibPagesItemUpdate,
)


router = APIRouter(prefix="/pages", tags=["core/engine/lib/pages"])


# =====
# NAV RESOLUTION
# =====


async def _resolve_nav_id(
    request: Request,
    explicit_nav_id: Optional[int],

```



```

    current_user: Optional[dict] = None,
) -> int:
    """
    Resolve the nav instance for the current request.

    Priority:
    1. Explicit nav_id – used ONLY if it belongs to the current
       user (or the user is a superadmin). If it does not belong
       to the user, we silently fall back to the user's own nav.
       This keeps the UX clean: no 403s, no error pages, the user
       always sees their own catalog.
    2. First nav of the current user (ORDER BY id ASC, is_delete=0).

    Raises:
    401 if there is no authenticated user.
    404 if the current user has no nav at all.
    """
    if current_user is None:
        current_user = await get_current_user(request)

    user_id = current_user.get("id") if isinstance(current_user, dict) else None
    if user_id is None:
        raise HTTPException(status_code=401, detail="Not authenticated")

    is_superuser = (
        current_user.get("is_superuser") is True
        or current_user.get("is_superuser") == 1
        or current_user.get("is_superuser") == "1"
    )

    async for session in get_db_sqlite():
        # ---- 1. Explicit nav_id – accept only if it belongs to the user ----
        if explicit_nav_id is not None:
            if is_superuser:
                return explicit_nav_id

            stmt = select(Nav).where(
                Nav.id == explicit_nav_id,
                Nav.user_id == user_id,
                Nav.is_delete == False,
            )
            nav = (await session.execute(stmt)).scalar_one_or_none()
            if nav is not None:
                return nav.id

            # Foreign nav – silently fall through to the user's own nav.

        # ---- 2. Fallback: current user's first nav ----
        stmt = (
            select(Nav)
            .where(Nav.user_id == user_id, Nav.is_delete == False)
            .order_by(Nav.id.asc())
            .limit(1)
        )
        nav = (await session.execute(stmt)).scalar_one_or_none()
        if not nav:

```

```

        raise HTTPException(
            status_code=404,
            detail="No nav found for the current user",
        )
    return nav.id

    raise HTTPException(status_code=500, detail="DB error")

# =====
# LIST
# =====

@router.get("/list")
async def get_pages_list(
    request: Request,
    nav_id: Optional[int] = Query(
        None,
        description="Nav instance ID (optional; defaults to the user's first nav)",
    ),
    page: int = Query(1, ge=1, description="Page number"),
    limit: int = Query(20, ge=1, le=100, description="Page size"),
    is_active: Optional[int] = Query(None, description="Filter: 1 – active, 0 – inactive"),
    is_template: Optional[int] = Query(None, description="Filter: 1 – templates only, 0 – regular only"),
) -> JSONResponse:
    """Get a paginated list of articles. Public endpoint."""
    resolved_nav_id = await _resolve_nav_id(request, nav_id)

    result = await CoreEngineLibPagesService.get_list(
        nav_id=resolved_nav_id,
        page=page,
        limit=limit,
        is_active=is_active,
        is_template=is_template,
    )

    return JSONResponse({
        "success": True,
        "data": result["items"],
        "total": result["total"],
        "page": result["page"],
        "limit": result["limit"],
    })

# =====
# MY LIST – для редактора (выбор ссылки)
# =====
#
# ВАЖНО: этот роут ДОЛЖЕН быть объявлен ДО /item/{item_id}.
# FastAPI матчит роуты в порядке добавления; иначе /my-list
# будет интерпретирован как item_id="my-list" и упадёт валидация.

@router.get("/my-list")
async def get_my_pages_list(
    request: Request,

```

```

exclude_page_id: Optional[int] = Query(
    None,
    description="Исключить страницу из списка (нельзя ссылаться на себя)",
),
current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """
    Список страниц текущего пользователя – для выбора ссылки
    в редакторе (trait page-link).

    Возвращает только активные, неудалённые, не-шаблонные страницы
    всех nav текущего юзера. Каждая с готовым URL вида
    /page/<nav_id>/<YYYYMMDD>/<HHMMSS>.

    Any authenticated user.
    """
    user_id = current_user.get("id")
    items = await CoreEngineLibPagesService.list_for_user(
        user_id,
        exclude_page_id=exclude_page_id,
    )
    return JSONResponse({
        "success": True,
        "data": items,
    })

# =====
# ONE ITEM BY ID
# =====

@router.get("/item/{item_id}")
async def get_page_item(
    item_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
) -> JSONResponse:
    """Get one article by ID. Public endpoint."""
    resolved_nav_id = await _resolve_nav_id(request, nav_id)

    item = await CoreEngineLibPagesService.get_item(item_id, nav_id=resolved_nav_id)

    if not item:
        raise HTTPException(status_code=404, detail="Article not found")

    return JSONResponse({
        "success": True,
        "data": item,
    })

# =====
# ONE ITEM BY DATETIME
# =====

@router.get("/bydatetime/{date}/{time}")

```

```

async def get_page_by_datetime(
    date: str,
    time: str,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
) -> JSONResponse:
    """
    Get one article by date and time.

    Example: /core/engine/lib/pages/bydatetime/20260914/153910

    date = "20260914" (YYYYMMDD)
    time = "153910" (HHMMSS)

    Public endpoint.
    """
    resolved_nav_id = await _resolve_nav_id(request, nav_id)

    item = await CoreEngineLibPagesService.get_by_datetime(
        date, time, nav_id=resolved_nav_id
    )

    if not item:
        raise HTTPException(status_code=404, detail="Page not found")

    return JSONResponse({
        "success": True,
        "data": item,
    })

# =====
# CREATE
# =====

@router.post("/item")
async def create_page_item(
    data: CoreEngineLibPagesItemCreate,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """
    Create a new article. Any authenticated user.

    Before creating – checks the page limit via BalanceChecked
    (lazy, based on the owner's tariff).

    On limit exceeded → 403 with detail 'pages_exhausted'.
    """
    # get_current_user raises 401 for guests – no extra permission check needed.
    user_id = current_user.get("id")
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    # Pre-check: is the user allowed to create a new page?
    bal, err, current_pages = await BalanceChecked.page_allowed(

```

```

        user_id, log=request.app.state.log
    )
    if err:
        raise HTTPException(status_code=403, detail=err)

    item = await CoreEngineLibPagesService.create_item(data, nav_id=resolved_nav_id)

    if not item:
        raise HTTPException(status_code=400, detail="Failed to create article")

    return JSONResponse({
        "success": True,
        "data": item,
    })

# =====
# UPDATE
# =====

@router.put("/item/{item_id}")
async def update_page_item(
    item_id: int,
    data: CoreEngineLibPagesItemUpdate,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """Update an article. Any authenticated user."""
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    item = await CoreEngineLibPagesService.update_item(
        item_id, data, nav_id=resolved_nav_id
    )

    if not item:
        raise HTTPException(status_code=404, detail="Article not found")

    return JSONResponse({
        "success": True,
        "data": item,
    })

# =====
# DELETE (SOFT)
# =====

@router.delete("/item/{item_id}")
async def delete_page_item(
    item_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """Soft-delete an article. Any authenticated user."""

```

```

        resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

        result = await CoreEngineLibPagesService.delete_item(
            item_id, nav_id=resolved_nav_id
        )

    if not result:
        raise HTTPException(status_code=404, detail="Article not found")

    return JSONResponse({
        "success": True,
        "message": "Article deleted",
    })

# =====
# RESTORE
# =====

@router.post("/item/{item_id}/restore")
async def restore_page_item(
    item_id: int,
    request: Request,
    nav_id: Optional[int] = Query(None, description="Nav instance ID (optional)"),
    current_user: dict = Depends(get_current_user),
) -> JSONResponse:
    """Restore a soft-deleted article. Any authenticated user."""
    resolved_nav_id = await _resolve_nav_id(request, nav_id, current_user=current_user)

    result = await CoreEngineLibPagesService.restore_item(
        item_id, nav_id=resolved_nav_id
    )

    if not result:
        raise HTTPException(status_code=404, detail="Article not found")

    return JSONResponse({
        "success": True,
        "message": "Article restored",
    })

=====
FILE: core/engine/lib/pages/schema.py
=====
# app/core/engine/lib/pages/schema.py

from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime as dt

# =====
# BASE SCHEMA
# =====

class CoreEngineLibPagesItemBase(BaseModel):

```

```

    """Base article fields"""
    datetime: Optional[dt] = Field(None, description="Publication date and time. If omitted, set automatically.")
    title: str = Field(..., min_length=1, max_length=255, description="Title")
    description: Optional[str] = Field(None, description="Short description")
    logo: Optional[str] = Field(None, description="Logo (URL or emoji)")
    is_active: int = Field(1, description="Active: 1 – yes, 0 – no")

    # Template system
    is_template: int = Field(0, description="Base template: 1 – yes, 0 – no")
    template_id: Optional[int] = Field(None, description="Base template ID (inheritance)")

# =====
# CREATE
# =====

class CoreEngineLibPagesItemCreate(CoreEngineLibPagesItemBase):
    """Article creation"""
    content: Optional[str] = Field(None, description="HTML for display")
    content_json: Optional[str] = Field(None, description="GrapesJS JSON for the editor")

# =====
# UPDATE
# =====

class CoreEngineLibPagesItemUpdate(BaseModel):
    """Article update – all fields optional"""
    datetime: Optional[dt] = None
    title: Optional[str] = Field(None, min_length=1, max_length=255)
    description: Optional[str] = None
    logo: Optional[str] = None
    content: Optional[str] = None
    content_json: Optional[str] = None
    is_active: Optional[int] = None

    # Template system
    is_template: Optional[int] = None
    template_id: Optional[int] = None

# =====
# LIST ITEM (without content/content_json)
# =====

class CoreEngineLibPagesItemListItem(BaseModel):
    """Article for list view – without heavy fields"""
    id: int
    datetime: dt
    title: str
    description: Optional[str] = None
    logo: Optional[str] = None
    is_active: int
    is_delete: int
    created_at: Optional[dt] = None
    updated_at: Optional[dt] = None

```

```

    rss_yandex_id: Optional[str] = None

    # Template system
    is_template: int = 0
    template_id: Optional[int] = None

    class Config:
        from_attributes = True

# =====
# FULL ARTICLE (with content/content_json)
# =====

class CoreEngineLibPagesItemResponse(CoreEngineLibPagesItemListItem):
    """Full article – with content and content_json"""
    content: Optional[str] = None
    content_json: Optional[str] = None

# =====
# ARTICLE LIST (response)
# =====

class CoreEngineLibPagesListResponse(BaseModel):
    """Response with a list of articles"""
    items: list[CoreEngineLibPagesItemListItem]
    total: int = 0
    page: int = 1
    limit: int = 20

=====
FILE: core/engine/lib/pages/service.py
=====
# neurocad/core/engine/lib/pages/service.py

from sqlalchemy import select, func
from datetime import datetime, timedelta
from typing import Optional, Dict, Any, List
from ...models.base import Page
from ...models.nav import Nav
from .schema import (
    CoreEngineLibPagesItemCreate,
    CoreEngineLibPagesItemUpdate,
)
from neurocad.utils.sqlite import get_db_sqlite
from neurocad.core.engine.lib.balance.checked import BalanceChecked

class CoreEngineLibPagesService:
    """Service for working with articles."""

    # =====
    # LIST
    # =====

```



```

@staticmethod
async def get_list(
    nav_id: int,
    page: int = 1,
    limit: int = 20,
    is_active: Optional[int] = None,
    is_template: Optional[int] = None,
) -> Dict[str, Any]:
    """
    Get a paginated list of articles.

    nav_id – nav instance (whose catalog this is)
    page – page number (1-based)
    limit – page size
    is_active – filter: 1 (active only), 0 (inactive only), None (all)
    is_template – filter: 1 (templates only), 0 (regular only), None (all)
    """
    offset = (page - 1) * limit

    async for session in get_db_sqlite():
        base_stmt = select(Page).where(
            Page.nav_id == nav_id,
            Page.is_delete == 0,
        )

        if is_active is not None:
            base_stmt = base_stmt.where(Page.is_active == is_active)

        if is_template is not None:
            base_stmt = base_stmt.where(Page.is_template == is_template)

        count_stmt = select(func.count()).select_from(base_stmt.subquery())
        total_result = await session.execute(count_stmt)
        total = total_result.scalar_one()

        stmt = (
            base_stmt
            .order_by(Page.datetime.desc())
            .offset(offset)
            .limit(limit)
        )
        result = await session.execute(stmt)
        rows = result.scalars().all()

        items = []
        for page_item in rows:
            items.append({
                "id": page_item.id,
                "datetime": page_item.datetime.isoformat() if page_item.datetime else None,
                "title": page_item.title,
                "description": page_item.description,
                "logo": page_item.logo,
                "is_active": page_item.is_active,
                "is_delete": page_item.is_delete,
                "is_template": page_item.is_template or 0,
                "template_id": page_item.template_id,
            })

```

```

        "created_at": page_item.created_at.isoformat() if page_item.created_at else None,
        "updated_at": page_item.updated_at.isoformat() if page_item.updated_at else None,
        "rss_yandex_id": page_item.rss_yandex_id,
    })

    return {
        "items": items,
        "total": total,
        "page": page,
        "limit": limit,
    }

    return {"items": [], "total": 0, "page": page, "limit": limit}

# =====
# LIST FOR USER – для редактора (выбор ссылки)
# =====

@staticmethod
async def list_for_user(
    user_id: int,
    exclude_page_id: Optional[int] = None,
) -> List[Dict[str, Any]]:
    """
    Список страниц пользователя – для выбора ссылки в редакторе.

    Возвращает только активные, неудалённые, не-шаблонные страницы
    всех nav текущего юзера. Каждая с готовым URL вида
    /page/<nav_id>/<YYYYMMDD>/<HHMMSS>.

    exclude_page_id – исключить конкретную страницу (обычно – ту,
    которую сейчас редактирует пользователь: нельзя ссылаться на
    себя).

    Формат элемента:
    {
        "id": <page.id>,
        "title": <page.title>,
        "url": "/page/<nav_id>/<YYYYMMDD>/<HHMMSS>"
    }
    """
    async for session in get_db_sqlite():
        # Найти все nav пользователя (по возрастанию id –
        # чтобы порядок был предсказуемым)
        nav_stmt = (
            select(Nav.id)
            .where(
                Nav.user_id == user_id,
                Nav.is_delete == False,
            )
            .order_by(Nav.id.asc())
        )
        nav_rows = (await session.execute(nav_stmt)).all()
        nav_ids = [r[0] for r in nav_rows]
        if not nav_ids:
            return []

```

```

# Все активные неудалённые не-шаблонные страницы этих nav
stmt = (
    select(Page)
    .where(
        Page.nav_id.in_(nav_ids),
        Page.is_delete == 0,
        Page.is_active == 1,
        Page.is_template == 0,
    )
    .order_by(Page.datetime.desc())
)
if exclude_page_id is not None:
    stmt = stmt.where(Page.id != exclude_page_id)

result = await session.execute(stmt)
rows = result.scalars().all()

items: List[Dict[str, Any]] = []
for p in rows:
    if not p.datetime:
        continue
    url = (
        f"/page/{p.nav_id}/"
        f"{p.datetime.strftime('%Y%m%d')}/"
        f"{p.datetime.strftime('%H%M%S')}"
    )
    items.append({
        "id": p.id,
        "title": p.title or f"Страница {p.id}",
        "url": url,
    })

return items

return []

# =====
# ONE ITEM BY ID
# =====

@staticmethod
async def get_item(item_id: int, nav_id: int) -> Optional[Dict[str, Any]]:
    """Get one article by ID (with content and content_json)."""
    async for session in get_db_sqlite():
        stmt = select(Page).where(
            Page.id == item_id,
            Page.nav_id == nav_id,
            Page.is_delete == 0,
        )
        result = await session.execute(stmt)
        page_item = result.scalar_one_or_none()

        if not page_item:
            return None

```

```

    return {
        "id": page_item.id,
        "datetime": page_item.datetime.isoformat() if page_item.datetime else None,
        "title": page_item.title,
        "description": page_item.description,
        "logo": page_item.logo,
        "content": page_item.content,
        "content_json": page_item.content_json,
        "is_active": page_item.is_active,
        "is_delete": page_item.is_delete,
        "is_template": page_item.is_template or 0,
        "template_id": page_item.template_id,
        "created_at": page_item.created_at.isoformat() if page_item.created_at else None,
        "updated_at": page_item.updated_at.isoformat() if page_item.updated_at else None,
        "rss_yandex_id": page_item.rss_yandex_id,
    }

    return None

# =====
# ONE ITEM BY DATETIME
# =====

@staticmethod
async def get_by_datetime(
    date: str,
    time: str,
    nav_id: int,
) -> Optional[Dict[str, Any]]:
    """
    Find a page by date and time within a nav.

    date = "20260914" (YYYYMMDD)
    time = "153910" (HHMMSS)

    datetime in the DB is stored with microseconds (15:39:10.666406),
    so we search in the range [dt_start, dt_start + 1 sec).
    """
    try:
        dt_start = datetime.strptime(f"{date}{time}", "%Y%m%d%H%M%S")
    except ValueError as e:
        print(f"[Pages] Invalid date/time: {date} {time} - {e}")
        return None

    dt_end = dt_start + timedelta(seconds=1)

    async for session in get_db_sqlite():
        stmt = select(Page).where(
            Page.nav_id == nav_id,
            Page.datetime >= dt_start,
            Page.datetime < dt_end,
            Page.is_delete == 0,
        )
        result = await session.execute(stmt)
        page_item = result.scalar_one_or_none()

```

```

    if not page_item:
        return None

    return {
        "id": page_item.id,
        "datetime": page_item.datetime.isoformat() if page_item.datetime else None,
        "title": page_item.title,
        "description": page_item.description,
        "logo": page_item.logo,
        "content": page_item.content,
        "content_json": page_item.content_json,
        "is_active": page_item.is_active,
        "is_delete": page_item.is_delete,
        "is_template": page_item.is_template or 0,
        "template_id": page_item.template_id,
        "created_at": page_item.created_at.isoformat() if page_item.created_at else None,
        "updated_at": page_item.updated_at.isoformat() if page_item.updated_at else None,
        "rss_yandex_id": page_item.rss_yandex_id,
    }

    return None

# =====
# CREATE
# =====

@staticmethod
async def create_item(
    data: CoreEngineLibPagesItemCreate,
    nav_id: int,
) -> Optional[Dict[str, Any]]:
    """
    Create a new article in the given nav.

    Before creating – checks the page limit via BalanceChecked
    (lazy, based on the owner's tariff). If the limit is hit,
    returns None – the route translates this into HTTP 403.

    If datetime is not provided, the current time is used.
    """
    # Resolve the owner of this nav – needed for the balance check.
    user_id = await CoreEngineLibPagesService._resolve_user_id(nav_id)
    if user_id is None:
        print(f"[Pages] create_item: nav {nav_id} has no owner")
        return None

    # Lazy limit check (BalanceChecked internally applies the
    # day-change accruals before checking).
    bal, err, current_pages = await BalanceChecked.page_allowed(user_id)
    if err:
        print(f"[Pages] create_item: user {user_id} – {err} "
              f"(pages={current_pages}, limit={bal.limit_pages if bal else '?'})")
        return None

    item_datetime = data.datetime
    if item_datetime is None:

```

```

        item_datetime = datetime.now()

    async for session in get_db_sqlite():
        new_item = Page(
            nav_id=nav_id,
            datetime=item_datetime,
            title=data.title.strip(),
            description=data.description,
            logo=data.logo,
            content=data.content,
            content_json=data.content_json,
            is_active=data.is_active,
            is_delete=0,
            is_template=getattr(data, "is_template", 0) or 0,
            template_id=getattr(data, "template_id", None),
            created_at=datetime.now(),
            updated_at=datetime.now(),
        )
        session.add(new_item)
        await session.commit()
        await session.refresh(new_item)

    return {
        "id": new_item.id,
        "datetime": new_item.datetime.isoformat() if new_item.datetime else None,
        "title": new_item.title,
        "description": new_item.description,
        "logo": new_item.logo,
        "content": new_item.content,
        "content_json": new_item.content_json,
        "is_active": new_item.is_active,
        "is_delete": new_item.is_delete,
        "is_template": new_item.is_template or 0,
        "template_id": new_item.template_id,
        "created_at": new_item.created_at.isoformat() if new_item.created_at else None,
        "updated_at": new_item.updated_at.isoformat() if new_item.updated_at else None,
    }

    return None

# =====
# UPDATE
# =====

@staticmethod
async def update_item(
    item_id: int,
    data: CoreEngineLibPagesItemUpdate,
    nav_id: int,
) -> Optional[Dict[str, Any]]:
    """Update an article."""
    async for session in get_db_sqlite():
        stmt = select(Page).where(
            Page.id == item_id,
            Page.nav_id == nav_id,
            Page.is_delete == 0,

```

```

    )
    result = await session.execute(stmt)
    page_item = result.scalar_one_or_none()

    if not page_item:
        return None

    update_data = data.model_dump(exclude_unset=True)

    # template_id can be explicitly set to None (unlink from template)
    for key, value in update_data.items():
        if not hasattr(page_item, key):
            continue
        if value is None and key != "template_id":
            continue
        setattr(page_item, key, value)

    page_item.updated_at = datetime.now()
    await session.commit()
    await session.refresh(page_item)

    return {
        "id": page_item.id,
        "datetime": page_item.datetime.isoformat() if page_item.datetime else None,
        "title": page_item.title,
        "description": page_item.description,
        "logo": page_item.logo,
        "content": page_item.content,
        "content_json": page_item.content_json,
        "is_active": page_item.is_active,
        "is_delete": page_item.is_delete,
        "is_template": page_item.is_template or 0,
        "template_id": page_item.template_id,
        "created_at": page_item.created_at.isoformat() if page_item.created_at else None,
        "updated_at": page_item.updated_at.isoformat() if page_item.updated_at else None,
    }

    return None

# =====
# DELETE (SOFT)
# =====

@staticmethod
async def delete_item(item_id: int, nav_id: int) -> bool:
    """Soft-delete an article."""
    async for session in get_db_sqlite():
        stmt = select(Page).where(
            Page.id == item_id,
            Page.nav_id == nav_id,
            Page.is_delete == 0,
        )
        result = await session.execute(stmt)
        page_item = result.scalar_one_or_none()

        if not page_item:

```

```

        return False

    page_item.is_delete = 1
    page_item.updated_at = datetime.now()
    await session.commit()
    return True

    return False

# =====
# RESTORE
# =====

@staticmethod
async def restore_item(item_id: int, nav_id: int) -> bool:
    """
    Restore a soft-deleted article.

    Restoring may push the user over the page limit – the check
    is NOT run here, because restored pages belong to the user
    (they were created earlier, under a valid tariff).
    """
    async for session in get_db_sqlite():
        stmt = select(Page).where(
            Page.id == item_id,
            Page.nav_id == nav_id,
            Page.is_delete == 1,
        )
        result = await session.execute(stmt)
        page_item = result.scalar_one_or_none()

        if not page_item:
            return False

        page_item.is_delete = 0
        page_item.updated_at = datetime.now()
        await session.commit()
        return True

    return False

# =====
# INTERNAL – NAV OWNER
# =====

@staticmethod
async def _resolve_user_id(nav_id: int) -> Optional[int]:
    """Return the owner (user_id) of a nav, or None."""
    async for session in get_db_sqlite():
        stmt = select(Nav.user_id).where(
            Nav.id == nav_id,
            Nav.is_delete.is_(False),
        )
        result = await session.execute(stmt)
        return result.scalar_one_or_none()
    return None

```



```
=====
FILE: core/engine/lib/pages/__init__.py
=====
```

```
=====
FILE: utils/cors.py
=====
```

```
# neurocad/utils/cors.py
```

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
```

```
from ..config import settings
```

```
def setup_cors(app: FastAPI) -> None:
```

```
    """
```

```
    Настройка CORS.
```

```
    На локальной разработке (APP_HOST == 127.0.0.1/localhost) CORS
    отключается полностью – фронт и бэк на одном origin, а наличие
    CORSMiddleware с allow_credentials=True ломает WebSocket-handshake.
```

```
    На проде – включается с конкретными origin'ами (без "*", т.к.
    при allow_credentials=True wildcard запрещён спецификацией).
```

```
    """
```

```
    host = settings.APP_HOST
```

```
    if host in ("127.0.0.1", "localhost", "::1"):
        # Локальная разработка: CORS не нужен вообще
        return
```

```
    app.add_middleware(
        CORSMiddleware,
        allow_origins=[
            f"https://{host}",
            f"https://www.{host}",
            # сюда добавляй остальные домены явно
        ],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )
```

```
=====
FILE: utils/crypto.py
=====
```

```
# neurocad/utils/crypto.py
```

```
"""
```

```
Symmetric encryption helpers.
```

```
Generic Fernet wrapper – no knowledge of which fields are secret.
Callers decide what to encrypt.
```

If NEUROCAD_SECRET_KEY is missing or invalid, encrypt()/decrypt() are no-ops (return input unchanged) and is_crypto_available() returns False.

Not to be confused with app/utils/hash.py – that's one-way password hashing, not reversible encryption.

"""

```
from typing import Optional
```

```
from cryptography.fernet import Fernet, InvalidToken
```

```
from neurocad.config import settings
```

```
_fernet: Optional[object] = None
```

```
def _get_fernet():
```

```
    """Return the cached Fernet instance, or None if crypto is disabled."""
```

```
    global _fernet
```

```
    if _fernet is not None:
```

```
        return _fernet if _fernet is not False else None
```

```
    key = getattr(settings, "NEUROCAD_SECRET_KEY", None)
```

```
    if not key:
```

```
        # No key – encryption disabled. Callers can check via
```

```
        # is_crypto_available() to warn the user.
```

```
        _fernet = False
```

```
        return None
```

```
    try:
```

```
        _fernet = Fernet(key.encode("utf-8") if isinstance(key, str) else key)
```

```
        return _fernet
```

```
    except Exception:
```

```
        # Invalid key – encryption disabled. Callers can check via
```

```
        # is_crypto_available().
```

```
        _fernet = False
```

```
        return None
```

```
def is_crypto_available() -> bool:
```

```
    """True if a valid NEUROCAD_SECRET_KEY is configured."""
```

```
    return _get_fernet() is not None
```

```
def encrypt(plaintext: Optional[str]) -> Optional[str]:
```

```
    """
```

```
    Encrypt a string with Fernet. Returns a base64 str.
```

```
    - None → None
```

```
    - '' → ''
```

```
    - If crypto is disabled → returns plaintext unchanged.
```

```

- On any error → returns plaintext unchanged.
"""
if plaintext is None:
    return None
if plaintext == "":
    return ""

f = _get_fernet()
if f is None:
    return plaintext

try:
    return f.encrypt(plaintext.encode("utf-8")).decode("ascii")
except Exception:
    return plaintext

def decrypt(ciphertext: Optional[str]) -> Optional[str]:
    """
    Decrypt a Fernet token. Returns the original plaintext.

    - None → None
    - '' → ''
    - If crypto is disabled → returns input unchanged.
    - If the token is invalid → returns input unchanged
      (key rotated, or the value was stored as plaintext).

    Never raises.
    """
    if ciphertext is None:
        return None
    if ciphertext == "":
        return ""

    f = _get_fernet()
    if f is None:
        return ciphertext

    try:
        return f.decrypt(ciphertext.encode("ascii")).decode("utf-8")
    except InvalidToken:
        # Key rotated, or the value was stored in plaintext.
        return ciphertext
    except Exception:
        return ciphertext

```

```

=====
FILE: utils/css.py
=====

```

```

# neurocad/utils/css.py

```

```

"""
CSS utilities.

```

```

Generic helpers for deriving cached CSS files from DB-stored CSS:

```

- `css_hash(css)` – short stable hash for ?v= cache-busting
- `ensure_css_file(...)` – write CSS to a file (atomic), return URL
- `split_style_from_html(...)` – extract legacy `<style>` blocks

Where the files live and what URL they map to is NOT decided here. The caller (the module that owns the CSS) passes ``directory`` and ``url_prefix`` explicitly – so each module keeps its own namespace, and this utility stays module-agnostic.

Permissions

When this utility creates a directory or a file, it does NOT rely on the process ``umask`` – services can start with ``umask 077``, which would make new files ``rw-----`` (mode 600), unreadable for the web server.

Instead, the created entry inherits the mode of its parent:

- a new directory gets the parent's mode, with the execute bit set (so it can be entered / traversed);
- a new file gets the parent's mode with all execute bits cleared (files are never executable).

Result: whatever mode the surrounding static tree uses (755, 775, ...) propagates to the files this utility writes.

Legacy helper `split_style_from_html()` is used for pages saved before CSS was split into a separate field (CSS embedded in HTML as `<style>`).
 """

```
import hashlib
import os
import re
import stat as stat_mod
import tempfile
from pathlib import Path
from typing import Optional, Tuple, Union

# Legacy <style> extraction.
_STYLE_RE = re.compile(r"<style[^>]*>([\s\S]*?)</style>", re.IGNORECASE)
```

```
# =====
# PERMISSION HELPERS
# =====
```

```
def _inherit_dir_mode(parent: Path) -> None:
    """
    Apply the parent directory's mode to `parent` itself?

    No – this function is unused. Kept for symmetry: use
    `_apply_dir_mode` on a newly created directory.
    """
    pass
```

```

def _apply_dir_mode(new_dir: Path) -> None:
    """
    Set `new_dir`'s mode to match its parent's mode, plus execute bits.

    Why: when the utility creates a directory (e.g. static/.../pages/),
    the process umask decides its mode – and for systemd services the
    umask is often 077, giving `rw-----` (mode 700). Nginx then can't
    traverse into it. Copying the parent's mode (plus x bits) keeps the
    whole static tree consistent, whatever mode it uses.

    Fails silently: a permission error here must not break the caller.
    """
    try:
        parent_mode = stat_mod.S_IMODE(os.stat(new_dir.parent).st_mode)
        # Directory must be traversable – ensure x bits are present
        # wherever r bits are present.
        mode = parent_mode | ((parent_mode & 0o444) >> 2)
        os.chmod(new_dir, mode)
    except OSError:
        pass


def _apply_file_mode(path: Path) -> None:
    """
    Set `path`'s mode to match its parent's mode, with execute bits cleared.

    Same rationale as `_apply_dir_mode`: files this utility writes must be
    readable by the web server regardless of the service umask. Files are
    never executable, so x bits are stripped.

    Fails silently.
    """
    try:
        parent_mode = stat_mod.S_IMODE(os.stat(path.parent).st_mode)
        os.chmod(path, parent_mode & ~0o111)
    except OSError:
        pass


# =====
# PUBLIC API
# =====


def css_hash(css: Optional[str]) -> str:
    """
    Short stable hash of CSS content.
    Returns '' for empty CSS – caller must handle.

    Used as ?v= in <link>, NOT stored in DB.
    """
    if not css or not css.strip():
        return ""
    return hashlib.sha256(css.encode("utf-8")).hexdigest()[:8]


def ensure_css_file(

```

```

page_id: int,
css: Optional[str],
directory: Union[str, Path],
url_prefix: str,
) -> Optional[str]:
    """
    Write <directory>/<page_id>.css from css (if not already present
    with the same content), return "<url_prefix>/<page_id>.css?v=<hash>".

```

Returns None if css is empty.

Args:

```

page_id:    numeric id used as the file name (e.g. Page.id).
css:        CSS content. Empty/whitespace → None.
directory:  filesystem directory where the file lives.
            Created if missing (parents included).
url_prefix: URL prefix that maps 1:1 to `directory`.
            For example, directory="static/foo/bar" and
            url_prefix="/static/foo/bar" – default Nginx
            serves URL → filesystem without rewrites.

```

Atomic write via temp file + os.replace – so concurrent readers never see a half-written file.

Permissions: new directories and files inherit the mode of their parent (files without execute bits). The process umask is ignored on purpose – see the module docstring.

The caller is responsible for passing a directory/url_prefix pair that actually corresponds (URL must resolve to the file on disk).

```

    """
    if not css or not css.strip():
        return None

    h = css_hash(css)
    dir_path = Path(directory)

    # ---- Ensure the directory exists; fix its mode if we created it ----
    dir_existed = dir_path.is_dir()
    dir_path.mkdir(parents=True, exist_ok=True)
    if not dir_existed:
        _apply_dir_mode(dir_path)

    path = dir_path / f"{page_id}.css"

    # Skip write if file already has the same content.
    need_write = True
    if path.exists():
        try:
            if path.read_text(encoding="utf-8") == css:
                need_write = False
        except OSError:
            need_write = True

    if need_write:
        fd, tmp_path = tempfile.mkstemp(

```

```

        dir=str(dir_path),
        prefix=f".{page_id}.",
        suffix=".css.tmp",
    )
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            f.write(css)
        # Move temp → final, atomically.
        os.replace(tmp_path, path)
        # Fix the final file's mode – parent mode, no x bits.
        _apply_file_mode(path)
    except Exception:
        try:
            os.unlink(tmp_path)
        except OSError:
            pass
        raise

    prefix = url_prefix.rstrip("/")
    return f"{prefix}/{page_id}.css?v={h}"

```

```

def split_style_from_html(html: str) -> Tuple[str, str]:
    """

```

Extract all <style> blocks from HTML.

Returns (css, html_without_style).

Used as a fallback for legacy pages where CSS was embedded in Page.content as <style>...</style> (before the css field was introduced).

```

    if not html:
        return "", ""

```

```

    styles = _STYLE_RE.findall(html)
    css = "\n".join(s.strip() for s in styles if s and s.strip())
    html_clean = _STYLE_RE.sub("", html)
    return css, html_clean

```

```

=====
FILE: utils/database.py
=====

```

```

# app/utils/mysql.py

```

```

from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine, async_sessionmaker
from sqlalchemy import text
from ..config import settings

```

```

# Переменные будут инициализированы только при вызове init_mysql()
engine = None
AsyncSessionLocal = None

```

```

async def init_mysql(log=None):
    """Инициализация MySQL (проверка подключения и создание engine)"""

```

```

global engine, AsyncSessionLocal

# Проверяем, нужно ли подключаться
...

if not settings.has_mysql:
    msg = "MySQL не настроен (DATABASE_URL отсутствует или не MySQL)"
    if log:
        await log.log_info(target="mysql", message=msg)
    else:
        print(f"❌ {msg}")
    return False
...

try:
    # Создаём engine ТОЛЬКО сейчас
    engine = create_async_engine(
        settings.DATABASE_URL,
        echo=settings.DEBUG,
        pool_size=10,
        max_overflow=20,
        pool_pre_ping=True
    )

    AsyncSessionLocal = async_sessionmaker(
        engine,
        class_=AsyncSession,
        expire_on_commit=False
    )

    # Проверяем подключение
    async with engine.begin() as conn:
        await conn.execute(text("SELECT 1"))

    if log:
        await log.log_info(target="mysql", message="Подключение к MySQL успешно")
    else:
        print("✅ MySQL connected")
    return True

except Exception as e:
    error_msg = f"Ошибка подключения к MySQL: {e}"
    if log:
        await log.log_error(target="mysql", message=error_msg)
    else:
        print(f"❌ {error_msg}")
    raise

async def get_db():
    """Генератор сессий для Dependency Injection"""
    if AsyncSessionLocal is None:
        raise RuntimeError("MySQL не инициализирован. Вызовите init_mysql() сначала.")

    async with AsyncSessionLocal() as session:
        yield session

```



```

async def close_mysql():
    """Закрытие соединения с MySQL"""
    global engine, AsyncSessionLocal

    if engine is not None:
        try:
            await engine.dispose()
            print("MySQL disconnected")
        except Exception as e:
            print(f"Error closing MySQL: {e}")
        finally:
            engine = None
            AsyncSessionLocal = None
    else:
        print("MySQL не был инициализирован, закрывать нечего")

=====
FILE: utils/hash.py
=====
# app/utils/hash.py

import hashlib
import base64

def get_hash_string(text: str) -> str:
    """
    Многослойное хэширование пароля:
    1. MD5(text) → base64
    2. Добавляем 'SD' → MD5 → base64
    3. Добавляем 'EN' → SHA1 → base64
    """
    # Шаг 1: MD5(text) → base64
    md5_hash = hashlib.md5(text.encode('utf-8'), usedforsecurity=False).digest()
    md5_base64 = base64.b64encode(md5_hash).decode()

    # Шаг 2: Добавляем 'SD' → MD5 → base64
    md5_base64_saltd = md5_base64 + 'SD'
    md5_hash_saltd = hashlib.md5(md5_base64_saltd.encode('utf-8'), usedforsecurity=False).digest()
    md5_base64_saltd_hash = base64.b64encode(md5_hash_saltd).decode()

    # Шаг 3: Добавляем 'EN' → SHA1 → base64
    sha_input = md5_base64_saltd_hash + 'EN'
    sha1_hash = hashlib.sha1(sha_input.encode('utf-8'), usedforsecurity=False).digest()
    sha1_base64 = base64.b64encode(sha1_hash).decode()

    return sha1_base64

def verify_password(raw_password: str, hashed_password: str) -> bool:
    """
    Проверяет, соответствует ли пароль хэшу из БД
    """
    return get_hash_string(raw_password) == hashed_password

=====

```

FILE: utils/lifespan.py

=====

neurocad/utils/lifespan.py

```
from fastapi import FastAPI
from contextlib import asynccontextmanager
```

```
from ..utils.log import Log
from ..utils.mysql import init_mysql, close_mysql
from ..utils.sqlite import init_sqlite, close_sqlite
from ..utils.paths import ensure_workdirs
from ..config import settings
```

```
def get_lifespan():
    """Return lifespan context manager."""
```

```
@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
    """Application lifecycle manager."""
```

```
    # Ensure working directories and files exist
    ensure_workdirs()
```

```
    # Logger
    app.state.log = Log()
```

```
    # Startup log
    await app.state.log.log_info(
        target="startup",
        message=f"Starting application on port {settings.APP_PORT}",
    )
```

```
    # MySQL – only if DATABASE_URL is set
    if settings.DATABASE_URL:
        await init_mysql(app.state.log)
    else:
        await app.state.log.log_info(
            target="startup",
            message="MySQL disabled (DATABASE_URL is empty)",
        )
```

```
    # SQLite – always.
    # Includes: migrations, superadmin, modules registration.
    await init_sqlite(app.state.log)
```

```
    yield
```

```
    # Shutdown MySQL – only if connected
    if settings.DATABASE_URL:
        await close_mysql()
```

```
    # Shutdown SQLite
    await close_sqlite(log=app.state.log)
```

```
    # Shutdown log
```

```

        await app.state.log.log_info(
            target="shutdown",
            message="Stopping application",
        )

        # Shutdown logger
        await app.state.log.shutdown()

    return lifespan

```

```
=====
FILE: utils/log.py
=====
```

```

# neurocad/utils/log.py
# Логирование событий

```

```

import os
import datetime
from aiologger import Logger
from aiologger.handlers.files import AsyncFileHandler
import logging

```

```
class Log:
```

```

    def __init__(self, log_dir: str = "log"):
        self.log_dir = log_dir
        os.makedirs(self.log_dir, exist_ok=True)
        self.handlers = {}

```

```

    def build_log_path(self, target: str, now: datetime.datetime) -> str:
        """

```

```

        Формируем путь к лог-файлу:
        <log_dir>/2025/10/04.log
        """

```

```

        year = f"{now.year}"
        month = f"{now:%m}"
        day = f"{now:%d}"

```

```

        base_dir = os.path.join(self.log_dir, year, month)
        os.makedirs(base_dir, exist_ok=True)

```

```

        filename = f"{day}.log"
        return os.path.join(base_dir, filename)

```

```

    async def get_logger(self, target: str, now: datetime.datetime) -> Logger:

```

```

        """Асинхронный логгер для target."""
        log_path = self.build_log_path(target, now)

```

```

        if target not in self.handlers or self.handlers[target]["path"] != log_path:
            handler = AsyncFileHandler(filename=log_path, mode="a", encoding="utf-8")
            target_logger = Logger(name=f"logger_{target}")
            target_logger.add_handler(handler)

```

```

        if target in self.handlers:
            try:
                await self.handlers[target]["logger"].shutdown()

```

```

        except Exception:
            pass

        self.handlers[target] = {
            "path": log_path,
            "logger": target_logger,
            "handler": handler,
        }

    return self.handlers[target]["logger"]

# =====
# АСИНХРОННЫЕ МЕТОДЫ
# =====

async def log_info(
    self,
    target: str = "",
    message: str = "",
    data: dict | None = None,
):
    if data is None:
        data = {}

    now = datetime.datetime.now()

    log_data_str = f"{now:%d.%m.%Y %H:%M:%S} {target}: {message}"
    if data:
        log_data_str += f": {self.safe_serialize(data)}"

    target_logger = await self.get_logger(target, now)
    await target_logger.info(log_data_str)

async def log_error(
    self,
    target: str = "",
    message: str = "",
    data: dict | None = None,
):
    await self.log_info(target, f"ERROR: {message}", data)

async def log_warning(
    self,
    target: str = "",
    message: str = "",
    data: dict | None = None,
):
    await self.log_info(target, f"WARNING: {message}", data)

# =====
# СИНХРОННЫЕ МЕТОДЫ
# =====

def log_info_sync(
    self,
    target: str = "",

```

```

        message: str = "",
        data: dict | None = None,
    ):
        if data is None:
            data = {}

        now = datetime.datetime.now()
        log_path = self.build_log_path(target, now)
        os.makedirs(os.path.dirname(log_path), exist_ok=True)

        log_data_str = f"{now:%d.%m.%Y %H:%M:%S} {target}: {message}"
        if data:
            log_data_str += f": {self.safe_serialize(data)}"

        logger = logging.getLogger(f"sync_logger_{target}")
        logger.setLevel(logging.INFO)

        if not logger.handlers:
            handler = logging.FileHandler(log_path, mode="a", encoding="utf-8")
            formatter = logging.Formatter("%(message)s")
            handler.setFormatter(formatter)
            logger.addHandler(handler)

        logger.info(log_data_str)

def log_error_sync(
    self,
    target: str = "",
    message: str = "",
    data: dict | None = None,
):
    self.log_info_sync(target, f"ERROR: {message}", data)

def log_warning_sync(
    self,
    target: str = "",
    message: str = "",
    data: dict | None = None,
):
    self.log_info_sync(target, f"WARNING: {message}", data)

# =====
# УТИЛИТЫ
# =====

def safe_serialize(self, obj):
    """
    Преобразуем объект в сериализуемый вид для JSON/log:
    - dict, list, tuple рекурсивно
    - Pydantic модели через model_dump
    - любые несериализуемые объекты → строка с типом
    """
    if obj is None:
        return None
    elif isinstance(obj, (str, int, float, bool)):
        return obj

```

```

    elif isinstance(obj, dict):
        return {k: self.safe_serialize(v) for k, v in obj.items()}
    elif isinstance(obj, (list, tuple, set)):
        return [self.safe_serialize(v) for v in obj]
    elif hasattr(obj, "model_dump"): # Pydantic
        return self.safe_serialize(obj.model_dump())
    elif hasattr(obj, "__dict__"):
        # игнорируем приватные атрибуты
        return {k: self.safe_serialize(v) for k, v in vars(obj).items() if not k.startswith("_")}
    else:
        # для всего остального выводим тип объекта
        return f"<{type(obj).__name__}>"

    async def shutdown(self):
        for h in list(self.handlers.values()):
            try:
                await h["logger"].shutdown()
            except Exception:
                pass

```

```

=====
FILE: utils/migrations.py
=====

```

```

# neurocad/utils/migrations.py

```

```

"""Apply Alembic migrations from package and user directories."""

```

```

from pathlib import Path
from alembic.config import Config
from alembic import command

```

```

from .paths import (
    package_alembic_dir,
    package_alembic_ini,
    user_alembic_versions_dir,
)

```

```

def _log_sync(log, level: str, message: str) -> None:
    """Write through app.state.log if available, else silently."""
    if log is None:
        return
    fn = getattr(log, f"log_{level}_sync", None)
    if fn is None:
        return
    try:
        fn(target="migrations", message=message)
    except Exception:
        pass

```

```

def _collect_version_locations(root: Path) -> list[str]:
    """
    Recursively collect all directories containing .py migration files.

    Alembic's `recursive_version_locations` does not work reliably

```

when version_locations is set programmatically, so we walk the tree ourselves and pass an explicit list of directories.

Supports nested layouts like:

```
versions/2026/09/19/011744_init.py
"""
locations: list[str] = []
if not root.exists():
    return locations

for path in [root] + [p for p in root.rglob("*") if p.is_dir()]:
    if any(path.glob("*.py")):
        locations.append(str(path))

return locations
```

```
def apply_migrations(sync_url: str | None = None, log=None):
    """
    Apply Alembic migrations.

    Migrations are loaded from two locations:
    1. Package: neurocad/alembic/versions/
    2. User:    <cwd>/alembic/versions/ (if exists)

    sync_url – optional, overrides sqlalchemy.url from alembic.ini.
    log       – app.state.log from init_sqlite. If None – silent.
    """
    from ..config import settings

    ini_path = package_alembic_ini()
    if not ini_path.exists():
        raise RuntimeError(f"alembic.ini not found in package: {ini_path}")

    cfg = Config(str(ini_path))
    cfg.set_main_option("script_location", str(package_alembic_dir()))
    cfg.set_main_option("sqlalchemy.url", sync_url or settings.SQLITE_URL_SYNC)

    # ===== Collect version_locations =====
    locations = _collect_version_locations(package_alembic_dir() / "versions")

    user_versions = user_alembic_versions_dir()
    if user_versions.exists() and user_versions.is_dir():
        user_locations = _collect_version_locations(user_versions)
        if user_locations:
            locations.extend(user_locations)
            _log_sync(log, "info", f"User migrations dirs: {user_locations}")

    if not locations:
        _log_sync(log, "warning", "No migration directories found.")

    cfg.set_main_option("version_locations", " ".join(locations))

    _log_sync(log, "info", "Applying Alembic migrations...")
    command.upgrade(cfg, "head")
    _log_sync(log, "info", "Migrations applied.")
```

```

=====
FILE: utils/mysql.py
=====
# neurocad/utils/mysql.py

from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine, async_sessionmaker
from sqlalchemy import text
from ..config import settings

# Переменные будут инициализированы только при вызове init_mysql()
engine = None
AsyncSessionLocal = None

async def init_mysql(log=None):
    """Инициализация MySQL (проверка подключения и создание engine).

    log – app.state.log from lifespan. If None – silent.
    """
    global engine, AsyncSessionLocal

    try:
        # Создаём engine ТОЛЬКО сейчас
        engine = create_async_engine(
            settings.DATABASE_URL,
            echo=settings.DEBUG,
            pool_size=10,
            max_overflow=20,
            pool_pre_ping=True
        )

        AsyncSessionLocal = async_sessionmaker(
            engine,
            class_=AsyncSession,
            expire_on_commit=False
        )

        # Проверяем подключение
        async with engine.begin() as conn:
            await conn.execute(text("SELECT 1"))

        if log is not None:
            await log.log_info(target="mysql", message="MySQL connected")
            return True

    except Exception as e:
        error_msg = f"MySQL connection error: {e}"
        if log is not None:
            await log.log_error(target="mysql", message=error_msg)
        raise

async def get_db():
    """Генератор сессий для Dependency Injection"""
    if AsyncSessionLocal is None:

```



```

        raise RuntimeError("MySQL не инициализирован. Вызовите init_mysql() сначала.")

    async with AsyncSessionLocal() as session:
        yield session

async def close_mysql(log=None):
    """Закрытие соединения с MySQL.

    log – app.state.log from lifespan. If None – silent.
    """
    global engine, AsyncSessionLocal

    if engine is not None:
        try:
            await engine.dispose()
            if log is not None:
                await log.log_info(target="mysql", message="MySQL disconnected")
        except Exception as e:
            if log is not None:
                await log.log_error(target="mysql", message=f"close error: {e}")
        finally:
            engine = None
            AsyncSessionLocal = None
    # engine is None → nothing to close, stay silent.

```

```

=====
FILE: utils/paths.py
=====
# neurocad/utils/paths.py

"""Package paths and user working directory helpers."""

import json
import shutil
from pathlib import Path
from importlib.resources import files

# =====
# PACKAGE PATHS
# =====

def package_dir() -> Path:
    """Package root (in site-packages)."""
    return Path(str(files("neurocad")))

def package_core_dir() -> Path:
    """Package core dir (neurocad/core/)."""
    return package_dir() / "core"

def package_engine_dir() -> Path:
    """Package engine dir (neurocad/core/engine/)."""
    return package_core_dir() / "engine"

```

```

def package_alembic_dir() -> Path:
    """Package alembic dir (neurocad/alembic/)."""
    return package_dir() / "alembic"

def package_alembic_ini() -> Path:
    """Package alembic.ini path."""
    return package_dir() / "alembic.ini"

def package_static_dir() -> Path:
    """Package static dir (neurocad/static/)."""
    return package_dir() / "static"

def package_demo_dir() -> Path:
    """
    Package demo dir (neurocad/base/demo/).

    Injected into the wheel from `base/demo/` in the repo via
    pyproject.toml → [tool.hatch.build.targets.wheel.force-include]:
        "base/demo" = "neurocad/base/demo"
    """
    return package_dir() / "base" / "demo"

# =====
# USER PATHS (relative to cwd)
# =====

def user_base_dir() -> Path:
    """User base dir (relative to cwd) – where neurocad.db lives."""
    from ..config import settings
    return Path(settings.BASE_PATH)

def user_alembic_dir() -> Path:
    """User migrations directory (relative to cwd)."""
    return Path("mig")

def user_alembic_versions_dir() -> Path:
    """User migrations versions directory."""
    return user_alembic_dir() / "versions"

def user_core_dir() -> Path:
    """User overrides directory (project1/core/)."""
    return Path("core")

def user_engine_dir() -> Path:
    """User engine overrides dir (project1/core/engine/)."""
    return user_core_dir() / "engine"

```

```

def user_static_dir() -> Path:
    """User static dir (project1/static/). Built by sync_static."""
    return Path("static")

def user_demo_dir() -> Path:
    """
    User demo dir – `base/demo/` inside cwd, next to neurocad.db.

    The manifest.json inside this folder carries the `applied` flag:
    - applied: false → demo has never been imported into this project;
    - applied: true  → demo has been imported once; do not re-import.

    The user controls demo data by this folder:
    - keep it          → demo stays applied (applied: true);
    - delete the DB    → clean DB, demo will NOT be re-imported
                        (applied is still true);
    - delete base/demo/ → demo will be re-copied from the package
                        and re-imported on the next run.
    """
    return user_base_dir() / "demo"

# =====
# ENSURE WORKDIRS
# =====

def ensure_workdirs():
    """
    Create working directories and files in cwd if missing.

    Created:
    - base/, log/, media/ – working dirs
    - static/ – synced static (nginx root)
    - app/ – with copy of mod/* (if empty)
    - core/engine/ – user overrides dir
    - alembic/versions/ – user migrations
    - base/demo/ – copy of package demo, unless applied:true or DB exists
    - main.py – entry point (if missing)
    - .env – settings (if missing, copied from package .env.example)
    - .gitignore – project gitignore (created or synced)

    Demo copy rule (see `_copy_demo_if_needed`):
    Copy package demo (neurocad/base/demo) to cwd/base/demo only when
    BOTH:
    - the DB does not exist yet (this is the first run), and
    - cwd/base/demo does not exist yet, and
    - cwd/base/demo/manifest.json is missing or has applied:false.

    Any of those being false → no copy.
    """
    from ..config import settings

# ===== 1. Working dirs =====

```

```

for p in [
    settings.BASE_PATH,
    settings.LOG_PATH,
    settings.MEDIA_PATH,
    Path("app"),
    user_engine_dir(),
    user_static_dir(),
]:
    Path(p).mkdir(parents=True, exist_ok=True)

# ===== 2. app/ - if empty, copy mod/* =====
app_dir = Path("app")
if not any(app_dir.iterdir()):
    _copy_builtin_app(app_dir)

# ===== 3. alembic/versions/ - user migrations =====
versions_dir = user_alembic_versions_dir()
versions_dir.mkdir(parents=True, exist_ok=True)

# ===== 4. base/demo/ - copy from package if not applied yet =====
_copy_demo_if_needed()

# ===== 5. main.py - if missing =====
main_py = Path("main.py")
if not main_py.exists():
    main_py.write_text(
        "from neurocad import NeuroCad\n"
        "\n"
        "app = NeuroCad()\n",
        encoding="utf-8",
    )

# ===== 6. .env - if missing =====
# Template lives in the package as neurocad/.env.example
# (injected via pyproject.toml → force-include).
# Single source of truth: edit the template, not this file.
env = Path(".env")
if not env.exists():
    try:
        env_example = package_dir() / ".env.example"
        if env_example.is_file():
            shutil.copy(env_example, env)
            print(f"[neurocad] .env created from template: {env}")
        else:
            print("[neurocad] .env.example not found in package - .env not created")
    except Exception as e:
        print(f"[neurocad] failed to create .env: {e}")

# ===== 7. .gitignore - create or sync =====
ensure_gitignore()

```

```

# =====
# DEMO COPY
# =====

```

```

def _read_applied(demo_dir: Path) -> bool:
    """
    Read the `applied` flag from <demo_dir>/manifest.json.

    Returns False if the file is missing, unreadable, or has no
    `applied` field. That means "treat as not yet applied" – safe
    default: we would rather re-import than silently skip demo on
    the first run.
    """
    manifest = demo_dir / "manifest.json"
    if not manifest.is_file():
        return False
    try:
        data = json.loads(manifest.read_text(encoding="utf-8"))
    except Exception:
        return False
    return bool(data.get("applied", False))

def _copy_demo_if_needed():
    """
    Copy `base/demo/` from the package into cwd/base/demo/ – but only
    when demo has never been applied.

    Skip the copy if ANY of these is true:
    - the DB exists (not the first run);
    - cwd/base/demo exists AND its manifest has applied:true
      (demo already imported once; respect the user's state);
    - the package has no demo.

    Otherwise – copy.

    Notes:
    - If cwd/base/demo exists with applied:false (user copied an
      unapplied demo folder by hand, or the very first copy) – we
      still don't touch it: the folder is already there, the
      importer will read it and set applied:true.
    - If cwd/base/demo exists with applied:true – never copy.
    - Deleting cwd/base/demo is the user's signal "I want demo
      again": the next run will copy it from the package and
      re-import (as long as the DB does not exist or the applied
      flag was reset by the deletion).
    """
    src = package_demo_dir()
    dst = user_demo_dir()
    db_file = user_base_dir() / "neurocad.db"

    if not src.is_dir():
        # This build ships without demo data – nothing to copy.
        return

    if dst.exists():
        # Folder already there. If applied:true – definitely skip.
        # If applied:false – also skip: the importer will pick it up.
        return

```

```

if db_file.exists():
    # Not the first run: the DB exists, so we are past the
    # "first copy" moment. Respect the current state: base/demo
    # is gone → the user deliberately removed it.
    return

try:
    dst.parent.mkdir(parents=True, exist_ok=True)
    shutil.copytree(src, dst)
    print(f"[neurocad] demo copied to: {dst}")
except Exception as e:
    print(f"[neurocad] failed to copy demo: {e}")

# =====
# .gitignore
# =====

_GITIGNORE_BEGIN = "# === NEUROCAD BEGIN ==="
_GITIGNORE_END = "# === NEUROCAD END ==="

def ensure_gitignore():
    """
    Create or sync the project .gitignore.

    Rules:
    1. .gitignore doesn't exist          → copy package version (full).
    2. Exists, identical to package       → skip.
    3. Exists, has no NEUROCAD markers    → full overwrite (package version).
    4. Exists, both have markers, differs → replace NEUROCAD block only.
    5. Exists, both have markers, same    → skip.

    User additions AFTER the NEUROCAD END marker are preserved
    once the file has markers.

    Source: neurocad/.gitignore (inside the installed package).
    Destination: ../.gitignore (cwd).
    """
    dst = Path(".gitignore")
    src = package_dir() / ".gitignore"

    if not src.exists():
        print("[neurocad] .gitignore template not found in package")
        return

    src_content = src.read_text(encoding="utf-8")

    # ===== Case 1: no .gitignore – copy =====
    if not dst.exists():
        shutil.copy(src, dst)
        print(f"[neurocad] .gitignore created from template: {dst}")
        return

    dst_content = dst.read_text(encoding="utf-8")

```

```

# ===== Case 2: identical – skip =====
if dst_content.strip() == src_content.strip():
    print("[neurocad] .gitignore already up to date – skipping")
    return

src_has_markers = _GITIGNORE_BEGIN in src_content and _GITIGNORE_END in src_content
dst_has_markers = _GITIGNORE_BEGIN in dst_content and _GITIGNORE_END in dst_content

# ===== Case 3: dst has no markers – full overwrite =====
if not dst_has_markers:
    shutil.copy(src, dst)
    print(f"[neurocad] .gitignore overwritten (no markers found): {dst}")
    return

# ===== Case 4: both have markers – replace block only =====
if src_has_markers and dst_has_markers:
    src_block = _extract_gitignore_block(src_content)
    dst_block = _extract_gitignore_block(dst_content)

    if src_block == dst_block:
        print("[neurocad] .gitignore NEUROCAD block up to date – skipping")
        return

    new_content = dst_content.replace(dst_block, src_block)
    dst.write_text(new_content, encoding="utf-8")
    print(f"[neurocad] .gitignore NEUROCAD block updated: {dst}")
    return

# Fallback: src has no markers (некорректный пакет)
print("[neurocad] .gitignore template has no markers – skipping")

def _extract_gitignore_block(content: str) -> str:
    """Extract the NEUROCAD block (BEGIN..END inclusive) from .gitignore content."""
    i = content.index(_GITIGNORE_BEGIN)
    j = content.index(_GITIGNORE_END) + len(_GITIGNORE_END)
    return content[i:j]

# =====
# BUILTIN APP COPY
# =====

def _copy_builtin_app(app_dir: Path):
    """
    Copy builtin mod/* into app/ (each module as a subfolder).

    Skips:
    - __init__.py
    - __pycache__
    - existing destinations
    """
    src = package_engine_dir() / "mod"
    if not src.exists() or not src.is_dir():
        return

```

```

for item in src.iterdir():
    # Skip dunder/service entries
    if item.name.startswith("_"):
        continue

    dst = app_dir / item.name
    if dst.exists():
        continue

    if item.is_dir():
        shutil.copytree(item, dst)
    else:
        shutil.copy(item, dst)

=====
FILE: utils/routes.py
=====
# neurocad/utils/routes.py

from typing import Optional

from fastapi import FastAPI, Request
from fastapi.responses import RedirectResponse, Response
from sqlalchemy import select

from neurocad.core.route import router as core_router
from neurocad.core.engine.lib.pages.public.route import router as pages_public_router

# Internal routes – called by Caddy, not by end users.
# Lives at /internal/* – outside the /core/engine/... tree
# on purpose: Caddy's On-Demand TLS `ask` URL must be short
# and stable, not coupled to the profile module's prefix.
from neurocad.core.engine.lib.base.profile.domain.internal_route import (
    router as internal_tls_router,
)

from neurocad.config import settings

# =====
# ROBOTS.TXT DEFAULTS
# =====

# Must match ROBOTS_OPEN / ROBOTS_CLOSED in
# neurocad/core/engine/lib/base/profile/domain/service.py and
# DEFAULT_ROBOTS_CLOSED in the domain schema. If you change one,
# change all three – they are checked by the test suite.
ROBOTS_OPEN = "User-agent: *\nDisallow:\n"
ROBOTS_CLOSED = "User-agent: *\nDisallow: /\n"

# =====
# SUBDOMAIN HELPERS
# =====

def _slug_from_host(host: Optional[str]) -> Optional[str]:

```



```
"""
```

```
Extract the user slug from a Host header.
```

```
Returns the slug ("testuser3") when the host is a subdomain of
settings.APP_DOMAIN ("testuser3.neurocad-dev.ru" with
APP_DOMAIN=neurocad-dev.ru). Returns None otherwise.
```

```
Guards:
```

- host is empty / missing → None
- port stripped, lowercased, trailing dot gone → canonical form
- host == APP_DOMAIN (no subdomain) → None
- host is "www.<APP_DOMAIN>" → None
("www" is a web convention, not a user slug)
- slug contains a dot ("a.b.<APP_DOMAIN>") → None
(only one level of subdomain is a user)
- host does not end with "<APP_DOMAIN>" → None

```
"""
```

```
if not host:
```

```
    return None
```

```
host = host.strip().lower()
```

```
if ":" in host:
```

```
    host = host.split(":", 1)[0]
```

```
if host.endswith("."):

```

```
    host = host[:-1]
```

```
if not host:
```

```
    return None
```

```
root = (settings.APP_DOMAIN or "").strip().lower()
```

```
if not root:
```

```
    return None
```

```
# Exact root domain – not a subdomain.
```

```
if host == root:
```

```
    return None
```

```
suffix = "." + root
```

```
if not host.endswith(suffix):
```

```
    return None
```

```
slug = host[: -len(suffix)]
```

```
if not slug:
```

```
    return None
```

```
# "www" is a web convention, not a user slug.
```

```
if slug == "www":
```

```
    return None
```

```
# Only one level of subdomain. "a.b.example.com" is not a user.
```

```
if "." in slug:
```

```
    return None
```

```
return slug
```

```
def _normalize_host(host: Optional[str]) -> Optional[str]:
```

```

"""
Shared host normalization for the custom-domain lookup:
strip port, lowercase, drop the trailing dot.

Returns None if the host is empty after normalization, or if it
equals APP_DOMAIN / starts with "www." (those never belong to
a custom-domain user – they are platform-hosted).
"""

if not host:
    return None

host = host.strip().lower()
if ":" in host:
    host = host.split(":", 1)[0]
if host.endswith("."):
    host = host[:-1]
if not host:
    return None

root = (settings.APP_DOMAIN or "").strip().lower()
if root:
    if host == root:
        return None
    if host.endswith("." + root):
        # A subdomain of APP_DOMAIN – handled by _slug_from_host,
        # not by the custom-domain lookup.
        return None

return host


async def _login_from_custom_domain(host: Optional[str]) -> Optional[str]:
    """
    If `host` is a custom domain registered by some user
    (users.domain == host), return that user's login. Otherwise None.

    This is the custom-domain counterpart of `_slug_from_host`:
    the subdomain lookup reads the login from the host
    ("testuser3.neurocad-dev.ru" → "testuser3"); the custom-domain
    lookup reads it from the database ("atou.ru" → "admin").

    Guards (all applied via _normalize_host):
    - empty / missing host → None
    - host is APP_DOMAIN itself → None
    - host is a subdomain of APP_DOMAIN → None (handled earlier)

    Never raises on DB errors – treated as "not a custom domain".
    """
    normalized = _normalize_host(host)
    if not normalized:
        return None

# Late imports – avoid pulling SQLAlchemy models at module import
# time (this file is loaded very early by main.py).
from neurocad.core.models.user import User
from neurocad.utils.sqlite import get_db_sqlite

```

```

try:
    async for session in get_db_sqlite():
        stmt = select(User).where(
            User.domain == normalized,
            User.is_delete.is_(False),
        )
        user = (await session.execute(stmt)).scalar_one_or_none()
        if user is None:
            return None
        return user.login
except Exception as e:
    print(f"[Engine] custom-domain lookup failed for {normalized!r}: {e}")
    return None

return None

```

```

# =====
# ROBOTS.TXT HELPERS
# =====

```

```

def _robots_response(text: str) -> Response:
    """
    Build the /robots.txt response.

    Always text/plain; charset=utf-8. Never cached – robots.txt is
    edited by the user in the modal and cached responses would make
    the change invisible to crawlers (and to the user debugging).
    """
    return Response(
        content=text or ROBOTS_CLOSED,
        media_type="text/plain; charset=utf-8",
        headers={"Cache-Control": "no-cache, no-store, must-revalidate"},
    )

```

```

async def _robots_for_slug(slug: str) -> str:
    """
    robots.txt body for a free third-level subdomain
    (<slug>.<APP_DOMAIN>).

    Source: users.robots_3. If the user does not exist or the field
    is NULL, ROBOTS_CLOSED is returned – the subdomain is closed to
    crawlers by default, and add_custom / remove_custom toggle this
    field automatically (see service.py).

    Never raises on DB errors – treated as "closed".
    """
    from neurocad.core.models.user import User
    from neurocad.utils.sqlite import get_db_sqlite

```

```

try:
    async for session in get_db_sqlite():
        stmt = select(User).where(
            User.login == slug,

```

```

        User.is_delete.is_(False),
    )
    user = (await session.execute(stmt)).scalar_one_or_none()
    if user is None:
        return ROBOTS_CLOSED
    return user.robots_3 or ROBOTS_CLOSED
except Exception as e:
    print(f"[Engine] robots_3 lookup failed for {slug!r}: {e}")
    return ROBOTS_CLOSED

return ROBOTS_CLOSED

async def _robots_for_custom_domain(host: str) -> str:
    """
    robots.txt body for a custom second-level domain.

    Source: users.robots_2, looked up by users.domain == host. If the
    user does not exist or the field is NULL, ROBOTS_CLOSED is
    returned – safer default for a domain that is registered but
    whose robots.txt has never been saved.

    Never raises on DB errors – treated as "closed".
    """
    from neurocad.core.models.user import User
    from neurocad.utils.sqlite import get_db_sqlite

    try:
        async for session in get_db_sqlite():
            stmt = select(User).where(
                User.domain == host,
                User.is_delete.is_(False),
            )
            user = (await session.execute(stmt)).scalar_one_or_none()
            if user is None:
                return ROBOTS_CLOSED
            return user.robots_2 or ROBOTS_CLOSED
    except Exception as e:
        print(f"[Engine] robots_2 lookup failed for {host!r}: {e}")
        return ROBOTS_CLOSED

    return ROBOTS_CLOSED

async def _resolve_home_url_for_slug(slug: str) -> Optional[str]:
    """
    Given a user slug (login), return the URL of their home page –
    or a fallback URL if the user exists but has no pages.

    Resolution order:

    1. users.home_page_id – explicit choice made by the user on
       the "Custom domains" page.

    2. First page of the user's first nav (ORDER BY datetime ASC,
       is_delete = 0). This makes "/" always show something

```

meaningful for a user who has at least one page.

3. Fallback: /core/engine/pages – the article catalog, so the user can create their first page.

Returns None if the user does not exist at all – the caller falls back to the Host → module resolution.

The final URL uses the public "pages" route:

/page/<nav_id>/<YYYYMMDD>/<HHMMSS>

which is what the existing public router understands.

"""

Late imports – avoid pulling SQLAlchemy models at module import

time (this file is loaded very early by main.py).

from neurocad.core.models.user import User

from neurocad.core.models.nav import Nav

from neurocad.core.models.page import Page

from neurocad.utils.sqlite import get_db_sqlite

async for session in get_db_sqlite():

---- 1. Find the user -----

user_stmt = select(User).where(

 User.login == slug,

 User.is_delete.is_(False),

)

user = (await session.execute(user_stmt)).scalar_one_or_none()

if user is None:

 return None

---- 2. Find the user's first nav -----

nav_stmt = (

 select(Nav)

 .where(

 Nav.user_id == user.id,

 Nav.is_delete.is_(False),

)

 .order_by(Nav.id.asc())

 .limit(1)

)

nav = (await session.execute(nav_stmt)).scalar_one_or_none()

if nav is None:

 # User exists but has no nav – send them to the catalog

 # so they can create one (or see an empty state).

 return "/core/engine/pages"

---- 3a. Explicit home page -----

page: Optional[Page] = None

if user.home_page_id:

 page_stmt = select(Page).where(

 Page.id == user.home_page_id,

 Page.nav_id == nav.id,

 Page.is_delete == 0,

)

 page = (await session.execute(page_stmt)).scalar_one_or_none()

 # If the explicit home page was deleted or moved to

 # another nav, fall through to the "first page" branch.

```

# ---- 3b. Fallback – first page by datetime -----
if page is None:
    first_stmt = (
        select(Page)
        .where(
            Page.nav_id == nav.id,
            Page.is_delete == 0,
        )
        .order_by(Page.datetime.asc(), Page.id.asc())
        .limit(1)
    )
    page = (await session.execute(first_stmt)).scalar_one_or_none()

if page is None:
    # Nav exists but has no pages – the user should create one.
    return "/core/engine/pages"

# ---- 4. Build the public URL -----
return _page_public_url(nav.id, page.datetime)

return None

```

```

def _page_public_url(nav_id: int, page_dt) -> str:
    """

```

Build the public page URL:

```

    /page/<nav_id>/<YYYYMMDD>/<HHMMSS>

```

Matches what the public router expects and what the editor produces when it opens an article.

```

    """

```

```

    date = page_dt.strftime("%Y%m%d")

```

```

    time = page_dt.strftime("%H%M%S")

```

```

    return f"/page/{nav_id}/{date}/{time}"

```

```

# =====
# ROUTES
# =====

```

```

def setup_routes(app: FastAPI) -> None:

```

```

    """Register all application routes."""

```

```

    # Core routes – /core/*

```

```

    app.include_router(core_router)

```

```

    # Public pages – /pages/*

```

```

    app.include_router(pages_public_router)

```

```

    # Internal TLS verification – /internal/tls/verify

```

```

    # Called by Caddy before issuing an On-Demand TLS certificate.

```

```

    app.include_router(internal_tls_router)

```

```

    # /robots.txt – three-step resolution:

```

```

    #

```

```

# 1. Host is a user subdomain (<login>.<APP_DOMAIN>):
#     testuser3.neurocad-dev.ru → users.robots_3
#     Default when the field is NULL: ROBOTS_CLOSED.
#
# 2. Host is a user's custom domain (users.domain):
#     atou.ru → users.robots_2
#     Default when the field is NULL: ROBOTS_CLOSED.
#
# 3. Anything else (apex APP_DOMAIN, IP, unknown host):
#     ROBOTS_CLOSED. Never redirects, never serves HTML –
#     crawlers must see a 200 text/plain response.
@app.get("/robots.txt")
async def robots_txt(request: Request):
    host = request.headers.get("host")

    # ---- Step 1: user subdomain -----
    slug = _slug_from_host(host)
    if slug is not None:
        text = await _robots_for_slug(slug)
        return _robots_response(text)

    # ---- Step 2: user custom domain -----
    normalized = _normalize_host(host)
    if normalized is not None:
        custom_login = await _login_from_custom_domain(normalized)
        if custom_login is not None:
            text = await _robots_for_custom_domain(normalized)
            return _robots_response(text)

    # ---- Step 3: fallback -----
    return _robots_response(ROBOTS_CLOSED)

# Root – three-step resolution:
#
# 1. If Host is a user subdomain (<login>.<APP_DOMAIN>), redirect
#    to that user's home page. The slug is read from the host:
#    testuser3.neurocad-dev.ru → "testuser3"
#
# 2. If Host is a user's custom domain (users.domain), redirect
#    to that user's home page. The login is read from the DB:
#    atou.ru → owner's login → home page
#
# 3. Otherwise, redirect to the module resolved by Host via
#    `domain` in the module's JSON. Fallback: APP_MAIN_PAGE.
#
# Set APP_MAIN_PAGE in .env to override the final fallback.
@app.get("/")
async def root(request: Request):
    host = request.headers.get("host")

    # ---- Step 1: user subdomain -----
    # e.g. testuser3.neurocad-dev.ru → "testuser3"
    slug = _slug_from_host(host)
    if slug is not None:
        target = await _resolve_home_url_for_slug(slug)
        if target is not None:

```

```

        return RedirectResponse(url=target, status_code=302)
    # slug was valid but user not found – fall through to
    # the module resolution below (e.g. a subdomain that
    # happens to be declared as a module domain).

# ---- Step 2: user custom domain -----
# e.g. atou.ru → owner's login → home page
custom_login = await _login_from_custom_domain(host)
if custom_login is not None:
    target = await _resolve_home_url_for_slug(custom_login)
    if target is not None:
        return RedirectResponse(url=target, status_code=302)
    # Custom domain is registered, but the owner has no
    # home page / no pages – fall through to module resolution.

# ---- Step 3: Host → module -----
from neurocad.core.engine.route import _resolve_module_by_host

host_module = _resolve_module_by_host(host)
if host_module:
    return RedirectResponse(url=f"/core/engine/{host_module}")

# ---- Step 4: fallback -----
main_url = settings.APP_MAIN_PAGE
if not main_url or main_url == "/":
    main_url = "/core/engine/admin"
return RedirectResponse(url=main_url)

```

```

=====
FILE: utils/sqlite.py
=====

```

```

# neurocad/utils/sqlite.py

```

```

from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine, async_sessionmaker
from sqlalchemy.pool import AsyncAdaptedQueuePool
from sqlalchemy import event, text, select
from ..config import settings

```

```

# SQLite URL
SQLITE_URL = settings.SQLITE_URL

```

```

# Engine with async pool

```

```

#

```

```

# Notes on the connect_args:

```

```

#   check_same_thread=False – required for SQLAlchemy's async adapter.

```

```

#   timeout=30 – seconds to wait on a locked DB (kernel level).

```

```

#

```

```

# PRAGMA tuning is done in _set_sqlite_pragma below, once per new

```

```

# connection. See the docstring there for the reasoning.

```

```

engine = create_async_engine(
    SQLITE_URL,
    echo=True,
    poolclass=AsyncAdaptedQueuePool,
    connect_args={
        "check_same_thread": False,
        "timeout": 30,
    }
)

```



```

    }
)

# =====
# SQLITE PRAGMA – once per new connection
# =====
#
# SQLite's default journal_mode is "delete": while one connection is
# writing (INSERT/UPDATE/DELETE), every other connection – including
# readers – blocks. With a FastAPI app that mixes HTTP saves,
# auto-saves, and WebSocket-driven LLM runs, that means a long write
# (e.g. saving a page) blocks the WS handler for the whole duration.
# In practice this looks like the WS "hanging" for tens of seconds.
#
# WAL (Write-Ahead Logging) fixes this: writers go to a separate
# -wal file, readers see a consistent snapshot. Writers and readers
# no longer block each other.
#
# The other PRAGMAs:
#   synchronous=NORMAL – safe default under WAL; faster than FULL.
#   busy_timeout=30000 – if a writer must wait, wait up to 30 s
#                       instead of failing immediately.
#   foreign_keys=ON    – SQLite does not enforce FKs by default.
#
# All four run on every new DB connection. journal_mode is persistent
# (stored in the DB file), the rest are per-connection.

@event.listens_for(engine.sync_engine, "connect")
def _set_sqlite_pragma(dbapi_conn, connection_record):
    cursor = dbapi_conn.cursor()
    try:
        cursor.execute("PRAGMA journal_mode=WAL")
        cursor.execute("PRAGMA synchronous=NORMAL")
        cursor.execute("PRAGMA busy_timeout=30000")
        cursor.execute("PRAGMA foreign_keys=ON")
    finally:
        cursor.close()

# Session factory
AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False
)

async def get_db_sqlite():
    """Session generator for Dependency Injection (SQLite)."""
    async with AsyncSessionLocal() as session:
        yield session

def _log_sync(log, level: str, message: str) -> None:
    """Write through app.state.log if available, else silently."""

```

```

    if log is None:
        return
    fn = getattr(log, f"log_{level}_sync", None)
    if fn is None:
        return
    try:
        fn(target="sqlite", message=message)
    except Exception:
        pass

# =====
# FRESH-DB CHECK
# =====

async def _is_db_fresh() -> bool:
    """
    Return True if the DB has no users yet.

    "No users" is our marker for "clean DB": a freshly created DB
    has an empty `users` table, and demo data (if any) can be
    imported into it.

    If the table does not exist yet (very first run, migrations
    not applied), we treat this as "not fresh" – the importer
    must not run before the schema is in place anyway.

    Any user – superadmin, regular, whatever – is enough to
    consider the DB "in use". Demo data must never be mixed
    with real data.
    """
    try:
        from ..core.models.base import User

        async with AsyncSessionLocal() as session:
            stmt = select(User.id).limit(1)
            result = await session.execute(stmt)
            return result.scalar_one_or_none() is None
    except Exception:
        # Table `users` not there yet, DB unreachable, etc.
        # Conservative default: not fresh → skip demo import.
        return False

# =====
# INIT
# =====

async def init_sqlite(log=None):
    """
    Initialize SQLite:
    1. create working directories,
    2. check connection,
    3. apply migrations,
    4. import demo data – only if the DB is fresh (no users yet)
       AND `base/demo/` exists in the working directory,

```

5. create superadmin,
6. register modules from mod/,
7. ensure the admin has a default Nav row (owns demo pages),
8. ensure the admin has a Balance row.

log – app.state.log from lifespan. If None – silent.

"""

try:

```
# 1. Working directories (base/, log/, media/, app/, alembic/versions/)
from .paths import ensure_workdirs
ensure_workdirs()
```

```
# 2. Check connection
async with engine.begin() as conn:
    await conn.execute(text("SELECT 1"))
```

```
if log is not None:
    await log.log_info(target="sqlite", message="SQLite connected")
```

```
# 3. Alembic migrations
from .migrations import apply_migrations
apply_migrations()
```

```
# 4. Demo data – only if the DB is fresh.
#
# "Fresh" means: no users yet. If the DB already has
# any user (superadmin or regular), it is considered
# "in use" and demo data must NOT be mixed into it.
#
# The importer itself also checks that `base/demo/`
# exists – so a user who deleted that folder gets a
# clean DB without demo content.
#
# Non-fatal: import errors do not block startup.
```

try:

```
if await _is_db_fresh():
    from .dbsqlite.demo_import import import_demo
    await import_demo(log=log)
else:
    _log_sync(
        log,
        "info",
        "SQLite: DB is not fresh (users exist) – "
        "demo import skipped",
    )
```

except Exception as e:

```
_log_sync(log, "warning", f"demo import failed: {e}")
```

```
# 5. Superadmin (auth bootstrap)
await ensure_superuser(log=log)
```

```
# 6. Modules (engine bootstrap).
# NOTE: temporary for 0.1.x – will be replaced by web UI in 0.2.x.
# Local import: utils/ must not depend on core/engine at module level.
from ..core.engine.modules import ensure_modules
from ..core.engine.route import MOD_ROOT
```

```

    await ensure_modules(MOD_ROOT, log)

    # 7. Default nav for admin (the object that owns demo pages).
    # Idempotent: does nothing if the row already exists.
    await ensure_default_nav(log=log)

    # 8. Balance row for admin (user_id = 1, created by step 5).
    # Idempotent: does nothing if the row already exists.
    await ensure_user_balance(user_id=1, log=log)

except Exception as e:
    error_msg = f"SQLite connection error: {e}"
    if log is not None:
        await log.log_error(target="sqlite", message=error_msg)
    raise

async def close_sqlite(log=None):
    """Close SQLite connection."""
    try:
        await engine.dispose()
        if log is not None:
            await log.log_info(target="sqlite", message="SQLite disconnected")
    except Exception as e:
        if log is not None:
            await log.log_error(target="sqlite", message=f"close error: {e}")

# =====
# SUPERADMIN
# =====

async def ensure_superuser(log=None):
    """
    Check for a specific superadmin (by login from .env).
    If missing – create from SUPERADMIN_LOGIN and SUPERADMIN_PASSWORD.

    Does not fail if there are multiple superadmins – checks only the specific one.

    log – app.state.log from init_sqlite. If None – silent.
    """
    from ..config import settings
    from ..core.models.base import User
    from ..utils.hash import get_hash_string

    login = getattr(settings, "SUPERADMIN_LOGIN", "admin")
    password = getattr(settings, "SUPERADMIN_PASSWORD", "admin")

    async with AsyncSessionLocal() as session:
        # Check the specific user by login (not "any superadmin")
        stmt = select(User).where(User.login == login)
        result = await session.execute(stmt)
        existing_admin = result.scalar_one_or_none()

        if existing_admin:
            # If user exists but is not superadmin – promote

```

```

        if not existing_admin.is_superuser:
            existing_admin.is_superuser = True
            await session.commit()
            _log_sync(log, "info", f"User '{login}' promoted to superadmin")
        else:
            _log_sync(log, "info", f"Superadmin already exists: {existing_admin.login}")
        return

# If user does not exist – create
hashed_password = get_hash_string(password)

new_admin = User(
    login=login,
    password=hashed_password,
    name="Админ",
    is_superuser=True,
    is_active=True,
    is_delete=False,
)
session.add(new_admin)
await session.commit()

_log_sync(log, "info", f"Superadmin created: {login}")

# =====
# DEFAULT NAV
# =====

async def ensure_default_nav(log=None) -> int:
    """
    Ensure the admin user (id=1) has at least one Nav row for the
    'default' module. Called on every startup – idempotent.

    This Nav row is the "object" that owns all the built-in demo
    pages, so URLs like /core/engine/pages/<nav_id>/<date>/<time>
    work for the default content.

    Returns the nav.id of the admin's default nav (found or created),
    or 0 if admin / module not found.
    """
    from ..core.models.base import Nav, Module, User

    async with AsyncSessionLocal() as session:
        # 1. Find admin.
        admin_stmt = select(User).where(User.id == 1, User.is_delete == False)
        admin = (await session.execute(admin_stmt)).scalar_one_or_none()

        if not admin:
            _log_sync(log, "info", "admin (id=1) not found – skip ensure_default_nav")
            return 0

        # 2. Find the 'default' module by name.
        mod_stmt = select(Module).where(
            Module.name == "default",
            Module.is_delete == False,

```

```

)
module = (await session.execute(mod_stmt)).scalar_one_or_none()

if not module:
    _log_sync(log, "info", "module 'default' not found – skip ensure_default_nav")
    return 0

# 3. Does admin already have a nav for this module?
nav_stmt = select(Nav).where(
    Nav.user_id == admin.id,
    Nav.module_id == module.id,
    Nav.is_delete == False,
)
existing = (await session.execute(nav_stmt)).scalar_one_or_none()

if existing:
    _log_sync(
        log, "info",
        f"admin nav for 'default' already exists: id={existing.id}",
    )
    return existing.id

# 4. Create it.
nav = Nav(
    user_id=admin.id,
    parent_id=None,
    card_type="link",
    sort_order=1,
    name="Каталог статей",
    description=None,
    icon=None,
    module_id=module.id,
    is_delete=False,
)
session.add(nav)
await session.commit()
await session.refresh(nav)

_log_sync(
    log, "info",
    f"created admin nav for 'default': id={nav.id}",
)
return nav.id

```

```

# =====
# USER BALANCE
# =====

```

```

async def ensure_user_balance(user_id: int, log=None) -> int:
    """

```

Ensure a balance row exists for the given user.

Idempotent: if the row already exists, returns its id without creating a new one.

New users start on the TRIAL tariff (code 3, 1 day, same limits as llm). The lazy reset in `BalanceChecked.reset_if_needed()` will switch them to free after 1 day.

Trial is granted WITH tokens already topped up to the trial cap – otherwise `llm_allowed()` sees `tokens=0` against a non-zero `limit_tokens` and reports 'tokens_exhausted' on the very first LLM request. `reset_if_needed()` does NOT top up trial tokens (trial has no daily/monthly accrual), so it must be done here.

Default values for a new user:

```

tarif          = 3          (trial, 1 day)
tokens         = 2_000_000 (same as limit_tokens – trial is
                           "unlimited" for its 1-day window)

gen            = 0
sum            = 0
mb             = 0
pages          = 0
price          = 0
refer_id       = None
day            = today.day (so billing/expiry math has a value)
acc_at         = today      (trial start date – the 24h clock)

limit_genday   = 0
limit_genmon   = 0
limit_mb       = 1024       (1 GB, same as llm)
limit_pages    = 0          (unlimited, same as llm)
limit_tokens   = 2_000_000 (2M, same as llm)

```

Returns the balance row id, or 0 on error / user not found.

```

"""
from datetime import date
from ..core.models.base import Balance, User

# Must match limit_tokens below. Kept local on purpose: utils/
# must not import from core/engine/lib at module level (see the
# note in init_sqlite about the utils/ → core/engine dependency).
TRIAL_TOKENS = 2_000_000

async with AsyncSessionLocal() as session:
    # 1. Verify the user exists (defensive – avoids FK errors).
    user_stmt = select(User).where(
        User.id == user_id,
        User.is_delete == False,
    )
    user = (await session.execute(user_stmt)).scalar_one_or_none()
    if not user:
        _log_sync(
            log, "info",
            f"ensure_user_balance: user {user_id} not found – skip",
        )
        return 0

    # 2. Does a balance row already exist?
    bal_stmt = select(Balance).where(

```

```

        Balance.user_id == user_id,
        Balance.is_delete == False,
    )
    existing = (await session.execute(bal_stmt)).scalar_one_or_none()
    if existing:
        _log_sync(
            log, "info",
            f"ensure_user_balance: balance for user {user_id} "
            f"already exists (id={existing.id})",
        )
        return existing.id

# 3. Create it with TRIAL defaults (code 3, same limits as llm),
#   tokens topped up so the first LLM call passes llm_allowed().
today = date.today()

bal = Balance(
    user_id=user_id,
    tarif=3,                      # trial
    day=today.day,                # needed for billing-day math
    gen=0,
    tokens=TRIAL_TOKENS,         # trial is "unlimited" for 1 day
    sum=0,
    mb=0,
    pages=0,
    price=0,
    refer_id=None,
    limit_genday=0,
    limit_genmon=0,
    limit_mb=1024,
    limit_pages=0,
    limit_tokens=TRIAL_TOKENS,
    acc_at=today,                # trial clock starts now
    is_delete=False,
)
session.add(bal)
await session.commit()
await session.refresh(bal)

_log_sync(
    log, "info",
    f"ensure_user_balance: created balance id={bal.id} "
    f"for user {user_id} (trial, tokens={TRIAL_TOKENS})",
)
return bal.id

```

```
=====
FILE: utils/static.py
=====
```

```
# neurocad/utils/static.py
```

```

import mimetypes
import shutil
import time
from pathlib import Path
from fastapi import FastAPI

```



```

from fastapi.staticfiles import StaticFiles
from fastapi.responses import RedirectResponse

from .paths import (
    package_engine_dir,
    package_static_dir,
    user_engine_dir,
    user_static_dir,
)

# =====
# MIME TYPES (Windows fix)
# =====
# Windows: mimetypes.guess_type(".js") returns ("text/plain", None),
# so browsers refuse to execute ES modules
# (strict MIME type checking for <script type="module">).
# Register explicitly BEFORE creating StaticFiles, because Starlette
# caches MIME types at init time.

mimetypes.init()

mimetypes.add_type("application/javascript", ".js")
mimetypes.add_type("application/javascript", ".mjs")
mimetypes.add_type("text/css", ".css")
mimetypes.add_type("image/svg+xml", ".svg")
mimetypes.add_type("font/woff", ".woff")
mimetypes.add_type("font/woff2", ".woff2")
mimetypes.add_type("font/ttf", ".ttf")
mimetypes.add_type("image/x-icon", ".ico")
mimetypes.add_type("image/vnd.microsoft.icon", ".ico")

STATIC_VERSION = str(int(time.time()))

# Static extensions to sync.
STATIC_EXTENSIONS = {
    '.css', '.js', '.mjs',
    '.woff2', '.woff', '.ttf',
    '.svg', '.png', '.jpg', '.jpeg', '.gif', '.ico', '.map',
}

def get_static_version() -> str:
    return STATIC_VERSION

def _sync_dir(src_dir: Path, dst_root: Path, dst_subdir: Path) -> tuple[int, int]:
    """
    Copy static files from src_dir to dst_root / dst_subdir.

    Copies only changed files (by mtime).

    Args:
        src_dir:    source directory (e.g. neurocad/core/engine/)
        dst_root:   destination root (e.g. project1/static/)

```

dst_subdir: subpath inside dst_root (e.g. core/engine/)

Returns:

(copied, skipped)

"""

if not src_dir.exists():

return 0, 0

copied = 0

skipped = 0

for filepath in src_dir.rglob('*'):

if not filepath.is_file():

continue

if filepath.suffix not in STATIC_EXTENSIONS:

continue

rel_path = filepath.relative_to(src_dir)

dest_path = dst_root / dst_subdir / rel_path

if dest_path.exists() and filepath.stat().st_mtime <= dest_path.stat().st_mtime:

skipped += 1

continue

dest_path.parent.mkdir(parents=True, exist_ok=True)

shutil.copy2(filepath, dest_path)

copied += 1

return copied, skipped

def sync_static() -> tuple[int, int]:

"""

Sync static files to USER static dir (project1/static/).

Order (later overwrites earlier, by mtime):

1. Package libs/ + fonts/ -> project1/static/

2. Package engine -> project1/static/core/engine/

3. User engine overrides -> project1/static/core/engine/

Returns total (copied, skipped).

"""

static_root = user_static_dir()

engine_subdir = Path("core") / "engine"

copied_total = 0

skipped_total = 0

1. Package libs/ + fonts/ -> project1/static/

c, s = _sync_dir(package_static_dir(), static_root, Path("."))

copied_total += c

skipped_total += s

2. Package engine -> project1/static/core/engine/

c, s = _sync_dir(package_engine_dir(), static_root, engine_subdir)

```

copied_total += c
skipped_total += s

# 3. User engine -> project1/static/core/engine/ (overrides)
c, s = _sync_dir(user_engine_dir(), static_root, engine_subdir)
copied_total += c
skipped_total += s

return copied_total, skipped_total

def setup_static(app: FastAPI, log=None) -> None:
    """
    Mount /static and /media.

    log – optional logger with log_info_sync(target, message).
    If not provided – prints to stdout.
    """
    static_root = user_static_dir()
    static_root.mkdir(parents=True, exist_ok=True)

    # Media dir is relative to cwd: <project>/media/
    media_root = Path("media")
    media_root.mkdir(parents=True, exist_ok=True)

    copied, skipped = sync_static()

    if log:
        log.log_info_sync(
            target="static",
            message=f"Sync: copied={copied}, skipped={skipped}",
        )
    else:
        print(f"[Static] Sync: copied={copied}, skipped={skipped}")

    app.mount("/static", StaticFiles(directory=str(static_root)), name="static")
    app.mount("/media", StaticFiles(directory=str(media_root)), name="media")

    @app.get("/favicon.ico", include_in_schema=False)
    async def favicon_redirect():
        return RedirectResponse(url="/static/core/engine/lib/base/images/favicon.ico")

=====
FILE: utils/templates.py
=====
# neurocad/utils/templates.py

from fastapi import Request
from fastapi.templating import Jinja2Templates
from pathlib import Path
import time

from .paths import package_dir

# Static version (changes on server restart).
STATIC_VERSION = str(int(time.time()))

```

```

# Jinja2Templates search order:
# 1. User overrides (project1/)
# 2. Package (neurocad/)
templates = Jinja2Templates(directory=[
    str(Path(".")),          # project1/ (cwd)
    str(package_dir()),      # neurocad/
])

# =====
# STATIC FILTER
# =====

def static_url(path: str) -> str:
    """Return static file URL with version."""
    return f"/static/{path}?v={STATIC_VERSION}"

templates.env.filters['static'] = static_url
templates.env.globals['static_version'] = STATIC_VERSION

# =====
# GLOBAL TEMPLATE FUNCTIONS
# =====

def get_user(request: Request):
    """Get user from request.state."""
    return getattr(request.state, "user", None)

def is_authenticated(request: Request):
    """Check if user is authenticated."""
    return getattr(request.state, "is_authenticated", False)

templates.env.globals['get_user'] = get_user
templates.env.globals['is_authenticated'] = is_authenticated

```

```

=====
FILE: utils/__init__.py
=====

```

```

=====
FILE: utils/llm/base.py
=====

```

```

# neurocad/utils/llm/base.py

```

```

"""

```

```

Base class for LLM providers.

```

All providers implement the same async interface, so the editor does not care which backend is used. The factory returns an instance, and the caller only sees ``get_response`` / ``generate_completion``.

Token accounting

Every provider keeps a ``tokens_used`` counter. It is reset to 0 at the start of each ``get_response()`` call and incremented:

- for input messages (prompt tokens),
- for every chunk yielded (completion tokens).

Callers read ``provider.tokens_used`` AFTER the generator is fully consumed – that is the total cost of the call (prompt + completion).

``count_tokens(text)`` is the provider's own tokenizer. The base implementation is an approximation (`len * 0.6`) used by providers that have no exact tokenizer available. Subclasses override it.

"""

```
from abc import ABC, abstractmethod
from typing import AsyncGenerator
```

```
class LLMPProvider(ABC):
```

```
    """
```

```
    Common interface for all LLM providers.
```

```
    Subclasses must:
```

- set ``name`` (short identifier used in the factory and UI);
- implement ``is_configured`` (whether credentials are present);
- implement ``get_response`` (async generator of chunks).

```
    `generate_completion` is a convenience wrapper that collects all chunks into a single string – provided here, no need to override.
```

```
    Token accounting:
```

- ``self.tokens_used`` is reset at the start of ``get_response()``;
- subclasses should call ``self._add_tokens(text)`` for input messages and for every yielded chunk;
- after the generator is fully consumed, ``self.tokens_used`` holds the total (prompt + completion) for that call.

```
    """
```

```
    #: Human-readable name (for UI / logs)
```

```
    name: str = "base"
```

```
    def __init__(self):
```

```
        # Total tokens for the LAST get_response() call.
```

```
        # Reset at the start of each call.
```

```
        self.tokens_used: int = 0
```

```
    # =====
```

```
    # TOKEN COUNTING
```

```
    # =====
```

```
    def count_tokens(self, text: str) -> int:
```

```
        """
```

```
        Approximate token count for arbitrary text.
```

The base implementation is a rough heuristic (`len * 0.6`), used only as a fallback. Providers with a real tokenizer override this method.

Never raises.

"""

if not text:

 return 0

return int(len(text) * 0.6)

def _reset_tokens(self) -> None:

 """Reset the per-call token counter. Called at the top of each `get_response()` implementation."""

 self.tokens_used = 0

def _add_tokens(self, text: str) -> int:

 """Add the token count of ``text`` to the running total.

 Returns the number added."""

 n = self.count_tokens(text)

 self.tokens_used += n

 return n

=====

PROVIDER INTERFACE

=====

@property

@abstractmethod

def is_configured(self) -> bool:

 """

 Whether the provider has all the credentials it needs to work.

 Used by the factory to list providers available to the user, and by the UI to disable providers that are not set up.

 """

 ...

@abstractmethod

async def get_response(

 self,

 messages_list: list,

 temperature: float | None = None,

 max_tokens: int | None = None,

 stream: bool = False,

) -> AsyncGenerator[str, None]:

 """

 Async generator of response chunks.

 stream=False → one chunk with the whole response.

 stream=True → chunks as they arrive.

 On errors the generator yields a human-readable error string instead of raising – the caller decides what to do with it.

 Implementations MUST call ``self._reset_tokens()`` at the very start, then ``self._add_tokens(...)`` for every input message

```

and every yielded chunk. After the generator is fully
consumed, `self.tokens_used` holds the total cost.
"""
...

async def generate_completion(
    self,
    messages_list: list,
    temperature: float | None = None,
    max_tokens: int | None = None,
) -> str:
    """
    Non-streaming convenience wrapper: collect all chunks into one string.

    After this returns, `self.tokens_used` holds the total cost.
    """
    full = ""
    async for chunk in self.get_response(
        messages_list,
        temperature=temperature,
        max_tokens=max_tokens,
        stream=False,
    ):
        full += chunk
    return full

```

```

=====
FILE: utils/llm/deepseek.py
=====
# neurocad/utils/llm/deepseek.py

```

```

"""
DeepSeek provider (OpenAI-compatible API).

Supports reasoning models (V4.1-Flash, R1) – thinking mode is
controlled by the `thinking` config flag:

    thinking=False (default) → payload carries "thinking": {"type":"disabled"}.
                                Fast, no reasoning, content comes back
                                directly.

    thinking=True              → payload omits the field entirely, so the
                                model uses its own default (thinking ON
                                for V4.1-Flash). Slower (2–5×), but the
                                model reasons before answering – better
                                for analysis / QA tasks.

```

When thinking is ON, the response carries:

- `reasoning\_content` – the internal monologue (ignored on success);
- `content` – the final answer, ready to parse.

If `max\_tokens` is exhausted during the thinking phase, `content` comes back empty and the whole answer lives in `reasoning\_content`. The provider falls back to `reasoning\_content` in that case.

Raising `max\_output\_tokens` (via config) and `timeout` (via config) is

```
recommended when thinking is enabled – the model has more work to do.  
"""
```

```
import json  
import os  
import httpx  
import tiktoken  
from datetime import datetime  
from typing import AsyncGenerator
```

```
from .base import LLMProvider
```

```
# =====  
# CONSTANTS  
# =====
```

```
DEFAULT_TEMPERATURE = 0.3
```

```
#: DeepSeek V4.1-Flash supports up to 1M tokens of context.  
MAX_CONTEXT_TOKENS = 1_000_000
```

```
#: Where to drop raw responses for post-mortem analysis.  
DUMP_DIR = "/tmp/neurocad_llm_dumps"
```

```
# =====  
# TOKENIZER  
# =====
```

```
def get_deepseek_tokenizer():  
    try:  
        return tiktoken.get_encoding("cl100k_base")  
    except Exception:  
        return None
```

```
def count_tokens(text: str) -> int:  
    if not text:  
        return 0  
    tokenizer = get_deepseek_tokenizer()  
    if tokenizer is None:  
        return int(len(text) * 0.6)  
    try:  
        return len(tokenizer.encode(text))  
    except Exception:  
        return int(len(text) * 0.6)
```

```
# =====  
# PROVIDER CLASS  
# =====
```

```
class DeepSeekProvider(LLMProvider):  
    """DeepSeek API client (OpenAI-compatible)."""
```



```

name = "deepseek"

def __init__(self, config: dict, log=None):
    super().__init__()

    self.api_key = config.get("api_key") or ""
    self.base_url = (config.get("base_url") or "https://api.deepseek.com").rstrip("/")
    self.model = config.get("model") or "deepseek-flash"
    self.max_output_tokens = config.get("max_output_tokens") or 12000
    self.timeout = config.get("timeout") or 150
    self.thinking = bool(config.get("thinking", False))

    self.log = log

# =====
# TOKEN COUNTING
# =====

def count_tokens(self, text: str) -> int:
    return count_tokens(text)

# =====
# LOG HELPERS
# =====

def _log_info(self, message: str) -> None:
    if self.log is not None:
        self.log.log_info_sync(target="deepseek", message=message)

def _log_error(self, message: str) -> None:
    if self.log is not None:
        self.log.log_error_sync(target="deepseek", message=message)

def _log_warning(self, message: str) -> None:
    if self.log is not None:
        self.log.log_warning_sync(target="deepseek", message=message)

# =====
# ERROR EXTRACTION
# =====

def _extract_error_detail(self, raw_text: str) -> str:
    if not raw_text:
        return "нет деталей"

    try:
        data = json.loads(raw_text)

        err = data.get("error")
        if isinstance(err, dict) and err.get("message"):
            return str(err["message"])
        if isinstance(err, str):
            return err

        if data.get("message"):
            return str(data["message"])

```

```

        if data.get("detail"):
            return str(data["detail"])

    except Exception:
        pass

    text = raw_text.strip()
    if not text:
        return "нет деталей"
    if len(text) > 400:
        text = text[:400] + "..."
    return text

# =====
# RESPONSE CONTENT EXTRACTION
# =====

@staticmethod
def _extract_message_content(message: dict) -> str:
    if not isinstance(message, dict):
        return ""

    content = message.get("content") or ""
    if content.strip():
        return content

    reasoning = message.get("reasoning_content") or ""
    if reasoning.strip():
        return reasoning

    return ""

# =====
# DUMP HELPERS
# =====

def _dump_response(self, result: dict, reason: str = "") -> None:
    try:
        os.makedirs(DUMP_DIR, exist_ok=True)
        ts = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
        safe_reason = "".join(
            c if c.isalnum() or c in "_-" else "_" for c in (reason or "dump")
        )[:40]
        path = os.path.join(DUMP_DIR, f"deepseek_{ts}_{safe_reason}.json")

        with open(path, "w", encoding="utf-8") as f:
            json.dump(
                {
                    "model": self.model,
                    "max_output_tokens": self.max_output_tokens,
                    "thinking": self.thinking,
                    "reason": reason,
                    "response": result,
                },
                f,
                ensure_ascii=False,
            )
    
```

```

        indent=2,
    )

    self._log_info(f"raw response dumped -> {path}")
except Exception as e:
    self._log_warning(f"dump failed: {e}")

def _log_reasoning_head_tail(self, reasoning: str) -> None:
    if not reasoning:
        return
    head = reasoning[:500].replace("\n", " ")
    tail = reasoning[-500:].replace("\n", " ")
    self._log_warning(f"reasoning HEAD: {head}")
    self._log_warning(f"reasoning TAIL: {tail}")

# =====
# CONFIG
# =====

@property
def is_configured(self) -> bool:
    return bool(self.api_key)

def _build_api_url(self) -> str:
    return f"{self.base_url}/chat/completions"

# =====
# CHAT
# =====

async def get_response(
    self,
    messages_list: list,
    temperature: float | None = None,
    max_tokens: int | None = None,
    stream: bool = False,
) -> AsyncGenerator[str, None]:
    self._reset_tokens()

    if not self.is_configured:
        yield "Ошибка: DeepSeek API ключ не настроен."
        return

    if temperature is None:
        temperature = DEFAULT_TEMPERATURE
    if max_tokens is None:
        max_tokens = self.max_output_tokens

    for m in messages_list or []:
        self._add_tokens(m.get("content", "") if isinstance(m, dict) else "")

    self._log_info(
        f"Отправляем {len(messages_list)} сообщений в DeepSeek API "
        f"(thinking={self.thinking}, max_tokens={max_tokens})"
    )

```

```

headers = {
    "Authorization": f"Bearer {self.api_key}",
    "Content-Type": "application/json",
}
payload = {
    "model": self.model,
    "messages": messages_list,
    "temperature": temperature,
    "max_tokens": max_tokens,
    "stream": stream,
}

# ---- thinking control ----
# OFF: explicitly disable the thinking phase (V4.1-Flash / R1
#      would otherwise default to ON and burn max_tokens on
#      reasoning).
# ON:  omit the field entirely so the model uses its own
#      default. Sending "enabled" is not guaranteed to be
#      accepted – omitting is the safest way to say "your
#      default, please".
if not self.thinking:
    payload["thinking"] = {"type": "disabled"}

url = self._build_api_url()

async with httpx.AsyncClient(timeout=self.timeout) as client:
    try:
        if stream:
            async with client.stream("POST", url, headers=headers, json=payload) as response:
                if response.status_code != 200:
                    error_bytes = await response.aread()
                    error_text = error_bytes.decode("utf-8", errors="replace")
                    detail = self._extract_error_detail(error_text)
                    self._log_error(
                        f"API error: {response.status_code} - {error_text}"
                    )
                    yield f"\n⚠ Ошибка DeepSeek ({response.status_code}): {detail}\n"
                    return
            async for line in response.aiter_lines():
                if line.startswith('data: '):
                    data = line[6:]
                    if data == '[DONE]':
                        break
                    try:
                        chunk = json.loads(data)
                        if 'choices' in chunk and len(chunk['choices']) > 0:
                            delta = chunk['choices'][0].get('delta', {})
                            content = (
                                delta.get('content')
                                or delta.get('reasoning_content')
                                or ''
                            )
                            if content:
                                self._add_tokens(content)
                                yield content
                    except json.JSONDecodeError:

```

```

        continue
    else:
        response = await client.post(url, headers=headers, json=payload)
        if response.status_code != 200:
            error_text = response.text
            detail = self._extract_error_detail(error_text)
            self._log_error(
                f"API error: {response.status_code} - {error_text}"
            )
            yield f"\n⚠ Ошибка DeepSeek ({response.status_code}): {detail}\n"
            return
        result = response.json()
        if 'choices' in result and len(result['choices']) > 0:
            choice = result['choices'][0]
            message = choice.get('message', {}) or {}
            finish = choice.get('finish_reason')
            reasoning = message.get('reasoning_content') or ''
            raw_content = message.get('content') or ''

            content = self._extract_message_content(message)

            if not content:
                self._log_warning(
                    f"empty content. "
                    f"finish_reason={finish!r}, "
                    f"reasoning_len={len(reasoning)}, "
                    f"max_tokens={max_tokens}, "
                    f"model={self.model}"
                )
                self._dump_response(result, reason=f"empty_{finish}")
                self._log_reasoning_head_tail(reasoning)
            elif not raw_content.strip():
                self._log_info(
                    f"content empty, used reasoning_content "
                    f"({len(reasoning)} chars, "
                    f"finish_reason={finish!r})"
                )
                if finish == "length":
                    self._dump_response(
                        result, reason="truncated_reasoning"
                    )
                    self._log_reasoning_head_tail(reasoning)

            self._add_tokens(content)
            yield content

    except httpx.TimeoutException:
        self._log_error("timeout")
        yield "\n⚠ Ошибка: Превышено время ожидания ответа\n"
    except Exception as e:
        self._log_error(str(e))
        yield f"\n⚠ Ошибка: {str(e)}\n"

```

```

=====
FILE: utils/llm/factory.py
=====

```

```

# neurocad/utils/llm/factory.py

"""
LLM provider factory.

Usage:
    from neurocad.utils.llm.factory import get_provider

    provider = await get_provider(log=request.app.state.log)          # active provider (from DB)
    provider = await get_provider("openai", log=request.app.state.log) # explicit

Providers are imported lazily on first use, so a missing optional
dependency (e.g. cryptography for GigaChat) does not break the whole
package at import time.

The provider config is loaded from the `settings` table
(key='llm') via CoreEngineLibBaseSetupLlmService – so changes made
in the admin UI take effect immediately, without a restart.

Both get_provider() and get_configured_providers() are async
(they hit the DB).

Logging: pass `log=app.state.log` from the endpoint (Request / WebSocket).
If `log` is None – provider works silently (CLI, tests, UI listing).
"""

import importlib
from typing import Optional

from .base import LLMPProvider

# =====
# REGISTRY
# =====

#: Map of provider name → (module path, class name).
#: Names are used in .env (LLM_PROVIDER), in the DB row (llm.provider),
#: and in the UI. Keep them lowercase, no spaces.
_PROVIDERS: dict[str, tuple[str, str]] = {
    "deepseek": ("neurocad.utils.llm.deepseek", "DeepSeekProvider"),
    "openai":   ("neurocad.utils.llm.openai",   "OpenAIProvider"),
    "yandex":   ("neurocad.utils.llm.yandex",   "YandexProvider"),
    "gigachat": ("neurocad.utils.llm.gigachat", "GigaChatProvider"),
    "gemini":   ("neurocad.utils.llm.gemini",   "GeminiProvider"),
}

#: Fallback if the DB row is missing and settings.LLM_PROVIDER is not set.
DEFAULT_PROVIDER = "deepseek"

# =====
# PUBLIC API
# =====

def list_providers() -> list[str]:

```

```

    """Return all registered provider names (regardless of configuration)."""
    return list(_PROVIDERS.keys())

async def get_provider(
    name: Optional[str] = None,
    log=None,
) -> LLMProvider:
    """
    Instantiate a provider by name.

    name=None → active provider from the DB (fallback: settings.LLM_PROVIDER,
    then DEFAULT_PROVIDER).

    log – app.state.log from the endpoint (Request / WebSocket).
    If None – provider works silently (CLI, tests).

    The provider config is loaded from the DB via
    CoreEngineLibBaseSetupLlmService.get_provider_config(), with
    .env / hardcoded defaults as a fallback for missing fields.

    Raises ValueError if the name is unknown.
    Raises ImportError if the provider module cannot be loaded.
    """
    from neurocad.config import settings
    from neurocad.core.engine.lib.base.setup.service import (
        CoreEngineLibBaseSetupLlmService,
    )

    if not name:
        try:
            name = await CoreEngineLibBaseSetupLlmService.get_active_provider(log=log)
        except Exception as e:
            if log is not None:
                log.log_warning_sync(
                    target="llm",
                    message=(
                        f"Failed to read active provider from DB ({e}); "
                        f"falling back to settings.LLM_PROVIDER"
                    ),
                )
            name = getattr(settings, "LLM_PROVIDER", None) or DEFAULT_PROVIDER

    name = name.lower()

    if name not in _PROVIDERS:
        raise ValueError(
            f"Unknown LLM provider: {name!r}. "
            f"Available: {sorted(_PROVIDERS.keys())}"
        )

    module_path, class_name = _PROVIDERS[name]
    module = importlib.import_module(module_path)
    cls = getattr(module, class_name)

    config = await CoreEngineLibBaseSetupLlmService.get_provider_config(name, log=log)

```

```

    return cls(config=config, log=log)

async def get_configured_providers() -> list[str]:
    """
    Return names of providers that have credentials set.

    Used by the UI to show only providers the user can actually use.
    Silently skips providers whose module fails to import.
    """
    result: list[str] = []
    for name in _PROVIDERS:
        try:
            provider = await get_provider(name)
            if provider.is_configured:
                result.append(name)
        except Exception:
            # UI helper – silent on purpose. A provider that fails to
            # import just doesn't show up in the list.
            pass
    return result

=====
FILE: utils/llm/gemini.py
=====
# neurocad/utils/llm/gemini.py

"""
Google Gemini provider.

Uses the Gemini API (vlbeta, generateContent).
All parameters come from the `config` dict passed by the factory.

Messages are converted from OpenAI-style to Gemini format:
- system → systemInstruction
- user → {role: "user", parts: [...]}
- assistant → {role: "model", parts: [...]}

Logging: `log` is passed explicitly (app.state.log from the endpoint).
If None – provider works silently (CLI, tests).

Error reporting:
    On a non-200 response the provider yields a human-readable message
    built from the response body (extracted via _extract_error_detail),
    not just the status code. This is what the user sees in the chat.

Token accounting:
    Gemini returns exact token counts in `usageMetadata`:
        promptTokenCount, candidatesTokenCount, totalTokenCount
    When present, we use those values instead of the heuristic
    `count_tokens`. If the field is missing (older models, error
    responses), the fallback heuristic is used.
"""

import httpx
from typing import AsyncGenerator

```



```

from .base import LLMPProvider

DEFAULT_TEMPERATURE = 0.3

class GeminiProvider(LLMPProvider):
    """
    Google Gemini client.

    Config keys (all optional, fallback to hardcoded defaults):
        api_key, model, max_output_tokens, timeout

    log – app.state.log from the endpoint (Request / WebSocket).
    """

    name = "gemini"

    BASE_URL = "https://generativelanguage.googleapis.com/v1beta"

    def __init__(self, config: dict, log=None):
        # Initialize LLMPProvider (sets self.tokens_used = 0)
        super().__init__()

        self.api_key = config.get("api_key") or ""
        self.model = config.get("model") or "gemini-1.5-flash"
        self.max_output_tokens = config.get("max_output_tokens") or 8192
        self.timeout = config.get("timeout") or 120

        # app.state.log – приходит из эндпоинта.
        self.log = log

    # =====
    # LOG HELPERS
    # =====

    def _log_info(self, message: str) -> None:
        if self.log is not None:
            self.log.log_info_sync(target="gemini", message=message)

    def _log_error(self, message: str) -> None:
        if self.log is not None:
            self.log.log_error_sync(target="gemini", message=message)

    def _log_warning(self, message: str) -> None:
        if self.log is not None:
            self.log.log_warning_sync(target="gemini", message=message)

    # =====
    # ERROR EXTRACTION
    # =====

    def _extract_error_detail(self, response) -> str:
        """
        Pull a human-readable message out of an API error response.

```

```

Tries, in order:
- JSON: {"error": {"message": "..."}} ← Gemini / Google format
- JSON: {"error": "..."}
- JSON: {"message": "..."}
- JSON: {"detail": "..."}
- fallback: raw text, truncated to ~400 chars
"""
try:
    data = response.json()

    err = data.get("error")
    # {"error": {"message": "...", "code": 400, "status": "..."}}
    if isinstance(err, dict) and err.get("message"):
        return str(err["message"])
    # {"error": "..."}
    if isinstance(err, str):
        return err

    if data.get("message"):
        return str(data["message"])
    if data.get("detail"):
        return str(data["detail"])

except Exception:
    pass

text = (response.text or "").strip()
if not text:
    return "нет деталей"
if len(text) > 400:
    text = text[:400] + "..."
return text

# =====
# CONFIG
# =====

@property
def is_configured(self) -> bool:
    return bool(self.api_key)

# =====
# CHAT
# =====

async def get_response(
    self,
    messages_list: list,
    temperature: float | None = None,
    max_tokens: int | None = None,
    stream: bool = False,
) -> AsyncGenerator[str, None]:
    # Reset the per-call token counter.
    self._reset_tokens()

```

```

if not self.is_configured:
    yield "Ошибка: Gemini API ключ не настроен."
    return

if temperature is None:
    temperature = DEFAULT_TEMPERATURE
if max_tokens is None:
    max_tokens = self.max_output_tokens

system_text = ""
contents = []
for m in messages_list:
    role = m.get("role")
    text = m.get("content", "")
    if role == "system":
        system_text = text
    elif role == "user":
        contents.append({"role": "user", "parts": [{"text": text}]})
    elif role == "assistant":
        contents.append({"role": "model", "parts": [{"text": text}]})

# Count input (prompt) tokens with the fallback heuristic.
# Overwritten below if Gemini returns usageMetadata.
self._add_tokens(text)

payload = {
    "contents": contents,
    "generationConfig": {
        "temperature": temperature,
        "maxOutputTokens": max_tokens,
    },
}

if system_text:
    payload["systemInstruction"] = {"parts": [{"text": system_text}]}

url = f"{self.BASE_URL}/models/{self.model}:generateContent?key={self.api_key}"
headers = {"Content-Type": "application/json"}

async with httpx.AsyncClient(timeout=self.timeout) as client:
    try:
        response = await client.post(url, headers=headers, json=payload)
        if response.status_code != 200:
            detail = self._extract_error_detail(response)
            self._log_error(
                f"API error: {response.status_code} - {response.text}"
            )
            yield f"\n⚠ Ошибка Gemini ({response.status_code}): {detail}\n"
            return
        result = response.json()

        # ---- Prefer exact usage from Gemini, if present ----
        usage = result.get("usageMetadata") or {}
        total = usage.get("totalTokenCount")
        if isinstance(total, int) and total > 0:
            # Replace the heuristic count with the exact one.
            self.tokens_used = total

```

```

        candidates = result.get("candidates") or []
        if candidates:
            parts = candidates[0].get("content", {}).get("parts") or []
            text = "".join(p.get("text", "") for p in parts)
            if text:
                yield text
    except httpx.TimeoutException:
        self._log_error("timeout")
        yield "\n⚠ Ошибка: Превышено время ожидания ответа\n"
    except Exception as e:
        self._log_error(str(e))
        yield f"\n⚠ Ошибка: {str(e)}\n"

```

```

=====
FILE: utils/llm/gigachat.py
=====

```

```
# neurocad/utils/llm/gigachat.py
```

```
"""
```

```
GigaChat provider (Sber).
```

```
Two-step flow: OAuth → chat/completions.
```

```
The `auth_key` from Sber Studio can be in one of two formats:
```

1. Plain "client_id:client_secret"
2. base64("client_id:client_secret") – the Authorization Key
(this is what Sber Studio actually gives you)

The provider detects the format automatically. It talks to the OLD endpoint (<https://ngw.devices.sberbank.ru:9443/api/v2/oauth> and <https://gigachat.devices.sberbank.ru/api/v1>) with Basic auth – the new endpoint (api.giga.chat) is blocked by Sber WAF from this server.

TLS: the Ministry of Digital Development root CA (Russian Trusted Root CA) is not in the system trust store. The PEM text is passed in `config["ca\_pem"]` (plain text) and loaded into an `ssl.SSLContext` via ``load_verify_locations(cadata=...)``. No temp files – the context is built in memory.

If `ca\_pem` is empty, the default system trust store is used.

Config keys:

<code>auth_key</code>	– Authorization Key from Sber Studio (required)
<code>scope</code>	– GIGACHAT_API_PERS GIGACHAT_API_B2B GIGACHAT_API_CORP
<code>model</code>	– e.g. GigaChat-2-Lite
<code>max_output_tokens, timeout</code>	
<code>ca_pem</code>	– PEM text of the Russian Trusted Root CA (optional)

Logging: `log` is passed explicitly (app.state.log from the endpoint).
If None – provider works silently (CLI, tests).

Error reporting:

On a non-200 response the provider yields a human-readable message built from the response body (extracted via `_extract_error_detail`),

not just the status code. This is what the user sees in the chat.
The same helper is used for the OAuth step – a failed token request
also surfaces a readable message.

Token accounting:

```
`self.tokens_used` is reset at the start of get_response(), then  
incremented for every input message and for the (single) response  
text. GigaChat does not expose a public tokenizer; `count_tokens`  
uses the base heuristic (len * 0.6).
```

```
"""
```

```
import base64  
import ssl  
import uuid  
from typing import AsyncGenerator, Optional
```

```
import httpx
```

```
from .base import LLMPProvider
```

```
DEFAULT_TEMPERATURE = 0.3
```

```
class GigaChatProvider(LLMProvider):
```

```
    """
```

```
    GigaChat client.
```

```
    Config keys:
```

```
    auth_key          – Authorization Key from Sber Studio.  
                      Can be "client_id:client_secret" (plain) or  
                      base64("client_id:client_secret").  
    scope             – GIGACHAT_API_PERS | GIGACHAT_API_B2B | GIGACHAT_API_CORP  
    model             – e.g. GigaChat-2-Lite  
    max_output_tokens, timeout  
    ca_pem            – PEM text of the Russian Trusted Root CA (optional).
```

```
    """
```

```
    name = "gigachat"
```

```
    # OAuth + chat/completions – old endpoint.
```

```
    # api.giga.chat is intentionally NOT used: it is blocked by Sber WAF
```

```
    # from this server and answers 403 with an nginx HTML page.
```

```
    OAUTH_URL = "https://ngw.devices.sberbank.ru:9443/api/v2/oauth"
```

```
    API_URL = "https://gigachat.devices.sberbank.ru/api/v1/chat/completions"
```

```
    def __init__(self, config: dict, log=None):
```

```
        # Initialize LLMPProvider (sets self.tokens_used = 0)
```

```
        super().__init__()
```

```
        self.auth_key = (config.get("auth_key") or "").strip()
```

```
        self.scope = config.get("scope") or "GIGACHAT_API_PERS"
```

```
        self.model = config.get("model") or "GigaChat-2-Lite"
```

```
        self.max_output_tokens = config.get("max_output_tokens") or 8192
```

```
        self.timeout = config.get("timeout") or 120
```

```

    # PEM text (not a path). If set – loaded into an SSLContext.
    self.ca_pem_text = (config.get("ca_pem") or "").strip() or None
    self._ssl_context: Optional[ssl.SSLContext] = None

    # app.state.log – приходит из эндпоинта (Request / WebSocket).
    self.log = log

    self._check_auth_key()

# =====
# TOKEN COUNTING
# =====
#
# GigaChat does not expose a public tokenizer – use the base
# heuristic (len * 0.6). No override needed; kept here as a
# documented "explicitly not overridden" point.

# =====
# LOG HELPERS
# =====

def _log_info(self, message: str) -> None:
    if self.log is not None:
        self.log.log_info_sync(target="gigachat", message=message)

def _log_error(self, message: str) -> None:
    if self.log is not None:
        self.log.log_error_sync(target="gigachat", message=message)

def _log_warning(self, message: str) -> None:
    if self.log is not None:
        self.log.log_warning_sync(target="gigachat", message=message)

# =====
# ERROR EXTRACTION
# =====

def _extract_error_detail(self, response) -> str:
    """
    Pull a human-readable message out of an API error response.

    Tries, in order:
    - JSON: {"error": {"message": "..."}}
    - JSON: {"error": "..."}
    - JSON: {"message": "..."}
    - JSON: {"detail": "..."}
    - fallback: raw text, truncated to ~400 chars

    GigaChat returns errors as:
    {"status": 404, "message": "No such model"}
    so the plain "message" branch is the common one.
    """
    try:
        data = response.json()

        err = data.get("error")

```

```

        if isinstance(err, dict) and err.get("message"):
            return str(err["message"])
        if isinstance(err, str):
            return err

        if data.get("message"):
            return str(data["message"])
        if data.get("detail"):
            return str(data["detail"])

    except Exception:
        pass

    text = (response.text or "").strip()
    if not text:
        return "нет деталей"
    if len(text) > 400:
        text = text[:400] + "..."
    return text

# =====
# AUTH KEY
# =====

def _check_auth_key(self) -> None:
    """Log if the auth_key looks malformed (for diagnostics only)."""
    if not self.auth_key:
        self._log_warning("auth_key is empty")
        return

    try:
        padded = self.auth_key + "=" * (-len(self.auth_key) % 4)
        decoded = base64.b64decode(padded).decode("utf-8")
        if ":" not in decoded:
            self._log_warning(
                "auth_key does not look like "
                "base64(client_id:client_secret)"
            )
    except Exception:
        self._log_warning("auth_key is not valid base64")

@property
def is_configured(self) -> bool:
    return bool(self.auth_key)

# =====
# SSL CONTEXT
# =====

def _build_ssl_context(self) -> ssl.SSLContext:
    ctx = ssl.create_default_context()

    if self.ca_pem_text:
        try:
            ctx.load_verify_locations(cadata=self.ca_pem_text)
            self._log_info("loaded custom CA from PEM text")

```

```

        except Exception as e:
            self._log_error(f"failed to load CA PEM: {e}")
    else:
        self._log_info("ca_pem not set – using system trust store")

    return ctx

def _get_ssl_context(self) -> ssl.SSLContext:
    if self._ssl_context is None:
        self._ssl_context = self._build_ssl_context()
    return self._ssl_context

# =====
# OAUTH
# =====

async def _get_access_token(
    self,
    client: httpx.AsyncClient,
) -> tuple[Optional[str], Optional[str]]:
    """
    Request an access token.

    Returns (token, error_detail):
    - on success: (token, None)
    - on failure: (None, human-readable error string)
    """
    headers = {
        "Authorization": f"Basic {self.auth_key}",
        "RqUID": str(uuid.uuid4()),
        "Content-Type": "application/x-www-form-urlencoded",
        "Accept": "application/json",
    }
    data = {"scope": self.scope}

    try:
        response = await client.post(
            self.OAUTH_URL, headers=headers, data=data, timeout=30
        )
        if response.status_code != 200:
            detail = self._extract_error_detail(response)
            self._log_error(
                f"OAuth error: {response.status_code} - {response.text}"
            )
            return None, detail
        return response.json().get("access_token"), None
    except Exception as e:
        self._log_error(f"OAuth exception: {e}")
        return None, str(e)

# =====
# CHAT
# =====

async def get_response(
    self,

```



```

messages_list: list,
temperature: float | None = None,
max_tokens: int | None = None,
stream: bool = False,
) -> AsyncGenerator[str, None]:
    # Reset the per-call token counter.
    self._reset_tokens()

    if not self.is_configured:
        yield "Ошибка: GigaChat не настроен. Нужен auth_key."
        return

    if temperature is None:
        temperature = DEFAULT_TEMPERATURE
    if max_tokens is None:
        max_tokens = self.max_output_tokens

    # Count input (prompt) tokens.
    for m in messages_list or []:
        self._add_tokens(m.get("content", "") if isinstance(m, dict) else "")

    ssl_ctx = self._get_ssl_context()

    async with httpx.AsyncClient(timeout=self.timeout, verify=ssl_ctx) as client:
        token, oauth_err = await self._get_access_token(client)
        if not token:
            yield f"\n⚠ Ошибка GigaChat (OAuth): {oauth_err or 'не удалось получить токен'}\n"
            return

        headers = {
            "Authorization": f"Bearer {token}",
            "Content-Type": "application/json",
        }
        payload = {
            "model": self.model,
            "messages": messages_list,
            "temperature": temperature,
            "max_tokens": max_tokens,
            "stream": False,
        }

        try:
            response = await client.post(
                self.API_URL, headers=headers, json=payload
            )
            if response.status_code != 200:
                detail = self._extract_error_detail(response)
                self._log_error(
                    f"API error: {response.status_code} - {response.text}"
                )
                yield f"\n⚠ Ошибка GigaChat ({response.status_code}): {detail}\n"
                return
            result = response.json()
            choices = result.get("choices") or []
            if choices:
                content = choices[0].get("message", {}).get("content", "")

```

```

        if content:
            # Count output (completion) tokens.
            self._add_tokens(content)
            yield content
    except httpx.TimeoutException:
        self._log_error("timeout")
        yield "\n△ Ошибка: Превышено время ожидания ответа\n"
    except Exception as e:
        self._log_error(str(e))
        yield f"\n△ Ошибка: {str(e)}\n"

=====
FILE: utils/llm/openai.py
=====
# neurocad/utils/llm/openai.py

"""
OpenAI provider (OpenAI-compatible API).

All parameters come from the `config` dict passed by the factory.
Supports streaming and non-streaming modes.

Logging: `log` is passed explicitly (app.state.log from the endpoint).
If None – provider works silently (CLI, tests).

Error reporting:
    On a non-200 response the provider yields a human-readable message
    built from the response body (extracted via _extract_error_detail),
    not just the status code. Works in both streaming and non-streaming
    modes.

Token accounting:
    `self.tokens_used` is reset at the start of get_response(), then
    incremented for every input message and every yielded chunk. After
    the generator is fully consumed, it holds the total cost.
    The exact tokenizer is tiktoken.encoding_for_model(self.model)
    (falls back to cl100k_base).
"""

import json
import httpx
import tiktoken
from typing import AsyncGenerator

from .base import LLMPProvider

DEFAULT_TEMPERATURE = 0.3

#: gpt-4o-mini supports 128K tokens of context.
MAX_CONTEXT_TOKENS = 128_000

# =====
# TOKENIZER (module-level helpers)
# =====

```

```

#
# NOTE: these are standalone helpers for use outside the class
# (CLI, tests). The provider class overrides `count_tokens` and
# uses the same tokenizer internally, with the configured model.

def get_openai_tokenizer(model: str | None = None):
    """
    Tokenizer for OpenAI.

    tiktoken ships encodings for OpenAI models directly.
    Falls back to cl100k_base if the model is unknown.
    """
    model_name = model or "gpt-4o-mini"
    try:
        return tiktoken.encoding_for_model(model_name)
    except KeyError:
        try:
            return tiktoken.get_encoding("cl100k_base")
        except Exception:
            return None
    except Exception:
        return None

def count_tokens(text: str, model: str | None = None) -> int:
    """Approximate token count using the model's encoding (fallback: len * 0.6)."""
    if not text:
        return 0
    tokenizer = get_openai_tokenizer(model)
    if tokenizer is None:
        return int(len(text) * 0.6)
    try:
        return len(tokenizer.encode(text))
    except Exception:
        return int(len(text) * 0.6)

# =====
# PROVIDER CLASS
# =====

class OpenAIProvider(LLMProvider):
    """
    OpenAI API client.

    Config keys (all optional, fallback to hardcoded defaults):
    api_key, base_url, model, max_output_tokens, timeout

    log – app.state.log from the endpoint (Request / WebSocket).
    """

    name = "openai"

    def __init__(self, config: dict, log=None):
        # Initialize LLMProvider (sets self.tokens_used = 0)
        super().__init__()

```

```

self.api_key = config.get("api_key") or ""
self.base_url = (config.get("base_url") or "https://api.openai.com/v1").rstrip("/")
self.model = config.get("model") or "gpt-4o-mini"
self.max_output_tokens = config.get("max_output_tokens") or 4096
self.timeout = config.get("timeout") or 120

# app.state.log – приходит из эндпоинта.
self.log = log

# =====
# TOKEN COUNTING
# =====

def count_tokens(self, text: str) -> int:
    """Exact count via tiktoken for self.model (fallback: len * 0.6)."""
    return count_tokens(text, model=self.model)

# =====
# LOG HELPERS
# =====

def _log_info(self, message: str) -> None:
    if self.log is not None:
        self.log.log_info_sync(target="openai", message=message)

def _log_error(self, message: str) -> None:
    if self.log is not None:
        self.log.log_error_sync(target="openai", message=message)

def _log_warning(self, message: str) -> None:
    if self.log is not None:
        self.log.log_warning_sync(target="openai", message=message)

# =====
# ERROR EXTRACTION
# =====

def _extract_error_detail(self, raw_text: str) -> str:
    """
    Pull a human-readable message out of an API error response.

    OpenAI / OpenAI-compatible providers return errors as:
    {"error": {"message": "...", "type": "...", "code": "..."}}
    """
    if not raw_text:
        return "нет деталей"

    try:
        data = json.loads(raw_text)

        err = data.get("error")
        if isinstance(err, dict) and err.get("message"):
            return str(err["message"])
        if isinstance(err, str):
            return err
    
```

```

        if data.get("message"):
            return str(data["message"])
        if data.get("detail"):
            return str(data["detail"])

    except Exception:
        pass

    text = raw_text.strip()
    if not text:
        return "нет деталей"
    if len(text) > 400:
        text = text[:400] + "..."
    return text

# =====
# CONFIG
# =====

@property
def is_configured(self) -> bool:
    return bool(self.api_key)

def _build_api_url(self) -> str:
    return f"{self.base_url}/chat/completions"

# =====
# CHAT
# =====

async def get_response(
    self,
    messages_list: list,
    temperature: float | None = None,
    max_tokens: int | None = None,
    stream: bool = False,
) -> AsyncGenerator[str, None]:
    # Reset the per-call token counter.
    self._reset_tokens()

    if not self.is_configured:
        yield "Ошибка: OpenAI API ключ не настроен."
        return

    if temperature is None:
        temperature = DEFAULT_TEMPERATURE
    if max_tokens is None:
        max_tokens = self.max_output_tokens

    # Count input (prompt) tokens.
    for m in messages_list or []:
        self._add_tokens(m.get("content", "") if isinstance(m, dict) else "")

    self._log_info(f"Отправляем {len(messages_list)} сообщений в OpenAI API")

```

```

headers = {
    "Authorization": f"Bearer {self.api_key}",
    "Content-Type": "application/json",
}
payload = {
    "model": self.model,
    "messages": messages_list,
    "temperature": temperature,
    "max_tokens": max_tokens,
    "stream": stream,
}
url = self._build_api_url()

async with httpx.AsyncClient(timeout=self.timeout) as client:
    try:
        if stream:
            async with client.stream("POST", url, headers=headers, json=payload) as response:
                if response.status_code != 200:
                    error_bytes = await response.aread()
                    error_text = error_bytes.decode("utf-8", errors="replace")
                    detail = self._extract_error_detail(error_text)
                    self._log_error(
                        f"API error: {response.status_code} - {error_text}"
                    )
                    yield f"\n⚠ Ошибка OpenAI ({response.status_code}): {detail}\n"
                    return
                async for line in response.aiter_lines():
                    if line.startswith('data: '):
                        data = line[6:]
                        if data == '[DONE]':
                            break
                        try:
                            chunk = json.loads(data)
                            if 'choices' in chunk and len(chunk['choices']) > 0:
                                delta = chunk['choices'][0].get('delta', {})
                                content = delta.get('content', '')
                                if content:
                                    # Count output (completion) tokens.
                                    self._add_tokens(content)
                                    yield content
                        except json.JSONDecodeError:
                            continue
                    else:
                        response = await client.post(url, headers=headers, json=payload)
                        if response.status_code != 200:
                            error_text = response.text
                            detail = self._extract_error_detail(error_text)
                            self._log_error(
                                f"API error: {response.status_code} - {error_text}"
                            )
                            yield f"\n⚠ Ошибка OpenAI ({response.status_code}): {detail}\n"
                            return
                        result = response.json()
                        if 'choices' in result and len(result['choices']) > 0:
                            content = result['choices'][0].get('message', {}).get('content', '')
                            # Count output tokens.

```

```

        self._add_tokens(content)
        yield content

    except httpx.TimeoutException:
        self._log_error("timeout")
        yield "\n△ Ошибка: Превышено время ожидания ответа\n"
    except Exception as e:
        self._log_error(str(e))
        yield f"\n△ Ошибка: {str(e)}\n"

=====
FILE: utils/llm/yandex.py
=====
# neurocad/utils/llm/yandex.py

"""
YandexGPT provider.

Uses the Yandex Cloud Foundation Models API.
All parameters come from the `config` dict passed by the factory.
Streaming is not implemented.

Logging: `log` is passed explicitly (app.state.log from the endpoint).
If None – provider works silently (CLI, tests).

Error reporting:
    On a non-200 response the provider yields a human-readable message
    built from the response body (extracted via _extract_error_detail),
    not just the status code.

Yandex returns errors as:
    {"error": {"grpcCode": 16, "httpCode": 401, "message": "...", "httpStatus": "..."}}
    so the nested "error.message" branch is the common one.

Token accounting:
    `self.tokens_used` is reset at the start of get_response(), then
    incremented for every input message and for the (single) response
    text. Yandex does not expose a public tokenizer; `count_tokens`
    uses the base heuristic (len * 0.6).
"""

import httpx
from typing import AsyncGenerator

from .base import LLMPProvider

DEFAULT_TEMPERATURE = 0.3

class YandexProvider(LLMPProvider):
    """
    YandexGPT client.

    Config keys (all optional, fallback to hardcoded defaults):
        api_key, folder_id, model, max_output_tokens, timeout

```

```

log = app.state.log from the endpoint (Request / WebSocket).
"""

name = "yandex"

API_URL = "https://llm.api.cloud.yandex.net/foundationModels/v1/completion"

def __init__(self, config: dict, log=None):
    # Initialize LLMPProvider (sets self.tokens_used = 0)
    super().__init__()

    self.api_key = config.get("api_key") or ""
    self.folder_id = config.get("folder_id") or ""
    self.model = config.get("model") or "yandexgpt-lite"
    self.max_output_tokens = config.get("max_output_tokens") or 4096
    self.timeout = config.get("timeout") or 120

    # app.state.log – приходит из эндпоинта.
    self.log = log

# =====
# TOKEN COUNTING
# =====
#
# Yandex does not expose a public tokenizer – use the base
# heuristic (len * 0.6). No override needed; kept here as a
# documented "explicitly not overridden" point.

# =====
# LOG HELPERS
# =====

def _log_info(self, message: str) -> None:
    if self.log is not None:
        self.log.log_info_sync(target="yandex", message=message)

def _log_error(self, message: str) -> None:
    if self.log is not None:
        self.log.log_error_sync(target="yandex", message=message)

def _log_warning(self, message: str) -> None:
    if self.log is not None:
        self.log.log_warning_sync(target="yandex", message=message)

# =====
# ERROR EXTRACTION
# =====

def _extract_error_detail(self, response) -> str:
    """
    Pull a human-readable message out of an API error response.

    Tries, in order:
    - JSON: {"error": {"message": "..."}} ← Yandex format
    - JSON: {"error": "..."}

```



```

- JSON: {"message": "..."}
- JSON: {"detail": "..."}
- fallback: raw text, truncated to ~400 chars
"""
try:
    data = response.json()

    err = data.get("error")
    if isinstance(err, dict) and err.get("message"):
        return str(err["message"])
    if isinstance(err, str):
        return err

    if data.get("message"):
        return str(data["message"])
    if data.get("detail"):
        return str(data["detail"])

except Exception:
    pass

text = (response.text or "").strip()
if not text:
    return "нет деталей"
if len(text) > 400:
    text = text[:400] + "..."
return text

# =====
# CONFIG
# =====

@property
def is_configured(self) -> bool:
    return bool(self.api_key and self.folder_id)

# =====
# CHAT
# =====

async def get_response(
    self,
    messages_list: list,
    temperature: float | None = None,
    max_tokens: int | None = None,
    stream: bool = False,
) -> AsyncGenerator[str, None]:
    # Reset the per-call token counter.
    self._reset_tokens()

    if not self.is_configured:
        yield "Ошибка: YandexGPT не настроен. Нужны api_key и folder_id."
        return

    if temperature is None:
        temperature = DEFAULT_TEMPERATURE

```

```

if max_tokens is None:
    max_tokens = self.max_output_tokens

system_text = ""
chat_messages = []
for m in messages_list:
    role = m.get("role")
    text = m.get("content", "")
    if role == "system":
        system_text = text
    else:
        chat_messages.append({"role": role, "text": text})

    # Count input (prompt) tokens for every message.
    self._add_tokens(text)

if system_text:
    chat_messages.insert(0, {"role": "system", "text": system_text})

payload = {
    "modelUri": f"gpt://{self.folder_id}/{self.model}",
    "completionOptions": {
        "stream": False,
        "temperature": temperature,
        "maxTokens": str(max_tokens),
    },
    "messages": chat_messages,
}

headers = {
    "Authorization": f"Api-Key {self.api_key}",
    "Content-Type": "application/json",
}

async with httpx.AsyncClient(timeout=self.timeout) as client:
    try:
        response = await client.post(self.API_URL, headers=headers, json=payload)
        if response.status_code != 200:
            detail = self._extract_error_detail(response)
            self._log_error(
                f"API error: {response.status_code} - {response.text}"
            )
            yield f"\n⚠ Ошибка YandexGPT ({response.status_code}): {detail}\n"
            return
        result = response.json()
        alternatives = result.get("result", {}).get("alternatives") or []
        if alternatives:
            text = alternatives[0].get("message", {}).get("text", "")
            if text:
                # Count output (completion) tokens.
                self._add_tokens(text)
                yield text
    except httpx.TimeoutException:
        self._log_error("timeout")
        yield "\n⚠ Ошибка: Превышено время ожидания ответа\n"
    except Exception as e:
        self._log_error(str(e))

```

```
yield f"\n⚠ Ошибка: {str(e)}\n"
```

```
=====
FILE: utils/llm/__init__.py
=====
```

```
=====
FILE: core/engine/api.js
=====
```

```
// app/core/engine/api.js

/**
 * Универсальный API-клиент для работы с бэкендом
 * Используется фабрикой для CRUD-операций
 */
export class API {
  constructor(baseUrl = '/core/engine') {
    this.baseUrl = baseUrl;
  }

  /**
   * Выполнить запрос к API
   */
  async request(endpoint, options = {}) {
    const url = `${this.baseUrl}${endpoint}`;
    const config = {
      headers: {
        'Content-Type': 'application/json',
        ...options.headers
      },
      ...options
    };

    if (options.body && typeof options.body === 'object') {
      config.body = JSON.stringify(options.body);
    }

    try {
      const response = await fetch(url, config);
      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.message || 'Ошибка API');
      }

      return data;
    } catch (error) {
      console.error('[API] Error:', error);
      throw error;
    }
  }

  async get(endpoint) {
    return this.request(endpoint, { method: 'GET' });
  }
}
```

```

    async post(endpoint, data) {
        return this.request(endpoint, { method: 'POST', body: data });
    }

    async put(endpoint, data) {
        return this.request(endpoint, { method: 'PUT', body: data });
    }

    async patch(endpoint, data) {
        return this.request(endpoint, { method: 'PATCH', body: data });
    }

    async delete(endpoint) {
        return this.request(endpoint, { method: 'DELETE' });
    }
}

```

```

=====
FILE: core/engine/binder.js
=====

```

```

// app/core/engine/binder.js

```

```

/**
 * Универсальный обработчик событий для фабрики
 * Навешивает события на элементы по описанию из JSON-конфига
 */
export class Binder {
    constructor() {
        // API будет загружен динамически
        this.api = null;
        this.handlers = {
            'click': {},
            'change': {},
            'input': {},
            'submit': {}
        };
    }

    /**
     * Привязать события к элементу
     */
    bind(element, events) {
        if (!events) return;

        Object.keys(events).forEach(eventType => {
            const handlers = events[eventType];
            if (!Array.isArray(handlers)) return;

            handlers.forEach(handler => {
                const action = handler.action;
                const params = handler.params || {};

                element.addEventListener(eventType, (e) => {
                    this.execute(action, params, e, element);
                });
            });
        });
    }
}

```

```

        });
    });
}

/**
 * Привязать все события из конфига
 */
bindAll(config) {
    if (config.events) {
        const events = Array.isArray(config.events) ? config.events : [config.events];
        events.forEach(eventConfig => {
            const { selector, event = 'click', action, params = {} } = eventConfig;
            if (!selector || !action) return;

            const elements = document.querySelectorAll(selector);
            elements.forEach(element => {
                element.addEventListener(event, (e) => {
                    this.execute(action, params, e, element);
                });
            });
        });
    }
}

/**
 * Выполнить действие
 */
execute(action, params, event, element) {
    switch (action) {
        case 'log':
            console.log('[Binder] Log:', params.message || '');
            break;

        case 'alert':
            alert(params.message || '');
            break;

        case 'api':
            this.apiRequest(params);
            break;

        case 'navigate':
            window.location.href = params.url || '/';
            break;

        case 'custom':
            if (params.function && typeof window[params.function] === 'function') {
                window[params.function](params, event, element);
            }
            break;

        default:
            console.warn('[Binder] Неизвестное действие:', action);
    }
}
}

```

```

/**
 * Запрос к API через API
 */
async apiRequest(params) {
  try {
    if (!this.api) {
      const { API } = await import('./api.js');
      this.api = new API();
    }

    const data = await this.api.request(params.url, {
      method: params.method || 'GET',
      headers: params.headers || {},
      body: params.body
    });

    if (params.onSuccess) {
      if (typeof params.onSuccess === 'function') {
        params.onSuccess(data);
      } else if (typeof window[params.onSuccess] === 'function') {
        window[params.onSuccess](data);
      }
    }

    return data;
  } catch (error) {
    console.error('[Binder] API error:', error);
    if (params.onError) {
      if (typeof params.onError === 'function') {
        params.onError(error);
      } else if (typeof window[params.onError] === 'function') {
        window[params.onError](error);
      }
    }
  }
}
}
}

```

```

=====
FILE: core/engine/engine.js
=====

```

```

// app/static/core/engine/engine.js

```

```

/**
 * CoreEngine – application entry point.
 *
 * Responsibilities:
 * - parse URL (path + params)
 * - load renderer / binder / config / components
 * - expose shared utilities (loadCSS, fetchJson, auth, base)
 * - orchestrate the initial render + bind
 *
 * engine.js is a GENERIC loader: it does NOT know about any
 * specific module (auth, profile, word, ...). Anything that a
 * module needs from the URL travels down as generic query params
 * in `props.params` – the module itself decides what to do with

```

```

* them.
*/
class CoreEngine {
  constructor(moduleName) {
    console.log('[CoreEngine] Constructor called, moduleName:', moduleName);

    this.moduleName = moduleName || 'assistent';
    this.config = null;
    this.container = document.body;
    this.components = {};
    this.loadedConfigs = {};
    this.renderer = null;
    this.binder = null;
    this.static_version = '';

    // Shared HTTP utility. Loaded in loadApi().
    // Available everywhere as window.coreEngine.fetchJson.
    this.fetchJson = null;

    // ===== URL parsing =====
    // /core/engine/aleksmir.ru/page/20260809/182504
    //   parts = ["core", "engine", "aleksmir.ru", "page", "20260809", "182504"]
    //   rest  = ["aleksmir.ru", "page", "20260809", "182504"]
    //
    // Rule: all-digit segments → params.
    //       everything before the first digit → pathParts.
    //       everything after → paramsList.
    const parts = window.location.pathname.split('/').filter(Boolean);
    const rest = parts.slice(2); // drop "core", "engine"

    this.pathParts = [];
    this.paramsList = [];
    let inParams = false;

    for (const part of rest) {
      if (!inParams && /^\d+$/.test(part)) {
        inParams = true;
      }
      if (inParams) {
        this.paramsList.push(part);
      } else {
        this.pathParts.push(part);
      }
    }

    // Empty pathParts (request to /core/engine/) – use default.
    if (this.pathParts.length === 0) {
      this.pathParts = [this.moduleName];
    }

    // Single segment (module) – duplicate: X → X/X
    // (module config lives at mod/X/X.json).
    if (this.pathParts.length === 1) {
      this.pathParts = [this.pathParts[0], this.pathParts[0]];
    }
  }
}

```

```

// ===== Config path =====
// If the template gave us an explicit config_path via
// <body data-config-path="...">, use it. Otherwise fall back
// to the path computed from the URL.
const configPathOverride = document.body.dataset.configPath || null;
this.configPath = configPathOverride
  || [...this.pathParts, ...this.paramsList].join('/');

// ===== Module base URL =====
// /core/engine/aleksmir.ru
// Used to build page links: {baseUrl}/page/{date}/{time}
this.baseUrl = '/core/engine/' + this.pathParts[0];

console.log('[CoreEngine] pathParts:', this.pathParts);
console.log('[CoreEngine] paramsList:', this.paramsList);
console.log('[CoreEngine] configPath:', this.configPath);
console.log('[CoreEngine] baseUrl:', this.baseUrl);

this.authRequired = document.body.dataset.authRequired === 'true';
this.authRedirect = document.body.dataset.authRedirect || null;

// ===== Query params =====
// All GET parameters from the URL, forwarded as-is to every
// component via props.params. engine.js does not interpret
// them – that is the job of the component that owns the
// feature (e.g. Base reads `auth` and `section` for the
// profile). This keeps engine.js free of any module-specific
// knowledge: adding a new module with its own query params
// never requires touching this file.
this.params = Object.fromEntries(
  new URLSearchParams(window.location.search)
);

// ===== Nav instance =====
// If the template set data-nav-id (e.g. on page-view URLs
// like /core/engine/pages/<nav_id>/...), every component
// that expects props.nav_id gets it injected below.
const navIdRaw = document.body.dataset.navId || null;
this.navId = (navIdRaw !== null && navIdRaw !== '')
  ? Number(navIdRaw)
  : null;

console.log('[CoreEngine] authRequired:', this.authRequired);
console.log('[CoreEngine] params:', this.params);
console.log('[CoreEngine] navId:', this.navId);
console.log('[CoreEngine] Calling init()...');

this.init();
}

async init() {
  console.log('[CoreEngine] init() START');
  try {
    console.log('[CoreEngine] Loading shared API...');
    await this.loadApi();
    console.log('[CoreEngine] Shared API loaded');
  }

```



```

    console.log('[CoreEngine] Loading Renderer and Binder...');
    await this.loadRendererAndBinder();
    console.log('[CoreEngine] Renderer and Binder loaded');

    console.log('[CoreEngine] Loading config...');
    await this.loadConfig();
    console.log('[CoreEngine] Config loaded:', this.config);

    const hasParams = Object.keys(this.params).length > 0;
    if (this.authRequired || hasParams || this.navId !== null) {
        console.log('[CoreEngine] Injecting runtime props...');
        this._injectRuntimeProps(this.config);
    }

    console.log('[CoreEngine] Resolving refs...');
    await this.resolveRefs(this.config);
    console.log('[CoreEngine] Refs resolved');

    console.log('[CoreEngine] Loading components...');
    await this.loadComponents();
    console.log('[CoreEngine] Components loaded:', Object.keys(this.components));

    console.log('[CoreEngine] Rendering...');
    await this.renderer.render(this.config, this.components);
    console.log('[CoreEngine] Render complete');

    console.log('[CoreEngine] Binding...');
    this.binder.bindAll(this.config);
    console.log('[CoreEngine] Bind complete');

    console.log('[CoreEngine] init() COMPLETE');
} catch (error) {
    console.error('[CoreEngine] Init error:', error);
    this.showError();
}
}

/**
 * Load the shared HTTP utility (fetchJson) from base/auth/api.js
 * with cache-busting version.
 *
 * After this, `window.coreEngine.fetchJson(url, options)` is
 * available everywhere – no static imports, no per-module
 * dynamic import() calls.
 */
async loadApi() {
    console.log('[CoreEngine] loadApi() START');
    const url = `/static/core/engine/lib/base/auth/api.js${this.static_version ? '?v=' + this.static_version : ''}`;

    try {
        const mod = await import(url);
        this.fetchJson = mod.fetchJson;
        console.log('[CoreEngine] loadApi() SUCCESS');
    } catch (error) {
        console.error('[CoreEngine] loadApi() error:', error);
    }
}

```

```

        throw error;
    }
}

/**
 * Inject runtime-only props into the config before rendering.
 *
 * Runtime props fall into two categories:
 *
 * 1. System flags read from <body data-*>:
 *     - authRequired    → Base
 *     - authRedirect    → Base
 *     - nav_id          → Word, Pages, and any other component
 *                        that needs to scope API calls to a nav
 *
 * 2. Generic query params from the URL, forwarded as-is:
 *     - params          → any component that owns a feature
 *                        keyed by a query param (e.g. Base
 *                        reads `auth` and `section`).
 *
 * The WHOLE component tree is walked (root + every nested
 * component in `config.components`), so that child components
 * like `pages` and `word` also receive `nav_id` and `params`.
 * Components that do not understand a given prop simply ignore it.
 */
_injectRuntimeProps(config) {
    if (!config) return;

    const hasParams = Object.keys(this.params).length > 0;

    const injectInto = (obj) => {
        // Auth flags (system-level)
        obj.authRequired = this.authRequired;
        if (this.authRedirect) {
            obj.authRedirect = this.authRedirect;
        }
        // Nav scoping (system-level)
        if (this.navId !== null) {
            obj.nav_id = this.navId;
        }
        // Generic query params – merged into any existing params
        // the component may already carry from its config.
        if (hasParams) {
            obj.params = { ...(obj.params || {}), ...this.params };
        }
    };

    const walk = (obj) => {
        if (!obj || typeof obj !== 'object') return;
        if (obj.component) {
            injectInto(obj);
        }
        if (Array.isArray(obj.components)) {
            obj.components.forEach(walk);
        }
    };
};

```

```

        walk(config);
    }

    async loadRendererAndBinder() {
        console.log('[CoreEngine] loadRendererAndBinder() START');
        const rendererUrl = `/static/core/engine/renderer.js${this.static_version ? '?v=' + this.static_version : ''}`;
        const binderUrl = `/static/core/engine/binder.js${this.static_version ? '?v=' + this.static_version : ''}`;

        console.log('[CoreEngine] Loading renderer:', rendererUrl);
        console.log('[CoreEngine] Loading binder:', binderUrl);

        try {
            const [{ Renderer }, { Binder }] = await Promise.all([
                import(rendererUrl),
                import(binderUrl)
            ]);

            this.renderer = new Renderer();
            this.binder = new Binder();
            console.log('[CoreEngine] loadRendererAndBinder() SUCCESS');
        } catch (error) {
            console.error('[CoreEngine] Renderer/Binder load error:', error);
            throw error;
        }
    }

    async loadConfig() {
        console.log('[CoreEngine] loadConfig() START');

        // configPath comes from data-config-path (if set) or from
        // the URL (computed in the constructor).
        // URL without .json: /core/engine/api/<configPath>
        const url = `/core/engine/api/${this.configPath}`;
        console.log('[CoreEngine] Loading config:', url);

        try {
            const response = await fetch(url);
            if (!response.ok) {
                throw new Error(`Failed to load config: ${url}, status: ${response.status}`);
            }
            this.config = await response.json();
            console.log('[CoreEngine] loadConfig() SUCCESS');
        } catch (error) {
            console.error('[CoreEngine] Config load error:', error);
            throw error;
        }
    }

    async resolveRefs(config) {
        if (!config) return;

        if (config.components && Array.isArray(config.components)) {
            for (let i = 0; i < config.components.length; i++) {
                const item = config.components[i];
                if (item.$ref) {

```

```

        console.log('[CoreEngine] Resolving ref:', item.$ref);
        const refConfig = await this.loadRefConfig(item.$ref);
        config.components[i] = refConfig;
    }
    if (config.components[i] && config.components[i].components) {
        await this.resolveRefs(config.components[i]);
    }
}
}
}

async loadRefConfig(refPath) {
    if (this.loadedConfigs[refPath]) {
        console.log('[CoreEngine] Ref already loaded:', refPath);
        return this.loadedConfigs[refPath];
    }

    const url = `/core/engine/block/${refPath}`;
    console.log('[CoreEngine] Loading ref:', url);

    try {
        const response = await fetch(url);
        if (!response.ok) {
            throw new Error(`Failed to load config: ${refPath}, status: ${response.status}`);
        }
        const config = await response.json();
        this.loadedConfigs[refPath] = config;
        console.log('[CoreEngine] Ref loaded:', refPath);
        return config;
    } catch (error) {
        console.error('[CoreEngine] Ref load error:', error);
        throw error;
    }
}

async loadComponents() {
    console.log('[CoreEngine] loadComponents() START');
    const componentPaths = this.collectComponentPaths(this.config);
    console.log('[CoreEngine] Components to load:', componentPaths);

    if (componentPaths.length === 0) {
        console.warn('[CoreEngine] No components to load');
        return;
    }

    const loadPromises = componentPaths.map(({ name }) => {
        return new Promise(async (resolve, reject) => {
            if (this.components[name]) {
                console.log('[CoreEngine] Component already loaded:', name);
                resolve();
                return;
            }

            try {
                const url = `/static/core/engine/lib/${name}/${name}.js${this.static_version ? '?v=' + this.static_version : ''}`;
                console.log('[CoreEngine] Loading component:', name, 'from', url);

```

```

const module = await import(url);
console.log('[CoreEngine] Module loaded:', name, module);

const ComponentClass = module.default ||
  module[`${name.charAt(0).toUpperCase() + name.slice(1)}`] ||
  module[name];

if (!ComponentClass) {
  throw new Error(`Class for component ${name} not found`);
}

console.log('[CoreEngine] Class found:', name, ComponentClass);
this.components[name] = ComponentClass;

if (!window.coreEngine.components) {
  window.coreEngine.components = {};
}
window.coreEngine.components[name] = ComponentClass;

  resolve();
} catch (error) {
  console.error(`[CoreEngine] Component load error ${name}:`, error);
  reject(error);
}
});
});

try {
  await Promise.all(loadPromises);
  console.log('[CoreEngine] loadComponents() SUCCESS');
} catch (error) {
  console.error('[CoreEngine] loadComponents() ERROR:', error);
  throw error;
}
}

collectComponentPaths(config) {
  const paths = [];
  const visited = new Set();

  const traverse = (item) => {
    if (!item) return;

    if (item.component) {
      const name = item.component;
      if (!visited.has(name)) {
        visited.add(name);
        paths.push({ name });
      }
    }

    if (item.components && Array.isArray(item.components)) {
      for (const child of item.components) {
        traverse(child);
      }
    }
  }
}

```

```

        }
    };

    traverse(config);
    return paths;
}

loadCSS(path) {
    const fullUrl = `/static/${path}${this.static_version ? '?v=' + this.static_version : ''}`;

    const existing = document.querySelector(`link[href="${fullUrl}"]`);
    if (existing) {
        return;
    }

    const link = document.createElement('link');
    link.rel = 'stylesheet';
    link.href = fullUrl;
    document.head.appendChild(link);
}

showError() {
    console.log('[CoreEngine] Showing error');
    if (this.container) {
        this.container.innerHTML = `
            <div style="display:flex;flex-direction:column;align-items:center;justify-
content:center;padding:60px 20px;color:#dc2626;text-align:center;">
                <div style="font-size:48px;margin-bottom:16px;"></div>
                <h2 style="font-size:20px;margin-bottom:8px;">Ошибка загрузки модуля</h2>
                <p style="color:#64748b;">Не удалось загрузить конфигурацию модуля</p>
                <p style="color:#94a3b8;font-size:13px;margin-top:8px;">${this.moduleName}</p>
            </div>
        `;
    }
}

}

// Log file load
console.log('[CoreEngine] engine.js loaded');

=====
FILE: core/engine/lib/word/actions.js
=====
// app/core/engine/lib/word/actions.js

/**
 * Word actions – high-level operations.
 * All utilities via word._utils.
 *
 * Uses window.coreEngine.fetchJson – loaded once by CoreEngine.loadApi().
 *
 * Scoping:
 * Every word API request carries ?nav_id=<id> from word._qs
 * (see word.js constructor). The single exception is the public
 * page URL – there nav_id goes into the path, not the query.
 */

```

```

/**
 * Write page title into the app header (.core-engine-lib-base-title)
 * and into document.title.
 *
 * Uses the shared caption helper (word._setCaption) if available –
 * it saves the previous title, so the page can restore it on destroy.
 * Falls back to direct DOM write if the helper is not available.
 */
export function setHeaderTitle(word, retries = 5) {
  const el = document.querySelector('.core-engine-lib-base-title');

  if (!el) {
    if (retries > 0) {
      setTimeout(() => setHeaderTitle(word, retries - 1), 50);
    } else {
      console.warn('[Word] .core-engine-lib-base-title not found in header');
    }
    return;
  }

  const title = word.pageData?.title || '';

  // Preferred path – shared caption helper.
  if (word._setCaption) {
    word._savedCaption = word._setCaption(title, title);
    return;
  }

  // Fallback – direct write, save the previous title on the word instance.
  if (word._originalHeaderTitle === null) {
    word._originalHeaderTitle = el.textContent;
  }
  el.textContent = title;
}

/**
 * Go back to the admin panel.
 */
export function goBack(word) {
  console.log('[Word] Going back to admin');

  if (window.history.length > 1) {
    window.history.back();
  } else {
    window.location.href = '/core/engine/default/';
  }
}

/**
 * Open public version of the page in a new tab.
 *
 * URL schema:
 * /page/<nav_id>/<date>/<time>
 *
 * nav_id comes from word.navId (injected via props). If it's missing,

```

```

* we cannot build a valid public URL – bail out instead of guessing.
*/
export function openPublicPage(word) {
  console.log('[Word] Opening public page');

  const datetime = word.pageData?.datetime;
  if (!datetime) {
    console.warn('[Word] No datetime – cannot open public page');
    return;
  }

  if (word.navId == null) {
    console.warn('[Word] No nav_id – cannot build public URL');
    return;
  }

  const date = word._utils.formatDateShort(datetime);
  const time = word._utils.formatTimeShort(datetime);

  if (!date || !time) {
    console.warn('[Word] Failed to format date/time');
    return;
  }

  const url = `/page/${word.navId}/${date}/${time}`;
  console.log('[Word] Public URL:', url);

  window.open(url, '_blank', 'noopener,noreferrer');
}

// =====
// SAVE (manual + auto)
// =====

/**
 * Persist page content to the server.
 *
 * HTML and CSS are sent as separate fields:
 *   content      = HTML (no <style>)
 *   content_json = GrapesJS project JSON (string)
 *   css          = CSS from editor.getCss()
 *
 * Also updates local word.pageData so the next render uses
 * the fresh values without an extra GET.
 *
 * Throws an Error with `.status` set to the HTTP status code –
 * so callers can distinguish 401 (session expired) from other errors.
 *
 * @param {Object} word
 * @param {Object} data – { html, css, project }
 */
async function _persistContent(word, data) {
  if (!word.pageId) {
    throw new Error('Unknown page id');
  }
}

```



```

const html = data.html || '';
const css = (data.css || '').trim();
const projectJson = JSON.stringify(data.project);

const url = `/core/engine/lib/word/${word.pageId}${word._qs}`;
const fetchJson = window.coreEngine?.fetchJson;

await fetchJson(url, {
  method: 'PUT',
  body: {
    content: html,
    content_json: projectJson,
    css: css,
  },
});

// Update local state so the next render uses fresh values.
word.pageData.content = html;
word.pageData.content_json = projectJson;
word.pageData.css = css;

console.log('[Word] Content saved (HTML + CSS separate)');
}

/**
 * Manual save – called from the editor toolbar / Ctrl+S.
 *
 * Persists, then closes the editor.
 */
export async function saveContent(word, data) {
  console.log('[Word] Saving content for id:', word.pageId);

  await _persistContent(word, data);

  // Close editor
  const version = window.coreEngine?.static_version || Date.now();
  const mod = await import(`./bridge.js?v=${version}`);
  await mod.closeEditor(word);
}

/**
 * Auto-save – called from the editor's background auto-save.
 *
 * Persists only. The editor stays open and untouched.
 */
export async function autoSaveContent(word, data) {
  console.log('[Word] Auto-saving content for id:', word.pageId);

  await _persistContent(word, data);

  console.log('[Word] Auto-save done');
}

// =====
// HISTORY
// =====

```

```

/**
 * List snapshots for the current page (metadata only, no html/content_json/css).
 *
 * GET /core/engine/lib/word/{page_id}/history?nav_id=<id>
 *
 * @param {Object} word – Word instance
 * @returns {Promise<Array>} – [{ id, action, note, created_at }, ...]
 */
export async function listHistory(word) {
  console.log('[Word] Loading history for id:', word.pageId);

  if (!word.pageId) {
    throw new Error('Unknown page id');
  }

  const url = `/core/engine/lib/word/${word.pageId}/history${word._qs}`;
  const fetchJson = window.coreEngine?.fetchJson;
  const json = await fetchJson(url);

  return json.data || [];
}

/**
 * Get one full snapshot (html + content_json + css).
 *
 * GET /core/engine/lib/word/{page_id}/history/{hist_id}?nav_id=<id>
 *
 * @param {Object} word – Word instance
 * @param {number} histId – snapshot id
 * @returns {Promise<Object>} – { id, page_id, html, content_json, css, action, note, created_at }
 */
export async function getHistoryItem(word, histId) {
  console.log('[Word] Loading history item:', histId);

  if (!word.pageId) {
    throw new Error('Unknown page id');
  }

  const url = `/core/engine/lib/word/${word.pageId}/history/${histId}${word._qs}`;
  const fetchJson = window.coreEngine?.fetchJson;
  const json = await fetchJson(url);

  return json.data;
}

/**
 * Roll the page back to the given snapshot.
 *
 * POST /core/engine/lib/word/{page_id}/rollback/{hist_id}?nav_id=<id>
 *
 * Also updates local pageData with the rolled-back values, so the
 * next render (article view / editor) uses the restored state.
 *
 * @param {Object} word – Word instance
 * @param {number} histId – snapshot id

```

```

* @returns {Promise<Object>} – { id, title, content, content_json, css, updated_at }
*/
export async function rollback(word, histId) {
  console.log('[Word] Rollback to snapshot:', histId);

  if (!word.pageId) {
    throw new Error('Unknown page id');
  }

  const url = `/core/engine/lib/word/${word.pageId}/rollback/${histId}${word._qs}`;
  const fetchJson = window.coreEngine?.fetchJson;

  const json = await fetchJson(url, { method: 'POST' });
  const data = json.data;

  // Update local state so next render uses restored values
  if (data && word.pageData) {
    if (data.content !== undefined) word.pageData.content = data.content;
    if (data.content_json !== undefined) word.pageData.content_json = data.content_json;
    if (data.css !== undefined) word.pageData.css = data.css;
  }

  return data;
}

```

```

=====
FILE: core/engine/lib/word/bridge.js
=====

```

```

// app/core/engine/lib/word/bridge.js

```

```

/**
 * Bridge – glue between Word (page view) and GrapesJS Editor.
 *
 * Responsibilities:
 * - openEditor: switch Word widget into edit mode, mount Editor.
 * - closeEditor: destroy Editor and restore the article view.
 *
 * Template handling:
 * If the page has template_id, the base template HTML is rendered
 * OUTSIDE the GrapesJS iframe (in .editor-template), and GrapesJS
 * mounts only into the [data-slot="content"] placeholder.
 * This way the template layout is visible but not editable.
 *
 * If the page has no template, GrapesJS mounts into .editor-canvas
 * directly – same as before.
 *
 * Save vs auto-save:
 * - onSave – manual save (toolbar / Ctrl+S). Persists and
 * closes the editor.
 * - onAutoSave – background auto-save. Persists only; the editor
 * stays open and untouched.
 *
 * Empty-article tutorial:
 * When the editor is opened for an EMPTY article (no content), a
 * short three-step walkthrough (toolbar → area-left → area-right,
 * see ./editor/tutorial.js) is shown. It runs every time, with no

```

```

* "never show again" flag.
*
* The tutorial CSS (./editor/tutorial.css) is loaded here, right
* before the JS module is imported. This guarantees the bubble is
* styled on the very first paint – otherwise it would render as a
* bare <div> until some other code happens to pull the stylesheet
* in. Loading is guarded by a <link> check so we do not append
* duplicates on every editor open.
*
* Race protection:
* Editor init is async (dynamic imports, GrapesJS init). If the user
* cancels while init is still running, we must NOT create a new Editor
* after close. A monotonic token `word._editorToken` guards against this:
* each open/close increments the token, and openEditor checks the token
* after each await boundary.
*/

/**
* Path to the tutorial stylesheet, relative to /static/.
* Kept in one place so the <link> guard and loadCSS() call cannot drift.
*/
const TUTORIAL_CSS_PATH = 'core/engine/lib/word/editor/tutorial.css';

/**
* Ensure the tutorial stylesheet is in the document exactly once.
*
* coreEngine.loadCSS() may or may not be idempotent depending on the
* build; we guard here anyway so repeated editor opens do not append
* duplicate <link> tags. The check matches both the absolute
* (/static/...) and versioned (?v=...) forms.
*/
function _ensureTutorialCss() {
  const href = `/static/${TUTORIAL_CSS_PATH}`;

  // Already present (any version) – nothing to do.
  const existing = document.querySelector(
    `link[rel="stylesheet"][href^="${href}"]`
  );
  if (existing) return;

  if (window.coreEngine?.loadCSS) {
    window.coreEngine.loadCSS(TUTORIAL_CSS_PATH);
  } else {
    // Fallback – coreEngine not ready yet. Should not happen in
    // practice because openEditor() runs after Word._init().
    console.warn('[Word] coreEngine.loadCSS not available – appending <link> manually');
    const link = document.createElement('link');
    link.rel = 'stylesheet';
    link.href = href;
    document.head.appendChild(link);
  }
}

/**
* Open the GrapesJS editor inside the Word widget.
*

```

```

* @param {Object} word – Word instance
* @returns {Promise<void>}
*/
export async function openEditor(word) {
  console.log('[Word] Opening GrapesJS editor');

  // Race token – invalidated by closeEditor
  word._editorToken = (word._editorToken || 0) + 1;
  const myToken = word._editorToken;

  word.isEditing = true;
  word.toolbarEl?.classList.add('core-engine-lib-word-toolbar-hidden');
  word._clearAllResizers();

  // Clear widget content (keep toolbar and widget itself)
  if (word.widgetContentEl) {
    while (word.widgetContentEl.firstChild) {
      word.widgetContentEl.removeChild(word.widgetContentEl.firstChild);
    }
  } else {
    console.warn('[Word] widgetContentEl not found – re-rendering');
    await word._view.render(word);
  }

  // ===== Load base template (if any) =====
  // Render template HTML into .editor-template, replace [data-slot="content"]
  // with the .editor-slot placeholder. GrapesJS will mount into that slot.
  const templateId = word.pageData?.template_id;
  let templateHtml = null;

  if (templateId) {
    const template = await word._data.loadTemplateById(word, templateId);

    // Race check – the editor could be closed during template load
    if (word._editorToken !== myToken) {
      console.log('[Word] Editor was closed during template load – aborting open');
      return;
    }

    if (template && template.content) {
      templateHtml = template.content;
      console.log('[Word] Template loaded for editor:', templateId);
    } else {
      console.warn('[Word] Template not loaded – editor opens without layout');
    }
  }

  // Lazy-load Editor with cache-busting version
  const version = window.coreEngine?.static_version || Date.now();
  const { loadEditor } = await import(`./editor/index.js?v=${version}`);
  const Editor = await loadEditor();

  // ★ Check: was the editor closed while we were loading?
  if (word._editorToken !== myToken) {
    console.log('[Word] Editor was closed during load – aborting open');
    return;
  }

```

```

}

// ===== Create Editor instance =====
word.editorInstance = new Editor(word.widgetContentEl, {
  title: word.pageData?.title || 'Страница',
  html: word.pageData?.content || '',
  project: word.pageData?.content_json
    ? word._utils.safeJsonParse(word.pageData.content_json)
    : null,
  pageId: word.pageId,
  pageData: word.pageData,

  // Template HTML – Editor renders it into .editor-template before
  // GrapesJS init, and mounts into .editor-slot.
  templateHtml: templateHtml,

  // Manual save: persist AND close the editor.
  onSave: async (data) => {
    await word._saveContent(data);
  },

  // Auto-save: persist only. Do NOT close the editor.
  onAutoSave: async (data) => {
    await word._autoSaveContent(data);
  },

  onCancel: () => {
    closeEditor(word);
  },
});

await word.editorInstance.waitForInit();

// Second race check – Editor could be closed during init too
if (word._editorToken !== myToken) {
  console.log('[Word] Editor was closed during init – destroying');
  try {
    word.editorInstance?.destroy?();
  } catch (e) {
    console.warn('[Word] destroy after race error:', e);
  }
  word.editorInstance = null;
  return;
}

// ===== Empty-article tutorial =====
// Shown when the article has no content. Runs on EVERY open of an
// empty page – no "never show again" flag. Destroyed together with
// the editor (see closeEditor).
_maybeStartTutorial(word, version);
}

/**
 * Show the tutorial if the article is empty.
 *
 * "Empty" uses the same notion as Word._isContentEmpty(): HTML is

```

```

* stripped, style tags are ignored, whitespace is collapsed.
*
* The tutorial is stored on `word._editorTutorial` so closeEditor()
* can tear it down cleanly.
*
* CSS is loaded BEFORE the JS module so the bubble is styled on the
* very first paint. _ensureTutorialCss() is idempotent – repeated
* editor opens do not append duplicate <link> tags.
*
* @param {Object} word
* @param {string|number} version – cache-busting version
*/
async function _maybeStartTutorial(word, version) {
  if (!word._isContentEmpty || !word._isContentEmpty()) return;

  try {
    // 1. CSS first – the bubble must be styled on the very first paint.
    _ensureTutorialCss();

    // 2. Then the JS module (versioned dynamic import).
    const { EditorTutorial } = await import(
      `./editor/tutorial.js?v=${version}`
    );

    word._editorTutorial = new EditorTutorial();
    word._editorTutorial.start();
  } catch (e) {
    console.warn('[Word] EditorTutorial load failed:', e);
  }
}

/**
* Close the GrapesJS editor and restore the article view.
*
* Safe against:
* - destroy() throwing (wrapped in try/catch)
* - stale widgetContentEl (re-queried before use)
* - pending openEditor (token invalidated at the top)
*
* @param {Object} word – Word instance
*/
export async function closeEditor(word) {
  console.log('[Word] Closing GrapesJS editor');

  // ★ Invalidate any pending openEditor
  word._editorToken = (word._editorToken || 0) + 1;

  // ★ Tear down the tutorial (if any) – the editor is going away.
  if (word._editorTutorial) {
    try {
      word._editorTutorial.stop();
    } catch (e) {
      console.warn('[Word] tutorial.stop() error:', e);
    }
    word._editorTutorial = null;
  }
}

```

```

word.toolbarEl?.classList.remove('core-engine-lib-word-toolbar-hidden');

// Destroy editor safely – never let errors break the cleanup
if (word.editorInstance?.destroy) {
  try {
    word.editorInstance.destroy();
  } catch (e) {
    console.warn('[Word] editor.destroy() error:', e);
  }
}
word.editorInstance = null;
word.isEditing = false;

// Remove any remaining resizer handles
word._clearAllResizers();

// Re-query widgetContentEl – it may be stale after re-render
const wc = word.container?.querySelector('.core-engine-lib-word-widget-content');
if (wc) word.widgetContentEl = wc;

// Restore article (guaranteed)
if (word.widgetContentEl) {
  while (word.widgetContentEl.firstChild) {
    word.widgetContentEl.removeChild(word.widgetContentEl.firstChild);
  }
  const article = await word._view.buildArticle(word);
  word.widgetContentEl.appendChild(article);
} else {
  await word._view.render(word);
}

console.log('[Word] Editor closed, article restored');
}

```

```

=====
FILE: core/engine/lib/word/data.js
=====

```

```

// neurocad/core/engine/lib/word/data.js

```

```

/**
 * Word data loading – fetch page by id or by date/time.
 * Pure functions, take `word` instance for context (qs, pageId).
 *
 * Uses window.coreEngine.fetchJson – loaded once by CoreEngine.loadApi().
 *
 * URL schemas:
 * /core/engine/pages/<nav_id>/<date>/<time> – page view URL
 *   engine.paramsList = [<nav_id>, <date>, <time>]
 *
 * /core/engine/lib/word/item/<page_id> – load by page id
 * /core/engine/lib/word/bydatetime/<date>/<time>?nav_id=<id>
 *   – load by date/time within a nav instance
 */

```



```

* Load page by URL params (nav_id/date/time from coreEngine.paramsList).
* Sets word.pageData and word.pageId.
*
* URL: /core/engine/pages/<nav_id>/<date>/<time>
* paramsList = [<nav_id>, <date>, <time>]
*
* @returns {Promise<boolean>} true if loaded, false if no params
*/
export async function loadByParams(word) {
  const engine = window.coreEngine;
  const params = engine?.paramsList || [];
  const fetchJson = engine?.fetchJson;

  // paramsList layout for the new URL schema:
  //   [0] → nav_id
  //   [1] → date   (YYYYMMDD)
  //   [2] → time   (HHMMSS)
  const navId = params[0] || null;
  const date  = params[1] || null;
  const time  = params[2] || null;

  if (!navId || !date || !time) {
    console.log('[Word] URL params missing');
    return false;
  }

  console.log(`[Word] Loading by date/time: nav=${navId} ${date} ${time}`);

  // nav_id comes from the URL itself here, so we don't use word.qs.
  // This avoids sending ?nav_id= twice (once from the path, once
  // from the query string).
  const url = `/core/engine/lib/word/bydatetime/${date}/${time}?nav_id=${encodeURIComponent(navId)}`;

  let result;
  try {
    result = await fetchJson(url);
  } catch (err) {
    if (err.status === 404) {
      throw new Error('Page not found');
    }
    throw err;
  }

  if (!result.success) {
    throw new Error(result.message || 'Page load error');
  }

  word.pageData = result.data;
  word.pageId = result.data.id;
  console.log('[Word] Page loaded:', word.pageData.title);
  return true;
}

/**
* Load page by word.pageId.
*

```

```

* URL: /core/engine/lib/word/item/<page_id>?nav_id=<id>
* nav_id comes from word._qs (see word.js constructor).
*/
export async function loadById(word) {
  console.log(`[Word] Loading by id: ${word.pageId}`);

  const fetchJson = window.coreEngine?.fetchJson;
  const url = `/core/engine/lib/word/item/${word.pageId}${word._qs}`;

  let result;
  try {
    result = await fetchJson(url);
  } catch (err) {
    if (err.status === 404) {
      throw new Error('Page not found');
    }
    throw err;
  }

  if (!result.success) {
    throw new Error(result.message || 'Page load error');
  }

  word.pageData = result.data;
  console.log('[Word] Page loaded:', word.pageData.title);
}

```

```

/**
* Load a template page by id (for template_id resolution).
* Does NOT touch word.pageData – returns the raw page object.
*
* URL: /core/engine/lib/word/item/<id>?nav_id=<id>
* nav_id comes from word._qs. Template must belong to the same
* nav as the page – the backend enforces this.
*
* @param {Object} word
* @param {number} id
* @returns {Promise<Object|null>}
*/

```

```

export async function loadTemplateById(word, id) {
  if (!id) return null;

  console.log(`[Word] Loading template id: ${id}`);

  const fetchJson = window.coreEngine?.fetchJson;
  const url = `/core/engine/lib/word/item/${id}${word._qs}`;

  let result;
  try {
    result = await fetchJson(url);
  } catch (err) {
    if (err.status === 404) {
      console.warn(`[Word] Template ${id} not found`);
      return null;
    }
    console.warn(`[Word] Template load error:`, err.message);
  }
}

```

```

        return null;
    }

    if (!result.success || !result.data) {
        console.warn(`[Word] Template ${id} – bad response`);
        return null;
    }

    return result.data;
}

```

```

=====
FILE: core/engine/lib/word/word.js
=====

```

```

// app/core/engine/lib/word/word.js

```

```

/**
 * Word – page content display component.
 *
 * Orchestrator. Delegates work to submodules:
 *   data.js      – loadById, loadByParams, loadTemplateById
 *   view.js      – render, buildArticle, buildToolbarButtons, renderError
 *   actions.js   – setHeaderTitle, goBack, openPublicPage, saveContent, autoSaveContent
 *   bridge.js    – openEditor, closeEditor (GrapesJS editor)
 *   utils.js     – date formatting, safe JSON parse
 *
 * All submodules are loaded dynamically with a version to avoid
 * browser cache issues on updates.
 *
 * Scoping:
 *   Every word API request carries ?nav_id=<id>, which identifies
 *   the nav instance this page belongs to. The value comes from
 *   props.nav_id (injected by CoreEngine from <body data-nav-id>),
 *   with a fallback to window.coreEngine.navId / body dataset.
 *
 * Permissions:
 *   - Guests: read-only view of the page.
 *   - Any authenticated user (including superadmin): full toolbar
 *     (back / edit / public), editor, save.
 *
 * Caption: on _init() we pick up the shared caption helpers from
 * window.coreEngine.base (_setCaption / _restoreCaption). They are
 * used by actions.setHeaderTitle() to swap the header/tab title to the
 * page title, and by destroy() to restore whatever was there before.
 *
 * Empty-article hint:
 *   When the page has no content AND the user can edit, an onboarding
 *   tooltip (EmptyArticleHint, see ./hint.js) is shown under the
 *   "Редактировать" button. It can be dismissed (x) or permanently
 *   suppressed via a "Не показывать больше" checkbox (localStorage).
 *
 * Effects: each effect is its own file under editor/effects/fx/,
 * loaded on every page – so that effects applied in the editor
 * (classes like .fx-shadow-top-n) render on the public page as well.
 * The list is fetched from GET /editor/effects (single source of
 * truth – effects/registry.json on the backend, served by the API).

```

```

* Loading order matches the editor canvas:
*   content.css → fx/*.css → blocks/*.css.
*
* Public API (to Renderer / Word):
*   - waitForInit()
*   - isInitialized()
*   - destroy()
*/
export class Word {
  constructor(container, props = {}) {
    console.log('[Word] Constructor', { container, props });

    this.container = container;
    this.props = props;

    // Page data
    this.pageId = props.page_id || null;
    this.pageData = props.page_data || null;

    // Nav instance for multi-tenant scoping.
    // Priority:
    //   1. props.nav_id          (injected by CoreEngine)
    //   2. window.coreEngine.navId (set by CoreEngine constructor)
    //   3. document.body.dataset (raw data-nav-id attribute)
    this.navId = props.nav_id
      || window.coreEngine?.navId
      || document.body.dataset.navId
      || null;

    // Query string appended to every word API request.
    // Empty string when navId is not known – the request will then
    // fail with 422 (nav_id is required) rather than silently
    // returning foreign data.
    this._qs = this.navId != null
      ? `?nav_id=${encodeURIComponent(this.navId)}`
      : '';

    // State
    this.editorInstance = null;
    this.isEditing = false;
    this._initialized = false;
    this._initPromise = null;
    this._editorToken = 0;

    // DOM refs
    this.widgetEl = null;
    this.toolbarEl = null;
    this.widgetContentEl = null;

    // Caption helpers – filled from window.coreEngine.base in _init().
    this._setCaption = null;
    this._restoreCaption = null;
    this._savedCaption = null;

    // Fallback header title (used only if caption helpers are unavailable).
    this._originalHeaderTitle = null;

```

```

// Onboarding hint for an empty article. Created in _init().
this._emptyHint = null;

// Path to Base icons
this._iconsBase = '/static/core/engine/lib/base/images';

// Submodules (loaded in _init)
this._data = null;
this._view = null;
this._actions = null;
this._utils = null;

// CSS is loaded async – errors do not block Word creation.
this._loadCSS().catch(e => {
  console.warn('[Word] _loadCSS() error:', e);
});

this._initPromise = this._init();
}

/**
 * Load page-level CSS into the main document.
 *
 * Shared CSS (word.css, llm.css, content.css) – hardcoded.
 * Effect CSS (fx/*.css) – fetched from GET /editor/effects
 * (single source of truth). Block CSS (elements, layout, ready, ...)
 * – taken from blocks/manifest.js (dynamic import, versioned).
 *
 * ORDER MATTERS:
 *   content.css – shared atoms (.btn, .card, .grid, .h1, .text, ...)
 *                 AND --theme-* variables (they are defined at the
 *                 top of content.css, scoped under
 *                 .core-engine-lib-word-blocks).
 *   fx/*.css     – effect classes (.fx-shadow-top-n, ...).
 *                 Must load AFTER content.css (needs the theme vars),
 *                 BEFORE blocks/*.css (blocks may override).
 *                 One file per effect, matching the editor canvas
 *                 (see editor/grapes/index.js → canvasCss).
 *   blocks/*.css – block-specific classes (.hero, .flex-shell, ...).
 *                 Must load AFTER content.css so block rules can
 *                 override shared rules at equal specificity.
 *
 * NOTE: editor/css/theme.css is NOT loaded here. That file is the
 * GrapesJS UI theme (light override of GrapesJS dark panels),
 * only needed inside the editor top document – loaded by
 * grapes.js → cssFiles when the editor opens.
 *
 * coreEngine.loadCSS() expects a path relative to /static/
 * (no leading slash, no /static/ prefix).
 */
async _loadCSS() {
  if (!window.coreEngine?.loadCSS) return;

  window.coreEngine.loadCSS('core/engine/lib/word/word.css');
  window.coreEngine.loadCSS('core/engine/lib/word/llm/llm.css');

```

```

window.coreEngine.loadCSS('core/engine/lib/word/hint.css');

// Shared content classes (.btn, .card, .grid, .hl, .text, ...)
// AND --theme-* variables (defined at the top of content.css).
// Scoped under .core-engine-lib-word-blocks.
window.coreEngine.loadCSS('core/engine/lib/word/editor/css/content.css');

// Effect classes – one file per effect.
// Same order as in the editor canvas: after content.css
// (theme vars), before blocks/*.css.
const version = window.coreEngine?.static_version || Date.now();
const effectIds = await this._loadEffectIds(version);

for (const id of effectIds) {
  window.coreEngine.loadCSS(
    `core/engine/lib/word/editor/effects/fx/${id}.css`
  );
}

// Block-specific classes – must load AFTER content.css,
// so block rules can override shared rules at equal specificity.
// List comes from blocks/manifest.js – dynamic, versioned
// import (same rule as every other module here).
try {
  const mod = await import(`./editor/blocks/manifest.js?v=${version}`);
  const blockCss = mod.blockCssUrls ? mod.blockCssUrls(version) : [];

  blockCss.forEach((url) => {
    // '/static/core/...?v=123' → 'core/...'
    const path = url
      .replace(/^\/static\/\//, '')
      .replace(/\?.*$/, '');
    window.coreEngine.loadCSS(path);
  });
} catch (e) {
  console.warn('[Word] blocks/manifest.js not loaded:', e);
}
}

/**
 * Fetch the list of effect ids from GET /editor/effects.
 *
 * The API is the single source of truth (backed by
 * editor/effects/registry.json on the backend). There is no
 * bundled fallback: on failure we return an empty list, and
 * the page just renders without effect CSS.
 *
 * NOTE: /editor/effects does not care about nav scoping – effects
 * are global. So we deliberately do NOT append this._qs here.
 * Effects are shared across all navs.
 *
 * @param {string|number} version – cache-busting version
 * @returns {Promise<string[]>}
 */
async _loadEffectIds(version) {
  try {

```

```

const url = `/core/engine/lib/word/editor/effects`;
console.log('[Word] GET', url);

const fetchJson = window.coreEngine?.fetchJson;
if (!fetchJson) throw new Error('coreEngine.fetchJson not available');

const data = await fetchJson(url);
if (data && data.success && Array.isArray(data.data)) {
  const ids = data.data.map(e => e.id).filter(Boolean);
  console.log('[Word] effects loaded from API:', ids.length);
  return ids;
}
throw new Error('unexpected payload');
} catch (e) {
  console.warn('[Word] effects API failed:', e);
  return [];
}
}

async _init() {
  console.log('[Word] _init() START');
  const version = window.coreEngine?.static_version || Date.now();

  try {
    // Pick up caption helpers from Base (loaded before Word).
    const base = window.coreEngine?.base;
    this._setCaption = base?._setCaption || null;
    this._restoreCaption = base?._restoreCaption || null;

    // Load all submodules in parallel
    const [data, view, actions, utils] = await Promise.all([
      import(`./data.js?v=${version}`),
      import(`./view.js?v=${version}`),
      import(`./actions.js?v=${version}`),
      import(`./utils.js?v=${version}`),
    ]);

    this._data = data;
    this._view = view;
    this._actions = actions;
    this._utils = utils;

    // Load page data if not provided
    if (!this.pageData) {
      if (this.pageId) {
        await data.loadById(this);
      } else {
        const loaded = await data.loadByParams(this);
        if (!loaded) {
          throw new Error('No data to load: no page_data, page_id, or URL params');
        }
      }
    }

    // Set header title and render
    actions.setHeaderTitle(this);
  }
}

```

```

    await view.render(this);

    // Empty-article hint – load the class and show if the page
    // has no content and the user can edit.
    const { EmptyArticleHint } = await import(`./hint.js?v=${version}`);
    this._emptyHint = new EmptyArticleHint();
    this._updateEmptyArticleHint();

    this._initialized = true;
    console.log('[Word] _init() COMPLETE');
  } catch (error) {
    console.error('[Word] Init error:', error);
    this._renderError(error.message);
    this._initialized = false;
    throw error;
  }
}

/**
 * Show the empty-article hint when the page has no content AND
 * the user can edit (has an "edit" button in the toolbar).
 *
 * Deferred to the next tick so the toolbar DOM exists by the time
 * we measure its position.
 */
_updateEmptyArticleHint() {
  if (!this._emptyHint) return;

  if (this._isContentEmpty() && this._canEdit()) {
    setTimeout(() => {
      if (this._emptyHint) this._emptyHint.show();
    }, 0);
  } else {
    this._emptyHint.hide();
  }
}

/**
 * Whether the article content is effectively empty.
 *
 * Strips HTML tags and collapses whitespace – "<p> </p>" or
 * "<p>&nbsp;</p>" counts as empty. Matches the same notion of
 * "empty" that view.buildArticle() uses when it substitutes
 * the "Контент пуст" placeholder.
 */
_isContentEmpty() {
  const html = this.pageData?.content;
  if (!html) return true;

  const text = String(html)
    .replace(/<style[^>]*>[\s\S]*?<\/style>/gi, '') // ignore <style>
    .replace(/<[^>]*>/g, '') // strip tags
    .replace(/&nbsp;/g, ' ')
    .trim();

  return text.length === 0;
}

```



```

}

// =====
// DELEGATION
// =====

_renderError(message) {
  this._view?.renderError(this, message);
}

_goBack() {
  this._actions?.goBack(this);
}

_openPublicPage() {
  this._actions?.openPublicPage(this);
}

/**
 * Manual save – delegated to actions.saveContent.
 * Persists AND closes the editor.
 */
async _saveContent(data) {
  await this._actions?.saveContent(this, data);
}

/**
 * Auto-save – delegated to actions.autoSaveContent.
 * Persists only; the editor stays open.
 */
async _autoSaveContent(data) {
  await this._actions?.autoSaveContent(this, data);
}

_setHeaderTitle() {
  this._actions?.setHeaderTitle(this);
}

// =====
// EDITOR (delegated to bridge.js)
// =====

async _openEditor() {
  // The user is about to edit – the hint has done its job.
  if (this._emptyHint) this._emptyHint.hide();

  const version = window.coreEngine?.static_version || Date.now();
  const mod = await import(`./bridge.js?v=${version}`);
  await mod.openEditor(this);
}

_closeEditor() {
  const version = window.coreEngine?.static_version || Date.now();
  import(`./bridge.js?v=${version}`).then(mod => {
    mod.closeEditor(this);
  });
}

```

```

}

// =====
// RESIZER HANDLES
// =====

_clearAllResizers() {
    document.querySelectorAll('.core-engine-lib-word-editor-resizer').forEach(h => h.remove());
}

// =====
// PERMISSIONS
// =====

/**
 * Whether the current user can edit pages (open the editor,
 * save, use the toolbar).
 *
 * Any authenticated user qualifies; guests do not.
 *
 * The check tries several auth shapes so it works regardless of
 * how BaseAuth exposes state:
 *   - auth.isAuth()           → boolean
 *   - auth.isAuthenticated → boolean
 *   - auth.getUser()         → user object or null
 */
_canEdit() {
    const auth = window.coreEngine?.auth;
    if (!auth) return false;

    if (typeof auth.isAuth === 'function' && auth.isAuth()) {
        return true;
    }
    if (typeof auth.isAuthenticated === 'boolean' && auth.isAuthenticated) {
        return true;
    }
    if (typeof auth.getUser === 'function') {
        return auth.getUser() !== null;
    }
    return false;
}

/**
 * Whether the current user is a superadmin.
 *
 * Kept for callers that specifically need superadmin-only
 * behaviour. is_superadmin may come from the backend as bool,
 * int (0/1) or string ("0"/"1"), so all three are accepted.
 */
_isSuperadmin() {
    const auth = window.coreEngine?.auth;
    if (!auth) return false;

    if (typeof auth.isSuperadmin === 'function') {
        return !!auth.isSuperadmin();
    }
}

```

```

    if (typeof auth.getUser === 'function') {
        const user = auth.getUser();
        if (!user) return false;
        const v = user.is_superuser;
        return v === true || v === 1 || v === "1";
    }
    return false;
}

/**
 * Deprecated alias for _canEdit().
 *
 * Kept temporarily so existing callers (view.js, etc.) don't
 * break if they still reference _isAdmin(). New code should
 * call _canEdit() (any authenticated user) or _isSuperadmin()
 * (superadmin only).
 */
_isAdmin() {
    return this._canEdit();
}

// =====
// PUBLIC METHODS
// =====

isInitialized() {
    return this._initialized;
}

async waitForInit() {
    if (this._initPromise) await this._initPromise;
    return this._initialized;
}

destroy() {
    console.log('[Word] destroy()');

    // Remove the onboarding hint (if visible).
    if (this._emptyHint) {
        this._emptyHint.remove();
        this._emptyHint = null;
    }

    // Restore the title that was on screen before Word opened.
    if (this._restoreCaption) {
        this._restoreCaption(this._savedCaption);
    }
    this._savedCaption = null;

    // Fallback – if caption helpers were unavailable, use the old path.
    if (!this._restoreCaption && this._originalHeaderTitle !== null) {
        const el = document.querySelector('.core-engine-lib-base-title');
        if (el) el.textContent = this._originalHeaderTitle;
        this._originalHeaderTitle = null;
    }
}

```

```

    // Destroy editor if open
    if (this.editorInstance?.destroy) {
        try {
            this.editorInstance.destroy();
        } catch (e) {
            console.warn('[Word] destroy() error:', e);
        }
    }
    this.editorInstance = null;
    this._editorToken = (this._editorToken || 0) + 1;

    this._clearAllResizers();

    this._initialized = false;
    this._initPromise = null;
}
}

```

```

=====
FILE: core/engine/lib/word/editor/editor.js
=====

```

```

// neurocad/core/engine/lib/word/editor/editor.js

```

```

/**
 * Editor – GrapesJS visual editor (part of the Word component).
 *
 * Orchestrator class. Does nothing itself, assembles submodules:
 *
 * widgets.js    -> builds DOM of three areas (left / center / right) + toolbar
 * styles.js     -> provides StyleManager sections
 * grapes/       -> loads GrapesJS CSS/JS and calls grapesjs.init()
 * assets.js     -> works with media library (GET /assets, POST /assets/upload)
 * blocks/       -> registers block library
 * resizer.js    -> handles for dragging borders between areas
 * template.js   -> base template rendering + slot height sync
 * toolbar.js    -> editor toolbar + hotkeys + device switcher
 * dataloader.js -> load project data + re-apply scope class / body traits
 * modals.js     -> HTML+CSS page modal and element CSS modal
 * history.js    -> page history modal (list, preview, rollback)
 * io/import.js  -> import page from .grp / .html / URL
 * io/export.js  -> export page to .html / .grp
 * formatter.js  -> js-beautify wrapper (CSS / HTML pretty-print)
 * base/modal    -> Base modals (incl. code modal for HTML + CSS)
 * ../llm/chat.js -> LLM chat panel (right)
 * ../llm/presets.js -> Presets list (left tab)
 *
 * All internal modules are loaded dynamically, with version from coreEngine,
 * to avoid browser cache on updates.
 *
 * Auto-save (interval-based):
 * Every AUTO_SAVE_INTERVAL_MS (30 s) we check a "dirty" flag that is
 * set by GrapesJS `update` events and cleared after a successful save.
 * Nothing is saved if there were no changes since the last save, so
 * an idle editor produces zero traffic.
 *
 * Auto-save does NOT touch UndoManager – Undo/Redo stays usable for

```

```

* the whole lifetime of the editor. Manual save (toolbar / Ctrl+S)
* persists and closes the editor via onSave.
*
* Auto-save persists via onAutoSave (editor stays open). If onAutoSave
* is not provided, falls back to onSave.
*
* If the session expires during auto-save, fetchJson emits
* auth:unauthorized and the editor is torn down by Base.
*
* Save-on-close (three-way):
* When the editor is closed with unsaved changes – either by the
* toolbar "x" / Escape, or externally (Base navigates to setup /
* profile) – a three-way confirm modal asks:
*
* Сохранить      → persist, then close (or proceed with navigation).
* Не сохранять   → close / proceed, dropping changes.
* Отмена         → do nothing, keep the editor open.
*
* Editor.confirmClose() is the entry point for external closes.
*
* Template mode:
* If props.templateHtml is provided:
* - Rendered into .editor-template (outside GrapesJS iframe).
* - If the template contains [data-slot="content"], the slot is
*   replaced with .editor-slot and GrapesJS mounts into it.
* - If the template has no slot, GrapesJS is NOT created at all –
*   the template is shown as a read-only preview. The toolbar is
*   still rendered (only the "Close" button).
*
* Device behavior (when the template has a slot):
* - desktop: template is visible, slot lives inside .editor-template.
* - tablet / mobile: template is hidden, slot is moved into
*   .editor-canvas so the user can test how the slot content fits
*   narrow viewports without the surrounding template layout.
*
* Otherwise (no template):
* - GrapesJS mounts into .editor-canvas – same as before.
*
* Exposes (to Word):
* - waitForInit()      – wait for readiness
* - isInitialized()    – check readiness
* - confirmClose()     – ask before external close
* - destroy()          – destroy editor
* - editor              – GrapesJS instance (or null in preview mode)
*
* Save / cancel – via onSave / onAutoSave / onCancel callbacks from props.
*/
export class Editor {
  constructor(container, props = {}) {
    console.log('[Editor] Constructor', { container, props });

    this.container = container;
    this.props = props;

    // Data to load
    this.initialHtml = props.html || '';

```

```

this.initialProject = props.project || null;
this.templateHtml = props.templateHtml || null;

// Callbacks
this.onSave = props.onSave || null;
this.onAutoSave = props.onAutoSave || null;
this.onCancel = props.onCancel || null;

// State
this.editor = null;
this._initialized = false;
this._initPromise = null;

// Auto-save – interval-based, dirty-flag driven.
//
// The interval just wakes up every AUTO_SAVE_INTERVAL_MS and
// checks `_dirty`. `_dirty` is set by GrapesJS `update` events
// (user-driven changes only – filtered below) and cleared after
// a successful save. No changes → no traffic.
this.AUTO_SAVE_ENABLED = true;
this.AUTO_SAVE_INTERVAL_MS = 30_000;
this._autoSaveTimer = null;
this._autoSaveInFlight = false;
this._autoSaveFailed = false;
this._dirty = false;

// Submodules (created in _init)
this._widgets = null;
this._stylesConfig = null;
this._grapes = null;
this._assets = null;
this._blocks = null;
this._resizer = null;
this._templateMgr = null; // template.js module
this._toolbarMgr = null; // toolbar.js module
this._dataLoader = null; // dataLoader.js module
this._createModal = null;
this._modals = null; // modals.js module
this._history = null; // history.js module
this._chat = null; // LLM chat
this._presets = null; // LLM presets
this._importer = null; // io/import.js module
this._exporter = null; // io/export.js module

// DOM elements (filled by widgets.build())
this.leftArea = null;
this.rightArea = null;
this.blocksEl = null;
this.presetsEl = null;
this.canvasEl = null; // whole center area
this.templateEl = null; // static template HTML (when template exists)
this.slotEl = null; // GrapesJS mount point (when template exists)
this.toolbarEl = null;
this.stylesEl = null;
this.traitsEl = null;
this.chatEl = null;

```

```

this.tabsEl = null;

// Page data for LLM chat / presets
this.pageId = props.pageId || null;
this.pageData = props.pageData || null;

// Media library API – with module_name for multi-site support.
const moduleName = window.coreEngine?.moduleName
  || document.body.dataset.module
  || '';
const qs = moduleName
  ? `?module=${encodeURIComponent(moduleName)}`
  : '';

this._assetsApi = `/core/engine/lib/word/assets${qs}`;
this._assetsUploadApi = `/core/engine/lib/word/assets/upload${qs}`;

// History API base
this._historyApi = `/core/engine/lib/word${qs}`;

// Scope class – must match GrapesLoader.scopeClass
this._scopeClass = 'core-engine-lib-word-blocks';

this._initPromise = this._init();
}

// =====
// INITIALIZATION
// =====

async _init() {
  console.log('[Editor] _init() START');
  try {
    const version = window.coreEngine?.static_version || Date.now();

    // 1. Load all submodules in parallel.
    // GrapesLoader now lives in grapes/index.js.
    const [
      { GrapesLoader },
      { WidgetsBuilder },
      { StylesConfig },
      { AssetsManager },
      { BlocksRegistry },
      { Resizer },
      { TemplateManager },
      { ToolbarManager },
      { DataLoader },
      modalsModule,
      { createModal },
      { LLMChat },
      { LLMPresets },
      { Importer },
      { Exporter },
    ] = await Promise.all([
      import(`./grapes/index.js?v=${version}`),
      import(`./widgets.js?v=${version}`),
    ]);
  }
}

```

```

import(`./styles.js?v=${version}`),
import(`./assets.js?v=${version}`),
import(`./blocks/index.js?v=${version}`),
import(`./resizer.js?v=${version}`),
import(`./template.js?v=${version}`),
import(`./toolbar.js?v=${version}`),
import(`./dataloader.js?v=${version}`),
import(`./modals.js?v=${version}`),
import(`../../base/modal/index.js?v=${version}`),
import(`../llm/chat.js?v=${version}`),
import(`../llm/presets.js?v=${version}`),
import(`./io/import.js?v=${version}`),
import(`./io/export.js?v=${version}`),
});

this._createModal = createModal;
this._modals = modalsModule;

// 2. Build DOM of three areas (left / center / right) + toolbar
this._widgets = new WidgetsBuilder(this);
this._widgets.build();

// 2.5. Template manager – render template HTML into .editor-template
this._templateMgr = new TemplateManager(this);
this._templateMgr.render();

// 2.6. Bind tabs [Styles|Traits|Blocks|Presets]
this._bindTabs();

// 2.7. Resizer handles
this._resizer = new Resizer(this);
this._resizer.build();

// 3. StyleManager config
this._stylesConfig = new StylesConfig(this);

// 4. GrapesJS init.
this._grapes = new GrapesLoader(this, {
  styleManagerSectors: this._stylesConfig.sectors(),
});
await this._grapes.load();
this.editor = this._grapes.init();

if (this.editor) {
  // ----- Everything that requires GrapesJS -----

  if (this.templateHtml) {
    this._templateMgr.setupSlotResize();
  }

  // 5. Block library
  this._blocks = new BlocksRegistry(this.editor);
  await this._blocks.register();

  // 6. Media library
  this._assets = new AssetsManager(this);

```



```

        await this._assets.load();

        // 7. LLM Chat
        this._chat = new LLMChat(this);
        await this._chat.init();

        // 8. LLM Presets
        this._presets = new LLMPresets(this);
        await this._presets.init();

        // 8.5. Import / export modules.
        //      Created before the data loader so that any
        //      post-load state (scope class, body traits) can
        //      be re-applied from the importer as well.
        this._importer = new Importer(this);
        this._exporter = new Exporter(this);

        // 9. Initial data.
        this._dataLoader = new DataLoader(this);
        this._dataLoader.loadInitial();

        // 9.5. Start tracking changes and auto-saving.
        this._bindAutoSave();
    } else {
        console.log('[Editor] Preview mode – GrapesJS not initialized, panels disabled');
    }

    // 10. Toolbar + hotkeys.
    this._toolbarMgr = new ToolbarManager(this);
    this._toolbarMgr.build();

    this._initialized = true;
    console.log('[Editor] _init() COMPLETE');
} catch (error) {
    console.error('[Editor] Init error:', error);
    this._initialized = false;
    throw error;
}
}

// =====
// AUTO-SAVE (interval + dirty flag)
// =====

/**
 * Subscribe to GrapesJS change events and start the auto-save timer.
 *
 * `update` fires on any user-driven change (component add/remove,
 * style, text). It also fires for internal things (selection, panel
 * tabs, layout) – but those don't matter here: we only set a flag,
 * and the actual save runs on the timer.
 */
_bindAutoSave() {
    if (!this.editor) return;

    if (!this.AUTO_SAVE_ENABLED) {

```

```

        console.log('[Editor] Auto-save disabled (AUTO_SAVE_ENABLED=false)');
        return;
    }

    // Any user-driven change sets the dirty flag. We do NOT save here –
    // just mark that there's something to save on the next tick.
    this.editor.on('update', () => {
        this._dirty = true;
    });

    this._startAutoSaveInterval();

    console.log(`[Editor] Auto-save enabled (interval ${this.AUTO_SAVE_INTERVAL_MS}ms)`);
}

/**
 * Start (or restart) the auto-save interval.
 */
_startAutoSaveInterval() {
    this._stopAutoSaveInterval();
    this._autoSaveTimer = setInterval(() => {
        if (!this._dirty) return; // nothing changed – skip
        if (this._autoSaveInFlight) return;
        this._autoSave();
    }, this.AUTO_SAVE_INTERVAL_MS);
}

/**
 * Stop the auto-save interval.
 */
_stopAutoSaveInterval() {
    if (this._autoSaveTimer) {
        clearInterval(this._autoSaveTimer);
        this._autoSaveTimer = null;
    }
}

/**
 * True if there are user-driven changes since the last save.
 *
 * Used by the save-on-close dialog. Note: this is stricter than
 * `_dirty` – UndoManager.hasUndo() reflects real history, while
 * `_dirty` is just "something happened since the last save".
 */
_hasUnsavedChanges() {
    const um = this.editor?.UndoManager;
    if (!um || typeof um.hasUndo !== 'function') {
        // No UndoManager – fall back to the dirty flag.
        return this._dirty;
    }
    return um.hasUndo() || this._dirty;
}

/**
 * Perform the auto-save.
 */

```

```

* Uses onSave if provided; otherwise falls back to onSave.
* The editor stays open either way (onAutoSave must not close it).
*
* Does NOT clear UndoManager – Undo/Redo must stay usable while
* the editor is open, even after an auto-save.
*/
async _autoSave() {
    if (!this.editor) return;

    const saveFn = this.onAutoSave || this.onSave;
    if (!saveFn) return;
    if (this._autoSaveInFlight) return;
    if (!this._dirty) return;

    this._autoSaveInFlight = true;

    // Notify UI – "Сохранение..."
    document.dispatchEvent(new CustomEvent('editor:autosave-pending'));

    try {
        const data = {
            html: this.editor.getHtml(),
            css: this.editor.getCss(),
            project: this.editor.getProjectData(),
        };

        await saveFn(data);
        console.log('[Editor] Auto-saved');

        // Clear dirty flag only after a successful save.
        this._dirty = false;

        // Reset failure flag on success.
        this._autoSaveFailed = false;

        // Notify UI – "Сохранено".
        document.dispatchEvent(new CustomEvent('editor:autosaved', {
            detail: { at: Date.now() }
        }));
    } catch (error) {
        console.error('[Editor] Auto-save error:', error);

        // On 401 – fetchJson already emitted auth:unauthorized.
        // Do not retry; the editor will be replaced by the login form.
        if (error?.status === 401) {
            console.warn('[Editor] Auto-save stopped (session expired)');
            return;
        }

        // On other errors – keep the dirty flag, mark as failed.
        // Next interval tick will retry.
        this._autoSaveFailed = true;
        document.dispatchEvent(new CustomEvent('editor:autosave-failed', {
            detail: { error }
        }));
    } finally {

```

```

        this._autoSaveInFlight = false;
    }
}

// =====
// TABS (Styles | Traits | Blocks | Presets)
// =====

_bindTabs() {
    if (!this.tabsEl) return;

    this.tabsEl.addEventListener('click', (e) => {
        const tab = e.target.closest('.core-engine-lib-word-editor-tab');
        if (!tab) return;

        const tabName = tab.dataset.tab;
        this._setTab(tabName);
    });
}

_setTab(name) {
    if (!this.tabsEl) return;

    this.tabsEl
        .querySelectorAll('.core-engine-lib-word-editor-tab')
        .forEach((btn) => {
            btn.classList.toggle('active', btn.dataset.tab === name);
        });

    const leftTop = this.leftArea?.querySelector('.core-engine-lib-word-editor-left-top');
    if (!leftTop) return;

    leftTop.querySelectorAll('[data-tab-panel]').forEach((panel) => {
        panel.hidden = panel.dataset.tabPanel !== name;
    });
}

// =====
// MODALS (delegated to modals.js)
// =====

async _openHtmlModal() {
    if (!this.editor) return;
    if (!this._modals) {
        console.warn('[Editor] modals module not loaded');
        return;
    }
    await this._modals.openHtmlCssModal({
        editor: this.editor,
        createModal: this._createModal,
    });
}

async _openCssModal() {
    if (!this.editor) return;
    if (!this._modals) {

```

```

        console.warn('[Editor] modals module not loaded');
        return;
    }
    await this._modals.openElementCssModal({
        editor: this.editor,
        createModal: this._createModal,
    });
}

/**
 * Open the page history modal (list, preview, rollback).
 */
async _openHistoryModal() {
    if (!this.editor) return;

    if (!this.pageId) {
        console.warn('[Editor] pageId not set – history unavailable');
        return;
    }

    try {
        const version = window.coreEngine?.static_version || Date.now();
        const mod = await import(`./history.js?v=${version}`);
        this._history = mod;

        await mod.openHistoryModal({
            editor: this.editor,
            createModal: this._createModal,
            pageId: this.pageId,
            qs: this._qsForHistory(),
            onRollback: (data) => this._applyRollback(data),
        });
    } catch (err) {
        console.error('[Editor] history modal error:', err);
    }
}

// =====
// IMPORT / EXPORT (delegated to io/*)
// =====

/**
 * Open the import dialog (.grp / .html / URL).
 * Delegates to io/import.js – no editor logic here.
 */
_openImportDialog() {
    if (!this._importer) {
        console.warn('[Editor] importer module not loaded');
        return;
    }
    this._importer.openDialog();
}

/**
 * Open the export dialog (HTML / .grp).
 * Delegates to io/export.js – no editor logic here.

```

```

    */
    _openExportDialog() {
        if (!this._exporter) {
            console.warn('[Editor] exporter module not loaded');
            return;
        }
        this._exporter.openDialog();
    }

    /**
     * Build ?module=<name> query string for history API.
     */
    _qsForHistory() {
        const moduleName = window.coreEngine?.moduleName
            || document.body.dataset.module
            || '';
        return moduleName
            ? `?module=${encodeURIComponent(moduleName)}`
            : '';
    }

    /**
     * Apply a rollback result to the editor.
     */
    _applyRollback(data) {
        this._dataLoader?.applyRollback(data);
    }

    // =====
    // SAVING / CLEAR / CANCEL
    // =====

    /**
     * Manual save – triggered by the toolbar / Ctrl+S.
     *
     * Stops the auto-save interval, persists via onSave (which also
     * closes the editor), and resets auto-save state.
     */
    async _handleSave() {
        if (!this.editor) {
            console.warn('[Editor] save: GrapesJS not initialized');
            return;
        }

        if (!this.onSave) {
            console.warn('[Editor] onSave not bound');
            return;
        }

        // Stop auto-save – the editor is about to close.
        this._stopAutoSaveInterval();
        this._dirty = false;

        console.log('[Editor] Saving...');

        const data = {

```

```

        html: this.editor.getHtml(),
        css: this.editor.getCss(),
        project: this.editor.getProjectData(),
    };

    try {
        await this.onSave(data);
        console.log('[Editor] Saved');

        // Reset auto-save state on manual save.
        this._autoSaveFailed = false;
    } catch (error) {
        console.error('[Editor] Save error:', error);

        // 401 – session expired. Redirect to login, preserving return URL.
        if (error?.status === 401) {
            this._redirectToLogin();
            return;
        }

        // Other errors – show a message modal (best-effort).
        this._showSaveError(error);
    }
}

/**
 * Clear the whole edited page.
 */
_handleClear() {
    console.log('[Editor] Clear page');

    if (!this.editor) return;

    try {
        this.editor.DomComponents.clear();
        this.editor.Css.clear();
        this.editor.select(null);

        console.log('[Editor] Page cleared');
    } catch (e) {
        console.warn('[Editor] clear failed:', e);
    }
}

/**
 * Redirect to login page with ?next=<current URL>.
 */
_redirectToLogin() {
    console.log('[Editor] Session expired – redirecting to login');

    const authRedirect = window.coreEngine?.authRedirect || '/login';
    const returnUrl = encodeURIComponent(window.location.href);
    const sep = authRedirect.includes('?') ? '&' : '?';

    window.location.href = `${authRedirect}${sep}next=${returnUrl}`;
}

```

```

/**
 * Show a small message modal with the save error text.
 */
async _showSaveError(error) {
  const message = error?.message || 'Не удалось сохранить';

  if (!this._createModal) {
    console.warn('[Editor] createModal not available – error not shown:', message);
    return;
  }

  try {
    const modal = await this._createModal('message');
    modal.open(message, 'Ошибка сохранения', 'Понятно');
    modal.setOnOk(() => modal.destroy());
  } catch (e) {
    console.warn('[Editor] failed to show save error modal:', e);
  }
}

/**
 * Cancel – triggered by the toolbar "x" / Escape.
 *
 * If there are unsaved changes, ask the user:
 * - "Сохранить" → run the manual save flow (persists + closes).
 * - "Не сохранять" → close via onCancel (dropping changes).
 * - "Отмена" → keep the editor open.
 *
 * If there are no unsaved changes – just close.
 */
async _handleCancel() {
  console.log('[Editor] Cancel');

  // No unsaved changes – close straight away.
  if (!this._hasUnsavedChanges()) {
    this._stopAutoSaveInterval();
    if (this.onCancel) this.onCancel();
    return;
  }

  // Ask the user what to do.
  const choice = await this._askSaveOnClose();

  if (choice === 'save') {
    // Save, then close (the toolbar x is "save and close").
    await this._handleSave();
    return;
  }

  if (choice === 'cancel') {
    // User cancelled – keep the editor open.
    return;
  }

  // choice === 'discard' – close without saving.

```



```

        this._stopAutoSaveInterval();
        if (this.onCancel) this.onCancel();
    }

    /**
     * Persist current editor state WITHOUT closing.
     *
     * Used by confirmClose() when the user picks "Сохранить" on an
     * external close (Base navigates away). The editor will be torn
     * down by Base right after – we just need the data on the server.
     *
     * Uses onAutoSave if set, otherwise onSave. Always resolves – a
     * failure is logged, but the navigation still proceeds (Base
     * cannot wait forever).
     */
    async saveForExternalClose() {
        if (!this.editor) return;

        const saveFn = this.onAutoSave || this.onSave;
        if (!saveFn) return;

        // Flush any pending auto-save state.
        this._stopAutoSaveInterval();

        const data = {
            html: this.editor.getHtml(),
            css: this.editor.getCss(),
            project: this.editor.getProjectData(),
        };

        try {
            await saveFn(data);
            this._dirty = false;
            this._autoSaveFailed = false;
            console.log('[Editor] Saved (before external close)');
        } catch (e) {
            console.error('[Editor] Save before external close failed:', e);
            // Do not throw – Base still needs to tear down.
        }
    }

    // =====
    // PUBLIC – EXTERNAL CLOSE
    // =====

    /**
     * Ask the user what to do with unsaved changes before the editor
     * is torn down externally (e.g. Base navigates to setup / profile).
     *
     * Returns:
     *   'save'      – user chose to save. Changes already persisted.
     *   'discard'   – user chose to close without saving.
     *   'cancel'    – user cancelled – keep the editor open.
     *
     * If there are no unsaved changes – returns 'discard' immediately
     * (nothing to lose, no dialog).

```

```

*/
async confirmClose() {
  if (!this._hasUnsavedChanges()) {
    return 'discard';
  }

  const choice = await this._askSaveOnClose();

  if (choice === 'save') {
    await this.saveForExternalClose();
  }

  return choice;
}

/**
 * Show a three-way dialog on close with unsaved changes.
 *
 * Returns: 'save' | 'discard' | 'cancel'.
 *
 * Сохранить      → 'save'      (persists, editor stays open)
 * Не сохранять   → 'discard' (close, drop changes)
 * Отмена        → 'cancel'   (do nothing, keep editor open)
 *
 * Uses the shared createModal('confirm') with three buttons.
 * Falls back to window.confirm() – 2-way (save / discard) – if
 * createModal is unavailable.
 */
async _askSaveOnClose() {
  if (!this._createModal) {
    const ok = window.confirm(
      'Есть несохранённые изменения. Сохранить перед закрытием?'
    );
    return ok ? 'save' : 'discard';
  }

  try {
    const modal = await this._createModal('confirm');

    return await new Promise((resolve) => {
      let settled = false;
      const settle = (v) => {
        if (settled) return;
        settled = true;
        resolve(v);
      };

      modal.open(
        'Есть несохранённые изменения. Сохранить перед закрытием?',
        'Закрытие редактора',
        'Сохранить',      // ok-btn
        'Отмена',        // cancel-btn
        'Не сохранять'   // no-btn (enables 3-button mode)
      );

      modal.setOnOk(() => { modal.destroy(); settle('save'); });
    });
  }
}

```

```

        modal.setOnCancel(() => { modal.destroy(); settle('cancel'); });
        modal.setOnNo(() => { modal.destroy(); settle('discard'); });
    });
} catch (e) {
    console.warn('[Editor] _askSaveOnClose modal failed:', e);
    const ok = window.confirm(
        'Есть несохранённые изменения. Сохранить перед закрытием?'
    );
    return ok ? 'save' : 'discard';
}
}

// =====
// PUBLIC METHODS
// =====

isInitialized() {
    return this._initialized;
}

async waitForInit() {
    if (this._initPromise) await this._initPromise;
    return this._initialized;
}

destroy() {
    console.log('[Editor] destroy()');

    // Stop auto-save.
    this._stopAutoSaveInterval();

    // Toolbar manager – detach global keydown
    if (this._toolbarMgr) {
        try { this._toolbarMgr.destroy(); } catch (e) { console.warn(e); }
        this._toolbarMgr = null;
    }

    // Template manager
    if (this._templateMgr) {
        try { this._templateMgr.destroy(); } catch (e) { console.warn(e); }
        this._templateMgr = null;
    }

    // Chat + presets
    if (this._chat) {
        try { this._chat.destroy(); } catch (e) { console.warn(e); }
        this._chat = null;
    }
    if (this._presets) {
        try { this._presets.destroy(); } catch (e) { console.warn(e); }
        this._presets = null;
    }

    // Import / export modules – remove any open dialogs
    if (this._importer) {
        try { this._importer.destroy(); } catch (e) { console.warn(e); }

```

```

        this._importer = null;
    }
    if (this._exporter) {
        try { this._exporter.destroy(); } catch (e) { console.warn(e); }
        this._exporter = null;
    }

    // Resizer
    if (this._resizer) {
        try {
            this._resizer.destroy();
        } catch (e) {
            console.warn('[Editor] resizer.destroy() error:', e);
        }
        this._resizer = null;
    }

    // Detach frame:load handler inside GrapesLoader
    if (this._grapes) {
        try {
            this._grapes.destroy(this.editor);
        } catch (e) {
            console.warn('[Editor] grapes.destroy() error:', e);
        }
    }

    // Destroy GrapesJS instance
    if (this.editor) {
        try {
            this.editor.destroy();
        } catch (e) {
            console.warn('[Editor] editor.destroy() error:', e);
        }
        this.editor = null;
    }

    // Clear DOM areas (in case GrapesJS didn't clean up)
    if (this.leftArea) this.leftArea.innerHTML = '';
    if (this.rightArea) this.rightArea.innerHTML = '';
    if (this.container) this.container.innerHTML = '';

    this._widgets = null;
    this._stylesConfig = null;
    this._grapes = null;
    this._assets = null;
    this._blocks = null;
    this._dataLoader = null;
    this._createModal = null;
    this._modals = null;
    this._history = null;

    this._initialized = false;
    this._initPromise = null;
}
}

```

```

=====
FILE: core/engine/lib/word/editor/index.js
=====

// app/core/engine/lib/word/editor/index.js

/**
 * GrapesJS editor entry point.
 *
 * Dynamically loads the Editor class from editor.js with a version query,
 * so the browser does not serve a stale cached copy after updates.
 *
 * Usage (from word.js):
 *
 *   const version = window.coreEngine?.static_version || Date.now();
 *   const { loadEditor } = await import(`./editor/index.js?v=${version}`);
 *   const Editor = await loadEditor();
 *
 *   this.editorInstance = new Editor(container, props);
 */

export async function loadEditor() {
  const version = window.coreEngine?.static_version || Date.now();
  const { Editor } = await import(`./editor.js?v=${version}`);
  return Editor;
}

```

```

=====
FILE: core/engine/lib/word/editor/formatter.js
=====

// app/core/engine/lib/word/editor/formatter.js

/**
 * Formatter – CSS / HTML pretty-print utilities for the editor.
 *
 * Uses js-beautify (loaded lazily from /static/libs/js-beautify-1.15.1/).
 * Falls back to raw strings if the library is unavailable.
 */

const BEAUTIFY_BASE = '/static/libs/js-beautify-1.15.1';

let _loaded = false;
let _loading = null;

/**
 * Load js-beautify CSS + HTML scripts once per session.
 *
 * @returns {Promise<void>}
 */
export async function loadBeautify() {
  if (_loaded) return;
  if (_loading) return _loading;

  const version = window.coreEngine?.static_version || Date.now();

  const loadScript = (src) => new Promise((resolve, reject) => {
    const existing = document.querySelector(`script[src="${src}"]`);

```

```

    if (existing) {
      if (existing.dataset.loaded === '1') return resolve();
      existing.addEventListener('load', resolve);
      existing.addEventListener('error', reject);
      return;
    }
    const s = document.createElement('script');
    s.src = src;
    s.onload = () => { s.dataset.loaded = '1'; resolve(); };
    s.onerror = reject;
    document.head.appendChild(s);
  });

  _loading = Promise.all([
    loadScript(`${BEAUTIFY_BASE}/beautify-css.min.js?v=${version}`),
    loadScript(`${BEAUTIFY_BASE}/beautify-html.min.js?v=${version}`),
  ]).then(() => {
    _loaded = true;
    console.log('[Formatter] js-beautify loaded');
  }).catch((e) => {
    console.warn('[Formatter] js-beautify load failed:', e);
  });

  return _loading;
}

/**
 * Pretty-print CSS.
 * Falls back to raw string if js-beautify is not available.
 *
 * @param {string} css
 * @returns {string}
 */
export function formatCss(css) {
  if (!css) return '';

  if (typeof window.css_beautify === 'function') {
    try {
      return window.css_beautify(css, {
        indent_size: 4,
        selector_separator_newline: true,
        newline_between_rules: true,
      });
    } catch (e) {
      console.warn('[Formatter] css_beautify error:', e);
    }
  }

  // Fallback: minimal formatter
  return css
    .replace(/\s*{\s*/g, ' {\n  ')
    .replace(/;\s*/g, ';\n  ')
    .replace(/\s*}\s*/g, '\n}\n')
    .replace(/\n\s*\n/g, '\n')
    .trim();
}

```

```

/**
 * Pretty-print HTML.
 * Falls back to raw string if js-beautify is not available.
 *
 * @param {string} html
 * @returns {string}
 */
export function formatHtml(html) {
  if (!html) return '';

  if (typeof window.html_beautify === 'function') {
    try {
      return window.html_beautify(html, {
        indent_size: 4,
        wrap_line_length: 0,
        preserve_newlines: true,
        max_preserve_newlines: 2,
      });
    } catch (e) {
      console.warn('[Formatter] html_beautify error:', e);
    }
  }

  // Fallback: minimal formatter
  const VOID = /^<(br|hr|img|input|meta|link|area|base|col|embed|source|track|wbr)\b/i;
  const tokens = html.split(/(<[>]+>)/g).filter(Boolean);

  let out = '';
  let indent = 0;

  for (const token of tokens) {
    if (token.startsWith('</')) {
      indent = Math.max(0, indent - 1);
      out += ' '.repeat(indent) + token + '\n';
    } else if (token.startsWith('<!--')) {
      out += ' '.repeat(indent) + token + '\n';
    } else if (token.startsWith('<')) {
      out += ' '.repeat(indent) + token + '\n';
      if (!token.endsWith('/>') && !VOID.test(token)) {
        indent++;
      }
    } else {
      const text = token.trim();
      if (text) {
        out += ' '.repeat(indent) + text + '\n';
      }
    }
  }

  return out.trim();
}

```

```

=====
FILE: core/engine/lib/word/editor/history.js
=====

```

```
// neurocad/core/engine/lib/word/editor/history.js

/**
 * History – page history modal (list, preview, rollback, delete, clear).
 *
 * Public API:
 *   openHistoryModal({ editor, createModal, pageId, qs, onRollback })
 *
 * Behavior:
 *   1. Fetch list of snapshots (metadata only) – GET /word/{id}/history
 *   2. Render list; first item selected by default.
 *   3. On click – fetch full snapshot – GET /word/{id}/history/{hist_id}
 *      Render preview in <iframe srcdoc="...">, wrapped in the scope class
 *      .core-engine-lib-word-blocks and with content.css + block CSS
 *      linked inside the iframe (isolation from editor styles).
 *      Snapshot CSS (data.css) is injected as <style> in the iframe head.
 *      For legacy snapshots without data.css, CSS is extracted from
 *      <style> tags inside data.html.
 *   4. On "Откатить" – confirm dialog, then POST /word/{id}/rollback/{hist_id}.
 *      On success – call onRollback(data) and close the modal.
 *   5. On "Удалить" – confirm dialog, then DELETE /word/{id}/history/{hist_id}.
 *      Removes ONE snapshot from history. Reloads the list.
 *   6. On "Очистить" – confirm dialog, then DELETE /word/{id}/history.
 *      Removes ALL snapshots of the page. Reloads the list (empty).
 *
 * CSS is loaded from ./history.css on module load.
 *
 * HTTP requests go through window.coreEngine.fetchJson
 * (loaded once by CoreEngine.loadApi()).
 *
 * The modal DOM follows Base Modal conventions:
 *   .core-engine-lib-word-editor-history
 *     └─ .window
 *         └─ .title-bar      (title + close button)
 *         └─ .content
 *             └─ .history-body
 *                 └─ .history-list    (left)
 *                 └─ .history-preview (right, iframe)
 *             └─ .actions-bar    (delete + clear + cancel + rollback)
 */

/* =====
CSS LOAD (once per session)
===== */

let _cssLoaded = false;

function _loadCss() {
  if (_cssLoaded) return;
  _cssLoaded = true;

  const version = window.coreEngine?.static_version || Date.now();
  const href = `/static/core/engine/lib/word/editor/css/history.css?v=${version}`;

  if (document.querySelector(`link[href="${href}"]`)) return;

```



```

    const link = document.createElement('link');
    link.rel = 'stylesheet';
    link.href = href;
    document.head.appendChild(link);
}

/* =====
API HELPERS
===== */

/**
 * Build a path WITHOUT query string.
 * _apiPath(3, '/history') → '/core/engine/lib/word/3/history'
 * _apiPath(3, '/history/42') → '/core/engine/lib/word/3/history/42'
 * _apiPath(3, '/rollback/42') → '/core/engine/lib/word/3/rollback/42'
 */
function _apiPath(pageId, suffix = '') {
    return `core/engine/lib/word/${pageId}${suffix}`;
}

/**
 * Append a query string (already starting with '?') to a URL.
 * _withQs('/word/3/history', '?module=default')
 * → '/word/3/history?module=default'
 * _withQs('/word/3/history', '')
 * → '/word/3/history'
 */
function _withQs(url, qs) {
    return qs ? `${url}${qs}` : url;
}

/* =====
PUBLIC API
===== */

/**
 * Open the history modal.
 *
 * @param {Object} opts
 * @param {Object} opts.editor – GrapesJS instance
 * @param {Function} opts.createModal – Base modal factory
 * @param {number} opts.pageId – current page id
 * @param {string} [opts.qs] – ?module=<name>
 * @param {Function} [opts.onRollback] – called after successful rollback with data
 */
export async function openHistoryModal({ editor, createModal, pageId, qs = '', onRollback }) {
    _loadCss();

    if (!pageId) {
        console.warn('[History] pageId is required');
        return;
    }

    const modal = new HistoryModal({ editor, createModal, pageId, qs, onRollback });
    await modal.open();
}

```

```

/* =====
HISTORY MODAL CLASS
===== */

class HistoryModal {
  constructor({ editor, createModal, pageId, qs, onRollback }) {
    this.editor = editor;
    this.createModal = createModal;
    this.pageId = pageId;
    this.qs = qs;
    this.onRollback = onRollback;

    this.container = null;
    this.listEl = null;
    this.previewEl = null;
    this.deleteBtn = null;
    this.clearBtn = null;
    this.rollbackBtn = null;
    this.statusEl = null;

    this.items = [];          // list of { id, action, note, created_at }
    this.selectedId = null;   // currently selected snapshot id
    this.previewData = null;  // full snapshot of selected item
    this._escHandler = null;
  }

  async open() {
    this._createDOM();
    this._bindEvents();
    this.container.classList.add('active');

    await this._loadList();
  }

  // -----
  // DOM
  // -----

  _createDOM() {
    // Reuse existing if present
    const existing = document.querySelector('.core-engine-lib-word-editor-history');
    if (existing) {
      this.container = existing;
      this._cacheElements();
      this._clearState();
      return;
    }

    const container = document.createElement('div');
    container.className = 'core-engine-lib-word-editor-history';
    container.innerHTML = `
      <div class="window">
        <div class="title-bar">
          <div class="title-bar-text">История изменений</div>
          <div class="title-bar-controls">

```

```

        <span class="close-btn">x</span>
      </div>
    </div>
    <div class="content">
      <div class="history-body">
        <aside class="history-list" data-role="list">
          <div class="history-list-empty">Загрузка...</div>
        </aside>
        <section class="history-preview" data-role="preview">
          <div class="history-preview-empty">Выберите версию слева</div>
        </section>
      </div>
      <div class="history-status" data-role="status"></div>
      <div class="actions-bar">
        <button class="delete-btn" type="button" disabled>Удалить</button>
        <button class="clear-btn" type="button" disabled>Очистить</button>
        <div class="actions-bar-spacer"></div>
        <button class="cancel-btn" type="button">Закрыть</button>
        <button class="ok-btn" type="button" disabled>Откатить</button>
      </div>
    </div>
  </div>
  `;
  document.body.appendChild(container);
  this.container = container;
  this._cacheElements();
}

_cacheElements() {
  this.listEl = this.container.querySelector('[data-role="list"]');
  this.previewEl = this.container.querySelector('[data-role="preview"]');
  this.statusEl = this.container.querySelector('[data-role="status"]');
  this.cancelBtn = this.container.querySelector('.cancel-btn');
  this.deleteBtn = this.container.querySelector('.delete-btn');
  this.clearBtn = this.container.querySelector('.clear-btn');
  this.rollbackBtn = this.container.querySelector('.ok-btn');
  this.closeBtn = this.container.querySelector('.close-btn');
}

_clearState() {
  this.items = [];
  this.selectedId = null;
  this.previewData = null;
  if (this.listEl) this.listEl.innerHTML = '<div class="history-list-empty">Загрузка...</div>';
  if (this.previewEl) this.previewEl.innerHTML = '<div class="history-preview-empty">Выберите версию слева</div>';
  if (this.statusEl) this.statusEl.textContent = '';
  if (this.rollbackBtn) this.rollbackBtn.disabled = true;
  if (this.deleteBtn) this.deleteBtn.disabled = true;
  if (this.clearBtn) this.clearBtn.disabled = true;
}

_bindEvents() {
  this.closeBtn.addEventListener('click', () => this.destroy());
  this.cancelBtn.addEventListener('click', () => this.destroy());

  this.rollbackBtn.addEventListener('click', () => this._handleRollback());
}

```

```

this.deleteBtn.addEventListener('click', () => this._handleDelete());
this.clearBtn.addEventListener('click', () => this._handleClear());

// Click on overlay (outside .window) – close
this.container.addEventListener('click', (e) => {
  if (e.target === this.container) this.destroy();
});

// Escape – close
this._escHandler = (e) => {
  if (e.key === 'Escape' && this.container.classList.contains('active')) {
    this.destroy();
  }
};
document.addEventListener('keydown', this._escHandler);

// Delegated click on list items
this.listEl.addEventListener('click', (e) => {
  const item = e.target.closest('[data-hist-id]');
  if (!item) return;
  const id = parseInt(item.dataset.histId, 10);
  if (!Number.isFinite(id)) return;
  this._selectItem(id);
});
}

// -----
// DATA – LIST
// -----

async _loadList() {
  try {
    const url = _withQs(_apiPath(this.pageId, '/history'), this.qs);
    const fetchJson = window.coreEngine?.fetchJson;
    const json = await fetchJson(url);
    this.items = json.data || [];

    this._renderList();
    this._updateActionButtons();

    // Auto-select first (newest) item
    if (this.items.length > 0) {
      this._selectItem(this.items[0].id);
    } else {
      this._setStatus('История пуста. Сохраните страницу – появится снимок.');
```

```

        this.listEl.innerHTML = '<div class="history-list-empty">Снимков нет</div>';
        return;
    }

    this.listEl.innerHTML = '';

    this.items.forEach((item) => {
        const row = document.createElement('div');
        row.className = 'history-list-item';
        row.dataset.histId = String(item.id);

        const actionLabel = this._actionLabel(item.action);
        const dateText = this._formatDate(item.created_at);

        row.innerHTML = `
            <div class="history-item-action">${actionLabel}</div>
            <div class="history-item-date">${dateText}</div>
            ${item.note ? `<div class="history-item-note">${this._escape(item.note)}</div>` : ''}
        `;

        this.listEl.appendChild(row);
    });
}

_actionLabel(action) {
    switch (action) {
        case 'user_edit':    return 'Изменение';
        case 'ai_edit':      return 'AI-редактирование';
        case 'preset_apply': return 'Применён пресет';
        case 'rollback':     return 'Откат';
        default:             return action || '-';
    }
}

_formatDate(iso) {
    if (!iso) return '';
    try {
        const d = new Date(iso);
        if (isNaN(d.getTime())) return iso;

        const pad = (n) => String(n).padStart(2, '0');
        return `${pad(d.getDate())}.${pad(d.getMonth() + 1)}${d.getFullYear()} `
            + `${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`;
    } catch (e) {
        return iso;
    }
}

_escape(str) {
    return String(str)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;');
}

```

```

/**
 * Update enabled/disabled state of action buttons based on current
 * list contents and selection.
 *
 * Очистить – enabled only when the list is non-empty.
 * Удалить – enabled only when an item is selected.
 * Откатить – enabled only when an item is selected (set elsewhere).
 */
_updateActionButtons() {
  const hasItems = this.items.length > 0;
  if (this.clearBtn) this.clearBtn.disabled = !hasItems;
  if (this.deleteBtn) this.deleteBtn.disabled = !this.selectedId;
}

// -----
// DATA – PREVIEW
// -----

async _selectItem(histId) {
  if (this.selectedId === histId && this.previewData) {
    return;
  }

  this.selectedId = histId;
  this.previewData = null;
  this.rollbackBtn.disabled = true;
  this._updateActionButtons();

  // Highlight active row
  this.listEl.querySelectorAll('[data-hist-id]').forEach((el) => {
    el.classList.toggle('active', parseInt(el.dataset.histId, 10) === histId);
  });

  this.previewEl.innerHTML = '<div class="history-preview-empty">Загрузка превью...</div>';
  this._setStatus('');

  try {
    const url = _withQs(_apiPath(this.pageId, `/history/${histId}`), this.qs);
    const fetchJson = window.coreEngine?.fetchJson;
    const json = await fetchJson(url);
    this.previewData = json.data;

    this._renderPreview(this.previewData);
    this.rollbackBtn.disabled = false;
    this._updateActionButtons();
  } catch (err) {
    console.error('[History] preview error:', err);
    this.previewEl.innerHTML = '<div class="history-preview-empty">Ошибка: ${this._escape(err.message)}</div>`;
    this._setStatus(`Не удалось загрузить превью: ${err.message}`);
  }
}

/**
 * Render preview inside an isolated <iframe srcdoc="...">.
 */
_renderPreview(data) {

```

```

const rawHtml = data.html || '<p style="color:#94a3b8;">Пустой снимок</p>';
const snapshotCss = (data.css || '').trim();

let htmlWithoutStyles = rawHtml;
let css = snapshotCss;

// Legacy path: CSS embedded in html as <style> tags
if (!css) {
    const styleMatches = [...rawHtml.matchAll(/<style[^>]*>([\s\S]*?)</style>/gi)];
    css = styleMatches.map(m => m[1]).join('\n').trim();
    htmlWithoutStyles = rawHtml.replace(/<style[^>]*>[\s\S]*?</style>/gi, '');
}

const version = window.coreEngine?.static_version || Date.now();
const cssBase = '/static/core/engine/lib/word/editor/css';
const blocksBase = '/static/core/engine/lib/word/editor/blocks';

// Same CSS set the canvas iframe uses
const cssLinks = [
    `${cssBase}/content.css?v=${version}`,
    `${blocksBase}/elements.css?v=${version}`,
    `${blocksBase}/layout.css?v=${version}`,
    `${blocksBase}/ready.css?v=${version}`,
    `${blocksBase}/aleksmir.ru.css?v=${version}`,
].map((href) => `<link rel="stylesheet" href="${href}">`).join('\n');

// Snapshot CSS as inline <style> in iframe head – it must come
// AFTER block CSS so page rules override block defaults.
const snapshotStyle = css
    ? `<style data-source="snapshot">${css}</style>`
    : '';

const srcdoc = `<!DOCTYPE html>

<html>
<head>
<meta charset="utf-8">
<base href="/">
${cssLinks}
${snapshotStyle}
<style>
    html, body { margin: 0; padding: 0; }
    body { padding: 12px; font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif; }
</style>
</head>
<body>
<div class="core-engine-lib-word-blocks">${htmlWithoutStyles}</div>
</body>
</html>`;

this.previewEl.innerHTML = '';

const iframe = document.createElement('iframe');
iframe.className = 'history-preview-frame';
iframe.setAttribute('sandbox', 'allow-same-origin allow-scripts');
iframe.srcdoc = srcdoc;

```

```

    this.previewEl.appendChild(iframe);
  }

  // -----
  // ROLLBACK
  // -----

  async _handleRollback() {
    if (!this.selectedId || !this.previewData) return;

    const item = this.items.find((i) => i.id === this.selectedId);
    const dateText = item ? this._formatDate(item.created_at) : '';

    // Confirm dialog
    const confirmed = await this._confirm(
      `Откатить страницу к версии от ${dateText}? Текущее состояние будет сохранено в истории.`,
      'Подтверждение отката',
      'Откатить',
    );
    if (!confirmed) return;

    this._setStatus('Откат...');
    this.rollbackBtn.disabled = true;

    try {
      const url = _withQs(_apiPath(this.pageId, `/rollback/${this.selectedId}`), this.qs);
      const fetchJson = window.coreEngine?.fetchJson;
      const json = await fetchJson(url, { method: 'POST' });
      const data = json.data;

      this._setStatus('Откат выполнен');

      if (typeof this.onRollback === 'function') {
        try {
          this.onRollback(data);
        } catch (e) {
          console.error('[History] onRollback callback error:', e);
        }
      }

      // Close modal shortly after – let user see the "done" status
      setTimeout(() => this.destroy(), 400);
    } catch (err) {
      console.error('[History] rollback error:', err);
      this._setStatus(`Ошибка отката: ${err.message}`);
      this.rollbackBtn.disabled = false;
    }
  }

  // -----
  // DELETE – one snapshot
  // -----

  async _handleDelete() {
    if (!this.selectedId) return;

```



```

const item = this.items.find((i) => i.id === this.selectedId);
const dateText = item ? this._formatDate(item.created_at) : '';

const confirmed = await this._confirm(
  `Удалить выбранный снимок от ${dateText}? Это действие необратимо.`,
  'Удаление снимка',
  'Удалить',
);
if (!confirmed) return;

this._setStatus('Удаление...');
this.deleteBtn.disabled = true;

try {
  const url = _withQs(
    _apiPath(this.pageId, `/history/${this.selectedId}`),
    this.qs,
  );
  const fetchJson = window.coreEngine?.fetchJson;
  const json = await fetchJson(url, { method: 'DELETE' });

  if (!json || !json.success) {
    throw new Error((json && json.error) || 'Не удалось удалить');
  }

  this._setStatus('Снимок удалён');

  // Clear current selection, reload list
  this.selectedId = null;
  this.previewData = null;
  this._clearPreview();
  await this._loadList();
} catch (err) {
  console.error('[History] delete error:', err);
  this._setStatus(`Ошибка удаления: ${err.message}`);
  this.deleteBtn.disabled = false;
}
}

// -----
// CLEAR – all snapshots
// -----

async _handleClear() {
  if (this.items.length === 0) return;

  const confirmed = await this._confirm(
    `Очистить всю историю изменений страницы (${this.items.length} снимков)? Это действие необратимо.`,
    'Очистка истории',
    'Очистить',
  );
  if (!confirmed) return;

  this._setStatus('Очистка...');
  this.clearBtn.disabled = true;
  this.deleteBtn.disabled = true;
}

```

```

this.rollbackBtn.disabled = true;

try {
  const url = _withQs(_apiPath(this.pageId, '/history'), this.qs);
  const fetchJson = window.coreEngine?.fetchJson;
  const json = await fetchJson(url, { method: 'DELETE' });

  if (!json || !json.success) {
    throw new Error((json && json.error) || 'Не удалось очистить');
  }

  this._setStatus('История очищена');

  // Reset state and reload (list will be empty)
  this.selectedId = null;
  this.previewData = null;
  this._clearPreview();
  await this._loadList();
} catch (err) {
  console.error('[History] clear error:', err);
  this._setStatus(`Ошибка очистки: ${err.message}`);
  this._updateActionButton();
}
}

_clearPreview() {
  if (this.previewEl) {
    this.previewEl.innerHTML = '<div class="history-preview-empty">Выберите версию слева</div>';
  }
}

// -----
// CONFIRM DIALOG
// -----

/**
 * Show a confirm dialog via Base Modal Confirm.
 * Returns a Promise<boolean>.
 *
 * @param {string} message – text of the dialog
 * @param {string} title – dialog title
 * @param {string} okLabel – label of the OK button
 */
_confirm(message, title = 'Подтверждение', okLabel = 'OK') {
  return new Promise(async (resolve) => {
    if (!this.createModal) {
      resolve(true);
      return;
    }

    try {
      const modal = await this.createModal('confirm');
      modal.open(message, title, okLabel, 'Отмена');

      let resolved = false;
      const done = (result) => {

```

```

        if (resolved) return;
        resolved = true;
        modal.destroy();
        resolve(result);
    };

    modal.setOnOk(() => done(true));
    modal.setOnCancel(() => done(false));
} catch (e) {
    console.warn('[History] confirm modal error:', e);
    resolve(true);
}
});
}

// -----
// MISC
// -----

_setStatus(text) {
    if (this.statusEl) this.statusEl.textContent = text || '';
}

destroy() {
    if (this._escHandler) {
        document.removeEventListener('keydown', this._escHandler);
        this._escHandler = null;
    }
    if (this.container) {
        this.container.classList.remove('active');
        this.container.remove();
        this.container = null;
    }
    this.previewData = null;
    this.items = [];
    this.selectedId = null;
}
}

```

```

=====
FILE: core/engine/lib/word/editor/grapes.js
=====

```

```

// app/core/engine/lib/word/editor/grapes.js

```

```

/**
 * GrapesLoader – load and init GrapesJS.
 *
 * Tasks:
 * 1. Load GrapesJS CSS/JS from /static/libs/grapesjs-0.21.13/.
 * 2. Load OUR CSS files (editor UI) – one <link> each.
 *    All are loaded AFTER grapes.min.css so they override.
 * 3. Load grapesjs-blocks-basic plugin.
 * 4. Inside canvas (GrapesJS iframe) load, in order:
 *     canvas.css      – canvas layout baseline
 *     content.css     – shared atoms (.btn, .card, .grid, ...)
 *     AND --theme-* variables

```

```

*     effects.css      – reusable effect classes (.effect-pulse, ...)
*     blocks/*.css     – block-specific classes (.hero, .flex-shell, ...)
*     All are scoped under .core-engine-lib-word-blocks.
*     Without them, block content looks "naked" in the editor.
*     List of block CSS is taken from blocks/manifest.js (dynamic import).
* 5. Build config and call grapesjs.init().
* 6. After init – reset inline styles on blocks set by GrapesJS.
* 7. Register 'link' component type with traits (href, target) –
*     so links can be set on buttons and other elements.
* 8. Add .core-engine-lib-word-blocks class to iframe body – single
*     scope wrapper for the whole page content.
* 9. Register body layout traits (Свойства panel) – height, display,
*     flex-direction, padding, margin, overflow for the wrapper (body).
* 10. Auto-remove empty <p> placeholder when a real block is added –
*     GrapesJS inserts <p></p> for empty canvas; it takes vertical
*     space and breaks full-height layouts.
* 11. Filter the context menu (right-click on a component): keep only
*     working items, replace the broken built-in "Link" with our own
*     command that wraps the selected component in <a> and selects it
*     (so the traits panel with href/target opens).
* 12. Filter UndoManager: record only real changes (add/remove/style/
*     rte exit), skip selection-only updates.
*
* Block categories (collapse behaviour):
* Categories are pre-created in BlocksRegistry.register() with the
* model flag `open: false` BEFORE any block is added. GrapesJS uses
* this flag when it renders the category DOM and when it re-renders
* after new blocks are added – so the state survives re-renders.
* No DOM patching here: click-to-toggle is standard GrapesJS and we
* do not interfere.
*
* Container selection:
* - If a template is set (editor.templateHtml), GrapesJS mounts ONLY
*   when the rendered template contains [data-slot="content"].
*   In that case the slot is replaced by editor.slotEl and GrapesJS
*   mounts into it; the rest of the template stays outside the iframe.
* - If a template is set but has no slot – GrapesJS is NOT created
*   at all. The template is shown as a read-only preview (no canvas,
*   no editor panels acting on it).
* - Otherwise (no template) – GrapesJS mounts into editor.canvasEl.
*
* Canvas size:
* width: '100%' – GrapesJS fills the whole container width.
* height: '100%' – fills the whole container height.
* Device 'desktop' has width: '' (auto), so the iframe takes the
* full container width when width: '100%' is set here.
*
* No editor logic here – only "bootstrap and run".
* Everything DOM-related is done by WidgetsBuilder.
*/
export class GrapesLoader {
  /**
   * @param {Editor} editor – parent Editor (we take DOM refs and API from it)
   * @param {Object} options – { styleManagerSectors: Array }
   */
  constructor(editor, options = {}) {

```

```

this.editor = editor;
this.options = options;

this.baseUrl = '/static/libs/grapesjs-0.21.13';

// Our CSS for editor UI (main document)
this.cssBase = '/static/core/engine/lib/word/editor/css';
this.cssFiles = [
    'layout.css',
    'toolbar.css',
    'blocks.css',
    'styles.css',
    'assets.css',
    'resizer.css',
    'theme.css',
    'responsive.css',
];

// Our CSS for iframe canvas.
// ORDER MATTERS: each file overrides the previous one at equal
// specificity. Block-specific rules (blocks/*.css) come last,
// so they can override shared classes (content.css).
//
// content.css – shared atoms AND --theme-* variables. Must come
// first, so blocks and effects can use the theme vars.
// effects.css – reusable effect classes (.effect-*). After
// content.css, before blocks/*.css (blocks may override).
//
// Block CSS list is appended in load() – from the dynamic
// import of blocks/manifest.js (versioned import).
this.cssCanvasBase = '/static/core/engine/lib/word/editor/css';
this.effectsBase = '/static/core/engine/lib/word/editor/effects';
this.blocksBase = '/static/core/engine/lib/word/editor/blocks';

this.canvasCss = [
    `${this.cssCanvasBase}/canvas.css`,
    `${this.cssCanvasBase}/content.css`,
    `${this.effectsBase}/effects.css`,
    // block CSS appended in load()
];

// Scope class – single wrapper for the whole page content.
// Added to iframe body on init and on every frame reload
// (device change re-creates the iframe).
this.scopeClass = 'core-engine-lib-word-blocks';

// Keep reference to the frame:load handler so we can detach on destroy.
this._frameLoadHandler = null;

// Keep reference to the empty <p> cleanup handler so we can detach
// on destroy.
this._emptyPHandler = null;

// Keep reference to the undo filter handler so we can detach on destroy.
this._undoFilterHandler = null;
}

```

```

// =====
// LOAD RESOURCES
// =====

/**
 * Load all external resources.
 *
 * ORDER:
 * 1. grapes.min.css – base GrapesJS styles.
 * 2. Our CSS (one <link> each) – AFTER, so they override.
 * 3. grapes.min.js
 * 4. grapesjs-blocks-basic.min.js
 *
 * Also: dynamically import blocks/manifest.js (versioned), and
 * append block CSS paths to this.canvasCss – so init() picks them up.
 */
async load() {
  console.log('[GrapesLoader] Loading resources');

  const version = window.coreEngine?.static_version || Date.now();

  // 1. GrapesJS CSS – base
  this._loadStylesheet(`${this.baseUrl}/grapes.min.css`);

  // 2. Our CSS – each file as a separate <link>.
  this.cssFiles.forEach((file) => {
    const href = `${this.cssBase}/${file}?v=${version}`;
    this._loadStylesheet(href);
  });

  // 3. GrapesJS JS
  await this._loadScript(`${this.baseUrl}/grapes.min.js`);

  // 4. grapesjs-blocks-basic JS
  await this._loadScript(`${this.baseUrl}/grapesjs-blocks-basic.min.js`);

  // 5. Load blocks manifest (dynamic, versioned) and append
  // block CSS to canvasCss. If manifest fails – canvasCss
  // stays with canvas.css + content.css + effects.css only
  // (graceful degrade).
  try {
    const mod = await import(`./blocks/manifest.js?v=${version}`);
    const blockCss = mod.blockCssUrls
      ? mod.blockCssUrls(null)
      : [];
    this.canvasCss.push(...blockCss);
    console.log('[GrapesLoader] block CSS appended:', blockCss.length);
  } catch (err) {
    console.warn('[GrapesLoader] blocks/manifest.js not loaded:', err);
  }
}

_loadStylesheet(href) {
  if (document.querySelector(`link[href="${href}"]`)) return;

```

```

    const link = document.createElement('link');
    link.rel = 'stylesheet';
    link.href = href;
    document.head.appendChild(link);
}

_loadScript(src) {
    return new Promise((resolve, reject) => {
        const existing = document.querySelector(`script[src="${src}"]`);
        if (existing) {
            resolve();
            return;
        }
        const script = document.createElement('script');
        script.src = src;
        script.onload = resolve;
        script.onerror = () => reject(new Error(`Script failed to load: ${src}`));
        document.head.appendChild(script);
    });
}

// =====
// GRAPESJS INIT
// =====

/**
 * Build config and call grapesjs.init().
 *
 * Returns GrapesJS instance, or null if there is nothing to mount:
 * - template is set but has no [data-slot="content"].
 * In that case the editor works in "preview only" mode – the
 * template is rendered read-only, no canvas is created.
 */
init() {
    console.log('[GrapesLoader] init()');

    if (typeof grapesjs === 'undefined') {
        throw new Error('[GrapesLoader] grapesjs not loaded');
    }

    const e = this.editor;

    if (!e.canvasEl) {
        throw new Error('[GrapesLoader] e.canvasEl not found – WidgetsBuilder.build() not called?');
    }

    // ===== Choose container =====
    //
    // Three cases:
    //
    // A. No template (templateHtml is empty):
    //     → mount into .editor-canvas (regular editor mode).
    //
    // B. Template with [data-slot="content"]:
    //     → replace that slot with .editor-slot and mount into it.
    //     The rest of the template stays as an outer frame.

```

```

//
// C. Template without a slot:
// → do NOT create GrapesJS. Render the template as a
// read-only preview. There is nothing to edit.
//
let container = null;
let containerKind = 'canvasEl';

if (e.templateHtml) {
  const slotInTemplate = e.templateEl
    ? e.templateEl.querySelector('[data-slot="content"]')
    : null;

  if (!slotInTemplate || !e.slotEl) {
    console.log(
      '[GrapesLoader] template has no [data-slot="content"] - ' +
      'skipping GrapesJS init (preview only)'
    );
    return null;
  }

  slotInTemplate.replaceWith(e.slotEl);
  container = e.slotEl;
  containerKind = 'slotEl (inside template)';
} else {
  container = e.canvasEl;
  containerKind = 'canvasEl';
}

console.log('[GrapesLoader] container:', containerKind);

const version = window.coreEngine?.static_version || Date.now();

// ===== Plugins =====
const plugins = [];
if (typeof grapesjsBlocksBasic !== 'undefined') {
  plugins.push(grapesjsBlocksBasic);
}

// ===== Canvas styles (inside iframe) =====
const canvasStyles = this.canvasCss.map(
  (path) => `${path}?v=${version}`
);

// ===== Config =====
const config = {
  container: container,
  height: '100%',
  width: '100%', // ← fills the whole container width
  fromElement: false,
  storageManager: false,

  plugins: plugins,
  pluginsOpts: {
    'grapesjs-blocks-basic': {
      flexGrid: true,

```



```

        // Separate category so the plugin does not create the
        // "Разметка" category BEFORE our LayoutBlocks register.
        // Otherwise the category order gets fixed by the plugin.
        category: 'Basic',
    },
},

// Disable built-in panels – we have our own toolbar
panels: { defaults: [] },

// Devices
deviceManager: {
    devices: [
        { name: 'desktop', width: '' },
        { name: 'tablet', width: '768px', widthMedia: '992px' },
        { name: 'mobile', width: '375px', widthMedia: '480px' },
    ],
},

// Asset manager – bound to our API
assetManager: {
    assets: [],
    upload: e._assetsUploadApi,
    uploadName: 'files',
    autoAdd: 1,
    dropzone: 1,
    openAssetsOnDrop: 1,
    headers: {},
    uploadText: 'Перетащите файлы сюда или нажмите для выбора',
    addBtnText: 'Добавить по ссылке',
    modalTitle: 'Медиаотека',
},

// ===== CSS inside canvas iframe =====
// Attached to the <head> of the iframe where blocks are rendered.
// Scoped under .core-engine-lib-word-blocks – isolated namespace.
canvas: {
    styles: canvasStyles,
},

// ===== Context menu filter =====
// GrapesJS shows a context menu on right-click / double-click.
// The built-in "Link" item does nothing useful (it expects an
// active RichTextEditor with a text selection), so we drop it
// and add our own command that wraps the component in <a> and
// selects it – then the traits panel shows href/target.
components: {
    contextMenu: (items) => {
        const KEEP = [
            'tlb-move',
            'tlb-clone',
            'tlb-delete',
            'tlb-select-parent',
        ];
        const filtered = items.filter(i => KEEP.includes(i.id));
    }
}

```

```

        // Append our custom "Link" item.
        filtered.push({
            id: 'tlb-custom-link',
            label: 'Ссылка',
            command: 'tlb-custom-link',
        });

        return filtered;
    },
},
};

// ===== BlockManager =====
if (e.blocksEl) {
    config.blockManager = {
        appendTo: e.blocksEl,
    };
}

// ===== TraitManager (right panel, above styles) =====
if (e.traitsEl) {
    config.traitManager = {
        appendTo: e.traitsEl,
    };
}

// ===== StyleManager (right panel, below styles) =====
if (e.stylesEl) {
    config.styleManager = {
        appendTo: e.stylesEl,
        sectors: this.options.styleManagerSectors || [],
    };
}

// ===== Init =====
const instance = grapesjs.init(config);
console.log('[GrapesLoader] GrapesJS initialized');

// ===== Custom command: wrap component in <a> and select it =====
// Replaces the broken built-in "Link" item.
//
// Behaviour:
// 1. If the selected component is already an <a> – just select it
//    (traits panel opens with href/target).
// 2. Otherwise – wrap its content in <a href="#">, replace the
//    component's content, then select the newly created <a>.
instance.Commands.add('tlb-custom-link', {
    run(editor) {
        const selected = editor.getSelected();
        if (!selected) return;

        // Already a link – just select it.
        if (selected.get('tagName') === 'a') {
            editor.select(selected);
            return;
        }
    }
});

```

```

        // Wrap in <a>.
        const html = selected.getEl()?.outerHTML || selected.toHTML();
        if (!html) return;

        // Replace the component's HTML with <a>...</a>.
        selected.replaceWith(`

```

```

/**
 * Add scope class to the iframe body.
 *
 * Called from event handlers – canvas:frame:load:body,
 * canvas:frame:load, load. No polling: if body is ready, the
 * event fires; if not, the next event fires later.
 *
 * GrapesJS re-creates the iframe on device change, so this must
 * run again on every 'canvas:frame:load:body' event.
 */
_applyScopeClass(instance) {
  try {
    const doc = instance.Canvas?.getDocument?();
    if (!doc || !doc.body) {
      // Body not ready yet – the next event will call us again.
      // Not an error, so no warning.
      return;
    }
    if (doc.body.classList.contains(this.scopeClass)) {
      return; // already applied – nothing to do
    }
    doc.body.classList.add(this.scopeClass);
    console.log('[GrapesLoader] scope class added to iframe body:', this.scopeClass);
  } catch (e) {
    console.warn('[GrapesLoader] failed to add scope class:', e);
  }
}

/**
 * Filter UndoManager: record only "real" changes.
 *
 * Problem:
 * By default GrapesJS records every `component:update` in the undo
 * stack – including pure selection changes (`selected` flag). So
 * Ctrl+Z after just clicking different elements "undoes" the
 * selection, which is useless and confusing.
 *
 * Solution:
 * 1. Save the original `um.track` (bound to the instance) and
 * replace it with a no-op – kills automatic recording.
 * 2. Manually call the saved `originalTrack` only for events that
 * represent real edits:
 *   - component:add / component:remove
 *   - component:update, unless the only changed attributes are
 *     `selected` / `status` / `hover`
 *   - styleable:change
 *   - rte:disable (exit from text editing)
 *
 * Why `um.track.bind(um)` and not `um.constructor.prototype.track`:
 * The prototype approach breaks across GrapesJS versions if the
 * internal method name changes. The instance-bound approach only
 * needs `um.track` to exist as a function – stable across 0.21.x.
 *
 * Idempotent: if called twice on the same UndoManager, the second
 * call is ignored (guarded by `um._undoFilterBound`).
 */

```

```

_bindUndoFilter(instance) {
  try {
    const um = instance.UndoManager;
    if (!um) {
      console.warn('[GrapesLoader] UndoManager not available – undo filter not bound');
      return;
    }

    // Guard against double-binding (hot reload, re-init, etc.)
    if (um._undoFilterBound) {
      console.log('[GrapesLoader] Undo filter already bound – skip');
      return;
    }
    um._undoFilterBound = true;

    // Save original track (bound to the instance) before replacing.
    const originalTrack = um.track.bind(um);

    // Replace with no-op – kills automatic recording.
    um.track = () => {};

    // Manual recording – only for real changes.
    const record = () => {
      try {
        originalTrack();
      } catch (err) {
        console.warn('[GrapesLoader] Undo record failed:', err);
      }
    };

    // 1. Add / remove components.
    instance.on('component:add', record);
    instance.on('component:remove', record);

    // 2. Component updates – skip selection-only changes.
    instance.on('component:update', (comp) => {
      try {
        const changed = (comp && typeof comp.changedAttributes === 'function')
          ? comp.changedAttributes()
          : null;

        if (changed) {
          const keys = Object.keys(changed);
          const onlySelection = keys.length > 0 && keys.every(
            (k) => k === 'selected' || k === 'status' || k === 'hover'
          );
          if (onlySelection) return;
        }

        record();
      } catch (err) {
        // Never let the filter break the editor – record anyway.
        console.warn('[GrapesLoader] component:update undo filter failed:', err);
        record();
      }
    });
  });
}

```

```

    // 3. Styles.
    instance.on('styleable:change', record);

    // 4. Text editing – record only on exit.
    instance.on('rte:disable', record);

    // Save record reference for destroy().
    this._undoFilterHandler = record;

    console.log('[GrapesLoader] Undo filter bound');
  } catch (err) {
    console.warn('[GrapesLoader] failed to bind undo filter:', err);
  }
}

/**
 * Auto-remove the empty <p> placeholder when a real block is added.
 *
 * GrapesJS inserts <p></p> as a placeholder for an empty canvas.
 * It takes vertical space and breaks full-height layouts.
 *
 * As soon as any real component lands in the wrapper (body), drop
 * the empty <p> – but only if it's truly empty (no content, no
 * children). User-authored paragraphs are left alone.
 *
 * Notes on safety:
 * - component may be already removed (e.g. we removed it in a
 *   previous iteration of this same handler, or GrapesJS is
 *   tearing down the tree during a drag). Guard against that.
 * - wrapper.components() may not be a Backbone collection in
 *   some versions – we use .each only if it exists.
 * - child.get('components') returns a collection; calling
 *   .length on it is safe, but we double-check it's truthy.
 */
_bindEmptyPCleanup(instance) {
  this._emptyPHandler = (component) => {
    try {
      // --- Guards: component may be gone ---
      if (!component) return;
      if (typeof component.parent !== 'function') return;

      const parent = component.parent();
      const wrapper = instance.getWrapper();

      // React only when adding directly into the wrapper (body)
      if (!parent || parent !== wrapper) return;

      // --- Guard: wrapper.components() may be missing ---
      if (typeof wrapper.components !== 'function') return;

      const children = wrapper.components();
      if (!children || typeof children.each !== 'function') return;

      // Collect first, then remove – mutating the collection
      // while iterating can skip items or re-enter the handler.

```

```

const toRemove = [];

children.each((child) => {
  if (!child || child === component) return;
  if (typeof child.get !== 'function') return;

  // Already removed from the tree
  if (child.removed || !child.collection) return;

  const tag = child.get('tagName');
  const content = String(child.get('content') || '').trim();

  let hasChildren = false;
  const comps = child.components && child.components();
  if (comps && typeof comps.length === 'number') {
    hasChildren = comps.length > 0;
  }

  const isEmptyP = tag === 'p'
    && !content
    && !hasChildren;

  if (isEmptyP) {
    toRemove.push(child);
  }
});

toRemove.forEach((child) => {
  if (child && !child.removed) {
    child.remove();
    console.log('[GrapesLoader] removed empty <p> placeholder');
  }
});

} catch (e) {
  // Cleanup is best-effort: never let it break the drop
  // or the editor flow. Log once at debug level.
  console.warn('[GrapesLoader] empty <p> cleanup failed:', e);
}

}

instance.on('component:add', this._emptyPHandler);
}

/**
 * Register custom trait types and assign them to the wrapper (body).
 *
 * Idempotent:
 * - trait types are registered only once (getType check);
 * - traits are always (re-)assigned on the current wrapper –
 *   GrapesJS replaces the wrapper on loadProjectData / setComponents,
 *   so re-assignment is required after each data load.
 *
 * These traits live in the Traits panel (Свойства) and write
 * directly to the component's inline style (component.addStyle).
 * Selecting the body shows layout controls (height, display,
 * flex-direction, padding, margin, overflow) – like StyleManager,

```

```

* but scoped to the wrapper only.
*/
_registerBodyStyleTraits(instance) {
  try {
    const tm = instance.TraitManager;

    // --- Custom trait type: <select> that writes to style ---
    if (!tm.getType('body-style-select')) {
      tm.addType('body-style-select', {
        createInput({ trait }) {
          const options = trait.get('options') || [];
          const el = document.createElement('select');
          el.className = 'body-style-select';

          const emptyOpt = document.createElement('option');
          emptyOpt.value = '';
          emptyOpt.textContent = '-';
          el.appendChild(emptyOpt);

          options.forEach((opt) => {
            const o = document.createElement('option');
            o.value = opt.id;
            o.textContent = opt.name;
            el.appendChild(o);
          });

          return el;
        },
        onEvent({ elInput, component }) {
          const prop = this.target.get('bodyProp');
          const value = elInput.value;
          if (value) {
            component.addStyle({ [prop]: value });
          } else {
            component.removeStyle(prop);
          }
        },
        onUpdate({ elInput, component }) {
          const prop = this.target.get('bodyProp');
          const value = component.getStyle()[prop] || '';
          elInput.value = value;
        },
      });
    }

    // --- Custom trait type: integer input + unit select ---
    if (!tm.getType('body-style-integer')) {
      tm.addType('body-style-integer', {
        createInput({ trait }) {
          const units = trait.get('units') || ['px'];
          const wrap = document.createElement('div');
          wrap.className = 'body-style-integer';

          const input = document.createElement('input');
          input.type = 'text';
          input.placeholder = 'auto';

```



```

        input.className = 'body-style-integer__input';

        const unitSel = document.createElement('select');
        unitSel.className = 'body-style-integer__unit';
        units.forEach((u) => {
            const o = document.createElement('option');
            o.value = u;
            o.textContent = u;
            unitSel.appendChild(o);
        });

        wrap.appendChild(input);
        wrap.appendChild(unitSel);
        return wrap;
    },
    onEvent({ elInput, component }) {
        const prop = this.target.get('bodyProp');
        const input = elInput.querySelector('.body-style-integer__input');
        const unitSel = elInput.querySelector('.body-style-integer__unit');
        const raw = input.value.trim();
        if (!raw) {
            component.removeStyle(prop);
            return;
        }
        const unit = unitSel.value || 'px';
        const hasUnit = /[a-z%]+$/i.test(raw);
        const value = hasUnit ? raw : `${raw}${unit}`;
        component.addStyle({ [prop]: value });
    },
    onUpdate({ elInput, component }) {
        const prop = this.target.get('bodyProp');
        const input = elInput.querySelector('.body-style-integer__input');
        const unitSel = elInput.querySelector('.body-style-integer__unit');
        const value = component.getStyle()[prop] || '';

        const m = value.match(/^(?![\d.]+)\s*([a-z%]*)$/i);
        if (m) {
            input.value = m[1];
            if (m[2] && [...unitSel.options].some(o => o.value === m[2])) {
                unitSel.value = m[2];
            }
        } else {
            input.value = value;
        }
    },
    });
}

// --- Assign traits to wrapper (body) ---
const wrapper = instance.getWrapper();
if (!wrapper) {
    console.warn('[GrapesLoader] wrapper not available – body traits not assigned');
    return;
}

const bodyTraits = [

```

```

{
  type: 'body-style-integer',
  name: 'height',
  label: 'Высота',
  bodyProp: 'height',
  units: ['px', '%', 'vh', 'vw', 'em', 'rem'],
},
{
  type: 'body-style-integer',
  name: 'min-height',
  label: 'Мин. высота',
  bodyProp: 'min-height',
  units: ['px', '%', 'vh', 'vw', 'em', 'rem'],
},
{
  type: 'body-style-integer',
  name: 'max-height',
  label: 'Макс. высота',
  bodyProp: 'max-height',
  units: ['px', '%', 'vh', 'vw', 'em', 'rem'],
},
{
  type: 'body-style-select',
  name: 'display',
  label: 'Display',
  bodyProp: 'display',
  options: [
    { id: 'block', name: 'block' },
    { id: 'flex', name: 'flex' },
    { id: 'grid', name: 'grid' },
    { id: 'inline', name: 'inline' },
    { id: 'inline-block', name: 'inline-block' },
    { id: 'none', name: 'none' },
  ],
},
{
  type: 'body-style-select',
  name: 'flex-direction',
  label: 'Flex direction',
  bodyProp: 'flex-direction',
  options: [
    { id: 'row', name: 'row' },
    { id: 'row-reverse', name: 'row-reverse' },
    { id: 'column', name: 'column' },
    { id: 'column-reverse', name: 'column-reverse' },
  ],
},
{
  type: 'body-style-select',
  name: 'justify-content',
  label: 'Justify content',
  bodyProp: 'justify-content',
  options: [
    { id: 'flex-start', name: 'flex-start' },
    { id: 'flex-end', name: 'flex-end' },
    { id: 'center', name: 'center' },
  ],
}

```

```

        { id: 'space-between', name: 'space-between' },
        { id: 'space-around', name: 'space-around' },
        { id: 'space-evenly', name: 'space-evenly' },
    ],
},
{
    type: 'body-style-select',
    name: 'align-items',
    label: 'Align items',
    bodyProp: 'align-items',
    options: [
        { id: 'stretch', name: 'stretch' },
        { id: 'flex-start', name: 'flex-start' },
        { id: 'flex-end', name: 'flex-end' },
        { id: 'center', name: 'center' },
        { id: 'baseline', name: 'baseline' },
    ],
},
{
    type: 'body-style-integer',
    name: 'padding',
    label: 'Внутренний отступ',
    bodyProp: 'padding',
    units: ['px', '%', 'em', 'rem'],
},
{
    type: 'body-style-integer',
    name: 'margin',
    label: 'Внешний отступ',
    bodyProp: 'margin',
    units: ['px', '%', 'em', 'rem'],
},
{
    type: 'body-style-select',
    name: 'overflow',
    label: 'Overflow',
    bodyProp: 'overflow',
    options: [
        { id: 'visible', name: 'visible' },
        { id: 'hidden', name: 'hidden' },
        { id: 'scroll', name: 'scroll' },
        { id: 'auto', name: 'auto' },
    ],
},
},
];

wrapper.set('traits', bodyTraits);
console.log('[GrapesLoader] body traits assigned:', bodyTraits.length);

} catch (err) {
    console.warn('[GrapesLoader] failed to register body style traits:', err);
}

}

/**
 * Detach canvas event handlers. Called from Editor.destroy().

```

```

*/
destroy(instance) {
  if (this._frameLoadHandler && instance) {
    try {
      instance.off('canvas:frame:load:body', this._frameLoadHandler);
      instance.off('canvas:frame:load', this._frameLoadHandler);
      instance.off('load', this._frameLoadHandler);
    } catch (e) {
      // ignore
    }
  }
  this._frameLoadHandler = null;

  if (this._emptyPHandler && instance) {
    try {
      instance.off('component:add', this._emptyPHandler);
    } catch (e) {
      // ignore
    }
  }
  this._emptyPHandler = null;

  if (this._undoFilterHandler && instance) {
    try {
      instance.off('component:add', this._undoFilterHandler);
      instance.off('component:remove', this._undoFilterHandler);
      instance.off('styleable:change', this._undoFilterHandler);
      instance.off('rte:disable', this._undoFilterHandler);
      // component:update listener is anonymous – cannot detach
      // it by reference. If a full detach is required, refactor
      // to store the handler in a field.
    } catch (e) {
      // ignore
    }
  }
  this._undoFilterHandler = null;
}

/**
 * Register 'link' component type with traits (href, title, target).
 * Allows setting links on buttons and other elements.
 */
_registerLinkType(instance) {
  try {
    instance.DomComponents.addType('link', {
      isComponent: (el) => el.tagName === 'A',
      model: {
        defaults: {
          traits: [
            'id',
            'title',
            {
              type: 'text',
              name: 'href',
              label: 'Ссылка (href)',
              placeholder: '/page/... или https://...',
            }
          ]
        }
      }
    });
  }
}

```

```

        },
        {
            type: 'select',
            name: 'target',
            label: 'Открывать',
            options: [
                { id: '', name: 'В текущем окне' },
                { id: '_blank', name: 'В новой вкладке' },
            ],
        },
    ],
},
},
});
console.log('[GrapesLoader] Link type registered');
} catch (err) {
    console.warn('[GrapesLoader] Failed to register link type:', err);
}
}

/**
 * Remove inline width/float/margin from block cards
 * that GrapesJS sets at render.
 */
_resetBlockInlineStyles(instance) {
    try {
        const blocks = instance.BlockManager.getAll();
        blocks.forEach((block) => {
            const el = block.get('el');
            if (el && el.style) {
                el.style.width = '';
                el.style.maxWidth = '';
                el.style.minWidth = '';
                el.style.float = '';
                el.style.margin = '';
                el.style.flexBasis = '';
            }
        });
        console.log('[GrapesLoader] Block inline styles reset');
    } catch (e) {
        console.warn('[GrapesLoader] Failed to reset block inline styles:', e);
    }
}
}

```

```

=====
FILE: core/engine/lib/word/llm/chat.js
=====

```

```
// neurocad/core/engine/lib/word/llm/chat.js
```

```

/**
 * LLMChat – thin facade over ./chat/*.js.
 *
 * Sub-modules are loaded lazily via dynamic import() with a version
 * query, matching the style of core/engine/lib/base/base.js.
 *

```

```

* LLMChatWS      (./chat/ws.js)      - WebSocket transport
* LLMChatUI      (./chat/ui.js)      - DOM: messages, progress, typing
* LLMChatAPI     (./chat/api.js)     - HTTP: history, active run, clear
* LLMChatDebug   (./chat/debug.js)   - console dumps
* buildLLMChatCatalog (./chat/catalog.js) - block catalog from GrapesJS
* createLLMChatHandler (./chat/handler.js) - incoming WS message handler
* getLLMChatSelection (./chat/selection.js) - current selection from GrapesJS
* applyLLMChatElementUpdate (./chat/apply.js) - replace a single element on canvas
* EditSession    (./chat/edit.js)    - effect edit mode
* CreateSession  (./chat/create.js)  - effect create mode
*
* What stays here
* -----
* This file is the ORCHESTRATOR. It owns:
*
* - the WS transport (connect / reconnect / state banners);
* - chat history + active run (HTTP);
* - the two sub-sessions (EditSession / CreateSession):
*   * created in _loadModules();
*   * bind() / unbind() called here;
*   * _onSend delegates to session.send(text) when a session is active.
*
* Everything specific to "edit effect" or "create effect" lives in the
* respective session file. This file only routes.
*
* Page generation from create_page
* -----
* The create_page agent builds a page either in ONE shot or
* STEPWISE (one section per LLM call). Both modes deliver HTML and
* CSS as two separate channels:
*
* Single-shot mode:
*   { type: 'html_update',    html: '...' }
*   { type: 'page_css_update', css: '...' }
*
* Stepwise mode:
*   { type: 'page_step', step: N, section_name: 'hero',
*     html: '<section...>', css: '.hero {...}', done: false }
*
* In both cases handler.js forwards the CSS through the chat's
* applyPageCss callback, which calls _applyPageCss(css) below.
* _applyPageCss feeds the string to editor.setStyle() – the same
* channel the Style Manager uses on save.
*
* In stepwise mode the handler accumulates all step fragments and
* calls applyHtml / applyPageCss with the WHOLE accumulated page
* after every step. That means _applyPageCss is called 5–10 times
* per run with monotonically growing CSS. To avoid useless
* re-renders when a step happened to return empty CSS, both
* _applyHtml and _applyPageCss skip their work if the incoming
* string is identical to the last one they applied for the
* current run.
*
* The _lastAppliedHtml / _lastAppliedCss caches are reset in
* _onSend() at the start of every new user request, so a fresh
* run always sees the first step as a change.

```

```

*
* The split between HTML and CSS is required by GrapesJS: it
* cannot parse <style> mixed into components and silently drops
* the whole tree if we inline the CSS into the HTML.
*
* Effect edit mode
* -----
* The palette emits `word:effect-edit-start`. EditSession._onStart
* fetches the current CSS, stores it, greets the user.
*
* LLMChat._onSend: if EditSession.isActive() – session.send(text).
*
* The server replies with `css_update`. handler.js calls the
* chat's `applyCss` callback with (effectId, css, opts), which
* delegates to EditSession.handleCss(effectId, css, opts).
* `opts` may carry `{ newId, newLabel }` if the server proposes to
* rename the effect on save.
*
* The user clicks the save icon (in the palette). The palette emits
* `word:effect-edit-save`. EditSession either PUTs the CSS in place
* (no rename) or POSTs a new effect + DELETES the old one (rename),
* commits the draft, clears state.
*
* Effect create mode
* -----
* The palette "+" button creates a PENDING block and emits
* `word:effect-create-session-started`. CreateSession._onStart resets
* state and greets the user.
*
* LLMChat._onSend: if CreateSession.isActive() – session.send(text).
* The session forwards `{ type: "create_effect", ... }` with the
* previous draft (if any) for iteration.
*
* The server replies with `effect_draft`. handler.js calls the
* chat's `applyEffectDraft` callback, which delegates to
* CreateSession.handleDraft(draft). Session applies CSS + class +
* updates the pending block in the palette.
*
* The user clicks the save icon (on the pending block). The palette
* emits `word:effect-create-save`. CreateSession POSTs the effect,
* injects the <link>, commits the draft, converts the pending
* block into a normal one.
*
* The user clicks the close icon. The palette emits
* `word:effect-create-cancel`. CreateSession reverts the draft,
* removes the class, removes the pending block.
*/
export class LLMChat {
  constructor(editor) {
    console.log('[LLMChat] Constructor');
    this.editor = editor;

    this._apiBase = '/core/engine/lib/word/llm';

    // Base for the effects editor API:
    // /core/engine/lib/word/editor/effects

```

```

this._effectsApiBase = '/core/engine/lib/word/editor/effects';

// Sub-modules – created in _loadModules()
this.ws = null;
this.ui = null;
this.api = null;
this.debug = null;
this._onWsMessage = null;
this._buildBlockCatalog = null;
this._getSelection = null;
this._applyElementUpdate = null;

// Effects live-edit helpers (loaded in _loadModules)
this._applyEffectDraft = null;
this._commitEffectDraft = null;
this._revertEffectDraft = null;
this._addEffectLink = null;
this._removeEffectLink = null;

// Sessions (created in _loadModules after their modules load)
this._editSession = null;
this._createSession = null;

// Run state
this._running = false;
this._currentRunId = null;
this._pendingStart = null;

// Stepwise create_page – dedup cache.
//
// In stepwise mode handler.js accumulates HTML/CSS fragments
// and calls _applyHtml / _applyPageCss with the whole page
// after every step. The CSS usually grows monotonically; the
// HTML as well. But a step may legitimately return empty
// html/css (e.g. the final "done":true frame). We do not
// want to rebuild the canvas for an unchanged string –
// setComponents / setStyle are not free.
//
// Both caches are reset in _onSend() at the start of every
// new user request.
this._lastAppliedHtml = '';
this._lastAppliedCss = '';

// Data
this.messages = [];
}

async init() {
  console.log('[LLMChat] init() START');

  if (!this.editor.chatEl) {
    console.error('[LLMChat] chatEl not found');
    return;
  }

  await this._loadModules();
}

```



```

this.debug.dumpEnv();

this.ui.mount();
this.ui.onSubmit(() => this._onSend());
this.ui.onClear(() => this._onClear());

await this._loadHistory();

if (this.messages.length === 0) {
  this.ui.addMessage({
    role: 'assistant',
    content: 'Опишите, что нужно сделать со страницей. Например: «собери лендинг для спа-салона», «выдели блок и скажи „заполни текстом“ или „закрась жёлтым“».',
    created_at: new Date().toISOString(),
  });
}

// Wire up the sessions (they subscribe to editor events).
this._editSession?.bind();
this._createSession?.bind();

this.ws.connect();
await this._checkActiveRun();

console.log('[LLMChat] init() COMPLETE');
}

// =====
// MODULE LOADING
// =====

async _loadModules() {
  const v = window.coreEngine?.static_version || Date.now();

  const [
    { LLMChatWS },
    { LLMChatUI },
    { LLMChatAPI },
    { LLMChatDebug },
    { buildLLMChatCatalog },
    { createLLMChatHandler },
    { getLLMChatSelection },
    { applyLLMChatElementUpdate },
    {
      applyDraft,
      commitDraft,
      revertDraft,
      addEffectLink,
      removeEffectLink,
    },
    { EditSession },
    { CreateSession },
  ] = await Promise.all([
    import(`./chat/ws.js?v=${v}`),
    import(`./chat/ui.js?v=${v}`),
  ]);
}

```

```

import(`./chat/api.js?v=${v}`),
import(`./chat/debug.js?v=${v}`),
import(`./chat/catalog.js?v=${v}`),
import(`./chat/handler.js?v=${v}`),
import(`./chat/selection.js?v=${v}`),
import(`./chat/apply.js?v=${v}`),
import(`../editor/effects/live.js?v=${v}`),
import(`./chat/edit.js?v=${v}`),
import(`./chat/create.js?v=${v}`),
]);

this.debug = new LLMChatDebug(this.editor);
this.ui = new LLMChatUI(this.editor);
this.api = new LLMChatAPI(this._apiBase);
this._buildBlockCatalog = buildLLMChatCatalog;
this._getSelection = getLLMChatSelection;
this._applyElementUpdate = applyLLMChatElementUpdate;

this._applyEffectDraft = applyDraft;
this._commitEffectDraft = commitDraft;
this._revertEffectDraft = revertDraft;
this._addEffectLink = addEffectLink;
this._removeEffectLink = removeEffectLink;

// Sessions – created after their modules and the UI/WS are ready.
// They do NOT subscribe to editor events yet – bind() is called
// from init() (so that the editor is fully initialized first).
this._editSession = new EditSession(this);
this._createSession = new CreateSession(this);

this.ws = new LLMChatWS({
  urlProvider: () => this._getWsUrl(),
  onOpen: () => this._onWsOpen(),
  onMessage: (m) => this._onWsMessage(m),
  onClose: (info) => this._onWsClose(info),
  onState: (state) => this._onWsState(state),
});

this._onWsMessage = createLLMChatHandler({
  ui: this.ui,
  onRunStart: (id) => { this._currentRunId = id; this._running = true; },
  onRunEnd: () => { this._currentRunId = null; this._running = false; },
  applyHtml: (html) => this._applyHtml(html),
  applyPageCss: (css) => this._applyPageCss(css),
  applyElement: (selector, html) => this._applyElement(selector, html),
  // handler.js passes a third `opts` argument that may carry
  // { newId, newLabel } for effect renames. Forward it to
  // EditSession.handleCss unchanged.
  applyCss: (effectId, css, opts) =>
    this._editSession?.handleCss(effectId, css, opts),
  applyEffectDraft: (draft) => this._createSession?.handleDraft(draft),
});
}

// =====
// URL

```

```

// =====

_getWsUrl() {
  const pageId = this.editor.pageId;
  if (!pageId) return null;

  const proto = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
  const host = window.location.host;
  return `${proto}://${host}${this._apiBase}/ws/${pageId}`;
}

/**
 * ?module=<name> for the effects editor API.
 */
_qsForEffects() {
  const moduleName = window.coreEngine?.moduleName
    || document.body.dataset.module
    || '';
  return moduleName
    ? `?module=${encodeURIComponent(moduleName)}`
    : '';
}

// =====
// HISTORY / ACTIVE RUN (HTTP)
// =====

async _loadHistory() {
  const pageId = this.editor.pageId;
  if (!pageId) return;

  const rows = await this.api.loadHistory(pageId);
  if (rows) {
    this.messages = rows;
    this.ui.setMessages(rows);
    console.log('[LLMChat] Loaded messages: ${this.messages.length}');
  }
}

async _checkActiveRun() {
  const pageId = this.editor.pageId;
  if (!pageId) return;

  const run = await this.api.checkActiveRun(pageId);
  if (!run) {
    console.log('[LLMChat] no active run');
    return;
  }

  console.log('[LLMChat] active run found:', run.id, run.status);
  this._currentRunId = run.id;
  this._running = true;
  this.ui.setSendingState(true);
  this.ui.setProgressText(run.message || `Run #${run.id} в процессе...`);
}

```

```

// =====
// CLEAR CHAT
// =====

async _onClear() {
  console.log('[LLMChat] _onClear()');

  const pageId = this.editor.pageId;
  if (!pageId) return;

  const deleted = await this.api.clearHistory(pageId);
  if (deleted === null) {
    console.error('[LLMChat] clear failed');
    this.ui.addMessage({
      role: 'assistant',
      content: '⚠ Не удалось очистить историю чата.',
      created_at: new Date().toISOString(),
    });
    return;
  }

  console.log(`[LLMChat] cleared ${deleted} messages`);
  this.messages = [];
  this.ui.setMessages([]);
  this.ui.finalizeProgress();
  this.ui.hideTyping();
  this.ui.addMessage({
    role: 'assistant',
    content: 'Чат очищен. Опишите, что сделать со страницей.',
    created_at: new Date().toISOString(),
  });
}

// =====
// WS CALLBACKS
// =====

_onWsOpen() {
  console.log('[LLMChat] WS open – readyState:', this.ws.readyState);

  this._setConnectionBanner(null);

  if (this._pendingStart) {
    const p = this._pendingStart;
    this._pendingStart = null;
    console.log('[LLMChat] flushing pending start');
    this.ws.send(p);
  }
}

_onWsClose(info = {}) {
  console.log('[LLMChat] WS close', info);

  if (info.auth) {
    this._setConnectionBanner({
      kind: 'error',
    });
  }
}

```

```

        text: 'Нет доступа к чату. Войдите как супер-администратор.',
    });
    this.ui.setSendingState(false);
    this.ui.hideTyping();
    return;
}

if (this._running) {
    this.ui.setProgressText('Соединение потеряно. Переподключаюсь...');
}

setTimeout(() => this._checkActiveRun(), 3500);
}

_onWsState(state) {
    switch (state) {
        case 'connecting':
            this._setConnectionBanner({ kind: 'info', text: 'Подключение к чату...' });
            break;
        case 'open':
            this._setConnectionBanner(null);
            break;
        case 'closed':
            this._setConnectionBanner({ kind: 'warn', text: 'Соединение потеряно. Переподключаюсь...' });
            break;
        case 'auth':
            this._setConnectionBanner({ kind: 'error', text: 'Нет доступа к чату. Войдите как супер-администратор.' });
            break;
        case 'dead':
            this._setConnectionBanner(null);
            break;
    }
}

_setConnectionBanner(spec) {
    if (typeof this.ui?.setConnectionBanner === 'function') {
        this.ui.setConnectionBanner(spec);
    } else if (spec) {
        console.log('[LLMChat] connection state:', spec.kind, spec.text);
    }
}

// =====
// HELPERS USED BY SESSIONS
// =====

/**
 * Access the EffectBlocks instance.
 *
 * The instance is registered by word/editor/index.js (BlocksRegistry
 * → register EffectBlocks) and stored somewhere on the editor /
 * word component. The exact field depends on which object holds it:
 *
 * - this.editor._effectBlocks          – Word component
 * - this.editor.editor._effectBlocks    – GrapesJS editor

```

```

* - this.editor.blocks._effectBlocks      - BlocksRegistry module
* - this.editor.editor.effectBlocks      - alternative field name
* - window._effectBlocks                 - last-resort global
*
* We probe all of them and log which one hit, so a missing
* registration is visible in the console instead of silently
* producing null.
*
* NOTE: this method is intentionally verbose on the console - it
* is the single point where the "save button stays disabled" bug
* manifests, and we want the exact hit / miss in the log.
*/
_effectBlocks() {
  const candidates = [
    ['this.editor._effectBlocks',      this.editor?._effectBlocks],
    ['this.editor.editor._effectBlocks', this.editor?.editor?._effectBlocks],
    ['this.editor.blocks._effectBlocks', this.editor?.blocks?._effectBlocks],
    ['this.editor.editor.effectBlocks', this.editor?.editor?.effectBlocks],
    ['this.editor.effectBlocks',      this.editor?.effectBlocks],
    ['window._effectBlocks',          window._effectBlocks],
  ];

  for (const [where, value] of candidates) {
    if (value) {
      console.log('[LLMChat] _effectBlocks() found at:', where);
      return value;
    }
  }

  console.warn(
    '[LLMChat] _effectBlocks() NOT FOUND. Probed:',
    candidates.map(([where, v]) => `${where}=${v ? 'set' : 'null'}`)
  );
  return null;
}

/**
* Load the current CSS of an effect from the server.
* Used by EditSession._handleStart.
*/
async _loadEffectCss(effectId) {
  const url = `${this._effectsApiBase}/${effectId}.css${this._qsForEffects()}`;
  console.log('[LLMChat] GET', url);

  try {
    const fetchJson = window.coreEngine?.fetchJson;
    const data = await fetchJson(url);
    if (data && data.success && data.data && typeof data.data.css === 'string') {
      return data.data.css;
    }
    console.warn('[LLMChat] loadEffectCss: unexpected payload', data);
    return null;
  } catch (e) {
    console.error('[LLMChat] loadEffectCss error:', e);
    return null;
  }
}

```

```

}

/**
 * Save the given effect CSS to the server (PUT).
 * Used by EditSession._handleSave.
 */
async _saveEffectCss(effectId, css) {
  const url = `${this._effectsApiBase}/${effectId}${this._qsForEffects()}`;
  console.log('[LLMChat] PUT', url, `${css.length} bytes`);

  try {
    const fetchJson = window.coreEngine?.fetchJson;
    const data = await fetchJson(url, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ css }),
    });
    return !!data && data.success;
  } catch (e) {
    console.error('[LLMChat] saveEffectCss error:', e);
    return false;
  }
}

// =====
// SEND / CANCEL
// =====

_onSend() {
  if (this._running) {
    this._onCancel();
    return;
  }

  const text = (this.ui.getInputValue() || '').trim();
  if (!text) return;

  if (!this.editor.pageId) {
    this.ui.addMessage({
      role: 'assistant',
      content: '⚠ Не удалось определить страницу (pageId не задан).',
      created_at: new Date().toISOString(),
    });
    return;
  }

  console.log('[LLMChat] _onSend() text:', text);

  // ---- reset stepwise dedup caches ----
  // A brand-new run must not skip its first applyHtml /
  // applyPageCss because the previous run happened to finish
  // with the exact same string. Cheap assignment, no
  // side-effects on the canvas.
  this._lastAppliedHtml = '';
  this._lastAppliedCss = '';

```

```

this.ui.addMessage({
  role: 'user',
  content: text,
  created_at: new Date().toISOString(),
});
this.ui.clearInput();

// ---- EFFECT CREATE MODE ----
// While a create session is active, every message goes to the
// create_effect agent. Saving is NOT done via chat – it is
// triggered by clicking the save icon on the pending block.
if (this._createSession?.isActive()) {
  console.log('[LLMChat] create-effect: draft request');
  this.ui.showTyping();
  this._running = true;
  this.ui.setSendingState(true);

  this._createSession.send(text);
  return;
}

// ---- EFFECT EDIT MODE ----
if (this._editSession?.isActive()) {
  this.ui.showTyping();
  this._running = true;
  this.ui.setSendingState(true);

  this._editSession.send(text);
  return;
}

// ---- REGULAR PAGE FLOW ----
this.ui.showTyping();
this._running = true;
this.ui.setSendingState(true);

const blockCatalog = this._buildBlockCatalog(this.editor);
const selection = this._getSelection(this.editor);

const payload = {
  type: 'start',
  message: text,
  block_catalog: blockCatalog,
  selection: selection,
};

this.debug.dumpBlocks(blockCatalog);
this.debug.dumpSelection(selection);
this.debug.dumpPayload(payload);

if (this.ws.isOpen()) {
  console.log('[LLMChat] sending start over open WS');
  this.ws.send(payload);
} else {
  console.warn('[LLMChat] WS not ready, queueing start');
  this._pendingStart = payload;
}

```



```

        this.ws.connect();
    }
}

_onCancel() {
    console.log('[LLMChat] _onCancel()');

    if (this._pendingStart) {
        this._pendingStart = null;
        this._running = false;
        this.ui.setSendingState(false);
        this.ui.hideTyping();
        this.ui.setProgressText('Отменено. ');
        this.ui.finalizeProgress();
        return;
    }

    this.ws.send({ type: 'cancel' });
    this.ui.setProgressText('Отмена...');
}

// =====
// APPLY HTML / CSS / ELEMENT
// =====

/**
 * Replace the whole canvas with the given HTML.
 *
 * Called by handler.js for two flows:
 * - single-shot agents → once, with the full page HTML
 *   (`html_update` frame);
 * - stepwise create_page → after every `page_step`, with the
 *   accumulated HTML of all sections so far.
 *
 * Dedup: if the incoming HTML is byte-for-byte identical to the
 * one we applied during the current run, skip the rebuild. This
 * happens when a step returns no HTML (e.g. the terminal
 * `done:true` frame) but the handler still re-applies the full
 * accumulated string. The cache is reset in _onSend().
 */
_applyHtml(html) {
    if (!html) return;
    if (!this.editor.editor) return;

    if (html === this._lastAppliedHtml) {
        console.log('[LLMChat] _applyHtml: unchanged, skipping');
        return;
    }

    try {
        console.log('[LLMChat] Applying HTML to canvas:', html.length);
        this.editor.editor.setComponents(html);
        this._lastAppliedHtml = html;
    } catch (e) {
        console.error('[LLMChat] Apply error:', e);
        this.ui.addMessage({

```

```

        role: 'assistant',
        content: `⚠ Ошибка применения HTML: ${e.message}`,
        created_at: new Date().toISOString(),
    });
}
}

/**
 * Replace the whole page CSS with the given string.
 *
 * Called by handler.js for two flows:
 * - single-shot agents → once, with the full page CSS
 *   (`page_css_update` frame);
 * - stepwise create_page → after every `page_step`, with the
 *   accumulated CSS of all sections so far.
 *
 * The CSS is passed as a plain string WITHOUT the surrounding
 * <style> tag. We feed it straight to editor.setStyle() – the
 * same channel the Style Manager uses on save. Doing this via
 * setStyle (and not by inlining <style> into the HTML) is
 * required: GrapesJS cannot parse <style> mixed into components
 * and would drop the whole tree if we tried.
 *
 * Dedup: same as _applyHtml – an identical string is skipped.
 * The stepwise loop tends to grow the CSS monotonically, but a
 * step without CSS would otherwise trigger an unnecessary
 * setStyle.
 */
_applyPageCss(css) {
    if (!css || !css.trim()) return;
    if (!this.editor.editor) return;

    if (css === this._lastAppliedCss) {
        console.log('[LLMChat] _applyPageCss: unchanged, skipping');
        return;
    }

    try {
        console.log('[LLMChat] Applying page CSS:', css.length);
        this.editor.editor.setStyle(css);
        this._lastAppliedCss = css;
    } catch (e) {
        console.error('[LLMChat] Page CSS apply error:', e);
        this.ui.addMessage({
            role: 'assistant',
            content: `⚠ Ошибка применения CSS: ${e.message}`,
            created_at: new Date().toISOString(),
        });
    }
}

_applyElement(selector, html) {
    if (!selector || !html) return;
    if (!this.editor.editor) return;

    try {

```

```

        console.log('[LLMChat] Applying element update:', selector, html.length);
        this._applyElementUpdate(this.editor, selector, html);
    } catch (e) {
        console.error('[LLMChat] Element apply error:', e);
        this.ui.addMessage({
            role: 'assistant',
            content: `⚠ Ошибка замены элемента ${selector}: ${e.message}`,
            created_at: new Date().toISOString(),
        });
    }
}

// =====
// PUBLIC
// =====

getMessages() {
    return this.messages.slice();
}

destroy() {
    console.log('[LLMChat] destroy()');

    // Sessions – detach from editor events.
    if (this._editSession) {
        try { this._editSession.unbind(); } catch (_) {}
        this._editSession = null;
    }
    if (this._createSession) {
        try { this._createSession.unbind(); } catch (_) {}
        this._createSession = null;
    }

    if (this.ws) this.ws.destroy();
    if (this.ui) this.ui.destroy();

    this.messages = [];
    this._running = false;
    this._currentRunId = null;
    this._pendingStart = null;
    this._lastAppliedHtml = '';
    this._lastAppliedCss = '';
}
}

```

```

=====
FILE: core/engine/lib/word/llm/presets.js
=====

```

```

// neurocad/core/engine/lib/word/llm/presets.js

```

```

/**
 * LLMPresets – presets list (left panel, "Presets" tab).
 *
 * Tasks:
 * - load presets from backend (GET /presets);
 * - render list with thumbnails;

```

```

* - on click – apply preset HTML+CSS to GrapesJS canvas;
* - create new preset from current canvas (POST /presets);
* - delete preset (DELETE /presets/{id});
* - upload PNG thumbnail (POST /presets/{id}/thumbnail).
*
* HTTP goes through window.coreEngine.fetchJson
* (loaded once by CoreEngine.loadApi()).
*
* Container:
*   editor.presetsEl – [data-js="editor-presets"]
*   Toolbar with "+" button is built by editor/widgets.js
*/
export class LLMPresets {
  constructor(editor) {
    console.log('[LLMPresets] Constructor');
    this.editor = editor;

    // DOM
    this.rootEl = editor.presetsEl;      // [data-js="editor-presets"]
    this.listEl = null;

    // API
    this._apiBase = '/core/engine/lib/word/llm/presets';

    // Data
    this.presets = [];                  // list from backend
    this.currentId = null;              // selected preset id
  }

  async init() {
    console.log('[LLMPresets] init() START');

    if (!this.rootEl) {
      console.error('[LLMPresets] presetsEl not found');
      return;
    }

    this._buildDOM();
    this._bindToolbarEvents();

    // Load list from backend
    await this._loadPresets();

    console.log('[LLMPresets] init() COMPLETE');
  }

  // =====
  // DOM
  // =====

  _buildDOM() {
    // Clear container
    while (this.rootEl.firstChild) {
      this.rootEl.removeChild(this.rootEl.firstChild);
    }
  }
}

```

```

    // Presets list
    const list = document.createElement('div');
    list.className = 'core-engine-lib-word-llm-presets-list';
    list.setAttribute('data-js', 'llm-presets-list');
    this.listEl = list;
    this.rootEl.appendChild(list);
}

_bindToolbarEvents() {
    // "+" button in presets toolbar
    const toolbar = this.editor.leftArea?.querySelector('[data-js="editor-presets-toolbar"]');
    if (!toolbar) return;

    toolbar.addEventListener('click', (e) => {
        const btn = e.target.closest('[data-action="preset-add"]');
        if (!btn) return;
        this._onCreate();
    });
}

_renderList() {
    if (!this.listEl) return;

    // Clear list
    while (this.listEl.firstChild) {
        this.listEl.removeChild(this.listEl.firstChild);
    }

    if (!this.presets || this.presets.length === 0) {
        const empty = document.createElement('div');
        empty.className = 'core-engine-lib-word-llm-presets-empty';
        empty.textContent = 'Нет настроек';
        this.listEl.appendChild(empty);
        return;
    }

    for (const preset of this.presets) {
        const item = this._buildItem(preset);
        this.listEl.appendChild(item);
    }
}

_buildItem(preset) {
    const item = document.createElement('div');
    item.className = 'core-engine-lib-word-llm-presets-item';
    item.setAttribute('data-id', preset.id);

    if (preset.id === this.currentId) {
        item.classList.add('active');
    }

    // ===== Thumbnail =====
    const thumb = document.createElement('div');
    thumb.className = 'core-engine-lib-word-llm-presets-thumb';

    if (preset.thumbnail_path) {

```

```

        const img = document.createElement('img');
        img.src = `/media/${preset.thumbnail_path}`;
        img.alt = preset.name || '';
        img.loading = 'lazy';
        thumb.appendChild(img);
    } else {
        // Placeholder if no thumbnail
        thumb.textContent = '□';
    }
    item.appendChild(thumb);

    // ===== Name =====
    const name = document.createElement('div');
    name.className = 'core-engine-lib-word-llm-presets-name';
    name.textContent = preset.name || 'Без названия';
    item.appendChild(name);

    // ===== Delete button (appears on hover) =====
    const delBtn = document.createElement('button');
    delBtn.type = 'button';
    delBtn.className = 'core-engine-lib-word-llm-presets-del';
    delBtn.setAttribute('title', 'Удалить пресет');
    delBtn.setAttribute('aria-label', 'Удалить пресет');
    delBtn.textContent = 'x';
    delBtn.addEventListener('click', (e) => {
        e.stopPropagation();
        this._onDelete(preset.id);
    });
    item.appendChild(delBtn);

    // ===== Click on item =====
    item.addEventListener('click', () => this._onSelect(preset.id));

    return item;
}

// =====
// API
// =====

async _loadPresets() {
    console.log('[LLMPresets] _loadPresets()');

    try {
        const fetchJson = window.coreEngine?.fetchJson;
        const result = await fetchJson(this._apiBase);

        if (result.success) {
            this.presets = result.data || [];
            console.log('[LLMPresets] Loaded presets: ${this.presets.length}`);

            // Detailed log for debugging CSS/html presence
            this.presets.forEach(p => {
                const htmlLen = (p.html || '').length;
                const cssLen = (p.css || '').length;
                console.log('[LLMPresets]   [{p.id}] "${p.name}" - html:${htmlLen} css:${cssLen}`);
            });
        }
    }
}

```

```

    });
  } else {
    console.warn('[LLMPresets] Response without success:', result);
    this.presets = [];
  }
} catch (error) {
  console.error('[LLMPresets] List load error:', error);
  this.presets = [];
}

this._renderList();
}

async _createPreset(name, description = '', html = '', css = '') {
  console.log('[LLMPresets] _createPreset()', {
    name,
    description,
    htmlLen: (html || '').length,
    cssLen: (css || '').length,
  });

  const fetchJson = window.coreEngine?.fetchJson;
  const result = await fetchJson(this._apiBase, {
    method: 'POST',
    body: { name, description, html, css },
  });

  console.log('[LLMPresets] _createPreset() result:', {
    id: result.data?.id,
    htmlLen: (result.data?.html || '').length,
    cssLen: (result.data?.css || '').length,
  });

  return result.data;
}

async _deletePreset(id) {
  const fetchJson = window.coreEngine?.fetchJson;
  await fetchJson(`${this._apiBase}/${id}`, {
    method: 'DELETE',
  });
}

async _updatePreset(id, data) {
  console.log('[LLMPresets] _updatePreset()', {
    id,
    htmlLen: (data.html || '').length,
    cssLen: (data.css || '').length,
  });

  const fetchJson = window.coreEngine?.fetchJson;
  const result = await fetchJson(`${this._apiBase}/${id}`, {
    method: 'PUT',
    body: data,
  });
}

```

```

        return result.data;
    }

    async _uploadThumbnail(id, blob) {
        const formData = new FormData();
        formData.append('file', blob, `preset-${id}.png`);

        const fetchJson = window.coreEngine?.fetchJson;
        const result = await fetchJson(`${this._apiBase}/${id}/thumbnail`, {
            method: 'POST',
            body: formData,
        });

        return result.data;
    }

    // =====
    // HELPERS – extract CSS from HTML
    // =====

    /**
     * Extract CSS from <style>...</style> blocks inside HTML.
     * Returns { html: string, css: string }.
     *
     * Used as a fallback when GrapesJS CsxComposer is empty
     * (e.g. when the page was loaded via setComponents(html) and
     * the <style> block was never pushed to the CsxComposer).
     */
    _extractCssFromHtml(html) {
        if (!html) return { html: '', css: '' };

        const matches = [...html.matchAll(/<style[^>]*>([\s\S]*?)<\/style>/gi)];
        const css = matches.map(m => m[1]).join('\n').trim();
        const htmlClean = html.replace(/<style[^>]*>[\s\S]*?<\/style>/gi, '').trim();

        return { html: htmlClean, css };
    }

    /**
     * Get current canvas HTML + CSS.
     *
     * Priority:
     * 1. CsxComposer (editor.getCss()) – if it has rules.
     * 2. <style> inside HTML – fallback.
     *
     * Returns { html, css }.
     */
    _getCurrentHtmlCss() {
        const ed = this.editor.editor;
        if (!ed) return { html: '', css: '' };

        const rawHtml = ed.getHtml() || '';
        const rawCss = ed.getCss() || '';
        const cssRulesCount = ed.Csx?.getAll?.()?.length || 0;

        console.log('[LLMPresets] _getCurrentHtmlCss()', {

```



```

        htmlLen: rawHtml.length,
        cssLen: rawCss.length,
        cssRulesCount,
    });

    // If CssComposer has rules – use it as primary source
    if (rawCss.trim() && cssRulesCount > 0) {
        return { html: rawHtml, css: rawCss };
    }

    // Fallback: extract <style> from HTML
    const extracted = this._extractCssFromHtml(rawHtml);
    console.log('[LLMPresets] fallback: extracted from HTML', {
        htmlLen: extracted.html.length,
        cssLen: extracted.css.length,
    });
    return extracted;
}

// =====
// ACTIONS
// =====

async _onSelect(id) {
    console.log('[LLMPresets] _onSelect() id:', id);

    const preset = this.presets.find(p => p.id === id);
    if (!preset) {
        console.warn('[LLMPresets] Preset not found:', id);
        return;
    }

    console.log('[LLMPresets] preset data:', {
        name: preset.name,
        htmlLen: (preset.html || '').length,
        cssLen: (preset.css || '').length,
    });

    this.currentId = id;

    // Highlight active
    this._renderList();

    const ed = this.editor.editor;
    if (!ed) {
        console.warn('[LLMPresets] editor not available');
        return;
    }

    // Prepare html + css (with fallback)
    let html = preset.html || '';
    let css = preset.css || '';

    // If CSS empty but HTML has <style> – extract
    if (!css.trim() && html.includes('<style>')) {
        const extracted = this._extractCssFromHtml(html);

```

```

    html = extracted.html;
    css = extracted.css;
    console.log('[LLMPresets] CSS extracted from HTML (preset has no separate css)');
  }

  console.log('[LLMPresets] applying:', {
    htmlLen: html.length,
    cssLen: css.length,
  });

  try {
    // 1. Clear ALL existing CSS rules (otherwise they accumulate on switch)
    if (ed.Css?.clear) {
      ed.Css.clear();
      console.log('[LLMPresets] CssComposer cleared');
    }

    // 2. Set HTML components
    ed.setComponents(html || '');
    console.log('[LLMPresets] components set');

    // 3. Apply CSS via CssComposer
    if (css && css.trim()) {
      if (ed.Css?.addRules) {
        ed.Css.addRules(css);
        console.log('[LLMPresets] CSS rules added via Css.addRules');
      } else {
        // Legacy fallback
        ed.setStyle(css);
        console.log('[LLMPresets] CSS applied via setStyle (legacy)');
      }
    }

    // 4. Verify
    const verifyCss = ed.getCss();
    console.log('[LLMPresets] verify – new getCss():', verifyCss.slice(0, 300));
  } catch (e) {
    console.error('[LLMPresets] apply preset error:', e);
  }
}

async _onCreate() {
  console.log('[LLMPresets] _onCreate()');

  const name = prompt('Название пресета:', 'Новый пресет');
  if (!name || !name.trim()) return;

  // Get current canvas HTML + CSS (with fallback)
  const { html, css } = this._getCurrentHtmlCss();

  console.log('[LLMPresets] _onCreate() – will save:', {
    name: name.trim(),
    htmlLen: html.length,
    cssLen: css.length,
    cssPreview: css.slice(0, 200),
  });
}

```

```

    try {
      const preset = await this._createPreset(name.trim(), '', html, css);

      console.log('[LLMPresets] _onCreate() - response:', {
        id: preset.id,
        htmlLen: (preset.html || '').length,
        cssLen: (preset.css || '').length,
      });

      // Add to local list
      this.presets.push(preset);
      this._renderList();

      console.log('[LLMPresets] Preset created:', preset.id);
    } catch (error) {
      console.error('[LLMPresets] Create error:', error);
      alert(`Ошибка: ${error.message}`);
    }
  }
}

async _onDelete(id) {
  console.log('[LLMPresets] _onDelete() id:', id);

  const preset = this.presets.find(p => p.id === id);
  if (!preset) return;

  if (!confirm(`Удалить пресет «${preset.name}»?`)) return;

  try {
    await this._deletePreset(id);

    // Remove from local list
    this.presets = this.presets.filter(p => p.id !== id);

    if (this.currentId === id) {
      this.currentId = null;
    }

    this._renderList();

    console.log('[LLMPresets] Preset deleted:', id);
  } catch (error) {
    console.error('[LLMPresets] Delete error:', error);
    alert(`Ошибка: ${error.message}`);
  }
}

/**
 * Save current canvas HTML + CSS to the selected preset.
 * Called from toolbar (Save) or from chat.
 */
async saveCurrentHtml() {
  if (!this.currentId) {
    console.log('[LLMPresets] No preset selected - skip saving');
    return;
  }
}

```

```

    }

    const { html, css } = this._getCurrentHtmlCss();

    console.log('[LLMPresets] saveCurrentHtml() – will save:', {
      currentId: this.currentId,
      htmlLen: html.length,
      cssLen: css.length,
    });

    try {
      await this._updatePreset(this.currentId, { html, css });

      // Update locally
      const preset = this.presets.find(p => p.id === this.currentId);
      if (preset) {
        preset.html = html;
        preset.css = css;
      }

      console.log('[LLMPresets] HTML+CSS saved to preset:', this.currentId);
    } catch (error) {
      console.error('[LLMPresets] Save HTML+CSS error:', error);
    }
  }

  destroy() {
    console.log('[LLMPresets] destroy()');
    this.rootEl = null;
    this.listEl = null;
    this.presets = [];
    this.currentId = null;
  }
}

```

```

=====
FILE: core/engine/lib/word/llm/chat/api.js
=====

```

```

// neurocad/core/engine/lib/word/llm/chat/api.js

```

```

/**
 * LLMChatAPI – HTTP part of the chat.
 *
 * Only GETs for history and active run; one DELETE for clearing
 * the chat history of a page.
 *
 * HTTP goes through window.coreEngine.fetchJson
 * (loaded once by CoreEngine.loadApi()).
 *
 * Note: this module is defensive – it returns null on any error
 * instead of throwing, so the chat UI can degrade gracefully.
 */
export class LLMChatAPI {
  constructor(apiBase) {
    this._apiBase = apiBase;
  }
}

```

```

// =====
// HISTORY
// =====

async loadHistory(pageId) {
  const url = `${this._apiBase}/chat/${pageId}/history`;
  console.log('[LLMChatAPI] GET', url);

  try {
    const fetchJson = window.coreEngine?.fetchJson;
    const data = await fetchJson(url);

    if (data && data.success && Array.isArray(data.data)) {
      return data.data;
    }

    console.warn('[LLMChatAPI] history: unexpected payload', data);
    return null;
  } catch (e) {
    console.error('[LLMChatAPI] history error:', e);
    return null;
  }
}

async clearHistory(pageId) {
  const url = `${this._apiBase}/chat/${pageId}/history`;
  console.log('[LLMChatAPI] DELETE', url);

  try {
    const fetchJson = window.coreEngine?.fetchJson;
    const data = await fetchJson(url, { method: 'DELETE' });
    return data?.deleted ?? 0;
  } catch (e) {
    console.error('[LLMChatAPI] clear error:', e);
    return null;
  }
}

// =====
// ACTIVE RUN
// =====

async checkActiveRun(pageId) {
  const url = `${this._apiBase}/chat/${pageId}/run/active`;
  console.log('[LLMChatAPI] GET', url);

  try {
    const fetchJson = window.coreEngine?.fetchJson;
    const data = await fetchJson(url);
    return data?.data || null;
  } catch (e) {
    console.warn('[LLMChatAPI] active run error:', e);
    return null;
  }
}

```

```
}
```

```
=====
FILE: core/engine/lib/word/llm/chat/apply.js
=====
```

```
// neurocad/core/engine/lib/word/llm/chat/apply.js
```

```
/**
```

```
 * applyLLMChatElementUpdate – replace a single element on the canvas.
```

```
 *
```

```
 * The server sends:
```

```
 * {
```

```
 *   "type": "element_update",
```

```
 *   "selector": "sel-abc12345",
```

```
 *   "html": "<section class='section hero effect-y3k9' data-selected-id='sel-abc12345'>...</section>"
```

```
 * }
```

```
 *
```

```
 * We find the component whose root element carries
```

```
 * `data-selected-id="sel-abc12345"` and replace it with the new HTML.
```

```
 *
```

```
 * Safety rules:
```

```
 * - Never call editor.setComponents() as a fallback. It would wipe  
 *   the whole canvas and replace it with a single element.
```

```
 * - Refuse to replace if the incoming HTML does not look like HTML  
 *   (for example, a plain error message from the LLM).
```

```
 * - Refuse to replace if the incoming HTML does not carry the  
 *   marker – otherwise the element would become untraceable on the  
 *   next update.
```

```
 * - The marker attribute `data-selected-id` is REMOVED from the new  
 *   HTML before inserting it, so the canvas is left clean.
```

```
 */
```

```
export function applyLLMChatElementUpdate(editor, selector, html) {
```

```
  if (!selector) {
```

```
    console.warn('[apply] empty selector, skipping');
```

```
    return;
```

```
  }
```

```
  if (!html) {
```

```
    console.warn('[apply] empty html, skipping');
```

```
    return;
```

```
  }
```

```
  const ed = editor?.editor;
```

```
  if (!ed) {
```

```
    console.error('[apply] editor.editor is null');
```

```
    return;
```

```
  }
```

```
  // ---- 1. html must look like html ----
```

```
  const trimmed = String(html).trim();
```

```
  if (!trimmed.startsWith('<')) {
```

```
    console.error(
```

```
      `[apply] html does not look like HTML – refusing to replace. ` +
```

```
      `First 80 chars: ${JSON.stringify(trimmed.slice(0, 80))}`
```

```
    );
```

```
    return;
```

```
  }
```

```

// ---- 2. html must contain the marker ----
if (!htmlContainsMarker(html, selector)) {
  console.error(
    `[apply] html does not contain marker "${selector}" - ` +
    `refusing to replace the element`
  );
  return;
}

// ---- 3. find the target on the canvas ----
const target = findElementByMarker(ed, selector);
if (!target) {
  console.error(
    `[apply] element "${selector}" not found on canvas. ` +
    `Nothing replaced. This usually means the marker was removed ` +
    `from the HTML by the LLM.`
  );
  return;
}

console.log(`[apply] replacing element "${selector}"`);

const parent = target.parent();
if (!parent) {
  console.error(`[apply] target has no parent, cannot replace`);
  return;
}

const index = target.index();

// Clean the marker attribute from the incoming HTML.
const cleanHtml = stripMarker(html, selector);

try {
  // Remove the old component, insert the new one at the same index.
  parent.components().remove(target);
  parent.components().add(cleanHtml, { at: index });
  console.log(`[apply] element "${selector}" replaced`);
} catch (e) {
  console.error(`[apply] replace failed:`, e);
}
}

// =====
// INTERNAL
// =====

/**
 * True if the html string contains data-selected-id="<selector>"
 * in any of the supported quote forms.
 */
function htmlContainsMarker(html, selector) {
  if (!html || !selector) return false;

  const escaped = escapeRegex(selector);

```

```

    // double or single quotes
    const quoted = new RegExp(
        `data-selected-id\\s*=\\s*["']${escaped}["']`
    );
    if (quoted.test(html)) return true;

    // unquoted (stops at whitespace, > or /)
    const unquoted = new RegExp(
        `data-selected-id\\s*=\\s*${escaped}(?=[\\s|>|/])`
    );
    if (unquoted.test(html)) return true;

    return false;
}

/**
 * Find a component whose root element has `data-selected-id="<selector>"`.
 * Walks the whole component tree from the wrapper.
 */
function findElementByMarker(ed, selector) {
    const wrapper = ed.getWrapper?();
    if (!wrapper) return null;

    const stack = [wrapper];
    while (stack.length) {
        const node = stack.pop();
        const attrs = node.getAttributes?() || {};
        if (attrs['data-selected-id'] === selector) {
            return node;
        }
        const children = node.components?();
        if (children && children.length) {
            for (let i = 0; i < children.length; i++) {
                stack.push(children.at(i));
            }
        }
    }
    return null;
}

/**
 * Remove the data-selected-id attribute from the html string.
 *
 * Matches both:
 * data-selected-id="sel-abc12345"
 * data-selected-id='sel-abc12345'
 * data-selected-id=sel-abc12345
 */
function stripMarker(html, selector) {
    if (!html || !selector) return html;

    const escaped = escapeRegex(selector);

    const patterns = [
        // double or single quotes

```



```

        new RegExp(
            `\\s+data-selected-id\\s*=\\s*["']${escaped}["']`,
            'g'
        ),
        // unquoted
        new RegExp(
            `\\s+data-selected-id\\s*=\\s*${escaped}(?=\\s|>|/)` ,
            'g'
        ),
    ];

    let out = html;
    for (const re of patterns) {
        out = out.replace(re, '');
    }
    return out;
}

function escapeRegex(s) {
    return String(s).replace(/[\.*+?^${}()|\\]/g, '\\$&');
}

=====
FILE: core/engine/lib/word/llm/chat/catalog.js
=====
// neurocad/core/engine/lib/word/llm/chat/catalog.js

/**
 * buildLLMChatCatalog – collect the block catalog from GrapesJS.
 *
 * Pure function: editor → [{id, label, category, html}].
 * Knows nothing about WS, UI, or HTTP.
 *
 * NOTE: this catalog is for HTML blocks (elements, layout, ready),
 * NOT for effects. The list of existing effect ids is collected
 * separately in llm/chat/edit.js from EffectBlocks._blocks.
 *
 * Excluded categories
 * -----
 * Blocks from the "Разметка" (layout) category – grids, containers,
 * flex-shell – are NOT sent to the backend. The LLM must compose pages
 * from ready-made sections and elements only; it must not use raw
 * layout blocks. This is enforced HERE, at the source, so:
 *   - they never reach the WS payload (less traffic);
 *   - the backend never has to filter them out of the prompt.
 *
 * The backend still keeps its own filter (llm/prompts/plan.py) as a
 * belt-and-suspenders safeguard, in case some other caller builds a
 * catalog without going through this function.
 */

//: Categories that must NOT be sent to the LLM on the planning step.
const EXCLUDED_CATEGORIES = new Set([
    'Разметка',
]);

```

```

export function buildLLMChatCatalog(editor) {
  const ed = editor.editor;
  if (!ed) {
    console.warn('[catalog] editor.editor is null');
    return [];
  }

  const catalog = [];
  const blocks = ed.BlockManager.getAll();
  console.log(`[catalog] BlockManager has ${blocks.length} blocks`);

  blocks.forEach((b) => {
    const rawCat = b.get('category');
    let category = 'other';
    if (typeof rawCat === 'string') {
      category = rawCat;
    } else if (rawCat && typeof rawCat === 'object') {
      category = rawCat.id || rawCat.get?('.')('label') || 'other';
    }

    // ---- Exclude layout blocks from the LLM catalog ----
    if (EXCLUDED_CATEGORIES.has(category)) {
      return;
    }

    const content = b.get('content');
    if (typeof content !== 'string') {
      // Component-type block – skipped
      return;
    }

    const html = content.trim();
    if (!html) return;

    catalog.push({
      id: b.id,
      label: b.get('label') || b.id,
      category,
      html,
    });
  });

  console.log(`[catalog] catalog for LLM: ${catalog.length} blocks ` +
    `(excluded categories: ${[...EXCLUDED_CATEGORIES].join(', ') || '-'})`);

  return catalog;
}

```

```

=====
FILE: core/engine/lib/word/llm/chat/create.js
=====

```

```

// neurocad/core/engine/lib/word/llm/chat/create.js

```

```

/**
 * CreateSession – the "effect create" mode of LLMChat.
 *

```

```

* Owned by LLMChat (one instance per chat). Handles everything that
* happens when the user clicks "+" in the "Эффекты" category header
* (editor/effects/index.js):
*
* 1. `bind()` subscribes to three editor events:
*     word:effect-create-session-started
*         - EffectBlocks created a pending
*           block (user clicked "+")
*     word:effect-create-save      - user clicked the save icon
*                                   on the pending block
*     word:effect-create-cancel   - user clicked the close icon
*                                   on the pending block
*
* NOTE: we deliberately do NOT listen on `word:effect-create-start`.
* That name is the *inbound* event for EffectBlocks itself
* (header.js → index.js → onCreateClick). If the chat subscribed
* to it *and* EffectBlocks re-emitted it after creating a pending
* block, the two would form an infinite recursion.
*
* 2. On `start`:
*     - reset `_state = { draft: null, hintShown: false }`;
*     - greet the user in the chat.
*     (The pending block in the palette is created by
*      EffectBlocks itself, not here.)
*
* 3. `send(text)` is called by LLMChat._onSend when this session
*     is active. It forwards `{ type: "create_effect", ... }` over
*     the WS, passing the previous draft (if any) so the LLM
*     revises instead of starting from scratch. It also passes
*     `existing_effect_ids` – the list of ids already registered
*     in the palette – so the server can pass it to the LLM and
*     avoid generating a colliding id.
*
* 4. `handleDraft(draft)` is called by the WS handler when
*     `effect_draft` arrives. It:
*     - stores `draft` in `_state.draft`;
*     - applies the draft CSS to the iframe
*       (live.js → applyDraft on a local id);
*     - applies the effect class to the selected element
*       (with UndoManager.skip);
*     - calls EffectBlocks.applyDraftToPendingBlock(draft) so the
*       pending block in the palette gets draft.media and enables
*       its save button.
*
* 5. On `save`:
*     - POST /editor/effects with the current draft;
*     - addEffectLink (real <link> in the iframe);
*     - commitDraft (remove the draft <style>, bump ?v=);
*     - EffectBlocks.finalizePending(effect) – converts the
*       pending block into a normal effect block;
*     - clear state, show a short success message.
*
* Collision handling:
*     - BEFORE the POST we check the local palette. If the draft
*       id is already there, we do not even try – we show a clear
*       message and keep the pending block, so the user can ask

```

```

*         for a different name.
*     - If the server still rejects the POST (409 Conflict), we
*       surface the server's `detail` string instead of the
*       generic "сервер отклонил запрос".
*
* 6. On `cancel`:
*     - revertDraft (remove the draft <style>);
*     - remove the effect class from the selected element;
*     - EffectBlocks.cancelPending() – remove the pending block;
*     - clear state, show "Создание эффекта отменено".
*
* All HTTP calls and UI updates go through the parent chat:
*   this.chat.ui.addMessage(...)
*   this.chat.ws.send(...)
*   this.chat.editor.editor
*   this.chat._applyEffectDraft / _commitEffectDraft / _revertEffectDraft
*   this.chat._addEffectLink
*   this.chat._effectBlocks()
*
* No static imports. Instantiated by LLMChat._loadModules.
*/
export class CreateSession {
  /**
   * @param {LLMChat} chat – parent chat instance
   */
  constructor(chat) {
    this.chat = chat;

    // { draft: { id, label, hint, css, media } | null, hintShown: bool }
    // or null when inactive.
    this._state = null;

    // Bound editor handlers – for unbind().
    this._onStart = null;
    this._onSave = null;
    this._onCancel = null;
  }

  // =====
  // LIFECYCLE
  // =====

  /**
   * Subscribe to editor events. Called once from LLMChat.init().
   */
  bind() {
    const ed = this.chat.editor?.editor;
    if (!ed) return;

    this._onStart = () => {
      this._handleStart();
    };
    this._onSave = () => {
      this._handleSave();
    };
    this._onCancel = () => {

```

```

        this._handleCancel();
    };

    // NOTE: `word:effect-create-start` is NOT listened to here –
    // it is the inbound event for EffectBlocks. We listen on
    // `word:effect-create-session-started`, which EffectBlocks
    // emits *after* the pending block has been created.
    ed.on('word:effect-create-session-started', this._onStart);
    ed.on('word:effect-create-save', this._onSave);
    ed.on('word:effect-create-cancel', this._onCancel);
}

/**
 * Unsubscribe. Called from LLMChat.destroy().
 */
unbind() {
    const ed = this.chat.editor?.editor;
    if (ed) {
        if (this._onStart) {
            try { ed.off('word:effect-create-session-started', this._onStart); } catch (_) {}
        }
        if (this._onSave) {
            try { ed.off('word:effect-create-save', this._onSave); } catch (_) {}
        }
        if (this._onCancel) {
            try { ed.off('word:effect-create-cancel', this._onCancel); } catch (_) {}
        }
    }
    this._onStart = null;
    this._onSave = null;
    this._onCancel = null;
}

/**
 * True if a create session is currently active.
 */
isActive() {
    return !!this._state;
}

/**
 * Current state, or null.
 * Shape: { draft, hintShown }
 */
getState() {
    return this._state;
}

// =====
// CREATE START
// =====

_handleStart() {
    console.log('[LLMChat] effect-create-session-started');

    // Reset any previous draft state.

```

```

    this._state = { draft: null, hintShown: false };

    this.chat.ui.addMessage({
        role: 'assistant',
        content: 'Опишите, какой эффект хотите для выделенного элемента, например: «мерцающие точки по фону» или «градиент',
нажмите дискету на новом блоке в палитре.',
        created_at: new Date().toISOString(),
    });
}

// =====
// SEND (called by LLMChat._onSend)
// =====

/**
 * Send the user's message to the LLM as a "create_effect" request.
 * LLMChat._onSend already added the user message to the UI and
 * set the sending state.
 */
send(text) {
    if (!this._state) return;

    const payload = {
        type: 'create_effect',
        message: text,
        previous_draft: this._state.draft || null,
        existing_effect_ids: this._collectExistingIds(),
    };

    this.chat.debug.dumpPayload?.(payload);

    if (this.chat.ws.isOpen()) {
        console.log('[LLMChat] sending create_effect over open WS');
        this.chat.ws.send(payload);
    } else {
        console.warn('[LLMChat] WS not ready, queueing create_effect');
        this.chat._pendingStart = payload;
        this.chat.ws.connect();
    }
}

/**
 * Collect the ids of all registered effects in the palette.
 * Passed to the server so it can pass them to the LLM and avoid
 * generating a colliding id.
 *
 * @returns {string[]}
 */
_collectExistingIds() {
    try {
        const fx = this.chat._effectBlocks?();
        if (!fx || !fx._blocks) return [];
        const ids = Array.from(fx._blocks.keys());
        console.log('[LLMChat] create: _collectExistingIds →', ids);
        return ids;
    } catch (e) {

```

```

        console.warn('[LLMChat] create: _collectExistingIds failed', e);
        return [];
    }
}

// =====
// DRAFT (called by handler via chat.js)
// =====

/**
 * Handle a draft coming from the WS (handler → effect_draft).
 *
 * The draft is applied IMMEDIATELY to the selected element:
 *
 * 1. draft CSS → <style id="fx-draft-<id>"> in the iframe;
 * 2. effect class → selected component, with undo skipped;
 * 3. pending block in the palette → applyDraftToPendingBlock.
 *
 * The user judges the result by looking at the element. No CSS
 * is shown in the chat. The agent's `assistant_message` already
 * tells the user what to do; we do not duplicate it here.
 */
handleDraft(draft) {
    console.log('[LLMChat] handleDraft: enter, draft=', draft ? draft.id : null);
    if (!draft) {
        console.warn('[LLMChat] handleDraft: draft is null – aborting');
        return;
    }

    if (!this._state) {
        console.warn('[LLMChat] handleDraft: no active create state – auto-enabling');
        // Auto-enable create mode if a draft arrives without a
        // prior "start" event (rare but safe).
        this._state = { draft: null, hintShown: false };
    }

    this._state.draft = draft;
    console.log('[LLMChat] handleDraft: state.draft stored:', draft.id);

    const ed = this.chat.editor.editor;

    // ---- 1. Apply the draft CSS to the iframe ----
    try {
        this.chat._applyEffectDraft?.(ed, draft.id, draft.css);
        console.log('[LLMChat] create-effect: draft CSS applied to iframe:', draft.id);
    } catch (e) {
        console.warn('[LLMChat] applyDraft failed:', e);
    }

    // ---- 2. Apply the effect class to the selected element ----
    try {
        const selected = ed?.getSelected?.();
        if (selected) {
            const classes = selected.getClasses() || [];
            const next = classes.filter(c => !c.startsWith('fx-'));
            next.push(draft.id);

```

```

        const um = ed.UndoManager;
        if (um && typeof um.skip === 'function') {
            um.skip(() => selected.setClass(next));
        } else {
            selected.setClass(next);
        }
        console.log('[LLMChat] create-effect: draft class applied to element:', draft.id);
    } else {
        console.warn('[LLMChat] create-effect: no selected element to apply draft to');
    }
} catch (e) {
    console.warn('[LLMChat] apply draft to element failed:', e);
}

// ---- 3. Update the pending block in the palette ----
try {
    const fx = this.chat._effectBlocks();
    console.log('[LLMChat] handleDraft: _effectBlocks() =', fx);
    if (!fx) {
        console.error('[LLMChat] handleDraft: _effectBlocks() returned null -
save button will stay disabled');
    } else if (typeof fx.applyDraftToPendingBlock !== 'function') {
        console.error(
            '[LLMChat] handleDraft: _effectBlocks() has no applyDraftToPendingBlock; keys:',
            Object.keys(fx)
        );
    } else {
        console.log('[LLMChat] handleDraft: → applyDraftToPendingBlock', draft.id);
        fx.applyDraftToPendingBlock(draft);
        console.log('[LLMChat] handleDraft: ← applyDraftToPendingBlock returned');
    }
} catch (e) {
    console.error('[LLMChat] handleDraft: applyDraftToPendingBlock threw:', e);
}

if (!this._state.hintShown) {
    this._state.hintShown = true;
}
}

// =====
// SAVE / CANCEL
// =====

/**
 * User clicked the save icon on the pending block.
 * Emitted as `word:effect-create-save` from EffectBlocks.
 */
async _handleSave() {
    console.log('[LLMChat] effect-create-save');

    if (!this._state || !this._state.draft) {
        console.warn('[LLMChat] effect-create-save: no draft');
        return;
    }
}

```



```

    this.chat.ui.showTyping();
    this.chat._running = true;
    this.chat.ui.setSendingState(true);

    try {
      await this._saveNewEffect(this._state.draft);
    } finally {
      this.chat._running = false;
      this.chat.ui.setSendingState(false);
      this.chat.ui.hideTyping();
    }
  }
}

/**
 * User clicked the close icon on the pending block.
 * Emitted as `word:effect-create-cancel` from EffectBlocks.
 *
 * Removes the draft <style> from the iframe, removes the effect
 * class from the selected element, and removes the pending block.
 */
_handleCancel() {
  console.log('[LLMChat] effect-create-cancel');

  const draftId = this._state?.draft?.id;

  // ---- 1. Remove the draft CSS ----
  if (draftId) {
    try {
      this.chat._revertEffectDraft?.(this.chat.editor.editor, draftId);
    } catch (e) {
      console.warn('[LLMChat] revertDraft failed:', e);
    }
  }

  // ---- 2. Remove the effect class from the selected element ----
  try {
    const ed = this.chat.editor.editor;
    const selected = ed?.getSelected?.();
    if (selected) {
      const classes = selected.getClasses() || [];
      const next = classes.filter(c => !c.startsWith('fx-'));
      if (next.length !== classes.length) {
        const um = ed.UndoManager;
        if (um && typeof um.skip === 'function') {
          um.skip(() => selected.setClass(next));
        } else {
          selected.setClass(next);
        }
      }
    }
  } catch (e) {
    console.warn('[LLMChat] remove draft class failed:', e);
  }

  // ---- 3. Remove the pending block ----

```

```

    try {
      const fx = this.chat._effectBlocks();
      console.log('[LLMChat] _handleCancel: _effectBlocks() =', fx);
      fx?.cancelPending();
    } catch (e) {
      console.warn('[LLMChat] cancelPending failed:', e);
    }

    // ---- 4. Clear state ----
    this._state = null;

    this.chat.ui.addMessage({
      role: 'assistant',
      content: 'Создание эффекта отменено.',
      created_at: new Date().toISOString(),
    });
  }

  /**
   * POST /editor/effects with the current draft.
   *
   * Flow:
   * 0. LOCAL COLLISION CHECK – if the draft id is already
   *    registered in the palette, show a clear message and keep
   *    the pending block (the user can ask for a different name).
   * 1. POST the draft.
   * 2. On success:
   *    - addEffectLink (real <link> in the iframe);
   *    - commitDraft (remove the draft <style>, bump ?v=);
   *    - EffectBlocks.finalizePending(effect) – converts the
   *      pending block into a normal effect block;
   *    - clear state, show success message.
   * 3. On failure:
   *    - extract the most specific error text we can
   *      (server `detail`, fetch error message, generic);
   *    - show it in the chat;
   *    - keep the pending block and the state, so the user can
   *      rephrase and try again.
   */
  async _saveNewEffect(draft) {
    const base = this.chat._effectsApiBase;
    const qs = this.chat._qsForEffects();
    const url = `${base}${qs}`;
    console.log('[LLMChat] POST', url, draft.id);

    // ---- 0. Local collision check ----
    if (this._idExistsLocally(draft.id)) {
      console.warn('[LLMChat] saveNewEffect: local collision for', draft.id);
      this.chat.ui.addMessage({
        role: 'error',
        content:
          `⚠ Имя «${draft.id}» уже занято другим эффектом в палитре. ` +
          `Опишите эффект чуть иначе – я предложу другое имя.`,
        created_at: new Date().toISOString(),
      });
      return false;
    }
  }

```

```

}

// ---- 1. POST ----
let ok = false;
let data = null;
let errText = '';

try {
  const fetchJson = window.coreEngine?.fetchJson;
  data = await fetchJson(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      id: draft.id,
      label: draft.label,
      hint: draft.hint || '',
      css: draft.css,
      media: draft.media || '',
    }),
  });
  ok = !!(data && data.success);
} catch (e) {
  console.error('[LLMChat] saveNewEffect error:', e);
  errText = this._extractErrorText(e);
}

if (!ok) {
  const reason =
    errText
    || (data && (data.detail || data.message))
    || 'сервер отклонил запрос';
  console.warn('[LLMChat] saveNewEffect: failed -', reason);
  this.chat.ui.addMessage({
    role: 'error',
    content: `⚠ Не удалось сохранить эффект: ${reason}.`,
    created_at: new Date().toISOString(),
  });
  return false;
}

// ---- 2. Inject the real <link> and drop the draft <style> ----
try {
  this.chat._addEffectLink?.(this.chat.editor.editor, draft.id);
  this.chat._commitEffectDraft?.(this.chat.editor.editor, draft.id);
} catch (e) {
  console.warn('[LLMChat] addEffectLink/commitDraft failed:', e);
}

// ---- 3. Convert the pending block into a normal effect block ----
try {
  const fx = this.chat._effectBlocks();
  console.log('[LLMChat] _saveNewEffect: _effectBlocks() =', fx);
  fx?.finalizePending({
    id: draft.id,
    label: draft.label,
    hint: draft.hint || draft.label,
  });
}

```

```

        media: draft.media || '',
        builtin: false,
    });
} catch (e) {
    console.warn('[LLMChat] finalizePending failed:', e);
}

// ---- 4. Clear state and notify ----
this._state = null;

this.chat.ui.addMessage({
    role: 'assistant',
    content: 'Эффект сохранён и добавлен в библиотеку.',
    created_at: new Date().toISOString(),
});

return true;
}

// =====
// HELPERS
// =====

/**
 * True if the given id is already registered in the palette.
 */
_idExistsLocally(id) {
    if (!id) return false;
    try {
        const fx = this.chat._effectBlocks?();
        if (!fx || !fx._blocks) return false;
        return fx._blocks.has(id);
    } catch (_) {
        return false;
    }
}

/**
 * Extract a human-readable message from an error thrown by
 * `fetchJson`.
 *
 * Different fetch wrappers put the useful text in different
 * places:
 *
 * - `e.message` – usually the most informative thing we get
 *   from a non-2xx response when the wrapper re-throws with
 *   the server's `detail`;
 *
 * - `e.response.detail` / `e.data.detail` – fallbacks for
 *   wrappers that attach the parsed body;
 *
 * - `e.toString()` – last resort.
 */
_extractErrorText(e) {
    if (!e) return '';
    if (typeof e === 'string') return e;

    const direct = e.message;
    if (direct && typeof direct === 'string') return direct;

```

```

    const fromResponse =
      (e.response && (e.response.detail || e.response.message))
      || (e.data && (e.data.detail || e.data.message));
    if (fromResponse) return String(fromResponse);

    try {
      return String(e);
    } catch (_) {
      return '';
    }
  }
}

```

```

=====
FILE: core/engine/lib/word/llm/chat/debug.js
=====
// neurocad/core/engine/lib/word/llm/chat/debug.js

/**
 * LLMChatDebug – debug dumps for the browser console.
 *
 * Prints environment, blocks, selection, and payload. Does not
 * compute anything – only formats and logs.
 */
export class LLMChatDebug {
  constructor(editor) {
    this.editor = editor;
  }

  // =====
  // ENV
  // =====

  dumpEnv() {
    console.group('[LLMChat][debug] environment');
    console.log('pageId:', this.editor.pageId);

    const auth = window.coreEngine?.auth;
    console.log('auth object:', auth);
    if (auth?.isSuperadmin) console.log('auth.isSuperadmin():', auth.isSuperadmin());
    if (auth?.getUser) console.log('auth.getUser():', auth.getUser());

    console.log('document.cookie:', document.cookie || '(empty)');
    console.log(
      'cookie names (visible):',
      (document.cookie || '')
        .split(';')
        .map(s => s.trim().split('=')[0])
        .filter(Boolean)
    );

    console.log('sessionStorage keys:', Object.keys(sessionStorage));
    console.log('localStorage keys:', Object.keys(localStorage));

    console.groupEnd();
  }
}

```

```

}

// =====
// BLOCKS
// =====

/**
 * Print the block catalog as a single object – expandable in the
 * console with a click. No tables, no loops.
 */
dumpBlocks(catalog) {
  console.log(
    '%c[BLOCKS] ' + catalog.length + ' items ready to send:',
    'background:#222;color:#3b82f6;padding:2px 6px;font-weight:bold;border-radius:4px;'
  );
  console.log('[BLOCKS]', catalog);
}

// =====
// SELECTION
// =====

/**
 * Print the current selection in the editor.
 */
dumpSelection(selection) {
  console.log(
    '%c[SELECTION] current selection:',
    'background:#222;color:#0f0;padding:2px 6px;font-weight:bold;border-radius:4px;'
  );
  console.log('[SELECTION]', selection);
}

// =====
// PAYLOAD
// =====

dumpPayload(payload) {
  const str = JSON.stringify(payload, null, 2);

  console.log(
    '%c[PAYLOAD] SEND TO BACKEND',
    'background:#111;color:#0f0;padding:4px 8px;font-weight:bold;border-radius:4px;'
  );
  console.log(
    '%c[PAYLOAD] type      : ' + payload.type,
    'color:#3b82f6;font-weight:bold;'
  );
  console.log(
    '%c[PAYLOAD] message : ' + JSON.stringify(payload.message),
    'color:#3b82f6;font-weight:bold;'
  );
  console.log(
    '%c[PAYLOAD] catalog : ' + (payload.block_catalog?.length || 0) + ' blocks',
    'color:#3b82f6;font-weight:bold;'
  );
}

```

```

        console.log(
            '%c[PAYLOAD] selection : ' + JSON.stringify(payload.selection),
            'color:#3b82f6;font-weight:bold;'
        );
        console.log(
            '%c[PAYLOAD] size      : ' + str.length + ' chars (' +
                new Blob([str]).size + ' bytes)',
            'color:#3b82f6;font-weight:bold;'
        );
        console.log(
            '%c[PAYLOAD] RAW (JSON):',
            'background:#222;color:#3b82f6;padding:2px 6px;font-weight:bold;'
        );
        console.log(str);
    }
}

```

```

=====
FILE: core/engine/lib/word/llm/chat/edit.js
=====

```

```
// neurocad/core/engine/lib/word/llm/chat/edit.js
```

```

/**
 * EditSession – the "effect edit" mode of LLMChat.
 *
 * Two distinct user actions are handled here:
 *
 * EDIT (pencil→save flow)
 *   User clicks the pencil icon → session opens with a greeting.
 *   User describes changes in the chat → LLM returns new CSS.
 *   User clicks SAVE (diskette) → PUT /editor/effects/<id>.
 *
 * RENAME (pencil-icon, auto flow)
 *   User clicks the rename icon in the block.
 *   The chat sends an `effect_rename` request to the server.
 *   The server proposes a new label and a new SVG miniature.
 *   The client then RELABELS the block in the palette – the id
 *   and the CSS are NOT changed. No POST, no DELETE, no class
 *   swap on the element. The class on the canvas stays `.fx-<id>`
 *   exactly as it was, and the .css file stays on disk untouched.
 *   The only feedback is a single chat message:
 *   "Переименовано в «...»".
 *
 * Events handled (subscribed in bind()):
 *   word:effect-edit-start   – user clicked the edit icon
 *   word:effect-edit-save   – user clicked it again (save)
 *   word:effect-edit-cancel – cancelled programmatically
 *   word:effect-edit-rename – user clicked the rename icon
 *
 * No static imports. Instantiated by LLMChat._loadModules.
 */
export class EditSession {
    constructor(chat) {
        this.chat = chat;
        this._state = null;
        this._onStart = null;
    }

```

```

    this._onSave = null;
    this._onCancel = null;
    this._onRename = null;
}

// =====
// LIFECYCLE
// =====

bind() {
    const ed = this.chat.editor?.editor;
    console.log('[LLMChat] edit: bind(), editor present =', !!ed);
    if (!ed) return;

    this._onStart = async ({ id }) => {
        console.log('[LLMChat] edit: event word:effect-edit-start', { id });
        await this._handleStart(id);
    };
    this._onSave = async ({ id }) => {
        console.log('[LLMChat] edit: event word:effect-edit-save', { id });
        await this._handleSave(id);
    };
    this._onCancel = ({ id }) => {
        console.log('[LLMChat] edit: event word:effect-edit-cancel', { id });
        this._handleCancel(id);
    };
    this._onRename = async ({ id }) => {
        console.log('[LLMChat] edit: event word:effect-edit-rename', { id });
        await this._handleRename(id);
    };

    ed.on('word:effect-edit-start', this._onStart);
    ed.on('word:effect-edit-save', this._onSave);
    ed.on('word:effect-edit-cancel', this._onCancel);
    ed.on('word:effect-edit-rename', this._onRename);
    console.log('[LLMChat] edit: subscribed to 4 editor events');
}

unbind() {
    const ed = this.chat.editor?.editor;
    if (ed) {
        if (this._onStart) {
            try { ed.off('word:effect-edit-start', this._onStart); } catch (_) {}
        }
        if (this._onSave) {
            try { ed.off('word:effect-edit-save', this._onSave); } catch (_) {}
        }
        if (this._onCancel) {
            try { ed.off('word:effect-edit-cancel', this._onCancel); } catch (_) {}
        }
        if (this._onRename) {
            try { ed.off('word:effect-edit-rename', this._onRename); } catch (_) {}
        }
    }
    this._onStart = null;
    this._onSave = null;

```



```

    this._onCancel = null;
    this._onRename = null;
    console.log('[LLMChat] edit: unbind() done');
  }

  isActive() {
    return !!this._state;
  }

  getState() {
    return this._state;
  }

  // =====
  // EDIT START
  // =====

  async _handleStart(id, opts = {}) {
    const silent = !!opts.silent;
    console.log('[LLMChat] edit: _handleStart ENTER', { id, silent });

    const css = await this.chat._loadEffectCss(id);
    if (css == null) {
      this.chat.ui.addMessage({
        role: 'error',
        content: '⚠ Не удалось загрузить CSS эффекта.',
        created_at: new Date().toISOString(),
      });
      this.chat._effectBlocks()?.cancelEdit(id);
      return;
    }

    const currentLabel = this._lookupLabel(id);

    this._state = {
      id,
      css,
      newLabel: null,
      newMedia: null,
      currentLabel,
      hintShown: silent, // silent sessions never show the save hint
      renamePending: false,
    };

    // Only the interactive EDIT flow greets the user. The RENAME
    // flow is silent – it fires and forgets.
    if (!silent) {
      this.chat.ui.addMessage({
        role: 'assistant',
        content: 'Режим правки эффекта. Опишите, что хотите изменить. Когда результат устроит –  
нажмите иконку-дискету «Сохранить» в блоке эффекта.',
        created_at: new Date().toISOString(),
      });
    }
  }
}

```

```

_lookupLabel(id) {
  try {
    const fx = this.chat._effectBlocks?();
    if (!fx || !fx._blocks) return id;
    const entry = fx._blocks.get(id);
    return (entry && entry.title) || id;
  } catch (_) {
    return id;
  }
}

_collectExistingIds() {
  try {
    const fx = this.chat._effectBlocks?();
    if (!fx || !fx._blocks) return [];
    const ids = Array.from(fx._blocks.keys());
    console.log('[LLMChat] edit: _collectExistingIds →', ids);
    return ids;
  } catch (e) {
    console.warn('[LLMChat] edit: _collectExistingIds failed', e);
    return [];
  }
}

// =====
// SEND (called by LLMChat._onSend)
// =====

send(text) {
  console.log('[LLMChat] edit: send() called, text =', text);
  if (!this._state) {
    console.warn('[LLMChat] edit: send() but no _state – aborting');
    return;
  }

  const effectiveLabel = this._state.newLabel
    || this._state.currentLabel
    || this._state.id;

  const payload = {
    type: 'effect_edit',
    message: text,
    effect_id: this._state.id,
    effect_label: effectiveLabel,
    effect_css: this._state.css || '',
    existing_effect_ids: this._collectExistingIds(),
  };

  this.chat.debug.dumpPayload?.(payload);

  if (this.chat.ws.isOpen()) {
    this.chat.ws.send(payload);
  } else {
    this.chat._pendingStart = payload;
    this.chat.ws.connect();
  }
}

```

```

}

// =====
// RENAME – relabel only, no id/CSS change
// =====

/**
 * User clicked the "pencil" rename button on an active effect
 * block.
 *
 * This is a FIRE-AND-FORGET action:
 * 1. Open a SILENT session (no greeting, no hint).
 * 2. Send a WS request of type `effect_rename`.
 * 3. When the response arrives, `handleCss` RELABELS the block
 *    in the palette (label + media). The id and CSS stay as
 *    they were – nothing on the canvas is touched.
 * 4. One chat message confirms the rename.
 */
async _handleRename(id) {
  console.log('[LLMChat] edit: _handleRename ENTER', { id });

  // Cancel any other running session first.
  if (this._state && this._state.id !== id) {
    this._handleCancel(this._state.id);
  }

  // Open a silent session if not already open for this effect.
  if (!this._state || this._state.id !== id) {
    await this._handleStart(id, { silent: true });
  }

  if (!this._state || this._state.id !== id) {
    console.warn('[LLMChat] edit: _handleRename could not open a session for', id);
    return;
  }

  // Mark the session so handleCss knows to relabel on reply.
  this._state.renamePending = true;
  this._state.hintShown = true; // never show the save hint

  const effectiveLabel = this._state.currentLabel || id;

  const payload = {
    type: 'effect_rename',
    message: '',
    effect_id: id,
    effect_label: effectiveLabel,
    effect_css: this._state.css || '',
    existing_effect_ids: this._collectExistingIds(),
  };

  this.chat.debug.dumpPayload?.(payload);

  if (this.chat.ws.isOpen()) {
    console.log('[LLMChat] edit: sending effect_rename over WS');
    this.chat.ws.send(payload);
  }
}

```

```

    } else {
        console.warn('[LLMChat] edit: WS not ready, queueing effect_rename');
        this.chat._pendingStart = payload;
        this.chat.ws.connect();
    }
}

// =====
// CSS UPDATE (called by handler via chat.js)
// =====

handleCss(effectId, css, opts) {
    console.log('[LLMChat] edit: handleCss ENTER',
        { effectId, cssLen: css ? css.length : 0, opts });

    if (!effectId || !css) return;

    if (!this._state || effectId !== this._state.id) {
        console.warn('[LLMChat] edit: css_update for non-edited effect – aborting');
        return;
    }

    const wasRename = !!this._state.renamePending;
    this._state.renamePending = false;
    this._state.css = css;

    // ---- Apply the CSS draft under the same id ----
    try {
        this.chat._applyEffectDraft?.(this.chat.editor.editor, effectId, css);
    } catch (e) {
        console.error('[LLMChat] edit: _applyEffectDraft THREW', e);
    }

    const newLabel = (opts && opts.newLabel) ? String(opts.newLabel).trim() : '';
    const newMedia = (opts && opts.newMedia) ? String(opts.newMedia).trim() : '';

    // ---- RENAME PATH ----
    if (wasRename) {
        if (!newLabel) {
            this.chat.ui.addMessage({
                role: 'assistant',
                content: 'Не удалось подобрать новое название – попробуйте ещё раз.',
                created_at: new Date().toISOString(),
            });
            this._state = null;
            return;
        }

        this._state.newLabel = newLabel;
        this._state.newMedia = newMedia || null;

        // Relabel the block in the palette: update label, hint
        // and media. The id and the CSS are NOT touched.
        try {
            const fx = this.chat._effectBlocks?();
            if (fx && typeof fx.relabelOne === 'function') {

```

```

        fx.relabelOne(effectId, {
            label: newLabel,
            hint: newLabel,
            media: newMedia || undefined,
        });
        console.log('[LLMChat] edit: relabelOne applied',
            { id: effectId, label: newLabel,
              mediaLen: newMedia ? newMedia.length : 0 });
    } else {
        console.warn('[LLMChat] edit: EffectBlocks.relabelOne not available');
    }
} catch (e) {
    console.warn('[LLMChat] edit: relabelOne failed', e);
}

this._state = null;

this.chat.ui.addMessage({
    role: 'assistant',
    content: `Переименовано в «${newLabel}»`,
    created_at: new Date().toISOString(),
});
return;
}

// ---- NORMAL EDIT PATH ----
// A css_update without renamePending means the user asked to
// rewrite the CSS. Stash newLabel/newMedia if they came in
// (the edit agent may occasionally propose them), and wait
// for the save click. We do NOT auto-apply anything here.
if (newLabel) {
    this._state.newLabel = newLabel;
    this._state.newMedia = newMedia || null;
}

if (!this._state.hintShown) {
    this._state.hintShown = true;
    this.chat.ui.addMessage({
        role: 'assistant',
        content: 'Если результат устраивает – нажмите иконку-дискету «Сохранить» в блоке эффекта.',
        created_at: new Date().toISOString(),
    });
}
}

// =====
// SAVE / CANCEL – used only by the EDIT flow
// =====

async _handleSave(id) {
    console.log('[LLMChat] edit: _handleSave ENTER', { id });

    if (!this._state || this._state.id !== id) {
        console.warn('[LLMChat] edit: _handleSave no active edit for', id);
        return;
    }
}

```

```

const state = this._state;
const css = state.css || '';

// Save the CSS via PUT. We do NOT change the id, we do NOT
// touch the label or the media – this is purely a CSS update.
const ok = await this.chat._saveEffectCss(id, css);
if (!ok) {
  this.chat.ui.addMessage({
    role: 'error',
    content: '⚠ Не удалось сохранить эффект.',
    created_at: new Date().toISOString(),
  });
  return;
}

this.chat._commitEffectDraft?.(this.chat.editor.editor, id);
this._state = null;
this.chat._effectBlocks()?.finishEdit(id);

this.chat.ui.addMessage({
  role: 'assistant',
  content: 'Эффект сохранён.',
  created_at: new Date().toISOString(),
});
}

_handleCancel(id) {
  console.log('[LLMChat] edit: _handleCancel ENTER', { id });

  if (!this._state || this._state.id !== id) return;

  try {
    this.chat._revertEffectDraft?.(this.chat.editor.editor, this._state.id);
  } catch (e) {
    console.warn('[LLMChat] edit: _handleCancel revertDraft failed', e);
  }

  this._state = null;
}
}

=====
FILE: core/engine/lib/word/llm/chat/handler.js
=====
// neurocad/core/engine/lib/word/llm/chat/handler.js

/**
 * createLLMChatHandler – factory for the incoming WS message handler.
 *
 * Returns a function (msg) => void, closed over the UI instance and
 * seven callbacks:
 *
 * - onRunStart(runId)           – a run has started
 * - onRunEnd()                  – a run has finished
 * - applyHtml(html)             – replace the whole canvas (create / create_page)
 * - applyPageCss(css)           – replace the whole page CSS (create_page)

```

```

* - applyElement(selector, html)      - replace a single element (fill / effect)
* - applyCss(effectId, css, opts)     - replace the CSS of an effect (edit).
*                                     `opts` may carry `{ newLabel, newMedia }`
*                                     for a rename round-trip, or
*                                     `{ newId, newLabel }` for a
*                                     meaning-changing edit that
*                                     proposes a rename on save.
* - applyEffectDraft(draft)           - receive a draft for a NEW effect
*                                     (create_effect – not yet saved)
*
* All "what to do with this message" logic lives here, not in chat.js.
*
* Streamed page generation (create_page)
* -----
* The create_page agent runs in a LOOP: one section per LLM call.
* Each section arrives as a separate `page_step` frame carrying its
* own HTML fragment and CSS. The handler accumulates them into a
* local buffer and re-applies the whole accumulated page to the
* canvas after every step. This gives the user progressive rendering
* instead of one big "wait 3 minutes then snap".
*
* Frame shape (from agent/create_page.py):
* {
*     type: 'page_step',
*     step: 3,
*     total_hint: 10,
*     section_name: 'features',
*     html: '<section ...>...</section>',
*     css: '.features { ... }',
*     done: false
* }
*
* The final step has `done: true` – at that point the accumulation
* stops, and the subsequent global `done` frame (sent by the
* dispatcher) finalizes the run as usual.
*
* Non-streamed agents (create, fill, effect, ...) keep using the
* classic `html_update` / `page_css_update` / `element_update`
* frames – the dispatcher only sends those when the agent did NOT
* set `streamed: true`.
*
* Error messages (type: "error") are rendered with role="error" –
* they get a distinct red bubble style
* (.core-engine-lib-word-llm-chat-msg-error) instead of the normal
* assistant bubble.
*
* Error actions:
* Some errors are actionable – e.g. "Закончились токены LLM"
* should offer a link to the balance page. The server MAY send
* `action: { label, href }` in the error frame; if it does not,
* the handler falls back to a client-side action chosen from
* the message text / code (currently only "tokens exhausted").
* `ui.addMessage()` renders the action as an inline link under
* the error text.
*
* User-facing strings are in Russian – they go straight to the chat UI.

```

```

* Internal log strings stay in English.
*/
export function createLLMChatHandler({
  ui,
  onStart,
  onRunEnd,
  applyHtml,
  applyPageCss,
  applyElement,
  applyCss,
  applyEffectDraft,
}) {
  // ---- streaming state (create_page stepwise) ----
  // Accumulated HTML and CSS across all `page_step` frames of the
  // current run. Reset on every `run_started` so that a new request
  // never inherits the previous page's tail.
  let streamHtml = '';
  let streamCss = '';
  let streamActive = false;

  /**
   * Reset the streaming buffers. Called on run start and after
   * every terminal frame (`done` / `cancelled` / `error`).
   */
  function resetStream() {
    streamHtml = '';
    streamCss = '';
    streamActive = false;
  }

  /**
   * Build a client-side action for an error when the server did
   * not send one. Currently only "tokens exhausted" maps to a
   * concrete action – a link to the balance page.
   *
   * Detection is by message text (and, if present, by `code`),
   * so it works whether or not the backend adds a `code` field
   * to the error frame.
   *
   * The action is a REAL URL (not an onClick handler):
   *   - survives in the address bar,
   *   - works with middle-click / right-click → "Open in new tab",
   *   - can be copied to the clipboard.
   *
   * Current page path is preserved (nav_id and other context
   * params survive); only `auth` and `section` are set. That way
   * clicking the link from the editor keeps the user inside the
   * same module – the profile page opens over the current page
   * and Base handles the rest via ?auth= + ?section=.
   *
   * @param {Object} msg – the error frame
   * @returns {{label: string, href: string} | null}
   */
  function _fallbackAction(msg) {
    if (!msg) return null;

```



```

const isTokensExhausted =
  msg.code === 'tokens_exhausted'
  || /tokens\s+LLM/i.test(msg.message || '');

if (isTokensExhausted) {
  const u = new URL(window.location.href);
  u.searchParams.set('auth', 'profile');
  u.searchParams.set('section', 'balance');

  return {
    label: 'Перейти к балансу',
    href: u.toString(),
  };
}

return null;
}

return function handle(msg) {
  const t = msg.type;
  console.log('[LLMChat] WS message:', t, msg);

  switch (t) {
    case 'pong':
      return;

    case 'run_started':
      // New run – wipe any stale streaming state from the
      // previous one before the first step arrives.
      resetStream();
      streamActive = true;
      onRunStart(msg.run_id);
      ui.setSendingState(true);
      // Do NOT hide typing here – it stays until the first
      // 'step' event, so the user sees the pulsing dots
      // while the router is deciding what to do.
      ui.ensureProgress();
      return;

    case 'step':
      ui.hideTyping();
      ui.setProgressText(msg.message || '');
      return;

    case 'plan':
      if (Array.isArray(msg.blocks) && msg.blocks.length) {
        const ids = msg.blocks.map(b => b.block_id).join(', ');
        ui.setProgressText(`Выбрано блоков: ${msg.blocks.length} (${ids})`);
      }
      return;

    case 'fill_progress':
      ui.setProgressText(
        `Заполняю текстом: запрос ${msg.request} из ${msg.total}...`
      );
      return;
  }
}

```

```

// -----
// SVG ILLUSTRATIONS (step 4): the agent sends one
// `svg_progress` frame BEFORE each placeholder image is
// sent to the LLM. The user sees "Генерация изображения
// 2 из 6..." updating live in the progress bubble.
//
// Frame shape (from step.py → step_svg_illustrations):
// { type: 'svg_progress', current: 2, total: 6, alt: '...' }
//
// We do not use `alt` in the visible text – it can be long
// or empty. Keep the message short and consistent with
// fill_progress.
// -----
case 'svg_progress':
    ui.setProgressText(
        `Генерация изображения ${msg.current} из ${msg.total}...`
    );
    return;

// -----
// PAGE STEP (create_page, stepwise): one section per frame.
//
// The agent runs in a loop and emits one `page_step` per
// generated section. We accumulate the fragments and
// re-apply the whole page after each step – so the user
// sees the page grow section by section instead of
// waiting for one giant response.
//
// Frame shape (from agent/create_page.py):
// {
//     type: 'page_step',
//     step: 3,
//     total_hint: 10,
//     section_name: 'features',
//     html: '<section ...>...</section>',
//     css: '.features { ... }',
//     done: false
// }
//
// Final frame: `done: true`. The subsequent global `done`
// (sent by the dispatcher) finalizes the run as usual.
// -----
case 'page_step':
    streamActive = true;

    // Accumulate. Fragments arrive in order; the final
    // section may come with `done: true` and its own html.
    if (typeof msg.html === 'string' && msg.html.trim()) {
        streamHtml += (streamHtml ? '\n' : '') + msg.html;
    }
    if (typeof msg.css === 'string' && msg.css.trim()) {
        streamCss += (streamCss ? '\n\n' : '') + msg.css;
    }

    // Re-apply the whole accumulated page to the canvas.

```

```

// This is a full replace (setComponents / setStyle),
// which is fine for a handful of sections – the editor
// rebuilds the tree once per step.
if (streamHtml) {
    applyHtml(streamHtml);
}
if (streamCss && typeof applyPageCss === 'function') {
    applyPageCss(streamCss);
}

// Progress text.
if (msg.error) {
    ui.setProgressText(
        `Ошибка на шаге ${msg.step}: ${msg.error}`
    );
} else if (msg.done) {
    ui.setProgressText('Завершаю страницу...');
} else if (msg.section_name) {
    ui.setProgressText(
        `Шаг ${msg.step}: секция «${msg.section_name}»`
    );
} else {
    ui.setProgressText(`Шаг ${msg.step}...`);
}
return;

// -----
// HTML UPDATE: replace the whole canvas.
// - create      – HTML built from our ready blocks.
// - create_page – free-form HTML (single-shot mode;
//                  for stepwise mode the content
//                  arrives via `page_step` instead).
// -----
case 'html_update':
    applyHtml(msg.html || '');
    return;

// -----
// PAGE CSS UPDATE: replace the whole page CSS.
//
// Sent by create_page in single-shot mode right after
// `html_update`. Carries the model-generated CSS WITHOUT
// the surrounding <style> tag. Applied through
// editor.setStyle(css) – the same channel that the Style
// Manager uses on save.
//
// This must be a separate frame: GrapesJS cannot parse
// <style> mixed into components and would drop the whole
// tree if we tried to inline it into `html`.
//
// Frame shape (from dispatch.py → _run_agent):
// { type: 'page_css_update', css: '...' }
// -----
case 'page_css_update':
    if (typeof applyPageCss === 'function') {
        applyPageCss(msg.css || '');
    }

```

```

    } else {
        console.warn('[LLMChat] page_css_update received but no applyPageCss handler');
    }
    return;

case 'element_update':
    applyElement(msg.selector || '', msg.html || '');
    return;

// -----
// EFFECT EDIT / EFFECT RENAME: server returns a new full
// CSS for one effect. The client applies it live in the
// iframe (see live.js). Nothing is written to disk –
// saving happens on the explicit save click (edit flow)
// or is not needed at all (rename flow).
//
// The server MAY also send optional fields in the same
// message:
//
//   new_id    – proposed new id (edit flow only, when the
//               edit changed the MEANING of the effect).
//               Handled by EditSession as "rename on save".
//
//   new_label – proposed new human-readable label. Used
//               both by the edit flow (meaning change) and
//               by the rename flow (label-only change).
//
//   new_media – proposed new SVG miniature. Used by the
//               rename flow only: the palette block gets
//               a fresh icon while its id and CSS stay
//               untouched.
//
// All three are optional. `null` means "not present in
// this message". They are forwarded as `opts` so the
// edit session can decide what to do.
// -----
case 'css_update':
    if (typeof applyCss === 'function') {
        const opts = {
            newId: typeof msg.new_id === 'string' ? msg.new_id : null,
            newLabel: typeof msg.new_label === 'string' ? msg.new_label : null,
            newMedia: typeof msg.new_media === 'string' ? msg.new_media : null,
        };
        console.log('[LLMChat] css_update opts:', {
            newId: opts.newId,
            newLabel: opts.newLabel,
            newMediaLen: opts.newMedia ? opts.newMedia.length : 0,
        });
        applyCss(msg.effect_id || '', msg.css || '', opts);
    } else {
        console.warn('[LLMChat] css_update received but no applyCss handler');
    }
    return;

// -----
// EFFECT CREATE: server returns a DRAFT for a new effect.

```

```

// Nothing has been written to disk yet. chat.js stores the
// draft and shows a "сохрани" hint. When the user confirms,
// chat.js POSTs to /editor/effects.
//
// The draft shape is:
// { id, label, hint, css, media }
// -----
case 'effect_draft':
  if (typeof applyEffectDraft === 'function') {
    applyEffectDraft(msg.draft || null);
  } else {
    console.warn('[LLMChat] effect_draft received but no applyEffectDraft handler');
  }
  return;

case 'assistant_message':
  ui.finalizeProgress();
  if (msg.content) {
    ui.addMessage({
      role: 'assistant',
      content: msg.content,
      created_at: new Date().toISOString(),
    });
  }
  return;

case 'done':
  onRunEnd();
  ui.setSendingState(false);
  ui.finalizeProgress();
  // Streaming run is over – wipe the accumulation buffer
  // so a subsequent request starts clean.
  if (streamActive) {
    resetStream();
  }
  return;

case 'cancelled':
  onRunEnd();
  ui.setSendingState(false);
  ui.setProgressText('Отменено. ');
  ui.finalizeProgress();
  resetStream();
  return;

case 'error': {
  onRunEnd();
  ui.setSendingState(false);
  ui.hideTyping();
  ui.finalizeProgress();

  // Error messages use role="error" – distinct red bubble.
  // The Δ prefix is kept for extra visual cue.
  //
  // Actionable errors get a link under the text:
  // - server may send `action: { label, href }`;

```

```

        // - otherwise the client picks an action from the
        //     message text / code (currently only "tokens
        //     exhausted" → balance page).
        const action = msg.action || _fallbackAction(msg);

        ui.addMessage({
            role: 'error',
            content: `⚠️ ${msg.message || 'Неизвестная ошибка'}`,
            created_at: new Date().toISOString(),
            action,
        });
        resetStream();
        return;
    }

    default:
        console.log('[LLMChat] Unknown WS message:', msg);
    }
}
};
}

```

```

=====
FILE: core/engine/lib/word/llm/chat/selection.js
=====

```

```

// neurocad/core/engine/lib/word/llm/chat/selection.js

```

```

/**
 * getLLMChatSelection – return the current selection in the editor.
 *
 * The selection is captured as an "element marker":
 * - a temporary attribute `data-selected-id="sel-XXXXXXX"` is put
 *   on the selected component;
 * - the component's outer HTML is captured as-is;
 * - the tag name and class list are captured for context.
 *
 * The marker is a random string. It travels to the backend and comes
 * back in the `element_update` event, so the client can find the same
 * component again – even if the component tree has changed.
 *
 * The marker is expected to survive the round-trip: the LLM is
 * instructed (via the prompt) not to remove or change attributes.
 * After the update is applied, the marker is removed by `apply.js`.
 *
 * Returns:
 * {
 *   "selector": "sel-abc12345",
 *   "tag": "section",
 *   "classes": ["section", "hero"],
 *   "outer_html": "<section class=\"section hero\">...</section>"
 * }
 * or null if nothing is selected.
 */
export function getLLMChatSelection(editor) {
    try {
        const ed = editor?.editor;
        if (!ed) return null;
    }
}

```

```

const selected = ed.getSelected?();
if (!selected) {
  console.log('[selection] nothing is selected');
  return null;
}

// ---- 1. unique marker ----
const marker = 'sel-' + makeId(8);
selected.addAttributes({ 'data-selected-id': marker });

// ---- 2. outer HTML after adding the marker ----
const outerHtml = selected.toHTML?() || '';
if (!outerHtml) {
  console.warn('[selection] selected component produced empty html');
  return null;
}

// ---- 3. tag + classes for context ----
const tag = (selected.get('tagName') || '').toString().toLowerCase();
const classes = (selected.getClasses?() || [])
  .map((c) => String(c))
  .filter(Boolean);

const selection = {
  selector: marker,
  tag,
  classes,
  outer_html: outerHtml,
};

console.log('[selection] captured element:', {
  selector: marker,
  tag,
  classes,
  html_len: outerHtml.length,
});

return selection;
} catch (e) {
  console.warn('[selection] error:', e);
  return null;
}
}

// =====
// INTERNAL
// =====

/**
 * Short random id generator: letters + digits, no dashes.
 */
function makeId(length) {
  const chars = 'abcdefghijklmnopqrstuvwxyz0123456789';
  let out = '';
  for (let i = 0; i < length; i++) {

```

```

        out += chars.charAt(Math.floor(Math.random() * chars.length));
    }
    return out;
}

```

```

=====
FILE: core/engine/lib/word/llm/chat/ui.js
=====

```

```

// neurocad/core/engine/lib/word/llm/chat/ui.js

```

```

/**
 * LLMChatUI – DOM part of the chat.
 *
 * Messages, progress bubble, typing indicator, input form, send/stop
 * button, and the "clear chat" button in the toolbar.
 *
 * Knows nothing about WebSocket or HTTP – the caller wires up the
 * onClear callback to whatever API method is appropriate.
 *
 * Message roles:
 *   user      – user message (right-aligned, blue bubble)
 *   assistant – assistant message (left-aligned, gray bubble)
 *   error     – error message (left-aligned, red bubble)
 *
 * Progress bubble
 * -----
 * The progress bubble is a single assistant-style line whose text is
 * replaced via setProgressText(text). It is used by BOTH agent modes:
 *
 * - single-shot agents (create, fill, effect):
 *   "Выбрано блоков: 5 (hero, features, ...)"
 *   "Заполняю текстом: запрос 2 из 4..."
 *   "Генерация изображения 1 из 6..."
 *
 * - stepwise create_page (one section per LLM call):
 *   "Шаг 3: секция «features»"
 *   "Завершаю страницу..."
 *
 * The UI does not parse or reformat the string – whatever the handler
 * passes in is what the user sees. The handler is responsible for
 * building the correct message for each mode.
 *
 * finalizeProgress() is called by the handler on terminal frames
 * (`assistant_message`, `done`, `cancelled`, `error`). In stepwise
 * mode the terminal `page_step` (with `done: true`) is followed by
 * the usual global `done` frame, so the progress bubble is removed
 * exactly once at the end of the run – no special-casing needed here.
 *
 * Message actions:
 *   `addMessage()` accepts an optional `msg.action` of shape
 *   `{ label, href?, onClick? }`. It is rendered as an inline link
 *   under the bubble text. Used by the error path in handler.js to
 *   offer "Перейти к балансу" when the LLM quota is exhausted.
 *
 * User-facing strings are in Russian.
 */

```



```

export class LLMChatUI {
  constructor(editor) {
    this.rootEl = editor.chatEl;

    this.messagesEl = null;
    this.inputEl = null;
    this.sendBtn = null;
    this.formEl = null;
    this.clearBtn = null;

    this._progressEl = null;
    this._progressText = '';

    this._typingEl = null;

    // Connection banner (shown between messages and the form).
    this._bannerEl = null;
  }

  // =====
  // MOUNT
  // =====

  mount() {
    while (this.rootEl.firstChild) {
      this.rootEl.removeChild(this.rootEl.firstChild);
    }

    const messages = document.createElement('div');
    messages.className = 'core-engine-lib-word-llm-chat-messages';
    messages.setAttribute('data-js', 'llm-chat-messages');
    this.messagesEl = messages;
    this.rootEl.appendChild(messages);

    // Connection banner slot – created here, shown/hidden later via
    // setConnectionBanner(). Sits between messages and the form so it
    // never overlaps the scrollable message list.
    const banner = document.createElement('div');
    banner.className = 'core-engine-lib-word-llm-chat-banner';
    banner.setAttribute('data-js', 'llm-chat-banner');
    banner.hidden = true;
    this._bannerEl = banner;
    this.rootEl.appendChild(banner);

    const form = document.createElement('form');
    form.className = 'core-engine-lib-word-llm-chat-form';

    const input = document.createElement('textarea');
    input.className = 'core-engine-lib-word-llm-chat-input';
    input.setAttribute('data-js', 'llm-chat-input');
    input.setAttribute('placeholder', 'Опишите, что сделать...');
    input.setAttribute('rows', '2');
    this.inputEl = input;
    form.appendChild(input);

    const sendBtn = document.createElement('button');

```

```

    sendBtn.type = 'submit';
    sendBtn.className = 'core-engine-lib-word-llm-chat-send';
    sendBtn.setAttribute('title', 'Отправить (Enter)');
    sendBtn.setAttribute('aria-label', 'Отправить');
    sendBtn.textContent = '►';
    this.sendBtn = sendBtn;
    form.appendChild(sendBtn);

    this.formEl = form;
    this.rootEl.appendChild(form);

    // Clear button in the toolbar (if the toolbar exists).
    this._mountClearButton();
}

_mountClearButton() {
    // The toolbar is rendered outside this component (widgets.js /
    // editor template), so we look it up by data-js attribute.
    const toolbar = document.querySelector('[data-js="editor-chat-toolbar"]');
    if (!toolbar) {
        console.warn('[LLMChatUI] toolbar not found – clear button skipped');
        return;
    }

    // Avoid duplicates on re-mount.
    const existing = toolbar.querySelector('.core-engine-lib-word-llm-chat-clear');
    if (existing) {
        this.clearBtn = existing;
        return;
    }

    const clearBtn = document.createElement('button');
    clearBtn.type = 'button';
    clearBtn.className = 'core-engine-lib-word-llm-chat-clear';
    clearBtn.setAttribute('title', 'Очистить чат');
    clearBtn.setAttribute('aria-label', 'Очистить чат');

    // Icon – same style as the other editor buttons.
    const icon = document.createElement('img');
    icon.className = 'core-engine-lib-word-editor-btn-icon';
    icon.src = '/static/core/engine/lib/base/images/reset.svg';
    icon.alt = '';
    icon.setAttribute('aria-hidden', 'true');
    clearBtn.appendChild(icon);

    this.clearBtn = clearBtn;
    toolbar.appendChild(clearBtn);
}

onClear(cb) {
    if (this.clearBtn) {
        this.clearBtn.addEventListener('click', (e) => {
            e.preventDefault();
            e.stopPropagation();
            cb();
        });
    }
}

```

```

    }
  }

  onSubmit(cb) {
    if (this.formEl) {
      this.formEl.addEventListener('submit', (e) => {
        e.preventDefault();
        cb();
      });
    }
    if (this.inputEl) {
      this.inputEl.addEventListener('keydown', (e) => {
        if (e.key === 'Enter' && !e.shiftKey) {
          e.preventDefault();
          cb();
        }
      });
    }
  }
}

// =====
// CONNECTION BANNER
// =====
//
// Called from LLMChat._setConnectionBanner(spec).
//
// spec = null → hide
// spec = { kind: 'info'|'warn'|'error', text } → show
//
// The banner is a single line above the form. It uses the same
// palette as the rest of the chat via CSS classes:
// .is-info – neutral (connecting)
// .is-warn – yellow (reconnecting)
// .is-error – red (auth failure)
//
// Hidden (not removed) so repeated show/hide does not cause
// layout thrash or re-create the DOM node.

setConnectionBanner(spec) {
  if (!this._bannerEl) return;

  if (!spec || !spec.text) {
    this._bannerEl.hidden = true;
    this._bannerEl.textContent = '';
    this._bannerEl.className = 'core-engine-lib-word-llm-chat-banner';
    return;
  }

  const kind = spec.kind || 'info';
  this._bannerEl.textContent = spec.text;
  this._bannerEl.className =
    `core-engine-lib-word-llm-chat-banner is-${kind}`;
  this._bannerEl.hidden = false;
}

// =====

```

```

// MESSAGES
// =====

/**
 * Add a message bubble.
 *
 * msg = { role, content, created_at?, action? }
 *   role = 'user' | 'assistant' | 'error'
 *   action = { label, href?, onClick? } – optional inline link
 *           under the bubble text (used by the error path to
 *           offer e.g. "Перейти к балансу").
 */
addMessage(msg) {
  this._render(msg);
  this._scroll();
}

/**
 * Convenience: add an error message.
 * Renders as a red bubble in the assistant column.
 */
addError(text) {
  this.addMessage({
    role: 'error',
    content: text || 'Неизвестная ошибка',
    created_at: new Date().toISOString(),
  });
}

setMessages(rows) {
  if (!this.messagesEl) return;
  while (this.messagesEl.firstChild) {
    this.messagesEl.removeChild(this.messagesEl.firstChild);
  }
  for (const m of rows) this._render(m);
  this._scroll();
}

_render(msg) {
  if (!this.messagesEl) return;

  const el = document.createElement('div');
  el.className = `core-engine-lib-word-llm-chat-msg core-engine-lib-word-llm-chat-msg-${msg.role}`;

  const bubble = document.createElement('div');
  bubble.className = 'core-engine-lib-word-llm-chat-bubble';

  // Text – via textContent (not innerHTML) so no XSS is possible
  // from server-controlled error strings.
  const text = document.createElement('span');
  text.className = 'core-engine-lib-word-llm-chat-bubble-text';
  text.textContent = msg.content || '';
  bubble.appendChild(text);

  // Optional inline action (link / button) under the text.
  // Shape: { label, href?, onClick? }.

```

```

// - if onClick is provided, it runs on click (href is a
//   no-op, mostly for cursor/hover styling);
// - otherwise the <a> navigates to href as usual.
if (msg.action && msg.action.label) {
  const link = document.createElement('a');
  link.className = 'core-engine-lib-word-llm-chat-bubble-action';
  link.href = msg.action.href || '#';
  link.textContent = msg.action.label;

  link.addEventListener('click', (e) => {
    if (typeof msg.action.onClick === 'function') {
      e.preventDefault();
      msg.action.onClick(e);
    }
  });

  bubble.appendChild(link);
}

el.appendChild(bubble);
this.messagesEl.appendChild(el);
}

_scroll() {
  if (this.messagesEl) {
    this.messagesEl.scrollTop = this.messagesEl.scrollHeight;
  }
}

// =====
// INPUT
// =====

getInputValue() {
  return this.inputEl?.value || '';
}

clearInput() {
  if (this.inputEl) this.inputEl.value = '';
}

// =====
// SEND / STOP BUTTON
// =====

setSendingState(sending) {
  if (!this.sendBtn) return;
  this.sendBtn.disabled = false;
  this.sendBtn.textContent = sending ? '■' : '►';
  this.sendBtn.classList.toggle('is-stop', !!sending);
  this.sendBtn.setAttribute('title', sending ? 'Остановить' : 'Отправить (Enter)');
  this.sendBtn.setAttribute('aria-label', sending ? 'Остановить' : 'Отправить');
}

// =====
// PROGRESS BUBBLE

```

```

// =====

ensureProgress() {
  if (this._progressEl) return;
  if (!this.messagesEl) return;

  const el = document.createElement('div');
  el.className = 'core-engine-lib-word-llm-chat-msg core-engine-lib-word-llm-chat-msg-assistant core-
engine-lib-word-llm-chat-progress';

  const bubble = document.createElement('div');
  bubble.className = 'core-engine-lib-word-llm-chat-bubble';

  const text = document.createElement('span');
  text.className = 'core-engine-lib-word-llm-chat-progress-text';
  text.textContent = this._progressText || '';

  bubble.appendChild(text);
  el.appendChild(bubble);

  this.messagesEl.appendChild(el);
  this._scroll();
  this._progressEl = el;
}

setProgressText(text) {
  this._progressText = text || '';
  this.ensureProgress();
  if (this._progressEl) {
    const t = this._progressEl.querySelector('.core-engine-lib-word-llm-chat-progress-text');
    if (t) t.textContent = this._progressText;
  }
  this._scroll();
}

finalizeProgress() {
  if (this._progressEl && this._progressEl.parentNode) {
    this._progressEl.parentNode.removeChild(this._progressEl);
  }
  this._progressEl = null;
  this._progressText = '';
}

// =====
// TYPING INDICATOR
// =====
//
// Three big pulsing dots, animated entirely by CSS. No JS timer
// is used here – the animation and stagger are declared in llm.css
// (keyframes core-engine-lib-word-llm-typing-pulse).

showTyping() {
  if (!this.messagesEl) return;
  if (this.messagesEl.querySelector('.core-engine-lib-word-llm-chat-typing')) return;

  const el = document.createElement('div');

```

```

        el.className = 'core-engine-lib-word-llm-chat-msg core-engine-lib-word-llm-chat-msg-assistant core-
engine-lib-word-llm-chat-typing';

        const bubble = document.createElement('div');
        bubble.className = 'core-engine-lib-word-llm-chat-bubble';

        // Three dots, each its own element, so CSS can stagger them.
        for (let i = 0; i < 3; i++) {
            const dot = document.createElement('span');
            dot.className = 'core-engine-lib-word-llm-chat-typing-dot';
            bubble.appendChild(dot);
        }

        el.appendChild(bubble);
        this.messagesEl.appendChild(el);
        this._scroll();

        this._typingEl = el;
    }

    hideTyping() {
        if (this._typingEl && this._typingEl.parentNode) {
            this._typingEl.parentNode.removeChild(this._typingEl);
        }
        this._typingEl = null;
    }

    // =====
    // DESTROY
    // =====

    destroy() {
        if (this.clearBtn && this.clearBtn.parentNode) {
            this.clearBtn.parentNode.removeChild(this.clearBtn);
        }
        this.rootEl = null;
        this.messagesEl = null;
        this.inputEl = null;
        this.sendBtn = null;
        this.formEl = null;
        this.clearBtn = null;
        this._progressEl = null;
        this._progressText = '';
        this._typingEl = null;
        this._bannerEl = null;
    }
}

=====
FILE: core/engine/lib/word/llm/chat/ws.js
=====
// neurocad/core/engine/lib/word/llm/chat/ws.js

/**
 * LLMChatWS – WebSocket transport for the chat.
 *

```

```

*   connect()      – open the socket, start heartbeat
*   send(payload)  – send JSON, return true/false
*   isOpen()      – readyState === OPEN
*   destroy()      – close, stop reconnect / heartbeat permanently
*
* Reconnect: exponential backoff (1s, 2s, 4s, 8s, 16s, 30s, 30s, ...).
* Heartbeat: ping every 30s, pong is silently dropped.
*
* Auth-close (4403) is treated as terminal: no reconnect, onClose fires
* with `{auth: true}` so the caller can show "нет доступа" instead of
* "переподключаюсь".
*
* State callback (`onState`) is called with one of:
*   "connecting" – trying to open
*   "open"       – connected
*   "closed"     – disconnected, will retry
*   "auth"       – closed with 4403, no retry
*   "dead"       – destroy() called, terminal
*/
export class LLMChatWS {
  // Exponential backoff steps (ms). Last value repeats.
  static BACKOFF = [1000, 2000, 4000, 8000, 16000, 30000];

  // WebSocket close code used by the server for auth failures.
  static CLOSE_AUTH = 4403;

  constructor({ urlProvider, onOpen, onMessage, onClose, onState }) {
    this._urlProvider = urlProvider;
    this._onOpen = onOpen;
    this._onMessage = onMessage;
    this._onClose = onClose;
    this._onState = onState;

    this._ws = null;
    this._reconnectTimer = null;
    this._heartbeatTimer = null;
    this._attempt = 0;
    this._destroyed = false;

    // Set to true right before close() in destroy(), so onClose
    // knows not to schedule another reconnect.
    this._closingByUser = false;
  }

  get readyState() {
    return this._ws?.readyState;
  }

  isOpen() {
    return this._ws && this._ws.readyState === WebSocket.OPEN;
  }

  connect() {
    if (this._destroyed) return;

    const url = this._urlProvider();

```



```

if (!url) {
  console.warn('[LLMChatWS] No URL – WS not opened');
  return;
}

console.log('[LLMChatWS] Opening WS:', url);
this._setState('connecting');

try {
  this._ws = new WebSocket(url);
} catch (e) {
  console.error('[LLMChatWS] create error:', e);
  this._scheduleReconnect();
  return;
}

this._ws.onopen = () => {
  console.log('[LLMChatWS] open – readyState:', this._ws?.readyState);
  this._attempt = 0; // reset backoff on success
  this._startHeartbeat();
  this._setState('open');
  this._onOpen?.();
};

this._ws.onmessage = (e) => {
  let msg;
  try {
    msg = JSON.parse(e.data);
  } catch (err) {
    console.warn('[LLMChatWS] Bad message:', e.data);
    return;
  }

  // pong is silent – do not pass it up
  if (msg.type === 'pong') return;

  this._onMessage?.(msg);
};

this._ws.onerror = (e) => {
  console.warn('[LLMChatWS] error:', e);
};

this._ws.onclose = (e) => {
  console.log(
    `[LLMChatWS] closed: code=${e.code} reason=${e.reason || '(none)'} wasClean=${e.wasClean}`
  );
  this._stopHeartbeat();
  this._ws = null;

  // destroy() already handled state and cleanup.
  if (this._closingByUser) {
    this._closingByUser = false;
    return;
  }
}

```

```

    // Auth failure – terminal, no retry.
    if (e.code === LLMChatWS.CLOSE_AUTH) {
        this._setState('auth');
        this._onClose?({ auth: true, code: e.code, reason: e.reason });
        return;
    }

    this._setState('closed');
    this._onClose?({ code: e.code, reason: e.reason });
    this._scheduleReconnect();
};
}

send(payload) {
    if (!this.isOpen()) {
        if (payload.type !== 'ping') {
            console.warn(
                `[LLMChatWS] not ready (readyState=${this._ws?.readyState}), drop: ${payload.type}`
            );
        }
        return false;
    }
    try {
        this._ws.send(JSON.stringify(payload));
        return true;
    } catch (e) {
        if (payload.type !== 'ping') {
            console.warn(`[LLMChatWS] send error:`, e);
        }
        return false;
    }
}

// =====
// RECONNECT
// =====

_scheduleReconnect() {
    if (this._destroyed) return;
    if (this._reconnectTimer) return;

    const steps = LLMChatWS.BACKOFF;
    const idx = Math.min(this._attempt, steps.length - 1);
    const delay = steps[idx];
    this._attempt += 1;

    console.log(`[LLMChatWS] reconnect attempt #${this._attempt} in ${delay}ms`);

    this._reconnectTimer = setTimeout(() => {
        this._reconnectTimer = null;
        this.connect();
    }, delay);
}

// =====
// HEARTBEAT

```

```

// =====

_startHeartbeat() {
  this._stopHeartbeat();
  this._heartbeatTimer = setInterval(() => {
    this.send({ type: 'ping' });
  }, 30000);
}

_stopHeartbeat() {
  if (this._heartbeatTimer) {
    clearInterval(this._heartbeatTimer);
    this._heartbeatTimer = null;
  }
}

// =====
// STATE
// =====

_setState(state) {
  try {
    this._onState?.(state);
  } catch (e) {
    console.warn('[LLMChatWS] onState error:', e);
  }
}

// =====
// DESTROY
// =====

destroy() {
  this._destroyed = true;
  this._stopHeartbeat();
  if (this._reconnectTimer) {
    clearTimeout(this._reconnectTimer);
    this._reconnectTimer = null;
  }
  if (this._ws) {
    this._closingByUser = true;
    try { this._ws.close(); } catch (e) { /* ignore */ }
    this._ws = null;
  }
  this._setState('dead');
}
}

=====
FILE: core/engine/lib/word/llm/chat/__init__.py
=====

=====
FILE: core/engine/lib/base/base.js
=====

```

```
// app/core/engine/lib/base/base.js

/**
 * Base component – application shell.
 *
 * Responsibilities:
 * - render the shell (header, area-left/center/right, footer);
 * - load the auth / modal / caption modules;
 * - react to auth state changes (auth:changed, auth:unauthorized);
 * - load and render the page config components into area-center.
 *
 * Delegated concerns (loaded dynamically in _loadModules):
 * - areas.js      → BaseAreas      (renderInArea / teardownAreas / side areas)
 * - pages.js      → BasePages      (showProfile / showSetup / showAuthPage)
 * - chat.js       → BaseChatController (chat panel lifecycle)
 *
 * Public API kept stable for external callers (auth.js, menu.js, ...):
 * base.renderInArea(ComponentClass, options)
 * base.teardownAreas()
 * base.showProfile(section)
 * base.showSetup(section)
 * base.openChat() / base.closeChat()
 *
 * Runtime params:
 * Generic query params from the URL are forwarded by CoreEngine
 * as `props.params`. Base interprets the ones it owns:
 *
 * - `auth`      – which AUTH form to open (login / register /
 *                  restore / password). Strictly authentication.
 *
 * - `page`      – which APP page to open (profile / setup).
 *
 * - `section`   – a sub-page inside `page`:
 *                  page=profile → section=balance / domain / ...
 *                  page=setup  → section=llm / ...
 *
 * `auth` and `page` are independent:
 *
 * ?auth=login           → login form
 * ?auth=register         → register form
 * ?page=profile          → profile landing
 * ?page=profile&section=balance → profile → balance
 * ?page=setup            → setup landing
 * ?page=setup&section=llm → setup → llm
 *
 * `` / `` /
 * `` are kept as fallbacks for server-rendered
 * pages that do not go through the URL query at all.
 *
 * Reload after login
 * -----
 * After a successful login (auth:changed with isAuthenticated=true)
 * the page is fully reloaded. This is deliberate:
 *
 * - `` is rendered by the SERVER, before login.
 * After logout + login (without a reload) it still carries the
```

```

*   value from the pre-login render – which may be empty, or the
*   previous user's nav. Word/Pages then build article URLs like
*   /core/engine/admin/page/<date>/<time> instead of
*   /core/engine/admin/page/<nav_id>/<date>/<time>, and the page
*   fails to load.
*
* - A full reload re-renders the shell with the new session, so
*   data-nav-id (and any other server-driven data-* attribute)
*   matches the authenticated user.
*
* The reload happens ONLY when the URL has no ?auth= left. If the
* URL still carries ?auth=<form>, it is first stripped and the
* navigation itself reloads the page – one reload per login.
*/
export class Base {
  constructor(container, props = {}) {
    console.log('[Base] Constructor called');
    this.container = container;
    this.props = props;

    this.showChat = props.showChat === true;
    this.authRequired = props.authRequired || false;
    this.authRedirect = props.authRedirect || null;

    // Query params forwarded by engine.js as-is. Base interprets
    // the ones it owns (auth, page, section). Fallbacks read the
    // <body data-*> attributes set by the server-rendered page.
    this.params = props.params || {};

    // `auth` – which AUTH form to open (login / register /
    // restore / password). Strictly authentication; not profile.
    this.authForm = this.params.auth
      || document.body.dataset.authForm
      || null;

    // `page` – which APP page to open (profile / setup).
    this.page = this.params.page
      || document.body.dataset.page
      || null;

    // `section` – a sub-page inside `page`.
    this.section = this.params.section
      || document.body.dataset.section
      || null;

    this.content = props.content || null;
    this.components = props.components || [];

    this.showHeader = props.showHeader !== false;
    this.showFooter = props.showFooter !== false;
    this.showLeft = props.showLeft !== false;
    this.showRight = props.showRight !== false;

    this.modules = {};
    this.header = null;
    this.headerEl = null;

```

```

this.footerEl = null;
this.leftEl = null;
this.rightEl = null;
this.centerEl = null;
this.auth = null;
this.chat = null;
this.savedContent = null;
this.redirectUrl = null;
this.childComponents = [];

// Active "page" instance rendered into area-center (profile, setup, ...).
// Not the same as childComponents – those come from the config.
this.areaInstance = null;

// Delegated managers – created in _loadModules().
this.areas = null;
this.pages = null;
this.chatController = null;

// Init state
this._initialized = false;
this._initPromise = null;

// Flag: authentication in progress (blocks renderContent)
this._isAuthenticating = false;

// Caption helpers (loaded in _loadModules).
this._setCaption = null;
this._restoreCaption = null;

document.body.style.display = 'none';

this._loadBaseCSS();

this._initPromise = this._init();
}

_loadBaseCSS() {
  console.log('[Base] _loadBaseCSS()');
  const version = window.coreEngine?.static_version || Date.now();

  const cssFiles = [
    'core/engine/lib/base/css/00_variables.css',
    'core/engine/lib/base/css/01_reset.css',
    'core/engine/lib/base/css/02_fonts.css',
    'core/engine/lib/base/css/03_layout.css',
    'core/engine/lib/base/css/04_blocks.css',
    'core/engine/lib/base/css/05_widgets.css',
    'core/engine/lib/base/css/06_buttons.css',
    'core/engine/lib/base/css/07_mobile.css',
  ];

  cssFiles.forEach(path => {
    const href = `/static/${path}?v=${version}`;
    const existing = document.querySelector(`link[href="${href}"]`);
    if (existing) return;
  });
}

```

```

        const link = document.createElement('link');
        link.rel = 'stylesheet';
        link.href = href;
        document.head.appendChild(link);
    });
}

async _init() {
    console.log('[Base] _init() START');
    try {
        await this._loadModules();
        this._initialized = true;
        console.log('[Base] _init() COMPLETE');
    } catch (error) {
        console.error('[Base] Init error:', error);
        this._initialized = false;
        throw error;
    }
}

async _loadModules() {
    console.log('[Base] _loadModules()');
    const version = window.coreEngine?.static_version || Date.now();

    try {
        const [
            { Header },
            { BaseAuth },
            { createModal },
            { setCaption, restoreCaption },
            { BaseAreas },
            { BasePages },
            { BaseChatController },
        ] = await Promise.all([
            import(`./header.js?v=${version}`),
            import(`./auth/auth.js?v=${version}`),
            import(`./modal/index.js?v=${version}`),
            import(`./caption.js?v=${version}`),
            import(`./areas.js?v=${version}`),
            import(`./pages.js?v=${version}`),
            import(`./chat.js?v=${version}`),
        ]);

        this.modules.Header = Header;
        this.modules.BaseAuth = BaseAuth;
        this.modules.createModal = createModal;
        this.modules.setCaption = setCaption;
        this.modules.restoreCaption = restoreCaption;
        this.modules.BaseAreas = BaseAreas;
        this.modules.BasePages = BasePages;
        this.modules.BaseChatController = BaseChatController;

        this._setCaption = setCaption;
        this._restoreCaption = restoreCaption;
    }
}

```

```

    // Delegated managers.
    this.areas = new BaseAreas(this);
    this.pages = new BasePages(this);
    this.chatController = new BaseChatController(this);

    this.header = new Header({
      logoText: this.props.logoText || '× Нейрокад',
      title: this.props.title || null,
      menu: this.props.menu || null,
      auth: null
    });

    await this.header.init();
    this.render();
    await this._initAuth();
  } catch (error) {
    console.error('[Base] Module load error:', error);
    throw error;
  }
}

async _initAuth() {
  console.log('[Base] _initAuth()');
  const { BaseAuth } = this.modules;

  if (this.showChat) {
    await this.chatController.init();
  }

  this.auth = new BaseAuth();

  this.auth.setCaption = this._setCaption;
  this.auth.restoreCaption = this._restoreCaption;

  if (window.coreEngine) {
    window.coreEngine.auth = this.auth;
    window.coreEngine.base = this;
  }

  if (this.header) {
    this.header.setAuth(this.auth);
  }

  // ===== React to auth state changes =====
  document.addEventListener('auth:changed', (e) => {
    const { user, isAuthenticated } = e.detail;
    console.log('[Base] auth:changed', { user, isAuthenticated });

    this._updateAuthUI(user, isAuthenticated);
    if (this.header) {
      this.header.setUser(isAuthenticated ? user : null);
    }

    if (isAuthenticated) {
      // If the URL still carries ?auth=<form> (from a shareable
      // register/login link), strip it and do a full reload to

```



```

// the clean URL. This mirrors a normal link navigation:
// - the URL no longer contains the auth marker,
// - pressing F5 loads the real page instead of the form.
try {
  const u = new URL(window.location.href);
  if (u.searchParams.has('auth')) {
    u.searchParams.delete('auth');
    console.log('[Base] hard redirect to clean URL:', u.toString());
    window.location.replace(u.toString());
    return; // page is reloading
  }
} catch (err) {
  console.warn('[Base] failed to strip ?auth= from URL:', err);
}

// No ?auth= left in the URL – but the page still carries
// the HTML from BEFORE login. In particular,
// `<body data-nav-id>` was rendered for the guest session
// and is empty (or holds the previous user's nav after
// logout+login without a reload). Word/Pages then build
// article URLs without the /page/<nav_id>/ segment, and
// the article fails to load.
//
// Do a full reload so the server re-renders the shell
// for the authenticated user: data-nav-id, data-auth-form
// and every other server-driven data-* attribute will
// match the new session.
//
// This runs once per login. If ?auth= was present, the
// branch above already returned and the navigation itself
// reloads the page – no double reload.
console.log('[Base] auth:changed → full reload for fresh server render');
window.location.reload();
return;
}

if (isAuthenticated && this.authRequired) {
  this.onAuthSuccess();
}
});

// ===== React to unauthorized access (401) =====
document.addEventListener('auth:unauthorized', () => {
  console.log('[Base] auth:unauthorized – tearing down the active page');

  this._isAuthenticating = true;

  try {
    this.areas._destroyAreaInstance();
  } catch (e) {
    console.warn('[Base] auth:unauthorized: _destroyAreaInstance error:', e);
  }
  try {
    this.areas._clearSideAreas();
  } catch (e) {
    console.warn('[Base] auth:unauthorized: _clearSideAreas error:', e);
  }
});

```

```

    }
  });

  if (this.auth._initPromise) {
    await this.auth._initPromise;
  }

  const user = this.auth.getUser();
  const isAuthenticated = this.auth.isAuth();

  this._updateAuthUI(user, isAuthenticated);
  if (this.header) {
    this.header.setUser(isAuthenticated ? user : null);
  }

  // ===== 1. Auth form requested =====
  // Comes from ?auth=<form> (via props.params.auth) or from
  // <body data-auth-form>. Only login / register / restore /
  // password - `profile` is not an auth form anymore.
  if (this.authForm) {
    const map = {
      login: () => this.auth.showLogin(),
      register: () => this.auth.showRegister(),
      restore: () => this.auth.showRestore(),
      password: () => this.auth.showPassword(),
    };
    const fn = map[this.authForm];
    if (fn) {
      this._isAuthenticating = true;
      fn();
      document.body.style.display = 'flex';
      document.body.style.flexDirection = 'column';
      return;
    }
    console.warn('[Base] Unknown auth form:', this.authForm);
  }

  // ===== 2. App page requested =====
  // Comes from ?page=<name> (via props.params.page) or from
  // <body data-page>. profile / setup. If the user is not
  // authenticated, the login form is shown instead; once the
  // user logs in, the reload (see auth:changed) re-runs the
  // whole init with the session in place.
  if (this.page) {
    if (!isAuthenticated) {
      console.log('[Base] page requested but not authenticated - showing login');
      this._isAuthenticating = true;
      this.auth.showLogin();
      document.body.style.display = 'flex';
      document.body.style.flexDirection = 'column';
      return;
    }

    document.body.style.display = 'flex';
    document.body.style.flexDirection = 'column';
    this._openPage(this.page, this.section);
  }

```

```

        return;
    }

    // ===== 3. Default flow =====
    if (this.authRequired) {
        if (!isAuthenticated) {
            this.pages.showAuthPage();
        } else {
            this.renderContent();
        }
    } else {
        this.renderContent();
    }

    if (this.header && typeof this.header.bindEvents === 'function') {
        this.header.bindEvents();
    }

    document.body.style.display = 'flex';
    document.body.style.flexDirection = 'column';
}

/**
 * Open an app page (profile / setup) with an optional section.
 *
 * Centralises the page → BasePages mapping so both _initAuth()
 * and the auth:changed handler stay in sync.
 *
 * @param {string} page – 'profile' | 'setup'
 * @param {string|null} section – sub-page inside `page`
 */
_openPage(page, section = null) {
    console.log('[Base] _openPage()', { page, section });

    if (page === 'profile') {
        this.showProfile(section || 'main');
        return;
    }
    if (page === 'setup') {
        this.showSetup(section || 'main');
        return;
    }
    console.warn('[Base] Unknown page:', page);
}

_updateAuthUI(user, isAuthenticated) {
    const actionIcon = document.querySelector('.core-engine-lib-base-action-icon');
    if (actionIcon) {
        if (isAuthenticated && user) {
            actionIcon.textContent = '✕';
            actionIcon.dataset.action = 'logout';
        } else {
            actionIcon.textContent = '🔑';
            actionIcon.dataset.action = 'auth';
        }
    }
}

```

```

    const usernameEl = document.querySelector('.core-engine-lib-base-username');
    if (usernameEl) {
        if (isAuthenticated && user) {
            usernameEl.textContent = user.name || user.login || 'Пользователь';
        } else {
            usernameEl.textContent = 'Гость';
        }
    }
}

// =====
// PUBLIC DELEGATES – area / pages / chat
// =====

/**
 * Mount a component into area-center. Delegates to BaseAreas.
 * Kept here so external callers (auth.js, menu.js) don't change.
 */
renderInArea(ComponentClass, options = {}) {
    if (!this.areas) {
        console.warn('[Base] renderInArea called before init');
        return null;
    }
    return this.areas.renderInArea(ComponentClass, options);
}

/**
 * Destroy the active area-center page (with confirmClose) and
 * clear side areas. Delegates to BaseAreas.
 */
teardownAreas() {
    if (!this.areas) return true;
    return this.areas.teardownAreas();
}

/**
 * Open a profile page. Delegates to BasePages.
 */
showProfile(section = 'main') {
    if (!this.pages) {
        console.warn('[Base] showProfile called before init');
        return null;
    }
    return this.pages.showProfile(section);
}

/**
 * Open a setup page (superadmin only). Delegates to BasePages.
 */
showSetup(section = 'main') {
    if (!this.pages) {
        console.warn('[Base] showSetup called before init');
        return null;
    }
    return this.pages.showSetup(section);
}

```

```

}

/**
 * Show the auth fallback page. Delegates to BasePages.
 */
showAuthPage() {
    if (!this.pages) return;
    return this.pages.showAuthPage();
}

/**
 * Open the chat panel. Delegates to BaseChatController.
 */
openChat() {
    if (!this.chatController) return;
    this.chatController.open();
}

/**
 * Close the chat panel. Delegates to BaseChatController.
 */
closeChat() {
    if (!this.chatController) return;
    this.chatController.close();
}

// =====
// RENDER CONTENT
// =====

async renderContent() {
    console.log('[Base] renderContent()');

    if (this._isAuthenticating) {
        console.log('[Base] Auth in progress, skipping renderContent');
        return;
    }

    // Restore side areas visibility (they may have been hidden on 401).
    if (this.areas) {
        this.areas._restoreSideAreas();
    }

    const center = document.querySelector('.core-engine-lib-base-area-center');
    if (!center) {
        console.error('[Base] center not found');
        return;
    }

    const renderer = window.coreEngine?.renderer;
    if (!renderer) {
        console.error('[Base] renderer not found');
        center.innerHTML = '';
        return;
    }
}

```

```

console.log('[Base] components:', this.components);
console.log('[Base] content:', this.content);

// ===== STEP 1: render components into a hidden staging area =====
const staging = document.createElement('div');
staging.style.display = 'none';
document.body.appendChild(staging);

if (this.childComponents && this.childComponents.length) {
  for (const inst of this.childComponents) {
    try {
      if (inst && typeof inst.destroy === 'function') inst.destroy();
    } catch (e) {
      console.warn('[Base] Child component destroy error:', e);
    }
  }
}
this.childComponents = [];

const componentElements = {};

if (this.components && this.components.length > 0) {
  for (const comp of this.components) {
    const alias = comp.alias || comp.component;
    console.log('[Base] Rendering component:', alias);
    try {
      const element = await renderer.renderComponent(comp, staging);
      if (element) {
        componentElements[alias] = element;

        if (element.__instance) {
          this.childComponents.push(element.__instance);
        }

        console.log(`[Base] Component ${alias} rendered`);
      } else {
        console.warn('[Base] Component not rendered:', alias);
      }
    } catch (error) {
      console.error('[Base] Component render error:', error);
    }
  }
}

// ===== STEP 2: assemble final DOM =====
center.innerHTML = '';

if (this.content) {
  let html = this.content;
  if (typeof this.content === 'function') {
    html = this.content();
  }

  if (typeof html === 'string') {
    const template = document.createElement('template');
    template.innerHTML = html;
  }
}

```

```

const fragment = template.content;

const walker = document.createTreeWalker(
  fragment,
  NodeFilter.SHOW_TEXT,
  null
);

const textNodes = [];
let node;
while ((node = walker.nextNode())) {
  if (node.nodeValue && node.nodeValue.includes('{{}}') {
    textNodes.push(node);
  }
}

for (const textNode of textNodes) {
  const parts = textNode.nodeValue.split(/(\{\{[^}]+\}\})/g);
  if (parts.length === 1) continue;

  const parent = textNode.parentNode;
  const refNode = textNode;

  for (const part of parts) {
    const m = part.match(/^\{\{([^\}]+)\}\}$/);
    if (m) {
      const alias = m[1].trim();
      const el = componentElements[alias];
      if (el) {
        console.log(`[Base] Inserting live element for {{{${alias}}}`);
        parent.insertBefore(el, refNode);
        delete componentElements[alias];
      } else {
        console.warn(`[Base] Marker {{{${alias}}} not found`);
        parent.insertBefore(document.createTextNode(part), refNode);
      }
    } else if (part) {
      parent.insertBefore(document.createTextNode(part), refNode);
    }
  }

  parent.removeChild(refNode);
}

center.appendChild(fragment);
}

// ===== STEP 3: components without a marker =====
for (const [alias, element] of Object.entries(componentElements)) {
  if (element) {
    console.log(`[Base] Appending unused component to DOM:`, alias);
    center.appendChild(element);
  }
}

```

```

    staging.remove();

    console.log('[Base] renderContent() complete');
}

onAuthSuccess() {
    console.log('[Base] onAuthSuccess()');
    const redirectUrl = sessionStorage.getItem('auth_redirect_url') || this.redirectUrl || '/';
    sessionStorage.removeItem('auth_redirect_url');

    if (redirectUrl && redirectUrl !== window.location.pathname) {
        window.location.href = redirectUrl;
    } else {
        this.renderContent();
        const center = document.querySelector('.core-engine-lib-base-area-center');
        if (center && this.savedContent) {
            center.innerHTML = this.savedContent;
            this.savedContent = null;
        }
    }
}

render() {
    console.log('[Base] render()');
    document.body.innerHTML = `
        ${this.header ? this.header.render() : ''}
        <main class="core-engine-lib-base-main">
            <div class="core-engine-lib-base-main-inner">
                <aside class="core-engine-lib-base-area-left"></aside>
                <div class="core-engine-lib-base-area-center"></div>
                <aside class="core-engine-lib-base-area-right"></aside>
            </div>
        </main>
        <footer class="core-engine-lib-base-footer">
            <div class="core-engine-lib-base-footer-inner">© 2026 NeuroCad - Смирнов Алексей Владимирович</div>
        </footer>
    `;

    this.headerEl = document.querySelector('.core-engine-lib-base-header');
    this.footerEl = document.querySelector('.core-engine-lib-base-footer');
    this.leftEl = document.querySelector('.core-engine-lib-base-area-left');
    this.rightEl = document.querySelector('.core-engine-lib-base-area-right');
    this.centerEl = document.querySelector('.core-engine-lib-base-area-center');

    if (this.headerEl) {
        this.headerEl.style.display = this.showHeader ? 'flex' : 'none';
    }
    if (this.footerEl) {
        this.footerEl.style.display = this.showFooter ? 'flex' : 'none';
    }
    if (this.leftEl) {
        this.leftEl.style.display = this.showLeft ? 'flex' : 'none';
    }
    if (this.rightEl) {
        this.rightEl.style.display = this.showRight ? 'flex' : 'none';
    }
}

```



```

    this._bindAuthEvents();
}

_bindAuthEvents() {
    const actionIcon = document.querySelector('.core-engine-lib-base-action-icon');
    if (actionIcon) {
        actionIcon.removeEventListener('click', this._handleAuthClick);
        actionIcon.addEventListener('click', this._handleAuthClick.bind(this));
    }
}

_handleAuthClick(e) {
    const action = e.currentTarget.dataset.action;
    if (action === 'auth') {
        if (this.auth) this.auth.showLogin();
    } else if (action === 'logout') {
        if (this.auth) this.auth.logout();
    } else if (action === 'profile') {
        // Profile is an APP page, not an auth form. Open it via
        // the public delegate so it goes through BasePages.
        this.showProfile('main');
    }
}

async openModal(type, title, message, options = {}) {
    const modal = this.modules.createModal(type);
    if (!modal) {
        console.warn(`[Base] Unknown modal type: ${type}`);
        return;
    }

    switch (type) {
        case 'message':
            modal.open(message || 'Это тестовое сообщение.', title || 'Уведомление', options.okText || 'Понятно');
            modal.setOnOk(() => modal.destroy());
            break;
        case 'confirm':
            modal.open(message || 'Вы уверены?', title || 'Подтверждение', options.okText || 'Да', options.cancelText || 'Отмена');
            modal.setOnOk(() => modal.destroy());
            modal.setOnCancel(() => modal.destroy());
            break;
        case 'input':
            modal.open(title || 'Ввод', message || 'Введите значение:', options.placeholder || '');
            modal.setOnOk(() => modal.destroy());
            modal.setOnCancel(() => modal.destroy());
            break;
        case 'date':
            modal.open();
            modal.setOnOk(() => modal.destroy());
            break;
        case 'interval':
            modal.open();
            modal.setOnOk(() => modal.destroy());
            break;
        default:
    }
}

```

```

        console.warn(`[Base] Unknown modal type: ${type}`);
    }
}

update(props) {
    console.log('[Base] update()');
    this.props = { ...this.props, ...props };
    this.showHeader = this.props.showHeader !== false;
    this.showFooter = this.props.showFooter !== false;
    this.showLeft = this.props.showLeft !== false;
    this.showRight = this.props.showRight !== false;
    this.showChat = this.props.showChat === true;
    this.authRequired = this.props.authRequired || false;
    this.content = this.props.content || null;
    this.components = this.props.components || [];
    this.render();
}

isInitialized() {
    return this._initialized;
}

async waitForInit() {
    if (this._initPromise) {
        await this._initPromise;
    }
    return this._initialized;
}

destroy() {
    console.log('[Base] destroy()');

    // Destroy active area page instance + side areas.
    if (this.areas) {
        try {
            this.areas.destroy();
        } catch (e) {
            console.warn('[Base] areas.destroy error:', e);
        }
    }

    // Destroy chat.
    if (this.chatController) {
        try {
            this.chatController.destroy();
        } catch (e) {
            console.warn('[Base] chatController.destroy error:', e);
        }
    }

    // Destroy child components.
    if (this.childComponents && this.childComponents.length) {
        for (const inst of this.childComponents) {
            try {
                if (inst && typeof inst.destroy === 'function') inst.destroy();
            } catch (e) {
            }
        }
    }
}

```

```

        console.warn('[Base] Child component destroy error:', e);
    }
}
}
this.childComponents = [];

document.body.innerHTML = '';

if (this.auth) {
    if (typeof this.auth.destroy === 'function') {
        this.auth.destroy();
    }
    this.auth = null;
}

this.areas = null;
this.pages = null;
this.chatController = null;

this._initialized = false;
this._initPromise = null;
}
}

```

```

=====
FILE: core/engine/lib/base/pages.js
=====

```

```

// app/core/engine/lib/base/pages.js

```

```

/**
 * BasePages – page controllers for area-center.
 *
 * Owns the three "page-level" entry points that replace area-center
 * with a full-page component:
 *
 * - showProfile(section) – user profile (main / balance / domain)
 * - showSetup(section) – superadmin setup (main / llm)
 * - showAuthPage() – auth fallback page (login/register/...)
 *
 * Each show* method:
 * 1. checks the guard (authenticated / superadmin);
 * 2. calls base.teardownAreas() – destroys the previous page, may
 * prompt the user via confirmClose();
 * 3. dynamically imports the target component with ?v=<static_version>;
 * 4. calls base.renderInArea(ComponentClass, options).
 *
 * Navigation between sections is done by passing `onNavigate` down to
 * the component – the component calls it with the next section name,
 * and it calls back into this module.
 *
 * All state lives on the owning Base instance:
 * - base.auth, base.savedContent, base.redirectUrl, base.authRedirect
 *
 * No imports. Loaded dynamically from Base._loadModules() with a
 * cache-busting ?v= query.
 */

```

```

export class BasePages {
  constructor(base) {
    this.base = base;
  }

  // =====
  // PROFILE
  // =====

  /**
   * Open a profile page in area-center.
   *
   * Available to any authenticated user (no superadmin check).
   *
   * Three sections, three separate components:
   * - 'main' → profile/profile.js (BaseProfile)
   * - 'balance' → profile/balance/balance.js (BaseProfileBalance)
   * - 'domain' → profile/domain/domain.js (BaseProfileDomain)
   *
   * Password change is NOT a profile section – it lives in
   * auth/password.js and is opened directly via auth.showPassword()
   * from the profile landing page.
   *
   * @param {string} section – 'main' (default), 'balance' or 'domain'
   */
  async showProfile(section = 'main') {
    console.log('[BasePages] showProfile()', section);

    const user = this.base.auth?.getUser?();
    if (!user) {
      console.warn('[BasePages] showProfile: access denied (not authenticated)');
      if (this.base.auth) this.base.auth.showLogin();
      return;
    }

    const proceed = await this.base.teardownAreas();
    if (!proceed) {
      console.log('[BasePages] showProfile cancelled by user');
      return;
    }

    const version = window.coreEngine?.static_version || Date.now();

    let ComponentClass = null;
    try {
      if (section === 'balance') {
        const mod = await import(`./profile/balance/balance.js?v=${version}`);
        ComponentClass = mod.BaseProfileBalance;
      } else if (section === 'domain') {
        const mod = await import(`./profile/domain/domain.js?v=${version}`);
        ComponentClass = mod.BaseProfileDomain;
      } else {
        const mod = await import(`./profile/profile.js?v=${version}`);
        ComponentClass = mod.BaseProfile;
      }
    } catch (err) {

```

```

        console.error('[BasePages] showProfile import error:', err);
        return;
    }

    if (!ComponentClass) {
        console.error('[BasePages] showProfile: component class not found for section', section);
        return;
    }

    await this.base.renderInArea(ComponentClass, {
        section,
        user,
        setCaption: this.base._setCaption,
        restoreCaption: this.base._restoreCaption,
        onNavigate: (nextSection) => this.showProfile(nextSection),
    });
}

// =====
// SETUP (superadmin only)
// =====

/**
 * Open a setup page in area-center.
 *
 * Available to superadmin only. The guard is checked here; server-side
 * permissions are enforced on the API endpoints.
 *
 * Two sections, two separate components:
 *   - 'main' → setup/setup.js (BaseSetup)      - setup landing page
 *   - 'llm' → setup/llm/llm.js (BaseSetupLlm) - LLM settings page
 *
 * @param {string} section - 'main' (default) or 'llm'
 */
async showSetup(section = 'main') {
    console.log('[BasePages] showSetup()', section);

    const user = this.base.auth?.getUser?();
    const isSuperadmin = user?.is_superadmin === true;
    if (!isSuperadmin) {
        console.warn('[BasePages] showSetup: access denied (not superadmin)');
        return;
    }

    const proceed = await this.base.teardownAreas();
    if (!proceed) {
        console.log('[BasePages] showSetup cancelled by user');
        return;
    }

    const version = window.coreEngine?.static_version || Date.now();

    let ComponentClass = null;
    try {
        if (section === 'llm') {
            const mod = await import(`./setup/llm/llm.js?v=${version}`);

```

```

        ComponentClass = mod.BaseSetupLlm;
    } else {
        const mod = await import(`./setup/setup.js?v=${version}`);
        ComponentClass = mod.BaseSetup;
    }
} catch (err) {
    console.error('[BasePages] showSetup import error:', err);
    return;
}

if (!ComponentClass) {
    console.error('[BasePages] showSetup: component class not found for section', section);
    return;
}

await this.base.renderInArea(ComponentClass, {
    section,
    user,
    setCaption: this.base._setCaption,
    restoreCaption: this.base._restoreCaption,
    onNavigate: (nextSection) => this.showSetup(nextSection),
});
}

// =====
// AUTH FALLBACK PAGE
// =====

/**
 * Show the auth fallback page in area-center.
 *
 * Called from Base._initAuth() when authRequired is true and the
 * user is not authenticated. Delegates to base.auth.showPage(),
 * which decides which form to render (login / register / ...).
 *
 * Also saves the current content so Base.onAuthSuccess() can restore
 * it after a successful login, and remembers the redirect target
 * in sessionStorage.
 */
showAuthPage() {
    console.log('[BasePages] showAuthPage()');

    const center = document.querySelector('.core-engine-lib-base-area-center');
    if (!center) return;

    this.base.savedContent = center.innerHTML;

    if (this.base.authRedirect) {
        this.base.redirectUrl = this.base.authRedirect;
    } else {
        this.base.redirectUrl = window.location.pathname;
    }
    sessionStorage.setItem('auth_redirect_url', this.base.redirectUrl);

    if (this.base.auth) {
        this.base.auth.showPage(center);
    }
}

```

```

    }
  }
}

=====
FILE: core/engine/lib/base/chat/chat.js
=====
// app/core/engine/lib/base/chat/chat.js

/**
 * BaseChat – floating chat widget.
 *
 * Two modes:
 * - modal (mobile / no sidebar) – opened by a floating button;
 * - sidebar (desktop) – attached into the right area via attach().
 *
 * HTTP goes through window.coreEngine.fetchJson
 * (loaded once by CoreEngine.loadApi()).
 */
export class BaseChat {
  constructor(options = {}) {
    console.log('[Chat] Constructor called');
    this.options = options;
    this.container = null;
    this.messages = [];
    this.isModalOpen = false;
    this.isSidebarMode = false;
    this.floatingBtn = null;
    this.onSend = null;
    this.apiUrl = options.apiUrl || '/core/chat/send';
    this.isLoading = false;

    // Init state
    this._initialized = false;
    this._initPromise = null;

    // Load CSS on creation
    this._loadCSS();

    // Build DOM
    this._createDOM();
    this._bindEvents();
    this._createFloatingButton();

    // Start async init
    this._initPromise = this._init();
  }

  async _init() {
    console.log('[Chat] _init() START');
    try {
      // Load message history (if API is available)
      await this._loadHistory();
      this._initialized = true;
      console.log('[Chat] _init() COMPLETE, messages:', this.messages.length);
    } catch (error) {

```

```

        console.error('[Chat] Init error:', error);
        this._initialized = false;
        // Do not rethrow – chat should work even without history
    }
}

/**
 * Load message history from the server.
 */
async _loadHistory() {
    console.log('[Chat] _loadHistory()');
    try {
        const fetchJson = window.coreEngine?.fetchJson;
        const data = await fetchJson('/core/chat/history');

        if (data.success && data.data) {
            this.messages = data.data.map(msg => ({
                id: msg.id || Date.now() + Math.random(),
                text: msg.text || msg.message || '',
                isUser: msg.is_user || msg.role === 'user',
                timestamp: msg.created_at || msg.timestamp || new Date()
            }));
            // Re-render messages
            if (this.messagesContainer) {
                this._renderMessages();
                this._scrollToBottom();
            }
            console.log('[Chat] History loaded:', this.messages.length);
        }
    } catch (error) {
        console.warn('[Chat] Failed to load history:', error);
    }
}

/**
 * Load CSS for the chat.
 */
_loadCSS() {
    console.log('[Chat] _loadCSS()');
    if (window.coreEngine && typeof window.coreEngine.loadCSS === 'function') {
        window.coreEngine.loadCSS('core/engine/lib/base/chat/chat.css');
    }
}

/**
 * Build the chat DOM structure.
 */
_createdDOM() {
    console.log('[Chat] _createdDOM()');
    const existing = document.querySelector('.core-engine-lib-base-chat');
    if (existing) {
        this.container = existing;
        this._cacheElements();
        return;
    }
}

```



```

const container = document.createElement('div');
container.className = 'core-engine-lib-base-chat';
container.style.display = 'none';
container.innerHTML = `
  <div class="chat-window">
    <div class="chat-header">
      <span class="chat-title">Чат с AI</span>
      <span class="chat-close-btn">x</span>
    </div>
    <div class="chat-messages"></div>
    <div class="chat-input-area">
      <textarea class="chat-input" placeholder="Введите сообщение..." rows="1"></textarea>
      <button class="chat-send-btn">></button>
    </div>
  </div>
`;
document.body.appendChild(container);
this.container = container;
this._cacheElements();

// Hidden by default
this.container.classList.add('hidden');
}

_cacheElements() {
  this.messagesContainer = this.container?.querySelector('.chat-messages');
  this.inputField = this.container?.querySelector('.chat-input');
  this.sendBtn = this.container?.querySelector('.chat-send-btn');
  this.closeBtn = this.container?.querySelector('.chat-close-btn');
  this.title = this.container?.querySelector('.chat-title');
}

_bindEvents() {
  console.log('[Chat] _bindEvents()');

  // Send on click
  if (this.sendBtn) {
    this.sendBtn.addEventListener('click', () => {
      this._handleSend();
    });
  }

  // Send on Enter (without Shift)
  if (this.inputField) {
    this.inputField.addEventListener('keydown', (e) => {
      if (e.key === 'Enter' && !e.shiftKey) {
        e.preventDefault();
        this._handleSend();
      }
    });
  }

  // Auto-height for textarea
  this.inputField.addEventListener('input', () => {
    this.inputField.style.height = 'auto';
    this.inputField.style.height = this.inputField.scrollHeight + 'px';
  });
}

```

```

    }

    // Close on ×
    if (this.closeBtn) {
        this.closeBtn.addEventListener('click', () => {
            this.close();
        });
    }

    // Close on overlay click (modal mode only)
    if (this.container) {
        this.container.addEventListener('click', (e) => {
            if (e.target === this.container && this.isModalOpen) {
                this.close();
            }
        });
    }

    // Close on Escape
    document.addEventListener('keydown', (e) => {
        if (e.key === 'Escape' && this.isModalOpen) {
            this.close();
        }
    });

    // Floating button click
    if (this.floatingBtn) {
        this.floatingBtn.addEventListener('click', () => {
            if (this.isModalOpen || this.isSidebarMode) {
                this.close();
            } else {
                this.open();
            }
        });
    }
}

/**
 * Create the floating button (for mobile).
 */
_createFloatingButton() {
    console.log('[Chat] _createFloatingButton()');
    const existing = document.querySelector('.core-engine-lib-base-chat-floating-btn');
    if (existing) {
        this.floatingBtn = existing;
        return;
    }

    const btn = document.createElement('button');
    btn.className = 'core-engine-lib-base-chat-floating-btn';
    btn.textContent = '□';
    btn.setAttribute('aria-label', 'Открыть чат');
    document.body.appendChild(btn);
    this.floatingBtn = btn;
}

```

```

/**
 * Handle message send.
 */
async _handleSend() {
  if (!this.inputField) return;

  const text = this.inputField.value.trim();
  if (!text || this.isLoading) return;

  // Add user message
  this.addMessage(text, true);
  this.inputField.value = '';
  this.inputField.style.height = 'auto';

  // Show loading indicator
  const loadingId = this._addLoadingIndicator();
  this.isLoading = true;

  try {
    // Send request to API
    const response = await this._sendToAPI(text);

    // Remove loading indicator
    this._removeLoadingIndicator(loadingId);
    // Add AI response
    this.addMessage(response, false);
  } catch (error) {
    this._removeLoadingIndicator(loadingId);
    this.addMessage('❌ Ошибка: ' + (error.message || 'Не удалось получить ответ'), false);
    console.error('[Chat] API error:', error);
  } finally {
    this.isLoading = false;
  }
}

/**
 * Send request to the API.
 */
async _sendToAPI(message) {
  console.log('[Chat] _sendToAPI()', message);

  const fetchJson = window.coreEngine?.fetchJson;
  const data = await fetchJson(this.apiUrl, {
    method: 'POST',
    body: {
      message: message,
      history: this.messages.slice(-10),
    },
  });

  if (data.success) {
    return data.data?.response || data.data?.message || 'Получен пустой ответ';
  } else {
    throw new Error(data.message || 'Неизвестная ошибка');
  }
}

```

```

/**
 * Add a message to the history.
 */
addMessage(text, isUser) {
  console.log('[Chat] addMessage()', text, isUser);
  const msg = {
    id: Date.now() + Math.random(),
    text: text,
    isUser: isUser,
    timestamp: new Date()
  };
  this.messages.push(msg);
  this._renderMessages();
  this._scrollToBottom();
}

/**
 * Add loading indicator.
 */
_addLoadingIndicator() {
  const id = 'loading-' + Date.now();
  const div = document.createElement('div');
  div.className = 'chat-message chat-message-ai loading';
  div.id = id;
  div.innerHTML = '<span class="chat-loading-dots">...</span>';
  if (this.messagesContainer) {
    this.messagesContainer.appendChild(div);
    this._scrollToBottom();
  }
  return id;
}

/**
 * Remove loading indicator.
 */
_removeLoadingIndicator(id) {
  const el = document.getElementById(id);
  if (el) {
    el.remove();
  }
}

/**
 * Render all messages.
 */
_renderMessages() {
  if (!this.messagesContainer) return;

  this.messagesContainer.innerHTML = '';
  this.messages.forEach(msg => {
    const div = document.createElement('div');
    div.className = `chat-message chat-message-${msg.isUser ? 'user' : 'ai'}`;
    div.textContent = msg.text;
    this.messagesContainer.appendChild(div);
  });
}

```

```

}

/**
 * Scroll to bottom.
 */
_scrollToBottom() {
  setTimeout(() => {
    if (this.messagesContainer) {
      this.messagesContainer.scrollTop = this.messagesContainer.scrollHeight;
    }
  }, 50);
}

// ===== Public methods =====

/**
 * Attach the chat into a sidebar container (desktop).
 */
attach(container) {
  console.log('[Chat] attach()');
  if (!container || !this.container) return;

  this.isSidebarMode = true;
  this.container.classList.add('sidebar-mode');
  this.container.classList.remove('modal-mode', 'hidden');
  this.container.style.display = 'flex';

  container.appendChild(this.container);

  // Hide the floating button in desktop mode
  if (this.floatingBtn) {
    this.floatingBtn.style.display = 'none';
  }

  // Show welcome message if empty
  if (this.messages.length === 0) {
    this.addMessage('Привет! Я AI-ассистент. Задайте мне вопрос.', false);
  }

  console.log('[Chat] Attached to sidebar');
}

/**
 * Open the chat as a modal (mobile).
 */
open() {
  console.log('[Chat] open()');
  if (!this.container) return;

  this.isModalOpen = true;
  this.container.classList.remove('hidden');
  this.container.classList.add('modal-mode');
  this.container.classList.remove('sidebar-mode');
  this.container.style.display = 'flex';

  setTimeout(() => {

```

```

        this.container.classList.add('active');
    }, 10);

    if (this.messages.length === 0) {
        this.addMessage('Привет! Я AI-ассистент. Задайте мне вопрос.', false);
    }

    setTimeout(() => {
        if (this.inputField) {
            this.inputField.focus();
        }
    }, 300);

    console.log('[Chat] Opened in modal mode');
}

/**
 * Close the chat (modal mode only).
 */
close() {
    console.log('[Chat] close()');
    if (!this.isModalOpen || !this.container) return;

    this.isModalOpen = false;
    this.container.classList.remove('active');

    setTimeout(() => {
        this.container.classList.add('hidden');
        this.container.style.display = 'none';
    }, 300);

    console.log('[Chat] Closed');
}

/**
 * Set a custom send handler.
 */
setOnSend(callback) {
    this.onSend = callback;
}

/**
 * Clear the message history.
 */
clearHistory() {
    console.log('[Chat] clearHistory()');
    this.messages = [];
    this._renderMessages();
}

/**
 * Whether the component is initialized.
 */
isInitialized() {
    return this._initialized;
}

```

```

/**
 * Wait for init to complete.
 */
async waitForInit() {
  if (this._initPromise) {
    await this._initPromise;
  }
  return this._initialized;
}

/**
 * Destroy the component.
 */
destroy() {
  console.log('[Chat] destroy()');
  if (this.floatingBtn) {
    this.floatingBtn.remove();
    this.floatingBtn = null;
  }
  if (this.container) {
    this.container.remove();
    this.container = null;
  }
  this._initialized = false;
  this._initPromise = null;
}
}

```

```

=====
FILE: core/engine/lib/pages/pages.js
=====

```

```

// app/core/engine/lib/pages/pages.js

```

```

/**
 * Pages component – article catalog.
 * Thin wrapper around BaseCards.
 *
 * If items are not passed in props, loads the list from the server.
 *
 * HTTP goes through window.coreEngine.fetchJson
 * (loaded once by CoreEngine.loadApi()).
 *
 * NAV_ID:
 * Requests may carry ?nav_id=<props.nav_id>. This scopes the
 * catalog to a specific nav instance. When omitted, the backend
 * resolves the nav from the authenticated user's session
 * (first nav by id ASC). So nav_id is optional here – pass it
 * only when a specific nav must be targeted.
 *
 * Permissions:
 * - Guests: read-only catalog view (no toolbar).
 * - Authenticated users (including superadmin): full toolbar –
 *   add, edit, delete, restore.
 *
 * Template system:

```

```

* - is_template: page can be used as a base template by other pages.
* - template_id: this page inherits layout from that template page;
*   its own `content` is inserted into [data-slot="content"] slot.
*
* Logo generation:
* The "logo" media field declares an `extraButtons` entry – the
* "Генерировать" button. Its onClick lives in logo.js and is loaded
* dynamically, like every other module here. BaseCardsEdit itself
* stays generic: it only renders the buttons and calls their
* onClick with a context object.
*
* Media sources:
* The "logo" field also declares `mediaSources` and `mediaSource`.
* The picker (BaseAssets) will show two tabs – «Медiateка» and
* «Логотипы» – and open on «Логотипы» by default. The user can
* still switch to «Медiateка» and pick any uploaded image.
*
* Onboarding hints:
* Two independent hints (see ./hint.js), both stored per-browser
* in localStorage:
*
*   EmptyArticlesHint – when the catalog is empty. Points at the
*   "+" button, explains how to create the first article.
*
*   CardActionsHint – after ANY card is created. Anchored to the
*   newly created card. Explains right-click → «Редактировать»
*   and double-click → open the article.
*
* Grid layout:
* Grid is defined in cards.css:
*   grid-template-columns: repeat(auto-fill, minmax(280px, 1fr))
*   grid-auto-rows: 220px
*
* Cards are NOT less than 280px wide, and stretch to fill the row
* (no empty space on the right). The row height is fixed at 220px.
* We deliberately do NOT pass `listView.gridColumns` – an inline
* `grid-template-columns` would override the minmax() and break
* the stretch.
*
* Caption: on _init() we pick up the shared caption helpers from
* window.coreEngine.base (_setCaption / _restoreCaption). They are
* used to set the header/tab title to "Каранор старей" while the
* catalog is on screen, and restore it on destroy.
*/

```

```

export class Pages {
  constructor(container, props = {}) {
    console.log('[Pages] Constructor', { container, props });

    this.container = container;
    this.props = props;

    // Nav instance this catalog belongs to. Comes from
    // <body data-nav-id="..."> via CoreEngine. If missing,
    // the backend resolves it from the session.
    this.navId = props.nav_id || null;
  }
}

```



```

this.cardsInstance = null;
this._initialized = false;
this._initPromise = null;

// Cached list of templates (is_template=1), for the dropdown
this._templates = [];

// Caption helpers – filled from window.coreEngine.base in _init().
this._setCaption = null;
this._restoreCaption = null;
this._savedCaption = null;

// Onboarding hints. Created in _init().
this._emptyHint = null;    // empty-catalog tooltip
this._cardHint = null;    // how-to-use-a-card tooltip

// Last known item count – used to detect "a card was created"
// between two _reload() calls (prevCount < newCount).
this._lastItemCount = 0;

this._loadCSS();

this._initPromise = this._init();
}

_loadCSS() {
  if (window.coreEngine?.loadCSS) {
    window.coreEngine.loadCSS('core/engine/lib/pages/pages.css');
    window.coreEngine.loadCSS('core/engine/lib/pages/hint.css');
  }
}

/**
 * Build a URL to the pages API with nav_id attached (if known).
 *
 * BaseCards appends endpoint paths to `apiBase` as-is, so we
 * keep `apiBase` clean and put nav_id directly into each
 * endpoint string. When navId is null, the backend resolves
 * the nav from the session – no query parameter needed.
 */
_apiUrl(endpoint, extraQuery = '') {
  const sep = endpoint.includes('?') ? '&' : '?';
  const navPart = this.navId != null
    ? `nav_id=${encodeURIComponent(this.navId)}`
    : '';
  const extra = extraQuery ? `&${extraQuery}` : '';

  // Avoid trailing "?" / "&" when both parts are empty.
  if (!navPart && !extra) return endpoint;
  if (!navPart) return `${endpoint}${sep}${extra}`;
  if (!extra) return `${endpoint}${sep}${navPart}`;
  return `${endpoint}${sep}${navPart}${extra}`;
}

async _init() {

```

```

console.log('[Pages] _init() START, navId =', this.navId);
try {
  // Pick up caption helpers from Base (loaded before Pages).
  const base = window.coreEngine?.base;
  this._setCaption = base?._setCaption || null;
  this._restoreCaption = base?._restoreCaption || null;

  const version = window.coreEngine?.static_version || Date.now();
  const { BaseCards } = await import(`../base/cards/cards.js?v=${version}`);

  // Logo generator button – used by the "logo" media field
  // below. Loaded dynamically, like every other module here.
  const { makeLogoGeneratorButton } = await import(
    `../logo.js?v=${version}`
  );

  // Onboarding hints (both variants live in the same module).
  const { EmptyArticlesHint, CardActionsHint } = await import(
    `../hint.js?v=${version}`
  );
  this._EmptyArticlesHint = EmptyArticlesHint;
  this._CardActionsHint = CardActionsHint;
  this._emptyHint = new EmptyArticlesHint();
  this._cardHint = new CardActionsHint();

  const canEdit = this._canEdit();

  // If items are not passed in props – load from server
  if (!this.props.items || this.props.items.length === 0) {
    console.log('[Pages] items empty – loading from server');
    await this._loadFromServer();
  }

  // Load templates list (for "Наследовать от" dropdown)
  if (canEdit) {
    await this._loadTemplates();
  }

  // Set header/tab title.
  if (this._setCaption && !this._savedCaption) {
    this._savedCaption = this._setCaption('Каталог статей', 'Каталог статей');
  }

  this.cardsInstance = new BaseCards(this.container, {
    items: this.props.items || [],
    isLoading: this.props.isLoading || false,
    error: this.props.error || null,

    // API
    //
    // nav_id is put directly into each endpoint, not into
    // apiBase, because BaseCards appends endpoints to
    // apiBase and we don't want to guess how it handles
    // query strings. When navId is null, the backend
    // resolves the nav from the session.
    apiBase: '/core/engine/lib/pages',

```

```

apiEndpoints: {
  list:    this._apiUrl('/list'),
  create:  this._apiUrl('/item'),
  update:  this._apiUrl('/item/{id}'),
  delete:  this._apiUrl('/item/{id}'),
  restore: this._apiUrl('/item/{id}/restore'),
},

// ===== FIELDS FOR CREATE/EDIT FORM =====
fields: [
  {
    key: 'title',
    label: 'Заголовок',
    type: 'text',
    required: true,
    minLength: 1,
    maxLength: 255,
    placeholder: 'Введите заголовок статьи',
  },
  {
    key: 'description',
    label: 'Краткое описание',
    type: 'textarea',
    maxLength: 500,
    rows: 3,
    placeholder: 'Описание (необязательно)',
  },
  {
    key: 'logo',
    label: 'Логотип',
    type: 'media',
    placeholder: 'Не выбрано',
    // Picker sources: two tabs in BaseAssets.
    // Opens on «Логотипы» by default, but the user
    // can switch to «Медиаотека» and pick any uploaded
    // image.
    mediaSources: ['media', 'logos'],
    mediaSource: 'logos',
    // Extra button: "Генерировать".
    // Rendered between "Выбрать" и "Очистить".
    // The behaviour lives in logo.js.
    extraButtons: [ makeLogoGeneratorButton() ],
  },

  // ===== Template system =====
  {
    key: 'is_template',
    label: 'сделать базовым шаблоном',
    type: 'checkbox',
    default: 0,
  },
  {
    key: 'template_id',
    label: 'Наследовать от',
    type: 'select',
    // Opt-in: values are numbers (page ids)
  }
]

```

```

        valueType: 'number',
        // Function – evaluated on every form render, so the
        // dropdown always shows the latest templates
        // (including ones created after init, without
        // a page reload).
        options: () => this._templates.map(t => ({
            value: t.id,
            label: t.title,
        })),
        placeholder: '- Без шаблона -',
    },
],

// ===== INITIAL DATA FOR "CREATE" FORM =====
initialData: (cards) => {
    const nextNum = this._nextArticleNumber(cards.items || []);
    return {
        title: `Статья ${nextNum}`,
        is_template: 0,
        template_id: null,
    };
},

// Fields for standard render
listFields: ['title', 'description'],

// Entity type (used in UI messages)
entityType: 'статью',

// Buttons – any authenticated user
showAddButton: canEdit,
showEditButton: canEdit,
showDeleteButton: canEdit,
showTrashButton: false,
showSearch: false,
showStatusFilter: false,

// Custom card render
renderCard: (item) => this._renderArticleCard(item),
cardOptions: { customClass: 'pages-card-wrapper' },

// Click → navigate to editor/view
onItemClick: (id) => this._openArticle(id),

// Callbacks
onReload: () => this._reload(),
onRetry: () => this._reload(),

widgetTitle: 'Статьи',
widgetStatus: `${this.props.items?.length || 0} статей`,
});

await this.cardsInstance._initPromise;

// Normalize payload types before they hit the API.
// BaseCardsEdit sends checkbox as boolean and empty select

```

```

    // as '', while the backend schema expects int (0/1) and null.
    // We patch the instance methods so cards.js and edit.js
    // stay untouched.
    this._patchPayloadNormalization();

    // Remember the current item count so the first _reload()
    // can tell "created" from "unchanged".
    this._lastItemCount = (this.cardsInstance?.props?.items
                          || this.props.items
                          || []).length;

    // Show the empty-catalog hint if the catalog is empty.
    this._updateEmptyHint();

    this._initialized = true;
    console.log('[Pages] _init() COMPLETE');
  } catch (error) {
    console.error('[Pages] Init error:', error);
    this._initialized = false;
    throw error;
  }
}

// =====
// ONBOARDING HINTS
// =====

/**
 * Show or hide the empty-catalog hint based on the current item count.
 *
 * Shown when:
 * - items array is empty;
 * - the user can edit (has a "+" button in the toolbar);
 * - the user has not chosen "never show again".
 *
 * Called from _init() (first paint) and _reload() (after create /
 * delete). The show() is deferred to the next tick so the toolbar
 * DOM node exists by the time we measure its position.
 */
_updateEmptyHint() {
  if (!this._emptyHint) return;

  const items = this.cardsInstance?.props?.items
                || this.props.items
                || [];

  if (items.length === 0 && this._canEdit()) {
    // Defer so the toolbar (with the "+" button) is in the DOM.
    setTimeout(() => {
      if (this._emptyHint) this._emptyHint.show();
    }, 0);
  } else {
    this._emptyHint.hide();
  }
}

```

```

/**
 * Show the "how to use a card" hint right after a card is created.
 *
 * Trigger: previous item count < current item count. This fires
 * for ANY new card, not only the first one. It stays visible
 * until the user closes it, checks "never show again", or the
 * next catalog reload happens without a new card.
 *
 * Suppressed by its own localStorage flag – independent from the
 * empty-catalog hint.
 *
 * @param {number} prevCount – item count before the reload
 * @param {number} newCount – item count after the reload
 */
_updateCardActionsHint(prevCount, newCount) {
  if (!this._cardHint) return;

  const created = newCount > prevCount;

  if (!created) {
    // Nothing was created – do not show, but keep an already
    // visible hint (the user may still be reading it).
    return;
  }

  // A card was created – hide the empty-catalog hint if it was
  // still on screen, then show the card-actions hint.
  if (this._emptyHint) {
    this._emptyHint.hide();
  }

  // Give the grid a beat to render the new card, then anchor
  // the hint to it.
  setTimeout(() => {
    if (!this._cardHint) return;

    // Newest card is at the top of the grid (backend returns
    // items sorted by datetime desc). Fall back to the first
    // card in the DOM if the class name ever changes.
    const cardEl = document.querySelector(
      '.core-engine-lib-base-cards-card, .pages-card-wrapper'
    );

    this._cardHint.show(cardEl);
  }, 150);
}

// =====
// PAYLOAD NORMALIZATION
// =====

_patchPayloadNormalization() {
  const cards = this.cardsInstance;
  if (!cards) return;

  const fields = cards.fields || [];

```

```

const normalize = (data) => {
  if (!data || typeof data !== 'object') return data;

  const out = { ...data };

  for (const field of fields) {
    const key = field.key;
    if (!(key in out)) continue;

    const v = out[key];

    switch (field.type) {
      case 'checkbox':
        out[key] = v ? 1 : 0;
        break;

      case 'select': {
        if (v === '' || v === null || v === undefined) {
          out[key] = null;
        } else if (field.valueType === 'number') {
          const n = Number(v);
          out[key] = Number.isNaN(n) ? null : n;
        }
        break;
      }

      case 'number': {
        if (v === '' || v === null || v === undefined) {
          out[key] = null;
        } else {
          const n = Number(v);
          out[key] = Number.isNaN(n) ? null : n;
        }
        break;
      }

      default:
        // text / textarea / media / date / time – as-is
        break;
    }
  }

  return out;
};

const origAdd = cards._addItem.bind(cards);
cards._addItem = async (data) => origAdd(normalize(data));

const origUpdate = cards._updateItem.bind(cards);
cards._updateItem = async (id, data) => origUpdate(id, normalize(data));

console.log('[Pages] _patchPayloadNormalization() applied');
}

// =====

```

```

// LOAD FROM SERVER
// =====

async _loadFromServer() {
  console.log('[Pages] _loadFromServer()');

  const url = this._apiUrl('/core/engine/lib/pages/list');
  try {
    const fetchJson = window.coreEngine?.fetchJson;
    const result = await fetchJson(url);

    if (result.success) {
      this.props.items = result.data || [];
      console.log('[Pages] Loaded from server:', this.props.items.length);
    } else {
      console.warn('[Pages] Response without success:', result);
      this.props.items = [];
    }
  } catch (error) {
    console.error('[Pages] List load error:', error);
    this.props.items = [];
  }
}

/**
 * Load templates list (pages with is_template=1).
 */
async _loadTemplates() {
  console.log('[Pages] _loadTemplates()');

  try {
    const url = this._apiUrl('/core/engine/lib/pages/list', 'is_template=1');
    const fetchJson = window.coreEngine?.fetchJson;
    const result = await fetchJson(url);

    if (result.success) {
      this._templates = result.data || [];
      console.log('[Pages] Loaded templates:', this._templates.length);
    } else {
      console.warn('[Pages] Templates response without success:', result);
      this._templates = [];
    }
  } catch (error) {
    console.error('[Pages] Templates load error:', error);
    this._templates = [];
  }
}

// =====
// CARD RENDER
// =====

_renderArticleCard(item) {
  const esc = (s) => String(s ?? '').replace(/&[<>"]/g, c => ({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'
  }[c]));

```



```

const date = item.datetime
  ? new Date(item.datetime).toLocaleDateString('ru-RU', {
    year: 'numeric', month: 'long', day: 'numeric'
  })
  : '';

const templateBadge = item.is_template
  ? `Шаблон</span>`
  : '';

return `
  <div class="pages-card">
    <div class="pages-card-glow"></div>
    ${item.logo ? `
      <div class="pages-card-logo">
        
      </div>
    ` : ''}
    <div class="pages-card-body">
      <h3 class="pages-card-title">${esc(item.title)}</h3>
      ${item.description ? `
```

```

*/
_openArticle(id) {
  console.log('[Pages] _openArticle() id =', id, '(type:', typeof id, ')');

  const cardsItems = this.cardsInstance?.props?.items;
  const ownItems = this.props.items;
  const items = cardsItems || ownItems || [];

  const item = items.find(i => String(i.id) === String(id));

  if (!item) {
    console.warn('[Pages] item not found');
    return;
  }
  if (!item.datetime) {
    console.warn('[Pages] item has no datetime');
    return;
  }

  const date = this._formatDate(item.datetime);
  const time = this._formatTime(item.datetime);

  // Base URL – the current module, e.g. /core/engine/default.
  const baseUrl = window.coreEngine?.baseUrl || '/core/engine/default';

  // Path: <module>/page/<nav_id>/<date>/<time>
  // nav_id is optional here; the backend resolves it from the
  // session when omitted. Keeping it explicit when known makes
  // the URL self-describing.
  const url = this.navId != null
    ? `${baseUrl}/page/${this.navId}/${date}/${time}`
    : `${baseUrl}/page/${date}/${time}`;

  console.log('[Pages] _openArticle() navigating to:', url);

  window.location.href = url;
}

// =====
// RELOAD
// =====

async _reload() {
  console.log('[Pages] _reload()');

  // Remember the count BEFORE we reload – to detect a creation.
  const prevCount = (this.cardsInstance?.props?.items
    || this.props.items
    || []).length;

  await this._loadFromServer();

  if (this._canEdit()) {
    await this._loadTemplates();
  }
}

```

```

    if (this.cardsInstance) {
      await this.cardsInstance.updateProps({
        items: this.props.items,
        isLoading: false,
        error: null,
        widgetStatus: `${this.props.items.length} статей`,
      });
    }

    const newCount = (this.props.items || []).length;

    // Update the empty-catalog hint after every reload.
    this._updateEmptyHint();

    // Show the card-actions hint if a card was just created.
    this._updateCardActionsHint(prevCount, newCount);

    this._lastItemCount = newCount;
  }

  // =====
  // UTILITIES
  // =====

  _nextArticleNumber(items) {
    let max = 0;

    for (const item of items) {
      const title = String(item.title || '');
      const m = title.match(/^Статья\s+(\d+)/i);
      if (m) {
        const num = parseInt(m[1], 10);
        if (num > max) max = num;
      }
    }

    return max + 1;
  }

  /**
   * Whether the current user can edit (add / update / delete / restore)
   * articles. Any authenticated user qualifies; guests do not.
   *
   * The check tries several auth shapes so it works regardless of
   * how BaseAuth exposes state:
   *   - auth.isAuth()      → boolean
   *   - auth.isAuthenticated → boolean
   *   - auth.getUser()     → user object or null
   */
  _canEdit() {
    const auth = window.coreEngine?.auth;
    if (!auth) return false;

    if (typeof auth.isAuth === 'function' && auth.isAuth()) {
      return true;
    }
  }

```

```

    if (typeof auth.isAuthenticated === 'boolean' && auth.isAuthenticated) {
        return true;
    }
    if (typeof auth.getUser === 'function') {
        return auth.getUser() !== null;
    }
    return false;
}

/**
 * Whether the current user is a superadmin.
 * Used only where superadmin-only behaviour is required.
 *
 * is_superadmin may come from the backend as bool, int (0/1)
 * or string ("0"/"1"), so all three are accepted.
 */
_isSuperadmin() {
    const auth = window.coreEngine?.auth;
    if (!auth) return false;

    if (typeof auth.isSuperadmin === 'function') {
        return !!auth.isSuperadmin();
    }
    if (typeof auth.getUser === 'function') {
        const user = auth.getUser();
        if (!user) return false;
        const v = user.is_superadmin;
        return v === true || v === 1 || v === "1";
    }
    return false;
}

_formatDate(isoString) {
    try {
        const d = new Date(isoString);
        const yyyy = d.getFullYear();
        const mm = String(d.getMonth() + 1).padStart(2, '0');
        const dd = String(d.getDate()).padStart(2, '0');
        return `${yyyy}${mm}${dd}`;
    } catch (e) {
        console.warn('[Pages] Date format error:', e);
        return '';
    }
}

_formatTime(isoString) {
    try {
        const d = new Date(isoString);
        const hh = String(d.getHours()).padStart(2, '0');
        const mi = String(d.getMinutes()).padStart(2, '0');
        const ss = String(d.getSeconds()).padStart(2, '0');
        return `${hh}${mi}${ss}`;
    } catch (e) {
        console.warn('[Pages] Time format error:', e);
        return '';
    }
}

```

```

    }

    // =====
    // PUBLIC METHODS
    // =====

    async waitForInit() {
        if (this._initPromise) await this._initPromise;
        return this._initialized;
    }

    destroy() {
        console.log('[Pages] destroy()');

        if (this._restoreCaption) {
            this._restoreCaption(this._savedCaption);
        }
        this._savedCaption = null;

        // Remove both onboarding hints (if visible).
        if (this._emptyHint) {
            this._emptyHint.remove();
            this._emptyHint = null;
        }
        if (this._cardHint) {
            this._cardHint.remove();
            this._cardHint = null;
        }

        if (this.cardsInstance?.destroy) {
            this.cardsInstance.destroy();
        }
        this.cardsInstance = null;
        this._initialized = false;
        this._initPromise = null;
    }
}

=====
FILE: core/engine/lib/pages/hint.js
=====
// app/core/engine/lib/pages/hint.js

/**
 * Onboarding tooltips for the article catalog.
 *
 * Two independent hints:
 *
 * EmptyArticlesHint – when the catalog is empty. Anchored to the
 * "+" button. Explains how to create the first article.
 *
 * CardActionsHint – after a card is created. Anchored to the newly
 * created card. Explains:
 *   - right click → «Редактировать» (card properties)
 *   - double click → open the article
 */

```

```

* Both hints:
*   - are position: fixed and are placed by JS relative to a target
*     element (getBoundingClientRect);
*   - carry a "Не показывать больше" checkbox that persists a flag
*     in localStorage;
*   - reposition themselves on window resize while visible.
*
* Each hint has its own storage key, so suppressing one does not
* suppress the other.
*
* Lifecycle (both classes):
*   const hint = new EmptyArticlesHint();
*   hint.show();           // shows it (if not suppressed & target exists)
*   hint.show(el);         // CardActionsHint also accepts a card element
*   hint.hide();           // removes it from the DOM
*   hint.remove();         // same as hide(), also detaches listeners
*
* No imports. Loaded dynamically from Pages._init() with ?v=<version>.
*/

// =====
// STORAGE KEYS
// =====

const KEY_EMPTY_HINT = 'neurocad.pages.empty_hint_hidden';
const KEY_CARD_HINT  = 'neurocad.pages.card_actions_hint_hidden';

// =====
// SMALL HELPERS
// =====

function _readFlag(key) {
  try {
    return localStorage.getItem(key) === '1';
  } catch {
    return false;
  }
}

function _writeFlag(key) {
  try {
    localStorage.setItem(key, '1');
  } catch {
    // localStorage may be unavailable (private mode) – ignore.
  }
}

/**
 * Place `el` below `target`.
 * options.offsetY    – vertical gap, default 12
 * options.alignRight – when true, align the hint's right edge with
 *                       the target's right edge; otherwise align the
 *                       left edges.
 */

```

```

function _anchorTo(el, target, options = {}) {
  if (!el || !target) return;

  const rect = target.getBoundingClientRect();
  const offsetY = options.offsetY ?? 12;

  el.style.top = `${rect.bottom + offsetY}px`;

  if (options.alignRight) {
    el.style.right = `${window.innerWidth - rect.right}px`;
    el.style.left = '';
  } else {
    el.style.left = `${rect.left}px`;
    el.style.right = '';
  }
}

// =====
// EMPTY CATALOG HINT
// =====

export class EmptyArticlesHint {
  constructor() {
    this.el = null;
    this._onResize = null;
  }

  static isSuppressed() {
    return _readFlag(KEY_EMPTY_HINT);
  }

  static suppress() {
    _writeFlag(KEY_EMPTY_HINT);
  }

  /**
   * Show the hint anchored to the "+" button.
   * Does nothing if the user suppressed it or the "+" is not in the DOM.
   */
  show() {
    if (EmptyArticlesHint.isSuppressed()) return;

    const target = document.querySelector('.cards-toolbar-btn-add');
    if (!target) {
      console.log('[EmptyArticlesHint] no "+" button in DOM – skipping');
      return;
    }

    this.remove();

    const el = document.createElement('div');
    el.className = 'core-engine-lib-pages-empty-hint';
    el.innerHTML = `
      <button type="button"
        class="core-engine-lib-pages-empty-hint-close"

```

```

        title="Заккрыть">x</button>

<div class="core-engine-lib-pages-empty-hint-title">
    У вас пока нет статей
</div>

<div class="core-engine-lib-pages-empty-hint-text">
    Нажмите на кнопку «+», чтобы создать первую статью.
</div>

<label class="core-engine-lib-pages-empty-hint-check">
    <input type="checkbox" data-js="hint-never">
    <span>Не показывать больше</span>
</label>
`;

_anchorTo(el, target, { offsetY: 12 });

document.body.appendChild(el);
this.el = el;

el.querySelector('.core-engine-lib-pages-empty-hint-close')
    ?.addEventListener('click', () => this.hide());

el.querySelector('[data-js="hint-never"]')
    ?.addEventListener('change', (e) => {
        if (e.target.checked) {
            EmptyArticlesHint.suppress();
            this.hide();
        }
    });

this._onResize = () => _anchorTo(this.el, target, { offsetY: 12 });
window.addEventListener('resize', this._onResize);
}

hide() {
    this.remove();
}

remove() {
    if (this._onResize) {
        window.removeEventListener('resize', this._onResize);
        this._onResize = null;
    }
    if (this.el) {
        this.el.remove();
        this.el = null;
    }
}
}

// =====
// CARD ACTIONS HINT
// =====

```



```

export class CardActionsHint {
  constructor() {
    this.el = null;
    this._onResize = null;
  }

  static isSuppressed() {
    return _readFlag(KEY_CARD_HINT);
  }

  static suppress() {
    _writeFlag(KEY_CARD_HINT);
  }

  /**
   * Show the hint anchored to a card.
   *
   * @param {HTMLElement} [cardEl] – card node to anchor to.
   * When omitted, the first card in the grid is used.
   */
  show(cardEl = null) {
    if (CardActionsHint.isSuppressed()) return;

    const target = cardEl || document.querySelector(
      '.core-engine-lib-base-cards-card, .pages-card-wrapper'
    );
    if (!target) {
      console.log('[CardActionsHint] no card in DOM – skipping');
      return;
    }

    this.remove();

    const el = document.createElement('div');
    el.className = 'core-engine-lib-pages-card-hint';
    el.innerHTML = `
      <button type="button"
        class="core-engine-lib-pages-card-hint-close"
        title="Заккрыть">×</button>

      <div class="core-engine-lib-pages-card-hint-title">
        Карточка создана
      </div>

      <div class="core-engine-lib-pages-card-hint-text">
        <div class="core-engine-lib-pages-card-hint-row">
          <span class="core-engine-lib-pages-card-hint-key">Правый клик</span>
          <span>– редактировать свойства карточки</span>
        </div>
        <div class="core-engine-lib-pages-card-hint-row">
          <span class="core-engine-lib-pages-card-hint-key">Двойной клик</span>
          <span>– открыть статью</span>
        </div>
      </div>
    `;
  }
}

```

```

        <label class="core-engine-lib-pages-card-hint-check">
            <input type="checkbox" data-js="hint-never">
            <span>Не показывать больше</span>
        </label>
    `;

    // Anchor below the card, aligned to its right edge so the
    // hint does not overlap the neighbouring card.
    _anchorTo(el, target, { offsetY: 12, alignRight: true });

    document.body.appendChild(el);
    this.el = el;

    el.querySelector('.core-engine-lib-pages-card-hint-close')
        ?.addEventListener('click', () => this.hide());

    el.querySelector('[data-js="hint-never"]')
        ?.addEventListener('change', (e) => {
            if (e.target.checked) {
                CardActionsHint.suppress();
                this.hide();
            }
        });

    this._onResize = () => _anchorTo(el, target, { offsetY: 12, alignRight: true });
    window.addEventListener('resize', this._onResize);
}

hide() {
    this.remove();
}

remove() {
    if (this._onResize) {
        window.removeEventListener('resize', this._onResize);
        this._onResize = null;
    }
    if (this.el) {
        this.el.remove();
        this.el = null;
    }
}
}

```

```

=====
FILE: core/engine/lib/pages/logo.js
=====

```

```

// app/core/engine/lib/pages/logo.js

```

```

/**
 * Logo generator – extra button for the "Логотип" media field
 * on the pages card form.
 *
 * Returns a button descriptor consumable by BaseCardsEdit's
 * `field.extraButtons`. BaseCardsEdit itself is unchanged – it only
 * knows how to render a button and call its onClick with a context.

```

```

*
* The button:
*   1. reads the current "title" and "description" from the form
*      (through getAllValues);
*   2. POSTs to /core/engine/lib/word/editor/images/generate;
*   3. on success – writes the returned URL into the media field
*      (setValue) so the hidden input and the preview update;
*   4. on failure – shows the error under the field (showError).
*
* Text of the button never changes – only the disabled state,
* so it does not "jump" during generation.
*
* No global state, no imports – the module is pure.
*/

export function makeLogoGeneratorButton() {
  return {
    label: 'Генерировать',
    className: 'edit-media-btn-generate',
    onClick: async ({ getAllValues, setValue, showError, clearError, button }) => {
      const values = getAllValues() || {};
      const title = String(values.title || '').trim();
      const description = String(values.description || '').trim();

      if (!title) {
        showError('Сначала введите заголовок');
        return;
      }

      const prompt = description
        ? `${title}. ${description}`
        : title;

      // ---- Busy state ----
      // Only the disabled flag changes. The text stays as-is
      // so the button does not "jump" during the request.
      button.disabled = true;
      clearError();

      try {
        const res = await fetch(
          '/core/engine/lib/word/editor/images/generate',
          {
            method: 'POST',
            credentials: 'same-origin',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
              prompt,
              alt: title,
              source: 'logo',
            }),
          },
        );

        const data = await res.json().catch(() => null);

```

```

        if (!res.ok || !data || !data.success) {
            const msg =
                (data && (data.detail || data.message)) ||
                `Ошибка генерации (${res.status})`;
            showError(msg);
            return;
        }

        const url = data?.data?.url;
        if (!url) {
            showError('Пустой ответ сервера');
            return;
        }

        setValue(url);
    } catch (e) {
        console.error('[logo] generate failed:', e);
        showError('Сеть недоступна');
    } finally {
        button.disabled = false;
    }
},
};
}

```

```

=====
FILE: core/engine/lib/balance/balance.js
=====
// neurocad/core/engine/lib/balance/balance.js

/**
 * Balance – admin balance table (superadmin-only).
 *
 * Rendered into area-center by the "balance" page config
 * (app/default/balance.json → content: "{{ balance }}").
 *
 * Loads all users + their balance via
 * GET /core/engine/lib/balance/list
 * and shows them in a table.
 *
 * Each row has two action buttons:
 * - "Редактировать" → opens edit/edit.js (modal, all Balance fields)
 * - "Оплата" → opens paid/paid.js (modal, sum += N)
 *
 * QR-код Сбера:
 * Above the table there's a small toolbar:
 * - "Загрузить QR" → opens a file picker (PNG only)
 * - "Посмотреть QR" → opens the QR in a modal (BaseModalImage)
 * - "Удалить QR" → removes the file (with confirm)
 * State (exists / url) is loaded from
 * GET /core/engine/lib/balance/qr/status
 *
 * The QR file itself is stored at media/<nav_id>/qr.png and is
 * used by the tarif modal when a user has insufficient funds.
 *
 * Columns:

```

```

*   - `gen`   – accumulated generations
*   - `mb`    – used storage, in megabytes; formatted as "100 МБ"
*               or "1 ГБ" via _formatStorage()
*
* Access:
*   Only superadmin. The server enforces this on /list (403);
*   this component also checks the same flag before rendering
*   anything, so a non-superadmin who somehow lands on the page
*   sees a "доступ запрещён" placeholder instead of an empty table.
*
* Rendering:
*   The Renderer instantiates components and awaits their
*   _initPromise – it does NOT call render() itself. So we call
*   render() ourselves, from _init(), after data is loaded
*   (or immediately, for the forbidden / error states).
*
*   IMPORTANT: the container passed to the constructor is a
*   staging <div> (hidden), not area-center. Base.renderContent()
*   takes our rootEl out of staging and moves it into
*   area-center – and then removes the staging div. So on
*   _reload() we must NOT re-append rootEl to this.container:
*   that div is gone, and rootEl would end up detached.
*   Instead, _reload() refreshes the table inside the existing
*   rootEl (which already lives in area-center).
*
* Caption:
*   On _init() we pick up the shared caption helpers from
*   window.coreEngine.base (_setCaption / _restoreCaption) and set
*   the header/tab title to "Управление балансом" while the page is
*   on screen. The previous caption is restored in destroy().
*
* HTTP goes through window.coreEngine.fetchJson
* (loaded once by CoreEngine.loadApi()).
*/

```

```

export class Balance {
  constructor(container, props = {}) {
    console.log('[Balance] Constructor', { container, props });

    this.container = container;
    this.props = props;

    // Data
    this.items = [];
    this.isLoading = false;
    this.error = null;

    // QR state
    this.qr = {
      exists: false,
      url: null,
      uploading: false,
    };

    // Active modals
    this.editModal = null;
  }
}

```

```

this.paidModal = null;

// Init state
this._initialized = false;
this._initPromise = null;

// Caption helpers – filled from window.coreEngine.base in _init().
this._setCaption = null;
this._restoreCaption = null;
this._savedCaption = null;

// DOM refs (filled in render())
this.rootEl = null;
this.tableBodyEl = null;
this.statusEl = null;
this.qrToolbarEl = null;
this.fileInputEl = null;

// Guard against double render (e.g. _init() + explicit call).
this._rendered = false;

this._loadCSS();

this._initPromise = this._init();
}

// =====
// LIFECYCLE
// =====

_loadCSS() {
  if (window.coreEngine?.loadCSS) {
    window.coreEngine.loadCSS('core/engine/lib/balance/balance.css');
  }
}

async _init() {
  console.log('[Balance] _init() START');
  try {
    const base = window.coreEngine?.base;
    this._setCaption = base?._setCaption || null;
    this._restoreCaption = base?._restoreCaption || null;

    if (!this._isSuperadmin()) {
      console.warn('[Balance] Access denied (not superadmin)');
      this.error = 'forbidden';
      this.render();
      this._initialized = true;
      return;
    }

    if (this._setCaption && !this._savedCaption) {
      this._savedCaption = this._setCaption(
        'Управление балансом',
        'Управление балансом'
      );
    }
  }
}

```

```

    }

    // Load list and QR status in parallel.
    await Promise.all([
        this._loadList(),
        this._loadQrStatus(),
    ]);

    this.render();

    this._initialized = true;
    console.log('[Balance] _init() COMPLETE');
} catch (error) {
    console.error('[Balance] Init error:', error);
    this.error = error.message || 'load_failed';
    this.render();
    this._initialized = true;
}
}

// =====
// API
// =====

get _apiBase() {
    return '/core/engine/lib/balance';
}

get _navId() {
    // Nav instance – for media/<nav_id>/qr.png.
    // Priority: window.coreEngine.navId → data-nav-id on body.
    const v = window.coreEngine?.navId ?? document.body?.dataset?.navId;
    return v != null && v !== '' ? Number(v) : null;
}

async _loadList() {
    console.log('[Balance] _loadList()');

    this.isLoading = true;
    try {
        const fetchJson = window.coreEngine?.fetchJson;
        const json = await fetchJson(`${this._apiBase}/list`);

        this.items = json.data || [];
        this.error = null;

        console.log('[Balance] Loaded items:', this.items.length);
    } catch (err) {
        console.error('[Balance] List load error:', err);
        this.items = [];
        this.error = err.message || 'load_failed';
    } finally {
        this.isLoading = false;
    }
}
}

```

```

async _loadQrStatus() {
  console.log('[Balance] _loadQrStatus()');

  try {
    const fetchJson = window.coreEngine?.fetchJson;
    const qs = this._navId != null ? `?nav_id=${this._navId}` : '';
    const json = await fetchJson(`${this._apiBase}/qr/status${qs}`);

    this.qr.exists = !!json.exists;
    this.qr.url = json.url || null;

    console.log('[Balance] QR status:', this.qr);
  } catch (err) {
    console.error('[Balance] QR status error:', err);
    this.qr.exists = false;
    this.qr.url = null;
  }
}

async _reload() {
  console.log('[Balance] _reload()');
  await this._loadList();

  if (this.rootEl && this.rootEl.parentNode) {
    this.tableBodyEl = this.rootEl.querySelector('[data-role="table-body"]');
    this.statusEl = this.rootEl.querySelector('[data-role="status"]');
    this._renderTable();
    console.log('[Balance] _reload() - table refreshed in place');
  } else {
    this._rendered = false;
    this.render();
    console.log('[Balance] _reload() - full re-render');
  }
}

// =====
// RENDER
// =====

render() {
  console.log('[Balance] render()');

  if (this._rendered && this.rootEl && this.rootEl.parentNode) {
    console.log('[Balance] render() skipped (already rendered)');
    return this.rootEl;
  }

  this.rootEl = document.createElement('div');
  this.rootEl.className = 'core-engine-lib-balance';

  // Superadmin denied
  if (this.error === 'forbidden' || !this._isSuperadmin()) {
    this.rootEl.innerHTML = `
      <div class="balance-body">
        <div class="balance-error">
          <div class="balance-error-icon"></div>
    `;
  }
}

```



```

        <div class="balance-error-text">Доступ запрещён</div>
        <div class="balance-error-hint">
            Раздел доступен только суперадминистраторам.
        </div>
    </div>
</div>
`;
this.container.appendChild(this.rootEl);
this._rendered = true;
return this.rootEl;
}

// Load error
if (this.error && this.error !== 'forbidden') {
    this.rootEl.innerHTML = `
        <div class="balance-body">
            <div class="balance-error">
                <div class="balance-error-icon">❌</div>
                <div class="balance-error-text">Не удалось загрузить баланс</div>
                <div class="balance-error-hint">${this._escape(this.error)}</div>
                <button type="button" class="balance-btn" data-action="reload">Повторить</button>
            </div>
        </div>
    `;
    this.container.appendChild(this.rootEl);
    this._rendered = true;

    const reloadBtn = this.rootEl.querySelector('[data-action="reload"]');
    if (reloadBtn) {
        reloadBtn.addEventListener('click', () => this._reload());
    }

    return this.rootEl;
}

// Normal state
this.rootEl.innerHTML = `
    <div class="balance-body">
        <div class="balance-titlebar">
            <div class="balance-titlebar-left"></div>
            <div class="balance-titlebar-title">Управление балансом</div>
            <div class="balance-titlebar-right">
                <button type="button" class="balance-icon-btn" data-action="reload" title="Обновить">
                    <span aria-hidden="true">↻</span>
                </button>
            </div>
        </div>
    </div>

    <div class="balance-status" data-role="status" style="display:none;"></div>

    <div class="balance-qr-toolbar" data-role="qr-toolbar"></div>

    <div class="balance-scroll">
        <table class="balance-table">
            <thead>
                <tr>

```

```

        <th>ID</th>
        <th>Логин</th>
        <th>Имя</th>
        <th>Тариф</th>
        <th>Сумма</th>
        <th>Токены</th>
        <th>Ген</th>
        <th>Страниц</th>
        <th>Хранилище</th>
        <th>Обновлено</th>
        <th>Действия</th>
    </tr>
</thead>
<tbody data-role="table-body"></tbody>
</table>
</div>
</div>
`;

this.tableBodyEl = this.rootEl.querySelector('[data-role="table-body"]');
this.statusEl = this.rootEl.querySelector('[data-role="status"]');
this.qrToolbarEl = this.rootEl.querySelector('[data-role="qr-toolbar"]');

this._renderQrToolbar();
this._renderTable();

this.container.appendChild(this.rootEl);
this._rendered = true;

this.bindEvents(this.rootEl);

return this.rootEl;
}

_renderQrToolbar() {
    if (!this.qrToolbarEl) return;

    const hasQr = this.qr.exists;

    this.qrToolbarEl.innerHTML = `
        <div class="balance-qr-info">
            <span class="balance-qr-label">QR-код Сбера:</span>
            <span class="balance-qr-state ${hasQr ? 'is-ok' : 'is-missing'}">
                ${hasQr ? 'загружен' : 'не загружен'}
            </span>
        </div>
        <div class="balance-qr-actions">
            <button type="button"
                class="balance-qr-btn balance-qr-btn-primary"
                data-action="qr-upload"
                ${this.qr.uploading ? 'disabled' : ''}>
                ${this.qr.uploading ? 'Загрузка...' : (hasQr ? 'Заменить QR' : 'Загрузить QR')}
            </button>
            <button type="button"
                class="balance-qr-btn"
                data-action="qr-view"
    
```

```

        ${hasQr ? '' : 'disabled'}>
        Посмотреть
    </button>
    <button type="button"
        class="balance-qr-btn balance-qr-btn-danger"
        data-action="qr-delete"
        ${hasQr ? '' : 'disabled'}>
        Удалить
    </button>
    <input type="file"
        class="balance-qr-file"
        data-role="qr-file"
        accept=".png,image/png"
        hidden>
</div>
`;

this.fileInputEl = this.qrToolbarEl.querySelector('[data-role="qr-file"]');
}

_renderTable() {
    if (!this.tableBodyEl) return;

    if (!this.items || this.items.length === 0) {
        this.tableBodyEl.innerHTML = `
            <tr>
                <td colspan="11" class="balance-empty">Нет пользователей</td>
            </tr>
        `;
        return;
    }

    this.tableBodyEl.innerHTML = this.items.map((item) => {
        const tarifClass = this._tarifClass(item.tarif);
        const updated = item.updated_at
            ? this._formatDate(item.updated_at)
            : '-';

        return `
            <tr data-user-id="${item.user_id}">
                <td>${item.user_id}</td>
                <td>${this._escape(item.login || '-')}</td>
                <td>${this._escape(item.name || '-')}</td>
                <td>
                    <span class="balance-tarif balance-tarif-${tarifClass}">
                        ${this._escape(item.tarif_label || 'Free')}
                    </span>
                </td>
                <td class="balance-num">${item.sum ?? 0}</td>
                <td class="balance-num">${item.tokens ?? 0}</td>
                <td class="balance-num">${item.gen ?? 0}</td>
                <td class="balance-num">${item.pages ?? 0}</td>
                <td class="balance-num">${this._formatStorage(item.mb ?? 0)}</td>
                <td class="balance-date">${updated}</td>
                <td class="balance-actions">
                    <button type="button"

```

```

        class="balance-action-btn"
        data-action="edit"
        data-user-id="${item.user_id}">
        Редактировать
    </button>
    <button type="button"
        class="balance-action-btn balance-action-btn-primary"
        data-action="paid"
        data-user-id="${item.user_id}">
        Оплата
    </button>
</td>
</tr>
`
    }).join('');
}

// =====
// EVENTS
// =====

bindEvents(container) {
    console.log('[Balance] bindEvents()');

    const root = this.rootEl || container;
    if (!root) return;

    // Reload button
    root.querySelectorAll('[data-action="reload"]').forEach((btn) => {
        if (btn.dataset.bound === '1') return;
        btn.dataset.bound = '1';
        btn.addEventListener('click', () => this._reload());
    });

    // QR toolbar buttons (delegated on root – DOM may be re-rendered)
    if (root.dataset.qrBound !== '1') {
        root.dataset.qrBound = '1';

        root.addEventListener('click', (e) => {
            const btn = e.target.closest('[data-action^="qr-"]');
            if (!btn) return;

            const action = btn.dataset.action;
            if (action === 'qr-upload') this._handleQrUploadClick();
            else if (action === 'qr-view') this._handleQrView();
            else if (action === 'qr-delete') this._handleQrDelete();
        });

        // File input change – bound once, on root (delegated)
        root.addEventListener('change', (e) => {
            const el = e.target;
            if (!el.matches('[data-role="qr-file"]')) return;
            const files = Array.from(el.files || []);
            if (files.length) this._handleQrFile(files[0]);
            el.value = '';
        });
    }
}

```

```

    }

    // Delegated click on table actions
    const tbody = root.querySelector('[data-role="table-body"]');
    if (tbody && tbody.dataset.bound !== '1') {
        tbody.dataset.bound = '1';
        tbody.addEventListener('click', (e) => {
            const btn = e.target.closest('[data-action]');
            if (!btn) return;

            const action = btn.dataset.action;
            const userId = parseInt(btn.dataset.userId, 10);
            if (!Number.isFinite(userId)) return;

            if (action === 'edit') {
                this._openEdit(userId);
            } else if (action === 'paid') {
                this._openPaid(userId);
            }
        });
    }
}

// =====
// QR – UPLOAD / VIEW / DELETE
// =====

_handleQrUploadClick() {
    if (!this.fileInputEl) return;
    this.fileInputEl.click();
}

async _handleQrFile(file) {
    console.log('[Balance] _handleQrFile()', file);

    // Client-side validation – only PNG.
    const isPng = file.type === 'image/png'
        || file.name.toLowerCase().endsWith('.png');

    if (!isPng) {
        this._showStatus(
            'Только PNG. Конвертируйте файл перед загрузкой.',
            'error'
        );
        return;
    }

    // Busy state
    this.qr.uploading = true;
    this._renderQrToolbar();
    this._hideStatus();

    try {
        const fetchJson = window.coreEngine?.fetchJson;
        const qs = this._navId !== null ? `?nav_id=${this._navId}` : '';
    }

```

```

    const form = new FormData();
    form.append('file', file);

    const json = await fetchJson(`${this._apiBase}/qr/upload${qs}`, {
      method: 'POST',
      body: form,
    });

    console.log('[Balance] QR uploaded:', json);

    this.qr.exists = true;
    this.qr.url = json.url || `/media/${this._navId}/qr.png`;
    this._showStatus('QR-код загружен', 'success');
  } catch (err) {
    console.error('[Balance] QR upload error:', err);
    const msg = err?.data?.detail || err.message || 'Ошибка загрузки';
    this._showStatus(msg, 'error');
  } finally {
    this.qr.uploading = false;
    this._renderQrToolbar();
  }
}

async _handleQrView() {
  if (!this.qr.exists) return;

  // Re-check status – url might be missing after a manual upload
  // to the media folder (outside the admin UI).
  if (!this.qr.url) {
    await this._loadQrStatus();
  }

  const src = this.qr.url || (this._navId !== null ? `/media/${this._navId}/qr.png` : null);
  if (!src) {
    this._showStatus('QR-код не найден', 'error');
    return;
  }

  // Lazy-load createModal (base/modal/index.js)
  const version = window.coreEngine?.static_version || Date.now();
  let createModal;
  try {
    const mod = await import(
      `/static/core/engine/lib/base/modal/index.js?v=${version}`
    );
    createModal = mod.createModal;
  } catch (err) {
    console.error('[Balance] createModal load error:', err);
    // Fallback – open in new tab
    window.open(src, '_blank', 'noopener,noreferrer');
    return;
  }

  const modal = await createModal('image');
  if (!modal) {
    window.open(src, '_blank', 'noopener,noreferrer');
  }
}

```

```

        return;
    }

    modal.open({
        src,
        title: 'QR-код Сбера',
        caption: 'Отсканируйте приложением банка для оплаты',
        alt: 'QR-код Сбера',
    });
}

async _handleQrDelete() {
    if (!this.qr.exists) return;

    const ok = window.confirm('Удалить QR-код?');
    if (!ok) return;

    try {
        const fetchJson = window.coreEngine?.fetchJson;
        const qs = this._navId != null ? `?nav_id=${this._navId}` : '';

        const json = await fetchJson(`${this._apiBase}/qr${qs}`, {
            method: 'DELETE',
        });

        console.log('[Balance] QR delete:', json);

        this.qr.exists = false;
        this.qr.url = null;

        this._renderQrToolbar();
        this._showStatus('QR-код удалён', 'success');
    } catch (err) {
        console.error('[Balance] QR delete error:', err);
        const msg = err?.data?.detail || err.message || 'Ошибка удаления';
        this._showStatus(msg, 'error');
    }
}

// =====
// MODALS (edit / paid)
// =====

async _openEdit(userId) {
    console.log('[Balance] _openEdit() user =', userId);

    const version = window.coreEngine?.static_version || Date.now();
    let BalanceEdit;
    try {
        const mod = await import(`./edit/edit.js?v=${version}`);
        BalanceEdit = mod.BalanceEdit;
    } catch (err) {
        console.error('[Balance] edit.js load error:', err);
        this._showStatus('Не удалось открыть форму редактирования', 'error');
        return;
    }
}

```

```

    this.editModal = new BalanceEdit({
      userId,
      onSave: async () => {
        await this._reload();
        this._showStatus('Изменения сохранены', 'success');
      },
      onClose: () => { this.editModal = null; },
    });

    if (this.editModal._initPromise) {
      await this.editModal._initPromise;
    }
  }

  async _openPaid(userId) {
    console.log('[Balance] _openPaid() user =', userId);

    const version = window.coreEngine?.static_version || Date.now();
    let BalancePaid;
    try {
      const mod = await import(`./paid/paid.js?v=${version}`);
      BalancePaid = mod.BalancePaid;
    } catch (err) {
      console.error('[Balance] paid.js load error:', err);
      this._showStatus('Не удалось открыть форму оплаты', 'error');
      return;
    }

    this.paidModal = new BalancePaid({
      userId,
      onSave: async () => {
        await this._reload();
        this._showStatus('Оплата зачислена', 'success');
      },
      onClose: () => { this.paidModal = null; },
    });

    if (this.paidModal._initPromise) {
      await this.paidModal._initPromise;
    }
  }

  // =====
  // UI HELPERS
  // =====

  _showStatus(text, type = 'info') {
    if (!this.statusEl) return;
    this.statusEl.textContent = text;
    this.statusEl.className = `balance-status balance-status-${type}`;
    this.statusEl.style.display = 'block';

    if (type === 'success') {
      setTimeout(() => {
        if (this.statusEl) this.statusEl.style.display = 'none';
      });
    }
  }

```



```

        }, 3000);
    }
}

_hideStatus() {
    if (!this.statusEl) return;
    this.statusEl.textContent = '';
    this.statusEl.style.display = 'none';
}

_tarifClass(tarif) {
    switch (tarif) {
        case 1: return 'pro';
        case 2: return 'llm';
        default: return 'free';
    }
}

_formatDate(iso) {
    if (!iso) return '-';
    try {
        const d = new Date(iso);
        if (isNaN(d.getTime())) return iso;
        const pad = (n) => String(n).padStart(2, '0');
        return `${pad(d.getDate())}.${pad(d.getMonth() + 1)}.${pad(d.getFullYear())} `
            + `${pad(d.getHours())}:${pad(d.getMinutes())}`;
    } catch (e) {
        return iso;
    }
}

_formatStorage(mb) {
    const v = Number(mb) || 0;
    if (v >= 1024) {
        const gb = v / 1024;
        return `${gb.toFixed(1).replace(/\.0$/, '')} ГБ`;
    }
    return `${v} МБ`;
}

_escape(str) {
    return String(str ?? '')
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;');
}

_isSuperadmin() {
    const auth = window.coreEngine?.auth;
    if (!auth) return false;

    if (typeof auth.getUser === 'function') {
        const user = auth.getUser();
        if (!user) return false;
        const v = user.is_superadmin;
    }
}

```

```

        return v === true || v === 1 || v === "1";
    }
    return false;
}

// =====
// PUBLIC
// =====

isInitialized() {
    return this._initialized;
}

async waitForInit() {
    if (this._initPromise) await this._initPromise;
    return this._initialized;
}

destroy() {
    console.log('[Balance] destroy()');

    if (this._restoreCaption) {
        this._restoreCaption(this._savedCaption);
    }
    this._savedCaption = null;

    if (this.editModal?.destroy) this.editModal.destroy();
    if (this.paidModal?.destroy) this.paidModal.destroy();
    this.editModal = null;
    this.paidModal = null;

    if (this.rootEl) {
        this.rootEl.remove();
        this.rootEl = null;
    }

    this.tableBodyEl = null;
    this.statusEl = null;
    this.qrToolbarEl = null;
    this.fileInputEl = null;
    this.items = [];
    this._rendered = false;

    this._initialized = false;
    this._initPromise = null;
}
}

=====
FILE: alembic.ini
=====
# alembic.ini

[alembic]
script_location = %(here)s/alembic
compare_type = true

```

```

compare_server_default = true
prepend_sys_path = .
recursive_version_locations = true

file_template = %(year)d/%(month).2d/%(day).2d/%(hour).2d%(minute).2d%(second).2d_%(slug)s

path_separator = os

sqlalchemy.url =

[post_write_hooks]
hooks = black
black.type = console_scripts
black.entrypoint = black
black.options = -l 79 REVISION_SCRIPT_FILENAME

[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARNING
handlers = console
qualname =

[logger_sqlalchemy]
level = WARNING
handlers =
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers =
qualname = alembic

[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic

[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S

```